

PIC-tutor v0.110

Contents

PIC-tutor	7
读者在本版可以学到什么	7
如何判断一个结论	7
建议的阅读方式	7
0. 写作说明	7
这本书为谁而写	7
如何使用本书	7
1. 动理学模型与 PIC 的基本思想	9
1.1 Vlasov 方程首先是相空间守恒律	9
1.2 碰撞项不是 Vlasov 的一部分, 但必须保留边界	10
1.3 Vlasov-Maxwell: 源项、约束与闭合	10
1.4 Vlasov-Maxwell 的能量与动量守恒边界	11
1.5 Vlasov-Poisson / electrostatic 极限不是另一套世界	11
1.6 宏粒子不是假粒子, 而是 coarse-grained 分布函数载体	11
1.7 权重可以不同, 但不是没有代价	12
1.8 shape factor 不是插值细节, 而是粒子-网格耦合规则	12
1.9 PIC 的噪声不是 bug, 而是模型代价	13
1.10 Debye 长度、粒子数与统计时间尺度	13
1.11 从连续模型到 PIC 离散变量	14
1.12 这一章对后面源码章节的真正约束	14
1.13 证据范围与继续阅读	15
1.14 练习与源码定位	15
2. PIC 总循环: 从 Vlasov-Maxwell 到离散时间推进	16
2.1 连续模型: Vlasov-Maxwell 系统	16
2.2 宏粒子表示与形函数	17
2.3 leapfrog 时间层	17
2.3.1 ω_p 不是背景常数, 而是时间离散必须尊重的最快等离子体尺度	18
2.3.2 λ_D 不只是一条长度定义, 它直接约束 Δx	18
2.4 FDTD 场更新的数学骨架	19
2.4.1 CFL 不是经验参数, 而是离散 Maxwell 更新的因果上界	20
2.4.2 数值色散从这里开始: 连续光锥被离散 stencil 改写	21
2.4.3 Yee / Nodal / CKC 的差别本质上是离散色散关系不同	21
2.5 PSATD 与 FDTD 的主循环差异	22
2.6 一个完整 PIC step 的离散组织	23
2.6.1 AMR subcycling: 两个时间步不是同一个时间步的重复调用	23
2.6.2 JRhom 与 implicit: 同一个外层 step 内部也可能有不同的时间合同	24
2.7 本章后的源码阅读入口	25
2.8 参数示例与最小运行案例	25
2.9 基础文献与证据范围	26
2.10 进一步阅读与练习	26
2.11 本章结论	27

3. WarpX 主演化路径：生命周期、初始化与 Evolve()	28
源码定位约定	28
3.1 顶层入口：main.cpp	28
3.2 WarpX 类与单例构造	29
3.3 ReadParameters(): 主循环分支的来源	29
3.3.1 构造函数只建“跨 level 外壳”，不直接建完整网格数据	30
3.3.2 AllocLevelData() / AllocLevelMFs() 里，四条 solver 分支的落点不同	30
3.4 InitData(): 把状态准备到可推进	32
3.5 ComputeDt(): 步长不是一个固定常数	32
3.6 Evolve() 外层时间步	33
3.6.1 步末 moving window: 连续坐标与整数网格平移	33
3.7 OneStep(): 求解器分派器	35
3.8 OneStep_nosub(): 显式电磁标准路径	36
3.9 SyncCurrentAndRho(): 源项不是沉积完就可用	38
3.10 PushParticlesandDeposit(): 进入粒子容器	38
3.11 OneStep_sub1() 与 JRhom 的位置	38
3.11.1 implicit 分支：一次物理步包含多次试探性 source 重建	40
3.11.2 nonlinear solver、JFNK 与 mass-matrix: J 的三种构造层	40
3.12 参数示例与最小运行闭环	41
3.13 进一步阅读与练习	42
3.14 本章小结	42
3A. WarpX 初始化链：从 InitData() 到初始粒子和外部场	43
3A.1 初始化链为什么值得单独成章	43
3A.2 启动层先于 InitData(): MPI、AMReX、FFT、PETSc 与启动前提	43
3A.2.1 参数系统先分命名空间，再分读取语义	44
3A.2.2 文档 alias、AMReX-owned 参数与 WarpX 自有 parser 不是一回事	45
3A.3 顶层入口：fresh run 与 restart 分叉	50
3A.4 InitFromScratch(): AMReX level 与粒子初始化	50
3A.5 外部场初始化：grid field 与 particle external field	51
3A.6 species 初始化：PlasmaInjector 是参数总容器	52
3A.7 密度和动量分布：从文本参数到 functor	52
3A.8 AddParticles(): 按注入类型进入创建函数	54
3A.9 体注入 AddPlasma(): 候选粒子、密度、动量和权重	55
3A.10 Gaussian beam: 显式束流列表注入	56
3A.11 openPMD 粒子文件：文件粒子列表注入	57
3A.12 Projection divergence cleaning: 外部 A/B 场的初始约束修正	58
Laser antenna 与 profile 分派：laser 初始化并不走 PlasmaInjector	59
3A.13 初始化验证入口：哪些 regressions 真正在兜底	65
3A.14 从 Birdsall 3A ES1 到 WarpX: 历史最小程序骨架的现代映射	70
3A.15 本章小结：初始化状态怎样进入第一步推进	71
3A.16 练习与最小复现	71
4. 粒子推进器：从 Lorentz 方程到 PushPX()	72
4.1 连续 Lorentz 方程	72
4.2 Boris 推进的结构	72
4.3 WarpX 如何选择 pusher	74
4.4 Vay pusher: 相对论速度变换一致性的更新	75
4.4.1 Vay Appendix A/B: 显式 γ 根与回旋半径边界	77
4.4.2 用推进器谱系约束 Vay 的结论范围	77
4.5 Higuera-Cary pusher: Boris-like 结构的相对论修正	78
4.6 从 MultiParticleContainer 到 PhysicalParticleContainer	80
4.7 PhysicalParticleContainer::Evolve() 的 tile loop	81
4.8 PushPX(): gather 和 push 的融合 kernel	82
4.9 doGatherShapeN(): 从网格场到粒子场	83
4.10 位置推进与无质量粒子	89
4.11 RR、implicit 与 photon path	90

4.12	Field Ionization: ADK 不是附加后处理, 而是粒子属性和沉积权重一起改写	92
4.13	Collisions: 不是旁路模块, 而是和 momentum push 强耦合的时间调度层	93
4.13.1	BinaryCollision 先产出事件表, ParticleCreationFunc 再真正创建 products	94
4.13.2	BackgroundMCC、PulsedDecay 与 DSMC: collision 还有三条完全不同的后半段	95
4.13.3	pairwisecoulomb、bremsstrahlung、background_stopping 与 nuclearfusion	97
4.13.4	kernel 细节层: Perez 有效碰撞密度、Bremsstrahlung photon 写回、fusion products 复制语义	98
4.13.5	更深一层的 event kernel: Perez 角分布、Bremsstrahlung 能谱反演与 fusion 概率控制	99
4.13.6	性能与数值后处理层: resampling、thermalizer 与 sorting	100
4.13.7	Vranic 2015: 两粒子 merge 的论文-源码边界	101
4.13.8	四类单粒子问题: 该测什么, 不能推出什么	102
4.13.9	粒子诊断与外场: 两条不经过主网格场的路径	104
4.13.10	边界缓冲区与 Python 粒子操作: 能控制什么, 尚未覆盖什么	106
4.13.11	边界上的粒子: 内建条件、Python 回调与可验证的物理后果	107
4.13.12	接触记录、PML 粒子与 AMR: 观察对象决定验证强度	109
4.13.13	Embedded boundary: 从几何、吸收和解析场选择验证量	111
4.14	QED: 先分清 source、product 与产生机制, 再谈参数	112
4.14.1	事件开始前: 谁是 source, 谁保存统计状态, 谁接收 product	112
4.14.2	初始化只准备采样条件, 事件要在后续时间步发生	113
4.14.3	时间位置: 在 field ionization 后、用户 injection 前处理 QED	113
4.14.4	两条 source-to-product 事件链: 转化时同时更新 source 与创建 product	113
4.14.5	Schwinger: 由网格场直接创建粒子对, 不存在 source species	114
4.14.6	三条机制分别该看什么: 数目之外还要看权重、方向和守恒	114
4.14.7	事件何时发生: 先演化 optical depth, 再决定是否创建 product	115
4.14.8	分工边界: WarpX 提供粒子与场, PICSAR-QED 提供采样内核	115
4.14.9	source 的命运不同: 辐射会保留 lepton, 成对产生会消耗 photon	115
4.14.10	lookup table 是采样器的一部分, 不是可有可无的附属文件	115
4.14.11	不要被共同的 photon 名称误导: 强场 QED 与碰撞 QED 是两棵树	116
4.15	本章结论	118
4.16	练习与复现实验	118
5.	电荷、电流沉积与形函数: 源项如何回到网格	119
5.1	电荷沉积的基本形式	120
5.2	ShapeFactors.H: WarpX 实际使用的 0 到 4 阶形函数	120
5.3	电流沉积的守恒要求	123
5.3.1	五条路径的责任边界总览	124
5.4	WarpX 的旧电荷、新电荷和半步电流	125
5.5	多物种层如何清零和汇总源项	126
5.6	AMR coarse-fine interface: 粒子怎样切到 aux/cax/buf 路径	126
5.7	WarpXParticleContainer::DepositCurrent() 分派	128
5.8	WarpXParticleContainer::DepositCharge() 入口	133
5.8.1	icomp 不只是分量号, 而是旧/新 rho 的时间层合同	134
5.8.2	普通 charge deposition 的桥接合同在 ABLASTR, 而不在 kernel 本体	135
5.8.3	shared-memory charge deposition 不是走 ABLASTR, 而是先变成 tile-binned 执行合同	136
5.9	ChargeDeposition.H: 电荷沉积 kernel 的逐项结构	137
5.10	Direct current deposition: 非守恒但直观的速度加权沉积	141
5.11	Esirkepov current deposition: 用新旧形函数差构造连续性方程	143
5.11.1	论文、源码、代数合同与 runtime 证据的分层	147
5.11.2	Villasenor-Buneman 公式级审计: 四边界、重复分段与三维交叉项	153
5.11.3	Esirkepov 运行级维度证据: 1D、2D 与 3D Langmuir	156
5.12	沉积不只回 rho/J: WarpX 还把线性响应矩阵和统计矩交回网格	156
5.12.1	mass matrices: 沉的是 J(E) 的线性响应, 不是当前步的 Maxwell 源项	157
5.12.2	temperature / variance deposition: 沉的是样本数、加权均值和去均值二次矩	157
5.12.3	这一节回头修正了本章的总边界	158
5.13	沉积后为什么还要同步	158
5.13.1	PSATD 与 FDTD 的同步时序不同	159
5.13.2	SyncCurrent() 的骨架: coarsen、buffer、owner mask、filter、SumBoundary	159
5.13.3	SyncRho() 平行但不完全等于 SyncCurrent()	160

5.13.4	SyncCurrentAndRho() 的最后一步其实是边界条件	160
5.13.5	本章对应的两条关键 regression, 分别在验证哪一层 source-synchronization 合同	161
5.14	形函数、guard cells 与稳定性	162
5.14.1	源码定位与结论范围	166
5.14.2	覆盖范围与已知空白	166
	读者主线: 从守恒问题走到可解释的输入选择	167
5.14.3	选择沉积算法: 先问约束, 再问精度	167
5.14.4	读懂证据: 公式、源码和运行结果各回答什么	168
5.14.5	RZ axis residual: 把局部诊断和算法错误分开	168
5.14.6	收敛研究: 描述性趋势不是正式阶数	168
5.15	本章结论	168
5.16	练习与源码定位	169
6.	电磁场求解器	170
	读者主线: 从同步源项到可检验的场	170
	选择路径前的检查表	171
6.1	FDTD 差分算子: Yee、Nodal 与 CKC	171
6.2	FDTD PML split-field 更新	171
6.3	PML sigma profile、damping 与电流源项	173
6.4	非 Cartesian FDTD: RZ、RCYLINDER 与 RSPHERE	173
6.5	PSATD 谱求解主流程	174
6.6	标准/Galilean PSATD 系数和 current correction	176
6.6.1	先按更新对象理解 PSATD 系数	176
6.6.2	Galilean PSATD 解决的是表示问题, 不是滤波开关	177
6.6.3	从 regression 反推可作出的结论	177
6.7	PSATD-JRhom: 多次源项沉积与一阶/二阶谱更新	177
6.7.1	JRhom 的选择依据是源项时间模型	180
6.7.2	组合限制与验证边界	180
6.8	RZ PSATD: Hankel transform、azimuthal modes 与 E_p/E_m	180
6.8.1	RZ 中先认清字段表示, 再读系数	182
6.8.2	Comoving 与 Galilean 都有相位, 但问题不同	182
6.8.3	用算法选择表约束结论范围	183
6.9	静电与静磁求解器	183
6.9.1	Poisson 边界条件	184
6.9.2	Lab-frame 静电	184
6.9.3	Relativistic self fields	185
6.9.4	Effective potential	185
6.9.5	Magnetostatic vector Poisson	186
6.10	Hybrid PIC: 广义 Ohm 定律与 B 场 RK 子步	188
6.10.1	参数与辅助场	188
6.10.2	顶层 field update	189
6.10.3	Ampere 电流与 Ohm kernel	190
6.10.4	电子压力与外部矢势 split field	191
6.10.5	Fluids/ 与 HybridPICModel 不是同一条流体链	192
6.11	FieldSolver regression 判据: 从 analysis 脚本反读物理检查量	192
6.11.1	NCI FDTD: 场能增长是否被 corrector 压住	192
6.11.2	NCI PSATD: 电场能量比与 Gauss law	193
6.11.3	Maxwell hybrid QED: 真空修正后的相速度偏移	194
6.11.4	K ₄ 、QPM 与 thermal-plasma 长期 figure of merit	195
6.11.5	静电球: 解析场 L2 误差与能量守恒	195
6.11.6	隐式 EM: 能量、Gauss law 与求解器迭代数	198
6.11.7	Hybrid Ohm solver: 哪些是强判据, 哪些只是输出回归	202
6.11.8	具体 regression 入口索引	203
6.11.9	从源码入口回查验证量	204
6.12	练习与运行验证	205
6.13	本章结论	205

7. 边界条件、PML 与 AMR	206
本章的阅读路线：边界是一个闭合系统	206
7.0 源码入口地图	206
7.1 field / particle 边界不是两套彼此独立的输入	207
7.2 电磁 field boundary 的顶层分派	207
7.3 PEC / PMC 不只是场边界，也是沉积对称性	208
7.4 PML 在 WarpX 里是独立子域，不是单个边界公式	209
7.5 PML split field 与 PML 电流	209
7.5.1 用正确的 observable 判断 PML	210
7.5.2 从 split field 到 runtime evidence	211
7.6 Embedded boundary 先是几何初始化和辅助标记系统	211
7.7 Embedded boundary 的 face extension: 把不稳定 cut face 变成 enlarged face	212
7.8 Embedded boundary 的粒子侧: signed-distance 判定、默认吸收与 scraped buffer	214
7.9 AMR transition zone: 为什么最终 plotfile 不足以证明路由正确	217
7.10 本章练习与源码定位	218
7.11 本章结论	218
8. 诊断、验证与案例	219
Dawson 统计诊断链：从 modal energy 到 normal-mode reconstruction	220
Langmuir wave	220
Uniform plasma	222
Checkpoint/restart 的运行证据	223
LWFA/PWFA	224
LWFA: laser_acceleration 是 runtime matrix, 不是统一 wake benchmark	225
PWFA: plasma_acceleration 是 workflow matrix, 不是解析 wake benchmark	225
LWFA/PWFA 案例能够支持的结论	226
Laser ion / plasma mirror / RPA/TNSA	226
laser_ion: 最强的 laser-target application entry	226
plasma_mirror: laser-solid surface-plasma workflow baseline	227
RPA/TNSA: 当前属于物理解释层，不属于独立应用目录层	227
Capacitive discharge	228
1D PICMI 是当前最强的 Turner benchmark 入口	228
当前最硬断言是 case-1 ion-density profile	228
DSMC 版不是孤立小 test, 而是同一 benchmark scaffold 的分支	229
2D native / PICMI 当前仍主要是 workflow baseline	229
既有 plasma_acceleration 目录边界	229
Magnetic reconnection	230
force-free sheet + reduced FieldProbe	230
当前 analysis 是 observable extraction, 不是强 assert	230
checksum 仍是 active coverage 的另一半	231
Beam-beam / luminosity / FEL / ion extraction	231
DifferentialLuminosity: reduced-diagnostic 强谱基准	231
beam_beam_collision: collider-QED 应用骨架	232
free_electron_laser: boosted rigid-beam + undulator + BTD 的强 benchmark	232
ion_beam_extraction: EB electrostatic extraction 的强应用入口	233
accelerator_lattice: beamline optics 的强回归层	233
诊断在源码中的位置	234
BoundaryScrapingDiagnostics 与 Python scraped-particle buffer	239
固定模板案例页	240
模板 A: plotfile	240
模板 B: openPMD	240
模板 C: checkpoint	241
模板 D: BoundaryScraping/openPMD	241
Python 边界 buffer 的最小消费模板	242
模式 A: 运行结束后统一检查	242
模式 B: callback 或在线控制里的即时事件流	243
这和 BoundaryScrapingDiagnostics 的分工	244

Python field wrapper 的最小强回归	244
MPI transport 环境注意	245
Langmuir 与均匀等离子体的运行证据	245
Reduced diagnostics 与 full-state reference 对照	246
LoadBalanceCosts: 性能诊断的 efficiency gate	247
ColliderRelevant: 束流统计与 luminosity-rate 聚合合同	247
DifferentialLuminosity: 1D/2D 解析谱与 AMR 对照	248
ParticleHistogram2D: 二维 openPMD writer 合同	249
BeamRelevant: 束流矩与截断高斯束合同	251
Native external-file Gaussian beam: 束斑理论包络检查	251
第 8 章验证矩阵: 观察量、结论与边界	252
8.14 从诊断入口到可解释证据	253
8.14.1 三类 reduced diagnostics 的最小起点	253
8.15 练习与复现实验	253
8.16 延伸验证路线	253
8.17 本章结论	253
9. 文献路线与延伸阅读	255
本章的读者用法: 文献是论证工具, 不是书目清单	255
9.1 文献证据的使用层级	255
9.2 支撑各章的核心文献	255
9.2.1 PIC foundations	255
9.2.2 Particle pusher	256
9.2.3 PSATD / Galilean / boosted-frame / NCI	256
9.3 关键来源的已知边界	256
9.4 各章的文献覆盖范围	257
9.5 延伸阅读的优先顺序	257
9.6 用一张文献判读卡连接论文、源码与算例	258
9.7 两条深读路线	258
路线 A: 第 5 章沉积文献	258
路线 B: 第 7 章 PML 文献	258
9.7.1 深读笔记的核查边界	258
9.8 如何阅读证据边界	259
9.9 本章结论	259
9.10 练习与复核	259
9.10.1 证据层分类练习	259
9.10.2 证据边界复核练习	259
9.10.3 延伸阅读排序练习	259
附录 A: 符号、时间层与源码变量	260
A.1 连续模型符号	260
ux, uy, uz 的两层约定	260
A.2 网格、时间层与离散量	261
A.3 沉积、推进与场求解记号	261
A.4 诊断、文件与验证术语	262
A.5 常用缩写	262
A.6 使用规则	262

PIC-tutor

这是面向读者的 PIC 教程开篇，不是开发日志。书稿从连续的 Vlasov-Maxwell 模型出发，逐步建立宏粒子、网格、时间推进、粒子推进器、沉积、场求解器、边界、AMR 与诊断之间的关系，最后用 WarpX 的源码和可运行案例把这些概念落到程序行为上。

读者在本版可以学到什么

- 用 Vlasov-Maxwell 方程解释 PIC 中粒子与场为什么必须相互交换源项。
- 从 leapfrog 时间层读懂一个显式 PIC step，而不是把 `Evolve()` 看成黑盒。
- 区分 Boris、Vay 与 Higuera-Cary 等粒子推进器的物理假设、离散结构和适用边界。
- 解释 shape factor、charge/current deposition、current correction、guard cell 与 AMR 同步如何共同影响守恒和噪声。
- 根据 CFL、Debye 长度、plasma frequency、边界和诊断量选择一个可解释的输入案例。
- 用源码路径、输入参数、输出量和回归分析共同判断“程序运行了”是否等于“物理结果可信”。

如何判断一个结论

每一项重要结论都应先问它属于哪一层证据：

1. 数学和文献说明连续模型、离散近似和适用假设；
2. 源码映射说明程序实际在哪个对象或函数中实现这一近似；
3. 输入和诊断说明某个案例究竟比较了什么 observable；
4. 运行结果只能支持该输入、几何、算法与容差下的结论，不能自动推广到其他分支。

因此，case 通过 regression 并不等于任意配置都正确；公式成立也不等于每条实现路径都已覆盖。书中出现“边界”“未覆盖”或“不能外推”时，指的是证据强度的范围，而不是回避结论。

建议的阅读方式

第一次阅读时，沿着每章的“问题 -> 方程或离散规则 -> 源码职责 -> 最小案例 -> 练习”完成主线。遇到函数名或测试名时，不必先记住它们；先回答它消费什么数据、产出什么数据、处在哪个时间层，以及应由哪个 observable 检验。第二次阅读再沿源码文件和输入案例回查细节。

0. 写作说明

这本书为谁而写

如果你会运行一个 PIC case，却还不能回答“粒子在什么时间层被推进”“电流为什么要沿轨迹沉积”“一个小的 `divE-rho` 残差究竟说明什么”，这本书就是从这里开始的。它不要求读者先熟悉 WarpX 的类层次，也不把源码中的函数名当作知识本身；每个实现细节都先放回物理量、离散方程和可观察结果中解释。

读完主线后，读者应当能够：

- 从连续方程推导出 PIC 中需要保存和交换的离散量；
- 沿着一个输入参数追踪到初始化、时间推进、沉积、场更新和诊断输出；
- 为一个新 case 选择时间步、网格、shape、solver 和诊断，而不是只复制输入文件；
- 看见 regression 通过时，准确说出它证明了什么、还没有证明什么。

如何使用本书

正文按“模型 -> 离散算法 -> 源码调用链 -> 案例诊断”的顺序展开。章节末的练习要求读者自己定位源码、检查时间层或复现实验；它们不是附加的项目验收清单，而是把阅读变成判断能力的训练。

版本号、运行合同和缺口登记记录的是书稿如何被维护，不是读者必须按时间顺序阅读的内容。正文只在需要解释证据范围时使用它们，不把它们作为学习前提。

本书不是 WarpX 官方文档的翻译，也不是只讲公式的 PIC 理论笔记。它的主线是：先从 Vlasov-Maxwell / Vlasov-Poisson 这类连续模型出发，说明为什么需要宏粒子；再把宏粒子、网格、形函数、沉积和场求解拼成 PIC 算法；最后回到 WarpX 的 `Source/`、`Docs/`、`Examples/` 和 `regression` 入口，解释一个现代高性能 PIC 程序如何把这些步骤组织成可运行、可扩展、可验证的模拟软件。

源码定位应优先依赖文件职责、函数名和调用关系，而不是固定 `commit` 或行号。例如，主循环从 `Evolve()` 与 `OneStep()` 开始，初始化从 `InitData()` 开始，粒子/源项主链从 `PushParticlesandDeposit()` 与 `SyncCurrentAndRho()` 开始。版本信息只用于复现实验和维护记录，不应改变读者对算法关系的理解。

本书的每个技术判断都应尽量落到六类证据：物理方程、离散公式、WarpX 源码路径、输入参数、示例或测试、文献。DeepWiki、Zread 等 AI 解读页面可以用来快速找到模块名，但不能作为最终依据。

遇到一个新的输入或源码分支时，可按四步阅读：先写出要描述连续物理量；再标出它在离散网格和时间层上的表示；随后定位 producer 和 consumer；最后选择一个能区分正确与错误的 observable，并写下该观察量不能支持的外推。这样，代码阅读始终服务于物理判断，而不是变成函数名清单。

本书默认使用以下记号：粒子位置为

$$\mathbf{x}_p$$

，粒子动量为

$$\mathbf{u}_p = \gamma \mathbf{v}_p$$

，电磁场为

$$\mathbf{E}, \mathbf{B}$$

，电荷和电流密度为

$$\rho, \mathbf{J}$$

，粒子权重为

$$w_p$$

，形函数为

$$S$$

。网格量的上标表示时间层，例如

$$\mathbf{B}^{n+1/2}$$

；粒子量一般按 leapfrog 交错在位置和动量时间层上。

阅读建议：先读第 1-3 章建立“物理-算法-代码调用链”的整体图，再读第 4-7 章理解各个核心模块，最后用第 8 章的 Langmuir wave 和 uniform plasma 案例检查自己是否真正能把输入参数、源码和输出诊断连起来。

1. 动理学模型与 PIC 的基本思想

PIC 代码不是先有“粒子数组”和“场数组”，再去给它们找物理意义。它的上游模型是动理学方程：对每个物种，真实对象首先是相空间分布函数

$$f_s(\mathbf{x}, \mathbf{p}, t),$$

而不是单个粒子轨道。本章的任务是把这条连续模型主线压实到后面源码会反复用到的几个边界：

1. Vlasov 方程本质上是相空间守恒定律，而不只是“一个偏微分方程”。
2. Vlasov-Maxwell 与 Vlasov-Poisson 不是两套无关模型，而是同一条自洽场闭合在不同物理极限下的分支。
3. 宏粒子、权重和 shape factor 不是数值技巧附会到物理上，而是 coarse-grained kinetic model 的一部分。
4. PIC 的主要误差不是单一来源，而是采样噪声、有限粒子权重、有限网格和离散时间层共同作用的结果。

本章的阅读支点是：

- Birdsall 1985
- Dawson 1983
- 第 2、3、5、6 章对 WarpX 主循环、沉积和场求解器的实现说明

Hockney-Eastwood 与 Yee 1966 在本书中只作为补充书目信息，而不作为可逐页核对的全文依据；本章不把无法核实的历史细节当作结论。读者应先把这里的连续模型、离散变量和误差边界读清，再进入源码章节。

1.1 Vlasov 方程首先是相空间守恒律

对物种 s ，若忽略碰撞、衰变、电离和其他 source/sink，分布函数满足无碰撞相对论 Vlasov 方程

$$\frac{\partial f_s}{\partial t} + \dot{\mathbf{x}} \cdot \nabla_{\mathbf{x}} f_s + \dot{\mathbf{p}} \cdot \nabla_{\mathbf{p}} f_s = 0.$$

对带电粒子，

$$\dot{\mathbf{x}} = \mathbf{v}, \quad \dot{\mathbf{p}} = q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}),$$

因此可写成更常见的形式

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = 0.$$

这里真正需要记住的不是式子本身，而是它的守恒含义。把相空间速度记成

$$\mathbf{Z} = (\mathbf{x}, \mathbf{p}), \quad \dot{\mathbf{Z}} = (\dot{\mathbf{x}}, \dot{\mathbf{p}}),$$

则 Vlasov 方程也可写成

$$\frac{\partial f_s}{\partial t} + \nabla_{\mathbf{Z}} \cdot (f_s \dot{\mathbf{Z}}) = 0.$$

若相空间流满足

$$\nabla_{\mathbf{Z}} \cdot \dot{\mathbf{Z}} = 0,$$

就得到 Liouville 图像：沿特征线传播时，分布函数值保持不变，相空间体积也不被压缩或膨胀。对后面的 PIC 来说，这一点比公式本身更基础，因为它解释了为什么：

- 粒子推进器不能随意破坏轨道拓扑；
- 离散时间推进若不尊重相空间结构，就会把数值伪耗散或伪加热写进分布函数；
- Boris / Vay / Higuera-Cary 这类 pusher 的比较，不只是在比轨道误差，而是在比它们怎样离散化相空间流。

1.2 碰撞项不是 Vlasov 的一部分，但必须保留边界

一旦考虑碰撞、衰变、外部源项或粒子数变化，更一般的 kinetic equation 应写成

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = C_s[f] + S_s - L_s.$$

这里：

- $C_s[f]$ 表示碰撞算子，可以是 Boltzmann、Landau、Fokker-Planck 或 Monte-Carlo 近似；
- S_s 表示外加源项，如电离生成、注入、衰变产物；
- L_s 表示损失项，如吸收、衰变消失、边界流出。

这条边界在 WarpX 里很重要，因为：

- 主 PIC loop 默认走的是无碰撞 Vlasov-Maxwell 主线；
- CollisionHandler、field ionization、QED、ContinuousFluxInjection、粒子边界吸收/反射，都是往这条无碰撞主线上附加 C/S/L；
- 它们不该被写成“另一个独立程序”，而应被理解成对同一 kinetic balance 的修改。

因此本书后面凡是讲 collisions、ionization、QED、scraping，都应问两个问题：

1. 它改的是分布函数右端的哪一项？
2. 它是在一个时间步的哪个时间层插入进去？

否则很容易把“物理过程存在”和“离散时间组织正确”混成一件事。

1.3 Vlasov-Maxwell：源项、约束与闭合

相对论动量与速度满足

$$\mathbf{p} = \gamma m_s \mathbf{v}, \quad \gamma = \sqrt{1 + \frac{|\mathbf{p}|^2}{m_s^2 c^2}}, \quad \mathbf{v} = \frac{\mathbf{p}}{\gamma m_s}.$$

分布函数的低阶矩给出 Maxwell 方程右端的源项：

$$\rho(\mathbf{x}, t) = \sum_s q_s \int f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p},$$

$$\mathbf{J}(\mathbf{x}, t) = \sum_s q_s \int \mathbf{v}(\mathbf{p}) f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}.$$

然后场满足

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}, \quad \nabla \cdot \mathbf{B} = 0,$$

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t}, \quad \nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \frac{1}{c^2} \frac{\partial \mathbf{E}}{\partial t}.$$

这四式里最容易被误写的是：Gauss 定律和 $\nabla \cdot \mathbf{B} = 0$ 不是“额外条件”，而是系统闭合的一部分。只要连续性方程

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{J} = 0$$

成立，并且初值满足约束方程，Maxwell 演化就会传播这些约束。反过来，如果离散沉积和离散场推进不一致，那么程序里最先坏掉的往往不是 curl 更新，而是：

- `divE-rho/epsilon0`
- `divB`

- 边界附近的伪电荷
- 以及随之而来的非物理电场和数值加热

这正是后面第 5 章和第 6 章为什么要反复围着 source synchronization、current correction、Gauss-law regression 打转。

1.4 Vlasov-Maxwell 的能量与动量守恒边界

在闭域、无外源、忽略边界通量时，连续系统满足总能量守恒：

$$\frac{d}{dt} \left[\sum_s \int \gamma m_s c^2 f_s d\mathbf{x} d\mathbf{p} + \int \left(\frac{\epsilon_0}{2} |\mathbf{E}|^2 + \frac{1}{2\mu_0} |\mathbf{B}|^2 \right) d\mathbf{x} \right] = 0.$$

它说明两件事：

1. 粒子动能和场能不是分开各自守恒，而是可以相互交换。
2. 程序里若只监控粒子能量或只监控场能量，都不足以判断离散系统是否健康。

同理，总动量守恒应理解为“粒子动量 + 电磁场动量 + 边界 Maxwell stress”一起守恒，而不是单看粒子束团动量曲线。对后面的 implicit、hybrid、electrostatic sphere、planar pinch 和 FEL 例子，这个边界都非常关键：很多 regression 真正检查的是完整能量账本，而不是某个单独变量“看起来没漂”。

1.5 Vlasov-Poisson / electrostatic 极限不是另一套世界

当系统关注的是电荷分离和纵向静电响应，而电磁波传播、辐射和横向磁反馈不是主导效应时，可以把自治场闭合约化到 Vlasov-Poisson：

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s \mathbf{E} \cdot \nabla_{\mathbf{p}} f_s = 0,$$

$$\mathbf{E} = -\nabla \phi, \quad -\nabla^2 \phi = \frac{\rho}{\epsilon_0}.$$

它不是凭空把 Maxwell 方程换掉，而是对应这样一组物理假设：

- transverse electromagnetic radiation 不是主要自由度；
- 场主要由电荷分离决定；
- 电磁传播时间尺度不是当前主导尺度；
- 所关心的现象更接近 Langmuir、space-charge、electrostatic expansion、Poisson boundary-value problem。

因此：

- electrostatic PIC 仍然是 kinetic PIC；
- 只是“粒子如何给场提供源项、场如何回馈粒子”这一闭合从 Maxwell 变成了 Poisson。

这也是为什么：

- 第 6 章不能把 electrostatic solver 当作“Maxwell solver 的低配版”；
- electrostatic sphere、Pierce diode、effective potential 这些例子要和 Poisson 边界条件、势能账本一起讲；
- WarpX::OneStep() 里 electrostatic / hybrid 路线的场解位置会和标准 electromagnetic loop 不同。

1.6 宏粒子不是假粒子，而是 coarse-grained 分布函数载体

PIC 的核心近似不是把等离子体变成少数真实粒子，而是用有限数量的宏粒子采样分布函数。形式上可写成

$$f_s(\mathbf{x}, \mathbf{p}, t) \approx \sum_{p \in s} w_p S_x(\mathbf{x} - \mathbf{x}_p(t)) S_p(\mathbf{p} - \mathbf{p}_p(t)).$$

这里：

- w_p 是宏粒子权重；
- S_x 是空间形函数；
- S_p 常在实际 PIC 中退化成粒子自身在动量空间的离散采样。

从 Dawson 1983 的角度，更准确的说法是：宏粒子不是“把许多真实粒子团成一个球”的形象化故事，而是 coarse-grained kinetic model 的载体。它的目标是：

1. 用有限自由度代表连续分布；
2. 保住真正重要的低阶矩和 collective behavior；
3. 接受某些细粒度 phase-space 结构会被采样误差和 coarse graining 吞掉。

1.7 权重可以不同，但不是没有代价

宏粒子并不必然等权。Dawson 1983 讨论了

$$q_i = -\alpha_i e, \quad m_i = \alpha_i m, \quad \frac{q_i}{m_i} = -\frac{e}{m}$$

这一类不同电荷和质量、但相同荷质比的电子群，并说明由它们组成的加权分布函数仍满足通常的 Vlasov 方程。

这条结论的意义是：

- weighted macroparticles 从一开始就是合法的 kinetic coarse graining；
- 它允许把 phase-space 分辨率集中到真正需要的区域；
- 但它也会引入新的统计与 collisional side effects。

所以“加权宏粒子”更准确的理解不是“自适应采样免费升级”，而是：

- 你获得了 phase-space resolution redistribution；
- 但要付出更复杂的噪声、散射和统计解释代价。

这条边界对后面理解 WarpX 中：

- species 权重
- Gaussian beam / flux injection
- collision/QED product creation
- reduced-dimension weighting compensation

都很重要。

1.8 shape factor 不是插值细节，而是粒子-网格耦合规则

若把粒子源项沉积到网格单元或网格点 i ，最基本的电荷密度形式是

$$\rho_i^n = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(\mathbf{x}_p^n).$$

场 gather 则用同一类 shape family 从网格插值回粒子位置：

$$\mathbf{E}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{E}_i^n, \quad \mathbf{B}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{B}_i^n.$$

但 shape factor 的意义远不止“双向插值”：

1. 它定义了宏粒子在空间上的 coarse-grained 电荷云。
2. 它决定了粒子-网格耦合的 stencil 宽度。
3. 它会系统改写短波 aliasing、self-force 和统计噪声。
4. 它直接影响 guard-cell 需求、通信宽度和算子局域性。

Birdsall 1985 与 Dawson 1983 的共同结论都指向这一点：finite-size particles 不是为了把图画得更平滑，而是为了软化 point-charge 的短程奇异作用、压低非物理 collisionality，并把系统真正保留成“长程 collective physics + 可控短程误差”。

这也是为什么后面的第 5 章必须把：

- shape factor
- charge/current deposition

- sampled density
- finite-grid effects
- aliasing

放在同一章里讲，而不是把 shape factor 单独缩成一个插值小节。

1.9 PIC 的噪声不是 bug，而是模型代价

把连续分布函数换成有限宏粒子之后，最基本的代价就是采样噪声。它不是代码写坏了才出现，而是：

- 有限粒子数
- 有限权重
- 有限网格
- 有限时间平均

共同带来的统计涨落。

从 Birdsall 1985 的 thermal-plasma 讨论看，这种噪声不能只被理解成“粒子数不够大”。更准确的图像是：

1. sampled density 会生成 alias branches;
2. shape factor 会修改 fluctuation spectrum;
3. finite Δx 和 finite Δt 会把 continuum 改写成带离散谱结构和 effective transport 的系统;
4. 若离散耦合处理不好，噪声会演化成 numerical heating、drag、diffusion，甚至弱不稳定增长率的误判。

因此，本书后面凡是说“噪声更小”“结果更平滑”，都不应只停在图像层，而应继续问：

- 是 modal fluctuation level 变了?
- 是 alias branch 被压了?
- 还是只把可见图像平滑了，但守恒与统计量并没有更好?

1.10 Debye 长度、粒子数与统计时间尺度

Birdsall 1985 对 sheet model 的讨论给了一个比教科书定义更适合写进程序书的视角。

首先，在线性、近 Maxwellian 的三维等离子体中，热速率 v_t 、等离子体频率和 Debye 长度可写为

$$v_t = \sqrt{\frac{k_B T_s}{m_s}}, \quad \omega_{ps} = \sqrt{\frac{n_s q_s^2}{\epsilon_0 m_s}}, \quad \lambda_{Ds} = \frac{v_t}{\omega_{ps}} = \sqrt{\frac{\epsilon_0 k_B T_s}{n_s q_s^2}}.$$

其中 n_s 是物种数密度， T_s 是温度， k_B 是 Boltzmann 常数。对应的 Debye 球内粒子数近似为

$$N_D \sim (4\pi/3)n_s \lambda_{Ds}^3.$$

这些量的精确定义会随单位制、速度矩约定、磁化程度和 reduced geometry 改变；这里使用它们是为了建立尺度判断，而不是把二维或轴对称计算机械地当成三维 Debye 球。Debye 长度 λ_D 和 Debye 球内粒子数 N_D 因而不是孤立的公式，而是“这个 plasma 是否能被当作 collective medium”与“统计噪声会以什么尺度渗入观测量”的共同边界。

其次，在 reduced model 下，

$$\tau \sim \frac{2N_D}{\omega_p}$$

更适合被理解成：

- randomization time
- correlation time
- 统计独立采样间隔

而不是整个分布完全热化成 Maxwellian 的总弛豫时间。后者通常更慢，量级更接近 N_D^2 。

对 PIC 用户来说，这比“记住 Debye 长度定义”更实用，因为它直接影响：

- uniform-plasma 噪声底怎么看；
- reduced diagnostics 应平均多久；
- 弱效应、弱不稳定和 Landau damping 的 measurement window 多大才可信。

1.11 从连续模型到 PIC 离散变量

前面的方程还没有直接变成程序里的数组。PIC 的第一步不是把分布函数存成一个高维网格，而是用带权粒子样本代表它，再用网格上的有限差分或谱变量承载场。可以把这条映射写成下面的最小对应关系：

连续对象	PIC 离散载体	典型时间层/位置	在 WarpX 代码中应如何理解
$f_s(x, p, t)$	物种粒子的位置、动量、权重集合	$x_p^n, p_p^{n-1/2}, w_p$	ParticleContainer 中的粒子样本，不是一个逐点存储的分布函数
$\rho(x, t)$	shape-weighted charge density	ρ^n 或 $\rho^{n+1/2}$	由粒子沉积得到；rho_fp 与 rho_buf 还可能分别属于 fine 与 coarse-buffer 路径
$J(x, t)$	trajectory-based current density	通常跨越 $n- > n+1$	Esirkepov/Villasenor 等 current deposition 需要 old/new 轨迹或等价的 crossing 信息
E, B	staggered/collocated grid fields	由 solver 规定	Efield_*, Bfield_* 的后缀表示时间层、网格位置或辅助副本，不能只按变量名猜物理时刻
连续性方程	离散 source synchronization	每个粒子推进步或 solver stage	SyncCurrentAndRho()、guard exchange、AMR average-down 等共同完成可供场求解器使用的 source

最容易被忽略的是 ρ 和 J 的时间语义不同。单时间层的 charge deposition 可以直接对粒子位置取样：

$$\rho_i^n = (1/\Delta V_i) \sum_p q_p w_p S_i(x_p^n),$$

而守恒的 current deposition 必须表达粒子从 x_p^n 到 x_p^{n+1} 的输运：

$$\Delta t (\nabla_h \cdot J)_i = \rho_i^n - \rho_i^{n+1}.$$

这里的第二式不是说所有 current deposition 都在程序中显式先算出左右两边，而是说明它们必须满足同一条离散连续性关系。也因此，

- DepositCharge() 负责单时间层的 ρ 采样及其时间层、几何和 AMR 桥接；
- DepositCurrent() 及其 Esirkepov/Villasenor 等路径负责轨迹输运产生的 J ；
- SyncCurrentAndRho() 负责把不同 level、边界和 source buffer 中的结果整理成 solver 可消费的源项。

这三层不能合并成“粒子把电荷写到网格”一句话。后续第 5 章会从 kernel 角度展开，第 6 章则会继续说明不同 field solver 如何消费这些 source。附录 A 给出 rho_fp、rho_buf、current_fp、current_buf 和 lev 等 WarpX 实现变量的速查定义。

1.12 这一章对后面源码章节的真正约束

到这里，后续读 WarpX 代码时至少要带着下面这些硬问题，而不是只盯函数名：

1. 粒子推进器是否在离散时间层上合理近似了 Liouville 流？
2. 沉积算法是否把连续性方程离散闭合到了 rho/J？
3. field solver 处理的是 Maxwell 还是 Poisson，约束方程怎样传播？
4. shape factor 和 finite-size particles 是如何改写噪声、aliasing 和 self-force 的？
5. diagnostics 到底在测真实物理量，还是只在看离散噪声底的一个投影？

如果没有这几层边界，后面源码里的：

- OneStep_nosub
- PushParticlesandDeposit
- SyncCurrentAndRho
- PushPSATD
- ElectrostaticSolver
- ImplicitSolver

都会被读成“工程控制流”，而不是“连续模型的离散化实现”。

1.13 证据范围与继续阅读

本章下列论述可直接回到两部已提供逐段讲解的基础来源：

- Birdsall 1985: sheet model 的 randomization / correlation / thermalization 时间尺度, 以及 finite-grid / aliasing / fluctuation / heating 主线。
- Dawson 1983: numerical experiment 视角、superparticle / weighted particles 的 kinetic 边界, 以及 finite-size particles、网格和 FFT-Poisson 的 electrostatic contract。

下面两项适合作为后续补充阅读, 但本章不依赖其未取得全文的细节：

- Hockney-Eastwood: 加权粒子、heating estimates 和 optimum path 的经典表述。
- Yee 1966: staggered FDTD 与离散约束传播的原始出处。

继续阅读时, 应优先回到原始论文、教材或官方文档, 并为每一条主张标明它属于连续模型、离散算法还是特定代码实现; 不要用其中任一层代替另外两层。第 2 章会把这里的 leapfrog、CFL、Debye 长度和数值色散接到同一条离散主循环上。

1.14 练习与源码定位

1. **变量桥接题**: 根据 1.11 的映射表, 说明为什么 `rho_fp/rho_buf` 不能直接当作两个不同物理量, 并指出它们分别在哪个 AMR/source-synchronization 场景出现。
2. **尺度判断题**: 给定 $\lambda_D/\Delta x = 0.5$ 和 $v_t \Delta t/\Delta x = 1.2$, 列出至少两个可能的数值风险, 并说明它们分别属于空间分辨率、粒子跨单元输运还是时间推进约束。
3. **源码定位题**: 从 `Source/Evolve/WarpXEvolve.cpp` 的 `OneStep_nosub()` 开始, 而不是从全局搜索结果中猜入口。完成下列三步, 并交付一个三行表格 (函数、连续对象、调用后可认为“已准备好”的数据):
 - 记录 `PushParticlesandDeposit()` 与 `SyncCurrentAndRho()` 的相对次序; 前者对应粒子输运和源项沉积, 后者应使场求解前的 J/ρ 经滤波、guard-cell/MR 同步和边界处理而可被消费。
 - 打开 `WarpX::SyncCurrentAndRho()` 的定义, 列出其中至少两项 source 数据准备动作, 并说明它们为什么不能由连续方程本身自动保证。
 - 任选一个场更新分支: PSATD 路线继续定位 `PushPSATD()` (定义在 `Source/FieldSolver/WarpXPushFieldsEM.cpp`); FDTD 路线则定位 `EvolveB()/EvolveE()` 的调用。说明该分支把哪一种离散 Maxwell 闭合推进到下一时间层。

2. PIC 总循环：从 Vlasov-Maxwell 到离散时间推进

本章先不急进入某一个 WarpX 函数。生产级 PIC 代码的困难不在于“有粒子、有网格、有 Maxwell 方程”这几个名词，而在于这些对象必须在离散时间层、离散空间布局、并行 guard cells、边界条件和守恒约束之间保持一致。后续逐行读 WarpX 时，本章给出判断代码是否“物理上在做正确事情”的基准。

本章给出的函数名和调用关系是读代码的定位方法，而不是某一份源码树的版本标签。下文的 Source/... 与 Examples/... 均相对于 WarpX 源码根目录；使用不同 WarpX 版本时，应先从 Evolve()、OneStep()、PushParticlesandDeposit() 和 SyncCurrentAndRho() 等职责明确的入口重新建立调用关系，而不要把行号或本机工作区布局当作稳定 API。

Yee 1966 在本书中只承担一个窄的历史定位：有限差分 Maxwell 方程通过合适的场点布置处理导体边界。它不能替代本章的 stencil、时间层或色散推导；这些内容由连续方程、离散推导和现代实现三层分别说明。CartesianYeeAlgorithm.H、FiniteDifferenceSolver.cpp、EvolveB.cpp 与 EvolveE.cpp 的交叉定位可供读者复查，但现代代码不构成对历史论文逐式等价的证明。

2.1 连续模型：Vlasov-Maxwell 系统

对物种 s ，相空间分布函数 $f_s(\mathbf{x}, \mathbf{p}, t)$ 满足 Vlasov 方程：

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = 0.$$

相对论动量与速度满足

$$\mathbf{p} = \gamma m_s \mathbf{v}, \quad \gamma = \sqrt{1 + \frac{|\mathbf{p}|^2}{m_s^2 c^2}}, \quad \mathbf{v} = \frac{\mathbf{p}}{\gamma m_s}.$$

电磁场满足 Maxwell 方程：

$$\frac{\partial \mathbf{B}}{\partial t} = -\nabla \times \mathbf{E},$$

$$\frac{\partial \mathbf{E}}{\partial t} = c^2 \nabla \times \mathbf{B} - \frac{\mathbf{J}}{\epsilon_0},$$

以及约束方程

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}, \quad \nabla \cdot \mathbf{B} = 0.$$

源项来自分布函数的矩：

$$\rho(\mathbf{x}, t) = \sum_s q_s \int f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p},$$

$$\mathbf{J}(\mathbf{x}, t) = \sum_s q_s \int \mathbf{v}(\mathbf{p}) f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}.$$

PIC 的核心任务就是把这套连续耦合系统离散成两个相互交换信息的对象：宏粒子和网格场。

2.2 宏粒子表示与形函数

宏粒子近似把 f_s 写成有限个带权粒子的和：

$$f_s(\mathbf{x}, \mathbf{p}, t) \approx \sum_{p \in s} w_p S_x(\mathbf{x} - \mathbf{x}_p(t)) S_p(\mathbf{p} - \mathbf{p}_p(t)).$$

这里 w_p 是宏粒子权重， S_x 是空间形函数。把粒子源项沉积到网格单元或网格点 i ，常见电荷密度形式是

$$\rho_i^n = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(\mathbf{x}_p^n).$$

场 gather 则使用同一类形函数从网格插值回粒子位置：

$$\mathbf{E}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{E}_i^n, \quad \mathbf{B}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{B}_i^n.$$

如果只看这两个公式，很容易以为 PIC 的网格-粒子耦合只是“双向插值”。这个理解不够。电流 $\mathbf{J}$ 的沉积必须表达粒子在一个时间步内的轨迹，否则离散电荷守恒会被破坏。

连续系统满足连续性方程：

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{J} = 0.$$

在网格上，电流沉积应尽量满足对应的离散形式：

$$\frac{\rho_i^{n+1} - \rho_i^n}{\Delta t} + (\nabla_h \cdot \mathbf{J}^{n+1/2})_i = 0.$$

这解释了为什么 WarpX 这样的代码会提供 Esirkepov、Villasenor-Buneman、Vay 等沉积分支。它们的差异不是表面上的“把电流放到哪里”，而是如何在离散网格上把粒子轨迹、电荷守恒、网格布局和并行边界结合起来。

2.3 leapfrog 时间层

显式电磁 PIC 的标准时间层是 leapfrog：

- 粒子位置在整数步： $\mathbf{x}^n, \mathbf{x}^{n+1}$ 。
- 粒子动量在半整数步： $\mathbf{p}^{n-1/2}, \mathbf{p}^{n+1/2}$ 。
- 电流自然在半整数步： $\mathbf{J}^{n+1/2}$ 。
- 电磁场在 Yee/FDTD 路径中按半步磁场和整步电场交错推进。

粒子位置推进可写成

$$\frac{\mathbf{x}^{n+1} - \mathbf{x}^n}{\Delta t} = \mathbf{v}^{n+1/2}.$$

动量推进写成抽象形式：

$$\frac{\mathbf{p}^{n+1/2} - \mathbf{p}^{n-1/2}}{\Delta t} = q (\mathbf{E}_p^n + \bar{\mathbf{v}} \times \mathbf{B}_p^n).$$

其中 $\bar{\mathbf{v}}$ 的具体定义取决于 pusher。Boris、Vay、Higuera-Cary 等 pusher 的差别留到粒子推进章节逐行讲解。本章只强调主循环必须给 pusher 提供正确时间层的 $\mathbf{x}$ 、 $\mathbf{p}$ 、 $\mathbf{E}$ 、 $\mathbf{B}$ 。

WarpX 的显式无 subcycling 路径在 Source/Evolve/WarpXEvolve.cpp 的 WarpX::OneStep_nosub() 开头直接把这个时间层写进注释：

```

Push particle from x^{n} to x^{n+1}
      from p^{n-1/2} to p^{n+1/2}
Deposit current j^{n+1/2}
Deposit charge density rho^{n}

```

这四行是读 WarpX 主循环的锚点。任何 field gather、collision、ionization、deposition、sync、field solve 的位置都应围绕这些时间层理解。

2.3.1 ω_p 不是背景常数，而是时间离散必须尊重的最快等离子体尺度

对电子等离子体，最基本的本征时间尺度是 plasma frequency:

$$\omega_p = \sqrt{\frac{n_e e^2}{m_e \epsilon_0}}.$$

它决定了最简单的 Langmuir 振荡周期

$$T_p = \frac{2\pi}{\omega_p}.$$

对显式 leapfrog PIC，稳定 和 分辨 不是同一件事。只要时间层排列正确、场更新满足 CFL，代码也许不会立刻炸掉；但如果

$$\omega_p \Delta t$$

已经接近或超过 1 的量级，那么单步内粒子和场已经跨过了 plasma oscillation 的核心相位结构，后面看到的高噪声、相位误差、非物理 heating，往往不是“分析脚本太苛刻”，而是主循环本身没有分辨这个最快内禀尺度。

这也是为什么 Birdsall 1985 后面会把

$$\omega_p \Delta t, \quad v_t \Delta t / \Delta x$$

都写成数值健康度的第一层控制量。对 WarpX 来说，这条边界不会自动由 ComputeDt() 替你保证。ComputeDt() 只根据 solver/CFL、const_dt、max_dt、maxParticleVelocity() 和 AMR refinement 给出一个可运行步长；它并不知道你要不要精确分辨 Langmuir 振荡、electrostatic shielding 或弱不稳定增长率。

所以本章这里先压实一个最重要的判断：

- ComputeDt() 保证的是一层离散稳定性和时间步组织约束；
- $\omega_p \Delta t$ 是否足够小，仍然是物理建模和分辨率设计问题。

2.3.2 λ_D 不只是一条长度定义，它直接约束 Δx

和 ω_p 对偶的空间尺度是 Debye length。对非相对论热电子，

$$\lambda_D = \sqrt{\frac{\epsilon_0 k_B T_e}{n_e e^2}} = \frac{v_{th,e}}{\omega_p}.$$

这条式子把热速度、plasma frequency 和 shielding length 绑在了一起。对 PIC 而言，能否把 plasma 当作 collective medium 与 网格是否真的分辨了 shielding 不是分开的两个问题。

如果

$$\Delta x \gg \lambda_D,$$

那么 cell 内已经把最基本的 shielding 结构粗化掉了。接下来即使宏观波形看起来还能跑，field fluctuation、aliasing、self-force 和 nonphysical collisionality 也会被系统性放大。这就是为什么第 1 章已经把 λ_D 、 N_D 和统计时间尺度单独拎出来；在第 2 章里，它进一步变成主循环的硬分辨率边界：

- Δt 决定是否分辨 ω_p ;
- Δx 决定是否分辨 λ_D ;
- 两者一起决定 leapfrog + grid PIC 到底是在近似同一个 plasma, 还是已经换成了另一个更噪、更热、更强 alias 的离散模型。

2.4 FDTD 场更新的数学骨架

忽略 PML、divergence cleaning、宏观介质和边界时, Yee/FDTD 的主更新可以写成三段:

$$\begin{aligned}\mathbf{B}^{n+1/2} &= \mathbf{B}^n - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^n, \\ \mathbf{E}^{n+1} &= \mathbf{E}^n + c^2 \Delta t \nabla_h \times \mathbf{B}^{n+1/2} - \frac{\Delta t}{\epsilon_0} \mathbf{J}^{n+1/2}, \\ \mathbf{B}^{n+1} &= \mathbf{B}^{n+1/2} - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^{n+1}.\end{aligned}$$

这说明一个电磁 PIC step 的顺序不能随意交换。电场更新需要本步沉积出来的 $\mathbf{J}^{n+1/2}$, 所以粒子推进与电流沉积必须在 `EvolveE(dt)` 之前完成。WarpX 的 FDTD 路径正是这样组织的:

对应的核心源码节选来自 `Source/Evolve/WarpXEvolve.cpp` 的 `WarpX::OneStep_nosub()`, 这里保留源项同步和 FDTD 场推进部分; PSATD 分支和 PML 后处理在第 3 章继续展开:

```
// Synchronize J and rho:
// filter (if used), exchange guard cells, interpolate across MR levels
// and apply boundary conditions
SyncCurrentAndRho();

// For extended PML: copy J from regular grid to PML, and damp J in PML
if (do_pml && pml_has_particles) { CopyJPML(); }
if (do_pml && do_pml_j_damping) { DampJPML(); }

ExecutePythonCallback("beforeEsolve");

EvolveF(0.5_rt * dt[0], /*rho_comp=*/0);
EvolveG(0.5_rt * dt[0]);
FillBoundaryF(guard_cells.ng_FieldSolverF);
FillBoundaryG(guard_cells.ng_FieldSolverG);

EvolveB(0.5_rt * dt[0], SubcyclingHalf::FirstHalf, a_cur_time); // We now have B~{n+1/2}
FillBoundaryB(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

if (m_em_solver_medium == MediumForEM::Vacuum) {
    // vacuum medium
    EvolveE(dt[0], a_cur_time); // We now have E~{n+1}
} else if (m_em_solver_medium == MediumForEM::Macroscopic) {
    // macroscopic medium
    MacroscopicEvolveE(dt[0], a_cur_time); // We now have E~{n+1}
} else {
    WARPX_ABORT_WITH_MESSAGE("Medium for EM is unknown");
}
FillBoundaryE(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

EvolveF(0.5_rt * dt[0], /*rho_comp=*/1);
EvolveG(0.5_rt * dt[0]);
EvolveB(0.5_rt * dt[0], SubcyclingHalf::SecondHalf, a_cur_time + 0.5_rt * dt[0]); // We now have B~{n+1}
```

数学动作	WarpX 源码位置
粒子从 $\mathbf{x}^n, \mathbf{p}^{n-1/2}$ 推到 $\mathbf{x}^{n+1}, \mathbf{p}^{n+1/2}$, 并沉积源项	Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_nosub() 对 PushParticlesandDeposit() 的调用
同步 $\rho, \mathbf{J}$: 滤波、guard cells、AMR、边界 F, G 半步、 $\mathbf{B}$ 半步	同一函数中的 SyncCurrentAndRho() 同一函数 FDTD 分支中的 EvolveF()、EvolveG()、 EvolveB(...FirstHalf...)
$\mathbf{E}$ 整步 F, G 半步、 $\mathbf{B}$ 半步	同一分支中的 EvolveE() 或 MacroscopicEvolveE() 同一分支中的第二个 EvolveF()、EvolveG()、 EvolveB(...SecondHalf...)
PML 与 guard cell 处理	同一分支中的 DampPML() 与 FillBoundary*()

这里的 F, G 是 divergence cleaning 相关辅助场, 不是最小 Maxwell 更新必需项。WarpX 把它们插在场推进两侧, 是为了在实际模拟中控制 $\nabla \cdot \mathbf{E}$ 或 $\nabla \cdot \mathbf{B}$ 误差及 PML 相关处理。

2.4.1 CFL 不是经验参数, 而是离散 Maxwell 更新的因果上界

WarpX 的 ComputeDt() 不会把 FDTD 时间步写成 $\min(\Delta x_i)/c$, 而是把这件事委托给具体差分算法。Source/Evolve/WarpXComputeDt.cpp 的 WarpX::ComputeDt() 直接在主类层完成 solver 分派:

```

} else if (electromagnetic_solver_id == ElectromagneticSolverAlgo::PSATD) {
    deltat = cfl * minDim(dx) / PhysConst::c;
} else {
    #if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER)
        if (electromagnetic_solver_id == ElectromagneticSolverAlgo::Yee) {
            deltat = cfl * CylindricalYeeAlgorithm::ComputeMaxDt(dx, n_rz_azimuthal_modes);
        }
    #elif defined(WARPX_DIM_RSPHERE)
        if (electromagnetic_solver_id == ElectromagneticSolverAlgo::Yee) {
            deltat = cfl * SphericalYeeAlgorithm::ComputeMaxDt(dx);
        }
    #else
        if (grid_type == GridType::Collocated) {
            deltat = cfl * CartesianNodalAlgorithm::ComputeMaxDt(dx);
        } else if (electromagnetic_solver_id == ElectromagneticSolverAlgo::Yee
            || electromagnetic_solver_id == ElectromagneticSolverAlgo::ECT) {
            deltat = cfl * CartesianYeeAlgorithm::ComputeMaxDt(dx);
        } else if (electromagnetic_solver_id == ElectromagneticSolverAlgo::CKC) {
            deltat = cfl * CartesianCKCAlgorithm::ComputeMaxDt(dx);
        }
    #endif
}

```

对标准 Cartesian Yee, 真正的 CFL 上界由 Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms/CartesianYeeAlgorithm::ComputeMaxDt() 给出:

```

static amrex::Real ComputeMaxDt ( amrex::Real const * const dx ) {
    using namespace amrex::literals;
    amrex::Real const delta_t = 1._rt / ( std::sqrt( AMREX_D_TERM(
        1._rt / (dx[0]*dx[0]),
        + 1._rt / (dx[1]*dx[1]),
        + 1._rt / (dx[2]*dx[2])
    ) ) * PhysConst::c );
    return delta_t;
}

```

也就是

$$\Delta t_{\max}^{\text{Yee}} = \frac{1}{c \sqrt{\Delta x^{-2} + \Delta y^{-2} + \Delta z^{-2}}}.$$

它表达的是离散光锥约束：单步内，Yee curl stencil 不能让信息传播得比离散网格所允许的因果速度更快。warpx.cfl 只是把这个严格上界再乘一个安全系数，不是“拍脑袋调参”。

2.4.2 数值色散从这里开始：连续光锥被离散 stencil 改写

一旦用有限差分近似 curl，连续真空色散关系

$$\omega^2 = c^2 |\mathbf{k}|^2$$

就不再原样保留。以 Yee 为例，差分导数对应的是

$$k_d \rightarrow \frac{2}{\Delta d} \sin \frac{k_d \Delta d}{2},$$

于是离散色散关系变成近似的

$$\sin^2 \frac{\omega \Delta t}{2} = c^2 \Delta t^2 \sum_d \frac{\sin^2(k_d \Delta d / 2)}{\Delta d^2}.$$

这意味着 phase velocity 和 group velocity 都会偏离连续值，且偏离大小依赖传播方向、波数和网格各向异性。数值色散不是后处理图里才会出现的现象，它在 UpwardDx() / DownwardDx() 这种最基本的差分定义处就已经写进去了。

Yee 的 x 向导数由同一文件中的 CartesianYeeAlgorithm::UpwardDx() 与 DownwardDx() 实现：

```
static amrex::Real UpwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    amrex::Real const inv_dx = coefs_x[0];
    return inv_dx * ( F(i+1,j,k,ncomp) - F(i,j,k,ncomp) );
}

template< typename T_Field>
static amrex::Real DownwardDx (
    T_Field const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    amrex::Real const inv_dx = coefs_x[0];
    return inv_dx * ( F(i,j,k,ncomp) - F(i-1,j,k,ncomp) );
}
```

也就是 staggered grid 上的前/后向一阶差分：

$$D_x^+ F_i = \frac{F_{i+1} - F_i}{\Delta x}, \quad D_x^- F_i = \frac{F_i - F_{i-1}}{\Delta x}.$$

这里的 Upward/Downward 不只是“正向/反向”，而是在 nodal 与 cell-centered 位置之间搬运离散导数。正是这种 staggered 几何，让 Yee 在保持二阶精度的同时把 E/B 交错布置起来。

2.4.3 Yee / Nodal / CKC 的差别本质上是离散色散关系不同

对 collocated/nodal solver, WarpX 的 CartesianNodalAlgorithm 不再用 staggered 前后差分，而是直接用中心差分。可从 Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms/CartesianNodalAlgorithm.H 的 UpwardDx() 与 DownwardDx() 回查：

```

static amrex::Real UpwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    Real const inv_dx = coefs_x[0];
    return 0.5_rt*inv_dx*( F(i+1,j,k,ncomp) - F(i-1,j,k,ncomp) );
}

static amrex::Real DownwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const n_coefs_x,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    return UpwardDx( F, coefs_x, n_coefs_x, i, j, k ,ncomp);
}

```

所以 nodal grid 上 UpwardDx 和 DownwardDx 等价, 说明这里已经没有 Yee 那种 staggered 位置语义, 而是一个 collocated 中心差分系统。它的 CFL 上界虽然在当前实现里和 Yee 一样都是

$$\Delta t_{\max} \sim \frac{1}{c\sqrt{\sum_d \Delta d^{-2}}},$$

但数值色散与奇偶模耦合特征已经不同。

CKC 则更进一步, 不再满足“一个方向只看一对最近邻”的局部导数定义。可从 Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceA 的 ComputeMaxDt() 与 UpwardDx() 回查:

```

static amrex::Real ComputeMaxDt ( amrex::Real const * const dx ) {
    #if (defined WARPX_DIM_1D_Z)
        amrex::Real const delta_t = dx[0]/PhysConst::c;
    #elif (defined WARPX_DIM_XZ)
        amrex::Real const delta_t = std::min( dx[0], dx[1] )/PhysConst::c;
    #else
        amrex::Real const delta_t = std::min( dx[0], std::min( dx[1], dx[2] ) ) / PhysConst::c;
    #endif
    return delta_t;
}

...
return alphax * (F(i+1,j ,k ,ncomp) - F(i, j, k ,ncomp))
    + betaxy * (F(i+1,j+1,k ,ncomp) - F(i ,j+1,k ,ncomp)
        + F(i+1,j-1,k ,ncomp) - F(i ,j-1,k ,ncomp))
    + betaxz * (F(i+1,j ,k+1,ncomp) - F(i ,j ,k+1,ncomp)
        + F(i+1,j ,k-1,ncomp) - F(i ,j ,k-1,ncomp))

```

这说明 CKC 的真实目标不是“换一套写法”, 而是通过更宽的横向耦合 stencil 改写离散色散关系, 从而改善高方向性传播下的 phase error。代价则是:

- stencil 更宽;
- 算法/geometry 适用范围更窄;
- guard-cell 和边界处理的组合空间更复杂。

2.5 PSATD 与 FDTD 的主循环差异

FDTD 在实空间用局部 stencil 近似 curl。PSATD 则在谱空间解析积分线性 Maxwell 方程的一部分。物理方程相同, 但离散算法不同:

- FDTD 的优势是局部、显式、边界和 AMR 工程路径相对直接。
- PSATD 能显著降低数值色散, 适合相对论束流、激光等问题, 但对 FFT、并行 domain decomposition、边界、current correction 和时间平均场有更复杂要求。

这条分界线和前面几节正好连起来:

- leapfrog 规定了粒子、场和源项的时间层关系；
- ω_p 与 λ_D 规定了 plasma 自身是否被给定的 $\Delta t/\Delta x$ 分辨；
- CFL 规定了 Maxwell 更新是否还能保持离散因果；
- Yee/Nodal/CKC/PSATD 则进一步决定同一组 $\Delta t, \Delta x$ 会把波动相速度、群速度和 aliasing 改写成什么样。

所以“主循环能跑”不等于“主循环近似的是对的物理系统”。真正的 PIC loop 要同时满足：

time-layer consistency + charge/source consistency + physical scale resolution + solver-dependent stability/dispersion control.

WarpX 在 `OneStep_nosub()` 内部把两者清楚分开：

- PSATD 分支在 `Source/Evolve/WarpXEvolve.cpp` 的 `WarpX::OneStep_nosub()` 内,核心是 `PushPSATD(a_cur_time)`。
- FDTD 分支在同一函数内, 核心是 `EvolveB/EvolveE/EvolveB`。

这意味着本书后续讲 field solver 时不能把“Maxwell solver”写成单一算法。`algo.maxwell_solver` 的选择会改变主循环内的场推进、同步、边界和可用功能。

2.6 一个完整 PIC step 的离散组织

把物理动作映射到程序实现, 一个时间步至少包含这些层次：

1. 用户 callback、信号、诊断、负载均衡和步长更新。
2. 场 gather 前的 guard cell 与 auxiliary field 准备。
3. 电离、QED、粒子注入等改变粒子集合的多物理模块。
4. 粒子推进、碰撞、沉积电流和电荷。
5. 源项同步：滤波、guard cells、AMR fine/coarse 交换、边界条件。
6. 场推进：FDTD、PSATD、implicit、electrostatic 或 hybrid。
7. PML、moving window、粒子边界、重分布、排序。
8. 诊断写出和终止条件检查。

WarpX 在 `Source/Evolve/WarpXEvolve.cpp` 的 `WarpX::Evolve()` 中正是围绕这些层次组织外层循环。真正的主循环不是教科书五行伪代码, 而是把守恒离散化、时间层一致性和大规模并行工程组合起来的控制流。

2.6.1 AMR subcycling: 两个时间步不是同一个时间步的重复调用

无 subcycling 时, 第 0 层和更细层使用同一个外层时间步, `OneStep_nosub()` 可以把粒子推进、source synchronization 和场推进看成一条统一的 $n - > n + 1$ 链。打开 subcycling 后, 这个图像不再成立。本书采用的源码快照中, `OneStep_sub1()` 明确限定：只支持两级 mesh refinement, 且每个方向的 refinement ratio 必须为 2。

令粗层时间步为 Δt_c , 细层时间步为

$$\Delta t_f = \frac{\Delta t_c}{2}.$$

一个粗层周期内的基本推进结构是：

阶段	细层	粗层/母网格	源项职责
第一个半周期	推进一次粒子和场, 步长 Δt_f	暂不完成整步	将 <code>current_fp</code> 、 <code>rho_fp</code> 限制到 coarse patch
中间同步	细层已到 $t + \Delta t_f$	粗层推进到相应中间时间	合并细层、粗层和 buffer 中的源项
第二个半周期	再推进一次粒子和场, 步长 Δt_f	继续完成粗层剩余半步	第二段细层源项限制/合并后参与粗层场更新
粗层周期末	到 $t + \Delta t_c$	到 $t + \Delta t_c$	粗细层的场、源项和 guard cells 重新同步

因此, subcycling 不是简单地把 `OneStep_nosub()` 调两次。粗层粒子只推进一次, 而细层粒子推进两次；粗层场的更新还要消费两个细层时间片上累积并平均/合成后的电流。源码中的调用顺序可以压缩成：

fine push/deposit at $t \rightarrow$ restrict source $\rightarrow$ fine field half/full/half,
 coarse push/deposit at $t \rightarrow$ combine coarse+fine source $\rightarrow$ coarse field advance,
 fine push/deposit at $t + \Delta t_f \rightarrow$ restrict source $\rightarrow$ fine field half/full/half,
 combine second fine source $\rightarrow$ coarse field completion.

这里的 **restrict** 和 **add** 不能与普通 guard-cell exchange 混为一谈：前者改变的是 coarse/fine source 的层级表示，后者只是同一层相邻 patch 间的数据可见性。对电荷来说，**rho_buf** 还可能来自 transition-zone 粒子在 coarse 几何上的直接沉积；因此 subcycling 中的 source 合成既不是“把 fine rho 全部平均下来”这么简单，也不是由场求解器自动补齐。

本书采用的源码快照显式禁止 electrostatic solver 与 subcycling 组合。这个限制写在 **OneStep_sub1()** 的入口断言中，原因不是 electrostatic 不能使用 AMR，而是这条 subcycling 例程的时间组织只为显式 electromagnetic field advance 编写，不能把 Poisson/electrostatic 路径的源项和场解时序悄悄套进来。

所以读 AMR PIC loop 时要同时检查四个不变量：

1. 细层是否确实使用 $\Delta t_c/2$ ，且在一个粗层周期内推进两次；
2. 粗层粒子是否只推进一次，避免重复计数粒子输运；
3. 两个细层时间片的 current/rho 是否分别完成 restrict、边界合并和 coarse source add；
4. 粗细层字段与 auxiliary/guard data 是否在下次 gather 前重新可见。

这四项共同构成 AMR subcycling 的时间一致性条件。缺少其中任何一项，都可能得到“每个 patch 都成功更新”但跨层电荷守恒、场相位或粒子 gather 已经不一致的结果。

2.6.2 JRhom 与 implicit: 同一个外层 step 内部也可能有不同的时间合同

标准 **OneStep_nosub()**、PSATD-JRhom 和 implicit solver 都可能被外层 **WarpX::OneStep()** 视为一次迭代，但它们内部对“源项在什么时候被求值”的定义不同。本书采用的源码快照中的分派关系是：

路径	外层入口	源项/粒子时间组织	场推进特点	组合边界
标准显式 electromagnetic PIC	OneStep_nosub()	一次粒子推进，得到 J 与 rho，随后统一 SyncCurrentAndRho()	FDTD 的 B-E-B 或一次 PSATD 推进	可与普通显式 collision placement 组合
PSATD-JRhom	OneStep_JRhom()	先推进粒子但跳过普通沉积，再按 rho/J 时间依赖在 Δt 内做多次相对时间沉积	每个 deposit interval 都执行一次谱空间场推进；可选跨 $2\Delta t$ 时间平均	只支持 PSATD； current_correction 不支持；split momentum collision push 不支持
implicit electromagnetic PIC	ImplicitSolver::OneStep()	θ 或中间场为猜测，在非线性/线性 RHS 评估中反复推进粒子并构造 $J^{n+1/2}$	通过 nonlinear solver 求自治中间电场，再完成粒子和场的后半步	不能把一次 RHS 评估误当成一次物理时间步；mass-matrix/JFNK 还会改变 J 的构造路径

JRhom 的关键不是“PSATD 多调用几次”，而是把时间依赖的源项显式建落成一组谱空间可消费的历史量。**OneStep_JRhom()** 的顺序可以压缩为：

1. 以 **skip_deposition=true** 把粒子从 $x^n, p^{n-1/2}$ 推到 $x^{n+1}, p^{n+1/2}$ ；
2. 把 E/B 以及可选的 divergence-cleaning 场变换到谱空间；
3. 按 **time_dependency_rho** 和 **time_dependency_J** 在相对时间上沉积 rho_new 与 J，每次执行 filter、guard/AMR 同步和 Fourier transform；
4. 对每个子区间执行 **PSATDPushSpectralFields()**，最后把场变回实空间并处理 PML、边界和 guard cells。

其中 **m_JRhom_subintervals** 把一个外层步拆成 **sub_dt = Δt / n_deposit** 的多个沉积区间；打开 time averaging 时，内部循环还会覆盖两个完整时间步的源/场积分。因而 JRhom 中的 rho 和 J 不是标准显式路径里“本步各沉积一次”的同义词，不能直接用 **OneStep_nosub()** 的单点时间图解释它。

implicit 路径的差异更加根本。以 **SemiImplicitEM::OneStep()** 为例，程序先保存粒子和旧场，将磁场推进到半步，然后由 nonlinear solver 反复调用 **ComputeRHS()**；**ImplicitSolver::PreRHSOp()** 在每次 RHS 构造里：

- 用当前猜测的中间电场推进粒子；

- 形成半步电流；
- 在需要时把 `current_fp_non_suborbit`、mass matrices 或 JFNK 线性阶段的贡献合并进 J；
- 最后调用 `SyncCurrentAndRho()`，再把同步后的源项交给隐式残差/线性系统。

所以 implicit 中必须区分三个层次：

1. 物理时间步 $t^n \rightarrow t^{n+1}$ ；
2. 非线性迭代中的中间场猜测 $E^{n+\theta}$ ；
3. 每次 RHS 或 Jacobian 评估中的粒子/source 重算。

如果把第 3 层误写成“程序又推进了一个物理时间步”，就会错误理解粒子数、能量账本和 `SyncCurrentAndRho()` 的调用次数。相反，如果把 JRhom 的多个 deposit interval 当成 nonlinear iteration，也会把时间积分和求解器迭代混为一谈。

本书后续章节的阅读规则因此固定为：先识别外层物理时间步，再识别内部的 source subinterval 或 nonlinear iteration，最后才判断某次 `PushParticlesandDeposit()` 是物理推进、试探性 RHS 构造，还是历史源项重建。

读者可以用下表快速定位一个输入卡采用的时间组织：

优先判断	进入路径	这一外层步内部发生什么
m_implicit_solver 已创建	<code>ImplicitSolver::OneStep()</code>	多次 nonlinear iteration/RHS source rebuild, 最后才提交一个物理时间步
psatd.JRhom 开启	<code>OneStep_JRhom()</code>	多个相对时间的 rho/J 沉积区间和对应的 PSATD 谱推进
AMR subcycling 开启	<code>OneStep_sub1()</code>	细层做两次 dt/2 推进并限制/合并源项，粗层做一次 dt 推进
以上均否	<code>OneStep_nosub()</code>	push/deposit、 <code>SyncCurrentAndRho()</code> ，再进行 FDTD 或 PSATD 场推进

图中三种“内部多次执行”含义不同：implicit 的重复是求解器试探，JRhom 的重复是同一物理时间步内的源项时间积分，subcycling 的重复则是真实的细层物理推进。只有最后完成的外层调用才代表一次可提交的物理时间步。

2.7 本章后的源码阅读入口

读者现在可以从三个源码入口继续：

目标	入口
看外层时间步如何组织	<code>Source/Evolve/WarpXEvolve.cpp: WarpX::Evolve()</code>
看显式电磁无 subcycling 的标准 step	<code>Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_nosub()</code>
看主循环如何进入粒子容器	<code>Source/Evolve/WarpXEvolve.cpp: WarpX::PushParticlesandDeposit()</code>
看两级 AMR subcycling 的细/粗层时间组织	<code>Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_sub1()</code>
看 JRhom 的多次相对时间沉积与谱推进	<code>Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_JRhom()</code>
看 implicit RHS 中的粒子推进与 source synchronization	<code>Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp: ImplicitSolver::PreRHSOp()</code>

2.8 参数示例与最小运行案例

如果把本章压回一个最小、可运行、可验证的输入骨架，可从下面的官方案例开始：

- `Examples/Tests/langmuir/inputs_test_1d_langmuir_multi`

它把本章真正讨论的五类量都放在同一个最小问题上：

- `geometry.dims = 1`

- `algo.current_deposition = esirkepov`
- `algo.field_gathering = energy-conserving`
- `warpx.cfl = 0.8`
- `max_step = 80`
- 周期场边界

也就是说，本章的抽象讨论并不是悬空的。这里的：

- leapfrog 时间层
- ω_p
- λ_D
- FDTD curl 更新
- ρ/J 连续性关系

都能在这条最小 Langmuir 主线上落到真实输入。

Examples/Tests/langmuir/CMakeLists.txt 将这张输入卡注册为 `test_1d_langmuir_multi`：它使用两个 MPI rank，并把末态 `plotfile diags/diag1000080` 交给 `analysis_1d.py`。这给出了一条读者可回查的闭环，而不是一串脱离上下文的历史数字：

1. 输入卡设置 `max_step = 80`、周期边界、`esirkepov` 沉积和第 40/80 步的 full diagnostics；
2. `analysis_1d.py` 从传入的末态 `plotfile` 读取 `Ez`，按脚本中的解析 Langmuir 解重建同一时刻的场，并要求 `error_rel < 0.05`；
3. 脚本随后调用 `analysis_utils.py` 的 `check_charge_conservation(data)`。对这张 `esirkepov`、非 PSATD、非 RZ 的输入，`helper` 实际检查

$$\frac{\max |\operatorname{div} E - \rho/\epsilon_0|}{\max |\rho/\epsilon_0|} < 10^{-11}.$$

这里的 `divE` 是诊断输出中的离散散度字段；这项 `gate` 检查的是该离散表示下的 Gauss-law/source 一致性，不是“任意几何、边界、粒子形函数和求解器组合都守恒”的证明。单次运行得到的具体误差会随构建、并行布局和案例修改而变化，因此读者应以分析脚本的定义和阈值为准，而不是把某次残差数值当成通用常数。

因此，完成这个案例的最低交付应包括：输入参数、所消费的末态诊断、解析场误差和离散 Gauss-law 误差分别说明什么，以及它们不能外推到哪些算法组合。这样才真正把连续模型、离散方程和可运行的验证量连起来。

2.9 基础文献与证据范围

本章直接依托的基础来源是：

- Birdsall 1985
 - leapfrog 最小教学骨架
 - $\omega_p \Delta t$
 - $v_t \Delta t / \Delta x$
 - finite-grid / aliasing / heating 主线
- Dawson 1983
 - electrostatic / full EM 的数值模型边界
 - full EM 时间步与 light mode / CFL 的关系
 - Darwin 作为 radiation-free low-frequency route

下列来源只作为继续阅读线索，不在本章中承担未核实的公式或数值结论：

- Yee 1966
- Hockney-Eastwood

继续扩展基础文献时，优先选择能够给出完整推导、适用假设和数值实验条件的原始教材或论文。书目线索本身只能帮助定位来源，不能代替公式、算法或案例结论的核查。

2.10 进一步阅读与练习

进一步阅读：

1. 第 3 章：WarpX 演化：把本章的 PIC loop 抽象结构接到 `main.cpp` -> `WarpX::Evolve()` 的真实调用链。

2. Birdsall 1985: 继续核对 $\omega_p \Delta t$ 、 $\lambda_D / \Delta x$ 、finite-grid aliasing 和 numerical heating 的条件与量纲。
3. Dawson 1983: 继续比较 full EM、Darwin、quiet start 和 statistical measurements 如何改变“PIC 总循环”的解释方式。

练习题:

1. 解释为什么 ComputeDt() 给出的可运行时间步, 不自动保证 $\omega_p \Delta t \ll 1$ 。
2. 用本章的 λ_D 讨论说明: 为什么 $\Delta x \gg \lambda_D$ 时, 即使主循环稳定, 也可能已经不是同一个物理 plasma。
3. 结合本章的 Langmuir 案例, 指出 analysis_1d.py 的两条核心断言分别对应本章哪两类理论边界。
4. **案例闭环题**: 依次读取 inputs_test_1d_langmuir_multi、langmuir/CMakeLists.txt 和 analysis_1d.py, 交付一张四行表 (输入设置、诊断 surface、分析断言、不可外推范围)。表中必须写清 test_1d_langmuir_multi 的两个 rank、analysis_1d.py diags/diag1000080 的绑定, 以及为什么不能只从 max_step = 80 猜测分析器实际消费的输出路径和检查量。

2.11 本章结论

PIC 总循环的关键不在于依次调用“推粒子、沉积、推场”三个动作, 而在于每个动作所使用的时间层和空间表示相容。读者应按以下顺序判断一条 PIC 路线:

1. **先确定连续问题与可分辨尺度**。等离子体频率、Debye 长度、光波 CFL 和预期物理时间窗决定 dt、网格和粒子采样是否有机会描述目标问题。
2. **再确定粒子与网格的交换方式**。gather、形函数和 rho/J deposition 必须共同满足连续性和离散布局要求; 仅有稳定的场更新不能补偿不相容的源项。
3. **区分外层时间步和内部重复**。implicit nonlinear iteration、JRhom 的 source 时间积分与 AMR subcycling 都可能在一个外层步内多次执行, 但它们分别代表试探、积分和真实细层推进, 不能混用同一类验证量解释。
4. **最后用与问题匹配的 observable 验证**。Langmuir 的解析场与 $\text{div} E - \rho / \epsilon_0$ 能检验指定输入下的波动和 Gauss-law 链; 程序退出或 checksum 本身不能证明所有几何、边界和算法组合正确。

这条顺序把连续 Vlasov–Maxwell 模型连接到真实的离散时间推进, 也为第 3 章的生命周期调用图、第 4、5 章的粒子/沉积链和第 6 章的场求解器选择提供共同的时间层坐标。

3. WarpX 主演化路径：生命周期、初始化与 Evolve()

本章开始进入 WarpX 源码。目标不是概括“WarpX 有一个 Evolve 函数”，而是建立一个可复查的调用图：程序从 main.cpp 进入，如何构造 WarpX 对象，如何读取参数和初始化数据，如何计算步长，最后如何在 WarpXEvolve.cpp 中把一个个 PIC step 推进下去。

本章的任务是让读者能从一个输入出发，追踪它何时变成网格、场、粒子和诊断，再进入一个物理时间步。使用任何 WarpX 源树时，都应优先按 main.cpp、InitData()、ComputeDt()、Evolve() 和 OneStep() 的职责与调用关系检索，而不要把固定行号或某个分支名称当作算法语义。

main.cpp 负责生命周期，WarpX 类建立模拟状态，WarpXEvolve.cpp 组织时间推进，WarpXInitData.cpp 则准备首个时间步之前的状态。OneStep_sub1()、PSATD-JRhom 和 implicit solver 的入口会在本章中定位；场算法的离散公式、粒子的 nonlinear solve 参数和 mass-matrix kernel 分别在后续相关章节展开，避免在调用图中打断物理主线。

源码定位约定

WarpX 的实现会继续演进，固定行号只能说明某一次快照的位置，不能说明算法语义。本章因此把 文件路径和函数/类符号 作为可迁移的定位锚点：例如，读者在 Source/Evolve/WarpXEvolve.cpp 中搜索 WarpX::OneStep_nosub，而不是依赖某个行号。引用小段源码时，重点是观察其输入、输出和先后关系；阅读自己的 WarpX 版本时，应先重新搜索同名符号，再比较实现是否改变。

3.1 顶层入口：main.cpp

WarpX 的可执行入口在 Source/main.cpp。主函数的控制流非常短：

```
initialize_external_libraries(argc, argv)
warpx = WarpX::GetInstance()
warpx.InitData()
warpx.Evolve()
is_warpx_verbose = warpx.Verbose()
WarpX::Finalize()
print timer if verbose
finalize_external_libraries()
```

核心调用的职责如下：

源码锚点	代码含义
Source/main.cpp: initialize_external_libraries	初始化 AMReX/MPI/GPU runtime 等 WarpX 依赖的底层运行环境。
Source/main.cpp: WarpX::GetInstance()	取得全局 WarpX 单例。
Source/main.cpp: warpx.InitData()	把参数、网格、场、粒子、诊断和边界准备好。
Source/main.cpp: warpx.Evolve()	进入时间推进主循环。
Source/main.cpp: WarpX::Finalize()	释放 WarpX 单例。
Source/main.cpp: finalize_external_libraries	结束外部库。

主函数中的核心段落如下：

```
warpx::initialization::initialize_external_libraries(argc, argv);

auto timer = ablastr::utils::timer::Timer{};
timer.record_start_time();

auto& warpx = WarpX::GetInstance();
warpx.InitData();
warpx.Evolve();
const auto is_warpx_verbose = warpx.Verbose();
WarpX::Finalize();

timer.record_stop_time();
if (is_warpx_verbose) {
    amrex::Print() << "Total Time" : "
```

```

        << timer.get_global_duration() << '\n';
    }

```

```
warpX::initialization::finalize_external_libraries();
```

逐块看，这段代码只有一个物理模拟对象 `warpX`。`InitData()` 之前还没有进入时间推进；`Evolve()` 返回之后模拟已经结束，后续只做计时、profiling 和库资源释放。这里没有任何粒子或场循环，因为 `WarpX` 把模拟状态封装在 `WarpX` 单例内部。

这个入口说明：`WarpX` 的主程序不直接管理粒子数组或场数组；它只管理生命周期。真正的模拟状态集中在 `WarpX` 对象及其持有的 `MultiParticleContainer`、field register、diagnostics、solver、PML 等成员中。

3.2 WarpX 类与单例构造

`WarpX` 类定义在 `Source/WarpX.H`。关键声明包括：

源码锚点	声明	作用
<code>Source/WarpX.H</code>	<code>class WarpX : public amrex::AmrCore</code>	<code>WarpX</code> 以 <code>AMReX</code> 的 <code>AMR</code> 基类为基础。
<code>WarpX::GetInstance()</code>	<code>static WarpX& GetInstance();</code>	全局单例入口。
<code>WarpX::Finalize()</code>	<code>static void Finalize();</code>	删除单例。
<code>WarpX::InitData()</code>	<code>void InitData();</code>	初始化模拟数据。
<code>WarpX::Evolve()</code>	<code>void Evolve(int numsteps=-1);</code>	外层时间推进。
<code>WarpX::OneStep*</code>	<code>OneStep</code> 、 <code>OneStep_nosub</code> 、 <code>OneStep_sub1</code> 、 <code>OneStep_JRhom</code>	主循环内部的单步推进路径。

单例实现位于 `Source/WarpX.cpp`：

- `WarpX::GetInstance()` 检查 `m_instance`，为空则调用 `MakeWarpX()`。
- `WarpX::Finalize()` 调 `ResetInstance()` 删除对象。
- `WarpX::WarpX()` 设置 `m_instance=this`，初始化 warning manager，调用 `ReadParameters()`，做向后兼容处理，初始化 EB，建立 `istep/nsubsteps/t_new/t_old/dt` 数组，并创建 `MultiParticleContainer`。

构造函数中最重要的调用是 `ReadParameters()`。这意味着 solver 类型、边界、步长策略、滤波、静电/电磁模式等会在 `InitData()` 前决定。

3.3 ReadParameters(): 主循环分支的来源

`WarpX::ReadParameters()` 定义在 `Source/WarpX.cpp`。完整参数系统很大，本章只列出直接影响主循环的部分。

参数或逻辑	读取位置	对主循环的影响
<code>max_step</code> 、 <code>stop_time</code>	<code>WarpX::ReadParameters()</code>	<code>Evolve()</code> 用它们限制循环终点。
<code>algo.maxwell_solver</code>	<code>WarpX::ReadParameters()</code>	选择 PSATD、Yee、CKC、ECT、HybridPIC、None 等路径。
PSATD 不支持 PEC/PMC 的断言	<code>WarpX::ReadParameters()</code>	solver 选择会反过来限制边界条件。
<code>algo.evolve_scheme</code>	<code>WarpX::ReadParameters()</code>	决定 explicit、theta implicit 等演化框架。
<code>warpX.cfl</code> 、 <code>verbose</code> 、 <code>regrid_int</code> 、 <code>do_subcycling</code>	<code>WarpX::ReadParameters()</code>	控制步长、输出、重网格和 AMR 子循环。
<code>warpX.do_electrostatic</code>	<code>WarpX::ReadParameters()</code>	静电 solver 非空时把 electromagnetic solver 设为 None。
<code>const_dt</code> 、 <code>max_dt</code> 、 <code>dt_update_interval</code>	<code>WarpX::ReadParameters()</code>	控制固定步长和运行中步长更新。
<code>filter</code> 默认开关	<code>WarpX::ReadParameters()</code>	显式 scheme 默认滤波，隐式 scheme 默认关闭滤波。

这里有一个阅读原则：输入文件里的参数名只是表层。要理解参数的真实含义，必须追到 `ReadParameters()` 中它如何被读入、被断言约束、被改写，并进一步影响 `Evolve()`、`ComputeDt()` 或 solver 对象。

3.3.1 构造函数只建“跨 level 外壳”，不直接建完整网格数据

仅从 `ReadParameters()` 还看不出一个常见实现边界: `WarpX::WarpX()` 构造函数里虽然已经决定了 solver 路线, 但此时还没有有效的 `BoxArray` 和 `DistributionMapping`. 构造函数中的注释明确说明:

```
// Geometry on all levels has been defined already.  
// No valid BoxArray and DistributionMapping have been defined.  
// But the arrays for them have been resized.
```

所以构造函数能做的是:

- 创建不依赖具体网格盒划分的跨 level 对象:
 - `MultiParticleContainer`
 - `m_electrostatic_solver`
 - `m_hybrid_pic_model`
 - solver 指针数组外壳
- 按 `maxLevel()+1` 先 `resize` 各种 level 容器:
 - `istep`
 - `nsubsteps`
 - `t_old/t_new`
 - `dt`

但它不能在这里直接分配真正的 `Efield_fp/Bfield_fp/current_fp/rho_fp`. 这些字段必须等到 `MakeNewLevelFromScratch()` `-> AllocLevelData()` `-> AllocLevelMFs()`, 拿到真实的 `ba/dm`、`index type` 和 `guard cell` 之后才创建。

这也是为什么:

- `effective potential` 在构造期只创建 `EffectivePotentialES` 对象;
- `hybrid PIC` 在构造期只创建 `HybridPICModel` 对象;
- `implicit solver` 即使已经存在, 也还没分配 `mass matrices`.

真正的 level 级附加字段要到后面才各自落地。

3.3.2 `AllocLevelData()` / `AllocLevelMFs()` 里, 四条 solver 分支的落点不同

`AllocLevelData()` 定义在 `Source/WarpX.cpp`. 它先调用 `guard_cells.Init(...)`, 再进入 `AllocLevelMFs(...)`. 这一层真正决定的是: 哪些 `MultiFab` 需要随着 level 一起出生。

先看隐式分支. `AllocLevelMFs()` 并不会一次性把全部隐式字段都分好, 它只先放两类最基础的 level 数据:

```
if (m_implicit_solver) {  
    m_fields.alloc_init(FieldType::current_fp_non_suborbit, Direction{0}, lev,  
                        amrex::convert(ba, jx_nodal_flag), dm, ncomps, ngJ, 0.0_rt);  
    ...  
    m_fields.alloc_init(FieldType::E_old, Direction{2}, lev,  
                        amrex::convert(ba, Ez_nodal_flag), dm, ncomps, ngEB, 0.0_rt);  
}
```

也就是说, 隐式路线在 level 分配期先保存:

- 非 suborbit 粒子的独立电流容器 `current_fp_non_suborbit`
- 旧电场时间层 `E_old`

而更重的 `MassMatrices_X/Y/Z/MassMatrices_PC` 不是在这里分配, 而是在后续 `ImplicitSolver::InitializeMassMatrices()` 里, 等标准 `current_fp/Efield_fp` 的 `index type`、`guard cells` 和 `deposition` 算法都稳定后再决定组件数。

再看 hybrid PIC. 它在构造期只有一个模型对象, 真正的 level 字段由 `HybridPICModel::AllocateLevelMFs(...)` 填入共享的 `m_fields` register:

```
if (WarpX::electromagnetic_solver_id == ElectromagneticSolverAlgo::HybridPIC)  
{  
    m_hybrid_pic_model->AllocateLevelMFs(  
        m_fields,  
        lev, ba, dm, ncomps, ngJ, ngRho, ngEB, jx_nodal_flag, jy_nodal_flag,
```

```

        jz_nodal_flag, rho_nodal_flag, Ex_nodal_flag, Ey_nodal_flag, Ez_nodal_flag,
        Bx_nodal_flag, By_nodal_flag, Bz_nodal_flag
    );
}

```

这一调用会分配：

- hybrid_electron_pressure_fp
- hybrid_rho_fp_temp
- hybrid_current_fp_temp
- hybrid_current_fp_plasma
- 可选的 hybrid_current_fp_external

若启用 add_external_fields，还会进一步分配 external vector potential 相关字段。也就是说，hybrid 不是在 WarpX 根层自己持有一套独立网格，而是把专用状态嵌进统一的 field register。

再看 effective potential electrostatic solver。这条线在构造期创建了 EffectivePotentialES 对象，但 level 分配期没有额外的 effective_potential_* 专用字段。它复用的是静电共享合同：

```

if( (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrame) ||
    (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameElectroMagnetostatic) ||
    (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameEffectivePotential) ||
    (electromagnetic_solver_id == ElectromagneticSolverAlgo::HybridPIC) ) {
    rho_ncomps = ncomps;
}

if (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrame ||
    electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameElectroMagnetostatic ||
    electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameEffectivePotential )
{
    m_fields.alloc_init(FieldType::phi_fp, lev, amrex::convert(ba, phi_nodal_flag), dm,
                        ncomps, ngPhi, 0.0_rt );
}

```

所以 effective potential 在根层最关键的差异不是“多了什么字段”，而是后续 solver 如何解释和消费 rho_fp/phi_fp。

最后是 PML。它同样不是在 AllocLevelMFs() 里出生。真正创建 PML 对象是在 InitFromScratch() 的最后一步：

```

AmrCore::InitFromScratch(time); // This will call MakeNewLevelFromScratch
...
mypc->AllocData();
mypc->InitData();

InitPML();

```

这条顺序说明：

- 先有普通 level 的 fields / particles
- 后有作为边界子系统的 PML

因此 PML 是初始化末段附加上的 patch 级边界对象，而不是和 Efield_fp/Bfield_fp 同时由 AllocLevelMFs() 常规分配的主网格字段。

把对象落点按路径分开读，会比把构造、分配和初始化末段塞进同一张宽表更清楚：

- **标准场**：构造期只准备容器外壳；AllocLevelMFs() 分配 Efield_fp、Bfield_fp、current_fp、rho_fp 和 phi_fp；InitLevelData() 再填入物理初值。
- **effective potential**：构造期创建 EffectivePotentialES；level 分配期不另建专用字段，而是复用 rho_fp/phi_fp；静电求解器随后按 effective-potential 语义消费这些字段。
- **hybrid PIC**：构造期创建 HybridPICModel；level 分配期把 hybrid_* 字段放入共享 m_fields；HybridPICModel::InitData() 再编译 parser 并准备外加电流和矢势。
- **implicit**：solver 对象在构造期已存在；level 分配期先分配 current_fp_non_suborbit 与 E_old；随后 Define()、CreateParticleAttributes() 和 InitializeMassMatrices() 逐步补齐隐式响应数据。

- **PML**: 构造期只知道参数和开关, `AllocLevelMFs()` 也不创建 `PML patch`; `InitPML()` 在获得真实 `boxArray`、`DistributionMapping`、`dt` 和 `m_fields` 后再创建边界对象。

3.4 InitData(): 把状态准备到可推进

`WarpX::InitData()` 定义在 `Source/Initialization/WarpXInitData.cpp`。它不是简单分配内存, 而是把一个模拟从“参数已读”变成“可以推进第一步”。

核心顺序如下:

源码动作	解释
进入 <code>InitData()</code> , 检查 MPI thread level	并行运行前的运行环境检查。
创建 <code>MultiDiagnostics</code> 和 <code>MultiReducedDiags</code>	诊断系统在初始化早期建立。
非 restart: <code>ComputeDt()</code> 、打印步长网格、 <code>InitFromScratch()</code> 、 <code>InitDiagnostics()</code>	从头运行时先确定步长, 再建立网格/粒子/诊断。
restart: <code>InitFromCheckpoint()</code> 、打印步长网格、 <code>PostRestart()</code>	checkpoint 恢复不走完全相同的初始化路径。
<code>ComputeMaxStep()</code> 、PML factors、NCI corrector、buffer masks	准备停止步数、吸收边界和数值不稳定修正。
宏观介质、静电 solver、HybridPIC 初始化	solver 相关对象拿到场布局 and 参数。
网格摘要、guard cell 检查、打印 PIC 参数、写 used inputs	把运行状态和输入记录下来。
初始 div cleaning、自洽静电/磁静场、外场叠加	从头运行时在第一个 step 前建立初始场。
初始 full/reduced diagnostics	允许输出第 0 步或 restart 后诊断。
性能提示和 solver issue 检查	给出已知风险提示。

`InitFromScratch()` 调用 `AmrCore::InitFromScratch(time)` 建立 AMR level, 然后让 `mypc->AllocData()` 和 `mypc->InitData()` 初始化粒子, 最后初始化 PML。

3.5 ComputeDt(): 步长不是一个固定常数

`WarpX::ComputeDt()` 定义在 `Source/Evolve/WarpXComputeDt.cpp`。

逻辑可以分成四类:

1. HybridPIC 必须显式给出 `warpX.const_dt`。
2. 纯静电或无 Maxwell solver 时, 必须给出 `const_dt` 或激活 `dt_update_interval`。
3. 若用户给了 `const_dt`, 直接使用。
4. 否则按 solver 计算 CFL 限制: 静电/PSATD 用最小 cell size 与 c , FDTD 调用具体几何和算法的 `ComputeMaxDt()`。

最终 `dt` 被 `resize` 到 `max_level+1`。若启用 `subcycling`, 粗层步长由细层步长乘 `refinement ratio` 得到。

把它写成决策顺序会比宽表更容易在阅读和排版中保持清楚:

1. 用户固定步长优先。设置 `warpX.const_dt` 后, `ComputeDt()` 直接采用它, 所有 CFL 估计都不再决定步长; 稳定性责任也随之转给用户。
2. 特殊模型先检查强制条件。HybridPIC 必须给出 `const_dt`。静电路径若未使用固定步长, 则以 `max_dt` 为初值; 若也未给出, 则退回到 $\text{CFL } \Delta x_{\min}/c$ 。
3. PSATD 使用最小网格尺度的光速尺度。在没有固定步长时, 初值为 $\text{CFL } \Delta x_{\min}/c$ 。这只是本函数的初始选择, 不能替代该配置的物理分辨率判断。
4. FDTD 按几何和网格布置选择稳定上限。Cartesian collocated grid 调用 `CartesianNodalAlgorithm::ComputeMaxDt`; Yee/ECT 调用 `CartesianYeeAlgorithm::ComputeMaxDt`; CKC 调用 `CartesianCKCAlgorithm::ComputeMaxDt`。RZ/RCYLINDER 的 Yee 路径还使用 azimuthal mode 数, SPHERE 则走 `SphericalYeeAlgorithm::ComputeMaxDt`。这些函数名比展开成长公式更适合作为源码阅读入口。

因此, `ComputeDt()` 不只是“取最小网格长度除以光速”, 而是在参数层先决定有没有用户强制时间步, 再按 solver 家族和几何去选真正的稳定上限公式。

运行中自适应步长由 `WarpX::ApplyDtLimiters()` 处理。它首先经 `ParticleGridSpeedMax()` 从 `mypc->maxParticleVelocity()` 计算最大粒子跨网格速度, 再与可选的等离子体频率、回旋频率和 `max_dt` 限制一起取最严格的步长。仅考虑粒子跨网格限制时, 公式是

$$\Delta t_{\text{new}} = \text{CFL} \frac{\Delta x_{\min}}{v_{\max}}$$

ApplyDtLimiters() 更新 finest level 的 dt, 再向粗层回推。因此它并不等同于一个只看粒子速度的旧接口: 开启 max_omegap_dt、max_omegac_dt 或 max_dt 时, 任何一个限制都可以成为最终瓶颈。

这里还要补一条参数层边界: warpx.const_dt 与 warpx.dt_update_interval 在 ReadParameters() 中就是互斥的。也就是说, 运行时 adaptive timestep 不是“在固定步长上再做微调”, 而是一条和 const_dt 完全不同的时间组织路线。Evolve() 里只有当 m_dt_update_interval.contains(step+1) 为真, 或首步需要应用 max_dt 时, 才会在步首调用 ApplyDtLimiters()。

3.6 Evolve() 外层时间步

WarpX::Evolve() 定义在 Source/Evolve/WarpXEvolve.cpp。它不是单纯调用 OneStep(), 而是在每个 step 前后管理大量状态。

外层结构是:

```
cur_time = t_new[0]
numsteps_max = max_step or istep[0] + numsteps
for step in range while cur_time < stop_time:
    check signals
    multi_diags->NewIteration()
    run beforestep callback
    CheckLoadBalance(step)
    maybe update dt from particle speeds
    ExplicitFillBoundaryEBUpdateAux()
    hybrid initialization if first step
    field ionization / QED / particle injection
    OneStep(cur_time, dt[0], step)
    resampling / mirrors
    increment istep and time
    diagnostics prepack, moving window, particle boundary handling
    electrostatic or hybrid field solve if selected
    optional velocity synchronization for diagnostics
    afterstep callback, diagnostics, warnings, signals, stop check
```

关键动作:

源码动作	读法
初始化 cur_time 和循环边界	numsteps=-1 时使用全局 max_step。
信号检查、诊断新迭代	支持运行中断、checkpoint 和诊断状态更新。
callback、负载均衡、可选步长更新	更新步长前需要同步粒子速度时间层。
ExplicitFillBoundaryEBUpdateAux()	显式 scheme 为后续 field gather 准备场。
field ionization、QED、particle injection	多物理事件在 OneStep() 前改变粒子集合。
OneStep(cur_time, dt[0], step)	进入单步推进分派。
更新 istep 和 t_old/t_new	单步推进后更新时间状态。
诊断预处理、moving window、粒子边界	OneStep() 后的工程处理同样影响物理结果。
静电或 HybridPIC 的场解	非标准电磁路径的场更新位置不同。
诊断需要时同步粒子速度	为输出把 p 与 x 放到同步时间层。
reduced/full diagnostics 和 callback	诊断写出发生在本步状态更新之后。
未使用输入检查、计时、信号、停止	第一步后检查输入 typo, 最后判断是否退出。

注意 Evolve() 中多物理和诊断并不都在 OneStep() 内部。比如 field ionization、QED 和 particle injection 在 OneStep() 之前, resampling、moving window、粒子边界和某些 electrostatic/hybrid 场解在 OneStep() 之后。

3.6.1 步末 moving window: 连续坐标与整数网格平移

要理解 moving window, 先把它放回 Evolve() 主循环中: MoveWindow(step+1, move_j) 发生在 OneStep() 完成、cur_time 和 t_new 更新之后, 粒子边界处理之前。对应逻辑位于 WarpX::Evolve():

```
cur_time += dt[0];
```

```

ShiftGalileanBoundary();

// sync up time
for (int i = 0; i <= max_level; ++i) {
    t_old[i] = t_new[i];
    t_new[i] = cur_time;
}
multi_diags->FilterComputePackFlush( step, false, true );

const bool move_j = m_is_synchronized;
// If m_is_synchronized we need to shift j too so that next step we can evolve E by dt/2.
// We might need to move j because we are going to make a plotfile.
const int num_moved = MoveWindow(step+1, move_j);

MoveWindow() 的第一层逻辑是维护一个连续窗口位置 moving_window_x, 但只有当它相对当前几何左边界跨过整数个 cell 时才真正平移网格数据。实现位于 Source/Utils/WarpXMovingWindow.cpp:

if (!moving_window_active(step)) { return 0; }

// Update the continuous position of the moving window,
// and of the plasma injection
moving_window_x += (moving_window_v - WarpX::beta_boost * PhysConst::c)/(1 - moving_window_v * WarpX::beta_boost);
const int dir = moving_window_dir;

// Update current injection position for all containers
::UpdateInjectionPosition(*mypc, gamma_boost, beta_boost, boost_direction, moving_window_dir, dt[0]);

// Update antenna position for all lasers
// TODO Make this specific to lasers only
mypc->UpdateAntennaPosition(dt[0]);

// compute the number of cells to shift on the base level
amrex::Real new_lo[AMREX_SPACEDIM];
amrex::Real new_hi[AMREX_SPACEDIM];
const amrex::Real* current_lo = geom[0].ProbLo();
const amrex::Real* current_hi = geom[0].ProbHi();
const amrex::Real* cdx = geom[0].CellSize();
const int num_shift_base = static_cast<int>((moving_window_x - current_lo[dir]) / cdx[dir]);

if (num_shift_base == 0) { return 0; }

```

这段代码中的速度变换是相对论速度合成公式：

$$v'_w = \frac{v_w - \beta_b c}{1 - v_w \beta_b / c}.$$

因此 boosted-frame 模拟中窗口速度不是简单使用输入的 moving_window_v。输入参数在 read_moving_window_parameters() 中先由以 c 为单位的无量纲数转成 SI 速度；运行时再按 boost 速度变换到模拟坐标系。

active 判定本身也不是模糊的“某个阶段窗口有效”，而是源码里一个明确的闭开区间：

$$\text{start_moving_window_step} \leq n < \text{end_moving_window_step},$$

若 end_moving_window_step < 0 则表示没有终止步。这也是为什么 Evolve() 调的是 MoveWindow(step+1, ...)：窗口平移是这一步结束后、下一步开始前的状态更新。

当 num_shift_base != 0 时, MoveWindow() 调用 ResetProbDomain() 更新几何域, 并用 shiftMF() 平移 E/B/current/PML/F/G/rho/flux 等 MultiFab。shiftMF() 的核心赋值为：

```

amrex::Box dstBox = mf[mfi].box();
if (num_shift > 0) {
    dstBox.growHi(dir, -num_shift);
} else {
    dstBox.growLo(dir, num_shift);
}
AMREX_PARALLEL_FOR_4D ( dstBox, nc, i, j, k, n,
{
    dstfab(i,j,k,n) = srcfab(i+shift.x,j+shift.y,k+shift.z,n);
})

```

即

$$F_{\text{new}}(\mathbf{i}) = F_{\text{old}}(\mathbf{i} + N_{\text{shift}} \hat{e}_{\text{dir}}).$$

新露出的边界层不是随便置零。对外部场，shiftMF() 可以用常量外场或 parser 外场重新初始化；对背景粒子，MoveWindow() 构造整数 cell 宽度的 particleBox 并调用 pc.ContinuousInjection(particleBox)。这解释了 moving window 的数值设计：连续窗口位置负责物理速度，整数 cell 平移负责保持网格离散结构和宏粒子 spacing。

这里还要和步末的 ContinuousFluxInjection(cur_time, dt[0]) 分开。二者不是同一件事：

- moving-window continuous injection: 在新露出的体网格区域里补背景体分布；
- continuous flux injection: 从定义好的注入面持续打入粒子通量。

前者是“窗口移动后补齐新计算域”，后者是“边界源项继续往域内送粒子”。

3.7 OneStep(): 求解器分派器

WarpX::OneStep() 定义在 Source/Evolve/WarpXEvolve.cpp。它按 solver 和 AMR 情况分派：

```

if m_implicit_solver:
    m_implicit_solver->OneStep(...)
else:
    if electromagnetic_solver_id is None or HybridPIC:
        push particles, optionally split collisions, skip deposition
    else:
        if finest_level == 0:
            OneStep_nosub or OneStep_JRhom
        else:
            OneStep_nosub or OneStep_sub1

```

这段代码体现了 WarpX 的设计：OneStep() 不直接写某一种 PIC 算法的全部细节，而是先把路径分开。

分支	源码锚点	含义
implicit solver	m_implicit_solver->OneStep(...)	交给隐式 solver 自己推进一整步。
electrostatic / HybridPIC	electromagnetic_solver_id == None/HybridPIC	粒子推进但跳过标准电磁沉积路径，场解在外层后处理。
标准电磁无 MR	finest_level == 0	进入 OneStep_nosub() 或 PSATD-JRhom。
有 MR 无 subcycling	!m_do_subcycling	仍进入 OneStep_nosub()，所有 level 使用同一步长推进。
有 MR 且 subcycling	m_do_subcycling	进入 OneStep_sub1()；当前实现限制最多两个 level。

几个断言值得后续单独讲：

- JRhom 与 split momentum collision 不能组合。
- subcycling 要求 finest_level == 1。

- subcycling 与 split momentum collision 也不能组合。

这些不是文档层面的“建议”，而是源码级功能边界。

还应补一条输入层边界：psatd.JRhom 不是布尔开关，而是一个编码了源项时间模型的字符串。ReadParameters() 里它会同时决定：

- J 的时间依赖是 constant / linear / quadratic;
- rho 的时间依赖是 constant / linear / quadratic;
- 一个 PIC step 里切成多少个 JRhom subinterval。

因此 JRhom 开启后，后续变的不是“另一个小优化开关”，而是 OneStep() 内部的时间组织、rho_fp 的组件语义和谱空间源项更新公式。

3.8 OneStep_nosub(): 显式电磁标准路径

WarpX::OneStep_nosub() 定义在 Source/Evolve/WarpXEvolve.cpp。这是本书第一个需要逐行读懂的核心函数。

它的结构分为四段。

第一段：粒子推进、碰撞与沉积。

源码原文：

```
// Push particle from  $x^{\{n\}}$  to  $x^{\{n+1\}}$ 
//                               from  $p^{\{n-1/2\}}$  to  $p^{\{n+1/2\}}$ 
// Deposit current  $j^{\{n+1/2\}}$ 
// Deposit charge density  $\rho^{\{n\}}$ 

ExecutePythonCallback("beforedeposition");

// with collisions placed in the middle of the momentum push
if (m_collisions_split_momentum_push) {
    // push particles (half momentum)
    PushParticlesandDeposit(
        a_cur_time,
        /*skip_deposition=*/true,
        PositionPushType::None,
        MomentumPushType::FirstHalf
    );
    // perform particle collisions
    ExecutePythonCallback("beforecollisions");
    mypc->doCollisions(a_step, a_cur_time, a_dt);
    ExecutePythonCallback("aftercollisions");

    // push particles (full position and half momentum)
    PushParticlesandDeposit(
        a_cur_time,
        /*skip_deposition=*/false,
        PositionPushType::Full,
        MomentumPushType::SecondHalf
    );
}
else {
    ExecutePythonCallback("beforecollisions");
    mypc->doCollisions(a_step, a_cur_time, a_dt);
    ExecutePythonCallback("aftercollisions");

    PushParticlesandDeposit(
        a_cur_time,
        /*skip_deposition=*/false,
        PositionPushType::Full,
        MomentumPushType::Full
    );
}
```

```
);
}
```

```
ExecutePythonCallback("afterdeposition");
```

- 注释明确时间层: $\mathbf{x}^n \rightarrow \mathbf{x}^{n+1}$, $\mathbf{p}^{n-1/2} \rightarrow \mathbf{p}^{n+1/2}$, 沉积 $\mathbf{J}^{n+1/2}$ 和 ρ^n 。
- 如果 `m_collisions_split_momentum_push` 为真, 先做半个动量 push, 再碰撞, 再做位置 push 和后半动量 push。
- 否则先做碰撞, 再调用 `PushParticlesandDeposit()` 完整推进粒子并沉积。

第二段: 源项同步。

源码原文:

```
// Synchronize J and rho:
// filter (if used), exchange guard cells, interpolate across MR levels
// and apply boundary conditions
SyncCurrentAndRho();
```

```
// At this point, J is up-to-date inside the domain, and E and B are
// up-to-date including enough guard cells for first step of the field
// solve.
```

```
// For extended PML: copy J from regular grid to PML, and damp J in PML
if (do_pml && pml_has_particles) { CopyJPML(); }
if (do_pml && do_pml_j_damping) { DampJPML(); }
```

- `SyncCurrentAndRho()` 会处理滤波、guard cells、AMR 跨层插值/加和和边界。
- PML 若含粒子或需要电流阻尼, 会复制和阻尼 PML 中的电流。

第三段: PSATD 或 FDTD 场推进。

FDTD 分支的核心源码如下:

```
} else {
    EvolveF(0.5_rt * dt[0], /*rho_comp=*/0);
    EvolveG(0.5_rt * dt[0]);
    FillBoundaryF(guard_cells.ng_FieldSolverF);
    FillBoundaryG(guard_cells.ng_FieldSolverG);

    EvolveB(0.5_rt * dt[0], SubcyclingHalf::FirstHalf, a_cur_time); // We now have B^{n+1/2}
    FillBoundaryB(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

    if (m_em_solver_medium == MediumForEM::Vacuum) {
        EvolveE(dt[0], a_cur_time); // We now have E^{n+1}
    } else if (m_em_solver_medium == MediumForEM::Macroscopic) {
        MacroscopicEvolveE(dt[0], a_cur_time); // We now have E^{n+1}
    } else {
        WARPX_ABORT_WITH_MESSAGE("Medium for EM is unknown");
    }
    FillBoundaryE(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

    EvolveF(0.5_rt * dt[0], /*rho_comp=*/1);
    EvolveG(0.5_rt * dt[0]);
    EvolveB(0.5_rt * dt[0], SubcyclingHalf::SecondHalf, a_cur_time + 0.5_rt * dt[0]); // We now have B^{n+1}
}
```

- PSATD 走 `PushPSATD(a_cur_time)`, 并处理 hybrid QED、PML、平均场、 F/G guard cells。
- FDTD 走 `EvolveF/G` 半步、`EvolveB(dt/2)`、`EvolveE(dt)`、`EvolveF/G` 半步、`EvolveB(dt/2)`。

第四段: 回调。

- `afterEsolve` callback 在场更新后执行。

从物理角度看, `OneStep_nosub()` 做的事情是: 用旧场 `gather` 推粒子, 得到新位置和半步电流; 把源项修整到网格和边界一致; 再用这些源项推进电磁场。

3.9 SyncCurrentAndRho(): 源项不是沉积完就可用

`SyncCurrentAndRho()` 定义在 `Source/Evolve/WarpXEvolve.cpp`。

它的分支很重要:

- PSATD 且 periodic single box 时, 会立即同步 J 和 ρ 。
- PSATD 非 periodic single box 时, 若没有 current correction 且不是 Vay deposition, 才在这里同步。
- Vay deposition 在特定情况下先只做 filter。
- FDTD 路径总是 `SyncCurrent("current_fp")` 和 `SyncRho()`。
- 最后对 ρ 和 J 施加 PEC 等边界处理。

这说明“沉积”与“可用于场解”之间有一段不可忽略的工程层: 滤波、guard cell、AMR 和边界会改变源项数组的可用状态。

3.10 PushParticlesandDeposit(): 进入粒子容器

`PushParticlesandDeposit()` 的两个重载定义在 `Source/Evolve/WarpXEvolve.cpp`。

第一层重载遍历所有 AMR level。第二层重载做三件事:

1. 根据 `do_current_centering` 和 `current_deposition_algo == Vay` 选择当前沉积字段名。
2. 调用 `mypc->Evolve(...)`, 把 field register、level、字段名、时间、`dt[lev]`、subcycling half、是否跳过沉积、位置/动量 push 类型传入粒子容器。
3. 对 RZ/柱/球几何做逆体积缩放, 并在有流体物种时调用流体容器演化。

因此, 下一阶段逐行阅读必须从 `mypc->Evolve()` 继续进入 `Source/Particles`。`PushParticlesandDeposit()` 是主循环到粒子模块的接口, 不是粒子 pusher 本身。

3.11 OneStep_sub1() 与 JRhom 的位置

理解这两个特殊分支时, 先把它们放回主循环时间层。

`OneStep_sub1()` 定义在 `Source/Evolve/WarpXEvolve.cpp`。当前 subcycling 只支持两个 level 和 refinement ratio 2: fine patch 用小步长推两次, coarse patch 和 mother grid 推一次, coarse 场使用两次 fine current 的平均效果。函数注释直接说明这一点:

```
* This version of subcycling only works for 2 levels and with a refinement
* ratio of 2.
* The particles and fields of the fine patch are pushed twice
* (with dt[coarse]/2) in this routine.
* The particles of the coarse patch and mother grid are pushed only once
* (with dt[coarse]). The fields on the coarse patch and mother grid
* are pushed in a way which is equivalent to pushing once only, with
* a current which is the average of the coarse + fine current at the 2
* steps of the fine grid.
```

第一段 fine step 的核心源码如下:

```
PushParticlesandDeposit(fine_lev, cur_time, SubcyclingHalf::FirstHalf);
RestrictCurrentFromFineToCoarsePatch(
    m_fields.get_mr_levels_alldirs(FieldType::current_fp, finest_level),
    m_fields.get_mr_levels_alldirs(FieldType::current_cp, finest_level, skip_lev0_coarse_patch), fine_lev)
RestrictRhoFromFineToCoarsePatch(fine_lev);

EvolveB(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev], SubcyclingHalf::FirstHalf, cur_time);
EvolveF(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev], /*rho_comp=*/0);
EvolveE(fine_lev, PatchType::fine, dt[fine_lev], cur_time);
EvolveB(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev], SubcyclingHalf::SecondHalf, cur_time + 0.5_rt * dt);
EvolveF(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev], /*rho_comp=*/1);
```

因此 subcycling 的物理时间层是

$$\Delta t_0 = 2\Delta t_1.$$

fine level 在一个 coarse step 内走两个完整 leapfrog 小步。RestrictCurrentFromFineToCoarsePatch() 和 RestrictRhoFromFineToCoarsePatch() 把 fine 层沉积源项平均到 coarse patch; StoreCurrent()/RestoreCurrent() 保证 coarse 粒子自身 current 能在两个 half coarse step 中分别叠加对应的 fine contribution。

这里 StoreCurrent()/RestoreCurrent() 的角色需要说得更硬一点: subcycling 不是简单把 fine current 直接覆写 coarse current, 而是要先保留 coarse 粒子本身在大步时间层上的电流, 再把两次 fine-step 的 restriction 结果分别叠回 coarse half-step。否则 coarse mother grid 看到的就不是“一个 coarse 大步上等效的平均源项”, 而会把 coarse 自身电流和 fine 补偿混在一起。

OneStep_JRhom() 定义在 Source/Evolve/WarpXEvolve.cpp。它是 PSATD-JRhom 专用路径, 会多次沉积 J 和 ρ , 在谱空间推进字段, 并支持时间平均场。入口先断言 solver 必须是 PSATD, 并且粒子 push 时跳过标准沉积:

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    WarpX::electromagnetic_solver_id == ElectromagneticSolverAlgo::PSATD,
    "JRhom algorithm not implemented with the FDTD solver"
);

// Push particle from  $x^{\{n\}}$  to  $x^{\{n+1\}}$ 
//                               from  $p^{\{n-1/2\}}$  to  $p^{\{n+1/2\}}$ 
const bool skip_deposition = true;
PushParticlesandDeposit(cur_time, skip_deposition);

// Initialize PSATD-JRhom loop:
```

```
// 1) Prepare E,B,F,G fields in spectral space
PSATDForwardTransformEB();
if (WarpX::do_dive_cleaning) { PSATDForwardTransformF(); }
if (WarpX::do_divb_cleaning) { PSATDForwardTransformG(); }
```

随后 JRhom 以

$$\delta t = \frac{\Delta t}{m}$$

为子区间, 多次在相对时间 t_deposit_current 和 t_deposit_charge 沉积源项:

```
const int n_deposit = WarpX::m_JRhom_subintervals;
const amrex::Real sub_dt = dt[0] / static_cast<amrex::Real>(n_deposit);
const int n_loop = (WarpX::fft_do_time_averaging) ? 2*n_deposit : n_deposit;

for (int i_deposit = 0; i_deposit < n_loop; i_deposit++)
{
    if (time_dependency_J != TimeDependencyJ::Constant) { PSATDMoveJNewToJOld(); }

    const amrex::Real t_deposit_current = (time_dependency_J == TimeDependencyJ::Linear) ?
        (i_deposit-n_deposit+1)*sub_dt : (i_deposit-n_deposit+0.5_rt)*sub_dt;

    const amrex::Real t_deposit_charge = (time_dependency_rho == TimeDependencyRho::Linear) ?
        (i_deposit-n_deposit+1)*sub_dt : (i_deposit-n_deposit+0.5_rt)*sub_dt;

    mypc->DepositCurrent( m_fields.get_mr_levels_alldirs(current_string, finest_level), dt[0], t_deposit_current, t_deposit_charge);
    SyncCurrent("current_fp");
    PSATDForwardTransformJ("current_fp", "current_cp");
```

所以 JRhom 的核心不是多次 gather, 也不是多次粒子 push, 而是用同一次粒子轨道在多个相对时间沉积源项, 让 PSATD 在一个 step 内看到更高阶的 $\tilde{\mathbf{J}}(t)$ 和 $\tilde{\rho}(t)$ 。

这条线路还有两条必须写清的功能限制：

1. JRhom 不支持 FDTD, 只能走 PSATD。
2. JRhom 和 `current_correction`、`Vay deposition` 都不兼容；源码层会把 `current_correction` 关掉，并显式禁止 `Vay deposition` 和 JRhom 组合。

所以这一支的真实定位是：它不是“PSATD 上再附加一个任意可叠加的小修正”，而是 PSATD 自身的一种替代性时间积分组织方式。

3.11.1 implicit 分支：一次物理步包含多次试探性 source 重建

在 `WarpX::OneStep()` 中，只要 `m_implicit_solver` 非空，程序就不会进入 `OneStep_nosub()`、`OneStep_sub1()` 或 `OneStep_JRhom()`，而是把整步交给 `m_implicit_solver->OneStep(...)`。

以 `SemiImplicitEM::OneStep()` 为代表，隐式电磁步的控制流是：

顺序	源码动作	时间层/物理含义
1	<code>SaveParticlesAtImplicitStepStart</code>	保存 x^n, p^n ，供非线性迭代和最终提交使用
2	初始化 $E^{n+\theta}$ 猜测、保存 <code>E_old</code>	构造 solver 的中间场未知量，而不是直接写最终 E^{n+1}
3	<code>EvolveB($\Delta t/2$)</code>	先把 WarpX 所有的磁场推进到半步
4	<code>m_nlsolver->Solve(...)</code>	反复调用 <code>ComputeRHS()</code> ，求粒子和中间电场自洽的离散方程
5	<code>SetElectricFieldAndApplyBCs()</code> 、 <code>FinishImplicitParticleUpdate()</code>	将收敛的中间场写回，并把粒子从半步状态完成到 t^{n+1}
6	第二个 <code>EvolveB($\Delta t/2$)</code>	完成磁场后半步，物理时间步才真正结束

因此 `m_nlsolver->Solve()` 不是一个普通的函数调用包装，而是这条路径的核心时间组织。`SemiImplicitEM::ComputerRHS()` 会先用当前猜测的 $E^{n+1/2}$ 更新 WarpX 持有的电场，然后调用 `PreRHSOp()`；后者定义在 `Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp`，完成：

1. 以当前中间场推进粒子位置和速度；
2. 形成 $J^{n+1/2}$ ；
3. 按需要合并 `current_fp_non_suborbit`、`mass-matrix` 或 JFNK 线性阶段贡献；
4. 做 RZ/柱/球几何的逆体积缩放；
5. 调用 `SyncCurrentAndRho()`，把源项整理后交给隐式 RHS 或 Jacobian。

这条链中的 `PushParticlesandDeposit()` 可能在多个 nonlinear iteration、Jacobian evaluation 和 particle Picard iteration 中重复出现，但这些重复并不代表多个物理时间步。它们是在同一个 $t^n \rightarrow t^{n+1}$ 问题上，对不同中间场猜测重新计算离散残差。

implicit 还有两个容易误读的实现边界：

- `CumulateJ()` 必须在 `SyncCurrentAndRho()` 之前，把 `mass-matrix` 路径之外的 J 贡献合入 `current_fp`；否则同步的是不完整源项。
- `m_use_mass_matrices_jacobian` 和 `m_particle_suborbits` 会让 Jacobian 阶段只推进 suborbit 粒子或直接用 `ComputeJfromMassMatrices()` 构造电流，因而不能假定每次 RHS 都走同一个粒子 kernel。

这也解释了为什么 implicit 的验证不能只复制显式 Langmuir 的“单步场误差”判据。至少要分别检查：非线性求解是否收敛、粒子最终状态是否只提交一次、RHS 期间的 source synchronization 是否完整，以及最终 E/B 时间层是否与 `FinishImplicitParticleUpdate()` 一致。

3.11.2 nonlinear solver、JFNK 与 mass-matrix: J 的三种构造层

`ImplicitSolver::parseNonlinearSolverParams()` 定义在 `Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp`。它首先读取 `nonlinear_solver`，再决定后续 J 是通过完整粒子响应、Newton/JFNK 近似，还是 PETSc SNES 的线性阶段构造：

配置	solver 对象	粒子/电流路径	关键边界
picard	PicardSolver	每次 RHS 直接用当前场推进粒子并沉积 J	当前实现把 <code>max_particle_iterations=1</code> 、 <code>particle_tolerance=0</code> 固定为最小 Picard 路径
newton	NewtonSolver	非线性外层配合 JFNK；可选 particle suborbits 与 mass-matrix Jacobian 由 PETSc 管理	<code>use_mass_matrices_jacobian</code> 和 <code>use_mass_matrices_pc</code> 只能在该类 solver 中启用
petsc_snes	PETScSNES	nonlinear/linear solve, 仍复用 <code>PreRHSOp()</code> 构造源项	必须以 <code>AMREX_USE_PETSC</code> 编译, 否则源码直接 abort

在普通 nonlinear RHS 阶段, `PreRHSOp()` 的电流来源可以概括为完整粒子响应:

$$J = J_{\text{particle}} + J_{\text{non-suborbit}}.$$

但在使用 mass matrices 的 Jacobian 阶段, 源码采用的是线性化响应:

$$J(E_0 + \delta E) = J_{\text{suborbit}} + J_0 + M \delta E,$$

其中 E_0 是 Newton step 开始时由 `SaveE()` 保存的电场, J_0 是以 E_0 推进的非 mass-matrix 粒子电流, M 是 `MassMatrices_X/Y/Z` 表示的离散响应算子。这个式子正是 `CumulateJ()` 和 `ComputeJfromMassMatrices()` 之间的职责分界:

- `CumulateJ()` 把 `current_fp_non_suborbit` 加回当前 `current_fp`, 补上不在 mass matrices 中的粒子贡献;
- `ComputeJfromMassMatrices()` 根据当前 $E-E_0$ 、 J_0 和各方向交叉响应, 把 $M \delta E$ 写入 `current_fp`;
- `SyncCurrentAndRho()` 只负责之后的滤波、边界、guard/level 通信, 不负责判断 J 应该由完整粒子还是线性响应产生。

`ComputeJfromMassMatrices()` 还必须处理 Yee/nodal staggering。源码先根据 $J_x/J_y/J_z$ 的 `ixType()` 计算 `offset_xx ... offset_zz`, 再用 $S_{xx}/S_{xy}/.../S_{zz}$ 的多分量 stencil 访问邻近电场。因此 mass matrix 不是一个可以在任意 centering 上直接相乘的标量系数; 它同时编码了方向耦合、空间 support 和网格位置偏移。把它简写成 $M = \partial J / \partial E$ 只足以说明物理意图, 不足以替代对 index type 和 component offset 的源码核对。

配置层也有明确的几何限制: 当前源码禁止 3D 使用 `use_mass_matrices_jacobian`, 禁止 RSPHERE 使用 mass matrices; `mass_matrices_pc_width` 只在非 3D 情况下读取。因而这条路径不能被描述成所有 implicit geometry 的通用加速开关。

最后, `particle_suborbits` 改变的是粒子响应如何被拆分, 而不是外层物理时间步。在线性 Jacobian 阶段, 若启用 suborbit, `PreRHSOp()` 可以只推进 suborbit 粒子并用 `ComputeJfromMassMatrices(J_from_MM_only)` 补齐响应; 若未启用, 则由 mass matrices 直接构造线性阶段的 J 。这正是 implicit 验证必须同时记录 solver 类型、particle suborbit、mass-matrix 开关和最终 source gate 的原因。

3.12 参数示例与最小运行闭环

如果把本章压成一个最小、可执行、可回查的演化入口, 可从下面的官方回归案例开始:

- `Examples/Tests/langmuir/inputs_test_1d_langmuir_multi`

输入文件直接消费了本章讲到的一组顶层参数:

- `max_step = 80`
- `algo.current_deposition = esirkepov`
- `algo.field_gathering = energy-conserving`
- `warpx.cfl = 0.8`
- 周期场边界

这里需要避免一个常见误读: 这个输入 没有 显式设置 `algo.maxwell_solver`。因此它不能用于证明某个 solver 参数由该输入给出; 实际采用的默认 solver 必须回到该版本的 `ReadParameters()` 与官方文档确认。

它成为可执行案例, 是因为 `Examples/Tests/langmuir/CMakeLists.txt` 把它注册为 `test_1d_langmuir_multi`: 一维、2 个 MPI rank, 分析命令为 `analysis_1d.py diags/diag1000080`。也就是说, `max_step = 80` 只规定步数; 最终 plotfile 名称和分析入口来自 CMake 注册, 不能单凭步数猜出。

对本章来说，这个案例最重要的意义不是“Langmuir 物理本身”，而是它把主循环连接到明确的消费端。对于这个显式电磁、无 AMR 的输入，主路径是：

```
main.cpp
-> WarpX::GetInstance()
-> InitData()
-> ComputeDt()
-> Evolve()
-> OneStep()
-> PushParticlesandDeposit()
-> SyncCurrentAndRho()
-> EvolveB/EvolveE/EvolveB
-> diagnostics
```

分析脚本 `analysis_1d.py` 读取最终 `plotfile`，按解析的 Langmuir 波重建 E_z ，并要求最大相对误差满足

$$\max |E_{z,\text{sim}} - E_{z,\text{theory}}| / \max |E_{z,\text{theory}}| < 0.05.$$

随后它还调用 `check_charge_conservation(data)`。对这个 Esirkepov、非 PSATD、非 RZ 的案例，辅助分析会检查离散 Gauss-law 残差；这补充了场形状的比较，但不等同于证明所有几何、所有 solver 或 AMR 分支的守恒性。

这样，`inputs_test_1d_langmuir_multi` 把本章的“主循环入口”压成一个可复现的闭环：

- 有参数入口：输入文件
- 有测试编排：`test_1d_langmuir_multi` 的 CMake 注册与 2-rank 规模
- 有明确消费端：`analysis_1d.py` `diags/diag1000080`
- 有两类检查： E_z 解析误差阈值和离散 Gauss-law 残差

这条路线把 `WarpX` 主类生命周期和 `Evolve()` 主链从静态调用图连接到输入、输出和可检查的物理量。它只覆盖这个 Langmuir 配置，不能据此推出所有 solver、几何或 AMR 分支都已验证。

3.13 进一步阅读与练习

进一步阅读：

1. 第 4 章：粒子推进器：继续从 `PushParticlesandDeposit()` 进入 `mypc->Evolve()` 和粒子推进器。
2. 第 5 章：沉积与形函数：继续展开 `SyncCurrentAndRho()`、沉积、guard/source synchronization。
3. 对 `lifecycle`、`subcycling`/`JRhom` 与 `moving window` 三条分支，继续分别追问：状态在哪个时间层更新、哪些网格层参与、以及结果应通过哪个 observable 判断。

练习题：

1. 说明为什么 `WarpX::WarpX()` 里只能创建跨-level 外壳，而不能直接分配完整 `MultiFab` 主字段。
2. 用本章的 `StoreCurrent()/RestoreCurrent()` 解释：为什么 `subcycling` 不能简单拿 fine current 覆盖 coarse current。
3. 结合本章的 Langmuir 案例，指出 `inputs_test_1d_langmuir_multi` 中哪些参数分别进入 `ReadParameters()`、`ComputeDt()` 和 `OneStep_nosub()` 的不同层次。
4. 案例闭环题。阅读该输入、`Examples/Tests/langmuir/CMakeLists.txt` 和 `analysis_1d.py`，写出四行表格：输入参数、测试规模、分析命令、通过条件。解释为什么 `max_step = 80` 不能单独推出输出目录或通过阈值，并说明两类检查各自能与不能支持什么结论。

3.14 本章小结

本章建立的主演化路线可压缩为五个读者检查点。它们按一个输入从启动到一次时间步的顺序排列，而不是按源码文件的出现顺序排列：

1. 启动与构造：从 `main.cpp` 进入 `GetInstance()`、`WarpX::WarpX()` 和 `ReadParameters()`。先问哪些参数与 solver 分支在建立网格前已经确定。
2. 初始化：依次检查 `InitData()`、`ComputeDt()`、`InitFromScratch()` 或 `InitFromCheckpoint()`。确认初始 level、粒子、场、PML 和 diagnostics 在第一步前已就绪。
3. 外层推进：在 `Evolve()` 中核对 callback、负载均衡、注入、边界与诊断分别发生在单步前还是单步后。
4. 单步分派：在 `OneStep()` 中判断该输入实际走 `implicit`、`electrostatic`/`HybridPIC`、`OneStep_nosub()`、`subcycling` 还是 `JRhom`。
5. 显式主链：从 `PushParticlesandDeposit()` 到 `SyncCurrentAndRho()`，再到 `PushPSATD()` 或 `EvolveB/EvolveE/EvolveB`，确认粒子运输、源项同步和场推进的时间层相容。

3A. WarpX 初始化链：从 InitData() 到初始粒子和外部场

本章把 `WarpX::InitData()` 展开成一条完整的初始化链。它补足第 3 章中“初始化”只作为主循环前置步骤的不足：这里开始逐块解释 fresh run / restart、AMR level 初始化、外部场、species 注入器、粒子创建 kernel、Gaussian beam、openPMD 文件注入和 projection divergence cleaning。

阅读初始化代码时，应先围绕四个主入口建立因果关系：`InitData()` 决定 fresh/restart 分叉，`InitFromScratch()` 建立初始 level，`InitDiagnostics()` 准备可观察输出，`AddExternalFields()` 把外部场加入初态。使用不同 WarpX 版本时，优先按这些函数的职责和调用关系检索，不要把行号或分支名称当作初始化语义。

读者问题	首先追踪的对象
程序启动后哪些参数在构造对象前已被锁定？	参数读取、 <code>WarpX::WarpX()</code> 与 <code>ReadParameters()</code>
参数、level 字段和 <code>InitData()</code> 的顺序如何落到对象上？	<code>InitFromScratch()</code> 、level allocation 与 field data
外场、species、粒子创建与散度修正分别如何进入初态？	<code>AddExternalFields()</code> 、 <code>PlasmaInjector</code> 、粒子创建与 projection cleaner
哪些案例能区分不同初始化路径？	initial distribution、space charge、external field 与 restart 的观察量

3A.1 初始化链为什么值得单独成章

理解初始化时，先把以下三层边界分开：

1. `WarpX::WarpX()` 构造期只完成参数读取和跨 level 外壳创建。
2. `InitData()` 才在 fresh run / restart 之间分叉，并把 AMR level、粒子、诊断、PML、外场和初始静电/磁静场组织到第一步之前。
3. species 注入、外部场、projection cleaning、openPMD/Gaussian beam 分别有自己的参数、时间层和验证边界，不能笼统地称为“初始化完成”。

PIC 程序的时间推进方程通常写成：

$$f^n, \mathbf{E}^n, \mathbf{B}^n \longrightarrow f^{n+1}, \mathbf{E}^{n+1}, \mathbf{B}^{n+1}.$$

但第一个时间步之前必须先构造一个满足几何、边界、粒子权重、外部场、离散约束和并行布局的初态。WarpX 的初始化不是“读入参数然后开始跑”，而是完成以下任务：

1. 选择 fresh run 或 checkpoint restart。
2. 建立 AMReX level、field MultiFab、PML、EB 和 solver 数据结构。
3. 初始化 diagnostics 和 reduced diagnostics。
4. 创建 species、解析密度/动量/位置分布，并生成宏粒子。
5. 读入或解析外部场，并把外部场叠加到 self field 或 particle external field。
6. 对外部 A/B 场做 projection divergence cleaning。
7. 输出第 0 步 diagnostics，并检查 guard cell、solver 配置和 known issue。

本章先建立从程序启动到第一个时间步的主干；需要实现级细节时，再沿源码文件、函数和调用关系逐项核对。

3A.2 启动层先于 InitData(): MPI、AMReX、FFT、PETSc 与启动前提

许多初始化说明从 `WarpX::InitData()` 往后讲，但 WarpX 真正的初始化链更早就开始了。`main.cpp` 最外层先做的是：

```
int
main (int argc, char* argv[]) {
    warpx::initialization::initialize_external_libraries(argc, argv);
    {
        auto& warpx = WarpX::GetInstance();
        warpx.InitData();
        warpx.Evolve();
        WarpX::Finalize();
    }
    warpx::initialization::finalize_external_libraries();
}
```

这说明 `InitData()` 之前还有一层独立的 bootstrap 逻辑, 负责:

1. 初始化 MPI、AMReX、FFT, 以及可选 PETSc。
2. 覆盖一批 AMReX 默认 parser 行为。
3. 解析 geometry、periodicity 和整数数组表达式。
4. 锁定 `geometry.dims`、moving window 和 warning policy 这类全局运行前提。

这一层主要分布在:

- `Initialization/WarpXAMReXInit.*`
- `Initialization/WarpXInit.*`
- `WarpX::MakeWarpX()`

3A.2.1 参数系统先分命名空间, 再分读取语义

WarpX 的参数不是“一个大表一次性读完”。它先按 `ParmParse` 前缀拆成局部命名空间, 然后再决定是:

- 立即求值成数值;
- 保留 parser 字符串, 稍后编译;
- 解释成 interval 切片;
- 还是先解析成枚举型算法选择。

最上游的运行控制参数就是无前缀直接读取:

```
const ParmParse pp; // Traditionally, max_step and stop_time do not have prefix.
utils::parser::queryWithParser(pp, "max_step", max_step);
utils::parser::queryWithParser(pp, "stop_time", stop_time);
```

而大部分初始化参数则先按前缀取视图:

```
const ParmParse pp_amr("amr");
const ParmParse pp_algo("algo");
ParmParse const pp_warpx("warpx");
const ParmParse pp_geometry("geometry");
const ParmParse pp_boundary("boundary");
const ParmParse pp_psatd("psatd");
```

接下来又会分成几种读取壳:

1. `queryWithParser/queryArrWithParser`
 - 适合 `warpx.cfl`、`const_dt`、`amr.n_cell` 这类“读入就该变成数值”的参数;
2. `Store_parserString + makeParser`
 - 适合 `ref_patch_function(x,y,z)`、外场 parser、diagnostics 过滤函数这类“先保留表达式, 再在别处编译执行”的参数;
3. `IntervalsParser`
 - 适合 `sort_intervals`、`dt_update_interval`、`diagnostics intervals` 这类切片语法, 不应误看成普通整数;
4. `query_enum_sloppy + AMREX_ENUM`
 - 适合 `algo.evolve_scheme`、`warpx.do_electrostatic`、`algo.current_deposition` 这类算法枚举入口。

这层结构解释了为什么同样都出现在 `parameters.rst` 里, `max_step`、`ref_patch_function(x,y,z)` 和 `algo.evolve_scheme` 在源码中会走三种完全不同的消费链。后续参数索引如果想真正有用, 就必须至少区分:

- 参数前缀;
- 第一次被哪个 `ParmParse` 命名空间读取;
- 它属于数值参数、parser 字符串、interval 还是枚举型算法选择。

例如 `WarpXAMReXInit.cpp` 并不是简单调用 `amrex::Initialize()`, 而是把一个缺省值覆盖回调一起传进去:

```
amrex::Initialize(
    argc,
    argv,
    build_parm_parse,
    MPI_COMM_WORLD,
    ::overwrite_amrex_parser_defaults
);
```

这个回调会统一注入常量、改写 `amrex.abort_on_out_of_gpu_memory`、`amrex.the_arena_is_managed`、`amrex.omp_threads`、粒子 `tiling` 缺省值，以及由 `boundary.field_* / boundary.particle_*` 反推 `geometry.is_periodic`。换句话说，WarpX 从程序一启动就已经不是“AMReX 原生默认设置”。

同一个文件还会在 AMReX 初始化后立即把 `geometry.prob_lo/prob_hi`、`amr.n_cell`、`max_grid_size*`、`blocking_factor*` 这类可能带表达式的输入预解析并写回 `parser`，保证后面的 `Geometry`、`warp_x_job_info` 和 `yt` 读到的是数值结果，而不是未展开的表达式字符串。

3A.2.2 文档 alias、AMReX-owned 参数与 WarpX 自有 parser 不是一回事

参数索引继续往下清理后，会发现还有一类参数不能简单用“有没有 `grep` 到同名字符串”来理解。典型例子是：

- `geometry.prob_lo/hi`
- `boundary.field_lo/hi`
- `boundary.potential_lo/hi_x/y/z`
- `psatd.nox/noy/noz`
- `qed_schwinger.xmin/ymin/zmin/xmax/ymax/zmax`

这些名字在文档里是 grouped alias，但源码里真实读取的仍然是拆开的 key。以 `geometry.prob_lo/hi` 为例，WarpX 真正做的是：

```
utils::parser::getArrWithParser(  
    pp_geometry, "prob_lo", prob_lo, 0, AMREX_SPACEDIM);  
utils::parser::getArrWithParser(  
    pp_geometry, "prob_hi", prob_hi, 0, AMREX_SPACEDIM);  
  
pp_geometry.addarr("prob_lo", prob_lo);  
pp_geometry.addarr("prob_hi", prob_hi);
```

所以 `geometry.prob_lo/hi` 只是文档层“成对参数”的写法，真正 parser key 仍然是 `prob_lo` 和 `prob_hi`。`boundary.field_lo/hi`、`boundary.particle_lo/hi`、`boundary.potential_lo/hi_x/y/z`、`psatd.nox/noy/noz` 和 `Schwinger` 区域边界框也都属于这一类。

还要再区分另一类：参数确实出现在 WarpX 手册里，但 WarpX 并不直接读取，而是由 AMReX 自己消费，WarpX 只消费结果。例如 `amr.ref_ratio / amr.ref_ratio_vect` 和 `amrex.async_out / amrex.async_out_nfiles`。本书引用的 WarpX 源树没有独立的：

```
ParmParse("amr").query("ref_ratio", ...)  
ParmParse("amrex").query("async_out", ...)
```

但后续代码会直接消费已经构造好的 `ref_ratios`，或者在 `plotfile` I/O 邻近层继续设置 WarpX 自己的 `field_io_nfiles / particle_io_nfiles`。因此，参数索引如果要稳定，至少要区分三类：

1. WarpX 自身直接 parse；
2. WarpX 子对象 parse；
3. AMReX-owned 输入，WarpX 只消费 materialized 结果。

另一条关键链在 `WarpX::MakeWarpX()`：

```
warp_x::initialization::check_dims();  
warp_x::initialization::read_moving_window_parameters(...);  
ConvertLabParamsToBoost();  
parse_field_boundaries();  
parse_particle_boundaries(...);  
CheckGriddingForRZSpectral();  
m_instance = new WarpX();
```

因此单例构造前就已经锁定了四类全局事实：

- 当前可执行文件与 `geometry.dims` 是否匹配；
- moving window 是否开启、方向和速度是什么；
- field / particle boundary 类型是什么；
- RZ spectral 的 gridding 约束是否满足。

例如 `check_dims()` 直接把编译维度和 `geometry.dims` 做强一致断言, 而 `read_moving_window_parameters()` 会把输入文件里的 `moving_window_v` 从“以光速为单位的无量纲参数”转换成真正的物理速度 $v = (\cdots)c$, 再写进 `WarpX` 的全局静态状态。

这里还要补一层工程来源。`check_dims()` 能做这种强断言, 前提不是“`WarpX` 在运行时随意切换几何”, 而是构建系统已经先把几何变体锁死了。当前主构建链在顶层 `CMakeLists.txt` 中通过 `WarpX_DIMS` 生成一组按维度后缀分裂的 `lib_${SD}` / `pyWarpX_${SD}` target; 旧 `GNUmake` 链则通过 `DIM`、`USE_RZ` 以及 `-DWARPX_DIM_3D`、`-DWARPX_DIM_XZ`、`-DWARPX_DIM_RZ` 这组宏编码几何变体。因此初始化阶段读到的 `geometry.dims` 实际是在和“当前可执行文件已经按哪一种几何编译出来”做一致性检查, 而不是在决定后面要不要临时切换到另一种维度。

再往下, `WarpX` 构造函数开头还会调用:

```
warpX::initialization::initialize_warning_manager();
```

也就是在任何 `ReadParameters()` 和 `InitData()` 之前, 先读入:

- `warpX.always_warn_immediately`
- `warpX.abort_on_warning_threshold`

这说明 `warning manager` 在 `WarpX` 里也是初始化启动层的一部分, 而不是后面运行时再临时决定的日志选项。

这一层再往下细分, 还要注意 `MakeWarpX()` 和构造期 `ReadParameters()` 之间并不是简单前后顺序, 而是“前者已经开始改写后者将要读取的参数”。例如 `ConvertLabParamsToBoost()` 不只是保存 `gamma_boost`, 而是会先把:

- `geometry.prob_lo/prob_hi`
- `warpX.fine_tag_lo/fine_tag_hi`
- `slice.dom_lo/dom_hi`

按 `boosted-frame` 规则直接回写进 `parser`。若同时启用 `moving window`, 它计算的转换系数还会显式依赖 `moving_window_v/c`。因此 `boosted-frame` 与 `moving-window` 的第一次耦合, 其实发生在 `ReadParameters()` 之前, 而不是发生在后面的 `Evolve()`。

同样, `CheckGriddingForRZSpectral()` 也不是单纯做断言。对 `WARPX_DIM_RZ + algo.maxwell_solver = psatd` 这条组合, 它会在构造 `WarpX` 对象之前直接改写:

- `amr.blocking_factor_x/y`
- `amr.max_grid_size_x/y`

并要求 `longitudinal` 方向至少满足“每个 `MPI rank` 至少有一个 `block`, 且每个 `rank` 至少有 8 个 `z cells`”的约束。也就是说, 构造函数里的 `ReadParameters()` 读到的并不总是用户输入文件里的原始 `AMR` 分块值, 而往往已经是启动层重写过的 `spectral-compatible` 版本。

把这一点看清之后, 初始化启动层就能更准确地拆成两段:

1. 预初始化与 `parser` 改写段:
 - `initialize_external_libraries()`
 - `amrex_init()`
 - `parse_geometry_input()`
 - `read_moving_window_parameters()`
 - `ConvertLabParamsToBoost()`
 - `CheckGriddingForRZSpectral()`
2. 构造与运行态落地段:
 - `initialize_warning_manager()`
 - `ReadParameters()`
 - `BackwardCompatibility()`
 - `InitEB()`
 - `MultiParticleContainer` / `ParticleBoundaryBuffer` / `solver objects` 的真正创建

这个分层很有用, 因为后面再读 `boosted-frame` 例子、`RZ spectral gridding`、`moving window` 连续注入或 `warning manager`, 就不会再把“参数在哪一步被锁定”与“对象在哪一步消费这些参数”混在一起。

再往前走一步, `ReadParameters()` 还不只是“把输入存进成员”。它已经在替后面的对象图做分叉。例如:

- `do_fluid_species` 先通过 `fluids.species_names` 决定, 随后直接控制是否创建 `MultiFluidContainer`;
- `electrostatic_solver_id` 与 `electromagnetic_solver_id` 先在 `ReadParameters()` 中互相覆盖和约束, 随后决定静电 `solver` 具体实例化成 `LabFrameExplicitES`、`EffectivePotentialES` 还是 `RelativisticExplicitES`;
- `electromagnetic_solver_id == HybridPIC` 时, 构造函数后半段才真正创建 `HybridPICModel`;
- `grid_type`、`do_current_centering`、`n_rz_azimuthal_modes` 等参数先在 `ReadParameters()` 中完成默认值推导和合法组合检查, 后面再影响 `nodal flags`、`centering coefficients`、`fluid 兼容性`与容器初始化。

同样, `ReadParameters()` 里还有一类“源码先推导默认, 再由用户输入覆盖”的派生状态。最典型的是:

```
if (!do_divb_cleaning
    && m_p_ext_field_params->B_ext_grid_type != ExternalFieldType::default_zero
    && m_p_ext_field_params->B_ext_grid_type != ExternalFieldType::constant
    ...
    && (electromagnetic_solver_id == Yee
        || electromagnetic_solver_id == HybridPIC
        || ...))
{
    m_do_initial_div_cleaning = true;
}
pp_warpx.query("do_initial_div_cleaning", m_do_initial_div_cleaning);
```

这里 `do_initial_div_cleaning` 不是单纯从输入文件机械读取, 而是先根据外部场类型、grid type 和 solver 组合推导一个默认值, 再允许用户显式覆盖。也就是说, 后面 `ProjectionDivCleaner` 是否参与初态构造, 不是孤立地由一个布尔开关决定, 而是由初始化启动层和 `ReadParameters()` 主体共同塑造的。

到这一层为止, 初始化阶段已经可以更清楚地压成一条完整链:

1. 启动层先改 parser、锁定全局静态状态。
2. `ReadParameters()` 再把这些值落到 `WarpX` 成员, 并决定对象图分叉。
3. 构造函数后半段据此创建 `MultiParticleContainer`、`ParticleBoundaryBuffer`、`MultiFluidContainer` 和 solver objects。
4. 最后 `InitData()` 才真正开始分配 level data、生成粒子和构造初态。

但这里还差最后一层经常被误读的东西: `ReadParameters()` 里那些看上去分散的算法检查, 其实会继续决定后面到底分配哪类 `MultiFab`、是否创建 implicit solver 额外状态, 以及 `ProjectionDivCleaner` 是否在初态构造中插队。

最紧的一组约束是:

- `grid_type=hybrid` 会把 `do_current_centering` 推成默认真值, 并要求 `algo.field_gathering=momentum-conserving`;
- 这又会让 `AllocLevelData()` 中的 `aux_is_nodal=true`, 从而把 `Efield_aux/Bfield_aux`、后续 `coarse-aux cax` 和 `gather buffer` 路径切到 nodal 版本;
- 如果同时显式要求 `do_current_centering=1`, 源码只允许 `grid_type=hybrid`, 并且在 level 分配时额外分配 `current_fp_nodal`。

换句话说, `grid_type`、`field_gathering_algo`、`do_current_centering` 在这里并不是三个平行开关, 而是一组会继续改写场 index type 和附加存储布局的上游选择器。

同样, `current_deposition_algo` 也不是只影响某个沉积 kernel。源码先把 PSATD、HybridPIC 和 electrostatic 的默认 deposition 拉回 Direct, 再对 Vay 加上三重限制:

- 只能配 PSATD
- 不能配 mesh refinement
- 不能配 `do_current_centering`

这条限制后面会直接落成额外字段分配: 如果真的选了 Vay, `AllocLevelMFs()` 会专门分配 `current_fp_vay`。所以它已经是初始化对象图的一部分, 而不只是运行时算法分派。

implicit 系列更明显。`ReadParameters()` 一旦看到

- `semi_implicit_em`
- `theta_implicit_em`
- `strang_implicit_spectral_em`

就会立即构造 `m_implicit_solver`, 并同时要求:

- current deposition 只能是 Esirkepov / Villasenor / Direct
- EM solver 只能是 Yee / CKC / PSATD
- particle pusher 只能是 Boris / HigueraCary
- field gather 不能是 momentum-conserving

然后这条链会继续穿到 `InitFromScratch()` 和 `AllocLevelMFs()`:

- `InitFromScratch()` 在 `mypc->InitData()` 之前调用 `m_implicit_solver->Define()` 和 `CreateParticleAttributes()`;
- `AllocLevelMFs()` 在 `fine patch` 上额外分配 `current_fp_non_suborbit` 与 `E_old`。

因此 `implicit` 不是“场求解器章节内部的一个局部话题”，而是在初始化阶段就已经改变粒子属性表和字段分配合同。

`particle_shape.filter` 和 `projection div cleaning` 也属于同一种“前置组合约束”。只要存在 `particles` 或 `lasers`, `algo.particle_shape` 就变成强制参数，并直接设定 `nox=noy=noz`；这反过来继续影响 `guard-cell` 配额、沉积 `stencil` 和排序默认值。`use_filter` 也不是后面可有可无的一步卷积，因为 `AllocLevelData()` 会先 `InitFilter()`，再把 `filter stencil` 长度交给 `guard_cells.Init(...)`，于是它会继续改写 `guard-cell` 和 `buffer` 需求。

最后，`m_do_initial_div_cleaning` 也不是孤立的布尔开关。源码先根据外部 `B` 场类型、`EM solver` 组合和是否已经启用 `do_divb_cleaning` 推导默认值，再允许用户覆盖；而它的下游消费点就在初始化主链里，直接决定 `ProjectionDivCleaner` 是否参与初始场构造。所以到这一步为止，`ReadParameters()` 的真实地位可以概括成一句话：

它不是单纯读参数，而是在 `AllocLevelData()` 和 `InitFromScratch()` 之前，先把“允许哪种物理-算法-存储组合存在”这件事裁成一个可执行的对象图。

接下来再往下走一步，就会看到 `AllocLevelMFs()` 真正把这张对象图摊成一组具体字段。这里最容易误判的是 `rho`、`aux`、外场、`Hybrid-PIC/fluid/macroscopic/EB` 这些分支。

先看 `rho_fp`。它并不是总存在的基础字段，而是只在几类路径下显式分配：

- `electrostatic / electromagnetostatic / effective-potential`
- `HybridPIC`
- `do_dive_cleaning`
- `PSATD` 且启用了 `update_with_rho` 或 `current_correction`

而且它的分量数也不是固定值：普通 `electrostatic` 或 `HybridPIC` 只需要 `ncomps`，`do_dive_cleaning` 一般把它升到 `2*ncomps`，`PSATD` 又会根据 `JRhom` 是否开启在 `ncomps` 和 `2*ncomps` 之间切换。也就是说，`WarpX` 初始化阶段持有一份“可演化的 `rho` 状态”，本身就是 `solver contract` 的一部分。

与此平行的还有三类不同的辅助标量/约束场：

- `phi_fp`：只属于 `electrostatic` 系列；
- `F_fp`：只属于 `div(E) cleaning`；
- `G_fp`：只属于 `div(B) cleaning`。

不要把它们混成“又分配了几个标量场”。它们分别服务于 `Poisson` 势解、电场散度清理和磁场散度清理，而且 `G` 的 `coarse-patch index type` 还会继续受 `grid_type` 控制。

再看外场分支。源码实际上维护了两套完全不同的合同：

1. `grid external fields` 它们分配成 `Efield_fp_external/Bfield_fp_external`，`index type` 必须匹配 `fp`，后面由 `AddExternalFields()` 加到主网格场上。
2. `particle external fields` 它们分配成 `E_external_particle_field/B_external_particle_field`，`index type` 必须匹配 `aux`，而且分量数直接来自外部粒子场元数据。

所以 `particle external field` 不是 `grid external field` 的别名；它跟粒子 `gather` 所看的 `aux` 路径绑定得更紧。这也解释了为什么只要 `mypc->m_E_ext_particle_s` 或 `m_B_ext_particle_s` 是 `read_from_file`，最常见的 `aux -> fp alias` 优化就会被打断，`Efield_aux/Bfield_aux` 必须改成单独分配。

剩下几条看似“模块化”的分支，其实也都在 `AllocLevelMFs()` 里继续扩展 `field registry`：

- `electromagnetic_solver_id == HybridPIC` 时，`m_hybrid_pic_model->AllocateLevelMFs(...)` 会追加 `hybrid` 自己的 `level` 字段；
- `do_fluid_species` 时，`myfl->AllocateLevelMFs(...)` 之后立刻 `InitData(...)`，说明 `fluid` 已经进入初始化主链，而不是仅仅挂了个容器；
- `m_em_solver_medium == Macroscopic` 时，`m_macroscopic_properties->AllocateLevelMFs(...)` 只允许 `lev==0`，直接把 `mesh refinement` 排除在外；
- `EB::enabled()` 时，所有 `level` 至少都会有 `distance_to_eb` 与 `m_eb_reduce_particle_shape`，而 `finest level` 上又会继续长出 `m_eb_update_E/B`；如果 `solver` 还是 `ECT`，则再额外长出 `edge_lengths`、`face_areas`、`area_mod`、`Venl`、`ECTRhofield` 和 `FaceInfoBox` 借用关系。

因此 `AllocLevelMFs()` 的真实角色不是“机械把 `MultiFab` 开出来”，而是把前面 `ReadParameters()` 裁出来的对象图真正展开成可执行状态。

更关键的是，这些特例字段不会等到 `Evolve()` 才第一次使用。在 `InitData()` 后半段，源码会立刻：

- `BuildBufferMasks()`
- `m_macroscopic_properties->InitData(...)`
- `m_electrostatic_solver->InitData()`
- `m_hybrid_pic_model->InitData(m_fields)`
- `ProjectionCleanDivB()`

这就说明初始化链的最后三步其实是：

1. `ReadParameters()` 决定哪些组合允许存在；
2. `AllocLevelMFs()` 把这些组合摊成真实字段和对象；
3. `InitData()` 后半段立刻消费这些字段，把它们变成一份可跑的初态。

再往后一步，`InitData()` 后半真正把这条合同闭合掉。它的顺序不是“初始化完就直接写 diagnostics”，而是先做一轮收尾和首次消费：

- `ComputeMaxStep()`
- `ComputePMLFactors()`
- 可选 `InitNCICorrector()`
- `BuildBufferMasks()`
- `m_macroscopic_properties->InitData(...)`
- `m_electrostatic_solver->InitData()`
- `m_hybrid_pic_model->InitData(m_fields)`
- `CheckGuardCells()`
- `ProjectionCleanDivB()`

这一步说明前面 `AllocLevelMFs()` 分配出来的对象并不是先放着，很多都会在这里立刻进入第一次初始化消费。

尤其要区分两步经常被混在一起的电场初始化动作：

1. `m_electrostatic_solver->InitData()` 这是 solver 对象自身的初始化，fresh run 和 restart 都会走。
2. `ComputeSpaceChargeField(reset_fields=false)` 这是 fresh-run 下的初始 self-field / electrostatic solve，只在需要时触发，而且还明确保留网格上已有的用户指定值，不会先把字段清空。

它的触发条件也不是“只有 electrostatic solver 才会跑”，而是三选一：

- 开启 electrostatic solver
- 任意 species 打开 `initialize_self_fields`
- 指定了 boundary potential

但如果当前走的是 HybridPIC，源码又会显式跳过这条初始 field-solve 路径。

在这之前，若 `m_do_initial_div_cleaning` 成立，还会先执行 `ProjectionCleanDivB()`。因此初始 B 场的散度修正顺序其实非常明确：先完成外场读入与对象初始化，再做 `div B cleaning`，随后才进入初始 self-field / electrostatic solve。

Python callback 也不是一个统一“初始化结束后调用”的事件，而是被拆成了三类窗口：

- `beforeInitEsolve`：只在 fresh run，发生在初始场求解前；
- `afterInitEsolve`：只在 fresh run，发生在初始场求解后、外加场提交前；
- `afterInitatRestart`：只在 restart，表示恢复态后处理，而不是 fresh-run 初始求解窗口的一部分。

外加场这里也有两步，不能混读：

1. `LoadExternalFields()` 先把 external grid fields 装到 `Efield_fp_external/Bfield_fp_external`，把 particle-only external fields 装到 `E_external_particle_field/B_external_particle_field`，并在 finest level 给 Python `loadExternalFields` callback 一个写这些 buffer 的机会。
2. `AddExternalFields()` 再把 grid external fields 统一加回 `Efield_fp/Bfield_fp` 主场。

所以第 0 步最终主场不是“纯 self-field 结果”，而是“初始 self-field / electrostatic solve 结果，再叠加 grid external fields”。

这之后才进入第 0 步 diagnostics：

- `multi_diags->FilterComputePackFlush(istep[0]-1)`
- `reduced_diags->ComputeDiags(...)`
- `reduced_diags->WriteToFile(...)`

因此第 0 步输出的并不是“原始输入快照”，而是“初始化主链已经全部完成后的第一份可运行状态快照”。对于 restart，这一步则取决于 `write_diagnostics_on_restart`，表示是否要把恢复态也立刻写成一份 diagnostics。

3A.3 顶层入口: fresh run 与 restart 分叉

源码文件: `Source/Initialization/WarpXInitData.cpp` 函数: `WarpX::InitData()`

第 3 章已经给过总表，这里看决定数据来源的核心分支。

```
if (!restart_chkfile.empty())
{
    InitFromCheckpoint(restart_chkfile);
    PrintDtDxDyDz();
    PostRestart();
}
else
{
    ComputeDt();
    PrintDtDxDyDz();
    InitFromScratch();
    InitDiagnostics();
}
```

这段分支决定初始化数据来源：

- restart: 从 checkpoint 恢复 mesh、field、particles 和时间层，再做 restart 后处理；
- fresh run: 先由 CFL、网格和 solver 计算 dt，再从零建立 AMR level、field、particles 和 diagnostics。

物理上，restart 应该恢复一个已经离散一致的状态；fresh run 则必须从输入参数构造这种一致状态。后面讲的外部场、初始粒子、Gaussian beam、openPMD 注入和 projection cleaning，主要属于 fresh run 路径。

可以把 fresh run 与 restart 的责任边界压缩成下面的初始化路线表：

输入状态	主要调用顺序	进入 <code>Evolve()</code> 前必须具备的状态
checkpoint restart	<code>InitFromCheckpoint()</code> -> <code>PostRestart()</code>	从 checkpoint 恢复 AMR level、fields、particles 和时间层，再完成 restart 后处理
fresh run	<code>ComputeDt()</code> -> <code>InitFromScratch()</code> -> <code>MakeNewLevelFromScratch()</code> -> <code>mypc->AllocData()/InitData()</code> -> PML -> <code>InitDiagnostics()</code>	根据输入重新建立 AMR、solver、粒子、外部场和初始 diagnostics

这张图的重点不是列出每一个初始化 helper，而是固定数据来源的不可互换性：restart 路径恢复已经离散化的状态，fresh run 路径才负责从参数重新物化 AMR、solver、粒子、外部场和初始 diagnostics。后面的 `PlasmaInjector`、Gaussian beam、openPMD 文件注入和 projection cleaning 都必须放在 fresh-run 分支内理解，不能被误写成 restart 的重复初始化。

3A.4 `InitFromScratch()`: AMReX level 与粒子初始化

源码文件: `Source/Initialization/WarpXInitData.cpp` 函数: `WarpX::InitFromScratch()`

AMReX 回调文件: `Source/WarpX.cpp` 回调函数: `WarpX::MakeNewLevelFromScratch()`

```
void
WarpX::InitFromScratch ()
{
    const Real time = 0.0;
    AmrCore::InitFromScratch(time); // calls MakeNewLevelFromScratch

    if (m_implicit_solver) {
```

```

    m_implicit_solver->Define(this, /*from_restart=*/false);
    m_implicit_solver->CreateParticleAttributes();
}

mypc->AllocData();
mypc->InitData();

InitPML();
ExecutePythonCallback("allocdata");
}

```

这里的顺序很重要：

1. `AmrCore::InitFromScratch(time)` 驱动 AMReX 创建 AMR level; 它会回调 `WarpX::MakeNewLevelFromScratch()`, 后者对每个 level 依次调用 `AllocLevelData(lev, ...)` 和 `InitLevelData(lev, time)`。因此 `AllocLevelData` 是这个回调链的一部分, 不是 `InitFromScratch()` 顶层直接调用的无参步骤。
2. 若启用了 implicit solver, `Define()` 与 `CreateParticleAttributes()` 在粒子初始化前建立求解器状态和所需粒子属性。
3. `mypc->AllocData()` 为粒子容器准备数据结构, `mypc->InitData()` 创建初始粒子。
4. `InitPML()` 初始化吸收边界数据结构; 随后 `allocdata` Python callback 才获得已经分配完的初始化对象。

因此, species 初始化发生在 field/level 数据结构已经存在之后, 但在正式时间推进之前。

粒子容器入口: 源码文件: `Source/Particles/PhysicalParticleContainer.cpp` 函数: `PhysicalParticleContainer::InitData()`

```

void PhysicalParticleContainer::InitData ()
{
    AddParticles(0); // Note - add on level 0
    Redistribute(); // We then redistribute
}

```

这两行是后续所有粒子创建逻辑的入口。 `AddParticles(0)` 负责生成初始粒子, `Redistribute()` 负责把粒子分配到正确 tile, 并清理 invalid 粒子。

3A.5 外部场初始化: grid field 与 particle external field

WarpX 支持两类外部场:

- grid external field: 外部场先放在 `Efield_fp_external/Bfield_fp_external` 网格 MultiFab 上, 随后由 `AddExternalFields()` 叠加到初始 E/B 网格场并施加 field boundary;
- particle external field: 外部场保存在 `E/B_external_particle_field` 中, 在 particle gather 时参与粒子受力, 并不经 `AddExternalFields()` 写入主 E/B 场。

外部场初始化的关键是: 外部场可以是常量、parser 函数、openPMD 文件或 Python 回调。读者应避免把“外部场”理解成单一数组。

源码文件: `Source/Initialization/WarpXInitData.cpp` 函数: `WarpX::LoadExternalFields(int lev)`

```

if (grid_B_from_file || grid_E_from_file ||
    particle_B_from_file || particle_E_from_file) {
    WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
        WarpX::do_moving_window == 0,
        "External fields from file are not compatible with the moving window.");
}

```

这里的布尔名是为阅读压缩后的语义名: 源码中展开检查 `B_ext_grid_type/E_ext_grid_type` 与 `m_B_ext_particle_s/m_E_ext_particle_s`。这个断言覆盖 **grid** 和 **particle** 两类 file-driven external field: moving window 移动后, 文件场的 MultiFab 没有自动重读/平移机制, 故当前实现直接拒绝这个组合。

断言之后, `LoadExternalFields()` 分三步处理数据: parser 或文件填充 grid external field; 在 finest level 调用 `loadExternalFields` Python callback; 再按 metadata 的每张 map 填充 particle external field。不要把这三步压成一个无参的“读文件函数”: 实际的 `ReadExternalFieldFromFile(path, MultiFab*, field, component, map_index)` 总是明确写出目标场与分量。

外部场的物理约束是: 如果读入的是 B 或矢量 A, 数值上还需要检查离散散度误差; 这就是后面 `projection divergence cleaner` 的用途。

3A.6 species 初始化: PlasmaInjector 是参数总容器

PlasmaInjector 的职责不是推进粒子, 而是把输入文件中一个 species 的初始化规则收集成一组可供 kernel 调用的对象。

源码文件: Source/Initialization/PlasmaInjector.cpp 函数: PlasmaInjector::PlasmaInjector()

```
std::string injection_style = "none";
utils::parser::query(pp_species, source_name, "injection_style", injection_style);
std::transform(injection_style.begin(),
               injection_style.end(),
               injection_style.begin(),
               ::tolower);

num_particles_per_cell_each_dim.assign(3, 0);

if (injection_style == "singleparticle") {
    setupSingleParticle(pp_species);
    return;
} else if (injection_style == "multipleparticles") {
    setupMultipleParticles(pp_species);
    return;
} else if (injection_style == "gaussian_beam") {
    setupGaussianBeam(pp_species);
} else if (injection_style == "nrandompercell") {
    setupNRandomPerCell(pp_species);
} else if (injection_style == "nfluxpercell") {
    setupNFluxPerCell(pp_species);
} else if (injection_style == "nuniformpercell") {
    setupNuniformPerCell(pp_species);
} else if (injection_style == "external_file") {
    setupExternalFile(pp_species);
} else if (injection_style != "none") {
    SpeciesUtils::StringParseAbortMessage("Injection style", injection_style);
}
```

这个分支定义了初始粒子创建的第一层分类:

injection_style	含义	后续创建路径
singleparticle	单个手工粒子	AddNParticles()
multipleparticles	手工粒子列表	AddNParticles()
gaussian_beam	空间高斯束流	AddGaussianBeam()
external_file	openPMD 粒子文件	AddPlasmaFromFile()
nrandompercell / nuniformpercell	体密度注入	AddPlasma()
nfluxpercell	面通量注入	AddPlasmaFlux()

在这一步之后, PlasmaInjector 还会把 host 侧 functor 拷贝到 device 侧, 保证后续 GPU kernel 可以调用。

3A.7 密度和动量分布: 从文本参数到 functor

密度解析由 SpeciesUtils::parseDensity() 完成。源码文件: Source/Utils/SpeciesUtils.cpp

```
if (rho_prof_s == "constant") {
    amrex::Real density = 0;
    utils::parser::getWithParser(pp_species, source_name, "density", density);
    h_inj_rho.reset(new InjectorDensity(
        (InjectorDensityConstant*)nullptr, density));
} else if (rho_prof_s == "parse_density_function") {
    std::string str_density_function;
```

```

utils::parser::Store_parserString(
    pp_species, source_name, "density_function(x,y,z)", str_density_function);
density_parser = std::make_unique<amrex::Parser>(
    utils::parser::makeParser(str_density_function,{"x","y","z"}));
h_inj_rho.reset(new InjectorDensity(
    (InjectorDensityParser*)nullptr, density_parser->compile<3>()));
} else if (rho_prof_s == "read_from_file") {
    std::string density_file;
    std::string field_name = "density";
    bool distributed = true;
    utils::parser::get(pp_species, source_name,
        "read_density_from_path", density_file);
    utils::parser::query(pp_species, source_name,
        "density_mesh_name", field_name);
    pp_species.query("read_density_distributed", distributed);
    h_inj_rho.reset(new InjectorDensity(
        (InjectorDensityFromFile*)nullptr,
        density_file, field_name, geom, distributed));
}

```

因此 file profile 不只是一个路径: density_mesh_name 选择文件中的 mesh record (默认 density), geom 提供目标几何, read_density_distributed 决定读取布局。复现实验时应把这四项与文件坐标/单位一起检查, 而不应只确认路径可打开。

体注入中的宏粒子权重由密度决定:

$$w_p \approx n(\mathbf{x}) \frac{\Delta V}{N_{ppc}}.$$

动量解析由 SpeciesUtils::parseMomentum() 完成。常见分支包括:

- at_rest: $u=0$;
- constant: 固定 $u_x/u_y/u_z$;
- gaussian: 每个方向正态采样;
- gaussianflux: 通量注入专用, 法向速度按 $v_n f(v)$ 加权;
- uniform: 每个方向均匀采样;
- maxwell_boltzmann: 非相对论 Maxwellian;
- maxwell_juttner: 相对论热平衡分布;
- parse_momentum_function: 空间解析函数。

InjectorMomentum 的关键工程实现是手写 tagged union。源码文件: Source/Initialization/InjectorMomentum.H 对象: InjectorMomentum

```

struct InjectorMomentum
{
    enum struct Type {
        constant,
        gaussian,
        gaussianflux,
        uniform,
        boltzmann,
        juttner,
        parser,
        gaussianparser
    };

    Type type;

    union {
        InjectorMomentumConstant constant;
    };
};

```

```

    InjectorMomentumGaussian gaussian;
    InjectorMomentumGaussianFlux gaussianflux;
    InjectorMomentumUniform uniform;
    InjectorMomentumBoltzmann boltzmann;
    InjectorMomentumJuttner juttner;
    InjectorMomentumParser parser;
    InjectorMomentumGaussianParser gaussianparser;
};

```

这不是普通虚函数多态，而是 GPU kernel 友好的平铺对象。后续 AddPlasma() 可以在 device 上用 getMomentum() 采样单粒子动量，用 getBulkMomentum() 得到平均漂移速度。

3A.8 AddParticles(): 按注入类型进入创建函数

源码文件:Source/Particles/ParticleCreation/AddParticles.cpp 函数:PhysicalParticleContainer::AddParticles(int lev)

```

void
PhysicalParticleContainer::AddParticles (int lev)
{
    ABLASTR_PROFILE("PhysicalParticleContainer::AddParticles()");

    for (auto const& plasma_injector : plasma_injectors) {

        if (plasma_injector->add_single_particle) {
            if (WarpX::gamma_boost > 1.) {
                MapParticleToBoostedFrame(plasma_injector->single_particle_pos[0],
                                           plasma_injector->single_particle_pos[1],
                                           plasma_injector->single_particle_pos[2],
                                           plasma_injector->single_particle_u[0],
                                           plasma_injector->single_particle_u[1],
                                           plasma_injector->single_particle_u[2]);
            }
            const amrex::Vector<ParticleReal> xp = {plasma_injector->single_particle_pos[0]};
            const amrex::Vector<ParticleReal> yp = {plasma_injector->single_particle_pos[1]};
            const amrex::Vector<ParticleReal> zp = {plasma_injector->single_particle_pos[2]};
            const amrex::Vector<ParticleReal> uxp = {plasma_injector->single_particle_u[0]};
            const amrex::Vector<ParticleReal> uyp = {plasma_injector->single_particle_u[1]};
            const amrex::Vector<ParticleReal> uzp = {plasma_injector->single_particle_u[2]};
            const amrex::Vector<amrex::Vector<ParticleReal>> attr = {{plasma_injector->single_particle_weight}};
            const amrex::Vector<amrex::Vector<int>> attr_int;
            AddNParticles(lev, 1, xp, yp, zp, uxp, uyp, uzp,
                        1, attr, 0, attr_int, 0);

            return;
        }
    }
}

```

后半段分派如下:

```

    if (plasma_injector->gaussian_beam) {
        AddGaussianBeam(*plasma_injector);
    }

    if (plasma_injector->external_file) {
        AddPlasmaFromFile(*plasma_injector,
                        plasma_injector->q_tot,
                        plasma_injector->z_shift);
    }

    if ( plasma_injector->doInjection() ) {

```

```

        AddPlasma(*plasma_injector, lev);
    }
}

```

这个函数说明: PlasmaInjector 中可能同时保存多种初始化信息, 但最终创建时按 flag 调用不同路径。

3A.9 体注入 AddPlasma(): 候选粒子、密度、动量和权重

体注入的核心思想是: 先按 cell 和 num_particles_per_cell 创建候选粒子, 然后用真实 density/bounds 筛掉无效粒子, 并把有效粒子写入 SoA。

源码文件: Source/Particles/ParticleCreation/AddParticles.cpp 函数: PhysicalParticleContainer::AddPlasma(...)

```

// count the number of particles that each cell in overlap_box could add
amrex::Gpu::DeviceVector<amrex::Long> counts(overlap_box.numPts(), 0);
amrex::Gpu::DeviceVector<amrex::Long> offset(overlap_box.numPts());
auto *pcounts = counts.data();
amrex::Box fine_overlap_box; // default Box is NOT ok().
if (refine_injection) {
    fine_overlap_box = overlap_box & amrex::shift(fine_injection_box, -shifted);
}
amrex::ParallelFor(overlap_box, [=] AMREX_GPU_DEVICE (int i, int j, int k) noexcept
{
    const amrex::IntVect iv(AMREX_D_DECL(i, j, k));
    auto lo = getCellCoords(overlap_corner, dx, {0._rt, 0._rt, 0._rt}, iv);
    auto hi = getCellCoords(overlap_corner, dx, {1._rt, 1._rt, 1._rt}, iv);

    lo.z = applyBallisticCorrection(lo, inj_mom, gamma_boost, beta_boost, t);
    hi.z = applyBallisticCorrection(hi, inj_mom, gamma_boost, beta_boost, t);

    if (inj_pos->overlapsWith(lo, hi))
    {
        auto index = overlap_box.index(iv);
        const amrex::Long r = (fine_overlap_box.ok() && fine_overlap_box.contains(iv)) ?
            (AMREX_D_TERM(rrfac[0], *rrfac[1], *rrfac[2])) : (1);
        pcounts[index] = num_ppc*r;
    }
}

```

applyBallisticCorrection() 用 bulk velocity 把 boosted-frame 位置反推到 lab-frame 初始面。其公式是:

$$z_{0,lab} = \gamma_b [z_{boost}(1 - \beta_b \beta_z) - ct_{boost}(\beta_z - \beta_b)].$$

随后用 prefix scan 得到写入 offset:

```

const amrex::Long max_new_particles = amrex::Scan::ExclusiveSum(counts.size(), counts.data(), offset.data());

amrex::Long pid;
{
    pid = ParticleType::NextID();
    ParticleType::NextID(pid+max_new_particles);
}

```

这种“两遍法”避免在 GPU kernel 中动态 push 粒子。

真正填充粒子时, 体注入权重系数为:

```

AMREX_GPU_HOST_DEVICE AMREX_FORCE_INLINE
amrex::Real compute_scale_fac_volume (const amrex::GpuArray<amrex::Real, AMREX_SPACEDIM>& dx,
                                       const amrex::Long pcount) {
    using namespace amrex::literals;
}

```

```

    return (pcount != 0) ? AMREX_D_TERM(dx[0],*dx[1],*dx[2])/pcount : 0.0_rt;
}

```

即

$$w_p = n(\mathbf{x}) \frac{\Delta V}{N_{ppc}}.$$

在 boosted-frame 分支中, 密度和纵向动量做 Lorentz 变换:

```

dens = gamma_boost * dens * ( 1.0_rt - beta_boost*betaz_lab );
u.z = gamma_boost * ( u.z -beta_boost*gamma_lab );

```

对应:

$$n' = \gamma_b n (1 - \beta_b \beta_z), \quad u'_z = \gamma_b (u_z - \beta_b \gamma).$$

最后写入粒子 SoA:

```

u.x *= PhysConst::c;
u.y *= PhysConst::c;
u.z *= PhysConst::c;

amrex::Real weight = dens;
weight *= scale_fac;

pa[PIdx::w][ip] = weight;
pa[PIdx::ux][ip] = u.x;
pa[PIdx::uy][ip] = u.y;
pa[PIdx::uz][ip] = u.z;

```

因此 InjectorMomentum 返回的 u 是无量纲 \gamma\beta, 粒子数组中存的是 \gamma v。

3A.10 Gaussian beam: 显式束流列表注入

Gaussian beam 不使用 InjectorDensity, 而是在 IO rank 上显式随机生成粒子列表。

源码文件:Source/Particles/ParticleCreation/AddParticles.cpp 函数:PhysicalParticleContainer::AddGaussianBeam(...)

```

if (ParallelDescriptor::IOProcessor()) {
    // If do_symmetrize, create either 4x or 8x fewer particles, and
    // Replicate each particle either 4 times (x,y) (-x,y) (x,-y) (-x,-y)
    // or 8 times, additionally (y,x), (-y,x), (y,-x), (-y,-x)
    if (do_symmetrize){
        npart /= symmetrization_order;
    }
    // compute the weight from N_tot if the user specified npart_real = N_tot
    // compute the weight from q_tot if the user specified q_tot
    // note that npart is the number of macroparticles
    const amrex::Real weight_3d = (N_tot > 0.0_rt) ? (N_tot / npart) : (q_tot / (npart*charge));
}

```

权重来自总真实粒子数或总电荷:

$$w_{3d} = \{N_{\text{tot}}/N_p, Q_{\text{tot}}/(N_p q)\}.$$

随后按高斯分布采样空间位置:

```

#if defined(WARPX_DIM_3D) || defined(WARPX_DIM_RZ)
    const amrex::Real weight = weight_3d;
    amrex::Real x = amrex::RandomNormal(x_m, x_rms);

```



```

    amrex::Real y = amrex::RandomNormal(y_m, y_rms);
    amrex::Real z = amrex::RandomNormal(z_m, z_rms);
#elif defined(WARPX_DIM_XZ)
    const amrex::Real weight = weight_3d/y_rms;
    amrex::Real x = amrex::RandomNormal(x_m, x_rms);
    constexpr amrex::Real y = 0._prt;
    amrex::Real z = amrex::RandomNormal(z_m, z_rms);

```

如果设置 focal_distance, 代码用弹道近似从焦平面束斑反推出初始位置。核心公式是:

$$t = \frac{(\mathbf{x}_f - \mathbf{x}) \cdot \mathbf{n}}{\mathbf{v} \cdot \mathbf{n}}, \quad \mathbf{x} \leftarrow \mathbf{x} - \mathbf{v}_\perp t.$$

同一函数在 focal-plane 分支中计算交叉时刻并反推横向初始位置:

```

// Compute the time at which the particle will cross the focal plane
const amrex::Real v_dot_n = v_x * n_x + v_y * n_y + v_z * n_z;
const amrex::Real t = ((x_f-x)*n_x + (y_f-y)*n_y + (z_f-z)*n_z) / v_dot_n;

// Displace particles in the direction orthogonal to the beam bulk momentum
// i.e. orthogonal to (n_x, n_y, n_z)
#if defined(WARPX_DIM_3D) || defined(WARPX_DIM_RZ)
x = x - (v_x - v_dot_n*n_x) * t;
y = y - (v_y - v_dot_n*n_y) * t;
z = z - (v_z - v_dot_n*n_z) * t;

```

如果设置 symmetrization, 代码为每个样本生成 4 或 8 个镜像粒子, 并把权重除以阶数。这降低横向低阶统计噪声。

3A.11 openPMD 粒子文件: 文件粒子列表注入

external_file 路径在构造期先打开 openPMD 文件, 读取可选 charge/mass。源码文件: Source/Initialization/PlasmaInjector.cpp
函数: PlasmaInjector::setupExternalFile()

```

void PlasmaInjector::setupExternalFile (amrex::ParmParse const& pp_species)
{
#ifdef WARPX_USE_OPENPMD
    WARPX_ABORT_WITH_MESSAGE(
        "WarpX has to be compiled with USE_OPENPMD=TRUE to be able"
        " to read the external openPMD file with species data");
#endif
    external_file = true;
    std::string str_injection_file;
    utils::parser::get(pp_species, source_name, "injection_file", str_injection_file);
    // optional parameters
    utils::parser::queryWithParser(pp_species, source_name, "q_tot", q_tot);
    utils::parser::queryWithParser(pp_species, source_name, "z_shift", z_shift);

```

质量和电荷优先级是:

input charge/mass > input species_type > openPMD charge/mass record

真正读入粒子时: 源码文件: Source/Particles/ParticleCreation/AddParticles.cpp 函数: PhysicalParticleContainer::AddPlasma

```

for (auto i = decltype(npart){0}; i < npart; ++i){

    amrex::ParticleReal const weight = ptr_w.get()[i]*w_unit;

    #if !defined(WARPX_DIM_1D_Z)
    amrex::ParticleReal const x = ptr_x.get()[i]*position_unit_x + ptr_offset_x.get()[i]*position_offset_u
    #else
    amrex::ParticleReal const x = 0.0_prt;

```

```

#endif
#if defined(WARPX_DIM_3D) || defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER) || defined(WARPX_DIM_RS)
    amrex::ParticleReal const y = ptr_y.get()[i]*position_unit_y + ptr_offset_y.get()[i]*position_offset_u
#else
    amrex::ParticleReal const y = 0.0_prt;
#endif
#if !defined(WARPX_DIM_RCYLINDER)
    amrex::ParticleReal const z = ptr_z.get()[i]*position_unit_z + ptr_offset_z.get()[i]*position_offset_u
#else
    amrex::ParticleReal const z = 0.0_prt;
#endif

```

openPMD 的 position 和 positionOffset 都乘以各自 unitSI, z_shift 是 WarpX 额外偏移。

动量换算:

```

if (plasma_injector.insideBounds(x, y, z)) {

    // The normalized momentum is u = p / m = gamma beta c
    // with m = m_e for photons, m the particle mass otherwise.
    amrex::ParticleReal const mass_eff = (m_mass > 0.0_prt) ? m_mass : PhysConst::m_e;
    amrex::ParticleReal const ux = ptr_ux.get()[i]*momentum_unit_x/mass_eff;
    amrex::ParticleReal const uz = ptr_uz.get()[i]*momentum_unit_z/mass_eff;
    amrex::ParticleReal uy = 0.0_prt;
    if (ps["momentum"].contains("y")) {
        uy = ptr_uy.get()[i]*momentum_unit_y/mass_eff;
    }
}

```

openPMD 文件中的 momentum 是物理动量 p。除以质量得到 $p/m=\gamma v$, 这正是 WarpX 粒子数组的 ux/uy/uz 量纲。文件中的 weighting 直接成为宏粒子权重; q_tot 只产生 warning, 不会重标定权重。

3A.12 Projection divergence cleaning: 外部 A/B 场的初始约束修正

如果外部加载的 B 或矢量 A 在离散网格上不满足散度约束, WarpX 可用 projection 方法清理。

数学上, 给定向量场 F, 构造:

$$\mathbf{F}' = \mathbf{F} + \nabla_h \phi, \quad \nabla_h \cdot \mathbf{F}' = 0.$$

于是

$$\nabla_h^2 \phi = -\nabla_h \cdot \mathbf{F}.$$

源码文件: Source/Initialization/DivCleaner/ProjectionDivCleaner.cpp

函数: ProjectionDivCleaner::setSourceFromField()

```

WarpX::ComputeDivB(
    *m_source[ilev],
    0,
    {&Bx, &By, &Bz},
    WarpX::CellSize(0)
);

```

```
m_source[ilev]->mult(-1._rt);
```

然后 ProjectionDivCleaner::solve() 用 AMReX MLMG 解 Poisson 方程。

配置文件: Source/Initialization/DivCleaner/ProjectionDivCleaner.H

```

amrex::MLMG mlmg(linop);
mlmg.setMaxIter(m_max_iter);
mlmg.setMaxFmgIter(m_max_fmg_iter);
mlmg.setBottomSolver(m_bottom_solver);
mlmg.setVerbose(m_verbose);
mlmg.setBottomVerbose(m_bottom_verbose);
mlmg.setConvergenceNormType(amrex::MLMGNormType::greater);
mlmg.solve({m_solution[lev].get()}, {m_source[lev].get()}, m_rtol, m_atol);

```

最后修正场:

```

Bx_arr(i,j,k) += T::DownwardDx(sol_arr, coefs_x, n_coefs_x, i, j, k);
By_arr(i,j,k) += T::DownwardDy(sol_arr, coefs_y, n_coefs_y, i, j, k);
Bz_arr(i,j,k) += T::DownwardDz(sol_arr, coefs_z, n_coefs_z, i, j, k);

```

这不是演化阶段的 `warpx.do_dive_cleaning/do_divb_cleaning`。后者是 Maxwell solver 时间推进中的清理变量或修正方程；本节讲的是初始化或外部场加载后的 Poisson projection。

Laser antenna 与 profile 分派: laser 初始化并不走 PlasmaInjector

species 初始化走的是 PlasmaInjector -> AddPlasma/AddGaussianBeam/AddPlasmaFromFile 这一条链；laser 初始化则完全不同。它的入口是:

- lasers.names
- LaserParticleContainer
- Laser/LaserProfiles.*

LaserParticleContainer 构造函数先统一读取天线几何和公共物理参数:

```

utils::parser::getArrWithParser(pp_laser_name, "position", m_position);
utils::parser::getArrWithParser(pp_laser_name, "direction", m_nvec);
utils::parser::getArrWithParser(pp_laser_name, "polarization", m_p_X);
utils::parser::getWithParser(pp_laser_name, "wavelength", m_wavelength);

```

然后要求 `e_max` 和 `a0` 二选一:

```

const bool e_max_is_specified =
    utils::parser::queryWithParser(pp_laser_name, "e_max", m_e_max);
Real a0;
const bool a0_is_specified =
    utils::parser::queryWithParser(pp_laser_name, "a0", a0);
...
AMREX_ALWAYS_ASSERT_WITH_MESSAGE(
    e_max_is_specified ^ a0_is_specified,
    "Exactly one of e_max or a0 must be specified for the laser.\n");

```

如果给的是 `a0`, WarpX 会立即按

$$E_{\max} = \frac{m_e \omega c}{q_e} a_0$$

换算成真实场强 `e_max`。这说明在 profile 实现层, `a0` 已经不再存在, 剩下的只是规范化后的公共参数。

profile 类型分派也不是一串手写 if-else, 而是 LaserProfiles.H 中的工厂字典:

```

laser_profiles_dictionary =
{
    {"gaussian",
     [] () {return std::make_unique<GaussianLaserProfile>();} },
    {"parse_field_function",
     [] () {return std::make_unique<FieldFunctionLaserProfile>();} },
    {"from_file",

```

```

[] () {return std::make_unique<FromFileLaserProfile>();} }
};

```

构造函数把 profile 字符串转成小写后直接做字典查找，再调用统一接口：

```

m_up_laser_profile = laser_profiles_dictionary.at(laser_type_s)();
...
m_up_laser_profile->init(pp_laser_name, common_params);

```

所以：

1. gaussian 走解析包络与聚焦/STC/chirp 公式；
2. parse_field_function 直接把 field_function(X,Y,t) 编译成 parser；
3. from_file 则走 lasy_file_name 或 binary_file_name，并用 time_chunk_size / delay 做时间分块读入。

更重要的是，laser pulse 不是直接“写入一个初始场数组”，而是通过人工天线粒子实现。LaserParticleContainer.H 的类注释写得很明确：这些粒子均匀分布在一个平面上，按预设位移沉积电流 J，再由 Maxwell solver 在网格上生成真正的激光场。因此 LaserParticleContainer 需要 current deposition，但不走普通 FieldGather，它也正因此直接继承 WarpXParticleContainer，而不是 PhysicalParticleContainer。

continuous injection 对 laser 还有额外几何约束：

```

AMREX_ALWAYS_ASSERT_WITH_MESSAGE(
    ...,
    "do_continuous_injection for laser particle only works"
    " if moving window direction and laser propagation direction are the same");

```

如果叠加 boosted frame，目前还进一步要求 boost 方向是 z。因此 laser 的 do_continuous_injection 虽然和普通 species 同名，但它的可用组合更窄，实际上是“moving-window / boosted-frame 下天线何时进入域内”的控制开关。

这还只是 laser 初始化链的入口。真正运行起来后，WarpX 并不会把解析式或文件 profile 直接写进 E/Bfield_fp。它先把 profile 解释成“天线平面上每个人工粒子的目标发射振幅”，再通过标准 J/rho 沉积把激光交给 Maxwell solver。

GaussianLaserProfile::init() 继续在公共参数外读取：

```

utils::parser::getWithParser(pp1, "profile_waist", m_params.waist);
utils::parser::getWithParser(pp1, "profile_duration", m_params.duration);
utils::parser::getWithParser(pp1, "profile_t_peak", m_params.t_peak);
utils::parser::getWithParser(pp1, "profile_focal_distance", m_params.focal_distance);
utils::parser::queryWithParser(pp1, "zeta", m_params.zeta);
utils::parser::queryWithParser(pp1, "beta", m_params.beta);
utils::parser::queryWithParser(pp1, "phi2", m_params.phi2);
utils::parser::queryWithParser(pp1, "phi0", m_params.phi0);

```

因此当前 gaussian profile 并不是“只有纵向和横向高斯包络”的最简形式，而是已经直接包含：

- 聚焦距离 profile_focal_distance
- carrier-envelope phase phi0
- spatial chirp zeta
- angular dispersion beta
- temporal chirp phi2

WarpX 随后还会把 stc_direction 归一化，并强制要求它与天线法向 nvec 正交：

```

WARPX_ALWAYS_ASSERT_WITH_MESSAGE(std::abs(dp2) < 1.0e-14,
    "stc_direction is not perpendicular to the laser plane vector");

```

这说明 stc_direction 的真实语义是“STC 在激光平面内的作用方向”，不是随手附带的一个参考向量。到了 fill_amplitude()，WarpX 再显式构造：

- diffract_factor
- inv_complex_waist_2
- stretch_factor

并按 3D/RZ、XZ、1D 分别选不同 prefactor。也就是说，当前 Gaussian 实现已经把 diffraction、Gouy phase、wavefront curvature、spatial chirp、angular dispersion 和 temporal chirp 全都折叠进同一个复 envelope 公式里，而不是后面再给场求解器额外修正。

from_file profile 的合同也比“读一个文件”更具体。源码强制要求：

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE (
    lasy_file_name.empty() != binary_file_name.empty(),
    "Exactly one of 'binary_file_name' and 'lasy_file_name' has to be specified");
```

因此 from_file 不是同时混用两种后端，而是在：

- lasy_file_name
- binary_file_name

之间二选一。并且它不是一次性把整份文件全读进来，而是先把 time_chunk_size 设成默认全时域，再允许用户把它改小，随后只预读第一块；真正运行时由 m_up_laser_profile->update(t_lab) 按需换块。delay 也不是写文件时的预处理，而是在 update() / fill_amplitude() 内统一平移 profile 时间轴。

更关键的是，LaserParticleContainer::InitData() 的产物不是“初始化场”，而是人工天线粒子。它先按 finest cell 和天线平面方向计算平面 spacing S_X/S_Y ，再建立实验室坐标与激光平面坐标之间的 Transform/InverseTransform，最后只在注入盒覆盖的区域生成粒子：

- Cartesian / XZ / 1D：每个平面位置生成一对 +w/-w 粒子
- RZ：围绕轴向展开成 spokes，并用 $2\pi r/n_spokes$ 修正权重

到了 LaserParticleContainer::Evolve()，运行时主链是：

1. 若在 boosted frame，先把数值时间 t 转回实验室系 t_lab
2. m_up_laser_profile->update(t_lab)，必要时推进 from_file 的时间块缓存
3. calculate_laser_plane_coordinates(...)，把真实粒子位置映回激光平面坐标
4. fill_amplitude(...)，为每个天线粒子求目标 E 振幅
5. update_laser_particle(...)，把振幅变成粒子动量和位置
6. DepositCurrent() / DepositCharge()，把人工天线粒子写入 current_fp/rho_fp，必要时也写入 coarse-fine current_buf/rho_buf

所以 WarpX 的 laser antenna 不是“边界直接给定场”，而是“人为构造一层发射粒子，通过普通沉积链把 profile 写成 J，再让 Maxwell solver 自己生成传播场”。这也解释了为什么 laser 在 mesh refinement 下照样要走 fp / buf 分流，以及为什么 lasers.deposit_on_main_grid 其实属于 AMR/沉积合同，而不只是一个 laser 小选项。

最后，laser 的 continuous injection 也应理解得更准确一点。ContinuousInjection() 检查的是更新后的 m_updated_position 是否第一次进入当前 injection_box；一旦进入，就调用一次 InitData() 生成那片天线粒子，之后再由普通 Evolve() 周期性更新它们的发射振幅。它不是每步“重新生成整束激光”，而是在 moving-window / boosted-frame 下决定“天线什么时候进入域内并开始工作”。

parse_field_function 这条分支也需要单独说明，因为它和前两类 profile 的数学合同并不一样。官方参数文档把它定义成：

```
<laser_name>.field_function(X,Y,t)
```

这里给出的不是包络，而是完整电场本身；X/Y 是垂直于激光传播方向的平面坐标，而不是固定的仿真坐标轴。FieldFunctionLaserProfile::init() 在源码里只做两件事：

```
utils::parser::Store_parserString(
    ppl, "field_function(X,Y,t)", m_params.field_function);
m_parser = utils::parser::makeParser(m_params.field_function,{"X","Y","t"});
```

随后 fill_amplitude() 也没有再加任何 envelope、phase 或 geometry 修正，而是直接逐粒子求值：

```
auto parser = m_parser.compile<3>();
amrex::ParallelFor(np, [=] AMREX_GPU_DEVICE (int i) noexcept
{
    amplitude[i] = parser(Xp[i], Yp[i], t);
});
```

这说明 parse_field_function 的 profile 层是三类实现里最薄的一层：用户写什么场函数，天线平面上就得到什么目标场值；后续所有“把目标场值变成可沉积粒子运动”的责任，都落在 LaserParticleContainer 的更新 kernel 上。

这几个 kernel 里，calculate_laser_plane_coordinates(...) 负责把真实粒子位置减去天线参考点后，投影回 m_u_X/m_u_Y 基底，因此 profile 接口始终统一在激光平面坐标上。ComputeWeightMobility() 则先用固定的峰值速度上限 $\epsilon_{ps} = 0.05$ 设定：

```

m_mobility = eps/m_e_max;
m_weight = PhysConst::epsilon_0 / m_mobility;
m_weight *= AMREX_D_TERM(1._rt, * Sx, * Sy);

```

也就是说, WarpX 不是先任意给天线粒子权重, 再让速度自己长出来; 而是先要求峰值场下粒子速度不超过 $0.05c$, 再反推单粒子权重。到了 `update_laser_particle()`, WarpX 再把 `amplitude[i]` 变成:

1. 沿主偏振方向 p_X 的速度
2. boosted-frame 下额外减去沿传播方向 $nvec$ 的平移速度
3. 相应的 relativistic momentum $ux/uy/uz$
4. 显式路径的整步位置推进, 或 implicit 路径基于 $x_n/y_n/z_n$ 的半步 time-centered 位置推进

因此 laser 的人工天线粒子并不是一个只服务显式 solver 的简单边界 hack。它同样要遵守 implicit particle-centering 合同, 并继续进入普通 `DepositCurrent()/DepositCharge()` 主链。

在本书引用的 WarpX 源树中, `parse_field_function` 的最明确真实用例是 `Examples/Tests/particle_absorbing_boundary/inputs_test` 这个输入把:

- `laser1.profile = parse_field_function`
- `laser1.field_function(X,Y,t) = ...`

嵌进了吸收边界测试里。但对应 `analysis.py` 检查的是边界附近的负向高速电子是否被抑制, 而不是直接检查激光场本身。这意味着 `parse_field_function` 目前是“有真实 regression 入口, 但没有独立 field-level 解析断言”的状态, 书稿里应把这个验证边界明确写出来。

把 laser 模块整体放回 regression 版图后, 还能看到另一个重要事实: 不同 laser tests 的证据强度差别很大。 `Examples/Tests/laser_injection/` 的 1D/2D analysis 会直接比较 Gaussian 注入场的包络和主频; implicit 1D/2D 变体也继续复用同一组 analysis, 因此并不只是“implicit 能跑通”的 checksum test。 `Examples/Tests/laser_injection_from_file/` 则继续给 lasy、legacy binary、boosted-frame 和 RZ thetaMode 文件提供 envelope/frequency 双断言。

但这一组还必须再分出一层 helper / prepare 边界。两个目录里的 `analysis_default_regression.py` 都只是 checksum helper: 职责是自动识别 plotfile/openPMD 并按测试目录名调用 `evaluate_checksum(...)`, 提供历史输出基线, 而不是新增 laser 物理断言。更重要的是, `laser_injection_from_file/` 里那批 `inputs_test_*_prepare.py` 并不是“待分析输入”, 而是被 `CMakeLists.txt` 先行注册成 dependency 的外部文件生成阶段:

- 普通 1D/2D/3D/RZ lasy 变体统一先写 `gaussian_laser_3d`
- legacy binary 变体先手工写 `gauss_2d`
- RZ thetaMode 变体先写 `laguerre_laser_RZ`

因此这组 regression 的正确结构不是单段输入, 而是:

1. prepare
 - 生成外部 laser 文件
2. inject
 - WarpX 按 `from_file / binary_file_name / RZ` 路径消费这些文件
3. analysis
 - 再对最终包络和主频做强断言

这条边界在解释 Laser/ 模块时很关键, 因为它说明“外部 laser 文件格式合同”本身已经是 active regression 的一部分, 而不只是示例配套脚本。

但到了 `Examples/Physics_applications/laser_acceleration/`, 情况就不一样了。这个目录本质上不是一组 laser-injection 单元测试, 而是一套 LWFA runtime matrix。 `README.rst` 自己都把 Analyze 章节留成了 TODO, 而当前大多数 active tests 在 `CMakeLists.txt` 中也都配置成 `analysis = OFF`, 只保留 checksum; 只有少数变体有明确 analysis:

- `analysis_1d_fluid_boosted.py`: 把 laser 驱动的 1D boosted fluid WFA 结果与理论 ODE 解对照, 检查 $Ez/Jz/\rho/Vz$
- `analysis_refined_injection.py`: 检查 `warpx.refine_plasma = 1` 场景下的总粒子数和 refinement edge 前方 ρ 切片均匀性
- `analysis_openpmd_rz.py`: 检查 RZ openPMD diagnostics 的 mesh shape、species ordering 和 `rho_<species>` 物理中心位置

更进一步, `inputs_base_1d/2d/3d/rz` 四个基础输入也说明了这组 family 先定义的是不同维度下的运行骨架:

- 1D: moving window + 连续电子注入 + Gaussian laser antenna + FieldProbe
- 2D: PML + moving window + refined patch + 连续背景电子 + Gaussian beam
- 3D: moving window + openPMD Full diagnostics + 自定义粒子属性

- RZ: `n_rz_azimuthal_modes = 2 + beam/plasma 共存 + species 变量输出`

因此 `laser_acceleration` 目录里的大多数条目当前更准确的定位应该是:

- LWFA application/runtime checksum baseline
- 以及 `boosted / MR / PICMI / Python callback / RZ / openPMD` 的路径覆盖

而不是统一的 `wake amplitude` 或 `laser envelope` 解析 benchmark。

此外, `Examples/Tests/boosted_diags/analysis.py` 对 `test_3d_laser_acceleration_btd` 的验证重点也不是 `laser` 包络本身, 而是:

1. BTD plotfile 与 BTD openPMD 的 `Ez` 是否逐点一致
2. `random_fraction` 粒子子采样是否真的生效

因此, 本书引用的 WarpX 源树对 `laser` 的回归支持应这样理解:

- 注入本体: 1D/2D 强, 3D 较弱
- `from_file`: 强
- `parse_field_function`: 有真实入口, 但主要是间接覆盖
- `laser_acceleration`: 多数是下游 LWFA/LPI 工作流回归, 不应误写成 `laser` 注入公式的直接解析验证
- BTD / openPMD / Python callback: 更偏 diagnostics 和 workflow 合同

这组边界在书稿中必须显式写出, 否则很容易把“有 `analysis.py`”误判成“已经有强物理断言”, 或者把 `laser_acceleration` 目录整体误判成 `laser injection` 的单元测试集合。

再往运行态交界看一层, `laser` 初始化还必须和 `moving window`、`boosted frame`、`continuous injection`、`external fields` 的更新合同一起理解。WarpX::MoveWindow() 在真正平移网格前, 会先做三件互不等价的更新:

```
moving_window_x += ...;
::UpdateInjectionPosition(*mypc, gamma_boost, beta_boost, boost_direction, moving_window_dir, dt[0]);
mypc->UpdateAntennaPosition(dt[0]);
```

这里:

1. `moving_window_x` 是窗口几何本身的位置;
2. `UpdateInjectionPosition(...)` 更新普通 species 的 `m_current_injection_position`;
3. `UpdateAntennaPosition(dt)` 更新 `laser antenna` 的 `m_updated_position`。

普通 species 的连续注入位置来自 `PlasmaInjector` 的 `bulk momentum`, 再换成速度并在 `boosted frame` 下做洛伦兹变换; `laser` 则完全不走 `PlasmaInjector`, 只在 `do_continuous_injection=1` 且 `gamma_boost>1` 时按 `boost velocity` 平移天线平面。因此两者虽然都叫 `continuous injection`, 但运行态位置更新机制不同。

两者的 `ContinuousInjection()` 语义也不同。普通物理粒子是在 `moving window` 新扫进来的 `level-0 particleBox` 中反复调用 `AddPlasma(...)`; `laser` 则只在 `m_updated_position` 第一次进入当前 `injection_box` 时调用一次 `InitData()`, 之后靠已有入天线粒子在 `Evolve()` 中持续更新并沉积 `J/rho`。AMR 下这两个尺度也不同: species 的 runtime 注入盒按 `level-0 cell` 对齐, 而 `LaserParticleContainer::InitData()` 仍然按 `maxLevel()` 的 `finest spacing` 建立天线粒子。

`external field` 的 `moving-window` 合同也在这里锁死。`LoadExternalFields()` 对 `B/E_ext_grid_type == read_from_file` 以及 `particle external field` 的 `read_from_file` 都直接断言:

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    WarpX::do_moving_window == 0,
    "External fields from file are not compatible with the moving window." );
```

原因不是 `openPMD` 不能读, 而是 `MoveWindow()` 在新进入的 `cells` 上只能通过 `shiftMF(...)` 用 `constant` 或 `parser` 两种方式重建主场背景:

```
srcfab(i,j,k,n) = external_field;
srcfab(i,j,k,n) = field_parser(x,y,z);
```

因此 `constant/parser` 外场可以跟着 `moving window` 继续生成, 而 `read_from_file` 缺少“窗口每推进一次就按新的 `physical coordinates` 增量重读”的实现, 所以被源码显式禁止。这也解释了为什么当前 `load_external_field*` regressions 都天然静态窗口场景, 而 `laser_acceleration_boosted`、`refined_injection`、`subcycling_mr` 这些例子才更贴近 `laser` 与 `moving-window` 交界的真实运行态合同。

再往应用层走, laser 已分叉成三种不同角色。laser_ion 是最典型的“laser 作为驱动器”的场景: 输入里同时绑了 Gaussian laser、solid-density target、full diagnostics、time-averaged diagnostics、ParticleHistogram、FieldProbe 和 ParticleHistogram2D。但它最硬的 regression 断言并不是离子能量标度, 而是 analysis_test_laser_ion.py 对 diagInst 最后 5 个 snapshot 的瞬时 Ez 平均值与 diagTimeAvg 原位 time-averaged Ez 的逐点比较。因此它最适合承担“laser 主链怎样进入复杂 diagnostics 组合场景”的角色。

free_electron_laser 则正好是反例: 它没有 lasers.names = ..., 而是通过刚性注入电子/正电子束、boosted frame、moving window 和外加 undulator B_y(z) 让辐射在束流中自发增长。analysis_fel.py 在 lab-frame 与 boosted-frame diagnostics 上分别拟合 gain length, 并通过 FFT 反推出 radiation wavelength。这说明这里的“laser/辐射”不是天线输入, 而是束流和 external particle field 共同产生的结果量, 所以它更像 laser 相关应用, 而不是 Laser 模块本身的 injection regression。

再往实现层拆, free_electron_laser 真正依赖的三块基础设施是:

1. RigidInjectedParticleContainer
2. particles.B_ext_particle_init_style = parse_B_ext_particle_function
3. BackTransformed diagnostics

它的 species 不会实例化普通 PhysicalParticleContainer, 而是被 MultiParticleContainer 切到 RigidInjectedParticleContainer。zinject_plane 和 rigid_advance 决定束团在注入面之前是按各自 v_z 还是按平均束流速度作刚体传播; boosted-frame 下 zinject_plane_levels 还会继续按 beta_boost c 平移。与此同时, undulator 场也不是写入主场 Bfield_fp, 而是通过 particle external field parser 直接在 gather 侧提供 B_y(z)。因此这里的主链其实是“刚性束流 + 粒子背景场 + BTD 恢复 lab-frame 物理”, 而不是 laser antenna 本体。

rigid_injection 和 boosted_diags 两组 tests 则给这条链提供了更基础的硬断言。前者分别在 lab frame 和 BTD 下检查: 刚性传播是否真的把束宽保持到 zinject_plane、以及 plotfile/openPMD 回写的束团位置与动量是否一致; 后者额外验证 BTD 两种 writer 的场数据一致性与 random_fraction 粒子子采样合同。也就是说, analysis_fel.py 负责最终 FEL 标度, rigid_injection* 负责 rigid propagation 本身, boosted_diags 负责 BTD 基础设施, 而这三层不应再被混写成一个笼统的“laser regression”。

laser_on_fine 则又是第三类。它确实使用真正的 Gaussian laser antenna, 但 CMakeLists.txt 里没有独立 analysis, 主要依赖 checksum; 输入重点在 max_level = 1、fine_tag_lo/hi、laser1.prob_lo/prob_hi 和 PML。也就是说它更像一个 AMR placement/solver 稳定性测试, 而不是下游应用 physics 场景。

因此, 后续书稿中的 laser 应用层不应只按 profile 分类, 而应按三种角色拆分:

1. laser 作为驱动器并配套 diagnostics 组合: laser_ion
2. 辐射/laser 作为输出结果量: free_electron_laser
3. laser 作为 AMR/placement 测试对象: laser_on_fine

还需要再补一个当前证据层更弱、但应用语义很典型的角色: plasma_mirror。它的输入已经把 Gaussian laser、solid-density target、前后指数梯度、PML、field filter 和双 species 固体靶骨架接在一起, 因此在应用语义上非常像“laser-solid surface-plasma 最小样板”; 但当前 active regression 只有 checksum helper, 没有独立 analysis, 也没有 PICMI 版输入。所以它更适合在书稿里承担“过密靶/表面等离子体应用骨架已经存在, 但强物理断言仍未单独压实”的角色, 而不应被写成 plasma-mirror 反射率或高次谐波 benchmark。

还需要再补一句边界: laser_ion 当前并不是“多物理全开”的综合 benchmark。它的输入确实给了三条可切换分叉:

- hydrogen.do_field_ionization = 1
- collisions.collision_names = ...
- 将来再接 do_qed_*

但在当前 regression 版本里, 这些开关都没有同时启用。它真正激活的是“Gaussian laser + 预电离 target + full/time-averaged/reduced diagnostics”。因此更准确的写法应该是: laser_ion 提供了一个 laser-target 骨架, field ionization、collisions 和 QED 都可以从这个骨架分叉出去, 但它们各自的物理正确性仍然主要由 field_ionization/、collision/、qed/ 这些独立 regression 目录兜底, 而不是由 analysis_test_laser_ion.py 一次性证明。

从源码链看, 这三条分叉接入的位置也不同。field ionization 在 species 构造期只先记住 do_field_ionization, 真正的 InitIonizationModule().mapSpeciesProduct() 和 doFieldIonization() 要到 MultiParticleContainer::InitMultiPhysics 与推进循环里才发生; collisions 则走 CollisionHandler 和 collision_names, 并额外受 collisions.split_momentum_push 的 operator ordering 影响; QED 又只在 #ifdef WARPX_QED 编译路径下才会继续增加 opticalDepthQSR/BW、product species 映射和 InitQED()。所以, laser_ion 更适合承担“应用输入如何把这些模块挂到同一目标骨架上”的说明, 而不应把不同层级的验证合同混写成一条单一主链。

3A.13 初始化验证入口：哪些 regressions 真正在兜底

前面的 3A.1-3A.12 讲的是“源码如何初始化”；若没有可执行 regression 对照，这些讲解很容易停留在静态阅读层。WarpX 对 Initialization 的验证并没有集中在一个目录里，而是分散在几组物理 test 中。

读这张验证地图时，始终先问四个问题：输入创建了什么初态？比较的 observable 是什么？analysis 真正断言了什么？它没有覆盖哪条分支？目录名、能否运行和 checksum 都不能替代这四个问题的答案。下面按读者最常遇到的初始化任务组织入口；同一条路径若只有 checksum，便只能作为回归基线，不能推出解析正确性。

第一组是 Langmuir。它通常被当成 evolve 基准，但对初始化同样关键，因为它直接覆盖：

- NUniformPerCell
- profile=constant
- parse_momentum_function
- 周期边界下的初始粒子/场一致性

例如 analysis_1d.py 的主断言不是 checksum，而是把输出 Ez 与理论 Langmuir 波逐点比较：

```
E_sim = data[("mesh", field)].to_ndarray()[:, 0, 0]
E_th = get_theoretical_field(field, t0)
max_error = abs(E_sim - E_th).max() / abs(E_th).max()
assert error_rel < tolerance_rel
check_charge_conservation(data)
```

这意味着 Langmuir 不只是“时间推进跑通”，而是在验证 parser 形式的初始扰动确实经过 PlasmaInjector、SpeciesUtils 和 AddPlasma() 正确落到了粒子和场上。

第二组是 space_charge_initialization。它最直接对应 species.initialize_self_fields = 1 这条初始化支线。输入文件显式打开：

```
beam.injection_style = "gaussian_beam"
beam.initialize_self_fields = 1
beam.momentum_distribution_type = "at_rest"
```

而 analysis 脚本把输出 Ex/Ey/Ez 与高斯电荷团理论场直接比较。因此这组 test 实际上在硬验证：

1. gaussian_beam 初始粒子云生成正确；
2. InitData() 检测到 has_initialize_self_fields；
3. ComputeSpaceChargeField(reset_fields=false) 在第一个时间步前给出了正确的 Coulomb 场。

第三组是 dive_cleaning。它验证的不是 projection cleaner，而是：

- 初始 Gaussian beam 状态进入演化后，
- warpx.do_dive_cleaning = 1 与 PML 能否把 $\text{div}(\mathbf{E}) - \rho / \epsilon_0$ 误差传播并吸收掉，
- 最终场是否回到理论 Gaussian beam 电场。

第四组是 gaussian_beam / external_file。这里要分两半看：

1. analysis_focusing_beam.py 和 analysis_rotated_beam.py 分别验证 focal_distance、束斑统计、旋转位置和旋转动量；
2. inputs_test_3d_focusing_gaussian_beam_from_openpmd_prepare.py + inputs_test_3d_focusing_gaussian_beam_ 则覆盖 external_file openPMD 粒子注入合同，包括 weighting、mass、charge、positionOffset 和 momentum.unit_SI = m_e c。

这一组现在还能再细一层：

- inputs_test_3d_focusing_gaussian_beam_photons 不是新物理 benchmark，而是把同一聚焦束斑统计合同重复到 species_type = photon 路径；
- inputs_test_3d_gaussian_beam_picmi.py 则主要覆盖 PICMI GaussianBunchDistribution 前端到 runtime attributes 的接线，当前主要依赖 checksum，而不是独立理论断言。

这里还要诚实记录一个源码边界：gaussian_beam/CMakeLists.txt 为 native test_3d_focusing_gaussian_beam_from_openpmd 指定了 analysis.py，但该目录没有这个文件。因而这条 native 路径只能说明 prepare -> external_file inject -> diagnostics 的输入与输出接口被覆盖，不能被称为官方的束斑物理强验证。相邻的 PICMI 入口复用 analysis_focusing_beam.py，可

以用来学习应比较的束斑统计量；它也不能自动补上 native 入口缺失的 analysis。读者若修改 native 输入，应自行对照粒子数、总权重、横向均方根束斑和纵向切片统计，并预先说明容差来自何处。

第五组是 electrostatic / EB 初始化：

- `effective_potential_electrostatic` 用电子径向密度和解析 adiabatic expansion 基准比较，验证 effective-potential electrostatic solver；
- `electrostatic_sphere_eb` 则用 ChargeOnEB reduced diag 和 `eb_covered` 场，验证 `InitEB()`、Poisson 边界条件和带导体球的初始势问题。

最后一组是 `projection_div_cleaner`，它对应 3A.12 的 Poisson projection，而不是演化阶段的 `do_dive_cleaning`。相关 tests 已覆盖：

1. RZ openPMD 文件外场版本；
2. 3D PICMI 文件外场版本；
3. Python callback 版本；
4. 2D 解析外场版本。

这些脚本的共同断言都是：初始化完成后，从 raw staggered `Bx_aux/By_aux/Bz_aux` 重建的离散 `divB` 必须足够接近零。

这里也要区分强断言的位置：

- `test_rz_projection_div_cleaner` 的强断言在独立 `analysis.py` 里；
- `test_3d_projection_div_cleaner_picmi`、`test_3d_projection_div_cleaner_callback_picmi` 和 `test_2d_projection_div_cleaner_initial_analytical_field_picmi` 则都把 `divB` 断言直接写在输入脚本尾部，所以 `CMakeLists.txt` 里虽然 `analysis=OFF`，但并不等于这些条目只是 checksum-only。

再往 `species` 入口侧补一组，`Examples/Tests/flux_injection/` 是直接锚定 `setupNFluxPerCell()` 的 regression 家族。这组 tests 分三条：

1. `analysis_flux_injection_3d.py`
 - 对 3D `NFluxPerCell` 场景同时检查总发射量、法向 Gaussian-flux 分布和切向 Gaussian 分布；
2. `analysis_flux_injection_rz.py`
 - 对 `flux_normal_axis = t` 的 RZ 连续注入检查粒子始终停留在预期 Larmor 半径带，并保持正确总通量；
3. `analysis_flux_injection_from_eb.py`
 - 对 `inject_from_embedded_boundary = 1` 的 2D/3D/RZ 变体检查发射总数、法向/切向速度统计，以及粒子不会落入 EB 内部。

因此 `flux_injection` 的意义不是普通 emitter 示例，而是 `NFluxPerCell`、Gaussian-flux rejection sampling 和 embedded-boundary surface emission 这三条运行态合同的直接验证入口。

第一组是 `initial_distribution`。它不是普通 smoke test，而是一组多 `species`、多分布的综合强基准：同一输入里同时覆盖

- gaussian
- maxwell_boltzmann
- maxwell_juttner
- gaussian_beam
- parser 温度
- parser bulk velocity
- uniform
- parser-Gaussian 动量统计

`analysis` 脚本把 reduced histogram、束斑统计和解析分布逐条对照。这意味着它真正验证的是 `PlasmaInjector`、`SpeciesUtils` 和 momentum-dispatch 层的 built-in / parser 初始化合同，而不只是“粒子能被建出来”。

由于初始化使用随机采样，checksum 的默认容差不应被理解为跨机器、跨 MPI 布局都逐位相同的物理合同。复现实验应优先比较 `analysis` 脚本定义的直方图、均值、方差、束斑等统计量，并在报告中写清样本量、随机数种子（若可控）、并行布局和容差；只有这些条件一致时，checksum 才能作为补充证据。

第二组是 `initial_plasma_profile`。这组当前没有独立 `analysis.py`，只有 checksum helper，但输入本身非常明确：

- `injection_style = NUniformPerCell`
- `profile = parse_density_function`

并把横向 parabolic channel 与纵向 ramp / plateau / ramp 组合成二维电子密度。所以它更准确地是：

- `parse_density_function` 抛物型通道初始化的 checksum-only 基线

而不是应继续留在 `general / to classify` 的未知条目。

再往 `initialize_self_fields` 这一支补一组, `repelling_particles` 是一个更小但更干净的两体基准。它只放两个同号 `SingleParticle` species, 却同时打开:

- `electron1.initialize_self_fields = 1`
- `electron2.initialize_self_fields = 1`

analysis 随后从连续 `plotfiles` 读取两粒子的间距和速度, 并用两体排斥的非相对论能量守恒关系构造理论 $\beta(d)$ 。因此这组 regression 的真实意义不是“一对粒子可能会分开”, 而是:

1. 初始 electrostatic self-field 确实被建立起来;
2. 后续 pusher 对这份初始场的消费是对的;
3. 两者联立后能回到解析两体减速关系。

接着把另外三组容易落单的入口也补上之后, 初始化验证图还能再细一层。

第一组是 `load_external_field`。它不是普通粒子轨道测试, 而是在验证两套初始化合同:

1. grid external field:
 - `LoadExternalFields()`
 - `ReadExternalFieldFromFile()`
 - `Bfield_fp_external/Efield_fp_external`
 - `AddExternalFields()`
2. particle external field:
 - `Particles/ExternalParticleFields.cpp`
 - `m_B_ext_particle_s / m_E_ext_particle_s`
 - `B_external_particle_field/E_external_particle_field`
 - `GetExternalFields.cpp`

`analysis_3d.py / analysis_rz.py` 通过磁镜中的单粒子最终位置做硬断言, 说明这组 regression 同时验证了“初始化写场”和“后续 gather 消费场”的接口契合。时间依赖变体 `analysis_time_scaling.py` 则不看粒子, 而是直接比较两个时刻 `plotfile` 上 `B` 分量的缩放比, 从而验证 `read_fields_*_dependency(t)` parser 和多 field map 的时间缩放合同。

这组 family 里还要再区分一层 restart 保真。当前活跃三条最小入口是:

- `test_3d_load_external_field_particle_time_restart`
- `test_rz_load_external_field_grid_restart`
- `test_rz_load_external_field_particles_restart`

它们都只是:

- 先继承对应非 restart 输入
- 再通过 `amr.restart = ../../chk000150` 从中间 checkpoint 恢复
- 然后复用 `analysis_default_restart.py` 逐字段比较 restart 与非 restart 输出

因此这三条 regression 验证的不是新的外场 physics, 而是:

1. `read_from_file` 的 grid external field 状态能否在 restart 后保持一致;
2. `read_from_file / dependency parser` 的 particle external field 状态能否在 restart 后保持一致;
3. 初始化阶段构造出的 external-field 寄存器, 不会在 checkpoint/restart 边界上丢失或漂移。

第二组是 `relativistic_space_charge_initialization`。它的输入和普通 `space_charge_initialization` 一样也打开了:

```
beam.initialize_self_fields = 1
beam.injection_style = "gaussian_beam"
```

但束流动量改成了 relativistic:

```
beam.uz = 100.0
```

因此这组 regression 实际验证的是 `RelativisticExplicitES::ComputeSpaceChargeField()`, 而不再只是静止高斯电荷团的 Coulomb 场。analysis 脚本把 `Ex` 与理论值比较, 并检查 `By` 是否满足相对论束流自场的 $By \approx Ex/c$ 结构。这说明 `initialize_self_fields` 在 relativistic solver 分支下对应的是另一份初始化合同。

第三组是 `open_bc_poisson_solver`。它把四个条件绑在一起：

```
boundary.field_lo = open open open
boundary.field_hi = open open open
warpx.do_electrostatic = relativistic
warpx.poisson_solver = fft
electron.initialize_self_fields = 1
```

同时粒子不是 `gaussian_beam`，而是 `parse_density_function + NUniformPerCell`。analysis 脚本用 Basseti-Erskine 公式逐个 z 截面比较 E_x/E_y ，因此它验证的不是一般的 electrostatic 初始化，而是：

- open boundary
- relativistic bunch
- FFT Poisson
- 以及可选 `warpx.use_2d_slices_fft_solver = 1`

共同定义的初始 Poisson 解是否正确。

接下来三组补足了文件驱动分布、带嵌入边界的自场，以及 `collocated` 采样的分支；它们与前面的束流注入和 Poisson 例子不共享同一个可观测量，因而应分别阅读。

第一层是 `load_density`。这组 regression 的输入明确使用：

```
electrons.profile = "read_from_file"
electrons.read_density_from_path = "../test_*_load_density_prepare/example-density.h5"
electrons.do_continuous_injection = 1
warpx.do_moving_window = 1
```

源码上它直接对应 `SpeciesUtils::parseDensity()` 里 `profile = read_from_file` 分支，把 openPMD density mesh 装进 `InjectorDensityFromFile`，然后交给 `PlasmaInjector` 和连续注入主链消费。analysis 脚本则逐个 iteration 读取 diagnostics 中的 ρ ，与 prepare 脚本定义的 ramp / parabolic channel profile 比较。因此 `load_density` 验证的不是一般 I/O，而是“file-driven density profile + moving-window 连续注入”这条初始化合同。

第二层是 `magnetostatic_eb`。原生 inputs 文件把：

```
warpx.do_electrostatic = labframe-electromagnetostatic
beam.initialize_self_fields = 1
warpx.eb_implicit_function = "(x**2+y**2-radius**2)"
warpx.eb_potential(x,y,z,t) = "1."
```

绑在一起，而 `WarpXInitData.cpp` 的 `fresh-run` 分支会在 `ComputeSpaceChargeField(reset_fields)` 之后继续调用 `ComputeMagnetostaticField()`。所以这组 test 验证的是 embedded boundary、边界 potential、初始 self-field 和 magnetostatic solve 在初始化阶段的联动。这里还必须区分两层证据：原生 `inputs_test_3d_magnetostatic_eb` 目前主要由 checksum 兜底；两个 PICMI 输入文件则在 `sim.step()` 之后内嵌了解析 $E_r/B_\perp$ 误差断言，因此它们不是简单的 checksum-only regression。

第三层是 `nodal_electrostatic`。这组输入把：

```
warpx.do_electrostatic = relativistic
warpx.grid_type = collocated
beam_p.initialize_self_fields = 1
beam_p.do_qed_quantum_sync = 1
```

放在同一条链上。它的 analysis 不直接比较 E/B ，而是用 reduced diagnostics 断言 `ParticleExtrema_beam_p` 给出的最大 χ 极小，且 `ParticleNumber` 中 photon 数始终为零。也就是说，这组 regression 验证的是 `collocated relativistic electrostatic` 初始 self-field 没有制造出会假触发 QED 的非物理场，它更准确地是一个“零触发基准”。

这样 `nodal_electrostatic`、`open_bc_poisson_solver` 和 `relativistic_space_charge_initialization` 就不再需要继续共用一个过粗的 electrostatic / Poisson 桶。它们分别对应的是：

- `collocated relativistic electrostatic` 零触发基准
- open boundary + FFT/sliced FFT 的 relativistic Poisson 初始化
- relativistic Gaussian beam 的初始 self-field

不过还有一组此前仍容易被写得过粗：`electrostatic_sphere`。它不该和一般 Poisson 条目混成一桶，因为它真正验证的是“一个静止均匀电子球在自身 Coulomb 场下膨胀”这条自场初始化主链。`analysis_electrostatic_sphere.py` 的第一层断言是解析电场对照：脚本用最终输出

时间 $t_{\max}$ 反解球半径 r_{end} , 再构造内外球解析 $E(r)$, 沿坐标轴比较 $E_x/E_y/E_z$ 或 RZ 下的 E_r/E_z 的相对 L_2 误差。这意味着它验证的不仅是 solver 跑通, 而是:

1. 初始电子球几何是否被正确装填;
2. 初始自场是否正确建立;
3. 后续 electrostatic 演化是否仍与解析膨胀解一致。

这组 test 的第二层断言只在 lab-frame 变体上才打开。只有当输入显式要求:

```
warpx.do_electrostatic = labframe
diag2.electron.variables = x y z ux uy uz w phi
```

analysis 才会利用粒子 phi 重建:

- 动能
- 自场势能 $0.5 \sum w q \phi$

并检查初末总能量。也就是说:

- 所有 electrostatic_sphere 变体都做解析电场对照;
- 只有写出 phi 的 lab-frame 版本额外验证能量账本。

各输入变体的角色也不同:

- inputs_test_3d_electrostatic_sphere
 - 3D relativistic electrostatic 自场膨胀基线
- inputs_test_3d_electrostatic_sphere_lab_frame
 - lab-frame 自场膨胀 + 能量守恒
- inputs_test_3d_electrostatic_sphere_lab_frame_mr_emass_10
 - lab-frame + MR; electron.mass = 10 主要是减慢膨胀, 便于短步数比较
- inputs_test_3d_electrostatic_sphere_rel_nodal
 - warpx.grid_type = colocated, 验证 colocated electrostatic 布局
- inputs_test_3d_electrostatic_sphere_adaptive
 - adaptive-dt 变体; analysis 仍用实际 $t_{\max}$ 做强对照
- inputs_test_rz_electrostatic_sphere
 - RZ 自场膨胀 + 能量守恒
- inputs_test_rz_electrostatic_sphere_uniform_weighting
 - 再额外覆盖 radial_numpercell_power = 1. 的 uniform-weighting 粒子装填

另外, 这组目录下还有两个容易误读的文件:

- analysis_default_regression.py 只是通用 checksum helper;
- catalyst_pipeline.py 只是 ParaView Catalyst 的可视化脚本。

因此, electrostatic_sphere 在初始化验证图里更准确的位置应单独写成:

14. electrostatic self-field expansion: electrostatic_sphere*

还剩另外两组此前也容易被写得过粗, 但它们和 electrostatic_sphere 不是一类问题。

第一组是 electrostatic_dirichlet_bc。这组不是带电粒子自场 benchmark, 而是纯粹在验证时变边界势是否真正进入 electrostatic 解。输入直接设置:

```
warpx.do_electrostatic = labframe
boundary.potential_lo_x = 150.0*sin(2*pi*6.78e+06*t)
boundary.potential_hi_x = 450.0*sin(2*pi*13.56e+06*t)
diag1.fields_to_plot = phi
```

analysis 并不读取内部粒子量, 而是逐个输出时刻取两侧边界上的平均 phi, 然后与理论正弦函数比较。因此它测的不是一般 Poisson 正确性, 而是:

- boundary.potential_lo_x / potential_hi_x
- time-dependent parser
- electrostatic Dirichlet boundary condition
- diagnostics 中 phi 的边界保真

PICMI 变体只是在 `Cartesian2DGrid(..., lower_boundary_conditions=["dirichlet", ...])`、`warpx_potential_lo_x`、`warpx_potential_hi_x` 这一层重复同一件事，所以它更准确的归类是：

15. time-dependent electrostatic boundary driving: `electrostatic_dirichlet_bc*`

第二组是 `effective_potential_electrostatic`。这组当前只有一个 PICMI test，而且它验证的也不是一般意义上的 electrostatic phi 场，而是 `warpx_effective_potential=True` 这条 solver 分支是否能在导体球约束下复现绝热膨胀 benchmark。PICMI 输入脚本同时做了三件关键事：

1. 用 `GaussianBunchDistribution` 初始化电子和离子球团；
2. 加入导体球 embedded boundary；
3. 选择：

```
picmi.ElectrostaticSolver(  
    method="Multigrid",  
    warpx_effective_potential=True,  
    warpx_effective_potential_factor=C_EP,  
)
```

analysis 则从 `sim_parameters.dpk1` 读回参数，构造 Connor et al. 风格的绝热膨胀近似电子密度，再把 openPMD 中的 `rho_electrons` 变成径向密度曲线，与理论值逐个输出时刻比较 RMS 误差。因此它的真实验证对象是：

- effective-potential electrostatic solver
- PICMI front-end
- 导体球约束下的电子密度膨胀近似

更准确的归类应是：

16. effective-potential electrostatic: `effective_potential_electrostatic`

3A.14 从 Birdsall 3A ES1 到 WarpX: 历史最小程序骨架的现代映射

Birdsall and Langdon 的 3A ES1 是一份很好的历史参照，因为它把一维静电 PIC 的最小程序压缩成一条容易检查的阶段链：

INIT -> SETRHO -> FIELDS -> SETV -> ACCEL -> MOVE -> HISTRY

这条链可以帮助读者理解“初始化结束后第一步推进究竟从哪里开始”，但不能把旧程序的子程序名直接当成 WarpX 的函数名。历史参照见 Birdsall 与 Langdon 的 *Plasma Physics via Computer Simulation*；现代实现应从 `Source/Initialization/WarpXInitData.cpp`、`Source/Particles/` 和 `Source/FieldSolver/` 的对应职责回查。

3A ES1 阶段	历史程序的物理职责	WarpX 中最接近的阶段	不能直接等同的部分
INIT	建立网格、粒子、权重、边界和初始参数	<code>ReadParameters()</code> 、 <code>WarpX::InitData()</code> 、 <code>InitFromScratch()</code> 、 <code>AllocLevelData()</code> 与 <code>mypc->AllocData()</code>	WarpX 还要处理 AMR、多几何、solver 分支、PML、外部场、restart 和并行布局
SETRHO	用初始粒子位置完成第一次 charge deposition	<code>PlasmaInjector/AddParticle</code> 之后的初始 rho 构造及相关 field-register 路径	现代 WarpX 的 rho 是否直接写入、重新沉积或由 solver 分支消费，取决于 geometry、solver 和初始化选项
FIELDS	从 rho 求解静电势和电场，并形成可供粒子使用的场	electrostatic solver 的 <code>InitData()</code> 、 <code>ComputeSpaceChargeField()</code> 、初始场填充与 projection cleaning	WarpX 不只有一条 FFT Poisson 路径；EM、PSATD、RZ、EB 和 external-field 分支会改变字段对象与约束
SETV	设置初始速度、热分布和漂移	<code>SpeciesUtils</code> 、 <code>InjectorMomentum</code> 、temperature/velocity functor 与粒子属性创建	WarpX 还可能创建 relativistic、spin、implicit 或 pusher-specific attributes；并非一次简单数组赋值

3A ES1 阶段	历史程序的物理职责	WarpX 中最接近的阶段	不能直接等同的部分
ACCEL	用当前电场/磁场更新粒子速度	Evolve() 内的 particle push 与 gather	历史 ES1 的静电推进不能覆盖现代 EM、Boris/Vay/Higuera-Cary、implicit 和 subcycling 路径
MOVE	用更新后的速度推进粒子位置	PushParticlesAndDeposit() 中的 position update、边界处理和 current deposition	WarpX 还要处理 AMR tile、moving window、particle boundary、suborbit/crossing 和 MPI 交换
HISTRY	记录历史量、能量或分布函数	full/reduced diagnostics、openPMD/plotfile writer 与 reader-side analysis	现代 diagnostics 是独立 writer/schema 合同，不等于旧程序中的一个历史数组

最容易误读的是 SETRHO -> FIELDS -> SETV 的顺序。对历史 ES1 来说，它表达的是“先由初始位置形成源，再求场，再给粒子速度”；对 WarpX fresh run，实际初始化链还要先完成 AMReX level 和 field data 分配，并根据 solver、外场和 initialize_self_fields 等条件决定哪些源/场对象被创建。因而更可靠的读法是：Birdsall 骨架提供物理阶段的语义，WarpX 源码决定这些语义在现代对象图和分支条件中的落点。

这条映射也解释了为什么 Langmuir regression 可以同时作为初始化和演化入口：它检查的不是某个名为 SETRHO 的函数，而是初始粒子/密度、初始场、后续推进和最终解析频率之间的组合合同。相反，initial_distribution、space_charge_initialization、load_external_field 与 projection_div_cleaner 分别覆盖粒子分布、初始 self-field、外部场装填和初始散度修正的更窄路径。它们共同支撑的是现代 WarpX 初始化链的分层验证，而不是对 3A ES1 原程序做逐行复现。

3A.15 本章小结：初始化状态怎样进入第一步推进

到 InitData() 结束时，WarpX 已经完成：

```
inputs
-> WarpX::ReadParameters()
-> WarpX::InitData()
-> InitFromScratch or InitFromCheckpoint
-> AMReX levels and WarpX field data
-> external fields
-> species PlasmaInjector
-> AddParticles / AddPlasma / AddGaussianBeam / AddPlasmaFromFile
-> projection div cleaner if needed
-> initial diagnostics
-> Evolve()
```

这个状态才是 PIC 时间推进的初始条件。后续 Evolve()、OneStep()、PushParticlesandDeposit()、field solver 和 diagnostics 都在这个已经离散化、分布式、带权重和边界条件的状态上工作。

初始化到推进的交界由 Evolve()、OneStep() 与 PushParticlesandDeposit() 承接：前者负责外层时间步，后者负责 solver/AMR 分派，最后才把已经初始化的粒子和场送入实际粒子推进与沉积路径。历史 ES1 的阶段名只提供物理职责的参照，不能替代这些现代对象和分支。

进一步学习可沿以下方向展开：

- 把 InitLevelData() 中每一类 field allocation 展开到 root/fieldsolver 章节；
- 把 Gaussian beam 的 emittance/focal distance 公式结合 accelerator beam optics 文献继续推导；
- 把 openPMD 文件格式与 WarpX 单位约定加入诊断/I/O 章节；
- 为不同初始化路径设计可重复的验证案例，并记录它们各自能说明的边界。

3A.16 练习与最小复现

1. **fresh/restart 定位题**：沿 WarpXInitData.cpp 追踪 InitData()，说明 ComputeDt() 为什么只出现在 fresh-run 分支，以及 restart 为什么必须进入 PostRestart()。
2. **初始化与复现题**：解释 AmrCore::InitFromScratch()、AllocLevelData()、myrc->AllocData()、myrc->InitData() 和 InitPML() 的先后关系。再在 Examples/Tests/initial_distribution/ 选择匹配输入，核对生效输入、首个 diagnostics 和 species 数，并说明输入不匹配导致的 abort 为什么既不构成初始化通过，也不构成其反证。

4. 粒子推进器：从 Lorentz 方程到 PushPX()

粒子推进器负责把单个宏粒子从一个时间层推进到下一个时间层。它表面上是局部单粒子算法，实际上依赖上一章建立的主循环条件：场必须已经填好 gather guard cells，粒子位置和动量必须处在正确 leapfrog 时间层，外场、电离电荷态、辐射反作用和 QED 选项也必须在进入 pusher 前准备好。

本章按一条读者可追踪的因果链展开：先用 Lorentz 方程固定时间层和动量记号；再比较 Boris、Vay 与 Higuera–Cary 如何更新动量；最后从 WarpX::PushParticlesandDeposit() 经 MultiParticleContainer::Evolve() 进入 PhysicalParticleContainer::Evolve() 与 PushPX()，检查 gather、外场、push、位置更新和沉积如何接在同一 tile loop 中。需要核对实现时，优先搜索 PushSelector.H 的 doParticleMomentumPush()、三个 UpdateMomentum*.H 函数，以及 PhysicalParticleContainer::PushPX()；不要把行号或一次源码快照当作算法语义。

阅读 Boris 推进时要特别区分半步磁旋转的 Birdsall–Langdon 半角关系，不能把旋转系数机械地除以二；Examples/Tests/particle_pusher 提供 Higuera–Cary force-free 路径的直接验证入口。

4.1 连续 Lorentz 方程

WarpX 对带质量粒子推进的基本方程是相对论 Lorentz 方程。令

$$\mathbf{u} = \gamma \mathbf{v}, \quad \gamma = \sqrt{1 + \frac{|\mathbf{u}|^2}{c^2}}, \quad \mathbf{v} = \frac{\mathbf{u}}{\gamma}.$$

这里 WarpX 源码中的 ux ， uy ， uz 是 $\mathbf{u}$ 的三个分量，而不是普通速度 $\mathbf{v}$ 。对质量 m 、电荷 q 的粒子，

$$\frac{d\mathbf{u}}{dt} = \frac{q}{m} (\mathbf{E} + \mathbf{v} \times \mathbf{B}),$$

$$\frac{d\mathbf{x}}{dt} = \mathbf{v} = \frac{\mathbf{u}}{\gamma}.$$

显式 leapfrog PIC 中，常用时间层是

$$\mathbf{x}^n \rightarrow \mathbf{x}^{n+1}, \quad \mathbf{u}^{n-1/2} \rightarrow \mathbf{u}^{n+1/2}.$$

因此位置推进使用更新后的半步动量：

$$\mathbf{x}^{n+1} = \mathbf{x}^n + \frac{\mathbf{u}^{n+1/2}}{\gamma^{n+1/2}} \Delta t.$$

WarpX 的显式位置更新位于 Source/Particles/Pusher/UpdatePosition.H 的 UpdatePosition()；它先通过 GetExplicitPusherDisplacement() 用完整三速度分量构造位移，再按编译几何写回实际存储的坐标分量。

这里可以补一个 Dawson 1983 的历史边界。那篇综述在 **electromagnetic particle models and fractional dimensional models** 一节明确指出：只要问题涉及自洽磁场或 electromagnetic radiation，particle model 就不能再被理解成“electrostatic 外面多加几个场分量”，而必须回到 full Maxwell equations 与 self-consistent current/charge representation。与此同时，低空间维度并不意味着低速度维度；为了在一维或二维空间里承载 electromagnetic wave，仍必须保留 transverse $\mathbf{E}/B/j$ 与 transverse velocity，于是才会自然出现 1 1/2-D、1 2/2-D 和 2 1/2-D 这些 fully electromagnetic reduced-dimension models。这个边界正好解释了为什么本章后面很多看似“低维”的 laser、beam 和 FEL benchmark，仍然必须认真讨论 magnetic rotation、relativistic momentum 与 transverse field coupling，而不能把它们误降成纯 electrostatic pusher。

4.2 Boris 推进的结构

Boris pusher 把动量更新拆成三个操作：

1. 电场半步；
2. 磁场旋转；
3. 电场半步。

设旧动量为 $\mathbf{u}^{n-1/2}$ ，先做电半步：

$$\mathbf{u}^- = \mathbf{u}^{n-1/2} + \frac{q\Delta t}{2m} \mathbf{E}^n.$$

再用 $\mathbf{u}^-$ 计算

$$\gamma^- = \sqrt{1 + \frac{|\mathbf{u}^-|^2}{c^2}}, \quad \mathbf{t} = \frac{q\Delta t}{2m\gamma^-} \mathbf{B}^n, \quad \mathbf{s} = \frac{2\mathbf{t}}{1 + |\mathbf{t}|^2}.$$

磁场旋转写成

$$\mathbf{u}' = \mathbf{u}^- + \mathbf{u}^- \times \mathbf{t},$$

$$\mathbf{u}^+ = \mathbf{u}^- + \mathbf{u}' \times \mathbf{s}.$$

最后做第二个电半步：

$$\mathbf{u}^{n+1/2} = \mathbf{u}^+ + \frac{q\Delta t}{2m} \mathbf{E}^n.$$

源码文件：Source/Particles/Pusher/UpdateMomentumBoris.H 函数：UpdateMomentumBoris()

源码原文如下：

```
const amrex::ParticleReal econst = 0.5_prt*q*dt/m;

if (momentum_push_type == MomentumPushType::FirstHalf || momentum_push_type == MomentumPushType::Full) {
    // First half-push for E
    ux += econst*Ex;
    uy += econst*Ey;
    uz += econst*Ez;
}
// Compute temporary gamma factor
constexpr auto inv_c2 = PhysConst::inv_c2_v< amrex::ParticleReal>;
const amrex::ParticleReal inv_gamma = 1._prt/std::sqrt(1._prt + (ux*ux + uy*uy + uz*uz)*inv_c2);
// Magnetic rotation
amrex::ParticleReal tx = econst*inv_gamma*Bx;
amrex::ParticleReal ty = econst*inv_gamma*By;
amrex::ParticleReal tz = econst*inv_gamma*Bz;
if (momentum_push_type == MomentumPushType::FirstHalf || momentum_push_type == MomentumPushType::SecondHalf) {
    const amrex::ParticleReal tsq = tx*tx + ty*ty + tz*tz;
    const amrex::ParticleReal factor = (tsq > 0._prt) ? (std::sqrt(1._prt + tsq) - 1._prt) / tsq : 0.5_prt;
    tx *= factor;
    ty *= factor;
    tz *= factor;
}
const amrex::ParticleReal tsqi = 2._prt/(1._prt + tx*tx + ty*ty + tz*tz);
const amrex::ParticleReal sx = tx*tsqi;
const amrex::ParticleReal sy = ty*tsqi;
const amrex::ParticleReal sz = tz*tsqi;
const amrex::ParticleReal ux_p = ux + uy*tz - uz*ty;
const amrex::ParticleReal uy_p = uy + uz*tx - ux*tz;
const amrex::ParticleReal uz_p = uz + ux*ty - uy*tx;
// - Update momentum
ux += uy_p*sz - uz_p*sy;
```

```

uy += uz_p*sx - ux_p*sz;
uz += ux_p*sy - uy_p*sx;
if (momentum_push_type == MomentumPushType::SecondHalf || momentum_push_type == MomentumPushType::Full) {
// Second half-push for E
    ux += econst*Ex;
    uy += econst*Ey;
    uz += econst*Ez;
}

```

更新阶段	代码动作	公式对应
电场系数	$econst = 0.5 * q * dt / m$	$\frac{q \Delta t}{2m}$
第一半部	FirstHalf 或 Full 时做电半步	$\mathbf{u}^{n-1/2} \rightarrow \mathbf{u}^-$
旋转参数	计算 inv_gamma 和 full-step t	γ^-, t
分裂修正	FirstHalf/SecondHalf 时按半角关系重标定 t	使两次 half push 合成一次 Full 的磁旋转
磁旋转	交叉乘更新动量	$\mathbf{u}^- \rightarrow \mathbf{u}^+$
第二半部	SecondHalf 或 Full 时做第二个电半步	$\mathbf{u}^+ \rightarrow \mathbf{u}^{n+1/2}$

函数注释说明 FirstHalf 和 SecondHalf 可以分裂执行，并且连续执行应与一次 Full 更新数学等价。这正好服务于 WarpXEvolve.cpp 中把碰撞放在 momentum push 中间的路径。

这里有一个必须区分的实现细节：WarpX 的 half momentum push 不是简单把 $q \, dt \, B / (2m \gamma)$ 再乘 $1/2$ 。UpdateMomentumBoris() 的 half-push 分支使用

$$\frac{|t_{\text{half}}|}{|t_{\text{full}}|} = \frac{\sqrt{1 + |t_{\text{full}}|^2} - 1}{|t_{\text{full}}|^2}$$

重标定 tx,ty,tz。这来自 $\tan(\alpha/2)$ 与 $\tan(\alpha/4)$ 的半角关系，目的是让 FirstHalf 后接 SecondHalf 的组合仍对应 full Boris rotation，而不是两个朴素半磁旋转的近似拼接。

4.3 WarpX 如何选择 pusher

源码文件：Source/Particles/Pusher/PushSelector.H 函数：doParticleMomentumPush()

条件	选择
进入函数后	species 电荷乘 ion_lev，得到当前粒子的有效电荷。
启用 classical radiation reaction	进入 Boris 家族分支：通常调用 UpdateMomentumBorisWithRadiationReaction(); 若编译 QED 且同步分支中的 χ 不低于门限，则调用普通 UpdateMomentumBoris()。
ParticlePusherAlgo::Boris	进入 UpdateMomentumBoris()。
ParticlePusherAlgo::Vay	进入 UpdateMomentumVay()。
ParticlePusherAlgo::HigueraCary	进入 UpdateMomentumHigueraCary()。

这说明输入参数选择的不是一个高层“粒子模块”，而是每个粒子在 PushPX() device loop 内调用的单粒子 momentum update。下面继续展开 Vay 与 Higuera-Cary 的源码。

这一节背后的经典来源可以直接回到 Birdsall-Langdon 1985 第一分卷 4-3 到 4-5。那里把磁推进的核心先写成几何分裂：电场部分是半部 impulse，磁场部分是速度空间旋转；随后再把旋转压成 $t = \tan(\theta/2)$ 、 $s = 2t/(1+t^2)$ 、 $c = (1-t^2)/(1+t^2)$ 这组半角变量，并进一步给出向量 Boris 形式。对本章来说，这个来源有两个价值。第一，WarpX 的 Boris 更新不是孤立经验公式，而是这条“half-accel + rotation + half-accel”离散合同的现代实现。第二，Birdsall 在 4-5 里明确区分了 1d2v/1d3v 和真正的一维动力学，这正好解释了为什么即使空间维数较低，本章后面讨论的 mover 仍必须保留多速度分量与磁旋转结构。

Boris 1970 的原始历史位置需要单独标注边界：本书给出 J. P. Boris 的会议论文书目和 DTIC ADA023511 入口，但目前没有可逐页核对的会议论文全文。因此，本章的算法推导采用 Birdsall-Langdon 1985 的完整讲解，WarpX 的实现说明则回到 Source/Particles/Pusher/UpdateMomentumBoris.H；Boris 1970 不能作为已经逐页核查的直接公式依据。

物理上可以先记住：

- Boris：经典、鲁棒、磁场部分近似旋转，长期性质好。
- Vay：针对相对论漂移和 Lorentz 变换一致性问题设计，在 boosted frame 场景常用。
- Higuera-Cary：相对论粒子推进的结构保持改进，常用于减少高相对论问题中的系统误差。

正式误差比较必须回到对应论文和 benchmark；本章先把 WarpX 的实际实现讲清楚。

4.4 Vay pusher：相对论速度变换一致性的更新

源码文件：Source/Particles/Pusher/UpdateMomentumVay.H 函数：UpdateMomentumVay()

源码注释明确引用 Vay 2008 的公式 (9)-(13)，并说明 FirstHalf 与 SecondHalf 连续执行应等价于 Full：

```
/** \brief Push the particle's positions over one timestep,
 *   given the value of its momenta `ux`, `uy`, `uz`
 *   Note that UpdateMomentumVay algorithm can be splitted into FirstHalf and SecondHalf
 *   momentum updates. FirstHalf and SecondHalf updates are constructed so that
 *   performing FirstHalf followed by SecondHalf is mathematically
 *   equivalent to a single Full update.
 *   For more details, see formulas (9)-(13) in J.L.Vay, "Simulation of beams or plasmas crossing at
 *   relativistic velocity", Phys. Plasmas 15, 056701 (2008).
 */
AMREX_GPU_HOST_DEVICE AMREX_INLINE
void UpdateMomentumVay(
    amrex::ParticleReal& ux, amrex::ParticleReal& uy, amrex::ParticleReal& uz,
    const amrex::ParticleReal Ex, const amrex::ParticleReal Ey, const amrex::ParticleReal Ez,
    const amrex::ParticleReal Bx, const amrex::ParticleReal By, const amrex::ParticleReal Bz,
    const amrex::ParticleReal q, const amrex::ParticleReal m, const amrex::Real dt,
    MomentumPushType momentum_push_type)
{
    using namespace amrex::literals;

    // Constants
    const amrex::ParticleReal econst = q*dt/m * ((momentum_push_type == MomentumPushType::Full) ? 1.0_prt
    const amrex::ParticleReal bconst = 0.5_prt*q*dt/m;
    // Compute initial gamma
    const amrex::ParticleReal inv_gamma = 1._prt/std::sqrt(1._prt + (ux*ux + uy*uy + uz*uz)*PhysConst::inv
    // Get tau
    const amrex::ParticleReal taux = bconst*Bx;
    const amrex::ParticleReal tauy = bconst*By;
    const amrex::ParticleReal tauz = bconst*Bz;
    const amrex::ParticleReal tausq = taux*taux+tauy*tauy+tauz*tauz;
    // Get U', gamma'^2
    const amrex::ParticleReal uxpr = ux + econst*Ex + ((momentum_push_type == MomentumPushType::SecondHalf
    const amrex::ParticleReal uypr = uy + econst*Ey + ((momentum_push_type == MomentumPushType::SecondHalf
    const amrex::ParticleReal uzpr = uz + econst*Ez + ((momentum_push_type == MomentumPushType::SecondHalf

    if (momentum_push_type != MomentumPushType::FirstHalf) {
        // Get gamma'^2
        const amrex::ParticleReal gprsq = (1._prt + (uxpr*uxpr + uypr*uypr + uzpr*uzpr)*PhysConst::inv_c2_
        // Get u*
        const amrex::ParticleReal ust = (uxpr*taux + uypr*tauy + uzpr*tauz)*PhysConst::inv_c_v<amrex::Part
        // Get new gamma
        const amrex::ParticleReal sigma = gprsq-tausq;
        const amrex::ParticleReal gisq = 2._prt/(sigma + std::sqrt(sigma*sigma + 4._prt*(tausq + ust*ust))
        // Get t, s
        const amrex::ParticleReal bg = bconst*std::sqrt(gisq);
        const amrex::ParticleReal tx = bg*Bx;
```

```

const amrex::ParticleReal ty = bg*By;
const amrex::ParticleReal tz = bg*Bz;
const amrex::ParticleReal s = 1._prt/(1._prt+tausq*gisq);
// Get t.u'
const amrex::ParticleReal tu = tx*uxpr + ty*uypr + tz*uzpr;
// Get new U
ux = s*(uxpr+tx*tu+uypr*tz-uzpr*ty);
uy = s*(uypr+ty*tu+uzpr*tx-uxpr*tz);
uz = s*(uzpr+tz*tu+uxpr*ty-uypr*tx);
}
else {
ux = uxpr;
uy = uypr;
uz = uzpr;
}
}

```

Vay pusher 和 Boris 的主要差别在求解更新后的相对论因子这一步。Boris 使用旧的 γ^- 构造磁旋转；Vay 先构造 $\mathbf{u}'$ ，再通过

$$\sigma = \gamma'^2 - \tau^2, \quad \gamma_{\text{new}}^{-2} = \frac{2}{\sigma + \sqrt{\sigma^2 + 4(\tau^2 + u_*^2)}}$$

解析得到新的 γ 相关量。源码中的 `gisq` 就是这里的 γ_{new}^{-2} ，`bg=bconst*sqrt(gisq)` 对应 $(q\Delta t/2m)/\gamma_{\text{new}}$ 。这使磁旋转使用与更新后状态一致的相对论因子，适合 boosted-frame 或高速束流穿越问题。

`FirstHalf` 路径只返回 $\mathbf{u}'$ ，`SecondHalf` 路径不会再加入旧速度磁项。这和 Boris 文件中的分裂设计一致，服务于碰撞、外力或隐式/显式混合路径。

把这段源码重新放回 Vay 2008 原文，边界会更清楚。Vay 论文的第一性问题不是“再发明一个 relativistic mover”，而是对 relativistic crossing / boosted-frame 场景，离散 Lorentz force 必须尽量保住 electric 和 magnetic contributions 的 cancellation property。作者先用

$$\mathbf{E} + \mathbf{v} \times \mathbf{B} = 0$$

这个试金石说明 Boris 在一般非零 $\mathbf{E}$ 、 $\mathbf{B}$ 情况下会出现 spurious force，然后才提出新的 leapfrog velocity average。于是 WarpX 的 `UpdateMomentumVay.H` 最值得保留的历史定位不是“Boris 的另一个变体”，而是：它实现的是一条以 frame-change consistency 为目标的专门修正路线。

Vay 2008 后半段把这条历史论证链又压实了两次。第一，II.C 的两个单粒子测试并不是普通轨道展示，而是同一物理系统在 laboratory frame 和 moving frame 下都要和解析解一致的 frame-consistency test。常量 B_z 例子中，粒子以 $v_x=10^{-2}c$ 起步、 $\Delta t=10^{-2} \times 2\pi/\omega_c$ ，新 pusher 在实验室系和沿 $\hat{y}$ 方向 $\gamma_f=2$ 的 moving frame 里都贴住解析轨道；Boris 即使加上 $\tan(\omega_c \Delta t)/(\omega_c \Delta t)$ 修正，也会在 moving frame 中明显偏离，而且误差在 $\gamma_f=3$ 后迅速放大。常量 $E_x=1 \text{ kV/m}$ ，电子在实验室系初始静止、100 步 1 ns 更新的例子更直接：三种 mover 在实验室系里都正确，只有新 pusher 在 $\gamma_f=100$ 的 moving frame 中仍保持解析一致。于是这篇文献给本章的最硬证据不是“Vay 在某些 case 里更稳”，而是 Boris 的误差会在 frame change 后由次要项变成主导项。

第二，这篇论文并没有顺手给出一个通用 Maxwell solver。它在 III 节明确把场求解边界限定在 waves 和 retardation 可忽略、并且对每个 species 可以在共动系里近似取

$$v_z \gg v_x, v_y, \quad \frac{\partial}{\partial t} \approx v_z \frac{\partial}{\partial z}$$

的场景。于是 field side 被压成带 γ_z 拉伸的 Poisson 型求解，并近似保留 electrostatic、magnetostatic 以及沿主流向的 inductive effect。对 N 个 species，代价就是 N 次这类 Poisson solve。这条 bounded Darwin-lite explicit approximation 解释了为什么 IV 节的 LHC-like ultrarelativistic beam / electron-cloud 应用会特意选在 $\gamma \approx 16.5$ 的 moving frame 中做 first-principles PIC：在那里 beam 与 electron cloud 的 self-electric / self-magnetic cancellation 最强，最能放大 mover 的 frame-consistency 缺陷。文中报告 Boris 无论是否带 $\tan$ 修正，都会让 beam 和 electron 宏粒子以非物理速度丢失；只有新 pusher 才能恢复预期的 hose-like instability，并且给出和实验室系 quasistatic WARP calculation 一致的 vertical emittance growth rate 与 saturation level。因此，对 WarpX 而言，`UpdateMomentumVay.H` 的历史角色应理解成 relativistic beam-crossing / boosted-frame consistency repair，而不是一个与一般场求解器或一般 relativistic mover 等价并列的“备选算法”。

4.4.1 Vay Appendix A/B: 显式 γ 根与回旋半径边界

Vay 2008 的 Appendix A 给出了源码中 `gisq / gamma_new` 公式为什么可以显式计算。磁旋转中先定义

$$\mathbf{u}^{i+1} = s [\mathbf{u}' + (\mathbf{u}' \cdot \mathbf{t})\mathbf{t} + \mathbf{u}' \times \mathbf{t}], \quad s = \frac{1}{1+t^2}, \quad \mathbf{t} = \frac{\boldsymbol{\tau}}{\gamma^{i+1}},$$

其中 $\boldsymbol{\tau} = (q\Delta t/2m)\mathbf{B}$ 。对上式与 $\mathbf{u}$ 做点积，利用 $\gamma^2 = 1 + u^2/c^2$ ，并令

$$\gamma' = \sqrt{1 + u'^2/c^2}, \quad u^* = \frac{\mathbf{u}' \cdot \boldsymbol{\tau}}{c}, \quad \sigma = \gamma'^2 - \tau^2,$$

可把隐式的 relativistic factor 压成一个关于 γ^2 的二次方程：

$$\gamma^4 + (\tau^2 - \gamma'^2)\gamma^2 - \tau^2 - u^{*2} = 0.$$

只保留正的实根，得到

$$\gamma^{i+1} = \sqrt{\frac{\sigma + \sqrt{\sigma^2 + 4(\tau^2 + u^{*2})}}{2}}.$$

这解释了 `UpdateMomentumVay.H` 的实现顺序：先用 $\mathbf{u}'$ 和 $\boldsymbol{\tau}$ 构造标量不变量，再取正根，最后由 $\mathbf{t} = \boldsymbol{\tau}/\gamma$ 和 $s = 1/(1+t^2)$ 完成旋转。`gisq` 存的是 γ^{-2} ，因此 device loop 不需要迭代求解 γ 。这是 Appendix A 与当前 kernel 的直接公式桥接，不是普通 Boris 旋转中的经验系数。

Appendix B 则给出常磁场、 $\mathbf{E}=0$ 时的 gyroradius 边界：

$$\Delta\theta = 2 \arctan\left(\frac{\omega_c \Delta t}{2}\right), \quad \mathbf{v}^{i+1/2} = \left[1 + \left(\frac{\omega_c \Delta t}{2}\right)^2\right] \frac{\mathbf{v}^i + \mathbf{v}^{i+1}}{2},$$

从圆周几何得到

$$R = \frac{\|\mathbf{v}^{i+1/2}\|}{\omega_c} = \left[1 + \left(\frac{\omega_c \Delta t}{2}\right)^2\right]^{1/2} \frac{\|\mathbf{v}^i\|}{\omega_c}.$$

所以“Vay 在任意时间步都给出正确 gyroradius”必须加限定：若用于位置推进的半步速度满足 $\|\mathbf{v}^{i+1/2}\| = v_0$ ，则 $R = v_0/\omega_c$ ；若把整数时刻速度直接当作 $\mathbf{v}_0$ ，仍会出现与 Boris 类似的放大因子。pusher 的动量更新、半步速度定义和位置更新必须一起检查。

一个有用的最小验证是去掉自洽场、AMR 和 PML，只保留均匀磁场，然后同时比较三类量：离散相位、由相邻位置差构造的速度 proxy、以及由该 proxy 得到的 gyroradius。这样的比较能够检查“公式中的半步速度是否与实际位置更新相容”，也能把 Higuera–Cary 的相位行为与 Boris/Vay 分开观察；它不能证明输出文件中存在可直接读取的 half-step 速度，更不能代替论文图形的逐点复现。读者应把 Appendix B 当成一个关于时间层的判别题，而不是把任何圆形轨道都视作该附录结论的证明。

Vay 2008 与 Higuera–Cary 2017 在本章承担不同作用：前者说明为何需要对相对论速度变换作修正，后者说明如何在 Boris-like 结构中构造相对论不变量。两篇论文都应回到原文公式和本章给出的 kernel 对照阅读；只有在输入、观察量和容差明确相同的情况下，才可以讨论某个 WarpX 例子是否复现了其间的专门结论。

4.4.2 用推进器谱系约束 Vay 的结论范围

Vay–Godfrey 2014 review 的读者价值，不是替 WarpX 的 `UpdateMomentumVay.H` 背书，而是把 Boris、Lorentz-invariant pusher、场更新、current deposition、field gather、filtering 与数值稳定性放在同一条 PIC 离散链上。它提醒读者：推进行为不能只凭一条单粒子轨迹判断，场与源项怎样被离散、怎样被 gather，同样会决定相对论计算的误差结构。

因此第 4 章应按四层证据阅读：Vay 2008 解释 frame-consistency 这一原始算法目标；该综述给出推进器在完整 PIC 方法谱系中的位置；WarpX 源码说明本书采用的实现中 kernel 的变量和时间层；明确输入和观察量的算例才说明某个条件下实际测到了什么。任何一层都不能替代另一层，尤其不能把综述中的历史算法图或其他 PIC 程序的结果写成 WarpX 的验证结论。

本节的核心判断不依赖于资料整理方式：Vay 是为特定的相对论 frame-consistency 问题设计的推进器。选择它之前，仍要同时检查粒子推进、场更新、沉积和诊断路径，而不能只依据一个 mover 名称或一条单粒子轨道。

4.5 Higuera-Cary pusher: Boris-like 结构的相对论修正

源码文件：Source/Particles/Pusher/UpdateMomentumHigueraCary.H 函数：UpdateMomentumHigueraCary()

```
template <typename T>
AMREX_GPU_HOST_DEVICE AMREX_INLINE
void UpdateMomentumHigueraCary(
    T& ux, T& uy, T& uz,
    const T Ex, const T Ey, const T Ez,
    const T Bx, const T By, const T Bz,
    const T q, const T m, const amrex::Real dt )
{
    using namespace amrex::literals;

    // Constants
    const T qmt = 0.5_prt*q*dt/m;
    // Compute u_minus
    const T umx = ux + qmt*Ex;
    const T umy = uy + qmt*Ey;
    const T umz = uz + qmt*Ez;
    // Compute gamma squared of u_minus
    T gamma = 1._prt + (umx*umx + umy*umy + umz*umz)*PhysConst::inv_c2_v<T>;
    // Compute beta and betam squared
    const T betax = qmt*Bx;
    const T betay = qmt*By;
    const T betaz = qmt*Bz;
    const T betam = betax*betax + betay*betay + betaz*betaz;
    // Compute sigma
    const T sigma = gamma - betam;
    // Get u*
    const T ust = (umx*betax + umy*betay + umz*betaz)*PhysConst::inv_c_v<T>;
    // Get new gamma inverse
    gamma = 1._prt/std::sqrt(0.5_prt*(sigma + std::sqrt(sigma*sigma + 4._prt*(betam + ust*ust))) );
    // Compute t
    const T tx = gamma*betax;
    const T ty = gamma*betay;
    const T tz = gamma*betaz;
    // Compute s
    const T s = 1._prt/(1._prt+(tx*tx + ty*ty + tz*tz));
    // Compute um dot t
    const T umt = umx*tx + umy*ty + umz*tz;
    // Compute u_plus
    const T upx = s*( umx + umt*tx + umy*tz - umz*ty );
    const T upy = s*( umy + umt*ty + umz*tx - umx*tz );
    const T upz = s*( umz + umt*tz + umx*ty - umy*tx );
    // Get new u
    ux = upx + qmt*Ex + upy*tz - upz*ty;
    uy = upy + qmt*Ey + upz*tx - upx*tz;
    uz = upz + qmt*Ez + upx*ty - upy*tx;
}
```

前半段仍是电场半步：

$$\mathbf{u}^- = \mathbf{u}^{n-1/2} + \frac{q\Delta t}{2m}\mathbf{E}.$$

随后源码用 `beta=qmt*B` 和 `u_minus` 计算

$$\sigma = \gamma_-^2 - \beta^2, \quad u_* = \frac{\mathbf{u}^- \cdot \beta}{c},$$

并通过平方根表达式得到新的 `gamma`，这里变量名 `gamma` 在这一行之后实际保存的是 γ^{-1} 。`tx,ty,tz` 是 β/γ ，`s=1/(1+t^2)`。`u_plus` 的形式像 Boris 的磁旋转，但最后不是单纯加第二个电半步，而是

$$\mathbf{u}^{n+1/2} = \mathbf{u}^+ + \frac{q\Delta t}{2m} \mathbf{E} + \mathbf{u}^+ \times \mathbf{t}.$$

这个额外叉乘项是 Higuera-Cary 结构和 Boris 结构最容易混淆的地方。源码中 `upy*tz-upz*ty`、`upz*tx-upx*tz`、`upx*ty-upy*tx` 正是 $\mathbf{u}^+ \times \mathbf{t}$ 的三个分量。

把这段 kernel 放回 Higuera-Cary 2017 原文，WarpX 里这条算法线的真实边界会更清楚。那篇论文并不是在 Vay 2008 的 boosted-frame cancellation 问题上继续竞争，而是把 Boris、Vay 和新方法放到三个并列判据下比较： $\mathbf{E}=0$ 时的能量守恒、crossed $\mathbf{E}/\mathbf{B}$ 场下正确的 $\mathbf{E} \times \mathbf{B}$ drift、以及 phase-space volume preservation。作者的核心判断是：Boris 保住 volume 但 drift 不对；Vay 保住 drift 但不 volume-preserving；Higuera-Cary 则是三者中唯一同时保住 volume 与 $\mathbf{E} \times \mathbf{B}$ drift 的二阶 relativistic momentum integrator。因此，`UpdateMomentumHigueraCary.H` 最准确的历史定位不是“又一个 Boris-like 变体”，而是：在 Boris 的 rotation skeleton 上，把 centered average 和 γ 的 prescription 改写成一条双结构保持路线。

Higuera-Cary 2017 的 III-VI 节还把把这个判断压成了 WarpX 读者真正需要的两层证据。第一层是实现证据：新方法表面上仍是 implicit centered scheme，但最终可以像 Boris/Vay 一样显式实现，真正的实现分叉点几乎全部浓缩在 γ_{new} 的求法上。这正好解释了为什么 WarpX 源码里 `UpdateMomentumHigueraCary.H` 的外形与 Boris 非常接近，却在 `sigma`、`ust` 和新的 relativistic factor 路径上分叉。第二层是数值/几何证据：作者用 Poincare surface of section 而不是普通能量曲线来比较 practical timestep 下的轨道拓扑。小时间步时三种方法都能给出嵌套曲线；但在更接近实际模拟的 $\Delta t=1/10$ 下，Vay 会出现 resonance island 和不同轨道 section 交叉，而 Boris 与 Higuera-Cary 仍保持正确的 phase-space topology。对本章来说，这意味着 Higuera-Cary 的价值判断标准不是 Vay 2008 那种 frame-change consistency，而是 geometric/topological preservation at practical timestep。

如果进一步把论文公式和 WarpX kernel 逐式对位，这条关系会更直观。Higuera-Cary 论文先定义

$$\vec{\epsilon} = \frac{q\Delta t}{2m} \vec{E}, \quad \vec{\beta} = \frac{q\Delta t}{2m} \vec{B}, \quad \vec{u}_- = \vec{u}_i + \vec{\epsilon},$$

而源码里的 `qmt`、`umx/umy/umz`、`betax/betay/betaz` 正是这些量。随后论文显式求解

$$\gamma_{new}^2 = \frac{1}{2} \left(\gamma_-^2 - \beta^2 + \sqrt{(\gamma_-^2 - \beta^2)^2 + 4(\beta^2 + |\vec{\beta} \cdot \vec{u}_-|^2)} \right),$$

WarpX 则不先存 γ_{new}^2 ，而是直接用 `sigma = gamma - betam`、`ust = (u_- \cdot beta)/c` 和下一行的平方根公式，把变量 `gamma` 改写成 γ_{new}^{-1} 。这一点很关键：代码里的 `gamma` 在函数中段被重载了，前半段是 γ_-^2 ，后半段则变成 rotation 要消费的 inverse relativistic factor。之后 `tx/ty/tz`、`s`、`umt`、`upx/upy/upz` 这条链，就是论文 Boris-like rotation equation

$$\vec{u}_+ - \vec{u}_- = (\vec{u}_+ + \vec{u}_-) \times \frac{\vec{\beta}}{\gamma_{new}}$$

的直接实现。因此，`UpdateMomentumHigueraCary.H` 最本质的实现差异可以压成一句话：它在 Boris 的 rotation skeleton 上，仅通过改写 γ prescription，就把 volume-preserving 与 $\mathbf{E} \times \mathbf{B}$ drift preservation 这两条性质同时保住了。

论文第 V 节的 Jacobian 证明也让这一点不再停留在口头层。作者证明新 integrator 的前后半步 Jacobian determinant 互为倒数，所以一步更新的总体 Jacobian 恰好等于 1；而 Vay 的 Jacobian 一般写成 $J(\mathbf{x}_i, \mathbf{u}_i)/J(\mathbf{x}_i, \mathbf{u}_f)$ 这样的比值，通常不会化成 1。这正是后面数值例子里 Vay 在 practical timestep 下出现 resonance island 和轨道交叉，而 Higuera-Cary 仍保持 phase-space topology 的几何根源。

如果把这段证明再往下压一层，最关键的中间对象其实是 $\mathbf{I} \cdot \Omega$ 。Higuera-Cary 论文先把后半步对 $\vec{u}_{new}$ 的 Jacobian 写成“Boris-like rotation 主干 $\mathbf{I} \cdot \Omega$ + 一个 rank-one correction”，其中 $\Omega \cdot V = (\beta \times V)/\gamma_{new}$ 。这一步意味着作者不是直接对整个复杂映射硬算 determinant，而是先把旋转骨架和 relativistic correction 拆开。随后利用 determinant lemma，把后半步体积变化写成

$$J_{f,new} = \det(I - \Omega) \times (\text{scalar correction}),$$

再经过代数整理压成

$$J_{f,new} = 1 + \frac{\beta^2 + (\vec{\beta} \cdot \vec{u}_{new})^2}{\gamma_{new}^4}.$$

前半步可得同一形式的 determinant, 因此真正的 volume-preserving 不是“每个子步都单独等于 1”, 而是前后半步 Jacobian determinant 互为逆, 整步更新的 Jacobian 才严格等于 1。这一点很重要, 因为它把 UpdateMomentumHigueraCary.H 的稳定性来源从经验判断提升成了明确的 Jacobian 结构断言。

这里还要额外提醒一个记号陷阱: 论文里 $J_{\{f,new\}}$ 和 $J_{\{i,new\}}$ 最后都被压成相同的显式标量函数, 但这不代表它们是“同一个 Jacobian”。它们对应的是后半步和前半步那两条相反方向映射上的 determinant, 因此恰恰是因为它们处在 reciprocal 位置, 整步 Jacobian 才会严格回到 1。同样, 论文对 Vay 的结论也不是“任何情况下都会立刻出现 attractor/repeller”。作者保留了一个例外边界: 若磁场在时空上恒定, $J(x_i, u_i)/J(x_i, u_f)$ 这串比值会 telescoping, 再结合 $J(x, u)$ 在有界区域里的有界性, 不能直接推出灾难性体积失真。真正的问题是它缺少一般性的 volume-preservation, 因此在 practical timestep 和更复杂轨道拓扑下更容易暴露出 resonance-island 与 trajectory-crossing 这类非物理后果。

这组 regression 和文献的配对仍需保守表述。Examples/Tests/particle_pusher 提供 force-free Higuera-Cary 强断言; Poincare 合同则验证 $x=0, p_x>0$ 截面、 I_y 顺序和解析 quartic reference. topology classifier 同时保留时间顺序和相空间中心角顺序; 在 32^3 、2201-frame 长轨道上, 后三种 pusher 的角排序候选均无自交或轨道间交叉, 说明原先时间折线的交叉计数是连接顺序伪影, 而不是物理 resonance-island 证据。14-species dense family 与 64^3 $p_y=1.6/1.8$ control 进一步显示 Vay 窗口漂移约 $6.5e-2$, 控制组约 $1e-3$, 但该 resonance-sensitive screen 仍不是 two-fold island 或 trajectory-crossing topology proof。

这些短轨道、长轨道、密集 p_y family、resonance screen 和 resolution screen 给出的证据等级应当分开理解: 短轨道采样不足; 长轨道的 invariant/reference 与 angular-order candidate 可以通过, 但 topology 仍需要人工复核; 密集族的 resonance-sensitive screen 可以通过, 但解析 reference curve 和跨 pusher 的特征尚未形成完整拓扑证明。因此, 最强可写结论是“invariant 与局部 resonance-sensitive screen 已建立, 论文等价的 topology gate 尚未启用”。

4.6 从 MultiParticleContainer 到 PhysicalParticleContainer

源码文件: Source/Evolve/WarpXEvolve.cpp 函数: WarpX::PushParticlesandDeposit()

它选择 current 字段名后调用 mypc->Evolve(...).

mypc 是 MultiParticleContainer。源码文件: Source/Particles/MultiParticleContainer.cpp 函数: MultiParticleContainer::

调度阶段	操作
开始	不跳过沉积时清零 current_fp/current_buf/rho_fp/rho_buf。
implicit 分支	处理隐式 solver 相关源项和 mass matrix 的额外清零逻辑。
容器遍历	调用每个 pc->Evolve(...).

这层只负责多物种调度。真正的单 species 粒子推进在 PhysicalParticleContainer::Evolve()。

但在继续进入 PushPX() 之前, 还要先看清容器层次和属性系统, 否则后面很多变量名会被误读。

Source/Particles/WarpXParticleContainer.H 中的 PIdx 定义编译期属性表。它规定每个粒子天生就有:

- 位置分量 x/y/z 或非笛卡尔等价量;
- 权重 w;
- proper velocity ux/uy/uz;
- 在 RZ/球坐标几何下额外的 theta/phi。

而 IntIdx::nattribs 默认是 0, 这意味着 WarpX 的整数粒子属性默认都不是编译期内建, 而是后续按需动态添加。

顶层类层次由 Source/Particles/MultiParticleContainer.cpp 与各容器头文件共同决定:


```
MultiParticleContainer
-> WarpXParticleContainer
    -> PhysicalParticleContainer
        -> PhotonParticleContainer
        -> RigidInjectedParticleContainer
    -> LaserParticleContainer
```

这里几类容器的物理职责不同：

- `WarpXParticleContainer` 是统一基类，提供粒子 SoA 骨架、gather/deposition/push 的共用接口；
- `PhysicalParticleContainer` 承担普通带质量 species 的主要物理路径；
- `PhotonParticleContainer` 继承 `PhysicalParticleContainer`，但其 `DepositCharge()` 和 `DepositCurrent()` 直接空实现，因此它保留很多粒子基础设施，却不承担带电沉积；定义见 `Source/Particles/PhotonParticleContainer.H`；
- `LaserParticleContainer` 则直接从 `WarpXParticleContainer` 继承，因为激光天线粒子只需要 prescribed motion 和 current deposition，不需要普通 `FieldGather`；定义见 `Source/Particles/LaserParticleContainer.H`。

这层分工直接决定了粒子属性的来源。`Source/Particles/PhysicalParticleContainer.cpp` 中的 `PhysicalParticleContainer` 构造函数会按模块开关注册第一批 runtime attributes：

- QED quantum synchrotron 时加 `opticalDepthQSR`；
- Breit-Wheeler 时加 `opticalDepthBW`；
- `addRealAttributes` / `addIntegerAttributes` 时加入用户 parser 驱动属性；
- `save_previous_position` 时加 `prev_x/prev_y/prev_z`。

电离模块的 `InitIonizationModule()` 稍后又会动态补上 `ionizationLevel`。因此 `WarpX` 的粒子属性系统不是一个静态结构体，而是：

1. 编译期 builtin real: `x/y/z/w/ux/uy/uz/...`；
2. 构造期或模块初始化期动态加入的持久物理状态：如 `opticalDepthQSR`、`opticalDepthBW`、`prev_*`、`ionizationLevel`；
3. 更晚才按算法路径加入的临时缓存属性。

最典型的临时属性来自 implicit solver。`Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp` 中的 `ImplicitSolver::CreateP` 会统一给粒子容器补加：

- `x_n/y_n/z_n`
- `ux_n/uy_n/uz_n`
- 如果启用 particle suborbits，再加 `nsuborbits`

而且都用 `comm = 0` 注册，所以这些量既不参与通信，也不写入 checkpoint。随后 `Source/FieldSolver/ImplicitSolvers/WarpXImplicitOps.cpp` 的 `step-start` 初始化会把当前位置和动量快照写入 `x_n` 和 `ux_n` 这组缓存，并把 `nsuborbits` 先置成 1。它们的角色不是长期物理属性，而是 implicit 时间推进器本步用的局部状态。

`WarpXParticleContainer::AddNParticles()` 进一步说明了这套系统怎样落地。`Source/Particles/WarpXParticleContainer.cpp` 先写 builtin `x/y/z/w/ux/uy/uz`，再写调用者显式提供的 runtime real/int，最后调用 `DefaultInitializeRuntimeAttributes()` 给剩余 runtime attrs 自动补默认值。于是：

- `opticalDepthQSR/BW` 可以由 QED engine 随机初始化；
- `ionizationLevel` 可以统一设成 `ionization_initial_level`；
- 用户 parser 属性可以按 `attribute.<name>(x,y,z,ux,uy,uz,t)` 自动求值。

也就是说，进入 `PushPX()` 之前，`WarpX` 已经把“粒子是什么物种、当前带哪些附加物理状态、哪些只是临时算法缓存”这三件事分清了。后面看到 `ux`、`ux_n`、`ionizationLevel`、`opticalDepthQSR` 这些名字时，必须先回到这里判断它们属于哪一层状态。

4.7 PhysicalParticleContainer::Evolve() 的 tile loop

本章最重要的入口是 `Source/Particles/PhysicalParticleContainer.cpp` 中的 `PhysicalParticleContainer::Evolve()`。它把一个 species 的粒子按 tile 遍历，并把沉积、gather、push、buffer、隐式路径和 load-balance cost 放在一个局部循环里。

核心顺序是：

tile-loop 阶段	操作	含义
准备	取得 <code>Efield_aux</code> 和 <code>Bfield_aux</code>	粒子 gather 使用 auxiliary fields。
条件判断	判断是否沉积 charge/current、是否 split particles	<code>skip_deposition</code> 和 <code>do_not_deposit</code> 会关掉沉积。

tile-loop 阶段	操作	含义
AMR 分区	遍历 tile, 必要时按 AMR buffer 分区粒子	fine/coarse gather 和 deposit 的粒子集合可能不同。
push 前	沉积 rho component 0	旧时间层电荷, 通常对应 ρ^n 。
fine patch	粒子调用 PushPX()	gather fine fields 并推进粒子。
buffer/coarse	粒子调用 PushPX()	AMR 边界附近可从 coarse auxiliary fields gather。
push 后	沉积 current	显式路径 <code>relative_time=-0.5*dt</code> , 对应 $\mathbf{J}^{n+1/2}$ 。
新时间层	沉积 rho component 1	新时间层电荷, 通常对应 ρ^{n+1} 。
可选后处理	particle splitting	subcycling 时避免 coarse level 重复沉积。

这段源码说明, 真实粒子推进不是“先推所有粒子, 再单独沉积”。WarpX 为了 AMR、缓存局部性、GPU/CPU 并行和时间层一致性, 在 tile 内完成 gather/push/deposit 的组合。

4.8 PushPX(): gather 和 push 的融合 kernel

源码文件: Source/Particles/PhysicalParticleContainer.cpp 函数: PhysicalParticleContainer::PushPX()

它是真正进入单粒子并行循环的地方。

核心源码原文如下, 省略了 QED 宏分支和部分属性准备:

```
amrex::ParallelFor(
    TypeList<CompileTimeOptions<no_exteb,has_exteb>, CompileTimeOptions<no_qed ,has_qed>>{},
    {exteb_runtime_flag, qed_runtime_flag},
    np_to_push,
    [=] AMREX_GPU_DEVICE (long ip, auto exteb_control, auto qed_control)
{
    amrex::ParticleReal xp, yp, zp;
    getPosition(ip, xp, yp, zp);

    amrex::ParticleReal Exp = Ex_external_particle;
    amrex::ParticleReal Eyp = Ey_external_particle;
    amrex::ParticleReal Ezp = Ez_external_particle;
    amrex::ParticleReal Bxp = Bx_external_particle;
    amrex::ParticleReal Byp = By_external_particle;
    amrex::ParticleReal Bzp = Bz_external_particle;

    if (gather_fields) {
        // first gather E and B to the particle positions
        doGatherShapeN(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
            ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
            ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
            dinv, xyzmin, lo, n_rz_azimuthal_modes,
            nox, galerkin_interpolation);
    }

    if constexpr (exteb_control == has_exteb) {
        getExternalEB(ip, Exp, Eyp, Ezp, Bxp, Byp, Bzp);
    }

    scaleFields(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp);

    doParticleMomentumPush<0>(ux[ip], uy[ip], uz[ip],
        Exp, Eyp, Ezp, Bxp, Byp, Bzp,
        ion_lev ? ion_lev[ip] : 1,
```

```

        mass, q, pusher_algo, do_crr,
        dt, momentum_push_type);

    if (position_push_type == PositionPushType::Full) {
        UpdatePosition(xp, yp, zp, ux[ip], uy[ip], uz[ip], dt, mass);
        setPosition(ip, xp, yp, zp);
    }
});

```

关键步骤:

kernel 阶段	操作
入口准备	检查 gather level、构造 gather box, 并按 ngEB 扩展 guard cells。
属性准备	准备粒子位置访问器、外场、动量数组、ionization level、旧位置缓存, 以及 pusher/RR/QED 选项。
并行分派	启动带 compile-time option 的 amrex::ParallelFor。
场构造	读取粒子位置, 调用 doGatherShapeN(); 随后叠加每粒子外场 callback, 再应用 scaleFields()。
动量更新	调用 doParticleMomentumPush() 更新动量。
位置更新	若 PositionPushType::Full, 调用 UpdatePosition() 更新位置。

这给出了 WarpX 显式粒子推进的真实顺序:

```

for particle in tile:
    read x^n and u^(n-1/2)
    gather E/B at x^n
    add external particle fields and apply field scaling
    push momentum to u^(n+1/2)
    push position to x^(n+1)

```

随后 PhysicalParticleContainer::Evolve() 沉积半步电流与新时间层电荷。

4.9 doGatherShapeN(): 从网格场到粒子场

PushPX() 在调用 pusher 前先调用 doGatherShapeN()。这一步的物理含义是把网格上的 Yee/PSATD 场插值到粒子位置:

$$\mathbf{E}_p = \sum_i \mathbf{E}_i S_i(\mathbf{x}_p), \quad \mathbf{B}_p = \sum_i \mathbf{B}_i S_i(\mathbf{x}_p).$$

但 WarpX 不能只用一个标量形函数, 因为 $E_x, E_y, E_z, B_x, B_y, B_z$ 在交错网格上的中心位置不同; 同时 Galerkin 插值会让某些分量使用低一阶形函数。运行时入口在 `../warpx/Source/Particles/Gather/FieldGather.H`:

```

void doGatherShapeN (const amrex::ParticleReal xp,
                    const amrex::ParticleReal yp,
                    const amrex::ParticleReal zp,
                    amrex::ParticleReal& Exp,
                    amrex::ParticleReal& Eyp,
                    amrex::ParticleReal& Ezp,
                    amrex::ParticleReal& Bxp,
                    amrex::ParticleReal& Byp,
                    amrex::ParticleReal& Bzp,
                    amrex::Array4<amrex::Real const> const& ex_arr,
                    amrex::Array4<amrex::Real const> const& ey_arr,
                    amrex::Array4<amrex::Real const> const& ez_arr,
                    amrex::Array4<amrex::Real const> const& bx_arr,

```

```

        amrex::Array4<amrex::Real const> const& by_arr,
        amrex::Array4<amrex::Real const> const& bz_arr,
        const amrex::IndexType ex_type,
        const amrex::IndexType ey_type,
        const amrex::IndexType ez_type,
        const amrex::IndexType bx_type,
        const amrex::IndexType by_type,
        const amrex::IndexType bz_type,
        const amrex::XDim3 & dinv,
        const amrex::XDim3 & xyzmin,
        const amrex::Dim3& lo,
        const int n_rz_azimuthal_modes,
        const int nox,
        const bool galerkin_interpolation)
{
    if (galerkin_interpolation) {
        if (nox == 1) {
            doGatherShapeN<1,1>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 2) {
            doGatherShapeN<2,1>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 3) {
            doGatherShapeN<3,1>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 4) {
            doGatherShapeN<4,1>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        }
    } else {
        if (nox == 1) {
            doGatherShapeN<1,0>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 2) {
            doGatherShapeN<2,0>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 3) {
            doGatherShapeN<3,0>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 4) {
            doGatherShapeN<4,0>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,

```

```

        ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
        dinv, xyzmin, lo, n_rz_azimuthal_modes);
    }
}

```

这里的 `nox` 不是运行时循环里的 `shape` 阶数变量，而是被转成模板参数 `depos_order`。这样 GPU kernel 内部可以用 `if constexpr` 展开阶数，避免每个粒子再做阶数分支。`galerkin_interpolation` 同理变成第二个模板参数，后面直接影响数组长度 `depos_order + 1 - galerkin_interpolation`。

模板主体开头在 `../warpX/Source/Particles/Gather/FieldGather.H`。下面只列 `x` 方向；`y/z` 方向同构，但按各自场分量的 `staggering` 选择 `node` 或 `cell`：

```

template <int depos_order, int galerkin_interpolation>
AMREX_GPU_HOST_DEVICE AMREX_FORCE_INLINE
void doGatherShapeN ([[maybe_unused]] const amrex::ParticleReal xp,
                    [[maybe_unused]] const amrex::ParticleReal yp,
                    [[maybe_unused]] const amrex::ParticleReal zp,
                    amrex::ParticleReal& Exp,
                    amrex::ParticleReal& Eyp,
                    amrex::ParticleReal& Ezp,
                    amrex::ParticleReal& Bxp,
                    amrex::ParticleReal& Byp,
                    amrex::ParticleReal& Bzp,
                    amrex::Array4<amrex::Real const> const& ex_arr,
                    amrex::Array4<amrex::Real const> const& ey_arr,
                    amrex::Array4<amrex::Real const> const& ez_arr,
                    amrex::Array4<amrex::Real const> const& bx_arr,
                    amrex::Array4<amrex::Real const> const& by_arr,
                    amrex::Array4<amrex::Real const> const& bz_arr,
                    const amrex::IndexType ex_type,
                    const amrex::IndexType ey_type,
                    const amrex::IndexType ez_type,
                    const amrex::IndexType bx_type,
                    const amrex::IndexType by_type,
                    const amrex::IndexType bz_type,
                    const amrex::XDim3 & dinv,
                    const amrex::XDim3 & xyzmin,
                    const amrex::Dim3& lo,
                    [[maybe_unused]] const int n_rz_azimuthal_modes)
{
    using namespace amrex;

    constexpr int NODE = amrex::IndexType::NODE;
    constexpr int CELL = amrex::IndexType::CELL;

    Compute_shape_factor< depos_order > const compute_shape_factor;
    Compute_shape_factor<depos_order - galerkin_interpolation > const compute_shape_factor_galerkin;

    #if !defined(WARPX_DIM_1D_Z)
        const amrex::Real x = (xp - xyzmin.x)*dinv.x;

        amrex::Real sx_node[depos_order + 1] = {0._rt};
        amrex::Real sx_cell[depos_order + 1] = {0._rt};
        amrex::Real sx_node_galerkin[depos_order + 1 - galerkin_interpolation] = {0._rt};
        amrex::Real sx_cell_galerkin[depos_order + 1 - galerkin_interpolation] = {0._rt};

        int j_node = 0;
    #endif
}

```

```

int j_cell = 0;
int j_node_v = 0;
int j_cell_v = 0;
if ((ey_type[0] == NODE) || (ez_type[0] == NODE) || (bx_type[0] == NODE)) {
    j_node = compute_shape_factor(sx_node, x);
}
if ((ey_type[0] == CELL) || (ez_type[0] == CELL) || (bx_type[0] == CELL)) {
    j_cell = compute_shape_factor(sx_cell, x - 0.5_rt);
}
if ((ex_type[0] == NODE) || (by_type[0] == NODE) || (bz_type[0] == NODE)) {
    j_node_v = compute_shape_factor_galerkin(sx_node_galerkin, x);
}
if ((ex_type[0] == CELL) || (by_type[0] == CELL) || (bz_type[0] == CELL)) {
    j_cell_v = compute_shape_factor_galerkin(sx_cell_galerkin, x - 0.5_rt);
}
const amrex::Real (&sx_ex)[depos_order + 1 - galerkin_interpolation] = ((ex_type[0] == NODE) ? sx_node
const amrex::Real (&sx_ey)[depos_order + 1] = ((ey_type[0] == NODE) ? sx_node : sx_cell
const amrex::Real (&sx_ez)[depos_order + 1] = ((ez_type[0] == NODE) ? sx_node : sx_cell
const amrex::Real (&sx_bx)[depos_order + 1] = ((bx_type[0] == NODE) ? sx_node : sx_cell
const amrex::Real (&sx_by)[depos_order + 1 - galerkin_interpolation] = ((by_type[0] == NODE) ? sx_node
const amrex::Real (&sx_bz)[depos_order + 1 - galerkin_interpolation] = ((bz_type[0] == NODE) ? sx_node
int const j_ex = ((ex_type[0] == NODE) ? j_node_v : j_cell_v);
int const j_ey = ((ey_type[0] == NODE) ? j_node : j_cell );
int const j_ez = ((ez_type[0] == NODE) ? j_node : j_cell );
int const j_bx = ((bx_type[0] == NODE) ? j_node : j_cell );
int const j_by = ((by_type[0] == NODE) ? j_node_v : j_cell_v);
int const j_bz = ((bz_type[0] == NODE) ? j_node_v : j_cell_v);
#endif

```

这段源码有三个关键点。

1. $x = (x_p - xyzmin.x) * d_{inv}.x$ 把物理坐标变成网格坐标。后面形函数都在无量纲网格坐标上计算。
2. x 和 $x - 0.5_{rt}$ 分别对应 node-centered 和 cell-centered 自由度。也就是说，场分量的 staggered center 不是后处理标签，而是直接改变粒子看到的插值权重。
3. Galerkin 路径给 ex/by/bz 使用 compute_shape_factor_galerkin，阶数是 depos_order - 1；非 Galerkin 时第二个模板参数为 0，因此阶数不变。

以 2D XZ 编译为例，真正累加网格场的源码在 ../warpx/Source/Particles/Gather/FieldGather.H:

```

#elif defined(WARPX_DIM_XZ)
    // Gather field on particle Eyp from field on grid ey_arr
    for (int iz=0; iz<=depos_order; iz++){
        for (int ix=0; ix<=depos_order; ix++){
            Eyp += sx_ey[ix]*sz_ey[iz]*
            ey_arr(lo.x+j_ey+ix, lo.y+l_ey+iz, 0, 0);
        }
    }
    // Gather field on particle Exp from field on grid ex_arr
    // Gather field on particle Bzp from field on grid bz_arr
    for (int iz=0; iz<=depos_order; iz++){
        for (int ix=0; ix<=depos_order-galerkin_interpolation; ix++){
            Exp += sx_ex[ix]*sz_ex[iz]*
            ex_arr(lo.x+j_ex+ix, lo.y+l_ex+iz, 0, 0);
            Bzp += sx_bz[ix]*sz_bz[iz]*
            bz_arr(lo.x+j_bz+ix, lo.y+l_bz+iz, 0, 0);
        }
    }
    // Gather field on particle Ezp from field on grid ez_arr

```

```

// Gather field on particle Bxp from field on grid bx_arr
for (int iz=0; iz<=depos_order-galerkin_interpolation; iz++){
    for (int ix=0; ix<=depos_order; ix++){
        Ezp += sx_ez[ix]*sz_ez[iz]*
            ez_arr(lo.x+j_ez+ix, lo.y+l_ez+iz, 0, 0);
        Bxp += sx_bx[ix]*sz_bx[iz]*
            bx_arr(lo.x+j_bx+ix, lo.y+l_bx+iz, 0, 0);
    }
}

// Gather field on particle Byp from field on grid by_arr
for (int iz=0; iz<=depos_order-galerkin_interpolation; iz++){
    for (int ix=0; ix<=depos_order-galerkin_interpolation; ix++){
        Byp += sx_by[ix]*sz_by[iz]*
            by_arr(lo.x+j_by+ix, lo.y+l_by+iz, 0, 0);
    }
}

```

公式上，第一段就是

$$E_{y,p} = \sum_{i=0}^p \sum_{k=0}^p S_{E_y,i}^{(x)} S_{E_y,k}^{(z)} E_y(j_{E_y} + i, l_{E_y} + k).$$

Exp/Bzp、Ezp/Bxp、Byp 的循环上限不同，是因为 Galerkin 插值会沿某些方向把 shape order 从 p 降到 $p-1$ 。这不是任意优化，而是和离散 Maxwell operator、field staggering 与能量/电荷性质匹配的插值选择。

RZ 编译下, gather 先得到柱坐标分量, 再转回笛卡尔粒子 pusher 需要的 E_x, E_y, B_x, B_y 。关键转换在 ../warpx/Source/Particles/Gather/FieldGa

```

amrex::Real costheta;
amrex::Real sintheta;
if (rp > 0.) {
    costheta = xp/rp;
    sintheta = yp/rp;
} else {
    costheta = 1.;
    sintheta = 0.;
}
const Complex xy0 = Complex{costheta, -sintheta};
Complex xy = xy0;

for (int imode=1 ; imode < n_rz_azimuthal_modes ; imode++) {

    // Gather field on particle Ethetap from field on grid ey_arr
    for (int iz=0; iz<=depos_order; iz++){
        for (int ix=0; ix<=depos_order; ix++){
            const amrex::Real dEy = (+ ey_arr(lo.x+j_ey+ix, lo.y+l_ey+iz, 0, 2*imode-1)*xy.real()
                - ey_arr(lo.x+j_ey+ix, lo.y+l_ey+iz, 0, 2*imode)*xy.imag());
            Ethetap += sx_ey[ix]*sz_ey[iz]*dEy;
        }
    }

    // Gather field on particle Erp from field on grid ex_arr
    // Gather field on particle Bzp from field on grid bz_arr
    for (int iz=0; iz<=depos_order; iz++){
        for (int ix=0; ix<=depos_order-galerkin_interpolation; ix++){
            const amrex::Real dEx = (+ ex_arr(lo.x+j_ex+ix, lo.y+l_ex+iz, 0, 2*imode-1)*xy.real()
                - ex_arr(lo.x+j_ex+ix, lo.y+l_ex+iz, 0, 2*imode)*xy.imag());
            Erp += sx_ex[ix]*sz_ex[iz]*dEx;
            const amrex::Real dBz = (+ bz_arr(lo.x+j_bz+ix, lo.y+l_bz+iz, 0, 2*imode-1)*xy.real()

```

```

        - bz_arr(lo.x+j_bz+ix, lo.y+l_bz+iz, 0, 2*imode)*xy.imag());
    Bzp += sx_bz[ix]*sz_bz[iz]*dBz;
}
}
// Gather field on particle Ezp from field on grid ez_arr
// Gather field on particle Brp from field on grid bx_arr
for (int iz=0; iz<=depos_order-galerkin_interpolation; iz++){
    for (int ix=0; ix<=depos_order; ix++){
        const amrex::Real dEz = (+ ez_arr(lo.x+j_ez+ix, lo.y+l_ez+iz, 0, 2*imode-1)*xy.real()
                                - ez_arr(lo.x+j_ez+ix, lo.y+l_ez+iz, 0, 2*imode)*xy.imag());
        Ezp += sx_ez[ix]*sz_ez[iz]*dEz;
        const amrex::Real dBx = (+ bx_arr(lo.x+j_bx+ix, lo.y+l_bx+iz, 0, 2*imode-1)*xy.real()
                                - bx_arr(lo.x+j_bx+ix, lo.y+l_bx+iz, 0, 2*imode)*xy.imag());
        Brp += sx_bx[ix]*sz_bx[iz]*dBx;
    }
}
// Gather field on particle Bthetap from field on grid by_arr
for (int iz=0; iz<=depos_order-galerkin_interpolation; iz++){
    for (int ix=0; ix<=depos_order-galerkin_interpolation; ix++){
        const amrex::Real dBy = (+ by_arr(lo.x+j_by+ix, lo.y+l_by+iz, 0, 2*imode-1)*xy.real()
                                - by_arr(lo.x+j_by+ix, lo.y+l_by+iz, 0, 2*imode)*xy.imag());
        Bthetap += sx_by[ix]*sz_by[iz]*dBy;
    }
}
xy = xy*xy0;
}

// Convert Erp and Ethetap to Ex and Ey
Exp += costheta*Erp - sintheta*Ethetap;
Eyp += costheta*Ethetap + sintheta*Erp;
Bxp += costheta*Brp - sintheta*Bthetap;
Byp += costheta*Bthetap + sintheta*Brp;

```

所以 PushPX() 里的 pusher 始终看见 Cartesian-like 的 Exp, Eyp, Ezp, Bxp, Byp, Bzp. RZ 的 Fourier mode 和柱坐标细节被 gather 层封装, 但不是消失: 它们决定粒子场的实际插值。

把这层再向下看, WarpX 的 gather 还不是“只剩 3D/XZ/RZ 三个分支”。FieldGather.H 后面还继续给出了:

- 1D_Z: 只沿 z 一个方向做 shape 累加, 但仍保留 Ez/Bx/By 这组可能走 p-1 的 Galerkin 降阶路径;
- RCYLINDER: 先 gather Fr/Ftheta/Fz, 再按粒子极角转回 Fx/Fy/Fz;
- RSPHERE: 先 gather Er/Etheta/Ephi, 再按球坐标角转回 Ex/Ey/Ez;
- 3D: 完整保留 Ex/Ey/Ez/Bx/By/Bz 六个分量各自不同的循环上限, 因此 Galerkin 不是一个全局“降一阶开关”, 而是对特定分量沿特定方向降阶。

这也能更准确地解释官方文档里 energy-conserving 与 momentum-conserving gather 的差别。doGatherShapeN() 本身并不读一个“gather family”枚举, 它真正消费的是传进来的 IndexType。因此两族 gather 的代码差异不在这个 wrapper 里, 而在更早的 field centering 上:

- energy-conserving: 直接从原来的 staggered 或 nodal 网格 gather;
- momentum-conserving: 先把场中心化到 node, 再把 nodal IndexType 送进 doGatherShapeN()。

这也解释了为什么 collocated grid 下两者等价, 而 hybrid grid 下必须强制 momentum-conserving。一旦所有分量都已经是 nodal/collocated, doGatherShapeN() 看到的就只是同一组 NODE/CELL 组合。

如果只停在这层实现差异, 这个问题还是讲浅了。Birdsall-Langdon 第一分卷 Chapter 10 给出的更硬边界是: energy-conserving 和 momentum-conserving 从来都不是同一套离散系统上“换一种 gather 插值”这么简单, 而是两套不同的守恒合同。momentum-conserving 路线保留的是更自然的零总力/粒子-粒子相互作用对称结构, 但它一般并不存在严格守恒的总能量; energy-conserving 路线则把

$$W_E = \frac{V_c}{2} \sum_j \rho_j \phi_j$$

当成第一性对象，再把粒子受力理解成

$$\mathbf{F}_i = -\frac{\partial W_E}{\partial \mathbf{x}_i},$$

也就是不再先“在网格上差分 ϕ 得 E 、再把 E gather 给粒子”，而是要求粒子受力、离散 Poisson 解和场能量账本共享同一套 reciprocity 合同。对 WarpX 来说，这当然不意味着当前 `FieldGather.H` 里就直接复现了 Birdsall 的整个 energy-conserving electrostatic 变分算法；更准确的说法是，官方文档和 regression 里保留下来的 `energy-conserving gather / momentum-conserving gather` 命名，只有放回这条更老的理论分叉里才不会被误解成“两个 wrapper 里哪一个 stencil 更光滑”。

因此，本章后面凡是提到 gather family、field centering、collocated grid、Langmuir 守恒基线时，都应该记住自己实际上在比较三层东西：

1. `IndexType` 与 stagger/nodal centering 的实现差别；
2. sampled field 怎样被回插到粒子；
3. 这套回插究竟服务于哪一种离散守恒合同。

还有一层容易漏掉：implicit path 并不复用显式 gather。`FieldGather.H` 的 `doGatherShapeNImplicit(...)` 会先按沉积算法分派：

- `Esirkepov`: `doGatherShapeNEsirkepovStencilImplicit`
- `Villasenor`: `doGatherPicnicShapeN`
- `Direct`: 才退回普通半步位置 gather

所以 implicit 里不是“先统一 gather，再换 deposition”，而是 gather stencil 本身就要和后面的 charge-conserving 轨道几何解释匹配。

最后，external particle fields 也不止一种接入方式。`PushPX()` 里真实顺序是：

1. 先把常量 `m_E_external_particle/m_B_external_particle` 加到粒子局部 `Exp...Bzp`
2. 再对主网格场调用 `doGatherShapeN()`
3. 最后再执行 `GetExternalEBField` functor

其中：

- `parse*_ext_particle_function` 和 `repeated_plasma_lens` 是逐粒子直接相加；
- `read_from_file` 则不会在 gather kernel 内逐粒子加场，而是先把外场加到 `Efield_aux/Bfield_aux` 这类 `MultiFab`，再让粒子像 gather 主场一样去 gather 它们。

对应的 regression 也已经有了比较清楚的分层：

- `energy_conserving_thermal_plasma`: 在 electrostatic 周期热等离子体里检查总能量增长不超过 0.3%，这是当前对 energy-conserving gather 最直接的物理断言；
- `langmuir_multi_psatd_momentum_conserving`: 在 PSATD + momentum-conserving gather 组合下继续用 Langmuir 问题做守恒/稳定性基线；
- `load_external_field`: 分别用 3D/RZ 磁镜单粒子轨道和时间缩放脚本验证 external particle fields 的 read-from-file、time dependency 和 multi-field superposition。

4.10 位置推进与无质量粒子

显式位置推进的实现位于 `Source/Particles/Pusher/UpdatePosition.H` 的 `GetExplicitPusherDisplacement()` 与 `UpdatePosition()`。`PhysicalParticleContainer::Evolve()` 的顺序是先调用 `doParticleMomentumPush(...)`，再在 `PositionPushType::Full` 分支调用 `UpdatePosition(...)`；因此这里消费的是推进后的时间中心动量，而不是另一个独立导出的速度数组。对有质量粒子，源码先计算

$$\gamma^{-1} = \frac{1}{\sqrt{1 + |\mathbf{u}|^2/c^2}},$$

再按编译维度更新坐标：

$$x \leftarrow x + u_x \gamma^{-1} \Delta t, \quad y \leftarrow y + u_y \gamma^{-1} \Delta t, \quad z \leftarrow z + u_z \gamma^{-1} \Delta t.$$

对无质量粒子，源码使用

$$\mathbf{v} = c \frac{\mathbf{u}}{|\mathbf{u}|}$$

更新位置。因此 photon container 可以复用位置推进形式，但动量和沉积行为不同；光子容器的专门逻辑后续多物理章节再展开。

这个调用顺序也限制了如何验证“半步速度”。UpdatePosition.H 的注释把显式位置更新写成 $x(t + \Delta t) = x(t) + v(t + \Delta t/2)\Delta t$ ，但常规 Full 输出通常稳定提供的是位置和机械动量，而不是独立的 half-step velocity attribute。因此，相邻输出位置差可以构造与时间中心速度比较的 proxy，却不能被说成直接测得半步速度。对读者而言，正确的验证问题是：你比较的动量究竟处在哪个时间层，还是仅用相邻位置构造了平均量？

另一个容易忽略的分叉在 PushSelector.H: Boris 接受 FirstHalf/SecondHalf/Full 的 momentum_push_type，而当前 UpdateMomentumHigueraCary() 接口没有这一参数。因此不能把两个 pusher 写成完全相同的 split-half 输出合同。做轨道或时间层诊断时，先在输入中确认 pusher 和 push type，再比较位置、动量与场的时间层；只有这三者对齐，误差才可被归因于算法，而不是采样时刻不同。

4.11 RR、implicit 与 photon path

前面 4.1 到 4.10 主要还是显式带电粒子的主线，但 WarpX 的粒子推进并不只有这一条路径。至少还有三条不能忽略的分支：

1. classical radiation reaction;
2. implicit particle push;
3. photon container 的无质量推进。

先看 RR。Source/Particles/Pusher/PushSelector.H 的 doParticleMomentumPush() 说明，RR 不是第四种独立 pusher，而是优先级高于 ParticlePusherAlgo 的一个分支。省略编译宏后的控制流可概括为：

```
if (do_crr) {
    if (qed_sync && chi >= t_chi_max) {
        UpdateMomentumBoris(...);
    } else {
        UpdateMomentumBorisWithRadiationReaction(...);
    }
} else if (pusher_algo == ParticlePusherAlgo::Boris) {
    UpdateMomentumBoris(...);
} else if (pusher_algo == ParticlePusherAlgo::Vay) {
    UpdateMomentumVay(...);
} else if (pusher_algo == ParticlePusherAlgo::HigueraCary) {
    UpdateMomentumHigueraCary(...);
}
```

因此，打开 do_crr 后，当前粒子不会再走 Vay 或 Higuera-Cary，而是进入 Boris 家族分支。通常它调用“Boris 加辐射反作用”；但在编译 QED 且同步开关生效时，代码会先计算 χ ： $\chi < t_{\chi, \max}$ 才调用该 RR 例程，较高 χ 则调用普通 Boris。这一门限的含义是：源码保证了 pusher 家族的选择，却不保证每一个 χ 都附加 classical RR。Source/Particles/Pusher/UpdateMomentumBorisWithRadiationReaction.H 中的 UpdateMomentumBorisWithRadiationReaction() 则表明低 χ 分支如何实现：它先调用普通 UpdateMomentumBoris()，再用新旧动量平均构造中间时刻的 γ_n 、 $\mathbf{v}_n$ 和 Lorentz force，最后再把辐射反作用力乘 dt 加回动量。代码结构上，这是一种 Boris 后附加阻尼项，而不是完全重写一套 relativistic mover。

再看 implicit path。它和显式 PushPX() 的根本区别，不在于换了另一个 UpdateMomentum*()，而在于时间层和收敛逻辑都改了。../warpX/Source/Particles/Pusher/ImplicitPushPX.cpp 的注释直接说明了顺序：

1. 先 position push 半步；
2. 再 gather 场；
3. 再做 velocity push；
4. 再把 old/new velocity 平均成 time-centered 值；
5. 位置和速度彼此依赖，因此做 Picard 固定点迭代，直到 step norm 收敛。

而这里真正把上一篇属性图接进来的，是 x_n/y_n/z_n/ux_n/uy_n/uz_n 和 nsuborbits。在 ../warpX/Source/Particles/Pusher/ImplicitPushPX.cpp 中，这些量被明确当成“the positions and velocities saved at the start of the step”取出；随后粒子初值直接从 x_n 和 ux_n 开始，而不是从当前位置盲目继续推进。也就是说，x_n/ux_n 在 implicit 路径里不是诊断缓存，而是 nonlinear solve 的参考态。

nsuborbits 则是 implicit 不收敛时的 fallback 状态。ImplicitPushXP() 在 ../warpX/Source/Particles/Pusher/ImplicitPushPX.cpp 中，如果粒子没收敛，就把 nsuborbits[ip] = 2，再通过 SetupSuborbitParticles() 把这些粒子的权重临时置零、单独收集索引。后续

`ImplicitPushXSubOrbits()` 又会强制把沉积算法切到 Villaseñor, 见 `ImplicitPushPX.cpp`。所以 suborbit 不只是“多分几步时间步”, 还会连带改变当前粒子的沉积路径。

最后看 photon container...`./warpX/Source/Particles/PhotonParticleContainer.cpp` 的 `PhotonParticleContainer::Evolve()` 并没有重写 species 外层循环, 而是继续调用 `PhysicalParticleContainer::Evolve(...)`。也就是说, tile loop、AMR buffer 分区、gather 外壳这些基础设施仍然复用。但 photon 通过两层专门改写改变了物理语义:

- `PhotonParticleContainer.H` 中 `DepositCharge()` 和 `DepositCurrent()` 都是空实现;
- `PhotonParticleContainer::PushPX()` 在 `./warpX/Source/Particles/PhotonParticleContainer.cpp` 里只做 gather、可选 Breit-Wheeler optical depth 演化、以及无质量 `UpdatePosition(...)`, 并不调用 Boris/Vay/Higuera-Cary 的带电动量更新。

因此 photon path 和普通 charged species 的差异不只是“不沉积电流”, 而是连 momentum update 的物理模型都变了。它消耗的 runtime attributes 主要是:

- builtin ux/uy/uz;
- opticalDepthBW;
- 可选 *_btd;

而不会消费普通 charged implicit 路径里的 `ionizationLevel`、`opticalDepthQSR`、`x_n/ux_n/nsuborbits`。

从这一层往后看, WarpX 粒子推进可以总结成三种结构:

- 显式主线: Boris / Vay / Higuera-Cary;
- 显式修正: Boris + classical RR;
- 非标准推进: implicit fixed-point / suborbit fallback / photon push。

它们共享的是 `PhysicalParticleContainer::Evolve()`、gather 外壳和粒子属性系统; 真正分叉的是时间层、收敛控制、沉积算法约束和具体消耗的 runtime attributes。

如果再往 implicit solver 深处走一步, 还要继续把“suborbit 轨道本身”和“JFNK 线性化源项拼装”分开看...`./warpX/Source/FieldSolver/ImplicitS` 明确把 linear stage 的电流写成

$$J(E) = J_{\text{suborbit}} + J_0 + \text{MM}(E - E_0).$$

这里:

- `J_0` 对应 `current_fp_non_suborbit`;
- `MM` 对应 `MassMatrices_X/Y/Z`;
- `J_suborbit` 才是那些真正需要 suborbit fallback 的粒子继续显式推进后沉到 `current_fp` 的部分。

这正好解释了为什么 `MultiParticleContainer::Evolve()` 在 implicit 模式下还要额外处理 `current_fp_non_suborbit` 和 `MassMatrices_PC` 的清零时机, 见 `./warpX/Source/Particles/MultiParticleContainer.cpp`。它不是普通 bookkeeping, 而是在维护 JFNK 的三项分拆。

WarpX 为这一节提供了一条直接的 regression 入口: `Examples/Tests/radiation_reaction/`。它不是应用级 checksum, 而是强 analysis:

- 平行动量 case 要求 gamma 保持不变;
- 垂直动量 case 要求 gamma(t) 满足解析 Landau-Lifshitz 衰减公式;
- 容差统一为 5%。

因此它正好锚定了上面这条“先 Boris, 再加 RR 修正”的源码路径, 而不是泛化地证明“高能粒子大概会辐射”。

`ImplicitPushXSubOrbits()` 里还有两条实现约束非常重要。第一, `./warpX/Source/Particles/Pusher/ImplicitPushPX.cpp` 强制把 suborbit 路径的 current deposition 切到 Villaseñor:

```
const auto depos_type = CurrentDepositionAlgo::Villaseñor;
```

所以一旦粒子进入 suborbit fallback, 用户原来选择的沉积算法并不会继续沿用。第二, `./warpX/Source/Particles/Pusher/ImplicitPushPX.cpp` 把 `deposit_mass_matrices` 限定成

```
use_mass_matrices_pc && !linear_stage_of_jfnk
```

这说明 suborbit 粒子的 mass-matrix deposit 当前只服务 preconditioner, 不直接服务 Jacobian 的 linear stage。

于是 implicit 粒子推进实际上还要再分成四层:

1. $x_n/ux_n/nsuborbits$ 这组 step-start 属性;
2. PushXPSingleStep() 的 fixed-point 单轨道求解;
3. 不收敛粒子的 suborbit fallback 与 Villasenor-only 重沉积;
4. $current_fp_non_suborbit + MM*(E-E_0) + J_suborbit$ 这套线性化源项拼装。

这也是为什么 implicit 路径不能被简单概括成“把显式 pusher 改成 Picard 迭代”。它已经把粒子推进、沉积算法、质量矩阵和 Newton/JFNK 线性化绑定了一套更大的数值系统。

4.12 Field Ionization: ADK 不是附加后处理, 而是粒子属性和沉积权重一起改写

前面已经出现过 ionizationLevel, 但如果不单独拆开, 很容易把 field ionization 理解成“额外生成几个电子”。源码里真实发生的是三件事一起改变:

1. 源离子 species 在运行时新增整数属性 ionizationLevel;
2. 电离事件通过 filterCopyTransformParticles 复制到 product electron species;
3. 后续 ρ/J 沉积时, 源离子的有效电荷按 ionizationLevel 动态乘上去。

PhysicalParticleContainer::InitIonizationModule() 会在拿到运行时 dt 后, 才真正初始化 ADK 所需的:

- ionization_energies
- adk_power
- adk_prefactor
- adk_exp_prefactor
- 可选 adk_correction_factors

同时还会在没有现成属性时补上:

```
if (!HasAttrib("ionizationLevel")) {
    AddIntComp("ionizationLevel");
}
```

这说明 ionizationLevel 属于“模块初始化阶段动态加入的持久物理状态”, 而不是 PIDx builtin。

这里还有两个容易被漏掉的源码边界。

第一, WarpX 的 field ionization 只有 ADK 主链, 没有独立 OTB 分支。官方参数文档对 do_field_ionization 的描述明确写的是 using the ADK theory, 源码里读到的也只有 do_adk_correction、adk_prefactor、adk_exp_prefactor 和 adk_power。因此如果后面讨论 OTB, 那只能作为“源码未实现的外部模型边界”, 不能写成 Ionization.* 已有的一条并行实现。

第二, physical_element 也不是“用户手工给一串电离能表”的接口。InitIonizationModule() 实际是通过 ion_map_ids、ion_atomic_numbers 和 ion_energy_offsets 从 WarpX 内建的 table_ionization_energies 里切出整段 successive ionization energies, 再按当前运行时 dt 现算 ADK prefactor。因此 species 构造期不能完成这一步, 真正原因不是代码组织习惯, 而是 ADK 率已经把本步 dt 吸进了系数里。

真正的单粒子电离判定在 Particles/ElementaryProcess/Ionization.H 的 IonizationFilterFunc::operator()。它不会只看实验室系 $|E|$, 而是先 gather 主场和 external particle field, 再结合当前 $ux/uy/uz$ 计算粒子系场幅值, 然后才用 ADK 公式给出概率:

```
Real w_dtau = (E <= 0._rt) ? 0._rt : 1._rt / ga * m_adk_prefactor[ion_lev] *
    std::pow(E, m_adk_power[ion_lev]) *
    std::exp( m_adk_exp_prefactor[ion_lev]/E );
const Real p = 1._rt - std::exp( - w_dtau );
```

因此 boosted-frame field_ionization regression 不是“换个坐标跑一遍”而已, 它实际上在验证这层相对论一致性。

更容易被忽略的是 transform 本身。IonizationTransformFunc 看起来只有一句:

```
src.m_runtime_idata[0][i_src] += 1;
```

但这并不意味着 product electron 没被创建。真正的 product species 写入, 是由 MultiParticleContainer::doFieldIonization() 里的:

```
filterCopyTransformParticles<1>(...)
```

统一完成的。也就是说，一次电离事件被拆成：

- 源离子 `ionizationLevel += 1`
- 新电子复制到 `ionization_product_species`

两部分。

最后，这个离化态不会只停留在 `diagnostics`。`WarpXParticleContainer::DepositCurrent()` 和 `DepositCharge()` 都会把 `ionizationLevel` 传下去，而底层沉积 `kernel` 会做：

```
if (do_ionization) { wq *= ion_lev[ip]; }
```

因此 field ionization 的真正闭环是：

ADK event $\rightarrow$ `ionizationLevel` $\rightarrow$ effective source charge $\rightarrow \rho, J \rightarrow$ fields and diagnostics.

从粒子推进的视角看，这说明 WarpX 的多物理不总是“在主循环旁边加一个模块”。像 field ionization 这样的过程，会直接改写粒子属性系统和沉积权重，所以它应该被视为 particle algorithm 本体的一部分。

更细一点说，这里的“改写沉积权重”并不是去改宏粒子统计权重 `w`，而是保持 `w` 不变、只把有效物理电荷改成

$$wq_e \times \text{ionizationLevel}.$$

这也是 `InitIonizationModule()` 一开始就把 `ionizable species` 的 `charge` 强制改回 `q_e` 的原因：源码要把“多少个 physical particles”与“每个 physical particle 当前带几个基本电荷”这两层语义拆开，前者留在 `w`，后者留在 `ionizationLevel`。

4.13 Collisions: 不是旁路模块，而是和 momentum push 强耦合的时间调度层

field ionization 还是“每个源 species 自己带 product species 逻辑”的模块，但 collision 从入口层开始就不是这样。`MultiParticleContainer` 构造时直接建：

```
collisionhandler = std::make_unique<CollisionHandler>(this);
```

运行时则统一走：

```
collisionhandler->doCollisions(step, cur_time, dt, this);
```

这说明 collision 在 WarpX 里的第一抽象层不是某个 species 的附加属性，而是一个面向整个 `MultiParticleContainer` 的统一调度器。

`CollisionHandler` 的第一职责也不是实现物理公式，而是把：

```
collisions.collison_names  
<collision_name>.type
```

映射成具体对象。当前入口层已经能分出：

- `pairwisecoulomb`
- `background_mcc`
- `pulsed_decay`
- `background_stopping`
- `dsmc`
- `nuclearfusion`
- `bremsstrahlung`
- `linear_breit_wheeler`
- `linear_compton`

其中有些是独立的 `CollisionBase` 派生类，有些则走模板化的 `BinaryCollision<CollisionFunc, CreationFunc>`。因此“collision family”在 WarpX 里从一开始就包含了既可能只改动动量、也可能会生成新粒子的过程。

真正的公共合同在 `CollisionBase`。它提供的不是一条统一散射公式，而是：

- `species`
- `ndt`
- `CollisionSteppingMode::`{Supercycle, Subcycle}

两种 stepping mode 的语义不同:

- Subcycle: 一个 PIC step 内跑 `ndt` 次, 每次用 `dt/ndt`
- Supercycle: 每 `ndt` 个 PIC steps 才跑一次, 但单次碰撞用 `dt*ndt`

这说明 collision 入口首先定义的是“碰撞算子相对于 PIC 主步怎样调度”, 而不是“碰撞算子本体长什么样”。

更关键的是, collision 从全局参数读取阶段就已经和 momentum pusher 强耦合。WarpX.cpp 只要看到:

```
collisions.collision_names = ...
```

默认就会打开:

```
m_collisions_split_momentum_push = true;
```

然后再允许用户用:

```
collisions.split_momentum_push
```

覆盖。这说明 collision 默认并不是“在完整粒子推进前或后统一做一次”, 而是插在 momentum push 的中间。源码还明确给了当前边界:

- implicit evolve scheme 下不真正支持 split momentum push
- Higuera-Cary 目前也不支持

因此, collision 入口从一开始就已经和:

- pusher 选择
- electrostatic / electromagnetic solver
- 主循环时间组织

绑在一起。

这层耦合并不是只靠源码阅读猜出来的, analysis_test_2d_collisions_split_momentum_push.py 直接拿 reduced diagnostics 的:

- field_energy
- particle_energy

做两类断言:

1. 总能量守恒误差必须足够小
2. 场能量涨落长期平均必须接近 equipartition 参考值

所以这组 regression 真正在验证的是 collision insertion ordering 的数值合同, 而不是某个散射截面是否长得对。

从粒子推进章节的视角看, 这意味着 WarpX 的多物理插入至少分成三种类型:

1. 改写粒子属性和沉积权重: field ionization
2. 改写 momentum push 的时间组织: collisions
3. 改写单粒子推进与产品粒子统计: QED / photon emission / pair creation

它们都不是“主循环旁边挂一个功能块”这么简单, 而是直接进入粒子算法本体。

4.13.1 BinaryCollision 先产出事件表, ParticleCreationFunc 再真正创建 products

如果继续沿 collisions 这一支往下读, 就会发现 BinaryCollision<CollisionFunc, CopyTransformFunc> 的 cell-level functor 并不直接往 product species 里塞粒子。它先输出的是几张事件表:

- p_mask
- p_pair_indices_1
- p_pair_indices_2
- p_pair_reaction_weight

也就是:

- 哪一对 parent 真发生了反应
- 这次反应对应哪两个 parent 粒子
- 这次反应要抽走多少权重交给 products

真正把这些事件落到 product species tile 里的, 是后面的 ParticleCreationFunc。

这层分工很重要, 因为它说明 collision 模块内部也不是“一次 kernel 既判断又创建”, 而是:

```
CollisionFunc
-> event tables
-> ParticleCreationFunc
-> type-specific momentum initialization
```

ParticleCreationFunc 里最关键的分支是:

```
const int products_per_reactant_factor =
    (m_collision_type == CollisionType::LinearCompton) ? 1 : 2;
```

它把 binary creation 分成两类:

1. LinearCompton
 - 每个 product species 每个事件只创建 1 个宏粒子
 - 散射 photon 继承入射 photon 位置
 - 散射 electron 继承入射 electron 位置
2. 几乎所有其他 binary creation
 - 包括 LinearBreitWheeler
 - 都会在两个 reactant 的位置各复制一套 products

这就是为什么 LinearBreitWheeler 虽然物理上只产生一对 electron + positron, 实现上却会生成:

- 2 个电子宏粒子
- 2 个正电子宏粒子

并且每个宏粒子只拿一半反应权重。WarpX 用这种“复制到两个 parent 位置”的实现换取了局域电荷守恒。

LinearCompton 则反过来是当前最明显的特例。它不会做双份复制, 而是:

- 1 个散射 photon
- 1 个散射 electron

然后用 LinearComptonInitializeMomentum() 在电子静止系里按 Klein-Nishina 分布抽样散射角, 再 Lorentz 变换回 lab frame, 并用总动量守恒补出散射后电子动量。

所以如果把 collisions 再往 product 创建层推进一步, 当前最值得记的不是某个碰撞截面公式, 而是:

- BinaryCollision 先做事件压缩
- ParticleCreationFunc 再做统一 product 落地
- LinearBreitWheeler 和 LinearCompton 在“一个事件到底创建几个宏粒子”这件事上故意采用了两种不同合同

这也解释了为什么 linear_breit_wheeler 与 linear_compton 的 regression 都把:

- 守恒量
- product species 数目
- 最终产额

看得非常重, 因为真正容易出错的地方就在 product 布局和权重分配, 不只是事件概率。

4.13.2 BackgroundMCC、PulsedDecay 与 DSMC: collision 还有三条完全不同的后半段

前一小节已经把 BinaryCollision -> ParticleCreationFunc 这条 product 创建主链打通了, 但 WarpX 的 collision 还至少有三条不能混写进去的实现分支。

第一条是 BackgroundMCCCollision。它不是“两个显式 species 配对后再散射”, 而是:

- 一个显式运动 species
- 一个由 background_density(x,y,z,t)、background_temperature(x,y,z,t) 或常数描述的背景气体
- 一个用 m_max_background_density 和截面表扫出来的 nu_max

因此它的核心上游稳定性门问是

$$\nu_{\max}\Delta t,$$

而不是前面 BinaryCollision 那种 pair enumeration。更关键的是,若打开 impact ionization,它也不是走 ParticleCreationFunc,而是走:

```
ImpactIonizationFilterFunc
-> filterCopyTransformParticles
-> 原电子动量更新 + 新电子/新离子创建
```

也就是说, BackgroundMCC 的 ionization 是 filter/copy/transform 分支, 不是 pair-event-table 分支。

第二条是 PulsedDecay。它甚至不再是 pairwise 碰撞, 而是 cell 内总权重衰变问题。源码先在每个 cell 统计 parent species 的总权重, 再按

$$W_{\text{prod}} = W_1 (1 - e^{-\nu(x,y,z,t)\Delta t})$$

算出本步应衰变掉多少权重, 然后再用 fixed_product_weight 把这份总权重离散成 product 宏粒子数。新粒子的位置和基底速度来自该 cell 内随机挑选的 parent 粒子, 再叠加方向相关 thermal speed。这里真正的物理合同是:

- parser 驱动的 decay_rate(x,y,z,t)
- fixed-weight 宏粒子离散化
- parent 权重逐步扣减

而不是“一对 parent 粒子 -> 一次散射事件”。

第三条是 DSMC 的 SplitAndScatterFunc。它虽然仍挂在 BinaryCollision 大框架下, 但前半段 DSMCFunc 只负责:

- 组 pair
- 调 CollisionPairFilter
- 写 p_mask/p_pair_indices/p_pair_reaction_weight
- 先从 reactants 扣权重

真正的后半段 product/child 创建则交给 SplitAndScatterFunc, 而不是前面那条 ParticleCreationFunc。这里最重要的 slot 语义是:

- slot 0: 第一 reactant 的 child copy
- slot 1: 第二 reactant 的 child copy
- slot 2/3: true products

于是:

- elastic / back 只在 slot 0/1 生成 weight-split child, 并在 COM frame 随机旋转或反转速度;
- charge_exchange / two_product_reaction 在 slot 2/3 生成 products, 并由 TwoProductComputeProductMomenta(...) 统一处理动量;
- charge_exchange 还会交换 products 的生成位置, 以维持局域电荷守恒;
- DSMC ionization 则会同时保留 slot 0 的 incident child, 并在 slot 2/3 生成 ejected electron 和 ion。

所以到这里, WarpX 的 collision 至少要分成四种后半段语义:

1. BackgroundMCC 的背景介质 + filter/transform
2. PulsedDecay 的 cell 内总权重衰变
3. BinaryCollision + ParticleCreationFunc
4. BinaryCollision + SplitAndScatterFunc

这也是为什么不能再用一套统一口径把 BackgroundMCC、PulsedDecay、DSMC、LinearCompton 和 QED 都简单写成“会产生新粒子的碰撞”。

在 regression 层, 这三条分叉也看完全不同的量:

- analysis_collision_3d_pulsed_decay.py 比的是最终 ion 总权重和 0D 衰变模型;
- analysis_ionization_dsmc_3d.py 比的是 n_e、n_n 和 n_eT_e 的全局模型; 3D ion-impact sibling 则是 ion_impact_ionization.dat + ions neutrals 的 checksum-only 分叉。
- analysis_charge_exchange_dsmc_1d.py 比的是通量指数衰减;
- analysis_two_product_reaction_dsmc_1d.py 比的是 products 的理论速度;

- `analysis_photoneutralization_dsmc_1d.py` 还把快电子的产物验证接到了 `BoundaryScraping` diagnostics。

4.13.3 `pairwisecoulomb`、`bremsstrahlung`、`background_stopping` 与 `nuclearfusion`

把上一节的 `BackgroundMCC` / `PulsedDecay` / `DSMC` 再往外扩, `WarpX` 当前 `collision` 里还剩四条不能并进同一套 `product` 语义的主分叉。

第一条是 `pairwisecoulomb`。它虽然也走 `BinaryCollision` 的 `cell` 内 `pairing` 外壳, 但 `PairWiseCoulombCollisionFunc` 最后只做一件事:

```
ElasticCollisionPerez(...)
```

也就是说, 它没有 `products`, 也不依赖后面的 `ParticleCreationFunc` 或 `SplitAndScatterFunc`。构造期真正重要的只有:

- `CoulombLog`
- `use_global_debye_length`

当 `CoulombLog < 0` 且不使用 `global Debye length` 时, 源码会进一步要求运行时 `local temperature`, 从而按局域状态估算碰撞尺度。它的 `regression` 口径也因此不是“有没有新粒子”, 而是:

- `analysis_collision_1d.py` 检查 `relaxation benchmark`
- `analysis_collision_1d_correct_conservation.py` 检查总动量和总动能守恒

第二条是 `bremsstrahlung`。它重新回到 `pair-event` 表, 但后半段 `product` 创建也不是通用 `ParticleCreationFunc`, 而是专门的 `PhotonCreationFunc`。前半段 `BremsstrahlungFunc` 只给出:

- 哪一对发生事件
- `photon` 权重
- `photon` 能量

后半段再由 `PhotonCreationFunc`:

1. 在第一 `reactant` 位置上创建 `photon`
2. 同时回写 `parent electron/ion` 动量
3. 用能量动量守恒补全 `photon` 动量

因此 `bremsstrahlung` 的真实语义是“先算失能和 `photon energy`, 再做 `parent+product` 一起更新”。`analysis_collision_1d_Bremsstrahlung`。也正是按这个口径验证:

- 总能量守恒
- 总动量守恒
- `dE/dx` 与解析估计
- 每步新 `photon` 数与解析截面估计

第三条是 `background_stopping`。这已经不再是二体散射, 而是对单 `species` 应用解析 `slowing-down law`。源码在 `background_type = electrons` 与 `ions` 之间分成两条公式:

- 对电子背景:

$$\frac{d\mathbf{u}}{dt} = -\alpha\mathbf{u} \quad \Rightarrow \quad \mathbf{u}^{n+1} = \mathbf{u}^n e^{-\alpha\Delta t}$$

- 对离子背景:

$$\frac{dW}{dt} = -\frac{\alpha}{\sqrt{W}}$$

再按解析积分结果缩放速度

因此 `background_stopping` 的 `regression` 口径也完全不同: `ion_stopping/analysis.py` 直接用 `Python` 重写同一组 `slowing-down` 公式, 对 `constant` / `parsed` 的电子与离子背景逐点对照最终粒子能量。

第四条是 `nuclearfusion`。它又回到 `pair-event` 框架, 但事件概率控制比普通 `BinaryCollision` 更复杂。构造期最关键的不是单个截面文件, 而是:

- `event_multiplier`
- `probability_threshold`
- `probability_target_value`
- `fusion type`

其中 `event_multiplier` 用来提高稀有 fusion 事件的统计量, 而 `probability_threshold / probability_target_value` 用来在单 pair 概率过大时自动把 multiplier 压回安全范围, 避免系统性低估 yield。源码上的截面模型至少分成:

- `BoschHaleFusionCrossSection`: D-D / D-T 之类经典两产物 fusion
- `ProtonBoronFusionCrossSection`: p-B11, 对应 Tentori-Belloni 2023 的拟合

它们的 regression 也相应分层:

- `analysis_two_product_fusion.py` 检查 reactant 消耗、product 数、能量动量守恒、各向同性和理论 yield
- `analysis_proton_boron_fusion.py` 进一步检查 p-B11 的三 alpha 产物、热率拟合, 以及 `probability_threshold` 过大的故意失真场景

所以如果把已经成文的 collision 分支一起看, WarpX 至少已经出现了下面这些互不等价的后半段:

1. 纯动量散射: `pairwisecoulomb`
2. filter/transform: `BackgroundMCC ionization`
3. 固定权重衰变: `PulsedDecay`
4. DSMC child/product 重建: `SplitAndScatterFunc`
5. 通用 products: `ParticleCreationFunc`
6. photon 专用 products: `bremsstrahlung` 的 `PhotonCreationFunc`
7. fusion event-probability + specialized kinematics: `nuclearfusion`

这说明确实到目前为止, “collision 模块”已经不能再被当作一个统一 product 创建器来讲, 而必须按物理合同和后半段实现机制分开处理。

4.13.4 kernel 细节层: Perez 有效碰撞密度、Bremsstrahlung photon 写回、fusion products 复制语义

把上一节四条分叉再往下读, 最容易误判的不是类型分派, 而是它们在 kernel 末端到底怎样把统计事件变成具体粒子动量。

对 `pairwisecoulomb` 来说, `PairWiseCoulombCollisionFunc` 真正交给 `ElasticCollisionPerez(...)` 的并不是“原始 cell 密度”, 而是一套先闭合局域尺度、再构造有效配对密度的量:

- 若 `CoulombLog < 0`, 先按局域 n, T 算 Debye 长度
- 再取 $b_{\max} = \max(\lambda_D, r_{\min})$, 保证 screening length 不小于原子间距
- 然后构造加权后的

$$n_{12}$$

来驱动 Perez 散射

因此 WarpX 当前的 weighted macroparticle Coulomb 碰撞, 真正进入单对散射 kernel 的强度不是简单的 cell 物理密度, 而是“局域 plasma 尺度 + 抽样配对统计”共同决定的有效量。这也解释了为什么 Coulomb regression 里一个重点看 relaxation, 另一个重点看守恒。

对 `bremsstrahlung` 来说, 上一节只讲到了前半段 `BremsstrahlungFunc` 如何给出 photon energy。再往后看 `PhotonCreationFunc`, 会发现它不是只把 energy 填进 product, 然后结束。它会先在离子静止系内解出:

- 更新后的 electron
- 更新后的 ion
- photon 三动量

再一起变回 lab frame。最后写到 photon species 的不是“真实质量意义下的速度”, 而是

$$u = \frac{p}{m_e}$$

型的统一接口变量:

```
upx = p3x_rel / PhysConst::m_e;
upy = p3y_rel / PhysConst::m_e;
upz = p3z_rel / PhysConst::m_e;
```

这样做不是修改 photon 物理, 而是为了让 collision-created photons 继续复用 WarpX 现有的粒子 SoA 与 diagnostics 链。

对 fusion 来说, 上一节已经区分了 D-D / D-T 两产物和 p-B11 三 alpha 两条物理路径; 这一层再补的是“为什么实现里看到的宏粒子数更多”。两产物 fusion 的 `TwoProductFusionInitializeMomentum(...)` 其实只算一次 products 动量, 然后把:

- 第一 product 的一组 (ux, uy, uz) 写两次
- 第二 product 的一组 (ux, uy, uz) 也写两次

所以实现里出现 4 个宏粒子，并不表示一次 fusion 物理上有 4 个独立 products，而是因为 WarpX 在两个 parent 位置各复制一套 child。

p-B11 的 ProtonBoronFusionInitializeMomentum(...) 进一步复杂一些。它先按

p + B11 -> alpha1 + Be8*

做一次两产物动量求解，再在 Be8* 静止系里把剩余 3.12 MeV 衰变能随机分到：

Be8* -> alpha2 + alpha3

最后再变回 lab frame。于是实现里会看到 6 个 alpha 宏粒子：

- alpha1 两份
- alpha2 两份
- alpha3 两份

本质上仍然是“在两个 parent 位置各复制一套 products”的事件记账方式，而不是 6 个不同物理 alpha 通道。

所以到这一层可以把 collision 剩余 kernel 细节压成三句：

1. Perez 路径最关键的是局域尺度闭合和加权 n12
2. Bremsstrahlung 最关键的是 photon 动量写回并与 parent 更新同时完成
3. Fusion 最关键的是“先算真实 kinematics，再做双 parent 位置复制”的宏粒子实现语义

4.13.5 更深一层的 event kernel: Perez 角分布、Bremsstrahlung 能谱反演与 fusion 概率控制

如果再往 BinaryCollision 的 event kernel 里读，collision 还有一层之前没写清的共同问题：上一节讲的是“哪些量会被写回”，这一层讲的是“这些量到底怎样被抽出来”。

对 pairwisecoulomb 而言，ElasticCollisionPerez(...) 上游已经准备好了：

- bmax
- sigma_max
- 加权后的 n12

但 UpdateMomentumPerezElastic(...) 并不是把这些量直接当成一次 Bernoulli 碰撞概率。它先在 COM frame 内构造归一化散射长度

$$s_{12},$$

然后按 s12 所处区间切到四段式角分布：

- $s_{12} \leq 0.1$: $\cos\chi_s = 1 + s_{12} * \log(r)$
- $0.1 < s_{12} \leq 3$: 用五次多项式近似的 Ainv
- $3 < s_{12} \leq 6$: 用 $A = 3 \exp(-s_{12})$ 的另一段反演
- $s_{12} > 6$: 直接退化成 $\cos\chi_s = 2r - 1$ 的各向同性散射

随后再抽一个独立方位角，把 post-collision 动量从 COM frame 变回 lab frame。更关键的是，最后并不是无条件同时更新两只 reactant，而是分别按：

```
if (w2 > r * max(w1, w2)) update particle 1
if (w1 > r * max(w1, w2)) update particle 2
```

做 weighted rejection。因此 WarpX 当前的 weighted macroparticle Coulomb collision，真正的语义是：

- n12 和 s12 决定散射强度
- reactant 是否真正写回，再由宏粒子权重比决定

对 bremsstrahlung 而言，前两节已经区分了 BremsstrahlungFunc 和 PhotonCreationFunc。更深一层看 BremsstrahlungEvent(...)，会发现它的前半段先把电子变到离子静止系，再结合电子密度构造 plasma-frequency cutoff：

$$E_{\text{cut}} = \hbar\omega_{pe}.$$

如果电子在离子静止系里的动能 KE_eV 低于这个阈值，就直接返回 0，不让软光子进入采样。若允许发生事件，代码并不是从表里拿一个最近点，而是先按电子能量方向插值，再在 k/T1 网络上逐段积累 trapezoidal cross section，最后在命中的区间内反演出连续 photon energy。

这一层还有一个必须记录的当前实现边界：event probability 用的核心量是

$$\arg = f_{\text{multi}} v_1 \sigma_{\text{total}} n_2 dt \frac{\gamma_1^{\text{rel}}}{\gamma_1 \gamma_2}.$$

当 $\arg > 1$ 时，源码会先把 $\arg$ 饱和到 1，再去改 f_{multi} 。按当前实现，这一分支直接生效的是 probability saturation，而不是像 fusion 那样形成真正有效的 multiplier backoff。这一点应当被视为当前源码边界，而不是已有功能。

nuclearfusion 的 SingleNuclearFusionEvent.H 则正好相反：这里的 probability control 是真的做完了。它先算

$$P_{\text{est}} = \text{multiplier_ratio} f_{\text{mult}} \text{lab_to_COM_factor} w_{\text{max}} \sigma_f(E_{\text{coll}}) v_{\text{coll}} \frac{dt}{dV},$$

如果超过 probability_threshold，就按 probability_target_value 回退到一个新的 fusion_multiplier_eff ≥ 1 ，再把 product weight 缩成

$$w_{\text{product}} = \frac{w_{\text{min}}}{f_{\text{mult,eff}}}.$$

最终真正使用的事件率也不是简单线性截断，而是

$$P = 1 - e^{-P_{\text{est}}},$$

在代码里写成 `-std::expm1(-probability_estimate)`，专门保证小概率时的数值稳定性。

这一层和 regression 的关系也更清楚了：

- 2D / 3D Coulomb tests 看的主要是指数 relaxation fit
- inputs_test_3d_collision_iso{, _subcycle} 看的是各向异性温度的 isotropization 解析解，以及 ndt_subcycle 是否保持同一 collision timestep
- inputs_test_rz_collision 看的是 cylindrical-cell 配对与临时动量旋转假设下，局域共线粒子几乎不发生有效散射
- background_mcc 的 capacitive_discharge 条目则不该再混成普通 collision 单元测试：1D PICMI 入口实际拆成 analysis_1d.py 与 analysis_dsmc.py 两条 case-1 profile 对照，前者验证 background_mcc + external Poisson solver callback，后者验证把 DSMC 分支接入同一骨架后的离子密度 profile；2D native / PICMI 入口当前仍主要是应用级 checksum 基线，而 test_2d_background_mcc_dp_psp 在当前 CMakeLists.txt 中仍是注释掉的遗留变体

4.13.6 性能与数值后处理层：resampling、thermalizer 与 sorting

在 collision/QED 这些多物理分叉之外，Particles/ 里还有一层更偏数值控制与性能优化的对象图：

- Resampling
- ParticleThermalizer
- Sorting

它们不直接改变场求解器，但会显著改变宏粒子数、粒子分布以及后续 kernel 的访存局部性。

Resampling 是 species 级开关。PhysicalParticleContainer.cpp 先读 do_resampling，再按 species 构造一个 Resampling 对象。它内部又拆成：

- ResamplingTrigger
- ResamplingAlgorithm

触发条件不是只有固定步数，而是

$$\text{intervals.contains(step)} \vee \left(\frac{N_{\text{global}}}{N_{\text{cells,global}}} > \text{resampling_trigger_max_avg_ppc} \right).$$

调用位置在主循环里对应的是 `istep[0]+1`，所以匹配的是“下一步编号”。一旦触发，WarpX 会先 `Redistribute()`，再在各 level/tile 上调用具体算法，最后统一 `deleteInvalidParticles()`。因此 resampling 和 collision、absorbing boundary、EB scraping 一样，仍沿用“kernel 内先标 invalid，之后统一删粒子”的合同。

当前正式接入的算法有两条。LevelingThinning 是按 cell 独立工作的：对每个 cell 先算平均权重

$$\bar{w}_{\text{cell}},$$

再定义

$$w_{\text{level}} = \bar{w}_{\text{cell}} \times \text{target_ratio}.$$

凡是 $w_i \leq w_{\text{level}}$ 的粒子, 就以

$$1 - \frac{w_i}{w_{\text{level}}}$$

的概率删除, 否则把权重直接抬到 w_{level} 。这条算法只会抬低权重粒子, 不会动大权重尾部。对应的 `test_2d_leveling_thinning` analysis 也比较强: 一类 species 检查最终粒子数和统一权重, 一类 species 检查 Gaussian 权重分布下的 level-weight 和 untouched heavy tail。

`VelocityCoincidenceThinning` 则不是简单抽稀, 而是 merge。它先在每个 cell 内按速度空间分 bin, 可以是 spherical grid, 也可以是 cartesian grid; 然后把同一个 velocity bin 内的 cluster 压成两个保留粒子。实现上会累计 cluster 的总权重、加权平均位置、加权平均动量和总动能, 再构造一对关于平均动量对称的新动量, 使 cluster 内的线动量和动能都守恒, 其余粒子全部标 invalid。这里 `resampling_algorithm_target_weight` 的实际语义也要记清: 内部会乘 2, 因为每个 cluster 最后固定压成两粒子。当前这条路径只有 checksum, 没有像 `LevelingThinning` 那样的独立 analysis。

`ParticleThermalizer` 则是单独的全局参数块 `particle_thermalizer.*`, 不放在 species 段里。它当前只支持 Cartesian 1D/2D/3D, 不支持 RZ/RCYLINDER/RSPHERE。在主循环中的插入位置非常具体:

1. moving window
2. particle boundary / EB scraping / invalid 删除 / sorting
3. `m_particle_thermalizer.applyThermalizer(*mypc)`
4. collisions

因此 thermalizer 的效果会先落到“已经通过边界筛选保留下来的粒子”上, 再进入碰撞模块。它也不是硬墙式 momentum reset, 而是在一个有法向、起止位置和渐进概率的 thermal region 内, 对超过 `momentum_threshold` 的动量分量抽样重置为方差由 `theta` 决定的 Gaussian。`particle_absorbing_boundary` 会显式打开 `particle_thermalizer.*`, 并用 `PhaseSpaceElectrons` 直方图断言吸收边界附近的反向高速电子权重显著降低; 但这仍是 absorbing boundary、field-function laser、reduced diagnostic 和 thermalizer 的耦合 regression, 不是 dedicated thermalizer-only 单测。

Sorting 则必须和前文 AMR coarse-fine 的 `PartitionParticlesInBuffers()` 区分开来。后者是物理分流, 前者是全局性能阶段。`WarpXEvolve.cpp` 在 boundary 处理之后检查 `sort_intervals.contains(step+1)`, 触发后调用:

```
mypc->SortParticlesByBin(
    sort_bin_size, m_sort_particles_for_deposition, m_sort_idx_type);
```

文档和 `WarpX.cpp` 一起看, 当前默认值是平台相关的:

- GPU 默认 `sort_intervals = 4`
- CPU 默认 `sort_intervals = -1`

目的是提升 memory locality, 不是物理修正。并且还有两种模式:

- `sort_particles_for_deposition = true`
– 走 deposition-specialized sort
- `sort_particles_for_deposition = false`
– 走 generic bin sort, 粒度由 `sort_bin_size` 控制

所以到这里, `Particles/` 的“非主物理 kernel 层”也形成了清晰分层:

- resampling: 调粒子数与权重/局域 phase-space 代表性
- thermalizer: 调局部高动量粒子的热化行为
- sorting: 调后续 deposition/gather 的访存局部性

4.13.7 Vranic 2015: 两粒子 merge 的论文-源码边界

Vranic 等人的论文把粒子合并明确放在六维 phase space 中: 先用空间 merge cell 和动量 cell 找到相近 cluster, 再把一个 cluster 压成两个新宏粒子。单个新粒子一般不能同时满足总权重、总动量、总能量和质量壳关系; 两粒子构造则可以令

$$w_t = w_a + w_b, \quad p_t = w_a p_a + w_b p_b, \quad \epsilon_t = w_a \epsilon_a + w_b \epsilon_b.$$

在等权、等能量的简化下, 两个新动量关于总动量方向对称, 论文用

$$\cos \theta = \frac{p_t}{w_t p_a}$$

确定夹角，并用动量 cell 对角线选择平面，避免在原来没有展宽的方向凭空制造分布。论文还强调位置不能机械地都放在 cluster 质心，而应从原粒子位置中抽取，以避免 merge cell 中心出现人为密度尖峰。论文的图和数值例子说明了这种构造怎样在具体分布中工作；阅读时应把它们视为合并算法的机制证据，而不是 WarpX 默认设置的性能承诺。

这与 WarpX 的 VelocityCoincidenceThinning 存在清晰的结构映射：源码同样在 cell 内按速度空间分 bin，把 cluster 压成两个粒子，并累计权重、加权动量和动能；resampling_algorithm_target_weight 还因“两粒子输出”而在内部乘 2。论文的 Poisson merging-rate 公式、QED cascade speed-up 和分布复现结果则属于论文自己的 OSIRIS/QED 案例，不能转写成 WarpX 已完成的 runtime physics proof。

WarpX 的 resampling regression 主要检查粒子数、权重或 checksum；它没有把论文中的 two-stream、magnetic-shower 和 QED-cascade 逐一搬到相同的输入和诊断上。因此这里能成立的结论是“论文方法与 WarpX 的结构可以对应”，而不是“WarpX 已复现论文算例”或“加速比已被证明”。要评估一个重采样设置，至少还应比较重采样前后的局部总权重、动量、能量、空间分布和权重尾，而不能只看最终粒子数。

Muraviev 2021: agnostic down-sampling 的论文-源码边界 Vranic 2015 解释了“一个 cluster 压成两个粒子时如何同时保持局部动量和能量”；Muraviev 等人进一步把重采样问题拆成 merging、thinning 和 complete resampling，并提出 agnostic down-sampling 原则：至少一个粒子的权重变为零，同时每个原粒子的期望新权重仍等于旧权重。由此得到的不是单次 realization 的严格局部不变，而是任意由位置、动量或其他粒子状态定义的分布在 ensemble average 下保持不变。

论文比较了 simple、leveling、globalLev、numberT、energyT、conserv、mergeAv 和 merge。其中 numberT 严格保持 cell 总权重，energyT 严格保持 cell 总能量，conserv 可以把能量、三分量动量、总权重和空间中心矩组成线性不变量；反过来，simple 虽然最容易实现，却会产生很宽的局部权重尾，在 QED cascade 中可能制造无法被时间步解析的局部等离子体频率和非物理场增长。

这篇论文与 WarpX 有三条可用的概念连接：一是 LevelingThinning 对低权重粒子的 leveling 思路；二是 VelocityCoincidenceThinning 在 velocity bin 内把 cluster 压成两个粒子的 merge 结构；三是“只看 checksum 不足以证明重采样物理质量”的验证要求。论文的 PICADOR/hi-chi 运行使用了自己的 QED cascade、Weibel 和 k-means 实验，不能把其 growth rate、运行时间、权重尾或图 1-12 数值直接写成 WarpX 结果。

对 WarpX 而言，这篇论文最重要的启发是验证标准：算法名称、粒子数下降或 checksum 一致都不足以说明分布质量。若要判断一次 thinning 或 merge 是否可接受，应在同一空间单元和动量区间比较局部总权重、能量和三分量动量，另外报告 density variance 与权重尾；这些量分别对应守恒、统计代表性和数值稳定性。论文中 PICADOR/hi-chi 的 QED cascade、Weibel 等结果属于其自身的物理设置，不能直接充当 WarpX 的运行证据。

4.13.8 四类单粒子问题：该测什么，不能推出什么

particle_pusher、single_particle、larmor 和 photon_pusher 都只有很少的粒子，却不是同一种“单粒子测试”：

- particle_pusher
- single_particle
- larmor
- photon_pusher

读者首先要问的不是粒子数，而是每个输入固定了哪一种物理情形、比较了哪个观察量，以及该比较没有覆盖什么。

particle_pusher 是最直接的一条。输入里固定：

```
algo.particle_pusher = "higuera"
particles.B_ext_particle_init_style = "constant"
particles.B_external_particle = 0.0 0.0 1.0
particles.E_ext_particle_init_style = "constant"
particles.E_external_particle = -2.994174829214179e+08 0.0 0.0
```

并让单个 positron 在满足

$$E_x = -v_y B_z$$

的 force-free 场中跑 10000 步。analysis.py 只检查最终

$$x \approx 0.$$

因此它并不是一般轨道画图，而是直接给：

- PushSelector.H 的 ParticlePusherAlgo::HigueraCary
- UpdateMomentumHigueraCary(...)
- UpdatePosition(...)

这条 relativistic Higuera-Cary push 主链做强断言。

该输入得到的

$$\max |x| = 1.1430664324 \times 10^{-4},$$

低于 10^{-3} 容差。它支持“此 force-free 条件下的相对论推进和位置更新彼此相容”，但只覆盖单粒子、恒定外场和横向偏移这一观察量，不能替代三种推进器的完整轨道、能量或相空间基准。

在同一输入中只替换 `algo.particle_pusher`，可得到一个有意窄化的算法对照：

pusher	末态 $\max x $	$1e-3$ gate	解释
Boris	2.3213958529e3	FAIL	force-free cancellation 在这个高相对论设置下明显失真
Vay	1.0795497978e-4	PASS	保留较好的 relativistic frame/cancellation 行为
Higuera-Cary	1.1430664324e-4	PASS	在该观察量上与 Vay 同量级

三组不是三份独立的官方输入，而是对同一个 force-free 问题的 pusher-only 对照。因此它只支持相对论 cancellation 的差异提示，不替代 boosted-frame、Poincare section 或长期能量/相空间 benchmark。

`single_particle` 则必须拆成两类。

第一类 `inputs_test_2d_bilinear_filter` 虽然也只有一个电子，但 analysis 完全不看轨道，而是手工构造未滤波 J_x 、再用二维 bilinear kernel 卷积，最后和 plotfile 里的 j_x 比较。它真正验证的是：

- 单粒子沉积后的 current stencil
- `warpx.use_filter`
- `warpx.filter_npass_each_dir`
- 对称边界 bilinear filtering 的实现

所以它更接近 deposition/filter regression。

第二类 `inputs_test_1d_synchronize_velocity` 则在 `algo.maxwell_solver = none` 下给一个电子施加常量 E_z ，同时打开：

`warpx.synchronize_velocity_for_diagnostics = 1`

analysis 先在 Python 里手动做 half-backward、5 步 leapfrog、再加 half-forward 得到同步后的速度，然后和 diagnostics 输出比较。这条 regression 直接对应：

- 参数 `warpx.synchronize_velocity_for_diagnostics`
- `WarpXEvolve.cpp` 里 diagnostics 前的 `SynchronizeVelocityWithPosition()`

也就是说，它验证的不是 Boris 物理轨道误差，而是“输出给 diagnostics 的速度是否和位置处在同一时间层”。

第 5 个诊断步的模拟/理论值分别为：

- $z = 2.2985203786002786/2.2985203756075920$;
- $u_z = 879410.0053860814/879410.0053860815$;
- 相对速度误差为 $1.3237889e-16 < 1e-15$ 。

这支持的是 diagnostics time-level synchronization，不应被误写成单粒子 pusher 的独立轨道精度 benchmark。

`photon_pusher` 又是另一类。输入里建了 16 个 photon species，覆盖：

- 六个坐标轴正负方向
- 对角方向
- 动量大小 1 和 10

analysis 用理论式

$$\mathbf{x}(t) = \mathbf{x}_0 + ct \hat{\mathbf{u}}, \quad \mathbf{p}(t) = \mathbf{p}_0$$

逐 species 比较最终位置和动量。因此它真正打到的是：

- PhotonParticleContainer::PushPX()
- UpdatePosition(...) 里的 massless branch
- photon 不做 charge/current deposition 的合同

16 个 photon species 的末态最大相对误差为:

- 位置直线传播: $6.0986372e-16 < 1e-14$;
- 动量保持: $1.7217530e-16 < 2.2204460e-16$ 。

这条证据支持的是无质量粒子的 c 速率传播、方向保持和不参与带电粒子 current deposition 的路径, 不应与 Boris/Vay/Higuera-Cary 的带电动量旋转混写。

最后 larmor 反而要最保守。它输入里组合了:

- electron 和 positron
- 常量外部粒子磁场 B_y
- amr.max_level = 1
- PML
- warpx.do_dive_cleaning = 1
- full/raw diagnostics

但 CMakeLists.txt 里没有独立 analysis, 只有 checksum。因此更准确的说法是:

- 这是 charged-particle gyro-motion 与 external-particle-field、MR、PML、div-cleaning 组合稳定性的 checksum 基线;
- 它不是已有独立解析半径/回旋频率对照的强单粒子 analysis。

若把这个组合输入直接同连续 uniform-B_y 轨道比较, 电子和正电子的轨迹相对位移误差都是 $1.28285096e-2$, 动量相对误差都是 $9.69641193e-2$ 。这否定 checksum 的作用; 它只说明 MR/PML/divergence-cleaning 的组合已经超出连续单粒子解析解的假设。因此本章把它保留为 checksum-only 基线, 不能用它升级为强物理 gate。

这四类输入共同给出一个实用的阅读法:

- Higuera-Cary force-free relativistic push: 检查相对论推进链。
- diagnostics 半步速度同步: 检查输出时间层。
- bilinear current filter: 检查沉积后的网格量。
- photon 直线传播与动量守恒: 检查无质量传播。
- Larmor 半径/频率独立解析对照: 这组输入没有提供, 不能夸大。
- Larmor 连续轨道比较: 只用于说明为何不能把该组合输入提升为强 gate。

把它们都叫作“单粒子测试”, 会掩盖观察量、源码路径和可支持结论之间最关键的差别。

4.13.9 粒子诊断与外场: 两条不经过主网格场的路径

在“单粒子/推进器”之外, particle_fields_diags 与 plasma_lens 分别回答两个不同的问题: 怎样把粒子属性归约为网格诊断量, 以及怎样在不读取主网格场的条件下给粒子施加外场。

- particle_fields_diags
- plasma_lens

它们都直接消费粒子位置、动量或外部粒子场, 但验证对象并不是普通 pusher 轨道。

particle_fields_diags 的关键输入不是单粒子初值, 而是 diagnostics 配置:

```
diag1.particle_fields_to_plot = z uz uz_filt zuz jz
diag1.particle_fields_species = electrons protons photons
diag1.particle_fields.z(x,y,z,ux,uy,uz) = z
diag1.particle_fields.uz(x,y,z,ux,uy,uz) = uz
diag1.particle_fields.uz_filt(x,y,z,ux,uy,uz) = uz
diag1.particle_fields.uz_filt.filter(x,y,z,ux,uy,uz) = (uz < 0)
diag1.particle_fields.zuz(x,y,z,ux,uy,uz) = z * uz
diag1.particle_fields.jz(x,y,z,ux,uy,uz) = uz*q_e
diag1.particle_fields.jz.do_average = 0
```

analysis 会同时做三件事:

1. 用 yt 从粒子数据直接手工重建 cell-centered quantity
2. 读取 Full plotfile 中的 boxlib particle-field meshes
3. 再读取 openPMD 中同名 meshes

然后逐一比较：

- zavg
- uzavg
- zuzavg
- uzavg_filt
- jz

因此它真正验证的是：

- Diagnostics.cpp 对 particle_fields_to_plot、.filter(...)、.do_average 的参数解析
- ParticleReductionFuntor 如何把粒子归约成 cell-centered diagnostics field
- plotfile 与 openPMD writer 在这一层是否一致

这说明 WarpX 不仅能把粒子作为散点写出去，还能按用户给出的 parser 表达式把 species 内的粒子数据重新投影成网格诊断量。

plasma_lens 则落在粒子侧外场主链上。analysis 不是读主网格场，而是直接比较两颗测试电子穿过 lens 序列后的：

- 最终横向位置
- 最终横向动量

与解析透镜串联模型是否一致。

这里要区分两条源码入口。

第一条是：

```
particles.E_ext_particle_init_style = repeated_plasma_lens
particles.B_ext_particle_init_style = repeated_plasma_lens
```

它对应：

- MultiParticleContainer.cpp 读取 lens 周期、起点、长度、强度
- GetExternalEBField 在粒子 gather 之后、push 之前按粒子位置实时叠加 lens focusing field

第二条是 hard-edged 版本：

```
lattice.elements = ...
plasmalens*.type = plasmalens
plasmalens*.dEdx = ...
```

它对应 accelerator lattice 分支里的 HardEdgedPlasmaLens。

两者共用同一个 analysis，因此真正被验证的是：

- repeated-plasma-lens 粒子侧外场
- accelerator-lattice hard-edged lens
- boosted-frame plasma-lens 反变换后一致性
- short-lens residence correction

这一组的教学意义不是“又一条轨道”，而是两条与普通自治场推进不同的路径：

- 粒子 diagnostics 可以把粒子属性重新压成 cell-centered field
- 粒子侧外场可以完全绕过主网格场寄存器，直接通过 GetExternalEBField 进入 PushPX()

accelerator lattice 还有一个直接的解析基准：Examples/Tests/accelerator_lattice/hard_edged_quadrupoles*。单电子穿过 drift + quad + drift + quad 串联，分析从输入重建 lattice.elements、drift.ds、quad.ds、quad.dEdx，再按 hard-edged quadrupole 的解析透镜公式逐段积分，要求最终 x 误差低于 1%、u_x 误差低于 0.2%。boosted-frame 与 moving-window 变体继续使用同一解析对照，因此这里检查的不是“lattice 参数能读入”，而是 HardEdgedQuadrupole、LatticeElementFinder 与 PushPX() 能否共同给出正确的粒子偏转。

这里还有一个必须分清的源码边界：drift 只把长度 ds 转成 beamline 的 zs/ze 几何区间，不直接返回外场。真正给粒子累加 E/B 的是 HardEdgedQuadrupole 与 HardEdgedPlasmaLens；LatticeElementFinder 按 tile 建立最近元素索引，把 boosted-frame 粒

子坐标和步末 $z+v_zdt$ 变回实验室系, 调用各元件的 `get_field(...)`, 再把累计场变回 boosted frame 后交给 `PushPX()`。所以 `drift + quad + drift + quad` 中的 `drift` 只影响解析 beamline 几何和 residence 区间判定, 不参与外场累加。

读这一节时可用两个问题自检: 写出的 particle-field mesh 是否等于从粒子数据重新归约的量? 粒子所见的透镜场是否来自外场元件, 而不是主网格场? 前者要求同时比较数据归约和 writer, 后者要求同时比较元件几何、参考系变换和位置/动量的解析结果。

4.13.10 边界缓冲区与 Python 粒子操作: 能控制什么, 尚未覆盖什么

`particle_boundary_scrape`、`particle_data_python` 和 single-precision particle fields 都不以轨道为主要观察量。它们分别回答: 粒子被边界删除后在哪里可见, Python 能否读写粒子状态并触发沉积, 以及单精度归约是否已有可执行的误差检查。

- `particle_boundary_scrape`
- `particle_data_python`
- `particle_fields_diags` single-precision FIXME

阅读这些输入时, 应把验证对象分成三层:

- scraped-particle buffer
- Python runtime attribute / injection / deposition wrapper
- 单精度粒子 diagnostics 误差边界

被删除的粒子: 主容器与边界缓冲区必须同时检查 `particle_boundary_scrape` 的 native 输入配置一个立方体 EB 和一束电子, `analysis_scrape.py` 做两个直接检查:

- 第 40 步还应有 612 个电子
- 第 60 步主 species 中电子数应变成 0

这说明 `ScrapeParticlesAtEB()` 确实把撞到 embedded boundary 的粒子删掉了。

PICMI 变体在 `sim.step(...)` 后构造 `ParticleBoundaryBufferWrapper()`, 并检查:

- EB buffer 中累计粒子数是 612
- `stepScraped` 全都大于 40
- 所有 rank 汇总后的 buffer 粒子总数仍是 612
- `clear_buffer()` 之后 buffer size 回到 0

因此读者应同时检查两个对象:

1. EB scraping 确实把粒子从主容器里删除
2. 删除掉的粒子确实进入了 Python 可访问、可清空的 boundary buffer

Python 操作粒子: 属性、注入与沉积是三条独立操作 `particle_data_python` 没有独立 `analysis.py`, 断言直接写在 PICMI 输入脚本里, 主要步骤是:

- `sim.initialize_warpx()`
- `sim.particles.get("electrons")`
- `add_real_comp("newPid")`
- 在 `beforestep` callback 里持续 `add_particles(...)`
- 再直接断言 `get_real_comp_index(...)`、tile 里的 `newPid` 值, 以及 Python wrapper 暴露的 `deposit_current(...)` 确实能把电流沉到 `current_fp`

`inputs_test_2d_prev_positions_picmi.py` 则验证:

- `warpx_save_previous_position=True`
- `prev_x/prev_z` runtime attributes 确实被加进 species
- `PushPX()` 在推进前确实保存了旧位置

这组输入不检查某条特定的场或轨道物理, 而是检查:

- Python 对 runtime attributes 的增删访问
- Python 注入粒子接口
- Python 手动沉积接口
- Python 到 C++ `save_previous_position` 运行时属性链

源码数据流也应分三层理解: PICMI `sim.particles` 返回 `pybind` 暴露的 `WarpX::GetPartContainer()`; `species` 操作最终调用 `WarpXParticleContainer.cpp` 的 `add_n_particles(...)`、`deposit_current(...)` 或 `deposit_charge(...)`; `beforestep/afterstep` 回调先进入 Python `CallbackFunctions`, 再由 `ExecutePythonCallback(name)` 调用。因而这组输入检查的是 PICMI 外观、`pybind` 粒子对象和 `callback bridge` 的联合行为, 而不是某个孤立 Python helper。

这里有一个明确的覆盖缺口: `test_2d_particle_attr_access_unique_picmi` 虽传入 `--unique`, 输入脚本的 `add_particles(...)` 仍硬编码 `unique_particles=True`, 没有消费 `args.unique`。因此它不能证明 `unique/non-unique` 两种注入语义都已经比较过。

同一条接口线还区分两类 `restart` 读取。 `id_cpu_read` 输入遍历 `pti["idcpu"]`, 用 `unpack_ids/unpack_cpus` 检查累计和为 5050/0, 所以它检查的是粒子 `id/cpu` 的解包读取。 `runtime-components` 输入把 `add_real_comp("newPid")`、`callback` 注入、`get_real_comp_index("newPid")` 和 `picmi.Checkpoint(...)` 放在一起, 说明动态 `component` 可以进入 `checkpoint` 前端; 其 `restart` 目标虽然存在, 但 `analysis/checksum` 都是 OFF, 因此不能据此断言 `restart` 后 `runtime attributes` 已获完整验证。

单精度: 有分析程序不等于已有活跃回归 `particle_fields_diags` 的 `single-precision` 变体要单独理解:

- `analysis_particle_diags_single.py` 已经存在
- 它复用同一实现, 只把容差放宽到 `5e-3`
- 但 `CMakeLists.txt` 里整条 `test_3d_particle_fields_diags_single_precision` 仍被 `# FIXME` 注释掉

因此可以支持的结论是:

- `single-precision particle-field reductions` 的 `analysis` 预案已经随源码提供
- 但它还不是活跃 `regression`

把这三组放在一起, 读者就能避免三种常见误读: 粒子从主容器消失不等于它可被 Python 取回; Python 能写入属性不等于 `restart` 语义已经完整验证; 源码提供单精度分析程序不等于该变体正在被自动回归覆盖。

4.13.11 边界上的粒子: 内建条件、Python 回调与可验证的物理后果

边界不是一个统一的“反射开关”。在 `particle_boundary_scrape` 和 `particle_data_python` 之外, 读者至少要区分三件事: 内建边界怎样更新粒子, `scraped buffer` 能交给用户多少撞击信息, 以及 Python 参数前端是否仍把同一物理模型送进 C++ 主链。下面四组测试分别覆盖这些问题:

- `particle_boundary_interaction`
- `particle_boundary_process`
- `particle_thermal_boundary`
- `plasma_lens_python`

它们不只看最终轨道, 而是分别通过 Python `callback`、`buffer`、`parser` 或 `reduced diagnostics`, 检查边界语义怎样暴露给用户。

自己写边界物理: scraped buffer 记录撞击事件, 不提供反射后的粒子 `particle_boundary_interaction` 的关键点首先不是 `analysis`, 而是输入脚本本身的结构。它不是直接依赖 WarpX 内建的 EB 反射, 而是:

1. 把电子打到一个球形 `embedded boundary` 上
2. 打开 `warp_x_save_particles_at_eb=1`
3. 在 `afterstep` `callback` 里用 `ParticleBoundaryBufferWrapper.get_particle_scraped_this_step(...)` 取出:
 - `deltaTimeScraped`
 - `r/theta/z`
 - `ux/uy/uz`
 - `nx/ny/nz`
4. 在 Python 里手工做镜面反射
5. 再按 `dt - delta_t` 把粒子推进到这一步的末尾并重新 `add_particles(...)`

因此它验证的是: `scraped buffer` 是否给出了足够的几何和时间信息, 让用户实现此例的边界相互作用模型。它记录的是接触时刻、接触位置和局部法向, 而不是可以直接继续推进的“反射后粒子”; 用户回调仍须决定反射、二次发射或吸收后的动量, 并补完 `dt-deltaTimeScraped`。后面的 `analysis.py` 用解析几何反射轨道比较最终 `x/z`, 所以能检验该再注入模型的几何正确性, 不能证明任意用户回调或任意材料模型正确。

内建 domain boundary: 分别检查粒子数、动量和位置 在这些更复杂的边界 `regression` 之外, 源码中还有一个更“教科书式”的最小强基准: `Examples/Tests/boundaries/`。它不碰 `embedded boundary`、不碰 `callback`, 也不依赖 `reduced diagnostics`, 而是把三类 `domain particle boundary` 直接拆开测试:

- x 方向 reflecting
- y 方向 absorbing
- z 方向 periodic

输入里三类 species 都用 MultipleParticles 直接给定初始位置和动量；analysis 则显式检查：

- absorbing species 最终只剩 1 个粒子
- reflecting species 的速度严格翻号
- periodic species 的速度保持不变
- 两类保留粒子的位置都与解析反射/wrap-around 位置逐点一致

因此 particle_boundary 验证的不是应用级“边界大致工作”，而是 ParticleBoundaries_K.H 最核心的三种 domain particle boundary 语义本体。

particle_boundary_process 又不同，它分成两条不同强度的检查。

第一条是 test_2d_particle_reflection_picmi。虽然 analysis = OFF，但输入脚本本身做了直接自检：

- warpx_reflection_model_zhi = "0.5"
- 打开 save_particles_at_zhi/zlo
- 跑完后直接检查：
 - z_hi buffer 中粒子数是 63
 - z_lo buffer 中粒子数是 67
 - z_hi 的 stepScraped 全都等于 4
 - z_lo 的 stepScraped 全都等于 8

这组输入内自检测的是：

- absorbing boundary 上的随机反射模型 parser
- 上下边界 scraped buffer 的分流
- stepScraped 时间戳语义

第二条是 test_3d_particle_absorption。这里 analysis 很简单，只检查：

- 第 40 步仍有 612 个电子
- 第 60 步电子数变成 0

所以它更接近“EB 吸收后主 species 中粒子确实消失”的强吸收断言。

particle_thermal_boundary 也必须和前面已经整理过的 ParticleThermalizer 区分开。这里输入打开的是：

- boundary.particle_lo = thermal thermal
- boundary.particle_hi = thermal thermal
- boundary.<species>.u_th = ...

也就是 domain particle boundary 本身就是 thermal boundary。对应 ParticleBoundaries_K.H 的语义是：先像反射边界一样处理位置，再对法向/切向动量做热化抽样。analysis 并不看单粒子散射角，而是读取 reduced diagnostics 的：

- FieldEnergy
- ParticleEnergy

并要求：

- 场能不能无界增长
- 粒子总能量相对初值偏离不超过 2%

所以它检验的是 thermal particle boundary 在长时间粒子出入边界时的总量稳定性；它并不是单粒子散射角分布的充分验证。

Python 参数前端：同一物理断言，增加的是参数传递覆盖 最后，plasma_lens_python 复用和 native/PICMI plasma-lens 相同的 analysis.py，所以物理断言没有变化，仍然是两颗测试电子穿过 lens 序列后的：

- 最终横向位置
- 最终横向动量

与解析模型一致。

但它新增覆盖的不是物理，而是 front-end：输入不再走 native 文件或 PICMI，而是直接用 pywarpx 参数对象设置：

- `particles.E_ext_particle_init_style = "repeated_plasma_lens"`
- `particles.B_ext_particle_init_style = "repeated_plasma_lens"`
- `particles.repeated_plasma_lens_* = ...`
- 最后 `warpx.init(); warpx.step(...)`

它新增覆盖的是纯 Python 参数前端到 `MultiParticleContainer / GetExternalEBField repeated-plasma-lens` 主链，而不是新的 lens 物理。阅读这些例子时，应按问题选择观察量：写 Python 表面物理就检查撞击几何、法向和剩余时间；确认内建边界就分别检查粒子数、动量和解析位置；确认 Python front-end 就使用与 native 例相同的物理观察量。三种证据不能互相替代。

同一条 Python scraped-buffer 物理链还有一个更直接的边界物理 regression: `secondary_ion_emission`。它不是只拿 buffer 做统计，而是在 `afterstep callback` 里直接用

- `r/theta/z`
- `ux/uy/uz`
- `nx/ny/nz`
- `deltaTimeScraped`

为撞击球形 EB 的离子生成次级电子。analysis 再要求：

1. 固定随机种子下最终恰好产生 2 个电子；
2. 电子反向传播到撞击时刻后，应落在解析球面撞击点附近

所以这组例子说明：`ParticleBoundaryBufferWrapper` 的几何和时间元数据足以支撑该 callback 驱动的二次发射模型，而不只是后处理统计；它不自动给出通用的表面材料模型。

4.13.12 接触记录、PML 粒子与 AMR：观察对象决定验证强度

`embedded boundary`、PML 和 `mesh refinement` 都会改变粒子所在的数值环境，但它们的正确性不能用同一个观察量判断。下面四类例子分别检查接触几何、残余场、组合稳定性和解析场一致性：

- `point_of_contact_eb`
- `particles_in_pml`
- `subcycling_mr`
- `Langmuir multi_mr`

输入、analysis 和 writer 路径表明，它们回答的是四个不同的问题。

接触记录：确认写出的事件几何，而不是只确认粒子消失 `point_of_contact_eb` 的关键是它打开了两套 diagnostics：

- `diag1 = Full`
- `diag2 = BoundaryScraping`

analysis 根本不看主 species 的最终位置，而是直接读：

`diags/diag2/particles_at_eb/`

也就是 `BoundaryScraping` 写出的 scraped-particle openPMD 输出。它比较的是：

- `stepScraped`
- `deltaTimeScraped`
- `x/y/z`
- `nx/ny/nz`

与解析接触点、接触时刻和表面法向是否一致。因此它验证的是：

- EB 接触事件是否被正确记录到 `particles_at_eb`
- `BoundaryScraping` 输出里的几何量和时间量是否正确

这和前面的 `particle_boundary_scrape` 有本质区别：后者更侧重“粒子被删掉并进了 buffer”，而 `point_of_contact_eb` 检查记录下来的接触几何和时间是否正确。

粒子进入 PML：用残余场检验清理是否有效 `particles_in_pml` 则不是看粒子轨道，而是看粒子离域后留下的场。analysis 的核心只有一句：

```
assert max_Efield < tolerance_abs
```

它先取最后一步的 $E_x/E_y/E_z$ ，再要求域内残余电场足够小。对应物理含义是：

- 如果 PML 只吸场不吸粒子，离开的带电粒子会留下伪电荷和伪场
- 打开 `warpx.pml_has_particles = 1` 后，这个 spurious field 必须被压到足够低

2D/3D 与 MR 版本共享同一 analysis，只是 tolerance 按：

- 维度
- `max_level`

分开设置。因此它检验的是 particle-aware PML 的 residual-field cleanup，而不是单纯的 PML 场反射率；比较不同维度或不同 AMR level 时，不能忽略容差本身的差异。

AMR 与 subcycling: checksum 基线和解析基准的证据强度不同 `subcycling_mr` 又更弱一层。它当前没有独立 analysis，只有 checksum。输入里同时打开了：

- `warpx.do_subcycling = 1`
- `amr.max_level = 1`
- moving window
- driver / beam / plasma continuous injection
- `particles.deposit_on_main_grid = plasma_e plasma_p`
- `n_current_deposition_buffer = 0`
- `n_field_gather_buffer = 0`

所以它只能建立 AMR + subcycling + moving window + continuous injection + deposit_on_main_grid 这一组合的 checksum 基线。由于没有独立观察量，它不能分辨 injection、gather、deposit 或 coarse/fine 同步的任何一个步骤，也不能写成独立的 refined-injection analysis。

最后，Langmuir multi_mr 与 subcycling_mr 正好相反。它虽然也在测 MR，但并不只是 checksum，而是继续复用 `analysis_2d.py` 的强验证：

1. 取 E_x/E_z
2. 与解析 Langmuir-wave 场解比较
3. 在满足条件时，再检查 $\text{div}E$ 与 ρ/ϵ_0 的相对误差

而不同 MR 变体测的是不同 refinement 组合：

- `mr`
 - `amr.max_level = 1`
 - `amr.ref_ratio = 4`
- `mr_anisotropic`
 - `amr.ref_ratio_vect = 4 2`
- `mr_maxlevel2`
 - `amr.max_level = 2`
- `mr_momentum_conserving`
 - gather scheme 改成 momentum-conserving
- `mr_psatd`
 - solver 改成 PSATD

因此它可支持“这些已覆盖的 MR 组合没有破坏该解析 Langmuir 粒子-场基准”，不能外推为任意 AMR 配置、任意注入模型或所有 solver 的证明。

把四类例子放在同一张心理地图中最有用：接触 writer 看几何与时间，particle-aware PML 看残余场，subcycling 例提供组合 checksum，而 Langmuir multi-MR 保留解析场和电荷关系的强基准。观察对象不同，能得出的结论也必须不同。

实际阅读或设计自己的测试时，可以按下列顺序选择观察量：

1. 若问题是“粒子何时、何处触及表面”，优先读取 `stepScraped`、`deltaTimeScraped`、位置和法向；只数剩余粒子无法判断接触几何。
2. 若问题是“离开物理域的带电粒子是否留下数值污染”，比较 PML 后时刻的残余 $E_x/E_y/E_z$ ，不要把它与入射波的反射率混为一谈。
3. 若问题是“一个复杂 AMR 输入能否保持可重复输出”，checksum 是必要的基线；若要判断场和电荷关系是否正确，则必须再选择有解析解或守恒量的基准，如 Langmuir multi-MR。
4. 若某个例子只提供 checksum，读者应把它当作该配置组合的漂移报警器，而不是将其当成每一条粒子、网格和同步路径都已经独立证明。

这套区分也解释了为什么一个通过的测试集合仍可能留下开放边界：它们的输入可以相似，但被比较的对象、容差和可外推的范围并不相同。

4.13.13 Embedded boundary: 从几何、吸收和解析场选择验证量

要验证 embedded boundary, 先要确定你想验证的是几何表示、场的边界条件、粒子吸收, 还是 Python 对几何数组的访问。下列例子涉及这些不同问题:

- particle_absorbing_boundary/plot_2d.py
- particle_absorbing_boundary/plot_phase.py
- embedded_boundary_cube
- embedded_boundary_rotated_cube
- embedded_boundary_diffraction
- embedded_boundary_em_particle_absorption
- embedded_boundary_python_api
- electrostatic_sphere_eb
- scraping

先分清工具和断言: plot_2d.py 与 plot_phase.py 都不是 regression analysis。

它们只是:

- 画 2D full diagnostics 的 Ez slice
- 画 PhaseSpaceElectrons reduced diagnostic 的相图

它们只是 visualization helper, 不是物理断言脚本; 图像适合帮助读者定位异常, 却不能代替定量误差或守恒检查。

PEC cavity: 用解析本征模检验几何与场边界 第一类是 cavity 模态解析对照:

- embedded_boundary_cube
- embedded_boundary_rotated_cube

analysis_fields.py、analysis_fields_2d.py 和 analysis_fields_3d.py 都是显式构造 PEC cavity 的解析本征模, 再比较 By、Bz 或 Ey/c 的相对 L2 误差。区别只是:

- cube: 轴对齐 cavity
- rotated_cube: 旋转后的 cavity, analysis 里要先把坐标和场分量反旋回解析坐标系
- cube_macroscopic: 同一模态, 但频率按介质 epsilon_r 修正

这两组例子回答的是“给定的 EB 几何和 PEC 场边界能否维持指定本征模”。旋转例还把坐标和场分量转回解析坐标系, 因而同时约束几何方向和矢量分量的处理; 它们不直接验证带电粒子撞击或任意复杂几何。

衍射与 Python API: 一个测物理图样, 一个测几何数组

- embedded_boundary_diffraction
- embedded_boundary_python_api

embedded_boundary_diffraction/analysis_fields.py 读取 RZ 下的 Ex, 提取衍射图样第一极小值半径, 再与 Airy pattern 的

$$\theta \sim 1.22\lambda/d$$

预测比较。因此它测的是 EB 产生的衍射图样是否在 Airy 第一极小值上与解析预测相符, 而不是对每个网格单元的几何误差证明。

embedded_boundary_python_api 更特殊。CMake 里虽然 analysis = OFF, 但 PICMI 输入脚本本身在运行时就会读取:

- edge_lengths
- face_areas

并在三个中间切片上重建 cavity 的 perimeter 和 area, 再和解析几何值比较。它虽没有独立 analysis.py, 却不是单纯 checksum: 输入本身验证了 PICMI wrapper 返回的 edge-length 与 face-area 几何量。这个检查不自动覆盖任意 EB 形状、所有 AMR level 或后续场演化。

吸收与 scraping: 分别观察伪电荷和粒子记账

- embedded_boundary_em_particle_absorption
- scraping

embedded_boundary_em_particle_absorption/analysis.py 做的不是看粒子是否消失, 而是把 divE 做时间平均, 去掉沿 EB 传播的真实波动分量后, 检查是否还残留静态伪电荷。因此它检验的是 EM 粒子吸收不会在该测试条件下积累非物理电荷, 而不是一般的 divE-rho 全时空闭合证明。

而 scraping/analysis_rz.py 与 analysis_rz_filter.py 测的是另一条 writer 合同:

- 最终剩余粒子数是否正确
- remaining + scraped = initial 是否逐步成立
- scraped buffer 中的 id 是否和初始全集闭合
- 打开 plot_filter_function 后, 是否真的只记录 $z > 0$ 半域的 scraped particles

因此 scraping 关注的是 BoundaryScraping 的粒子记账和 plot_filter_function 的选择语义。它应与前一例的场量检查分开阅读: 前者能发现 particle identity 或输出选择错误, 后者能发现吸收后的静态场污染。

静电球: 用解析势与电荷把三维、RZ 和 AMR 分层比较 electrostatic_sphere_eb 至少分成三层:

1. 3D analysis.py
 - reduced diagnostic eb_charge.txt
 - 理论球导体电荷

$$q = 4\pi\epsilon_0\phi_0 R$$

- eb_covered 场是否满足内 1 外 0

2. RZ analysis_rz.py

- 比较解析

$$\phi(r) = A + B \log r, \quad E_r(r) = -B/r$$

3. RZ analysis_rz_mr.py

- 把同一 \phi/Er 对照扩展到每个 refinement level

只有 inputs_test_3d_electrostatic_sphere_eb_mixed_bc 没有独立 analysis, 因此它只提供 mixed-BC 配置的 checksum 基线。其余例子分别把导体总电荷、覆盖区域、RZ 解析势/径向场和各 refinement level 的误差作为观察量; 不能把通过的 checksum 当作解析 phi/Er 比较, 也不能把一个对称球的结果外推为任意电极几何。

综上, embedded-boundary 验证至少应同时间四个问题: 几何是否表示正确, 场是否满足已知解析解, 吸收是否留下数值电荷, Python 或 writer 是否导出了预期的几何/粒子数据。不同问题需要不同的输出量, 任何一个单独通过都不等于整个 EB 物理闭合。

4.14 QED: 先分清 source、product 与产生机制, 再谈参数

从 Particles/ 入口再往下看, WarpX 当前的 QED 主链至少分成三种完全不同的事件类型:

1. Quantum Synchrotron: lepton source 产生 photon product
2. Breit-Wheeler: photon source 产生 electron/positron products
3. Schwinger: 不以 source species 为起点, 而是直接由场在网格上创建对

4.14.1 事件开始前: 谁是 source, 谁保存统计状态, 谁接收 product

PhysicalParticleContainer.cpp 在构造期先决定:

```
pp_species_name.query("do_qed_quantum_sync", m_do_qed_quantum_sync);
if (m_do_qed_quantum_sync) {
    AddRealComp("opticalDepthQSR");
}

pp_species_name.query("do_qed_breit_wheeler", m_do_qed_breit_wheeler);
if (m_do_qed_breit_wheeler) {
    AddRealComp("opticalDepthBW");
}
```

这不是实现细节, 而是 QED 与 field ionization 一样, 先把“事件统计状态”写进 species 的 runtime attribute 系统。读者随后追踪每一次事件时, 应先找到:

- opticalDepthQSR
- opticalDepthBW

这两个持久属性。

与之配套, PhotonParticleContainer.cpp 又明确禁止 photon species 再开 quantum synchrotron:

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(  
    test_quantum_sync == 0,  
    "ERROR: do_qed_quantum_sync can't be enabled for photon particles!");
```

因此, WarpX 在 species 层已经把“谁提供待转化粒子、谁保存 optical depth、谁接收新粒子”分开; 只有先分清这些角色, 才能正确解读后续的粒子数、动量与权重变化。

4.14.2 初始化只准备采样条件, 事件要在后续时间步发生

MultiParticleContainer::InitMultiPhysicsModules() 在 InitData()/PostRestart() 前做:

```
mapSpeciesProduct();  
CheckQEDProductSpecies();  
InitQED();
```

这里三步分别对应:

1. 把 qed_quantum_sync_phot_product_species、qed_breit_wheeler_ele_product_species、qed_breit_wheeler_pos_product_species 从字符串映射成容器索引;
2. 检查 product species 类型是否正确;
3. 创建 QuantumSynchrotronEngine / BreitWheelerEngine 并按 qed_qs.*、qed_bw.* 初始化 lookup tables.

因此 InitQED() 不是“提前跑一遍 QED”, 而是在模拟开始或 restart 后把下列采样条件固定:

- 谁参与 quantum synchrotron
- 谁参与 Breit-Wheeler
- 表从 builtin/load/generate 哪条路径来

这一区分决定了排错顺序: 初始化失败时先检查 product species 与 lookup table; 粒子数或能谱异常时才追踪后续 push 与 event pass.

4.14.3 时间位置: 在 field ionization 后、用户 injection 前处理 QED

顶层主循环 WarpXEvolve.cpp 里, 多物理顺序非常直接:

```
doFieldIonization();  
  
#ifdef WARPX_QED  
doQEDEvents();  
mypc->doQEDSchwinger();  
#endif  
  
ExecutePythonCallback("particleinjection");  
OneStep(cur_time, dt[0], step);
```

这说明 QED 事件发生在:

- field ionization 之后
- 用户 particleinjection callback 之前
- 正常 OneStep() 之前

这说明 QED 不像 collisions 那样嵌在 split-momentum push 的组织里, 而是更早地消费当下的 Efield_aux/Bfield_aux。因此, 若用户 callback 在 particleinjection 中新增粒子, 这些粒子不会在同一轮的 QED event pass 中被处理。

4.14.4 两条 source-to-product 事件链: 转化时同时更新 source 与创建 product

doQedQuantumSync() 的骨架是:

```
const auto Filter = phys_pc_ptr->getPhotonEmissionFilterFunc();  
const auto CopyPhot = copy_factory_phot.getSmartCopy();
```

```

auto Transform = PhotonEmissionTransformFunc(
    m_shr_p_qs_engine->build_optical_depth_func(),
    pc_source->GetRealCompIndex("opticalDepthQSR") - pc_source->NArrayReal,
    m_shr_p_qs_engine->build_phot_em_func(),
    pti, lev, Ex.nGrowVect(), ...);

filterCopyTransformParticles<1>(
    *pc_product_phot, dst_tile, src_tile, np_dst,
    Filter, CopyPhot, Transform);

doQedBreitWheeler() 则是双 product 版本:

const auto Filter = phys_pc_ptr->getPairGenerationFilterFunc();
const auto pair_gen_func = m_shr_p_bw_engine->build_pair_func();

auto Transform = PairGenerationTransformFunc(pair_gen_func,
    pti, lev, Ex.nGrowVect(), ...);

filterCopyTransformParticles<1>(
    *pc_product_ele, *pc_product_pos,
    dst_ele_tile, dst_pos_tile, src_tile,
    np_dst_ele, np_dst_pos,
    Filter, CopyEle, CopyPos, Transform);

```

这两条链有一个关键共同点: QED 不是只在 product species 上增加一批新粒子, 而是同时做三件事:

1. 用 source 粒子的 optical depth 判断是否触发事件
2. 在 Transform 里读取主网格场和 external particle fields
3. 同时更新 source 状态并创建 product particles

这与前面 field ionization 的 filterCopyTransformParticles 思路相近, 但它依赖 QED engine 表和 optical-depth 统计变量。验证时因此不能只计数 product, 还要同时检查 source 的剩余状态与能量/动量变化。

4.14.5 Schwinger: 由网格场直接创建粒子对, 不存在 source species

doQEDSchwinger() 不再从某个 source species 复制, 而是直接在网格上用:

- Efield_aux/Bfield_aux
- Schwinger 激活区域
- filterCreateTransformFromFAB<1>(...)

生成 ele_schwinger 和 pos_schwinger。而且它目前硬性要求:

- collocated grid 或 momentum-conserving gather
- 无 mesh refinement
- 非 RZ
- 非 1D

所以不能把 Schwinger 和前两条 source-to-product 路径混写成同一类机制: 它的观察起点是激活区域内的场和 pair 产额, 而不是某一 source species 的 optical depth。

4.14.6 三条机制分别该看什么: 数目之外还要看权重、方向和守恒

- analysis_quantum_sync.py:
 - 检查 photon 数、权重、发射方向、能谱、以及 source/product optical depth 分布;
 - 对应 Quantum Synchrotron 的主合同。
- analysis_breit_wheeler_core.py:
 - 检查 pairs 数、残余 photon 动量、单事件能量守恒、pair 能谱和 optical depth;
 - 对应 Breit-Wheeler 的 product-species 合同。
- analysis_schwinger.py:
 - 检查理论率对应的 pair 数窗口, 以及电子/正电子权重数组一致性;
 - 对应 Schwinger 的强场真空对产生合同。

因此“QED 测试通过”不是单一结论：Quantum Synchrotron 需要同时看 photon 发射与 lepton 的统计状态，Breit-Wheeler 需要比较残余 photon、pairs 与单事件守恒，Schwinger 则以理论率窗口和电子/正电子权重配对为主。三条路径必须分别验证，不能用其中一条替代另外两条。

4.14.7 事件何时发生：先演化 optical depth，再决定是否创建 product

把入口层再往下读到 QEDPhotonEmission.H、QEDPairGeneration.H 和两个 engine wrapper，会发现 QED 事件触发并不是“在 event pass 里直接抽一次随机数”。

PhotonEmissionFilterFunc 和 PairGenerationFilterFunc 都只做一件事：

```
return (opt_depth < 0.0_rt);
```

这表示 event pass 不负责从零开始抽样，而是消费先前 push 已更新的状态。真正的统计演化发生在更早的 push 阶段：

- PhysicalParticleContainer::PushPX() 里先用 QuantumSynchrotronEvolveOpticalDepth 推进 opticalDepthQSR
- PhotonParticleContainer::PushPX() 里先用 BreitWheelerEvolveOpticalDepth 推进 opticalDepthBW

只有当 optical depth 在 push 中被推进到负值，后面的：

- doQedQuantumSync()
- doQedBreitWheeler()

才会在 event pass 里通过 filterCopyTransformParticles 触发事件。调试“没有产生 photon/pair”时，应先检查 chi、时间步和 optical depth 是否能跨过零，而不是只检查 product species 是否存在。

4.14.8 分工边界：WarpX 提供粒子与场，PICSAR-QED 提供采样内核

QuantumSyncEngineWrapper.H 和 BreitWheelerEngineWrapper.H 不是又实现了一遍 QED 理论，而是把 PICSAR-QED core 包成 WarpX 可在 GPU kernel 中调用的 functor：

- QuantumSynchrotronEvolveOpticalDepth 最终调用 pxx_qs::evolve_optical_depth
- QuantumSynchrotronPhotonEmission 最终调用 pxx_qs::generate_photon_update_momentum
- BreitWheelerEvolveOpticalDepth 最终调用 pxx_bw::evolve_optical_depth
- BreitWheelerGeneratePairs 最终调用 pxx_bw::generate_breit_wheeler_pairs

WarpX 在这一层真正负责的是：

1. gather 主网格与 external particle fields
2. 提供 source/product species 的 SoA 指针
3. 存取 opticalDepthQSR/BW
4. 组织 filterCopyTransformParticles

这条分工线给出了排错边界：gather、species 属性、tile 调度和 product 容器属于 WarpX；给定 chi 后的概率演化与动量采样属于 PICSAR-QED。不要把 table 或采样偏差误诊为 WarpX 的粒子容器问题，也不要错误的 field gather 归咎于采样器。

4.14.9 source 的命运不同：辐射会保留 lepton，成对产生会消耗 photon

两条 event kernel 都会先 gather E/B，但 source 处理方式并不一样：

- PhotonEmissionTransformFunc 会原地改写 source lepton 动量，并把 source optical depth 重新抽样初始化；
- PairGenerationTransformFunc 会生成 electron/positron 两个 product，并把 source photon 直接标记成 invalid。

这解释了为什么两类分析需要不同的守恒与统计检查：

- analysis_quantum_sync.py 要重点检查 source/product optical-depth 分布能否继续保持指数；
- analysis_breit_wheeler_core.py 要重点检查 residual photons、丢失 photon 数和新 pairs 数之间的对应关系。

4.14.10 lookup table 是采样器的一部分，不是可有可无的附属文件

如果继续往 QEDInternals/QuantumSyncEngineWrapper.cpp 和 BreitWheelerEngineWrapper.cpp 读，会发现 WarpX 当前的 QED wrapper 内部都不是只持有一张表，而是：

- 一张 dndt 表
- 一张真正用于事件采样的二维表
- 一个最小 chi 门槛

更具体地说：

- Quantum Synchrotron 持有 `m_dndt_table + m_phot_em_table + m_qs_minimum_chi_part`
- Breit-Wheeler 持有 `m_dndt_table + m_pair_prod_table + m_bw_minimum_chi_phot`

因此前面看到的：

- `build_optical_depth_functor()`
- `build_phot_em_functor()`
- `build_pair_functor()`

都会先要求 `m_lookup_tables_initialized == true`。这说明 QED kernel 不是“有 wrapper 就能跑”，而是必须先完成表的生命周期；出现初始化或运行时 table 错误时，应先确认模式、输入文件和最小 `chi`，再解释粒子统计结果。

`InitQED()` 里 `qed_qs.lookup_table_mode` 和 `qed_bw.lookup_table_mode` 只允许三种模式：

- `builtin`
- `load`
- `generate`

其中：

- `builtin` 直接使用 `wrapper.cpp` 中硬编码的低分辨率测试表
- `load` 从外部二进制文件读 raw bytes，再反序列化成两张子表
- `generate` 运行时现生成表，但最后仍然导出成同一种 raw binary 格式，并通过 MPI 广播给所有 rank

所以 `generate` 和 `load` 的真正共同点是：后续 kernel 消费的不是“生成态”或“加载态”的 C++ 对象，而是同一种序列化结果重新装配出来的 wrapper。

这一点在 `QuantumSyncGenerateTable()` / `BreitWheelerGenerateTable()` 里最清楚：

1. 只在 `IOProcessor` 上按 `tab_dndt_*`、`tab_em_*` 或 `tab_pair_*` 生成两张子表
2. 调 `export_lookup_tables_data()` 导出 raw bytes
3. 把 bytes 写入 `save_table_in`
4. 再把同一份 bytes 广播给所有 rank
5. 非 `IOProcessor` 用 `init_lookup_tables_from_raw_data(...)` 重建 wrapper

因此，完整主链应写成：

```
runtime attributes / product species
-> InitQED()
-> builtin/load/generate tables
-> build device functors
-> PushPX() 中演化 optical depth
-> event pass 中查表并生成 product particles
```

所以 examples 中的 `lookup_table_mode = builtin / load / generate` 不只是输入模板，而是直接选择 QED 采样器的可执行上游；同一物理案例切换模式时，仍须核对表分辨率和来源是否适合目标精度。

4.14.11 不要被共同的 photon 名称误导：强场 QED 与碰撞 QED 是两棵树

继续沿源码往下追，会碰到一组很容易被误写进同一章法的名字：

- `QedChiFunctions`
- `do_qed_virtual_photons`
- `linear_breit_wheeler`
- `linear_compton`

它们都带着 QED / photons 的标签，却不共享同一套事件骨架。选模型前，先问过程由局部强场驱动，还是由 cell 内两粒子配对驱动。

`QedChiFunctions.H` 本身只有两个薄包装：

- `QedUtils::chi_ele_pos(...)`
- `QedUtils::chi_photon(...)`

它们只把 SI 单位下的动量与 E/B 场交给 PICSAR 的 `chi` 公式。当前源码里，这两个函数主要只服务三类地方：

1. PushSelector.H 里 RR 与 QED 联动时的 chi 阈值判断
2. QuantumSyncEngineWrapper 的 optical-depth 与 photon-emission 采样
3. BreitWheelerEngineWrapper 的 optical-depth 与 pair-generation 采样

因此 QedChiFunctions 属于强场 QED 树:

```
gather E/B
-> compute chi
-> evolve optical depth
-> lookup-table sampling
-> update source and create products
```

但 virtual photons 走的不是这条路。

PhysicalParticleContainer.cpp 允许 lepton species 打开:

- do_qed_virtual_photons
- qed_virtual_photon_species_name
- qed_virtual_photons_do_beam_size_effect

随后 CollisionHandler::doCollisions() 在真正做任何碰撞之前, 会统一先调用:

```
collision::binarycollision::virtualphotons::GenerateVirtualPhotons(mypc);
```

这说明 virtual photons 的运行位置在 collision 调度层, 而不是:

- InitQED()
- doQedEvents()
- QEDPhotonEmission.H
- QEDPairGeneration.H

GenerateVirtualPhotons() 的语义是: 每个 coarse step 都从 lepton species 重新采样一批辅助 photon species; 旧 virtual photons 会被下一步覆盖。3D 下若打开 beam-size effect, 还会在垂直于动量的平面内给这些虚光子加上有限半径位移。

因此 virtual photons 不是强场 QED 事件生成、随后继续 push 的 product photons, 而是碰撞模块每个 coarse step 重建的辅助 photon 分布。若分析关注其空间分布或谱, 应以 virtual-photon 专用输出和 analysis 为准, 不能借用 opticalDepthBW 的解释。

接下来的 linear_breit_wheeler 与 linear_compton 也继续留在碰撞树里。

CollisionHandler.cpp 对 type = linear_breit_wheeler 的分派是:

```
std::make_unique<
    BinaryCollision<LinearBreitWheelerCollisionFunc, ParticleCreationFunc>
>(...
```

这说明它不走前面的:

- doQedBreitWheeler()
- PairGenerationFilterFunc
- BreitWheelerEngineWrapper
- opticalDepthBW

而是走另一套碰撞骨架:

```
optional GenerateVirtualPhotons()
-> BinaryCollision pairing in each cell
-> p_mask / reaction weight
-> ParticleCreationFunc
```

LinearBreitWheelerCollisionFunc 与 LinearComptonCollisionFunc 自己也写得很清楚: 它们实现的是 Higginson 风格的 binary-collision 算法, 控制参数是:

- event_multiplier
- probability_threshold
- probability_target_value

而不是强场 QED 那套:

- `chi_min`
- `lookup_table_mode`
- `photon_creation_energy_threshold`

因此, WarpX 至少有两棵名字都带 QED / photons 的树:

1. 强场 QED / `ElementaryProcess`
 - `QedChiFunctions`
 - `optical depth`
 - `tables`
 - `filterCopyTransformParticles`
2. 碰撞 QED / `BinaryCollision`
 - `optional virtual photons`
 - `cell-local pairing`
 - `reaction masks`
 - `ParticleCreationFunc`

相应地, 测试也分成两组不同的证据:

- `analysis_quantum_sync.py`, `analysis_breit_wheeler_core.py`, `analysis_schwinger.py`
– 验证强场 QED 主链
- `analysis_virtual_photons.py`, `analysis_beamsize_effect.py`, `analysis_many_photons.py`
– 验证 virtual-photon 采样与 linear Breit-Wheeler 碰撞分叉

实际选择模型时, 可用一个简短判据: 若观察量是强场中的 `chi`、`optical depth`、`emission spectrum` 或 `source photon/lepton` 的转化, 沿 `ElementaryProcess` 树阅读; 若观察量是两个粒子或虚光子的 `cell-local` 配对、`reaction weight` 或碰撞概率, 沿 `BinaryCollision` 树阅读。两棵树的输入参数、随机变量、守恒检查与适用范围不能互换。

4.15 本章结论

粒子推进器不等于孤立的 Boris 公式。一次 WarpX 粒子更新同时连接时间层、场插值、动量更新、位置更新、沉积、边界和多物理创建。读者面对异常轨道、错误粒子数或不守恒的场时, 可以按下列顺序缩小问题:

1. 先确定观察量和时间层。位置、机械动量、`rho`、`current`、`scraped buffer` 和 `reduced diagnostic` 不一定在同一时刻; 先写清比较的是单粒子轨道、粒子数、场残余还是守恒量。
2. 再追 `tile` 的带电粒子主链。`PushParticlesandDeposit()` 经 `MultiParticleContainer::Evolve()` 进入 `PhysicalParticleContainer::Evolve()`; `rho component 0` 保存旧时间层, `doGatherShapeN()` 取得粒子场, `doParticleMomentumPush()` 选择 Boris、Vay、Higuera-Cary 或 RR, `UpdatePosition()` 用半步速度更新位置, 随后沉积 `current` 与下一时间层的 `rho component 1`。
3. 把边界与 AMR 当成独立观察问题。粒子消失、`scraped` 接触几何、PML 残余场和 `coarse/fine` 组合 `checksum` 分别需要不同输出; 其中只有解析基准或守恒量比较能支持更强的物理结论。
4. 最后选择正确的产生模型。强场 QED 沿 `chi` -> `optical depth` -> `lookup table` -> `source/product` 路线读取; `virtual photons`、linear Breit-Wheeler 与 linear Compton 则沿 `cell-local BinaryCollision` 路线读取。两者不能以相同参数或相同守恒检查互相替代。

这条路线也给出了后续章节的接口: 第 5 章解释 `DepositCurrent/DepositCharge()` 的守恒沉积, 第 6 章解释场如何推进, 第 7 章解释 PML、AMR 与边界, 第 8 章说明如何把上述观察量写成可判断的 `diagnostics`。

4.16 练习与复现实验

1. **pusher 对照题:** 比较 Boris、Vay 与 Higuera-Cary 的同类案例结果, 解释为什么 Boris 的大位移不能简单归结为“代码运行失败”, 而应联系 `force-free relativistic pusher` 的适用条件判断。
2. **源码定位题:** 从 `PhysicalParticleContainer::Evolve()` 定位 `doGatherShapeN()`、`momentum push`、`UpdatePosition` 和 `current/charge deposition`, 画出一粒子 `tile loop` 的四个时间层节点。
3. **最小复现实验:** 运行官方 `particle_pusher` 或 `photon_pusher analysis`, 并分别记录位置、动量和是否发生电流沉积的观测量; 说明带电 `pusher` 的位置误差和无质量 `photon` 的位置/动量误差为何不能共用同一容差。
4. **QED 路线题:** 选择 `Examples/Tests/qed/` 的 `quantum-synchrotron` 或 Breit-Wheeler case, 列出 `source`、`product`、`opticalDepthQSR/BW`、主观察量和一个不能由该 case 证明的结论; 再说明为什么同一张表不能用 `linear_breit_wheeler` 的碰撞参数替代。

5. 电荷、电流沉积与形函数：源项如何回到网格

上一章从粒子侧解释了 field gather 和 pusher。本章看反方向：粒子推进后如何把电荷和电流交回网格。沉积不是输出或后处理，而是 PIC 离散方程的一部分。它直接决定离散连续性方程、Gauss 定律误差、数值噪声、guard cell 需求和 AMR fine/coarse 同步方式。

本章按一条从粒子状态到求解器源项的因果链展开：先用形函数定义一个粒子如何采样网格；再区分旧电荷、半步电流和新电荷的时间层；随后进入 WarpXParticleContainer::DepositCurrent() 与 DepositCharge(), 比较 Direct、Esirkepov、Villasenor 和 Vay 的局部构造；最后沿 WarpX::SyncCurrentAndRho() 检查 guard cells、物种求和、AMR 和边界如何把 tile 局部写入变成场求解器可消费的 ρ/J 。

需要回查实现时, 优先从 Source/Particles/ShapeFactors.H 的 Compute_shape_factor / Compute_shifted_shape_factor, Source/Particles/WarpXParticleContainer.cpp 的 DepositCurrent() / DepositCharge(), Source/Particles/Deposition/ 的四类 current kernel, 以及 Source/Evolve/WarpXEvolve.cpp 的 SyncCurrentAndRho() 开始。文件路径和函数符号表达算法职责；不要把某次源码的行号或笔记目录当成沉积算法本身。

Examples/Tests/langmuir/analysis_utils.py 与 Examples/Tests/vay_deposition/analysis.py 提供代表性的 divE-rho/epsilon_0 consumer, 但每个 consumer 都只覆盖给定的几何、时间层和输入条件。本章会持续区分：公式解释离散构造，源码定位实现分派，而案例分析只检验其指定条件下的结果。

本章引用 Esirkepov 2001 的作者预印本来解释 $W^1/W^2/W^3$, Eq. (23) 和二阶 spline 的构造；CPC 发表版的书目信息和摘要已核实，但没有可逐页核对的 publisher PDF。Villasenor-Buneman 1992 则可作为 crossing-based deposition 的全文来源。两条路径的论文、源码和运行证据将在 5.11 与 5.14 分层说明，不能相互替代。

Birdsall-Langdon 在 Plasma Physics via Computer Simulation 第一分卷的 4-6 到 4-8 给了一个很硬的理论边界：只要粒子通过空间网格被观测和求场，它就不再表现成零厚度 point particle，而必须被理解成具有有效形状因子 $S(x)$ 、频域响应 $S(k)$ 的 finite-size cloud。这样一来，shape order 不是单纯“更光滑的插值公式”，而是同时改写三件事：

1. 粒子如何把 ρ/J 交回网格；
2. 网格场如何再被 gather 回粒子；
3. 粒子间短程相互作用怎样被平滑，以及 grid force / aliasing 从哪里进入离散系统。

因此本章不能把 shape、deposition 和 finite-grid effects 分开讲。对 WarpX 来说，ShapeFactors.H、charge/current deposition kernel、AMR coarse-fine buffer 和后续的 current correction 共同实现的，正是这条 Birdsall 已经提前写清的离散合同。

Birdsall-Langdon 在 Chapter 8 又把这条线再往前推了一步：aliasing 不是“把结果做 FFT 之后才看见的谱污染”，而是在

$$\rho(k) = q \sum_p S(k_p) n(k_p), \quad k_p = k - pk_g$$

这一步就已经发生。也就是说，particle continuum information 被 sample 到 grid 上时，不同 aliases 已经混进同一个 $\rho(k)$ 。这正是为什么本章既要讲 shape factor，也必须讲 finite-grid effects 和 sampled density 的合同；否则只讨论 ShapeFactors.H 的局部公式，会把真正的 alias source 讲窄。

Birdsall 到 Chapter 10 又把这条线往前压了一步：energy-conserving 和 momentum-conserving 不是同一套沉积/受力合同上“谁更准一点”的实现差别，而是两条不同的离散守恒路线。前者把离散场能量

$$W_E = \frac{V_c}{2} \sum_j \rho_j \phi_j$$

当成第一性对象，再从 $-\partial \text{W}_E / \partial \text{x}_i$ 构造粒子受力；后者则保持更常见的 grid force / zero-total-force 结构。因此本章后面讨论 ShapeFactors.H、charge/current deposition 和 sampled density 时，必须把它们同时视作“守恒合同”的一部分，而不是孤立的插值技术细节。

Birdsall 在 Chapter 13 又把这条 shape-factor 主线往“长期数值健康度”推进了一步：对 thermal plasma, weighting order 与 short-wavelength smoothing 不只是决定瞬时噪声有多平滑，还会直接改写 self-heating time τ_H 。一维结果和 Hockney 的 2d2v 长时间实验都说明：

- 更高阶 particle shape 会更强地削弱 alias coupling；
- 更激进的高波数截断会进一步拉长 τ_H ；
- 但 collisional slowing-down time τ_s 未必同步等比例变化。

所以本章讨论 shape order 时, 不能只写“更高阶更光滑、噪声更低”。更准确的说法是: shape order、cloud width 和 smoothing policy 一起决定了热等离子体多久会因为 finite-grid effects 累积出不可忽略的数值自热。

Hockney 1971 的可用证据限于摘要级: 它支持 tau_coll/tau_pe、电场能量涨落、(omega_pe Delta t)_opt 和 K_2 的定量路线, 但不能支持对正文或图表的逐段解读。Abe et al. 1975 的摘要级 sigma(K_g) 与 correlation-time 观测补充短时 fluctuation 的统计量, 不能替代 Hockney 的长时 tau_H 结论。

两篇 1974 摘要级来源补足了 particle-mesh 的历史定位: QPM/PPPM 把 Gaussian cloud、potential shaping、mesh noise 和 sub-mesh resolution 放到同一模型谱系; force shaping 则把 NGP/CIC/九点 charge-sharing hierarchy、potential correction 与 force-law angular anisotropy 联系起来。它们没有全文支撑, 因而不能把摘要中的数值或设置当作 WarpX kernel 的复现结果。

Dawson 1983 则把同一条线往前退回到更基础的动机层: finite-size particles 的第一性目的不是“插值更方便”, 而是先把 point-charge 的近距离大冲量软化掉, 从而压低不想要的 collisional effects, 同时保住长程 Coulomb collective behavior。也正因为粒子已经被改写成有限尺寸 cloud, 空间上比 cloud 更细的电荷起伏本来就不再分辨, grid 才成为一种自然的 coarse-grained source representation。这条综述级表述很适合放在本章开头, 因为它比直接从 ShapeFactors.H 展开更清楚地交代了: shape、charge sharing 和 sampled density 本来就是同一个物理建模决定的三个侧面。

5.1 电荷沉积的基本形式

对宏粒子 p , 电荷、权重和位置分别为 $q_p, w_p, \mathbf{x}_p$ 。最基本的网格电荷沉积是

$$\rho_i = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(\mathbf{x}_p).$$

这里 S_i 是粒子形函数对网格自由度 i 的权重。形函数阶数越高, 粒子影响的网格范围越大, 噪声通常越低, 但 stencil、guard cell 和通信成本也更高。

WarpX 中 shape 阶数通过 nox/noy/noz 等内部变量进入 gather 和 deposition 分派。Source/Particles/ShapeFactors.H 定义 0–4 阶权重; WarpXParticleContainer::DepositCurrent() 再根据 WarpX::nox 与 CurrentDepositionAlgo 选择 doEsirkepovDepositionShapeN<N>()、doVillasenorDepositionShapeN* <N>()、doVayDepositionShapeN<N>() 或 direct doDepositionShapeN<N>()。因此读者应把 nox/noy/noz 看成“shape order 的全局分派键”, 而不是某一个 kernel 的局部参数。

5.2 ShapeFactors.H: WarpX 实际使用的 0 到 4 阶形函数

形函数不是抽象参数。WarpX 在 Source/Particles/ShapeFactors.H 的 Compute_shape_factor 中直接给出 0 到 4 阶的权重和最左网格点索引:

```
template <int depos_order>
struct Compute_shape_factor
{
    template< typename T >
    AMREX_GPU_HOST_DEVICE AMREX_FORCE_INLINE
    int operator()(
        T* const sx,
        T xmid) const
    {
        if constexpr (depos_order == 0){
            const auto j = static_cast<int>(xmid + T(0.5));
            sx[0] = T(1.0);
            return j;
        }
        else if constexpr (depos_order == 1){
            const auto j = static_cast<int>(xmid);
            const T xint = xmid - T(j);
            sx[0] = T(1.0) - xint;
            sx[1] = xint;
            return j;
        }
        else if constexpr (depos_order == 2){
            const auto j = static_cast<int>(xmid + T(0.5));
```



```

    const T xint = xmid - T(j);
    sx[0] = T(0.5)*(T(0.5) - xint)*(T(0.5) - xint);
    sx[1] = T(0.75) - xint*xint;
    sx[2] = T(0.5)*(T(0.5) + xint)*(T(0.5) + xint);
    // index of the leftmost cell where particle deposits
    return j-1;
}
else if constexpr (depos_order == 3){
    const auto j = static_cast<int>(xmid);
    const T xint = xmid - T(j);
    sx[0] = (T(1.0))/(T(6.0))*(T(1.0) - xint)*(T(1.0) - xint)*(T(1.0) - xint);
    sx[1] = (T(2.0))/(T(3.0)) - xint*xint*(T(1.0) - xint/(T(2.0)));
    sx[2] = (T(2.0))/(T(3.0)) - (T(1.0) - xint)*(T(1.0) - xint)*(T(1.0) - T(0.5)*(T(1.0) - xint));
    sx[3] = (T(1.0))/(T(6.0))*xint*xint*xint;
    // index of the leftmost cell where particle deposits
    return j-1;
}
else if constexpr (depos_order == 4){
    const auto j = static_cast<int>(xmid + T(0.5));
    const T xint = xmid - T(j);
    sx[0] = (T(1.0))/(T(24.0))*(T(0.5) - xint)*(T(0.5) - xint)*(T(0.5) - xint)*(T(0.5) - xint);
    sx[1] = (T(1.0))/(T(24.0))*(T(4.75) - T(11.0)*xint + T(4.0)*xint*xint*(T(1.5) + xint - xint*xint));
    sx[2] = (T(1.0))/(T(24.0))*(T(14.375) + T(6.0)*xint*xint*(xint*xint - T(2.5)));
    sx[3] = (T(1.0))/(T(24.0))*(T(4.75) + T(11.0)*xint + T(4.0)*xint*xint*(T(1.5) - xint - xint*xint));
    sx[4] = (T(1.0))/(T(24.0))*(T(0.5) + xint)*(T(0.5) + xint)*(T(0.5) + xint)*(T(0.5) + xint);
    // index of the leftmost cell where particle deposits
    return j-2;
}
else{
    WARPX_ABORT_WITH_MESSAGE("Unknown particle shape selected in Compute_shape_factor");
    amrex::ignore_unused(sx, xmid);
}
return 0;
}
};

```

对一阶, 若 $x_{\text{mid}} = j + \xi$, $0 \leq \xi < 1$, 源码就是

$$S_0 = 1 - \xi, \quad S_1 = \xi.$$

对二阶, 源码先把最近节点/中心取成 $j = \text{int}(x_{\text{mid}} + 0.5)$, 再使用

$$S_0 = \frac{1}{2} \left(\frac{1}{2} - \xi \right)^2, \quad S_1 = \frac{3}{4} - \xi^2, \quad S_2 = \frac{1}{2} \left(\frac{1}{2} + \xi \right)^2,$$

并返回 $j-1$ 作为 stencil 左端。三阶和四阶同样是 B-spline 形函数的展开式。源码里 0/2/4 阶用 $x_{\text{mid}}+0.5$ 找中心, 1/3 阶用 x_{mid} 找左端, 这是因为偶数阶和奇数阶 shape 的自然支撑中心不同。

Esirkepov 沉积还需要把旧位置的 shape 写进与新位置对齐的数组。对应实现是同一文件中的 `Compute_shifted_shape_factor`:

```

template <int depos_order>
struct Compute_shifted_shape_factor
{
    template< typename T >
    AMREX_GPU_HOST_DEVICE AMREX_FORCE_INLINE
    int operator()(
        T* const sx,

```

```

const T x_old,
const int i_new) const
{
    if constexpr (depos_order == 0){
        const auto i = static_cast<int>(std::floor(x_old + T(0.5)));
        const int i_shift = i - i_new;
        sx[1+i_shift] = T(1.0);
        return i;
    }
    else if constexpr (depos_order == 1){
        const auto i = static_cast<int>(std::floor(x_old));
        const int i_shift = i - i_new;
        const T xint = x_old - T(i);
        sx[1+i_shift] = T(1.0) - xint;
        sx[2+i_shift] = xint;
        return i;
    }
    else if constexpr (depos_order == 2){
        const auto i = static_cast<int>(x_old + T(0.5));
        const int i_shift = i - (i_new + 1);
        const T xint = x_old - T(i);
        sx[1+i_shift] = T(0.5)*(T(0.5) - xint)*(T(0.5) - xint);
        sx[2+i_shift] = T(0.75) - xint*xint;
        sx[3+i_shift] = T(0.5)*(T(0.5) + xint)*(T(0.5) + xint);
        // index of the leftmost cell where particle deposits
        return i - 1;
    }
    else if constexpr (depos_order == 3){
        const auto i = static_cast<int>(x_old);
        const int i_shift = i - (i_new + 1);
        const T xint = x_old - T(i);
        sx[1+i_shift] = (T(1.0))/(T(6.0))*(T(1.0) - xint)*(T(1.0) - xint)*(T(1.0) - xint);
        sx[2+i_shift] = (T(2.0))/(T(3.0)) - xint*xint*(T(1.0) - xint/(T(2.0)));
        sx[3+i_shift] = (T(2.0))/(T(3.0)) - (T(1.0) - xint)*(T(1.0) - xint)*(T(1.0) - T(0.5)*(T(1.0) - xint));
        sx[4+i_shift] = (T(1.0))/(T(6.0))*xint*xint*xint;
        // index of the leftmost cell where particle deposits
        return i - 1;
    }
    else if constexpr (depos_order == 4){
        const auto i = static_cast<int>(x_old + T(0.5));
        const int i_shift = i - (i_new + 2);
        const T xint = x_old - T(i);
        sx[1+i_shift] = (T(1.0))/(T(24.0))*(T(0.5) - xint)*(T(0.5) - xint)*(T(0.5) - xint)*(T(0.5) - xint);
        sx[2+i_shift] = (T(1.0))/(T(24.0))*(T(4.75) - T(11.0)*xint + T(4.0)*xint*xint*(T(1.5) + xint - xint*xint));
        sx[3+i_shift] = (T(1.0))/(T(24.0))*(T(14.375) + T(6.0)*xint*xint*(xint*xint - T(2.5)));
        sx[4+i_shift] = (T(1.0))/(T(24.0))*(T(4.75) + T(11.0)*xint + T(4.0)*xint*xint*(T(1.5) - xint - xint*xint));
        sx[5+i_shift] = (T(1.0))/(T(24.0))*(T(0.5) + xint)*(T(0.5) + xint)*(T(0.5) + xint)*(T(0.5) + xint);
        // index of the leftmost cell where particle deposits
        return i - 2;
    }
    else{
        WARPX_ABORT_WITH_MESSAGE("Unknown particle shape selected in Compute_shifted_shape_factor");
        amrex::ignore_unused(sx, x_old, i_new);
    }
    return 0;
}

```

```
};
```

这里的 `i_shift` 是 Esirkepov 的关键工程细节：旧位置和新位置可能跨过 cell 边界，不能把两个 shape 数组各自放在自己的左端后直接相减。WarpX 把旧 shape 平移到以 `i_new` 为参考的数组里，后面才能逐项计算 `sx_old[i] - sx_new[i]`。

再往下一层看，`ShapeFactors.H` 里这三个 functor 其实对应三种不同的离散合同，而不只是“都能算一组权重”：

1. `Compute_shape_factor`
 - 给出单时间层的 shape
 - 同时返回当前 stencil 的最左写入点；
2. `Compute_shifted_shape_factor`
 - 不把 old shape 独立排到自己的左端
 - 而是把它平移进“以 new shape 为参考”的数组框架；
3. `Compute_shape_factor_pair`
 - 给同一 Villasenor segment 的横向 old/new 权重提供共同 leftmost index。

对第 5 章来说，这条区分很重要，因为后面 Esirkepov 与 Villasenor 看起来都在“算 old/new shape”，但它们真正需要的不是同一类 helper：前者需要 old/new difference 的同框对齐，后者需要 segment-local 横向共同支撑。

这里还有一条容易忽略的数值实现边界：`ShapeFactors.H` 文件头已经说明，current deposition 中这些 functor 可以用 `double` 参数求值，以避免粒子一步只移动很短距离时，单精度把 old/new 差分结构磨掉。`CurrentDeposition.H` 的 implicit Esirkepov 里也确实把 `x_new/x_old` 与 `sx_new/sx_old` 明确声明成 `double`。因此这里的 `double` 不是泛泛的“更高精度更好”，而是离散守恒实现的一部分。

5.3 电流沉积的守恒要求

电荷沉积只看某一时间层的粒子位置。电流沉积必须看粒子在时间步内穿过网格的轨迹。离散电磁 PIC 希望满足

$$\frac{\rho_i^{n+1} - \rho_i^n}{\Delta t} + (\nabla_h \cdot \mathbf{J}^{n+1/2})_i = 0.$$

如果这个式子不成立，Maxwell solver 即使形式上正确，离散 Gauss 定律也会漂移：

$$\nabla_h \cdot \mathbf{E}^{n+1} - \frac{\rho^{n+1}}{\epsilon_0} \neq 0.$$

这解释了为什么 WarpX 需要 Esirkepov、Villasenor、Vay、Direct 等多种 current deposition。Direct deposition 直观但不自动保证电荷守恒；Esirkepov 和 Villasenor 属于 charge-conserving 路径；Vay deposition 与 PSATD/current correction 等算法组合有关。

把这个合同再往前压一步，可以直接写成单粒子 shape difference：

$$\rho_i^n = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(x_p^n),$$

因此一步中的净电荷变化本质上就是

$$\rho_i^{n+1} - \rho_i^n = \frac{q_p w_p}{\Delta V_i} (S_i(x_p^{n+1}) - S_i(x_p^n)).$$

charge-conserving deposition 真正要做的，不是“再估一个差不多的 $\mathbf{J}$ ”，而是构造某个离散电流，使得

$$\frac{\rho_i^{n+1} - \rho_i^n}{\Delta t} = -(\nabla_h \cdot \mathbf{J}^{n+1/2})_i.$$

这给后面的算法分叉一个更稳定的读法：

- **Esirkepov**: 围绕 old/new shape difference 直接构造守恒电流；
- **Villasenor**: 把轨迹按 cell crossing 切 segment，再让每段局部输运共同满足同一离散守恒；
- **Direct**: 直接写 $q \mathbf{v} / \Delta V$ ，所以不自动满足这个合同；

- **Vay**: 属于显式-only 的两阶段 D-field 重组算法, 离散守恒不是通过 Esirkepov/Villasenor 这种单阶段 charge-conserving kernel 来实现。

5.3.1 五条路径的责任边界总览

为了避免把“沉积到网格”误读成一个单一 kernel, 先把本章涉及的五类路径放在同一张表中。表中的“第一性对象”指该路径在局部循环里真正组织和累加的数学对象; “外层职责”则指它不能单独承担、必须由容器或同步层完成的工作。

路径	第一性对象	是否以离散连续性为设计目标	一步轨迹如何处理	WarpX 当前实现	主要边界
Direct current	$q w v$ 与单点 shape 权重	否	不恢复守恒轨迹; 按当前速度和 shape 直接写入 J	doDepositionShapeN<1..4>() 和 CurrentDepositionKernel	不能保证 $\text{div}_h J$ 与 $\rho^{(n+1)} - \rho^{(n)}$ 一致
Esirkepov current	old/new shape difference 的方向分解与 prefix accumulation	是	使用 old/new 端点; 跨 cell 时先做 shifted-shape 对齐	doEsirkepovDepositionShapeN<1..4>(), sdx/sdyj/sdzk	需要 $\text{shapeN} < 1..4 > ()$, shape、轨迹端点和足够 stencil; implicit 前端的 suborbit 不是原文的原样内容
Villasenor current	crossing-driven segment 的局部 boundary flux	是	按最早 cell crossing 反复切分 segment, 再逐段写回 this_J*	doVillasenorDepositionCellCrossings / num_segments	需要 $\text{shapeN} < 1..4 > ()$, crossing 几何; 不能把它简化成一次 old/new shape 差分
Vay current	两阶段 D-field 沉积与局部重组	由专用组合路径承担	显式路径中按 Vay 专用的 D-field 组织, 不走 Esirkepov/Villasenor 单阶段结构	doVayDepositionShapeN<1..4>() 与 current_fp_vay	Cartesian-only、非 shared-memory; 最终 source consistency 还依赖 PSATD/current-correction 同步链
DepositCharge()	单时间层 ρ 的 shape-weighted sample	不是 current mover	由外层 relative_time 取样和内层 icomp/time_shift_delta 选择时间层参考位置	WarpXParticleContainer -> ABLASTR / ChargeDepositionDifference	不能直接写: DepositCharge() old/new shape difference、不选择 current algorithm; guard、PEC、volume scaling、AMR 整理在外层完成

这张表的用法是: 先问当前代码在累加哪一种“第一性对象”, 再问守恒属性由哪一层承担。例如, Esirkepov 的 sdx/sdyj/sdzk 属于 current kernel 内部的方向分解; DepositCharge() 的 icomp 却只是时间层和网格参考原点的桥接参数, 不能据此推断它正在执行 implicit current deposition。类似地, Vay 的 current_fp_vay 不是 Esirkepov 电流的另一种 shape 写法, 而是专门的两阶段目标字段。

从软件职责看, 这五条路径还共享一个更高层的收口: 局部 kernel 只负责把粒子信息写入某个 tile-local 或 field component; SumBoundary、fine/coarse source synchronization、PEC source-boundary、inverse-volume scaling、filter 和最终 field-solver handoff 不应被反向归因给局部 shape loop。后文的 5.8、5.9 和 5.13 分别展开 charge bridge、charge kernel 与沉积后同步, 正是因为这三层合同不能压缩成同一个“沉积算法”名词。

5.4 WarpX 的旧电荷、新电荷和半步电流

Source/Particles/PhysicalParticleContainer.cpp 中的 PhysicalParticleContainer::Evolve() 组织单 species 的沉积顺序。

关键源码节选如下，来自同一个 tile loop；这里省略了 buffer/coarse gather 和隐式 suborbit 的长分支，但保留 push 前电荷、粒子推进、半步电流和 push 后电荷的原始调用形态：

```
if (deposit_charge) {
    // Deposit charge before particle push, in component 0 of MultiFab rho.
    const int* const AMREX_RESTRICT ion_lev = (do_field_ionization)?
        pti.GetiAttribs("ionizationLevel").dataPtr():nullptr;

    amrex::MultiFab* rho = fields.get(FieldType::rho_fp, lev);
    DepositCharge(pti, wp, ion_lev, rho, 0, 0,
        np_to_deposit, thread_num, lev, lev);
}

if (! do_not_push)
{
    if (push_type == PushType::Explicit) {
        PushPX(pti, exfab, eyfab, ezfab,
            bxfab, byfab, bzfab,
            Ex.nGrowVect(), e_is_nodal,
            0, np_to_push, lev, gather_lev, dt,
            ScaleFields(false), subcycling_half,
            position_push_type, momentum_push_type);
    }

    // Current Deposition
    if (deposit_current)
    {
        // Deposit at t_{n+1/2} with explicit push
        const amrex::Real relative_time = (push_type == PushType::Explicit ? -0.5_rt * dt : 0.0_rt);

        amrex::MultiFab * jx = fields.get(current_fp_string, Direction{0}, lev);
        amrex::MultiFab * jy = fields.get(current_fp_string, Direction{1}, lev);
        amrex::MultiFab * jz = fields.get(current_fp_string, Direction{2}, lev);
        DepositCurrent(pti, wp, uxp, uyp, uzp, ion_lev, jx, jy, jz,
            0, np_to_deposit, thread_num,
            lev, lev, dt, relative_time, push_type);
    }
}

if (deposit_charge) {
    // Deposit charge after particle push, in component 1 of MultiFab rho.
    // (Skipped for electrostatic solver, as this may lead to out-of-bounds)
    if (WarpX::electrostatic_solver_id == ElectrostaticSolverAlgo::None) {
        amrex::MultiFab* rho = fields.get(FieldType::rho_fp, lev);
        DepositCharge(pti, wp, ion_lev, rho, 1, 0,
            np_to_deposit, thread_num, lev, lev);
    }
}
```

tile-loop 阶段	动作	时间层解释
push 前	沉积 rho component 0	旧电荷，通常是 ρ^n 。
推进	调用 PushPX()	$\mathbf{x}^n, \mathbf{u}^{n-1/2} \rightarrow \mathbf{x}^{n+1}, \mathbf{u}^{n+1/2}$ 。

tile-loop 阶段	动作	时间层解释
push 后	沉积 current	显式路径 <code>relative_time=-0.5*dt</code> , 对应 $\mathbf{J}^{n+1/2}$ 。
状态落点	沉积 rho component 1	新电荷, 通常是 ρ^{n+1} 。

为什么电流在 push 后沉积还要 `relative_time=-0.5*dt`? 因为粒子位置已经是 $\mathbf{x}^{n+1}$, 而电流应位于半步 $n + 1/2$ 。WarpX 在 `DepositCurrent()` 的注释中说明:`relative_time` 非零时会临时修改粒子位置以匹配沉积时间, 见 `Source/Particles/WarpXParticleContainer` 这也是读源码时必须区分“粒子当前数组中的位置”和“沉积物理时间层”的原因。

5.5 多物种层如何清零和汇总源项

`Source/Particles/MultiParticleContainer.cpp` 中的 `MultiParticleContainer::Evolve()` 是多物种粒子推进入口。

若不跳过沉积, `MultiParticleContainer::Evolve()` 先把本 level 的当前步源项清零:

- `current_fp` 的三个方向;
- `current_buf` 的三个方向;
- `rho_fp`;
- `rho_buf`。

随后遍历 `allcontainers`, 每个 species 各自沉积到同一组源项数组中。也就是说, 最终的 ρ 和 $\mathbf{J}$ 是所有物种贡献之和。

独立调用的 `DepositCurrent()` 和 `DepositCharge()` 也有类似结构:

- `MultiParticleContainer::DepositCurrent()` 位于 `Source/Particles/MultiParticleContainer.cpp`, 先清零多层 J , 再逐 species 调 `pc->DepositCurrent()`, RZ/RCYLINDER/RSPHERE 下随后做 inverse-volume scaling。
- `MultiParticleContainer::DepositCharge()` 位于 `Source/Particles/MultiParticleContainer.cpp`, 先清零 ρ , 若 `relative_time != 0` 则临时 `PushX(relative_time)`, 逐 species 沉积后再推回; 在 RZ / RCYLINDER / RSPHERE 下, 最后还会对整张 rho 做 `ApplyInverseVolumeScalingToChargeDensity(...)`。

这些函数主要服务于 PSATD-JRhom、多时间层 charge/current、静电场或诊断等场景。

这里还需要把两层时间语义拆开, 否则很容易把后面的 `charge deposition` 读错。`MultiParticleContainer::DepositCharge(relative_time)` 这层 `PushX(relative_time)` 做的是外层粒子位置平移: 它先把粒子整体挪到需要诊断或沉积的物理时刻, 沉积完成后再推回。后面 `WarpXParticleContainer::DepositCharge(..., icomp, ...)` 里的 `time_shift_delta` 做的则是内层 Galilean 网格参考框架校正: 即使粒子已经在正确时间层上, `xyzmin = LowerCorner(tilebox, depos_lev, time_shift_delta)` 仍可能因为 `icomp=0/1` 而对应不同的移动网格坐标原点。这两层都与“时间”有关, 但一个改的是粒子位置数组本身, 另一个改的是沉积 kernel 所看到的 tile 几何参考系。

5.6 AMR coarse-fine interface: 粒子怎样切到 aux/cax/buf 路径

前面几章已经讲过 AMR 的 substitution 公式

$$F(a) = F(r) + I[F(s) - F(c)]$$

以及 `UpdateAuxiliaryData*` 在 WarpX 里的实现

$$\text{aux}(\ell) = \text{fp}(\ell) + I[\text{aux}(\ell - 1) - \text{cp}(\ell)].$$

但只知道这条公式, 还不知道粒子在 coarse-fine transition zone 到底怎样用它。真正把这层逻辑接到粒子 kernel 上的是 `PhysicalParticleContainer::Evolve()`。

它不会在每个 gather/deposition kernel 内实时查 coarse-fine mask, 而是先做一次粒子重排:

```
long nfine_deposit = np;
long nfine_gather = np;
if (has_buffer && !do_not_push) {
    PartitionParticlesInBuffers( nfine_deposit, nfine_gather, np,
```

```

        pti, lev, WarpX::n_field_gather_buffer,
        WarpX::n_current_deposition_buffer, current_masks, gather_masks );
}

```

源码位置: Source/Particles/PhysicalParticleContainer.cpp.

这一步之后:

- 前 `nfine_gather` 个粒子继续从 fine patch gather;
- 后 `np-nfine_gather` 个粒子改从 lower refinement level gather;
- 前 `nfine_deposit` 个粒子继续沉积到 fine patch;
- 后 `np-nfine_deposit` 个粒子改沉积到 lower refinement level buffer.

接下来的 gather 分成两段。先看 fine interior 粒子:

```

const auto np_to_push = np_gather;
const auto gather_lev = lev;
PushPX(pti, exfab, eyfab, ezfab,
        bxfab, byfab, bzfab,
        Ex.nGrowVect(), e_is_nodal,
        0, np_to_push, lev, gather_lev, dt, ...);

```

源码位置: Source/Particles/PhysicalParticleContainer.cpp.

这里的 `exfab/eyfab/...` 来自 `Efield_aux/Bfield_aux`, 也就是已经做完 substitution 的 full solution.

随后是 transition-zone 粒子:

```

amrex::MultiFab & cEx = *fields.get(FieldType::Efield_cax, Direction{0}, lev);
...
PushPX(pti, cexfab, ceyfab, cezfab,
        cbxfab, cbyfab, cbzfab,
        cEx.nGrowVect(), e_is_nodal,
        nfine_gather, np-nfine_gather,
        lev, lev-1, dt, ...);

```

源码位置: Source/Particles/PhysicalParticleContainer.cpp.

这里不再使用 fine-level aux, 而是使用 coarse-aux 副本 `E/Bfield_cax`, 并且把 `gather_lev` 显式设为 `lev-1`.

因此, transition-zone 的粒子不是“先从 fine patch gather 再做后处理修正”, 而是一开始就改用 lower-level full solution.

沉积也完全平行地拆成两段:

```

DepositCurrent(... jx, jy, jz,
                0, np_to_deposit, thread_num,
                lev, lev, dt, relative_time, push_type);
...
DepositCurrent(... cjx, cjy, cjz,
                np_to_deposit, np-np_to_deposit, thread_num,
                lev, lev-1, dt, relative_time, push_type);

```

以及

```

DepositCharge(... rho, 0, 0,
               np_to_deposit, thread_num, lev, lev);
...
DepositCharge(... crho, 0, np_to_deposit,
               np-np_to_deposit, thread_num, lev, lev-1);

```

源码位置: Source/Particles/PhysicalParticleContainer.cpp.

这里:

- `current_fp/rho_fp` 接收 fine interior 粒子;
- `current_buf/rho_buf` 接收 transition-zone 粒子;

- `depos_lev = lev-1` 时, 不是“先沉积在 `fine` 上再 `restrict`”, 而是一开始就在 `coarse` buffer patch 的几何上沉积。

这一点在 `WarpXParticleContainer::DepositCurrent()` 和 `DepositCharge()` 的 `tilebox` 处理里写得非常直接:

```
if (lev == depos_lev) {
    tilebox = pti.tilebox();
} else {
    const IntVect& ref_ratio = WarpX::RefRatio(depos_lev);
    tilebox = amrex::coarsen(pti.tilebox(), ref_ratio);
}
```

源码位置: `Source/Particles/WarpXParticleContainer.cpp`。

也就是说, `buffer deposition` 真正变化的是:

- `offset`
- `np_to_deposit`
- `depos_lev`
- 以及由此确定的 `tilebox`、`dinv`、`xyzmin`

但电流/电荷沉积算法本身并没有为 `AMR` 单独再写一套。无论是 `Esirkepov`、`Villasenor`、`Vay` 还是 `Direct`, `WarpX` 都是在“同一个 `deposition` 数学 + 不同 `level` 的几何解释”上复用。

所以, `AMR coarse-fine interface` 在粒子层的闭环可以概括为:

1. `UpdateAuxiliaryData*`() 先构造 `fine` patch 的 `aux`;
2. `BuildBufferMasks*`() 给出 `transition-zone` 的 `gather/current` masks;
3. `PartitionParticlesInBuffers()` 先重排粒子;
4. `fine interior` 粒子走 `aux + fp` 路径;
5. `transition-zone` 粒子走 `cax + buf` 路径;
6. 最后再由 `SyncCurrent()` / `SyncRho()` 把 `coarse-fine source` 合并回通信链。

5.7 WarpXParticleContainer::DepositCurrent() 分派

`tile` 级 `current deposition` 在 `Source/Particles/WarpXParticleContainer.cpp`。

入口先做安全检查和局部数组准备:

分派前阶段	操作
入场条件	检查 <code>deposition level</code> , 只处理非空粒子且 <code>do_not_deposit</code> 为假。
<code>stencil</code> 容量	取得 <code>ng_J</code> , 检查粒子 <code>shape</code> 是否放得进 <code>tile/guard cells</code> 。
几何准备	准备沉积 <code>level</code> 的 <code>cell size</code> 、 <code>tilebox</code> 、 <code>field array</code> 和边界 <code>cropping</code> 。
组合限制	<code>Esirkepov/Villasenor</code> 不能用于 <code>collocated grid</code> 。

随后按沉积算法分派。这里真正重要的不是把 `ShapeN<1..4>` 四套模板参数全部重复抄一遍, 而是看清分派的逻辑骨架: `Esirkepov` 在 `explicit/implicit` 下分别进入 `doEsirkepovDepositionShapeN<N>()` 与 `doChargeConservingDepositionShapeNImplicit<N>()`, `Villasenor` 在 `explicit/implicit` 下分别进入 `doVillasenorDepositionShapeNExplicit<N>()` 与 `doVillasenorDepositionShapeNImplicit<N>()`, `Vay` 只允许 `explicit`, 而 `Direct` 则保留 `explicit/implicit` 两条非守恒路径。源码位置统一在 `Source/Particles/WarpXParticleContainer.cpp`。

	分派分支	算法
<code>shared-memory</code>		只支持 <code>direct</code> ; <code>Esirkepov</code> 、 <code>Villasenor</code> 、 <code>Vay</code> 会 <code>abort</code> 。
<code>Esirkepov explicit</code>		调用 <code>doEsirkepovDepositionShapeN<N>()</code> 。
<code>charge-conserving implicit</code>		进入对应 <code>implicit deposition</code> 入口。
<code>Villasenor</code>		按 <code>explicit/implicit</code> 选择两个入口。
<code>Vay</code>		只允许 <code>explicit</code> ; 隐式路径直接 <code>abort</code> 。
<code>Direct</code>		保留 <code>explicit/implicit</code> 两条分支。

这个分派地图只是入口。下面继续进入 Source/Particles/Deposition/ChargeDeposition.H 和 CurrentDeposition.H 的 kernel, 把 shape 权重、电荷归一化、direct current、Esirkepov 守恒电流, 以及 Villasenor/Vay/implicit 的时间层与几何边界逐块展开。

如果把 WarpXParticleContainer::DepositCurrent() 只理解成“按算法名切 kernel”, 会漏掉调用层真正更强的合同。源码在进入 normal-path 分派之前, 先固定了四层边界:

1. depos_lev 只能是 lev 或 lev-1, 也就是 current buffer 只允许 coarse 一层;
2. 当前维度下的 shape_extent 必须落在 tile/guard-cell 允许范围内, 否则直接触发 numParticlesOutOfRange(...) == 0 断言;
3. Esirkepov / Villasenor 只要配上 colocated grid 就在入口 abort, 而不会等到 kernel 内部再失败;
4. 若打开 do_shared_mem_current_deposition, 则整条路径先转成 DenseBins + shared_tilesize + max_tbox_size 的 tile-binned performance contract, 并且这条合同当前只接受 explicit direct deposition, implicit / Esirkepov / Villasenor / Vay 都在入口直接 abort。

因此 WarpX 当前的 dispatch 次序其实是:

- 先判断这批粒子在当前 tile/guard-cell 几何上是否允许沉积;
- 再决定是否走 shared-memory performance path;
- 只有在 normal path 里, 才继续分成 Direct / Esirkepov / Villasenor / Vay 四个 kernel 家族。

这里还有一个容易讲混的细节: domain_double 与 do_cropping 虽然在入口统一构造, 但并不会发给所有算法。它们只会继续传给 Esirkepov 和 Villasenor 这两条 charge-conserving 路径; Direct 与 Vay 即使运行在同一 tile 上, 也只拿到时间层、dinv 和 xyzmin 这组几何缩放信息, 而拿不到可裁剪轨迹合同。也就是说, DepositCurrent() 自己就已经把 near-boundary/charge-conserving 的接口能力划给了特定算法家族, 而不是留给 kernel 临时自选。

如果只看 AMR coarse-fine buffer, 这几种算法的差异可以压缩成“共享同一套 coarse patch 几何, 但恢复粒子轨迹的方式不同”。

首先, 进入 current_buf 的粒子和进入 current_fp 的粒子, 在接口层共享同样的沉积壳:

```
if (lev == depos_lev) {
    tilebox = pti.tilebox();
} else {
    const IntVect& ref_ratio = WarpX::RefRatio(depos_lev);
    tilebox = amrex::coarsen(pti.tilebox(), ref_ratio);
}
tilebox.grow(ng_J);
const amrex::XDim3 dinv = WarpX::InvCellSize(std::max(depos_lev, 0));
const amrex::XDim3 xyzmin = WarpX::LowerCorner(tilebox, depos_lev, 0.5_rt*dt);
```

源码位置: Source/Particles/WarpXParticleContainer.cpp。

因此, AMR buffer 本身只改变:

- depos_lev
- tilebox
- dinv
- xyzmin
- 以及后续 domain_double / do_cropping

并不会为 coarse-fine interface 再定义另一套 current deposition 数学。

真正的分界线是各算法如何恢复 old/new/mid 轨迹:

- Vay: 显式-only; 接口层直接禁止 implicit。
- Villasenor explicit: 使用当前粒子位置、relative_time 和 dt 回推 x_old/x_new。
- Villasenor implicit: 不再用 relative_time, 改为显式使用 x_n、u_n、u_{n+1/2} 恢复轨迹。
- Esirkepov implicit: 时间层输入与 implicit Villasenor 相似, 但后续仍走 old/new shape 差分的守恒电流构造。

例如显式 Villasenor 会先写:

```
amrex::Real const xp_new = xp + (relative_time + 0.5_rt*dt)*uxp[ip]*gaminv;
amrex::Real const xp_old = xp_new - dt*uxp[ip]*gaminv;
```

源码位置: Source/Particles/Deposition/CurrentDeposition.H。

而隐式 Villasenor / Esirkepov 则改为:

```
amrex::ParticleReal const xp_npl = 2._prt*xp_nph - xp_n;
```

源码位置:

- Villasenor implicit: Source/Particles/Deposition/CurrentDeposition.H
- Esirkepov implicit: Source/Particles/Deposition/CurrentDeposition.H

所以, AMR coarse-fine buffer 下 current deposition 的最小结论是:

1. coarse-fine 只决定粒子在哪个 level 的几何上沉积;
2. 时间层恢复方式仍由 current deposition 算法自身决定;
3. 也正因为如此, `current_buf` 可以统一复用 Villasenor、Vay、Esirkepov、Direct 的现有 kernel, 而不需要 AMR 专用变体。

这里还要补一条论文边界, 否则很容易把 WarpX 今天的 implicit 路径误读成两篇经典论文的“原样实现”。无论是 Esirkepov 2001 还是 Villasenor-Buneman 1992, 原始论文讨论的核心都还是显式 PIC 的守恒电流构造: 前者围绕 old/new shape difference 的唯一线性分解, 后者围绕 crossing-driven local flux mover。WarpX 当前 explicit 分支仍直接继承这两条主干; 但 implicit 分支已经额外引入了 x_n 、 u_n 、 $u_{\{n+1/2\}}$ 、suborbit endpoint reconstruction 和 matching gather 兼容性这些现代工程语义。也就是说:

- 论文原始结构: 守恒电流该怎样由一条 one-step orbit 构造出来;
- WarpX implicit 扩展: 当 orbit 本身来自隐式推进、suborbit fallback 和 near-boundary gather 约束时, 怎样先把那条 orbit 恢复出来, 再送回原来的守恒沉积骨架。

对 Villasenor 来说, 这里还要再往下压一层: `doVillasenorDepositionShapeNExplicit(...)` 和 `doVillasenorDepositionShapeNImplicit(...)` 自己并不实现 segment loop。两条入口在完成各自的 endpoint reconstruction 之后, 都会把

- old/new 轨迹端点
- wq
- 中间时间层动量 `uxp_mid/uyy_mid/uzp_mid`
- `gaminv`
- `domain_double/do_cropping`

统一交给同一个 `VillasenorDepositionShapeNKernel(...)`。也就是说, explicit/implicit 这条分界回答的是“端点怎么恢复”, 而不是“segment 数学怎么改写”; 真正的 crossing 统计、segment 推进、cell/node 权重构造和局部 `this_J*` 写回都属于共享后端。

如果继续往 kernel 本体再走一步, 就会发现 Villasenor 和 Esirkepov 的 charge-conserving 结构并不只是“名字不同”, 而是两种不同的守恒组织方式:

- Esirkepov: 先把整条轨迹压成同框的 old/new shape difference, 再沿沉积方向做前缀累加;
- Villasenor: 先按真实 cell crossing 切出多个局部 segment, 再对每个 segment 分别沉积。

这也是为什么源码注释会说 Villasenor “results in a tighter stencil”: 它的支持域围绕真实 crossing segment 局部组织, 而不是像 Esirkepov 那样由整条 old/new difference support 一次性决定。更细的 3D 循环、cell/node 权重和 `this_J*` 写回细节, 放到后面的 Esirkepov current deposition 小节再展开。

更关键的是, Villasenor 的“segment-by-segment”不只是数学风格差异, 而是它的接口本身已经把边界裁剪写死了。`VillasenorDepositionShapeNKernel` 直接接收:

- `xp_old/xp_new`
- `domain_double`
- `do_cropping`

进入 kernel 后, 第一批动作就是 `ParticleUtils::crop_at_boundary(...)`, 然后才统计 `cell_crossings`、得到 `num_segments`、逐段恢复 `x0_old -> x0_new` 并沉积。源码见 `Source/Particles/Deposition/CurrentDeposition.H`。因此第 5 章不应把 Villasenor 理解成“另一种 charge-conserving 公式”, 而应把它看成:

1. 以完整轨迹端点为输入;
2. 允许在 PEC/PECInsulator 邻近几何上先裁剪轨迹;
3. 再把剩余轨迹按 crossing 切成局部 segment 的沉积路线。

这条合同与 direct 的“单个时间中心位置 + 速度加权沉积”完全不是同一层次。

这也是 `ImplicitPushPX.cpp` 在 suborbit fallback 里直接强制改成 Villasenor 的原因。源码注释写得很直白: 为了 energy conservation, suborbit push 必须使用 matching gather, 因此这里会覆盖 runtime-selected deposition type, 强制改成

CurrentDepositionAlgo::Villasenor。见 Source/Particles/Pusher/ImplicitPushPX.cpp。换句话说，WarpX 在这里需要的不是“任意一个能沉 J 的 kernel”，而是：

- 与 implicit gather stencil 配套；
- 保留 boundary crop + segment decomposition；
- 在 near-boundary/suborbit 情形下仍维持局部守恒几何的那一条沉积合同。

因此在 implicit/suborbit 一侧，WarpX 真正需要的不是“任意一个能沉 J 的 kernel”，而是保留 boundary crop、segment decomposition 和 matching gather 兼容性的那条沉积合同。

对 Esirkepov 也应作同样辨析。doChargeConservingDepositionShapeImplicit<N>() 当然仍保留了论文那条 old/new shape-difference 守恒主线，但它前面多出来的 x_n to x_{n+1} 恢复、几何分支坐标改写、double 精度 shape functor 以及 implicit gather 配套语义，都属于 WarpX 在原始论文主干之外加上的工程前端。换句话说，读第 5 章时应把

1. “守恒电流如何由 old/new shape difference 或 segment flux 构造出来”，和
2. “隐式推进下怎样先把这条轨迹恢复成可沉积对象”

严格分成两层：前一层是论文主结果，后一层是现代代码为把论文主结果接进更复杂时间推进框架而增加的实现层。

还有一条算法要单独区分出来：Vay deposition。它和 Direct、Esirkepov、Villasenor 的差别，不只是“权重系数不同”，而是整个执行拓扑都不同。

CurrentDeposition.H 的注释写得很直接：

deposit D in real space and store the result in Dx_fab, Dy_fab, Dz_fab

源码位置：Source/Particles/Deposition/CurrentDeposition.H。

这说明 Vay 路径的第一目标不是直接形成普通意义上的 $J_x/J_y/J_z$ ，而是先沉积一组 D 量。对应地，它一开始就会额外分配一个 temporary FAB：

```
#if defined(WARPX_DIM_3D)
amrex::FArrayBox temp_fab{Dx_fab.box(), 4};
#elif defined(WARPX_DIM_XZ)
amrex::FArrayBox temp_fab{Dx_fab.box(), 2};
#endif
temp_fab.setVal<amrex::RunOn::Device>(0._rt);
```

源码位置：Source/Particles/Deposition/CurrentDeposition.H。

也就是说，Vay deposition 不是单阶段沉积，而是两阶段：

1. 粒子 loop 先写 temp_arr 中间量；
2. 再由 box 级 ParallelFor 把它们重组为三个方向。

例如 3D 的第二阶段就是：

```
const amrex::Real t_a = temp_arr(i,j,k,0);
const amrex::Real t_b = temp_arr(i,j,k,1);
const amrex::Real t_c = temp_arr(i,j,k,2);
const amrex::Real t_d = temp_arr(i,j,k,3);
Dx_arr(i,j,k) += (1._rt/6._rt)*(2._rt*t_a + t_b + t_c - 2._rt*t_d);
Dy_arr(i,j,k) += (1._rt/6._rt)*(2._rt*t_a + t_b - 2._rt*t_c + t_d);
Dz_arr(i,j,k) += (1._rt/6._rt)*(2._rt*t_a - 2._rt*t_b + t_c + t_d);
```

源码位置：Source/Particles/Deposition/CurrentDeposition.H。

这正是它和 Esirkepov / Villasenor 的根本区别：

- **Esirkepov / Villasenor**：粒子 loop 内直接形成 charge-conserving J；
- **Vay**：粒子 loop 只形成 D 的中间组合量，真正的三个方向要靠第二阶段线性重组。

同时，Vay 还有一组清晰的实现边界：

- 不支持 implicit；
- 不支持 RZ；
- 不支持 1D / RCYLINDER / RSPHERE；
- 不支持 shared-memory current deposition。

这组边界不是上层文档约定, 而是 kernel 内部直接 abort:

```
#if defined(WARPX_DIM_RZ)
    WARPX_ABORT_WITH_MESSAGE("Vay deposition not implemented in RZ geometry");
#endif

#if defined(WARPX_DIM_1D_Z) || defined(WARPX_DIM_RCYLINDER) || defined(WARPX_DIM_RSPHERE)
    WARPX_ABORT_WITH_MESSAGE("Vay deposition not implemented in 1D geometry");
#endif
```

源码位置: Source/Particles/Deposition/CurrentDeposition.H。

与此相对, implicit charge-conserving 和 Villasenor 路径反而把几何差异显式展开了。比如 implicit charge-conserving 里直接分成:

- RZ / RCYLINDER
 - 从 (x,y) 恢复半径 r
 - 再用 costheta/sintheta 重建分量
- RSPHERE
 - 再进一步恢复 r, \theta, \phi
- 1D_Z
 - 空间支撑只剩 z
 - 但横向速度分量仍可能进入 current 分量的几何解释

源码原文如下, 位置为 Source/Particles/Deposition/CurrentDeposition.H:

```
#if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER)
    ...
    const amrex::Real costheta_mid = (rp_mid > 0._rt ? xp_mid/rp_mid : 1._rt);
    const amrex::Real sintheta_mid = (rp_mid > 0._rt ? yp_mid/rp_mid : 0._rt);
#elif defined(WARPX_DIM_RSPHERE)
    ...
    const amrex::Real cosphi_mid = (rp_mid > 0. ? rpxy_mid/rp_mid : 1._rt);
    const amrex::Real sinphi_mid = (rp_mid > 0. ? zp_mid/rp_mid : 0._rt);
#elif defined(WARPX_DIM_1D_Z)
    amrex::Real const vx = uxp_nph[ip]*gaminv;
    amrex::Real const vy = uyp_nph[ip]*gaminv;
#endif
```

但这段几何恢复还不能和后面的 shape factor 分开看。implicit Esirkepov 的真实链路是:

1. 从 x_n 与 $x_{\{n+1/2\}}$ 恢复 $x_{\{n+1\}}$;
2. 按 RZ / RCYLINDER / RSPHERE / 1D_Z 各自的坐标语义把位置与速度改写到沉积所需变量;
3. 必要时先做 crop_at_boundary(...);
4. 再用 Compute_shape_factor 与 Compute_shifted_shape_factor 在这个几何分支下生成对齐后的 old/new stencil。

也就是说, shape factor 在这里不是几何恢复之后“顺手再算的权重”, 而是这条 implicit 几何链的最后离散化步骤。若把这两层拆开写, 就会把 ShapeFactors.H 误解成和几何无关的通用插值库。

再进一步, 1D_Z / RCYLINDER / RSPHERE 的几何分支还直接改写了 Jx/Jy/Jz 三个分量各自的写回合同, 而不只是“先恢复什么坐标”。例如 implicit Esirkepov 在 1D_Z 下写成:

- Jx/Jy
 - 用 $0.5*(sz_{old} + sz_{new})$ 的 old/new 平均写回;
- Jz
 - 用 $sz_{old} - sz_{new}$ 的前缀累加承担唯一空间方向上的守恒运输。

而在 RCYLINDER / RSPHERE 下则反过来是:

- Jx
 - 实际承担径向主运输角色, 走 $sx_{old} - sx_{new}$ 的守恒差分;
- Jy/Jz
 - 作为切向分量, 只走 $0.5*(sx_{old} + sx_{new})$ 的横向平均写回。

对应地, Villasenor 在这两组几何里也保持同样的物理分工, 只是把“守恒主分量”的写回改成 segment-local 的 cell weights, 而把另外两个分量继续放在 node-average 支撑上。也就是说, 几何分支真正改变的是“哪个分量承担连续性方程的主差分, 哪个只沿横向平均写回”。

RZ 还要再多看一层: $m=0$ 的 Jr/Jz 确实沿 XZ 主干继续用 sx_old-sx_new 、 sz_old-sz_new 的守恒差分, 但 $m>0$ 并不是简单把 mode-0 的三个分量统一乘上一个相位。源码在 Source/Particles/Deposition/CurrentDeposition.H 里把

- Jr 模态写成 $djr_cmplx = 2 * sdx_i * xy_mid$,
- Jz 模态写成 $djz_cmplx = 2 * sdz_k * xy_mid$,
- Jtheta 模态则单独写成包含 $xy_new / xy_mid / xy_old$ 、 $1/imode$ 和 Davidson 符号约定修正的 djt_cmplx 。

因此更准确的说法是: RZ 的 mode-0 径向/轴向守恒结构与 XZ 同构, 但 $m>0$ 特别是 Jtheta 有自己独立的复模态重建合同, 不能概括成“XZ 再做一次 Fourier 复制”。

另外, CurrentDeposition.H 的 kernel 写回对 RZ / RCYLINDER / RSPHERE 还不是最终物理电流密度。MultiParticleContainer::DepositC 在所有 species 沉积之后, 会额外调用 WarpX::ApplyInverseVolumeScalingToCurrentDensity(...) (Source/Particles/MultiParticleContainer.cpp 单 species 路径也有同样调用)。对应实现位于 Source/FieldSolver/WarpXPushFieldsEM.cpp, 源码注释直接写明“the inverse volume factor was not included in the current deposition”。它做的事情包括:

- 先把轴附近负半径 guard cell 的沉积 wrap 回到轴上方;
- 对 RZ / RCYLINDER 的非轴点按 $2\pi r$ 缩放;
- 对 RSPHERE 的非轴点按 $4\pi r^2$ 型球壳体积因子缩放;
- 在轴上把 Jr/Jtheta 强制置零, 而 Jz 则按 Verboncoeur 修正体积因子 $axis_volume_factor$ 走专门处理。

这意味着柱/球对称几何里真正供场求解器消费的 J, 不是 kernel 原子加的直接输出, 而是“守恒写回 + inverse-volume scaling”两层合同共同定义的结果。

因此第 5 章里更稳定的组织方式不能只按算法名字排目录, 还必须同时保留:

1. 离散连续性合同;
2. Direct / Esirkepov / Villasenor / Vay 的实现差异;
3. implicit / RZ / 1D_Z / RCYLINDER / RSPHERE 的时间层与几何边界。

对正文来说, 最重要的结论不是再重复列一遍全部 #if, 而是明确:

1. Direct deposition 不自动保证

$$\frac{\rho^{n+1} - \rho^n}{\Delta t} + \nabla_h \cdot J = 0$$

2. Esirkepov / Villasenor 的第一性目标都是让这条离散连续性合同成立;
3. implicit 路线改写的是轨迹端点恢复方式, 不是守恒目标本身;
4. Vay deposition 是显式-only, 且当前只在 3D/XZ 这一组笛卡尔路径上有实现。

因此, Vay 应该被看作一个显式、笛卡尔、两阶段重组的专用沉积路径, 而不是一般 current deposition kernel 的简单变种。

5.8 WarpXParticleContainer::DepositCharge() 入口

上面讲的是 Vay deposition 的实现拓扑; 更直接的验证入口是 Examples/Tests/vay_deposition/。这组测试不是拿解析单粒子轨道去对照, 而是只看经过一小段推进后, 最终 full diagnostics 里是否仍满足

$$\frac{\max |\nabla \cdot E - \rho/\epsilon_0|}{\max |\rho/\epsilon_0|} < 10^{-3}.$$

也就是说, 它真正测的是:

- algo.current_deposition = vay
- algo.maxwell_solver = psatd
- warpx.grid_type = collocated

这组 Vay 专用实现边界下, D-field 两阶段重组后的离散电荷守恒是否仍成立。它给了一个比 Langmuir 家族更窄、更直接的 Vay deposition 自证入口。

和它互补的另一组 regression 是 Examples/Tests/langmuir/ 里的 PSATD current-correction 变体。那组测试不是只看 $\text{div}E-\rho/\epsilon_0$, 而是两层断言一起做:

1. 先把 $E_x/E_y/E_z$ 或 E_x/E_z 与解析 Langmuir-wave 场解比较;

2. 再由 `analysis_utils.py` 在特定组合下追加 `divE-rho/\epsilon_0` 检查。

更关键的是, 这个 helper 明确写死了适用边界:

- `current_correction`
 - 始终检查, 容差 $1e-9$
- `current_deposition = vay`
 - 始终检查, 容差 $1e-3$
- `current_deposition = esirkepov`
 - 只在非 RZ 且非 PSATD 时检查

因此, Langmuir + `current_correction` 和 `vay_deposition` 两组 regression 的角色并不相同:

- Langmuir + `current_correction`
 - 是 解析场解 + source consistency 的组合验证;
 - 典型输入还会显式打开 `psatd.current_correction = 1` 与 `psatd.periodic_single_box_fft = 1`。
- `vay_deposition`
 - 是更窄的 PSATD + collocated + Vay current deposition source-synchronization 验证;
 - 只断言离散 Gauss law, 不再做解析波对照。

这正好也对应上一节的 `SyncCurrentAndRho()` 分叉:

- `current-correction` 变体对应 PSATD + periodic single box 下仍立即同步的那条路径;
- Vay 变体对应非 periodic-single-box 下 `current_fp_vay` 单独过滤、再交给后续 PSATD 同步链的那条专门路径。

tile 级 charge deposition 入口在 `Source/Particles/WarpXParticleContainer.cpp` 的 `WarpXParticleContainer::DepositCharge` 阅读时先区分 component 和几何准备, 再区分 shared-memory 与普通 kernel 分派。

阶段	操作
component 检查	检查 rho component 数量是否足够。
shared-memory 前置	检查非空粒子、 <code>ng_rho</code> 、粒子 shape 与 guard cells。
tile 准备	取得 species 电荷、tilebox 和 GPU/CPU 本地 <code>rho_fab</code> 。
时间层参考	根据 <code>icomp</code> 计算 <code>time_shift_delta</code> , 再确定 <code>xyzmin</code> 和 <code>dinv</code> 。
shared-memory 分派	根据 <code>WarpX::nox</code> 调 <code>doChargeDepositionSharedShapeN<1..4>()</code> 。
普通分派	重新建立 <code>ng_rho/tilebox/xyzmin</code> 后委托 <code>ablastr::particles::deposit_charge(...)</code> 。

`time_shift_delta` 对理解 rho component 很关键:`icomp==0` 表示旧时间层;`icomp==1` 表示新时间层,它和 `PhysicalParticleContainer::Evolve` 中 push 前/后两次 charge deposition 对应。

这里有一个比 current deposition 更容易讲混的分叉: `DepositCharge()` 不像 current deposition 那样在这个文件里直接按 `Esirkepov/Villasenor/Vay/Direct` 多算法展开。共享内存路径显式调用 `doChargeDepositionSharedShapeN<1..4>()`; 普通路径则先交给 `ablastr::particles::deposit_charge(...)`,再由 ABLASTR 桥接到 `doChargeDepositionShapeN<1..4>()`。也就是说, charge deposition 的“主差别”首先不是算法名字, 而是:

1. shared-memory 还是普通路径;
2. 旧/新时间层的 rho component;
3. CPU thread-local 暂存还是 GPU 直接 alias 目标 rho。

这一点和 current deposition 很不一样。current deposition 的主入口主要负责算法选择; 而普通 charge deposition 的主入口更像是在整理 bridge contract, 把 `depos_lev/ref_ratio/icomp/xyzmin`、guard-cell 检查和 CPU/GPU 暂存策略都布置好, 再统一交给 `ChargeDeposition.H`。

5.8.1 icomp 不只是分量号, 而是旧/新 rho 的时间层合同

`WarpXParticleContainer::DepositCharge()` 的注释写得很明确:

- `icomp = 0`
 - 沉旧值, before particle push

- `icomp = 1`
— 沉新值, after particle push

而源码真正把这个时间层差异落到几何上的地方是:

```
const amrex::Real dt = warpx.getdt(lev);
const amrex::Real time_shift_delta = (icomp == 0 ? 0.0_rt : dt);
const amrex::XDim3 xyzmin = WarpX::LowerCorner(tilebox, depos_lev, time_shift_delta);
```

源码位置: `Source/Particles/WarpXParticleContainer.cpp`。

也就是说, 旧/新 `rho` component 的区别不只是 `MultiFab` 里“写第几块分量”, 还会改变用于沉积的 `tile` 物理左下角参考时间层。后面的 `shape` kernel 本身不再判断“现在沉的是旧电荷还是新电荷”, 因为 `xyzmin` 在桥接层已经按时间层对齐好了。

如果再把 `WarpX::LowerCorner(...)` 展开一层, 这个 `time_shift_delta` 的真实作用会更清楚。 `Source/WarpX.cpp` 中,

```
const amrex::Real cur_time = warpx.gett_new(lev);
const amrex::Real time_shift = (cur_time + time_shift_delta - warpx.time_of_last_gal_shift);
amrex::Array<amrex::Real,3> galilean_shift = { warpx.m_v_galilean[0]*time_shift,
                                              warpx.m_v_galilean[1]*time_shift,
                                              warpx.m_v_galilean[2]*time_shift };
```

然后 `LowerCorner(...)` 才把这份 `galilean_shift` 加到 `grid_min` 上。换句话说, `icomp=0/1` 并不是简单地在注释里区分“old/new `rho`”, 而是真的通过 `time_shift_delta` 改写了 moving-window / Galilean 坐标下沉积 kernel 所看到的 `tile` 原点。对成书叙述来说, 这一点比“分量号不同”重要得多, 因为它说明 `charge deposition` 的旧/新时间层差异已经被压进了几何参考框架本身。

5.8.2 普通 `charge deposition` 的桥接合同在 `ABLASTR`, 而不在 `kernel` 本体

普通路径的桥接在 `Source/ablastr/particles/DepositCharge.H`。它做了四件真正影响正文理解的事:

1. 把运行时 `shape` 阶数压成统一的 `particle_shape`, 因此调用点先 `assert WarpX::nox == WarpX::noy == WarpX::noz`。
2. 检查 `depos_lev` 只能是 `lev` 或 `lev-1`, 把 coarse-fine buffer 限定在这一级别差上。
3. 再做一次 `numParticlesOutOfRange(pti, range) == 0` 的 guard-cell 安全检查, 也就是 `charge deposition` 的 `tile/guard` 合法性并没有完全下沉到 `ChargeDeposition.H`。
4. 在 CPU/GPU 上切换不同的暂存策略:
 - GPU: `rho_fab` 直接 alias 到真实 `rho`
 - CPU: 先沉到 `local_rho`, 再 `lockAdd(...)` 回去

对应源码如下:

```
#ifdef AMREX_USE_GPU
amrex::MultiFab rhoi(*rho, amrex::make_alias, icomp*nc, nc);
auto & rho_fab = rhoi.getOut(pti);
#else
local_rho.resize(tb, nc);
local_rho.setVal(0.0);
auto & rho_fab = local_rho;
#endif
...
(*rho)[pti].lockAdd(local_rho, tb, tb, 0, icomp*nc, nc);
```

源码位置: `Source/ablastr/particles/DepositCharge.H`。

这里也因此可以把外层和内层两套时间合同并排摆清:

1. `MultiParticleContainer::DepositCharge(relative_time)`
 - 通过 `PushX(relative_time)` 临时改写粒子位置数组;
 - 主要服务于“要在当前数组时间层之外取样”的场景;
2. `WarpXParticleContainer::DepositCharge(..., icomp, ...)`
 - 通过 `time_shift_delta` 改写 `LowerCorner(tilebox, ...)` 的 Galilean 参考原点;
 - 主要服务于“同一歩里 old/new `rho` component 各自该落在哪个移动网格参考框架”。

这两者叠加起来, 才构成了 `WarpX` 里 `charge deposition` 完整的时间语义。

因此第 5 章里更准确的调用链不是：

DepositCharge -> ChargeDeposition.H

而是：

```
WarpXParticleContainer::DepositCharge
-> 选择 shared-memory 或普通路径
-> 确定 icomp / xyzmin / depos_lev / ref_ratio
-> ABLASTR deposit_charge(...) 做 guard-check 与 CPU/GPU 暂存
-> ChargeDeposition.H 做 shape-factor 和原子加
```

5.8.3 shared-memory charge deposition 不是走 ABLASTR, 而是先变成 tile-binned 执行合同

如果把这一节再往 shared-memory 那半边压一层, 就会发现 do_shared_mem_charge_deposition 不是简单把普通 kernel 放进 shared memory, 而是先把整条调用链改写成一套 tile-binned 执行合同。对应源码在 Source/Particles/WarpXParticleContainer.cpp 与 Source/Particles/Deposition/ChargeDeposition.H。

容器层做的第一件事不是调用 ABLASTR, 而是：

1. 仍先检查 depos_lev 只能是 lev 或 lev-1;
2. 仍先用 shape_extent 与 ng_rho / rho->nGrowVect() 检查粒子 shape 能否放进 tile 或 guard cells;
3. 然后把 pti.validbox().grow(ng_rho) 这块区域按 WarpX::shared_tilesize 切成 bins;
4. 为每个 bin 反推出对应 tile box, 再做一次 reduction 得到 max_tbox_size;
5. 最后才把 bins + box + geom + max_tbox_size + shared_tilesize 一起传给 doChargeDepositionSharedShapeN<1..4>()

也就是说, shared-memory charge deposition 的主入口负责的已经不再只是“选 shape 阶数”, 而是把粒子先组织成

DenseBins + tile boxes + max_tbox_size

这一套可供 GPU block 或 CPU tile 复用的局部执行几何。

这一点和普通路径有实质差别。普通路径的主问题是 icomp / xyzmin / local_rho 如何经 ABLASTR 桥接到统一 kernel; shared-memory 路径的主问题则变成：

1. 每个 tile 内有哪些粒子;
2. 这个 tile 最多需要多大的局部沉积缓冲区;
3. 该缓冲区能否装进单个 block 的 shared memory。

doChargeDepositionSharedShapeN<...>() 本体也正按这个思路组织。Source/Particles/Deposition/ChargeDeposition.H 里, 它先根据 a_tbox_max_size 构造一个 sample tile, 转成 ix_type 后再 grow(depos_order), 据此计算

```
const auto npts = sample_tbox_x.numPts();
std::size_t shared_mem_bytes = npts*sizeof(amrex::Real);
const std::size_t max_shared_mem_bytes = amrex::Gpu::Device::sharedMemPerBlock();
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(shared_mem_bytes <= max_shared_mem_bytes,
    "Tile size too big for GPU shared memory charge deposition");
```

这说明 max_tbox_size 不是旁枝信息, 而是 shared-memory 路径能否成立的硬约束: WarpX 先从所有实际 tile 里取一个逐方向最大包围盒, 再据此估算每个 block 需要的临时 rho 缓冲区大小; 若超出设备每个 block 的 shared-memory 预算, 就在 kernel 启动前直接 abort。

进入 GPU kernel 以后, 执行拓扑也和普通路径不同。shared-memory 版本不是对所有粒子直接做一个平铺 ParallelFor, 而是：

1. one block per bin/tile;
2. block 内线程按 stride 处理本 tile 的粒子;
3. 先在 shared memory 上分配 tile-local buf;
4. 先把 buf 清零, 再把当前 tile 粒子沉到 buf;
5. 最后才把 buf 原子加回全局 rho_arr。

因此 shared-memory charge deposition 虽然在数学上仍调用同样的 shape-factor 逻辑, 但它的工程合同已经明显不同于普通 ABLASTR 路径: 后者的 CPU/GPU 差别主要体现在 local_rho 还是 alias 目标场; 前者则把“tile-local 暂存”提升成了第一性执行结构, 并围绕它重新组织 binning、tile geometry、shared-memory 容量和 block-to-tile 对应关系。

这也解释了为什么 shared-memory 路径没有再走 ABLASTR deposit_charge(...)。它需要的不只是统一的 shape 调度, 而是：

- DenseBins 粒子重排;

- shared_tilesize 控制的 tile 划分;
- max_tbox_size 驱动的 shared-memory 预算检查;
- one block per tile 的专门 launch 拓扑。

这些信息都属于执行拓扑,而不是普通 charge kernel 的纯桥接参数,所以只能在 WarpX 容器层先整理完,再直接进入 doChargeDepositionSharedShapeN<.

5.9 ChargeDeposition.H: 电荷沉积 kernel 的逐项结构

普通路径的中间桥接在 Source/ablastr/particles/DepositCharge.H: 它接收 WarpXParticleContainer 的 particle iterator、本地/目标 rho.ng_rho.depos_lev.ref_ratio 与 icomp/nc,再按 WarpX::noz 选择 doChargeDepositionShapeN<1..4>(). 最终的 WarpX-specific shape kernel 位于 Source/Particles/Deposition/ChargeDeposition.H. 下面按 3D 主干摘出核心源码,并保留 XZ/RZ 与 3D 的原始写入分支;完整维度条件见原文件同一函数:

```
template <int depos_order>
void doChargeDepositionShapeN (const GetParticlePosition<PIdx>& GetPosition,
                               const amrex::ParticleReal * const wp,
                               const int* ion_lev,
                               amrex::FArrayBox& rho_fab,
                               long np_to_deposit,
                               const amrex::XDim3 & dinv,
                               const amrex::XDim3 & xyzmin,
                               amrex::Dim3 lo,
                               amrex::Real q,
                               [[maybe_unused]] int n_rz_azimuthal_modes)
{
    const bool do_ionization = ion_lev;
    const amrex::Real invvol = dinv.x*dinv.y*dinv.z;
    amrex::Array4<amrex::Real> const& rho_arr = rho_fab.array();
    amrex::IntVect const rho_type = rho_fab.box().type();

    amrex::ParallelFor(
        np_to_deposit,
        [=] AMREX_GPU_DEVICE (long ip) {
            amrex::Real wq = q*wp[ip]*invvol;
            if (do_ionization){
                wq *= ion_lev[ip];
            }

            amrex::ParticleReal xp, yp, zp;
            GetPosition(ip, xp, yp, zp);

            Compute_shape_factor< depos_order > const compute_shape_factor;
            const amrex::Real x = (xp - xyzmin.x)*dinv.x;

            amrex::Real sx[depos_order + 1] = {0._rt};
            int i = 0;
            if (rho_type[0] == NODE) {
                i = compute_shape_factor(sx, x);
            } else if (rho_type[0] == CELL) {
                i = compute_shape_factor(sx, x - 0.5_rt);
            }

            const amrex::Real y = (yp - xyzmin.y)*dinv.y;
            amrex::Real sy[depos_order + 1] = {0._rt};
            int j = 0;
            if (rho_type[1] == NODE) {
                j = compute_shape_factor(sy, y);
            } else if (rho_type[1] == CELL) {
```

```

        j = compute_shape_factor(sy, y - 0.5_rt);
    }

    const amrex::Real z = (zp - xyzmin.z)*dinv.z;
    amrex::Real sz[depos_order + 1] = {0._rt};
    int k = 0;
    if (rho_type[WARPX_ZINDEX] == NODE) {
        k = compute_shape_factor(sz, z);
    } else if (rho_type[WARPX_ZINDEX] == CELL) {
        k = compute_shape_factor(sz, z - 0.5_rt);
    }

    #if defined(WARPX_DIM_XZ) || defined(WARPX_DIM_RZ)
        for (int iz=0; iz<=depos_order; iz++){
            for (int ix=0; ix<=depos_order; ix++){
                amrex::Gpu::Atomic::AddNoRet(
                    &rho_arr(lo.x+i+ix, lo.y+k+iz, 0, 0),
                    sx[ix]*sz[iz]*wq);
            }
        }
    #elif defined(WARPX_DIM_3D)
        for (int iz=0; iz<=depos_order; iz++){
            for (int iy=0; iy<=depos_order; iy++){
                for (int ix=0; ix<=depos_order; ix++){
                    amrex::Gpu::Atomic::AddNoRet(
                        &rho_arr(lo.x+i+ix, lo.y+j+iy, lo.z+k+iz),
                        sx[ix]*sy[iy]*sz[iz]*wq);
                }
            }
        }
    #endif
    }
    );
}

```

这段代码对应的 3D 公式是

$$\rho_{i+\alpha, j+\beta, k+\gamma} \leftarrow \rho_{i+\alpha, j+\beta, k+\gamma} + q w_p \frac{1}{\Delta x \Delta y \Delta z} S_\alpha(x_p) S_\beta(y_p) S_\gamma(z_p).$$

源码里 `wq = q*wp[ip]*invvol` 已经包含体积归一化, 因此 `rho_arr` 存的是电荷密度而不是 cell 总电荷。`ion_lev` 非空时再乘电离态, 说明 field ionization species 的有效粒子电荷是在沉积时按粒子属性修正的。

这一层还有两个实现事实值得固定下来:

1. `rho_type = rho_fab.box().type()` 控制每个方向使用 node-centered 还是 cell-centered shape, 所以 charge deposition 并不是“永远拿一套 cell-centered $S(x)$ 到处乘”。
2. `ChargeDeposition.H` 本体看不到 `icomp`。旧/新时间层的 component 偏移已经在桥接层通过 `make_alias(..., icomp*nc, nc)` 或 `lockAdd(..., icomp*nc, nc)` 处理掉了。
3. `ChargeDeposition.H` 里看到的 `rho_fab` 也不总是最终那张 `rho`: GPU 路径直接 alias 目标 `MultiFab`, CPU 路径则可能只是 thread-local `local_rho`, 最后再 `lockAdd(...)` 回真实网格。

`Gpu::Atomic::AddNoRet` 是并行正确性必须条件: 不同粒子可能同时向同一个网格点沉积。没有 atomic, GPU/多线程下源码会出现竞态; 物理上表现为非确定的 ρ 和 $\mathbf{J}$ 误差。

这里还应再补一条维度语义, 否则很容易把 charge deposition 误读成“同一个 3D kernel 只是在低维时少掉几层循环”。源码其实不是这么组织的。`Source/Particles/Deposition/ChargeDeposition.H` 里, kernel 会先按几何重写粒子坐标, 再决定到底保留哪几个方向的 shape:

- 1D_Z

- 不再构造 x/y shape;
- 只保留 $z = (z_p - \text{xyzmin.z}) * \text{dinv.z}$ 和 $sz[\dots]$;
- 写回时也只循环 iz 。
- **RCYLINDER**
 - 先把笛卡尔粒子位置压成 $rp = \sqrt{x_p * x_p + y_p * y_p}$;
 - 再用 $x = (rp - \text{xyzmin.x}) * \text{dinv.x}$ 生成径向 shape $sx[\dots]$;
 - z 方向 shape 在这个 kernel 分支里根本不再构造。
- **RSPHERE**
 - 更进一步, 把位置压成球半径 $rp = \sqrt{x_p * x_p + y_p * y_p + z_p * z_p}$;
 - 同样只生成单个径向 $sx[\dots]$;
 - 写回时也只保留一维径向循环。

因此在 RCYLINDER / RSPHERE 下, ChargeDeposition.H 做的并不是“先在多维网格上沉积, 再由外层解释成柱/球几何”, 而是 kernel 一开始就把粒子位置改写成径向变量, 只在这一维上构造 support 并做原子加。换句话说:

1. 坐标压缩 已经发生在 kernel 内部;
2. shape 维数降低 不是后处理, 而是前端事实;
3. 后面的 inverse-volume scaling 负责的是几何体积因子, 不负责把一份“笛卡尔多维沉积结果”再翻译成柱/球坐标。

shared-memory 版本在这件事上和普通版本保持完全同构。doChargeDepositionSharedShapeN<...>() 的 WARPX_DIM_1D_Z / RCYLINDER / RSPHERE 分支同样先把坐标压成 z 或 r , 再只构造对应的一维 sz 或 sx , 最后沉到 tile-local buf。因此 shared-memory 路径改变的是执行拓扑, 不改变这些几何分支的物理语义。

这里还值得再补一层 RZ 语义, 否则读者容易把 charge deposition 误解成“把 2D kernel 原样照搬到柱坐标”。实际 WARPX_DIM_RZ 在写完 mode 0 之后, 还会继续沿

$$e^{im\theta}$$

的实部/虚部分量, 把 $m>0$ 的 azimuthal modes 显式写进额外 component。也就是说, RZ charge deposition 的 kernel 不只是决定 r - z 平面上的 node/cell shape 支撑, 还在同一轮原子加里把 Fourier 模态结构也一并 materialize 到 rho_arr。这条事实和前面的 icomp*nc component 偏移一起看尤其重要: 桥接层负责先把“旧/新时间层写到哪一块 component”整理好, 而 ChargeDeposition.H 则在那块已经选定的 component 空间里, 继续展开 mode 0 + $m>0$ 的 RZ 模态写回。

但这还不是 RZ charge deposition 的最后一步。对独立的 MultiParticleContainer::DepositCharge() 调用来说, 所有 species 沉积完成后, Source/Particles/MultiParticleContainer.cpp 还会统一执行

```
WarpX::GetInstance().ApplyInverseVolumeScalingToChargeDensity(rho[lev], lev);
```

也就是说, ChargeDeposition.H 写进去的首先仍是“尚未乘柱/球体积因子倒数”的局部源项; 真正带上 $2\pi r$ 、 $4\pi r^2$ 这类几何体积语义的最终 rho, 要到 species 汇总完以后才在容器层统一完成。这一点和前面 current deposition 的 inverse-volume scaling 是平行的: kernel 负责局部 shape 与守恒支持, 几何体积修正则留到更外层统一做。

对 RZ 问题, 验证这一层时应同时比较解析场、总电荷和粒子权重。官方 test_rz_electrostatic_sphere 以场的解析误差和总能量变化检验静电球; 在同一个诊断面上, 读者还可以把 mode-0 rho 乘以柱坐标 cell volume

$$\Delta V_{ij} = 2\pi r_i \Delta r \Delta z$$

并与粒子权重乘电荷比较。这个积分只能检验最终输出是否采用了相容的体积语义, 不能逐 cell 证明 ApplyInverseVolumeScalingToChargeDensity(); 若积分不一致, 也必须继续区分沉积、边界、诊断和积分区域。RZ 输入必须使用 warpx.rz, 不能使用同为二维编译但几何合同为 Cartesian XZ 的 warpx.2d。

RZ 的非零模态也必须单独理解。官方 test_rz_langmuir_multi 默认 warpx.n_rz_azimuthal_modes = 1; 若设置多个模态并输出 dump_rz_modes = 1, 应检查 $m>0$ 的实部/虚部是否写出, 并在给定方位角重建场。把 $\theta=0$ 视为

$$F(r, z, 0) = F_0(r, z) + F_{1,\text{real}}(r, z) + F_{2,\text{real}}(r, z)$$

后, 重建误差检验的是 diagnostics/writeback 与模态组合是否一致。官方 analysis_rz.py 的单模解析场断言不适用于多模态输入, 不能把它升级成多模态精确解 gate; 多模态结果需要与相应的解析解或独立收敛设计比较。

因此普通 charge deposition 的更准确调用链还应再细一层:

WarpXParticleContainer::DepositCharge

- > 选择 shared-memory 或普通路径
- > 确定 icode / xyzmin / depos_lev / ref_ratio
- > ABLASTR deposit_charge(...) 做 guard-check、CPU/GPU 暂存与 component 偏移
- > ChargeDeposition.H 做 node/cell shape、RZ modes 与原子加

这里还要把容器层接口的“做什么 / 不做什么”再拆开，否则很容易把 DepositCharge() 误读成一次调用就自动完成所有同步与边界修正。Source/Particles/WarpXParticleContainer.cpp 实际上把这些职责分成几层开关：

- reset
 - 只决定是否在本次 species 沉积前对 rho->setVal(0., icode*nc, nc, rho->nGrowVect());
 - 它不改变 shape kernel，也不改变时间层语义。
- local
 - 只决定沉积后是否做 SumBoundary(...);
 - local = true 时，结果可以停留在本地 guard/valid 区布局，不强制做跨 patch 通信。
- apply_boundary_and_scale_volume
 - 在 RZ / RCYLINDER / RSPHERE 下触发 ApplyInverseVolumeScalingToChargeDensity(...);
 - 在非柱/球几何下则改为 ApplyRhoFieldBoundary(...), 也就是按 PEC 之类边界条件去反射/修正 rho;
 - 它做的是沉积后的外层场语义整理，不参与粒子到网格的局部 shape 写入。

如果再看多层入口 WarpXParticleContainer::DepositCharge(const MultiLevelScalarField& rho, ..., interpolate_across_levels, ...), 还会再多一层：

- interpolate_across_levels
 - 只在每个 level 都沉完之后，执行 fine -> coarse 的 Coarsen(...) + ParallelAdd(...);
 - 它处理的是 AMR 层间一致性，而不是单 level 内的 tile/guard/source kernel。

这样一来，DepositCharge() 这组接口的职责边界就更清楚了：

1. DepositCharge(pti, ...)
 - 负责单 tile、单批粒子的 bridge contract 与 kernel launch;
2. DepositCharge(rho, lev, local, reset, apply_boundary_and_scale_volume, icode)
 - 负责单个 level 上的 reset、逐 tile 遍历、volume scaling / PEC 边界修正与 guard-cell 通信;
3. DepositCharge(multilevel rho, ..., interpolate_across_levels, icode)
 - 负责多 level 汇总后是否继续做 fine-to-coarse average down。

这组分层还有一个对读者很重要的负面结论：**charge deposition kernel 本身并不承担 AMR average-down、PEC 边界反射、或全局 guard-cell 交换**。这些都属于更外层的容器/通信语义。把这一点讲清以后，第 5 章里“粒子怎样写入局部 ρ ”和“最终可用于 Maxwell/diagnostics 的 ρ 怎样整理完成”才不会混成一件事。

这里再往 AMR 邻接关系上压一层，会看到 DepositCharge() 其实同时服务两种不同场景，而它们的跨层语义不能混看。

第一种是**独立 charge 诊断/取样**。例如 MultiParticleContainer::DepositCharge(rho, relative_time) 里，WarpX 明确设置的是：

- local = true
- reset = false
- apply_boundary_and_scale_volume = false
- interpolate_across_levels = false

然后逐 species 往调用者给定的 rho 上累加。也就是说，这条接口在这里的任务只是“把所有 species 的局部沉积加到同一张多层 rho 上”；真正的跨 species 清零由最外层统一先做，柱/球 inverse-volume scaling 也被推迟到 species 全部累加完成之后再统一做。它并不在 species 级别就把 fine level 自动 average-down 到 coarse level。

第二种是**运行时 AMR coarse-buffer source routing**。PhysicalParticleContainer::Evolve() 里，当 has_buffer 为真时，PartitionParticlesInBuffers(...) 会先把粒子数组重排成：

1. 前 n_fine_deposit 个粒子沉到 fine patch 主场 rho_fp
2. 后 np-n_fine_deposit 个粒子直接沉到 coarse buffer 场 rho_buf

对应源码就是：

```
DepositCharge(pti, wp, ion_lev, rho, 0, 0,
              np_to_deposit, thread_num, lev, lev);
```

```
...
DepositCharge(pti, wp, ion_lev, crho, 0, np_to_deposit,
              np-np_to_deposit, thread_num, lev, lev-1);
```

以及 push 后对 `icomp=1` 的完全平行调用。这里最重要的事实是：buffer 粒子不是先沉到 fine rho_fp 再 restrict 到 coarse。它们从一开始就以 `depos_lev = lev-1` 进入 `DepositCharge(...)`，因此：

- tile box 会先被 `coarsen(pti.tilebox(), RefRatio(lev-1))`
- `dinv` 直接取 coarse level 的 `InvCellSize(lev-1)`
- `xyzmin` 也直接按 coarse level 几何和对应 `time_shift_delta` 计算

换句话说，`rho_buf` 不是事后从 fine rho 裁出来的一块通信缓存，而是在 coarse patch 几何上直接生成的源项场。

这也解释了为什么前面那组多层接口参数不能直接替代运行时 source synchronization。运行时粗细层一致性真正依赖的是后面的 `SyncRho()` / `AddRhoFromFineLevelandSumBoundary(...)` 骨架：先把 fine rho_fp coarsen 成 `rho_cp`，再和 `rho_buf` 合并，再通过临时 coarse-side `fine_lev_cp + OwnerMask` 去重后并回 coarse 主场。也就是说：

1. `DepositCharge(..., depos_lev = lev-1)` 解决的是“buffer 粒子该按哪一层几何直接沉积”；
2. `interpolate_across_levels` 解决的是“一个显式给定的多层 rho 是否要在函数尾做平均下传”；
3. `SyncRho()` 解决的才是运行时 field-solver source 使用前的 coarse/fine/buffer 一致性整理。

把这三层分开以后，就不会误以为只要 `DepositCharge()` 有多层入口，运行时所有 `rho_buf` 邻接问题都已经在那一层自动解决了。

最后还要把 charge deposition 与 current deposition 的 implicit/守恒语义明确拆开。`ChargeDeposition.H` 的任务是把某一个已选定时间层上的粒子 cloud 写成 ρ ，它只消费当前位置、`ion_lev`、`rho_type`、`xyzmin`、`dinv` 和几何分支；它并不恢复一步轨迹，也不构造 old/new shape difference，更不判断 `Esirkepov` / `Villasenor` / `Direct` / `Vay`。这些算法名属于 current deposition 的选择空间，而不是 charge deposition 的选择空间。

因此 `DepositCharge()` 的“implicit 细节”不能按 current deposition 的 implicit 路径去读。对 ρ 来说，真正需要固定的是：

1. 旧/新时间层由外层 `icomp=0/1` 与 `time_shift_delta` -> `LowerCorner(...)` 表达；
2. 粒子位置若需要额外时间平移，由更外层 `MultiParticleContainer::DepositCharge(relative_time)` 临时 `PushX(...)` 表达；
3. coarse-buffer 粒子若要沉到粗层，由 `depos_lev = lev-1` 直接改写 `tilebox`、`dinv` 与 `xyzmin`；
4. shared-memory 分支只改变 binning、tile-local buffer 和回写拓扑，不引入新的物理算法；
5. PEC、guard exchange、inverse-volume scaling 和 AMR average-down 都在容器/通信层完成，不在 `ChargeDeposition.H` kernel 内完成。

读者在升级 WarpX 后，应重新核对 `icomp/time_shift_delta`、`ABLAstr` 的越界保护与 CPU/GPU 暂存、形状函数分派、RZ 模式和 atomic writeback；这些入口共同决定本节的旧/新时间层与局部写回解释是否仍适用。源码定位只能说明入口没有漂移，不能替代这里的物理推导或数值验证。

这组负面边界是本节的收口点：charge deposition 的局部 kernel 是“单时间层 shape 加权的源项采样器”，而不是 charge-conserving current mover。离散连续性方程仍由后面的 current deposition 与 `SyncCurrentAndRho()` 共同承担；`DepositCharge()` 在这条链里提供的是与旧/新时间层、AMR buffer 和几何体积因子一致的 ρ^n/ρ^{n+1} 输入。

5.10 Direct current deposition: 非守恒但直观的速度加权沉积

Direct current deposition 的核心 kernel 是 `Source/Particles/Deposition/CurrentDeposition.H`。下面两段分别取自同一函数的前半段和写入段；中间省略的是与 x 方向同构的 y/z 方向 shape 初始化。先看粒子电流权重和半位置：

```
template <int depos_order>
AMREX_GPU_HOST_DEVICE AMREX_INLINE
void doDepositionShapeNKernel([[maybe_unused]] const amrex::ParticleReal xp,
                             [[maybe_unused]] const amrex::ParticleReal yp,
                             [[maybe_unused]] const amrex::ParticleReal zp,
                             const amrex::ParticleReal wq,
                             const amrex::ParticleReal vx,
                             const amrex::ParticleReal vy,
                             const amrex::ParticleReal vz,
                             amrex::Array4<amrex::Real> const& jx_arr,
                             amrex::Array4<amrex::Real> const& jy_arr,
                             amrex::Array4<amrex::Real> const& jz_arr,
```

```

        amrex::IntVect const& jx_type,
        amrex::IntVect const& jy_type,
        amrex::IntVect const& jz_type,
        const amrex::Real relative_time,
        const amrex::XDim3 & dinv,
        const amrex::XDim3 & xyzmin,
        const amrex::Real invvol,
        const amrex::Dim3 lo,
        [[maybe_unused]] const int n_rz_azimuthal_modes)
{
    // wqx, wqy wqz are particle current in each direction
    #if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER)
        const amrex::Real xpmid = xp + relative_time*vx;
        const amrex::Real ypmid = yp + relative_time*vy;
        const amrex::Real rpmid = std::sqrt(xpmid*xpmid + ypmid*ypmid);
        const amrex::Real costheta = (rpmid > 0._rt ? xpmid/rpmid : 1._rt);
        const amrex::Real sintheta = (rpmid > 0._rt ? ypmid/rpmid : 0._rt);
        const amrex::Real wqx = wq*invvol*(+vx*costheta + vy*sintheta);
        const amrex::Real wqy = wq*invvol*(-vx*sintheta + vy*costheta);
        const amrex::Real wqz = wq*invvol*vz;
    #else
        const amrex::Real wqx = wq*invvol*vx;
        const amrex::Real wqy = wq*invvol*vy;
        const amrex::Real wqz = wq*invvol*vz;
    #endif

    Compute_shape_factor< depos_order > const compute_shape_factor;
    const double xmid = ((xp - xyzmin.x) + relative_time*vx)*dinv.x;
    double sx_node[depos_order + 1] = {0.};
    double sx_cell[depos_order + 1] = {0.};
    int j_node = 0;
    int j_cell = 0;
    if (jx_type[0] == NODE || jy_type[0] == NODE || jz_type[0] == NODE) {
        j_node = compute_shape_factor(sx_node, xmid);
    }
    if (jx_type[0] == CELL || jy_type[0] == CELL || jz_type[0] == CELL) {
        j_cell = compute_shape_factor(sx_cell, xmid - 0.5);
    }
}

```

relative_time 在显式路径通常是 $-0.5*dt$ 。由于 DepositCurrent() 被调用时粒子位置已经是 $\mathbf{x}^{n+1}$ ，这行

$$x_{\text{mid}} = \frac{x^{n+1} - x_{\text{min}} - \frac{1}{2}v_x \Delta t}{\Delta x}$$

把沉积位置移回半步。Direct deposition 的电流权重就是 $qw_p \mathbf{v}_p / \Delta V$ 。

最终写入数组的实现位于 Source/Particles/Deposition/CurrentDeposition.H 的 direct current kernel:

```

// Deposit current into jx_arr, jy_arr and jz_arr
#if defined(WARPX_DIM_XZ) || defined(WARPX_DIM_RZ)
    for (int iz=0; iz<=depos_order; iz++){
        for (int ix=0; ix<=depos_order; ix++){
            amrex::Gpu::Atomic::AddNoRet(
                &jx_arr(lo.x+j_jx+ix, lo.y+l_jy+iz, 0, 0),
                sx_jx[ix]*sz_jx[iz]*wqx);
            amrex::Gpu::Atomic::AddNoRet(
                &jy_arr(lo.x+j_jy+ix, lo.y+l_jy+iz, 0, 0),
                sx_jy[ix]*sz_jy[iz]*wqy);

```

```

        amrex::Gpu::Atomic::AddNoRet(
            &jz_arr(lo.x+j_jz+ix, lo.y+l_jz+iz, 0, 0),
            sx_jz[ix]*sz_jz[iz]*wqz);
    }
}
#elif defined(WARPX_DIM_3D)
    for (int iz=0; iz<=depos_order; iz++){
        for (int iy=0; iy<=depos_order; iy++){
            for (int ix=0; ix<=depos_order; ix++){
                amrex::Gpu::Atomic::AddNoRet(
                    &jx_arr(lo.x+j_jx+ix, lo.y+k_jx+iy, lo.z+l_jx+iz),
                    sx_jx[ix]*sy_jx[iy]*sz_jx[iz]*wqx);
                amrex::Gpu::Atomic::AddNoRet(
                    &jy_arr(lo.x+j_jy+ix, lo.y+k_jy+iy, lo.z+l_jy+iz),
                    sx_jy[ix]*sy_jy[iy]*sz_jy[iz]*wqy);
                amrex::Gpu::Atomic::AddNoRet(
                    &jz_arr(lo.x+j_jz+ix, lo.y+k_jz+iy, lo.z+l_jz+iz),
                    sx_jz[ix]*sy_jz[iy]*sz_jz[iz]*wqz);
            }
        }
    }
#endif
}

```

3D 形式可以写为

$$J_{x,i+\alpha,j+\beta,k+\gamma} \leftarrow J_{x,i+\alpha,j+\beta,k+\gamma} + qw_v v_x \frac{1}{\Delta V} S_{\alpha}^{J_x}(x_p^{n+1/2}) S_{\beta}^{J_x}(y_p^{n+1/2}) S_{\gamma}^{J_x}(z_p^{n+1/2}),$$

J_y, J_z 同理，但使用各自 staggering 对应的 shape 数组 $sx_jy/sy_jy/sz_jy$ 与 $sx_jz/sy_jz/sz_jz$ 。Direct 路径的优点是简单，缺点是这个公式没有强制把 ρ^n 和 ρ^{n+1} 的差精确写成离散散度。

这里还要补一条容易被略掉的接口边界：即使到了 implicit 版本，direct deposition 的 contract 也没有升级成“恢复完整轨迹，再按边界裁剪后沉积”。doDepositionShapeNImplicit(...) 只是把 γ^{-1} 改成由 u_n 与 $u_{n+1/2}$ 共同恢复，然后把

```
const amrex::Real relative_time = 0._rt;
```

喂回同一个 doDepositionShapeNKernel(...)。源码见 Source/Particles/Deposition/CurrentDeposition.H。也就是说，direct 的 implicit 语义依旧是：

1. 选定一个时间中心位置；
2. 用该时间层速度形成 $qwv/\Delta V$ ；
3. 按当前 staggering shape 直接沉回 Jx/Jy/Jz。

它并不显式接收 domain_double、do_cropping，也不把粒子轨迹当成一条可以在 PEC/PECInsulator 附近截断、再按 cell crossing 重分段的几何对象。这也是为什么在 near-boundary 守恒场景里，direct 不能简单充当 Villasenor 的“廉价替身”。

5.11 Esirkepov current deposition: 用新旧形函数差构造连续性方程

阅读 Esirkepov 论文时，最容易发生的记号错位是把论文的方向分解、WarpX 的前缀累加变量和最终网格电流分量直接画等号。它们可以建立结构对应，但不是同一个层次的对象。本章采用的源码快照以下表作为记号入口：

论文层对象	WarpX 当前实现	读者应保留的边界
W^1 : x 向 shape difference	<code>sx_old-sx_new</code> 沿 i 累加到 <code>sdx_i</code> ，再写入 Jx	这是 3D Esirkepov kernel 的方向分工，不等于所有 RZ/XZ 数组布局
W^2 : y 向 shape difference	<code>sy_old-sy_new</code> 沿 j 累加到 <code>sdy_j</code> ，再写入 Jy	RZ/XZ 中 out-of-plane 分量会使用不同的几何分支
W^3 : z 向 shape difference	<code>sz_old-sz_new</code> 沿 k 累加到 <code>sdz_k</code> ，再写入 Jz	1D/2D/RZ 会减少实际循环维度，不能照搬 3D 下标

论文层对象	WarpX 当前实现	读者应保留的边界
old/new form factor	Compute_shape_factor 与 Compute_shifted_shape_factor 生成 sx/sy/sz old/new 数组	shifted shape 的首索引对齐是源码合同，不是论文排版中的隐含步骤
transverse tensor-product factor	one_third/one_sixth 组成 old-old、old-new、new-old、new-new 混合平均	该对应由预印本与源码核对得到，不表示 CPC 定稿已逐页比较
current normalization	invdtd.x/y/z = transverse inverse cell area / dt	不能把三个分量都简化成单独的 1/dt

表中的对应关系应逐项回查源码；它消除的是论文记号到该源码快照变量的映射歧义，不替代 SyncCurrent()、AMR coarse-fine、边界同步或全 geometry/order runtime regression。

Esirkepov 入口在 Source/Particles/Deposition/CurrentDeposition.H:

```
template <int depos_order>
void doEsirkepovDepositionShapeN (const GetParticlePosition<PIdx>& GetPosition,
                                   const amrex::ParticleReal * const wp,
                                   const amrex::ParticleReal * const uxp,
                                   const amrex::ParticleReal * const uyp,
                                   const amrex::ParticleReal * const uzp,
                                   const int* ion_lev,
                                   const amrex::Array4<amrex::Real>& Jx_arr,
                                   const amrex::Array4<amrex::Real>& Jy_arr,
                                   const amrex::Array4<amrex::Real>& Jz_arr,
                                   long np_to_deposit,
                                   amrex::Real dt,
                                   amrex::Real relative_time,
                                   const amrex::XDim3 & dinv,
                                   const amrex::XDim3 & xyzmin,
                                   amrex::Dim3 lo,
                                   amrex::Real q,
                                   [[maybe_unused]] int n_rz_azimuthal_modes,
                                   const amrex::Array4<const int>& reduced_particle_shape_mask,
                                   bool enable_reduced_shape
                                   )
{
    bool const do_ionization = ion_lev;

    amrex::XDim3 const invdtd = amrex::XDim3{(1.0_rt/dt)*dinv.y*dinv.z,
                                              (1.0_rt/dt)*dinv.x*dinv.z,
                                              (1.0_rt/dt)*dinv.x*dinv.y};

    Real constexpr inv_c2 = PhysConst::inv_c2;
    Real constexpr one_third = 1.0_rt / 3.0_rt;
    Real constexpr one_sixth = 1.0_rt / 6.0_rt;

    enum eb_flags : int { has_reduced_shape, no_reduced_shape };
    const int reduce_shape_runtime_flag = (enable_reduced_shape && (depos_order>1)) ? has_reduced_shape : no_reduced_shape;

    invdtd.x=(1/dt)*dinv.y*dinv.z 不是普通的  $1/\Delta t$ 。因为  $J_x$  位于 x-face，离散连续性中  $J_x$  的差分还会除以  $\Delta x$ ，所以 current 的量纲需要配合横截面积  $1/(\Delta y \Delta z)$ 。源码使用  $dinv.y*dinv.z/dt$ ，后续再由差分 operator 处理 x 向差分。

    粒子旧/新位置和 shape 数组由 CurrentDeposition.H 的 Esirkepov 入口生成:

    amrex::ParallelFor( TypeList<CompileTimeOptions<has_reduced_shape,no_reduced_shape>>>{ },
                        {reduce_shape_runtime_flag},
                        np_to_deposit, [=] AMREX_GPU_DEVICE (long ip, auto reduce_shape_control) {
                            Real const gaminv = 1.0_rt/std::sqrt(1.0_rt + uxp[ip]*uxp[ip]*inv_c2
```



```

+ uyp[ip]*uyp[ip]*inv_c2
+ uzp[ip]*uzp[ip]*inv_c2);

Real wq = q*wp[ip];
if (do_ionization){
    wq *= ion_lev[ip];
}

ParticleReal xp, yp, zp;
GetPosition(ip, xp, yp, zp);

double const x_new = (xp - xyzmin.x + (relative_time + 0.5_rt*dt)*uxp[ip]*gaminv)*dinv.x;
double const x_old = x_new - dt*dinv.x*uxp[ip]*gaminv;
double const y_new = (yp - xyzmin.y + (relative_time + 0.5_rt*dt)*uyp[ip]*gaminv)*dinv.y;
double const y_old = y_new - dt*dinv.y*uyp[ip]*gaminv;
double const z_new = (zp - xyzmin.z + (relative_time + 0.5_rt*dt)*uzp[ip]*gaminv)*dinv.z;
double const z_old = z_new - dt*dinv.z*uzp[ip]*gaminv;

const Compute_shape_factor< depos_order > compute_shape_factor;
const Compute_shifted_shape_factor< depos_order > compute_shifted_shape_factor;
const Compute_shifted_shape_factor< 1 > compute_shifted_shape_factor_order1;

double sx_new[depos_order + 3] = {0.};
double sx_old[depos_order + 3] = {0.};
const int i_new = compute_shape_factor(sx_new+1, x_new );
const int i_old = compute_shifted_shape_factor(sx_old, x_old, i_new);

double sy_new[depos_order + 3] = {0.};
double sy_old[depos_order + 3] = {0.};
const int j_new = compute_shape_factor(sy_new+1, y_new);
const int j_old = compute_shifted_shape_factor(sy_old, y_old, j_new);

double sz_new[depos_order + 3] = {0.};
double sz_old[depos_order + 3] = {0.};
const int k_new = compute_shape_factor(sz_new+1, z_new );
const int k_old = compute_shifted_shape_factor(sz_old, z_old, k_new );

int dil = 1, diu = 1;
if (i_old < i_new) { dil = 0; }
if (i_old > i_new) { diu = 0; }
int djl = 1, dju = 1;
if (j_old < j_new) { djl = 0; }
if (j_old > j_new) { dju = 0; }
int dkl = 1, dku = 1;
if (k_old < k_new) { dkl = 0; }
if (k_old > k_new) { dku = 0; }

```

这里 x_{new} 与 x_{old} 是同一时间步轨迹的两端。显式路径中 $\text{relative_time} = -0.5 \cdot dt$ ，而粒子数组位置是 push 后位置，于是

$$x_{\text{new}} = x^{n+1} \left(-\frac{1}{2} \Delta t + \frac{1}{2} \Delta t \right) v_x = x^{n+1}, \quad x_{\text{old}} = x^{n+1} - v_x \Delta t = x^n.$$

最后的 3D 电流公式位于同一 Esirkepov kernel 的写回部分：

```

for (int k=dkl; k<=depos_order+2-dku; k++) {
    for (int j=djl; j<=depos_order+2-dju; j++) {
        amrex::Real sdx_i = 0._rt;
        for (int i=dil; i<=depos_order+1-diu; i++) {

```

```

        sdxi += wq*invdtd.x*(sx_old[i] - sx_new[i])*
            one_third*(sy_new[j]*sz_new[k] + sy_old[j]*sz_old[k])
            +one_sixth*(sy_new[j]*sz_old[k] + sy_old[j]*sz_new[k]));
        amrex::Gpu::Atomic::AddNoRet( &Jx_arr(lo.x+i_new-1+i, lo.y+j_new-1+j, lo.z+k_new-1+k), sdxi);
    }
}
}
for (int k=dkl; k<=depos_order+2-dku; k++) {
    for (int i=dil; i<=depos_order+2-diu; i++) {
        amrex::Real sdyj = 0._rt;
        for (int j=djl; j<=depos_order+1-dju; j++) {
            sdyj += wq*invdtd.y*(sy_old[j] - sy_new[j])*
                one_third*(sx_new[i]*sz_new[k] + sx_old[i]*sz_old[k])
                +one_sixth*(sx_new[i]*sz_old[k] + sx_old[i]*sz_new[k]));
            amrex::Gpu::Atomic::AddNoRet( &Jy_arr(lo.x+i_new-1+i, lo.y+j_new-1+j, lo.z+k_new-1+k), sdyj);
        }
    }
}
for (int j=djl; j<=depos_order+2-dju; j++) {
    for (int i=dil; i<=depos_order+2-diu; i++) {
        amrex::Real sdzk = 0._rt;
        for (int k=dkl; k<=depos_order+1-dku; k++) {
            sdzk += wq*invdtd.z*(sz_old[k] - sz_new[k])*
                one_third*(sx_new[i]*sy_new[j] + sx_old[i]*sy_old[j])
                +one_sixth*(sx_new[i]*sy_old[j] + sx_old[i]*sy_new[j]));
            amrex::Gpu::Atomic::AddNoRet( &Jz_arr(lo.x+i_new-1+i, lo.y+j_new-1+j, lo.z+k_new-1+k), sdzk);
        }
    }
}
}
}

```

这不是 $q\mathbf{v}S$ 的 direct 形式。它用新旧 shape 的差构造电流。例如 J_x 内部累计量可概括为

$$\Delta J_x(i, j, k) \propto \sum_{i' \leq i} q \frac{S_x^n(i') - S_x^{n+1}(i')}{\Delta t \Delta y \Delta z} \overline{S_y S_z}(j, k),$$

其中横向平均 $\overline{S_y S_z}$ 在源码中由 one_third 和 one_sixth 组合给出。这种构造保证对 Jx/Jy/Jz 做离散散度时，望远镜求和会还原新旧电荷形函数差：

$$\nabla_h \cdot \mathbf{J}^{n+1/2} = -\frac{\rho^{n+1} - \rho^n}{\Delta t}$$

在同一 shape、同一网格中心和同一边界同步规则下成立。这就是 Esirkepov 路径比 direct 路径更适合作为显式电磁 PIC 默认守恒沉积的原因。

如果把这段源码和 Esirkepov 2001 的 paper-level 推导对起来，逻辑会更清楚。论文第 3 节先把单粒子位移导致的总电荷变化写成

$$W^1 + W^2 + W^3 = S(x + \Delta x, y + \Delta y, z + \Delta z) - S(x, y, z),$$

然后再要求三个方向的电流差分分别去承担 $W^1/W^2/W^3$ 。WarpX 没有显式保留 $W(i, j, k, m)$ 这个中间数组，但实现上的等价结构就是：

1. Compute_shifted_shape_factor(...) 先把 old/new shape 放到统一索引框架内；
2. sx_old-sx_new、sy_old-sy_new、sz_old-sz_new 分别承担三个方向的差分源；
3. one_third/one_sixth 横向平均把多维 tensor-product shape 的耦合补回。

因此源码里的 prefix-like sdxi/sdyj/sdzk 不是经验配方，而是论文 density decomposition 在二阶 spline/tensor-product family 下的一种直接程序化实现。

如果把这条对应再压到循环级，关系会更清楚。3D Esirkepov kernel 并不是先算一个总的 $S_{\text{new}} - S_{\text{old}}$ 再数值拆账，而是从三组 prefix loop 的拓扑上就已经把 $W^1/W^2/W^3$ 固定下来了。对固定的 j, k , Jx 那组循环实际在做

`sdxi += (sx_old[i] - sx_new[i]) * yz_mixed_average(j,k);`

其中 `yz_mixed_average` 正是 $1/3, 1/6, 1/6, 1/3$ 的 `old/new` 双横向组合；然后沿 `i` 方向把它累加并写回 `Jx`。`Jy` 和 `Jz` 两组循环只是把同一结构置换到 `y`、`z` 方向，于是分别承担 W^2 、 W^3 。也就是说：

- `sdxi` 前缀和承担 W^1 ；
- `sdyj` 前缀和承担 W^2 ；
- `sdzk` 前缀和承担 W^3 。

如果把论文里的八项结构再和这三组 `loop` 并排看，分工其实非常具体： W^1 总是把“`x` 方向 `old/new` difference”当成主变量，而把 `y,z` 两方向放进 $1/3, 1/6$ 的 `mixed average`； W^2 则把主变量换成 `y` 方向差分； W^3 再换成 `z`。所以源码里真正被“前缀累加”的不是一个抽象的三维总差分，而是三份已经在论文里各自分好工的方向性 `shape` 变化。这样读的时候，`sdxi/sdyj/sdzk` 就不只是三个长得很像的循环，而是：

- `sdxi`：先提取 `x` 向直接变化，再让 `y,z` 耦合平均去补横向几何；
- `sdyj`：先提取 `y` 向直接变化，再让 `x,z` 去补横向几何；
- `sdzk`：先提取 `z` 向直接变化，再让 `x,y` 去补横向几何。

这条对应第 5 章很关键，因为它说明 Eq. (23) 在源码里并没有“蒸发成一堆经验循环”，而是被压缩进了三组方向前缀累加的循环骨架本身。

预印本已足够把 Eq. (23) 的结构写清：每个方向的 W^m 都是八个 `old/new` corner-like `shape` 值的线性组合，而且只出现两种系数 $1/3$ 与 $1/6$ 。这说明 WarpX 里显式写出来的 `one_third / one_sixth` 不是局部数值调味，而是论文唯一性分解的直接遗留物；源码快照中横向平均项的结构，并不是“为了让公式看起来对称”，而是为了让三方向分解在加总后精确回到总的 `shape` 差分。

这里还应把论文的 `claim` 再说硬一点。Esirkepov 2001 并没有把 `density decomposition` 当成众多可选配方之一，而是明确声称：在线性、零位移退化、坐标置换对称和总差分守恒这些自然条件下，这就是定义粒子相关电流的唯一允许过程。这样回头看 WarpX 的 `sdxi/sdyj/sdzk + one_third/one_sixth`，它们就不再只是“实现选用的一组常数”，而更接近一条被论文唯一性条件挑出来、随后在现代 `tensor-product kernel` 里程碑化保存下来的结构。

这里需要把证据层级说清。关于 Esirkepov 的论文论证使用者 arXiv 预印本 physics/9901047，而不是 Elsevier Computer Physics Communications 135(2) 的 publisher-formatted 定稿 PDF。因此可直接依赖的结论是：

1. Eq. (23) 的 $W^1/W^2/W^3$ 结构与 $1/3, 1/6$ 系数；
2. `density decomposition` 的唯一性口径；
3. 二阶 `spline / tensor-product form-factor` 如何压成可编程局部算法；
4. 这些 paper-level 结构在 WarpX `sdxi/sdyj/sdzk + one_third/one_sixth` 中的程序化对应。

但本章不应声称“2001 CPC 发表版已逐行核对”。可读的预印本支撑 Eq. (23) 到源码 `loop` 的主叙述；发表版只支持书目信息和摘要级事实。两种标题确有稳定差别：预印本是

- Exact charge conservation scheme for Particle-in-Cell simulations for a big class of form-factors

而 CPC 发表版是

- Exact charge conservation scheme for Particle-in-Cell simulation with an arbitrary form-factor

标题差异不能推出正文内容有何变化，但说明预印本与发表版不能被预设为逐字相同。发表版身份为 Computer Physics Communications 135(2), 144-153 (2001), DOI 为 10.1016/S0010-4655(00)00228-9; abstract、section wording、Eq. (23) 排版和二阶 `spline` 说明仍须以 publisher PDF 为准。

5.11.1 论文、源码、代数合同与 runtime 证据的分层

为了避免把不同强度的证据压成一句“Esirkepov 已验证”，本章把当前可复核材料分成四层：

证据层	当前材料	可以支持的结论	不能支持的结论
论文/索引摘要	作者 arXiv 预印本、CPC 书目信息与 indexed abstract	$W^1/W^2/W^3$ 、Eq. (23)、arbitrary form-factor、直线轨迹、无需 Poisson solve 的 paper-level 叙述	CPC 定稿的逐页排版、section 编号和逐式编辑差异
源码快照	ShapeFactors.H、CurrentDeposition.H、WarpXParticleContainer.cpp	old/new shape 对齐、sdxi/sdyj/sdzk 前缀循环、 $1/3/1/6$ 混合平均、几何/执行分支	所有 geometry/order 组合都已端到端等价

证据层	当前材料	可以支持的结论	不能支持的结论
代数/源码核查	记号映射、密度分解和有限样本公式恒等式	记号映射、密度分解和有限样本公式恒等式在当前定义下成立	公式恒等式自动等价于 GPU kernel 或 AMR source synchronization
runtime consumer	1D/2D/3D Langmuir、RZ、RCYLINDER/RSPHERE 与 MR contracts	指定案例和边界下的 field/charge/observable 结果及其 PASS/BOUNDARY 分类	从局部案例外推完整 Cartesian product、默认参数修复或正式收敛阶

因此，本节后文的“论文、源码、运行”是证据层叠加，不是把最弱层自动升级成最强层。尤其是 `runtime consumer` 只能回答某个输入、几何和诊断量是否成立；它不能反向证明 CPC 发表版逐式一致，也不能替代 `SyncCurrentAndRho()` 的独立同步合同。

发表版的书目信息和索引摘要可支持“任意 form-factor、直线轨迹假设、无需 Poisson solve、2D/3D demonstration”等摘要级事实；可读的预印本支撑 Eq. (23) 到 `sdx/sdyj/sdzk` 的主论证。由于 publisher PDF 尚未取得，abstract 的正式排版、section numbering、公式排版和二阶 spline 段落不能声称已经逐页核对。

同样，当前预印本也已经足够把论文内部的 section 结构稳定绑定到第 5 章的主叙述，而不必等发表版 PDF 才能继续写。更准确地说：

1. **Section 2 Continuity equation in finite differences**
 - 先把离散 Maxwell + leapfrog mover 压成 $(\rho^{n+1} - \rho^n)/dt + \nabla_h \cdot J = 0$;
 - 这正对应本章先讲“为什么 direct qvS 不自动守恒”，再讲 source-side continuity contract 的必要性。
2. **Section 3 Density decomposition**
 - 先定义 $W^1/W^2/W^3$ ，再要求它们加总恢复总的 shape 差分；
 - 这正对应 WarpX 里 `sx_old-sx_new`、`sy_old-sy_new`、`sz_old-sz_new` 三组方向差分源，以及三组 prefix loop 的分工。
3. **Section 4 Computing of the current with second-order polynomial form-factor**
 - 把算法明确压成 `old shape -> push -> new shape -> difference -> directional current` 的可编码步骤；
 - 这正对应 `Compute_shifted_shape_factor(...)`、old/new arrays 同框、再到 `sdx/sdyj/sdzk` 前缀累加这条现代 kernel 骨架。

这层结构性对应很重要，因为它说明第 5 章当前并不是只拿 Eq. (23) 这一条孤立公式去贴源码，而是已经能把预印本的问题设定、唯一性分解、到二阶 spline 算法化三个层次，分别对回 WarpX 的守恒目标、directional decomposition、kernel skeleton。

为了避免把“论文结构已能对回源码”误读成“发表版逐页已核对”，当前证据矩阵固定如下：

论文层次	可用证据	WarpX 对应	证据等级
Section 2: 离散连续性方程	arXiv 预印本全文、MinerU Markdown	<code>SyncCurrentAndRho()</code> 、 <code>divE-rho/epsilon_0</code> regression	preprint-backed + source-grounded
Section 3: $W^1/W^2/W^3$ density decomposition	预印本公式与中文讲解	<code>sx_old-sx_new</code> 、 <code>sy_old-sy_new</code> 、 <code>sz_old-sz_new</code>	preprint-backed + source-grounded
Section 4: 二阶 spline 算法骨架	预印本算法段落	shifted-shape helper; <code>sdx/sdyj/sdzk</code> prefix loops	preprint-backed + source-grounded
CPC 发表版题名、卷期、页码、DOI 和公开摘要	ScienceDirect/公开书目元数据	<code>Esirkepovcpc01</code> bibliography key	publication-metadata verified
CPC 发表版 abstract、section numbering、Eq. (23) 排版、二阶 spline 文字	indexed abstract compare 已完成; publisher PDF 仍未取得	摘要级主张可绑定，逐页公式仍无证据	abstract verified / PDF open

这张表给出本章的证据边界：前三行可以直接进入正文，第四行用于出版身份和引用信息，第五行可以支持摘要级算法主张，但不能写成发表版全文逐页核对。

发表版公开索引摘要与 arXiv 预印本摘要在 Cartesian geometry、arbitrary quasi-particle form-factor、straight-line trajectory、无需 Poisson solve、唯一线性组合和 2D/3D demonstration 等主张上可以逐项对照。这个层级只支持摘要级结论；Eq. (23) 排版、section numbering、发表版图表和二阶 spline 正文仍保留 PDF 缺口。

发表版证据边界 因此，本章可使用的最强但不过度的结论是：**Esirkepov** 的守恒分解已有预印本公式、该源码快照和代表性运行案例的三层交叉复核；**CPC** 发表版身份和摘要级事实已核实，但 **publisher-PDF** 的逐行比较仍未完成。这条边界避免把可读预印本、索引摘要和出版定稿混为同一来源。

公式层还应做独立的代数核查：对任意 old/new shape 分量检查

$$W^1 + W^2 + W^3 = (S_x^{old} + Delta S_x)(S_y^{old} + Delta S_y)(S_z^{old} + Delta S_z) - S_x^{old} S_y^{old} S_z^{old}.$$

这只验证论文 Eq.(23) 的代数分解，不替代 WarpX kernel、网格散度或端到端 regression；它说明 1/2 横向平均和 1/3 三重差分是可直接检验的局部恒等式，而不是仅凭文字接受的解释。

源码层可从 `CurrentDeposition.H` 依次核对 `doEsirkepovDepositionShapeN`、`Compute_shifted_shape_factor`、`invdtd`、`one_third/one_sixth`、`sdx/sdyj/sdzk`、三方向 old/new shape difference 和 `Jx/Jy/Jz` writeback。这证明正文所描述的 skeleton 有源码对应，但不是数值 kernel regression。

在这个 Esirkepov skeleton 之上，还要读清 geometry/order 的分支约束：`CurrentDeposition.H` 分别编译 `1D_Z/XZ/RZ/RCYLINDER/RSPHERE/3D` Vay 对 `RZ/1D` 有显式拒绝，Vay 与 `implicit` 互斥，径向 geometry 不进入 shared-memory current kernel。这些事实说明可用入口和拒绝条件，不能替代所有 geometry × order 组合的运行验证。

Villasenor 的组织方式则完全不同。`VillasenorDepositionShapeNKernel(...)` 在完成轨迹恢复和 boundary crop 之后，第一件事不是构造 shape difference，而是先统计整条轨迹的 `cell_crossings_x/y/z`，得到 `num_segments`，再按 crossing 逐段推进。3D 情况下，它甚至不会平均切段，而是每一轮都比较哪个方向先撞到下一条 crossing，并用最早发生的那个 crossing 定义当前段终点。也就是说，Villasenor 的第一性对象不是“一对 old/new shape 数组”，而是一条被真实 cell crossing 切开的粒子轨迹。

这条结构和 Villasenor-Buneman 1992 的 paper-level 写法也是一致的。原论文先从最简单的 four-boundary case 出发，把一整步运动写成

$$J_{x1}, J_{x2}, J_{y1}, J_{y2}$$

四个局部 boundary flux；若一步跨过更多网格线，再继续拆成 seven-boundary / ten-boundary 的多段 four-boundary 子移动。这里真正应抓住的是它背后的组织原则：

- 第一性对象是局部 boundary flux，而不是全轨道 old/new shape difference；
- 遇到 crossing 就继续切段，而不是先做整轨道全局差分；
- 三维时再通过局部交叉项和分段权重，把严格守恒推广到 face-local stencil。

这也解释了为什么 Villasenor-Buneman 1992 会专门强调“不把一般位移拆成两次正交 move”。如果先把一条真实二维或三维轨迹硬拆成若干彼此独立的正交子移动，那么虽然也可能写出某种守恒公式，但它描述的已经不再是同一条几何扫描。Villasenor 真正坚持的是：每一段局部通量都必须来自粒子云这一步实际先撞到哪条 boundary 的几何事实；现代 WarpX 把这条历史动机改写成 `earliest-crossing` 判据，本质上正是在程序里拒绝“先假设可以正交拆分、再回头拼装”的思路。

所以 WarpX 现代源码虽然不再显式保留 `four/seven/ten-boundary` 这套手工分类，但 `cell_crossings -> num_segments -> local this_J*` writeback 这条段式结构，正是那篇 1992 论文的现代化程序版本。

可以把这层 paper-to-code 关系压缩成下面的流程。论文用 `four-/seven-/ten-boundary` 给若干典型几何子移动命名；WarpX 则先统计 crossing 数，再在同一个循环中重复执行“选最早 crossing、截断 segment、局部写回”的过程。因此图中的 `four-boundary` 不是某个固定的 WarpX 枚举值，`seven/ten-boundary` 也不是源码中的两个分支名，而是 repeated segmentation 可能产生的论文级几何结果。

这条分段循环可按以下步骤阅读：

1. 对一个时间步的粒子轨迹统计各方向的 cell crossing，并确定 `num_segments`。
2. 若仍有未处理的 crossing，比较候选 crossing，选择最早发生的一个。
3. 用该 crossing 截断当前 segment，并为这个局部段构造 cell/node 权重。
4. 将该段的 `this_J*` 局部通量写回；若还有剩余轨迹，则以新的局部原点重复第 2 步。
5. 所有 segment 完成后，局部通量之和就是整条轨迹的沉积结果。

论文级几何名称	它表达的内容	WarpX 中的运行时表示
four-boundary	一个 elementary local submove 的 boundary-flux 分配	一个 crossing-defined segment 的 <code>this_J*</code> 局部写回

论文级几何名称	它表达的内容	WarpX 中的运行时表示
seven-boundary	在基本子移动上增加 crossing 后的多边界几何	<code>num_segments > 1</code> 后重复执行 earliest-crossing loop; 不是固定的 <code>seven</code> 分支
ten-boundary	更复杂的多 crossing 几何组合	同一 loop 继续累加更多 segment; 具体 segment 数由端点 cell index 差决定

这张图也解释了源码注释中“valid for an arbitrary number of cell crossings”的含义：实现的可扩展性来自循环和 crossing 计数，而不是预先枚举有限个几何 case。论文中的 seven/ten-boundary 仍然适合用来帮助读者建立几何直觉，但阅读 WarpX 时应优先追踪 `cell_crossings_*`、`num_segments`、`ns` 和局部 `this_J*`，不要寻找同名的 case label。

更具体一点，论文里从 four-boundary 进入 seven-boundary 或 ten-boundary，本质上是在回答同一个问题：当前这一步里，粒子云先撞到哪一条新的网格边界？现代 WarpX 不再把答案写成手工 case table，而是直接在 kernel 里做“最早 crossing”判据。XZ/RZ 分支会比较当前候选 x-crossing 和 z-crossing 哪个先发生：

```
if (dzp == 0. || std::abs(dxp_seg) < std::abs(dxp/dzp*dzp_seg)) {
    Xcell = x0_new;
    dzp_seg = dzp/dxp*dxp_seg;
    z0_new = z0_old + dzp_seg;
} else {
    Zcell = z0_new;
    dxp_seg = dxp/dzp*dzp_seg;
    x0_new = x0_old + dxp_seg;
}
```

而 3D 分支则把同一个逻辑扩成三方向竞争：比较 x/y/z 三个候选 crossing 中谁最早发生，就用那个方向来定义当前 segment 的终点。这样一来，论文里“先切成 seven-boundary 还是 ten-boundary 的哪一段”这件事，在今天的 WarpX 代码里已经被改写成了一个更通用的程序判据：

1. 为每个方向构造下一条候选 crossing;
2. 比较谁最早发生;
3. 用最早 crossing 截断当前 segment;
4. 把余下轨迹继续送回同一套局部沉积循环。

所以 seven-/ten-boundary 在现代实现里不再是离散命名的几何 case，而是 repeated earliest-crossing segmentation 的自然结果。

如果再把二维 four-boundary 公式和今天的 XZ/RZ kernel 对一下，关系还可以写得更硬一点。论文里的

$$J_{x1} + J_{x2} = \Delta x, \quad J_{y1} + J_{y2} = \Delta y$$

说明二维 Villasenor 的纵向总输运，本质上仍由该方向位移本身控制；几何信息真正改变的是“这份输运怎样在两条局部 boundary 之间分配”。WarpX 当前 XZ/RZ segment kernel 恰好保留了这层结构：Jx 与 Jz 都写成“该段主方向输运量”乘上横向 old/new node weights 的简单平均，

```
this_Jx = wqx*sx_cell[i]*(sz_old_node[k] + sz_new_node[k])/2 * seg_factor_x;
this_Jz = wqz*sz_cell[k]*(sx_old_node[i] + sx_new_node[i])/2 * seg_factor_z;
```

也就是说，二维情形里最核心的 four-boundary 几何仍然能读成：

1. 主方向总输运由 `dx_seg` 或 `dz_seg` 决定；
2. 横向 old/new 平均决定它落在哪两条局部 boundary 上、各占多少；
3. crossing 变多时，就继续把整步拆成多个 obey 同一规则的局部 segment。

这正是论文里“two-boundary split of one directional flux”到现代 kernel 的直接对应关系。

如果把对应再压到 Eq. (6)–(9) 本身，可以写得更细。论文里的

$$J_{x1} = \Delta x \left(\frac{1}{2} - y - \frac{1}{2} \Delta y \right), \quad J_{x2} = \Delta x \left(\frac{1}{2} + y + \frac{1}{2} \Delta y \right)$$

可以直接读成：“同一段 x 向总运输 Δx ，按横向旧位置 y 和横向位移 Δy 改写成上下两条 boundary 上的两份局部 flux”。WarpX 今天的 XZ/RZ kernel 不再显式把这两份 flux 分别命名成 J_{x1}, J_{x2} ，而是把同一件事改写成

$$\text{cell-based support in } x \times \frac{S_{\text{old}}^{(z)} + S_{\text{new}}^{(z)}}{2} \times \frac{dt_{\text{seg}}}{dt}.$$

其中：

1. $sx_cell[i]$ 承担论文里“当前这段 flux 落在哪个主方向 cell support 上”；
2. $\frac{1}{2}(sz_{\text{old}} + sz_{\text{new}})$ 承担论文里由 $y, \Delta y$ 决定的“两条 boundary 怎样分流”；
3. $seg_factor_x = dt_{\text{seg}}/dt = dx_{\text{seg}}/dx$ 则把这一段局部运输从整步 Δx 缩回当前 crossing-defined 子移动。

而与之完全对称的另外两式

$$J_{y1} = \Delta y \left(\frac{1}{2} - x - \frac{1}{2} \Delta x \right), \quad J_{y2} = \Delta y \left(\frac{1}{2} + x + \frac{1}{2} \Delta x \right)$$

则说明论文并不是把 x 向和 y 向分开用两套不同逻辑处理；它坚持的是同一条局部几何规则：某一方向的总运输由该方向位移给出，而它分到哪两条相邻 boundary，则由正交方向上的旧位置与位移共同决定。换到今天的 WarpX 语言里，这正对应 XZ/RZ kernel 中 J_x/J_z 彼此镜像的两条写法：主方向分量落在 cell-based support 上，正交方向只通过 old/new 节点权重平均来决定两边界分流，而不会再额外引入第二套独立守恒机制。

若再贴近 Eq. (6)–(9) 本身，四个通量的角色可以直接读成：

- J_{x1} : x 向运输落到“下侧”那条局部 boundary 的份额；
- J_{x2} : 同一份 x 向运输落到“上侧”那条局部 boundary 的份额；
- J_{y1} : y 向运输落到“左侧”那条局部 boundary 的份额；
- J_{y2} : 同一份 y 向运输落到“右侧”那条局部 boundary 的份额。

于是 $\frac{1}{2} \mp y \mp \frac{1}{2} \Delta y$ 这类因子，本质上不是在给 Δx 再乘一个神秘修正，而是在回答：当 charge cloud 沿 x 方向推进时，它有多少面积扫过了上/下两条相邻 boundary。WarpX 当前 XZ/RZ kernel 用 old/new node average 来写这件事，只是把论文里显式的几何宽度改写成了现代 shape-weight 语言；它保留下来的核心物理量，仍然是“哪一条局部边界分到多少横向扫掠面积”。

所以从 paper-level 读到 code-level，真正保持不变的不是表面符号，而是这条组织关系：先有主方向运输，再有横向分流，最后才由 crossing segmentation 决定这一份局部 flux 属于哪一段真实轨迹。

两篇论文还有一条共同的实现边界，值得在这里顺手点明。Villasenor 1992 在 2D 讨论里直接把 timestep 约束和 Courant condition 连在一起；Esirkepov 2001 第 4 节则要求 $|\Delta x|, |\Delta y|, |\Delta z|$ 不超过单个网格步长。它们说法不同，但物理边界一致：这两条严格守恒沉积都默认 one-step orbit 仍是局部对象。WarpX 现代实现对此条前提的处理方式，则是把它拆到 dt/CFL、implicit/suborbit endpoint reconstruction，以及 cell_crossings $\rightarrow$ segment loop 这些程序结构里，而不再在正文里单独保留一个“几何 case table”。

更进一步，每个 segment 内部也不是只拿段长乘平均速度。源码会同时构造两组权重：

- 以段中心 $x0_bar/y0_bar/z0_bar$ 为输入的 cell-based weights，由 Compute_shape_factor<depos_order-1> 生成；
- 以段两端 $x0_old \rightarrow x0_new, y0_old \rightarrow y0_new, z0_old \rightarrow z0_new$ 为输入的 node-based old/new weights，由 Compute_shape_factor_pair<depos_order> 生成。

然后对当前段直接形成局部 $this_Jx/this_Jy/this_Jz$ ：

```
this_Jx = wqx*sx_cell[i]*( ... )*seg_factor_x;
this_Jy = wqy*sy_cell[j]*( ... )*seg_factor_y;
this_Jz = wqz*sz_cell[k]*( ... )*seg_factor_z;
```

这里的 seg_factor_* 本质上就是当前段的 dt_seg/dt 。因此 Villasenor 的守恒组织方式不是 Esirkepov 那种沿方向累加 $sdxi/sdyj/sdzk$ 的 whole-orbit prefix sum，而是：

1. 用 crossing 把整条轨迹切成多个局部 segment；
2. 对每个 segment 单独构造 cell/node 权重；
3. 将该段的局部输运量直接写回 $this_Jx/this_Jy/this_Jz$ ；
4. 最后由所有 segment 的局部输运量和闭合整条轨迹的离散守恒。

这里还可以再压实一层。3D kernel 里这三个 seg_factor

```
seg_factor_x = dxp_seg/dxp;
seg_factor_y = dyp_seg/dyp;
seg_factor_z = dzp_seg/dzp;
```

并不是抽象的“分段修正系数”，而是当前局部 segment 在三个方向上分别占整步总位移的比例。也就是说，WarpX 在每个方向上做的不是“把整条轨迹的通量平均分给若干 segment”，而是：

1. 先由 earliest-crossing 规则确定当前 segment 的真实终点；
2. 再把这一段在 x/y/z 三个方向上实际走过的位移，分别写成整步位移的分数；
3. 最后用这些方向分数去缩放当前段的 `this_Jx/this_Jy/this_Jz`。

这条结构和论文里 seven-/ten-boundary 的子移动通量是同一个物理意思：局部段只承担自己那一小段真实扫描所对应的那部分 face flux，而不会提前替后续 segment“多沉一截”。因此 `seg_factor_x/y/z` 不是附带修正项，而是现代 WarpX 用程序方式保存 Villasenor 局部守恒子移动语义的关键部件。

这里还有一个容易被误读的细节。`one_third / one_sixth` 并不只出现在 Esirkepov 路径里：Villasenor 的每个 segment 也在横向两个方向上使用同样的 `1/3, 1/6` 组合，对 `old/new node weights` 做局部平均。区别不在于“是否使用这组系数”，而在于它们服务的对象完全不同：

- 在 Esirkepov 里，这组系数属于整条轨迹 `old/new shape difference` 的唯一分解；
- 在 Villasenor 里，这组系数属于单个 segment 内横向 `old/new node weights` 的局部 face-flux 平均。

对 Villasenor 1992 的 3D 推导来说，这条差别还有一层更具体的意义。论文里专门冒出来的 `\Delta x \Delta y \Delta z / 12`，强调的是三维局部通量不再是三个方向彼此独立的简单并列，而会出现真正的 mixed-direction coupling。WarpX 当前 3D kernel 虽然不再把这类项显式写成单个 `\Delta x \Delta y \Delta z / 12` 单项式，但它并没有把这层耦合抹掉；相反，这层耦合正是通过每个方向电流里那组

```
old*old * one_third
old*new * one_sixth
new*old * one_sixth
new*new * one_third
```

的双横向 `old/new` 混合平均被程序化保存下来的。现代源码中的 `one_third/one_sixth` 在 Villasenor 3D 路径里不只是“平滑一下横向权重”，而是在离散实现层面承担了论文 3D 交叉耦合项的角色：它保证每个方向的局部 face flux 在做双横向平均时，仍然保留 `old/new` 端点之间的混合信息，而不是把三维守恒退化三个互不相干的一维沉积。

如果把论文 Eq. (36) 本身也放进来，这层对应还能再硬一点。那一式给出的 `x` 向四个 face contribution 不是四份彼此独立的 `\Delta x`，而是

1. 一份 `+\Delta x, \bar{\eta}, \bar{z}`；
2. 两份带负号的 `-\Delta x \Delta y \Delta z / 12` 修正；
3. 以及一份带正号的 `+\Delta x \Delta y \Delta z / 12` 修正。

它真正表达的是：`x` 向 face flux 要同时感受到 `y/z` 两个横向方向的局部体积重叠，因此四个相邻 `x`-face 上的份额既有“横向平均面积”主项，也有“旧端点与新端点不能简单分离”的 mixed-direction coupling 修正。WarpX 当前 3D kernel 虽然把这层结构改写成 `old*old / old*new / new*old / new*new` 四项 `old/new` 混合平均，但这四项的符号与权重组织，承担的正是同一种职责：让每个 `this_Jx/this_Jy/this_Jz` 在分到四个局部横向角点时，不会丢掉论文 Eq. (36) 里那条 `+, -, -, +` 型耦合信息。

因此两条算法虽然都继承了同一类 tensor-product 守恒平均结构，但一个把它组织成 whole-orbit decomposition，另一个把它组织成 segment-local flux closure。这解释了为什么两段 kernel 看起来都会出现 `one_third/one_sixth`，但循环骨架、support 范围和几何语义仍然截然不同。

这里同样值得把证据边界讲清。和 Esirkepov 那条线不同，Villasenor-Buneman 1992 不只是 preprint-backed，而是已有 full-text PDF 与 MinerU 资产归档在论文目录，因此第 5 章对 Villasenor 的 paper-backed 论证不再需要退回“只有源码、没有论文”的口径。正文可以稳定依赖的层次包括：

1. “不把一般位移拆成正交 move”这条历史动机；
2. four-/seven-/ten-boundary move 的局部 boundary-flux 组织；
3. cell_crossings -> num_segments -> local this_J* writeback 与论文几何 case 的现代对应；
4. XZ/RZ 下 directional transport * (old+new)/2 * dt_seg/dt 这条 four-boundary 到 segment kernel 的直接映射；
5. 3D 路径里 `one_third/one_sixth` 与 `\Delta x \Delta y \Delta z / 12` 类交叉耦合的程序化对应。

这条线当前已经完成四边界、重复分段和三维交叉项的第一轮公式级审计；仍未完成的是论文图示逐图回填、记号统一，以及所有现代 geometry/order 分支的逐项等价性审查。因此本章现在对 Villasenor 最稳妥的证据等级是 **paper-backed + source-grounded + formula-audited**，而不是把尚未完成的出版级图示精修误写成公式缺口。

5.11.2 Villasenor-Buneman 公式级审计：四边界、重复分段与三维交叉项

Villasenor-Buneman 1992 的全文 PDF 与 MinerU Markdown 已完成第一轮公式级核对。论文以局部原点为最近的 cell-boundary 交点，对单位方形粒子的四边界运动写出：

$$\begin{aligned} J_{x1} &= \Delta x \left(\frac{1}{2} - y - \frac{1}{2} \Delta y \right), & J_{x2} &= \Delta x \left(\frac{1}{2} + y + \frac{1}{2} \Delta y \right), \\ J_{y1} &= \Delta y \left(\frac{1}{2} - x - \frac{1}{2} \Delta x \right), & J_{y2} &= \Delta y \left(\frac{1}{2} + x + \frac{1}{2} \Delta x \right). \end{aligned}$$

这四式的核心不是某个固定阶数的 shape kernel，而是“主方向位移 × 横向扫描宽度的旧/新平均”。因此在本书使用的源码快照的 XZ/RZ kernel 中，对应关系应读成：主方向的 displacement 或 cell weight，乘以横向 old/new node weight 的平均，再乘 `seg_factor = dt_seg/dt`。现代源码还要额外承载 arbitrary shape order、几何分支、boundary crop 和 segment-local writeback，所以不能把源码中的表达式当成论文四式的逐字复制。

论文对 seven-boundary 和 ten-boundary 也没有另起两套独立电流公式。seven-boundary 先按第一次 complementary-mesh crossing 把轨迹分成两段，例如：

$$\Delta x_1 = \frac{1}{2} - x, \quad \Delta y_1 = \frac{\Delta y}{\Delta x} \Delta x_1, \quad qquad \Delta x_2 = \Delta x - \Delta x_1, \quad qquad \Delta y_2 = \Delta y - \Delta y_1.$$

随后对两段分别套用四边界公式；ten-boundary 则重复同一过程三次。这个映射正好解释了 WarpX 当前的 `cell_crossings_* -> num_segments -> earliest-crossing -> local this_J*` 循环：论文中的几何名称是结果分类，源码中的可扩展性来自“截断一段、写回一段、继续推进”的循环，而不是七/十边界的固定分支标签。

下面的示意图把这层对应关系压成读者侧流程。它不是 Villasenor-Buneman 论文原图，也不表示源码里存在 `seven_boundary` 或 `ten_boundary` 两个分支；它只把论文的结果分类和 WarpX 当前循环骨架放在同一张图中：

对应关系可直接读成：论文的 four-boundary move 对应一个 segment-local `this_J*` 写回；第一次 complementary-mesh crossing 后若还有残余位移，WarpX 更新局部原点并重复同一段式写回。两段的几何结果可被论文称为 seven-boundary case，三段可称为 ten-boundary case；这些名称是结果分类，不是源码中的固定分支。

读图时应把实线理解成现代实现的执行顺序，把虚线理解成论文中的结果分类。也就是说，seven-/ten-boundary 不是额外的物理守恒律，而是同一局部四边界构造在一条轨迹上重复执行后出现的几何计数；这也是为什么源码只需要一个可重复的 crossing loop，就能覆盖论文中多个 case。

三维部分还给出了一个不能省略的交叉项。对某个 x-face，论文写成：

$$\Phi_x = \Delta x \bar{\eta} \bar{\zeta} + \frac{\Delta x \Delta y \Delta z}{12},$$

其余三个 x-face 按横向因子和交叉项符号变化，y/z 分量由循环置换得到。论文明确指出 $\Delta x \Delta y \Delta z / 12$ 是三维新增项。WarpX 当前 3D Villasenor kernel 不把它保留为一个独立的单项式，而是通过 `one_third/one_sixth` 组成的四个 old/new 横向权重乘积表达同一类 mixed-direction coupling。因而“源码没有显式的 /12”不能被解释成“三维交叉耦合不存在”。

因此，Villasenor 线可概括为 **论文支撑、源码定位和公式核查**；尚未完成的是论文图示逐图回填、记号统一和所有现代 geometry/order 分支的逐项等价性审查。阅读论文副本时也必须区分可核查的正文与出版版本身份，不能由文件可读性推断 publisher provenance。

这里还应补一条维度差异，否则读者容易误以为 Villasenor 在所有几何里都完全按同一平均式沉积。实际并不是这样。WarpX 的 3D kernel 中， $J_x/J_y/J_z$ 三个分量都要面对真正的双横向耦合，因此都会写成 `cell-based weight * (old/new node weights 的 1/3, 1/6 组合) * seg_factor`。但在 XZ/RZ kernel 里，in-plane 的 J_x/J_z 只需要处理单个横向方向，所以退化成 $(old+new)/2$ 的简单平均；只有 out-of-plane 的 J_y 仍保留 1/3, 1/6 组合。换句话说：

- 2D/XZ/RZ：主平面运输更接近论文 four-boundary 的“两条局部边界分流”，
- 3D：每个方向都必须面对双横向耦合，因此每个分量都需要完整的 tensor-product 局部平均。

这条维度差异，正是 Villasenor 1992 二维公式和 WarpX 三维通用 kernel 之间最重要的实现延伸。

这也是为什么源码注释会强调 Villasenor “results in a tighter stencil”。它更紧，不是因为放弃了高阶修正。恰恰相反，`depos_order >= 3` 时，源码仍会对 cell-based weights 做

$$\frac{4S_{\text{bar}} + S_{\text{old}} + S_{\text{new}}}{6}$$

式的 higher-order 修正；只是这些修正现在被局部化到了每个 segment 内，而不是像 Esirkepov 那样围绕整条 old/new shape difference 一次性组织。更稳定的结论可以直接写成：

- **Esirkepov**: 把整条轨迹压成 old/new shape arrays, 并在统一索引框架内做方向前缀累加；
- **Villasenor**: 把整条轨迹按真实 cell crossing 切段，再让每个 segment 的局部输运逐段闭合守恒。

两条路径满足的是同一条离散连续性方程，但“守恒是怎样被压进代码里的”完全不同。

为了把这条差异固定成可复查的出版级摘要，可以把论文对象、源码对象和验证边界并排写成下表：

对照层	Esirkepov 2001	Villasenor-Buneman 1992	读者应保留的边界
第一性对象	整条轨迹两端的 tensor-product shape difference	crossing-defined segment 的局部 face flux	两者都服务离散连续性，但不是同一个局部变量
论文结构	$W^1 + W^2 + W^3$ density decomposition; 二阶 form-factor 使用 mixed averages	four-boundary flux; 多 crossing 递归成 seven-/ten-boundary 子移动; 3D 有 mixed-direction cross term	论文 case 名称不能直接当作现代源码分支名
WarpX 入口	doEsirkepovDepositionShapeCellKernelsDepositCurrent()	doVillasenorDepositionShapeCellKernelsDepositCurrent() 及 explicit/implicit 前端	分派，不能由 DepositCharge() 推断
源码循环	sx_old-sx_new、sy_old-sy_new、sz_old-sz_new 加 sdx/sdy/sdzk prefix accumulation	cell_crossings_* -> num_segments -> earliest-crossing -> this_J* segment loop	一个是 whole-orbit directional decomposition, 一个是 segment-local closure
1/3, 1/6 的职责	论文唯一性分解中的横向 old/new mixed average	单个 segment 的局部横向 face-flux average	数值常数相同不代表算法组织相同
几何支持	old/new shape 先在同一索引框架中对齐; 需要 shifted shape	轨迹按真实 crossing 裁剪和分段; 支持域随 segment 局部化	near-boundary 语义不能把 Direct 当作任一路径的替身
当前验证	代数恒等式, 加 Langmuir/current-correction 的端到端 Gauss-law gate	公式审计, 加 RZ/3D kernel 源码映射	公式级检查不等于 kernel bitwise equivalence 或所有 geometry/order 的端到端证明

表中最后一列是本章的出版边界：它允许读者在一页内看清“论文中的守恒对象如何进入源码”，同时阻止两个常见的过度推断。第一，one_third/one_sixth 只是共享的 tensor-product 平均结构，不意味着 Villasenor 就是 Esirkepov 的另一种循环写法；第二，公式恒等式和源码行级映射只能证明局部结构，不能自动升级成所有维度、边界、shape order 和 implicit/suborbit 分支都已逐项等价。

这条局部公式边界可用独立的代数检查验证：二维 crossing 轨迹和三维中点/位移样本应先满足

$$J_{x1} + J_{x2} = \Delta x, \quad J_{y1} + J_{y2} = \Delta y,$$

再把任意轨迹按所有 cell crossing 切成 segment, 验证各段位移之和仍恢复整条轨迹；同时验证 Eq.(36) 四个 face contribution 的三维交叉项和体积分数差分闭合。它把 Eq.(6)-(9)、Eq.(36) 与 repeated segmentation 落为可重复的代数/几何证据，但仍不替代 WarpX kernel 的 bitwise、边界或全量 Gauss-law regression。

Villasenor-Buneman formula contract

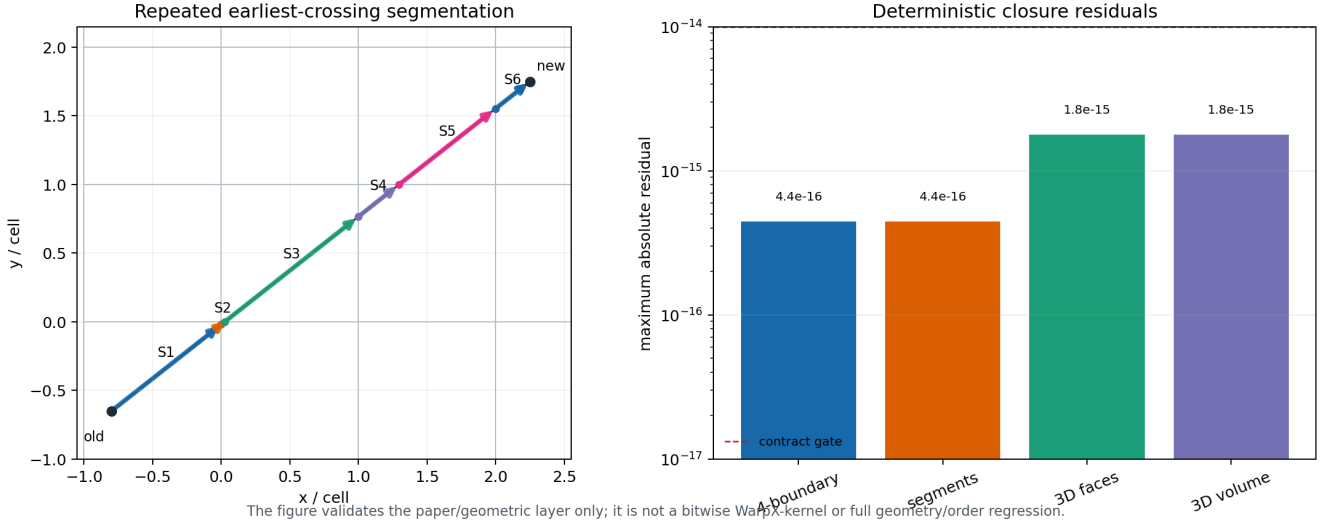


图 5-1: Villasenor 公式合同的两层证据。左侧把一条跨越多个 cell 的轨迹按 earliest crossing 切成局部 segment; 右侧汇总四边界、segment、3D face 和 3D volume closure 的最大残差。该图只展示论文/几何层闭合, 不把它升级为 WarpX kernel 等价或全 geometry/order 回归。

在公式核查之外, 可在源码中依次定位 VillasenorDepositionShapeNKernel、explicit/implicit entrypoint、三方向 cell_crossings_* 计数、num_segments 循环、final-segment/continuation 分支、seg_factor_* 和 this_Jx/this_Jy/this_Jz 写回。它说明正文中的 crossing-driven segment skeleton 与源码一致, 但仍不替代数值 kernel regression。

官方 test_2d_theta_implicit_jfnk_vandb 将 implicit Villasenor 的证据从源码和公式层推进到 2-rank 运行级: 它使用 shape=2、周期边界和 theta-implicit Newton/JFNK; analysis_vandb_jfnk_2d.py 比较总能量和 Gauss-law。这个案例支持二维周期主路径, 不能外推到所有 geometry/order。

同一条独立 contract 又读取官方 test_2d_theta_implicit_jfnk_vandb_cropping: 该 sibling 把 shape 提升到 4、网格缩小到 16x16, 并打开 near-boundary cropping; 官方 analysis 与独立读取均通过, 末态 Gauss-law 最大绝对误差为 $8.2275e-14 < 1e-13$, RMS 为 $3.0023e-14$ 。这两条结果可以支持“2D implicit Villasenor 的普通 shape=2 路径和 shape=4 boundary-cropping 路径均有运行级守恒证据”, 但不能外推到所有 geometry/order。

同一 family 的 test_2d_theta_implicit_jfnk_vandb_filtered 只把 warpx.use_filter 打开为 1, 保留 shape=2、周期边界和 JFNK 配置不变。官方 analysis 与要求显式确认 filter 输入的独立 contract 均通过: 最大总能量相对变化 $3.8931e-15 < 2e-14$, Gauss-law RMS $5.1401e-16 < 2e-15$, 且末态字段全部 finite。于是当前 2D 证据不只覆盖“Villasenor 能守恒”, 还覆盖了 implicit current 同步之后再经过 field filter 的组合路径。

官方 test_2d_theta_implicit_jfnk_vandb_picmi 又提供同一物理合同的 Python 前端路径。生成输入应显式包含 algo.current_deposition = "villasenor"、algo.evolve_scheme = "theta_implicit_em" 与 algo.particle_shape = 2; 读者需要检查实际生效输入, 而不是只信任前端对象。该路径还出现过 newton.liner_solver unused-input 提示, 说明“能运行”不等于每个前端参数都被消费。

维度边界也必须如实记录: 官方 RZ theta-implicit Villasenor 输入要求 newton.linear_solver=petsc_ksp, 当前 build_full binary 未启用 AMREX_USE_PETSC, 因此在 NewtonSolver::Define() 初始化阶段直接拒绝, 未进入物理计算。随后只用命令行把同一输入的线性求解器覆盖为 amrex_gmres 做 control; 它仍未进入物理时间推进, 而是在 WarpX::InitData() -> ThetaImplicitEM::Define() -> InitializeCurlCurlBCMasks() 触发 SIGILL。因此当前 RZ blocker 不只是 PETSc 缺失, 还包含 arm64 build_full 的 RZ theta-implicit boundary-mask 初始化失败。官方 1D planar-pinch sibling 则在 Newton 后的粒子边界处理路径出现 SIGILL, 只落入初始诊断帧。三者均记录为构建/运行边界, 而不是伪造为 Villasenor physics failure 或 pass。

运行级证据按可支持的结论可压缩为六项:

1. test_2d_theta_implicit_jfnk_vandb: 2D、shape=2、周期。总能量变化 $4.0980e-15$, Gauss RMS $9.2951e-16$, 官方与独立读取均 PASS, 覆盖 2-rank 主路径。
2. test_2d_theta_implicit_jfnk_vandb_cropping: 2D、shape=4、near-boundary cropping。最大 charge error $8.2275e-14$, PASS; 它没有单独的强 energy ledger。
3. test_2d_theta_implicit_jfnk_vandb_filtered: 2D、shape=2、warpx.use_filter=1。总能量变化 $3.8931e-15$, Gauss RMS $5.1401e-16$, PASS, 并显式确认 filter 输入。

4. `test_2d_theta_implicit_jfnk_vandb_picmi`: 2D PICMI, shape=2, Python GMRESLinearSolver。总能量变化 $4.0980\text{e-}15$, Gauss RMS $9.5730\text{e-}16$, PASS; Python-enabled build 仍保留 unused-input warning。
5. `test_rz_theta_implicit_dynamic_pinch`: RZ, shape=2, axis/insulator。PETSc 官方路径在 `NewtonSolver::Define()` 拒绝; `amrex_gmres` control 在 `InitializeCurlCurlBCMasks()` 触发 SIGILL, 未进入物理计算。
6. `test_1d_theta_implicit_planar_pinch`: 1D、shape=2、planar pinch。Newton 后触发 SIGILL, 仅有初始帧, 因此不作为通过证据。

5.11.3 Esirkepov 运行级维度证据: 1D、2D 与 3D Langmuir

前面的公式恒等式和源码合同只能证明局部结构; 为了避免把它们误写成端到端证据, 运行产物直接采用 WarpX 官方 Langmuir 输入。官方 1D 和 3D 输入本身显式设置 `algo.current_deposition = esirkepov`; 官方 2D 测试卡默认是 `direct`, case-local 副本只将这一项覆盖为 `esirkepov`, 并补上官方 `analysis` 所需的 `rho/divE` 输出字段, 因此它是可复查的验证 sibling, 而不是 WarpX 上游已注册的 2D Esirkepov regression。

运行验证应使用官方 `analysis` 的理论 Langmuir 场误差与内置 charge-conservation gate, 并检查最终 plotfile 的 `Ex/Ey/Ez/Bx/By/Bz/jx/jy/jz/rh` 是否有限; 其中一个清晰的 charge observable 是

$$\epsilon_{\text{charge}} = \frac{\|\text{divE} - \rho/\epsilon_0\|_{\infty}}{\|\rho/\epsilon_0\|_{\infty}}.$$

结果如下:

结果按维度与 shape 可直接阅读:

- **1D, 128x1x1, shape 1**: 场误差 $1.7028\text{e-}3 < 0.05$, charge residual $8.3450\text{e-}12 < 1\text{e-}11$, PASS。
- **2D, 128x128x1, shape 1-4**: shape 1/2/3 的场误差为 $1.2201\text{e-}2/3.4096\text{e-}2/4.6336\text{e-}2$, 均低于 0.0503; shape 4 的误差为 $6.0165\text{e-}2 < 0.07$ 。四种 shape 的 charge residual 为 $3.5650\text{e-}12/3.1326\text{e-}12/4.5607\text{e-}12/2.8977\text{e-}12$, 均低于 $1\text{e-}11$, 因此均 PASS。
- **3D, 64x64x64, shape 1**: 场误差 $3.4040\text{e-}2 < 0.05$, charge residual $1.3029\text{e-}12 < 1\text{e-}11$, PASS。
- **2D MR, max_level=1, ratio 4, CKC/filter**: 理论场误差 $3.8068\text{e-}2 < 0.0503$, 但逐层 charge residual 为 L0 0.8828、L1 1.2005, 所以是 BOUNDARY, 不是 AMR 守恒通过。

这些证据覆盖 **1D/2D/3D + 2D shape=1/2/3/4 + 3D shape=1/2/3/4**。2D shape=4 的 0.07 场误差阈值来自官方 `analysis_2d.py` 对测试名中 `particle_shape_4` 的分支, 而不是临时放宽; 2D shape=1/2/3 使用 0.0503, 3D shape=1/2/3/4 使用官方 0.05 field gate, 所有 shape 都使用独立 $1\text{e-}11$ charge residual gate。

3D shape=2 的 field error 为 $3.5970\text{e-}2$ 并通过。shape=3/4 在 64^3 的 field error 为 $6.7792\text{e-}2/8.7344\text{e-}2$, 但同一输入的 128^3 refined sibling 降至 $2.3515\text{e-}2/3.0644\text{e-}2$ 并通过 field gate, charge residual 分别为 $4.3288\text{e-}12/3.0001\text{e-}12$ 。因此, shape=3/4 的低分辨率 field boundary 具有分辨率敏感性, 尚不足以包装成正式 convergence order。Source/WarpX.cpp 的初始化检查拒绝 shape=0, 源码只允许 `particle_shape=1..4`, 所以这是 unsupported boundary, 而不是失败的 physics case。

MR overlay 的理论场 gate 通过, 但逐层 reader contract 在 L0/L1 分别得到 0.8828/1.2005, 因此只能标记为 BOUNDARY, 不能升级为 AMR 守恒通过。15-anchor AMR source contract 证明路由/同步源码骨架存在, 7-anchor Python observability audit 证明 generic register API 存在; 两者都不能替代中间场与 route-count 的专门验证。现有 1-4 阶运行证据也不能推出 AMR buffer、边界裁剪、RZ/RCYLINDER/RSPHERE 或 implicit 分支都已逐项等价, 更不能替代尚未取得的 CPC publisher-PDF bounded compare。

2D case 的 `direct` -> `esirkepov` 覆盖和 `rho/divE` 诊断字段来自仅修改沉积选项的验证 sibling, 不能写成上游官方注册回归; 3D shape=2/3/4 及 refined sibling 也不改变上游测试注册。

shape=2 的 128^3 refined sibling 的 field error 为 $1.2523\text{e-}2$, charge residual 为 $5.4174\text{e-}12$, 同样通过双 gate。三种 shape 的 refined controls 均通过, 但这仍是 case-local 分辨率证据, 不足以包装成正式 convergence order。

5.12 沉积不只回 rho/J: WarpX 还把线性响应矩阵和统计矩交回网格

如果只盯 `DepositCharge()` 和 `DepositCurrent()`, 会把本章的边界讲窄。WarpX 里至少还有两条同样属于 particle-to-grid deposition 的源项支线:

1. **mass matrices deposition**
 - 服务 implicit/JFNK 的线性化电流响应;
 - 不是 Maxwell 方程一时间步要直接消费的 `current_fp`。
2. **temperature / variance deposition**

- 服务 per-species 温度和速度方差统计;
- 不是 ρ/J 的附属输出, 而是一条独立的 weighted-moment deposition 主线。

这三条线共享同一套 particle-to-grid 工程骨架:

- shape order
- tilebox / buffer
- guard-cell 安全检查
- boundary sum / fill-boundary

但物理语义不同。

5.12.1 mass matrices: 沉的是 $J(E)$ 的线性响应, 不是当前步的 Maxwell 源项

MultiParticleContainer::DepositMassMatrices() 会先把

- MassMatrices_X
- MassMatrices_Y
- MassMatrices_Z

三组方向块清零,再逐 species 调 `pc->DepositMassMatrices(fields, lev, dt)`。源码见 `Source/Particles/MultiParticleContainer.`

species 级的 PhysicalParticleContainer::DepositMassMatrices() 则取:

- Bfield_aux
- 九个矩阵块 Sxx..Szz

再调用 WarpXParticleContainer::DepositMassMatrices(...),源码见 `Source/Particles/PhysicalParticleContainer.cpp`。

因此这条线的第一性对象不是

$$\rho, \quad \mathbf{J},$$

而是 implicit 线性化里近似

$$\mathbf{J}(\mathbf{E}) \approx \mathbf{J}_0 + \mathbf{M}(\mathbf{E} - \mathbf{E}_0)$$

所需的粒子响应块。也正因为它不是普通 current deposition 的平移版, 源码一开始就拒绝:

- Esirkepov
- Vay
- collocated grid

只保留:

- Villasenor
- Direct

两条 mass-matrix deposition 路径, 见 `Source/Particles/WarpXParticleContainer.cpp`。

5.12.2 temperature / variance deposition: 沉的是样本数、加权均值和去均值二次矩

species 打开 `do_temperature_deposition` 后, PhysicalParticleContainer::AllocData() 会按 `current_fp` 的 `box/stagger/guard` 规格额外分配 `T_<species>` 三个方向场, 并创建 VarianceAccumulationBuffer。源码见 `Source/Particles/PhysicalParticleContainer.cpp`。

真正的多物种入口不在主 Evolve() 里, 而是 MultiParticleContainer::DepositTemperatures():

1. 找出 `T_<species>`;
2. 清零;
3. 调 `pc->AccumulateVelocitiesAndComputeTemperature(T_vf, relative_time)`;

见 `Source/Particles/MultiParticleContainer.cpp`。

这条线的工作前提也比普通 J 沉积更窄。DepositTemperature() 直接要求:

- `current_deposition_algo == Direct`
- `push_type == Explicit`
- 关闭 shared-memory current deposition

否则 abort, 见 `Source/Particles/PhysicalParticleContainer.cpp`。

内部统计对象不是“直接沉温度”，而是：

- `n`
- `w`
- `wv`
- `w(v - \bar{v})^2`

更具体地，当前实现硬编码走 `DOUBLE_PASS`：

1. 第一遍沉 sample count、权重和、加权速度和；
2. boundary sum；
3. 第二遍用第一遍得到的 `\bar{v}` 再沉去均值二次矩；
4. 再按

$$\text{var} = \frac{n}{(n-1) \sum w} \sum w(v - \bar{v})^2$$

做 unbiased normalization；

5. 最后乘 `m/k_B` 变成 Kelvin。

源码位置：

- `Source/Particles/PhysicalParticleContainer.cpp`
- `Source/Particles/Deposition/TemperatureDeposition.H`
- `Source/Particles/Deposition/VarianceAccumulationBuffer.cpp`

因此，这条温度沉积主线更准确地说是：

- particle-to-grid weighted-moment deposition
- 再经过 boundary sum / filter / 归一化
- 最后得到 `T_x/T_y/T_z`

而不是“顺手从 `J` 推一个温度”。

5.12.3 这一节回头修正了本章的总边界

到这里，本章里“粒子把东西交回网格”至少已经分成三类：

1. `rho/J`
 - 服务 Maxwell 源项与离散连续性；
2. `MassMatrices_*`
 - 服务 implicit/JFNK 的线性响应近似；
3. `T_<species>` / variance buffers
 - 服务统计矩和温度 diagnostics。

它们的共同点是都共享 `shape`、`tilebox`、`guard-cell` 和 `boundary-sum` 的 deposition 工程骨架；不同点是物理语义完全不同。

5.13 沉积后为什么还要同步

沉积 kernel 只把粒子贡献放到本地数组、tile、本 level 或 buffer 中。它还不能保证场求解器马上可以使用这些源项。主循环在 `Source/Evolve/WarpXEvolve.cpp` 调 `SyncCurrentAndRho()`，源码注释直接把它定义成：

- filter
- exchange guard cells
- interpolate across MR levels
- apply boundary conditions

但如果只停在这句注释，本章会把真正的 source-synchronization 合同讲窄。更准确的分层如下。

5.13.1 PSATD 与 FDTD 的同步时序不同

SyncCurrentAndRho() 位于 Source/Evolve/WarpXEvolve.cpp。

- PSATD + periodic single box
 - 即使启用了 current correction 或 Vay deposition, 也立即同步;
 - 若是 Vay, 则同步的名字就是 current_fp_vay, 不是 current_fp。
- PSATD + 非 periodic single box
 - 只有在
 - * 没开 current_correction
 - * 且不是 Vay deposition 时才立刻 SyncCurrent("current_fp") + SyncRho();
 - 否则把更完整的同步推迟到 PushPSATD。
- FDTD
 - 直接 SyncCurrent("current_fp") + SyncRho()。

因此 source synchronization 的时序本身就是 solver-algorithm contract 的一部分, 不是沉积后永远立刻做的固定尾声。

这里还有一条容易被一句话摘要吃掉的实现边界: PSATD + 非 periodic single box + Vay deposition 并不是“什么都不做, 等后面统一处理”, 而是会先对 current_fp_vay 单独做 filter, 而且源码旁边直接留着注释:

```
// TODO This works only without mesh refinement
```

也就是说, current_fp_vay 在这条分支里当前仍带着“只假定无 MR”的现实边界。对第 5 章来说, 这一点很重要, 因为它说明 source synchronization 不只受 Maxwell solver 与 deposition algorithm 影响, 还受当前实现是否已经覆盖 AMR 组合情形影响。

5.13.2 SyncCurrent() 的骨架: coarsen、buffer、owner mask、filter、SumBoundary

SyncCurrent() 的底层在 Source/Parallelization/WarpXComm.cpp。

它的关键结构不是一句“通信电流”, 而是:

1. 若开 do_current_centering
 - 先把 current_fp_nodal 中心化回 staggered current_fp。
2. finest-to-coarsest 迭代
 - 先把未过滤、未求和的 J_fp coarsen 成 J_cp。
3. 若有 current_buf
 - 先把 J_cp 加到 buffer, 再统一作为跨 level 的通信源。
4. coarse level 接收 finer 源项时
 - 先 ParallelAdd 到临时 fine_lev_cp
 - 再用 OwnerMask(...) 去掉 nodal overlap / periodic overlap 的 double counting
 - 然后才并入本 level J_fp
5. 最后对每个 level 再做:
 - ApplyFilterMF(...)
 - SumBoundaryJ(...)

这里尤其要注意 owner-mask 这一步: 它说明 fine-to-coarse 源项并不能直接并进当前 level J_fp, 否则 nodal overlap 点会被重复加两次。

这里可以把源码里的三个中间对象明确分开:

1. J_cp
 - 表示当前 fine level 先 coarsen 下来的 coarse representation;
2. mf_comm
 - 表示“这一轮真正要跨 level 发送的源项”;
 - 若存在 current_buf, 它其实 alias 到 J_buffer, 也就是“buffer + coarsened current”的合并体;
 - 若不存在 current_buf, 才直接等于 J_cp;
3. fine_lev_cp
 - 表示 coarse level 接收完 fine 源项后的临时落点;
 - 它不是最终要保留的场, 而只是为了在并回 J_fp 之前先做 owner-mask 去重。

这层区分很重要, 因为它说明 WarpX 的 fine/coarse 同步并不是“fine 直接往 coarse 主场里加一遍”, 而是:

```
J_fp(fine)
-> coarsen 成 J_cp
```

- > 若有 `current_buf` 则先进 `buffer`, 得到 `mf_comm`
- > `ParallelAdd` 到 `coarse` 侧临时 `fine_lev_cp`
- > 用 `OwnerMask` 去重
- > 再并回 `coarse` 的 `J_fp`

只有这样, `coarse-fine transition zone` 和 `nodal/periodic overlap` 才不会在同一张 `coarse` 主场上被重复计数。

这里还有一个容易被“通信电流”四个字掩盖掉的结构: 如果打开 `do_current_centering`, `SyncCurrent()` 的第一步并不是通信, 而是先把 `current_fp_nodal` 中心化回 `staggered current_fp`。也就是说, 某些 `current` 甚至在同层 `box` 间求和之前, 都还没有处在 `field solver` 最终消费的 `staggering` 上。对成书叙述来说, 这比简单说“同步后电流可用”更准确, 因为它明确区分了:

1. `nodal/staggered` 源项重排;
2. `same-level / fine-coarse` 一致性;
3. `filter` 与 `guard-cell` 求和;
4. 边界条件。

`SyncCurrent()` 的最后一步也不只是“把所有 `ghost cells` 累加一遍”。`SumBoundaryJ()` 会先从 `get_ng_depos_J()` 出发, 再叠加 `current centering` 与 `bilinear filter` 真正需要的 `stencil` 宽度, 最后只对那部分 `guard region` 做 `WarpXSumGuardCells(...)`。因此这里的 `same-level` 同步范围并不是整个 `nGrow`, 而是“沉积和后续 `filter` 真正需要的最小安全区”。

5.13.3 SyncRho() 平行但不完全等于 SyncCurrent()

`SyncRho()` 位于 `Source/Parallelization/WarpXComm.cpp`, 其高层结构和平行于 `SyncCurrent()`:

1. `rho_fp -> rho_cp coarsen`;
2. 若有 `rho_buf`, 先和 `rho_cp` 合并;
3. `coarse level` 也通过临时 `fine_lev_cp + OwnerMask` 去重后再并回 `rho_fp`;
4. 最后对每个 `level` 调 `ApplyFilterandSumBoundaryRho(...)`。

但它和 `current` 不完全相同:

- 没有 `do_current_centering`
- 过滤和求和由 `ApplyFilterandSumBoundaryRho(...)` 统一处理

后者在 `Source/Parallelization/WarpXComm.cpp` 中:

- 若 `use_filter`
 - 先把 `filtered rho` 写进临时 `rf`
 - 再用 `WarpXSumGuardCells(rho, rf, ...)` 把 `filtered` 值并回
- 否则直接 `WarpXSumGuardCells(rho, ...)`

所以 `SyncCurrent()` 与 `SyncRho()` 虽然共享 `fine/coarse + buffer + owner-mask` 的大骨架, 但具体 `filter/sum` 实现并不一样。

更进一步说, `rho` 这条线比 `current` 少了一层 `staggering` 重排, 但并没有因此退化“更简单的数组相加”。`ApplyFilterandSumBoundaryRho(...)` 先决定是否构造 `filtered` 临时 `rf`, 然后再用 `WarpXSumGuardCells(...)` 把 `filtered` 或 `unfiltered` 结果合并回去。也就是说, `rho` 的 `source synchronization` 也同样把“物理要不要 `filter`”写进了同步合同本身, 而不是在同步完成后额外修饰。

`rho` 路径里也有和 `current` 完全平行的“临时接收层”问题: `fine level` 传下来的 `rho_cp` 或 `rho_buf + rho_cp` 并不是直接加回 `coarse rho_fp`, 而是同样先落到临时 `fine_lev_cp`, 再通过 `OwnerMask(...)` 选出 `coarse patch` 上真正该保留的那一份。也就是说, `fine/coarse` 去重并不是 `current` 独有技巧, 而是 `source synchronization` 对所有守恒源项都共享的一条骨架。

5.13.4 SyncCurrentAndRho() 的最后一步其实是边界条件

顶层同步完成后, `SyncCurrentAndRho()` 还会继续对:

- `fine patch` 的 `rho_fp/current_fp`
- `coarse patch` 的 `rho_cp/current_cp`

调用:

- `ApplyRhofieldBoundary(...)`
- `ApplyJfieldBoundary(...)`

源码见 `Source/Evolve/WarpXEvolve.cpp`。

因此这条函数的完整职责不是:

- “做完 guard-cell 通信就结束”

更准确地说，这里调用的也不是抽象的“边界后处理”。WarpXEvolve.cpp 在注释里直接写的是：

- Reflect charge and current density over PEC boundaries, if needed.

而 ../warpX/Source/BoundaryConditions/WarpX_PEC.H 又把底层语义钉得更死：

- ApplyReflectiveBoundarytoRhofield(...): 把沉积到 PEC 外侧的电荷密度反射回计算域；
- ApplyReflectiveBoundarytoJfield(...): 把沉积到 PEC 外侧的电流密度反射回计算域。

这意味着 ApplyRhofieldBoundary(...) / ApplyJfieldBoundary(...) 在 source synchronization 里承担的不是“最后顺手修一下边界层”，而是把已经完成 same-level 求和、fine/coarse 合并、filter 和 OwnerMask 去重后的源项，再按实际场边界几何物化成可供 Maxwell solver 消费的 patch-local rho/J。如果边界是 PEC，那一步会把落到导体外侧或 guard 区的沉积量，按镜像位置与分量符号规则折回域内；如果不是需要反射的边界，则这里才退化成相对平直的 boundary pass。

还有一个容易被一句话略过的实现点：WarpX 并不是只对 fine patch 做这件事，而是按 patch 类型分别处理：

- level lev 上始终对 fine patch 的 rho_fp/current_fp 调 PatchType::fine
- 当 lev > 0 时，再对 coarse patch 的 rho_cp/current_cp 调 PatchType::coarse

也就是说，source synchronization 的终点不是“得到一份全局一致的守恒源项”，而是“得到 fine/coarse 两套都已经与边界条件相容的源项 MultiFab”。这对后面的场推进很关键，因为 solver 消费的是 patch-local rho/J，不是一个抽象的、尚未带边界语义的守恒和。

而是：

1. solver/algorithm 条件分叉；
2. same-level 与 fine/coarse 源项同步；
3. filter；
4. boundary reflection / mirror / PEC-compatibility。

所以完整源项路径更准确地写成：

```
particle trajectory
-> tile-level charge/current deposition
-> species sum
-> MultiFab/local or buffer accumulation
-> filter / guard-cell exchange / AMR synchronization
-> boundary conditions
-> field solver
```

漏掉：

- owner-mask 去重、
- fine/coarse buffer 合并、
- filter 的时序、
- 或最后的 ApplyRhofieldBoundary/ApplyJfieldBoundary

都会把 WarpX 的 source synchronization 合同讲错。

5.13.5 本章对应的两条关键 regression，分别在验证哪一层 source-synchronization 合同

到这里，source synchronization 的源码主链已经闭合；对应的 regression 也可以更明确地挂回这条链，而不只当成零散 smoke test。

第一条是 Examples/Tests/langmuir/ 里的 PSATD + current_correction 变体。它不是只看 divE-\rho/\epsilon_0，而是两层断言同时成立：

1. Ex/Ey/Ez 或 Ex/Ez 仍要和解析 Langmuir-wave 场解匹配；
2. analysis_utils.py 在 current_correction 路径下还会追加 divE-\rho/\epsilon_0 检查，容差固定放宽到 1e-9。

因此它验证的不是“某个 deposition kernel 单独正确”，而是：

- Esirkepov deposition
- PSATD current correction
- 立即同步后的源项一致性

这三层组合后，解析波解和离散 Gauss law 都没有被破坏。

第二条是 Examples/Tests/vay_deposition/。这组测试更窄，也更直接：它不再对照解析波，只断言

$$\frac{\max |\nabla \cdot E - \rho/\epsilon_0|}{\max |\rho/\epsilon_0|} < 10^{-3}.$$

在当前输入里，这条断言专门落在

- algo.current_deposition = vay
- algo.maxwell_solver = psatd
- warpx.grid_type = collocated

这组实现边界上。结合 SyncCurrentAndRho() 的源码分叉，vay_deposition 真正测到的是：current_fp_vay 在 filter-first、后续再交给 PSATD 同步链的专门路径里，最终是否仍能保住离散 Gauss law。

所以对本章来说，这两组 regression 的职责应分开理解：

- Langmuir + current_correction
 - 更宽，测 physics solution + source consistency;
- vay_deposition
 - 更窄，测 specialized source-synchronization path 本身。

这样第 5 章的验证闭环就不再只是“有些测试目录可以参考”，而是已经能说清：哪一条 regression 在替本章的哪一层 source contract 做断言。

5.14 形函数、guard cells 与稳定性

更高阶 shape 会影响三个方面：

1. gather/deposition stencil 更宽，需要更多 guard cells;
2. 粒子噪声降低，但局部性和通信成本增加;
3. 在边界、AMR coarse-fine interface、PML 和 embedded boundary 附近，shape 的截断或修正会影响守恒。

源码中 current deposition 和 charge deposition 都会检查 shape 是否能放进 guard cells, WarpXParticleContainer::DepositCurrent() 计算 shape_extent 和 range; 同一文件中的 shared-memory charge 路径做类似检查，普通 charge deposition 则经 Source/ablastr/particles/DepositCharge.H 的桥梁路径处理本地 tile 与 guard 区。这些检查不是性能细节，而是物理离散化安全条件。

下面的 geometry/order 证据表把“实现有分派入口”和“指定输入已有运行结果”分开列出；运行证据只覆盖已经实际比较过的组合，不能由源码入口自动推导。

路径	源码覆盖	代表性运行证据	仍未关闭的边界
DepositCharge() ordinary/shared	1D_Z、XZ、RZ、 RCYLINDER、 RSPHERE、3D; shape 1/2/3/4	1D/2D/3D charge/Gauss-law siblings; 2D shape 1/2/3/4; RZ charge/inverse-volume	RCYLINDER/RSPHERE 的逐阶 charge/Gauss-law runtime 矩阵仍不完整
Direct current	1D_Z、XZ、RZ、 RCYLINDER、 RSPHERE、3D; shape 1/2/3/4; implicit 入口	既有 Langmuir、Vay/Direct 相关回归和源码 contract	不能据此推出所有几何、边界裁剪和 implicit 组合等价
Esirkepov	shape 1/2/3/4; 显式与 implicit skeleton	1D/2D/3D Langmuir; 2D shape 1/2/3/4; RCYLINDER/RSPHERE shape 1/2/3/4 径向 Er 与 rho/divE 观测; RZ Er/Ez field PASS; 2D MR 为 BOUNDARY	RZ charge residual 为 BOUNDARY; RCYLINDER/RSPHERE 径向 charge shape=1/2/3/4 均为 BOUNDARY; 完整 AMR route-count 仍未形成强 runtime 闭环
Villasenor	shape 1/2/3/4; 显式与 implicit skeleton	2D implicit native、 filtered、shape=4 cropping、PICMI; 公式级 contract	RZ 因 PETSc/build 边界未 形成运行级证据，其他几何/阶数组 合仍需逐项核对

路径	源码覆盖	代表性运行证据	仍未关闭的边界
Vay	shape 1/2/3/4	既有 <code>vay_deposition</code> regression	几何与边界裁剪的全组合覆盖仍未完 成

因此, 本节可以说明实现提供哪些分派入口, 但不能把它缩写成“所有入口都已验证”。尤其是 RCYLINDER/RSPHERE 只确认了编译期 geometry branch; 它们与 RZ 的坐标压缩、逆体积和模式写回语义不能互相替代。

RZ + Esirkepov 还需要单独保留一个诊断边界: 代表性 2-rank case 的 $\mathbf{E_r}/\mathbf{E_z}$ field contract 通过, 但同面 $\text{div}\mathbf{E}-\rho/\epsilon_0$ residual 仍约为 $3.6\text{e-}3$, 而官方 `analysis_utils.py` 也明确跳过 RZ Esirkepov 的强 charge gate。原因不能简单归结为“kernel 已经错误”: `DivEFunctor` 与 `RhoFunctor` 分别从场求解器和重新沉积的 charge density 构造诊断量, 再经过各自的 node/cell、mode 与插值路径。本书将其标为 BOUNDARY, 不把 field PASS 升级为守恒 PASS。

同一条证据又做了 `warpx.do_dive_cleaning=1/0` 的 paired control: 全局 charge residual 从 $3.593\text{e-}3$ 增至 $9.693\text{e-}2$, 约为 26.98 倍; 第一径向 cell 之外的 residual 则为 $4.293\text{e-}4/6.540\text{e-}12$ 。两个 case 的全局最大值都由 axis cell 主导, $\mathbf{E_r}/\mathbf{E_z}$ field error 也改变为 $2.427\text{e-}2/4.941\text{e-}3$ 。这说明边界同时对 axis treatment 和 cleaning 路径敏感, 但不能据此把 cleaning 认定为唯一根因; 比较结果归档为 AXIS_DOMINATED_CLEANING_SENSITIVE_DIAGNOSTIC_BOUNDARY。

进一步只切换 `boundary.verboncoeur_axis_correction`: 在一个 64×128 、`particle_shape=1` 对照中, 开启时 charge residual 约为 $3.6\text{e-}3$, 关闭时降到约 $5.5\text{e-}12$, 且 off-axis residual 处于同一量级; 两者 $\mathbf{E_r}/\mathbf{E_z}$ field error 都低于该案例的场门限。因此这个对照能恢复双 gate, 但不足以要求修改 WarpX 默认值。

在默认轴修正保持开启的条件下, RZ Esirkepov Langmuir 的 shape=1/2/3/4 都有 field-level 覆盖, $\mathbf{E_r}$ 最大相对误差均低于 0.12; 相应的 charge residual 约为 10^{-3} , 且最大值均由 axis cell 主导。因此这里补足的是 field shape coverage, 不是 RZ charge boundary。

将同样的 shape=2/3/4 case 切换到 `verboncoeur_axis_correction=false` 后, charge residual 降到约 10^{-12} , 但 $\mathbf{E_r}$ field error 会超过 0.12。因此轴修正对照不是一个可以全局套用的“修复开关”, 而是随 shape 改变的 charge/field tradeoff。

对 shape=1 做三档加密后, correction-on 的场误差下降、axis charge residual 也下降; correction-off 在中等分辨率可同时通过 field/charge gate, 但在更高分辨率又可能越过强 charge gate。这组趋势支持“分辨率和轴修正存在耦合”, 却明确否定了“correction-off 是通用修复”的表述。

同样的比较用于 shape=2 时, coarse correction-off case 出现 field boundary, 而加密后可通过 field/charge 双 gate; correction-on 的高分辨率场误差通过, 但 axis charge residual 仍约为 10^{-3} 。因此 shape=2 的 coarse field failure 可归为分辨率边界, 但 correction-on charge boundary 仍未闭合。

把 correction-off 对照扩展到 shape=3/4 后, coarse 场误差仍可能超过门限, 而加密后可通过双 gate。这说明高阶 shape 的粗网格失败是分辨率诊断, 不关闭 correction-on axis charge residual, 也不外推到其他 geometry、AMR 或 implicit 路径。

把高阶 shape 与更高分辨率一并比较后, 结论保持一致: correction-on 的 field gate 可以通过, 但 axis charge residual 仍处于 10^{-3} 量级; correction-off 只在部分 shape/resolution 组合上同时通过 field/charge gate。默认轴修正开启时 field PASS 而 axis charge 仍是 BOUNDARY; 关闭轴修正只提供局部诊断对照, 不能替代默认配置。更高 shape 与更高分辨率可以改善部分 case, 不能被写成全局修复或正式收敛阶。

RZ Esirkepov correction/shape tradeoff

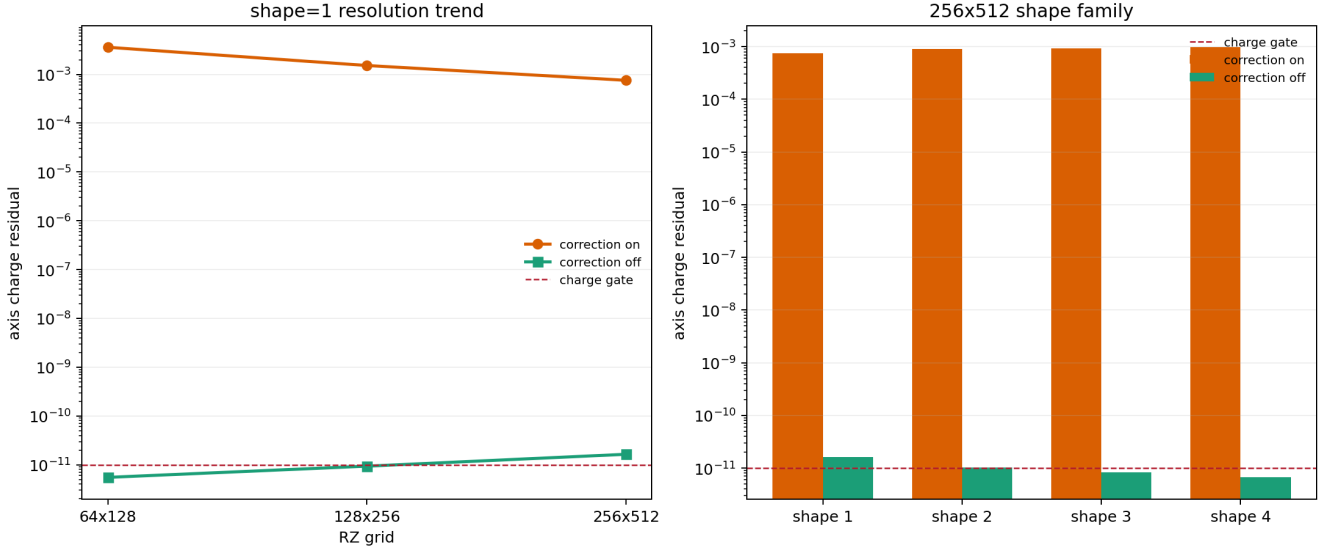


图 5-3: RZ Esirkepov axis-correction/shape tradeoff. 左侧是 shape=1 的三档分辨率趋势, 右侧是 256x512 下 shape=1/2/3/4 的 correction-on/off 对照; 红色虚线是 $1e-11$ charge gate. 所有 field gate 均通过, 但 correction-on 的 axis residual 仍约为 $0(1e-3)$, correction-off 的 charge 结果随 shape 变化, 不能据此修改全局默认值或宣称正式收敛阶。

为避免把不同诊断量混成一个结论, 应单独比较 ρ , $\rho_{\text{electrons}}$ 和 ρ_{ions} . 在代表性 refined case 中, $\rho - (\rho_{\text{electrons}} + \rho_{\text{ions}})$ 可达到机器精度; 这只说明 ρ -side species decomposition 一致, 不能替代 $\text{divE-}\rho$ 、current closure 或完整 Gauss-law contract, 因为同一批 case 的 axis residual 仍为 10^{-3} 量级。

将同一 reader-side observable 扩展到 shape=1/2/3/4 的统一 family 后, 四个 shape 的末态 $\rho - (\rho_{\text{electrons}} + \rho_{\text{ions}})$ 相对差为 $9.124e-15/1.303e-14/1.228e-14/1.343e-14$, integrated- ρ 漂移相对 $\text{abs}(\rho)$ scale 为 $6.495e-6/2.371e-6/2.729e-6/3.124e-6$. 这补齐的是 ρ -side species decomposition 的 shape coverage, 不是 $\text{divE-}\rho$ 守恒闭合; 同面 axis residual 仍保持 BOUNDARY。

对同一批 256x512、2-rank RZ sibling 做径向 profile: 把同面 $\text{abs}(\text{divE-}\rho/\epsilon_0)$ 按 $r=0$ 、 $r=1$ 和 $r \geq 2$ 分层。correction-on/default 的 shape=1/2/3/4 最大值分别为 $7.554e-4/8.990e-4/9.289e-4/9.729e-4$, 而 correction-off 为 $1.639e-11/1.020e-11/8.399e-12/6.669e-12$; 8 个 case 的全局 profile maximum 都落在 $r=0$, 且 $r=0$ 高于近轴与 off-axis 分层。这将 reader-side 观测定位为 axis-dominated, 但不区分 axis volume scaling、staggering/interpolation、mode handling 和 deposition kernel, 也不关闭默认 correction-on 的 $\text{divE-}\rho$ boundary。

将该 profile 扩展到相同 8 个 case 的全部数值 plotfile: diag1000000 初始化帧、diag1000040 中间帧和 diag1000080 末帧共 24 帧。初始化帧排除在 evolved-time 分类外, 因为 $t=0$ 的零场基线不适合与推进后的 $\text{divE-}\rho$ 残差使用同一解释。排除初始化帧后, 16 个 evolved frames 的最大值全部仍在 $r=0$, 因此 axis dominance 不是单一末帧偶然现象。这仍不能区分轴体积缩放、staggering/interpolation、mode handling 与 deposition kernel, 也不关闭 $\text{divE-}\rho$ 、current closure 或 formal convergence boundary。

对同一 8 个 case 的 ρ 、 $\rho_{\text{electrons}}$ 和 ρ_{ions} 做全时间 species decomposition: 初始化 diag1000000 的相对差约为 $1.37e-2/1.93e-2/1.96e-2/2.28e-2$, 但排除该 pre-evolution baseline 后, 16 个 evolved frames 的最大相对差分别为 correction-on $1.854e-14/1.636e-14/1.591e-14/1.435e-14$, correction-off $1.599e-14/1.389e-14/1.341e-14/1.347e-14$, 全部通过 $1e-12$ gate。这说明 ρ 组装在 evolved-time reader-side 已与物种和保持机器精度一致; 仍不关闭独立的 $\text{divE-}\rho$ axis residual、current closure、轴体积耦合或正式收敛。

对两组 RZ/RSPHERE family 做 correction-on axis charge repeat stability 检查后, 6 个 correction-on level 的 axis residual 在两组 family 之间全部通过 $1e-10$ 相对重复容差, 且每个 level 的 axis residual 都高于对应 off-axis residual。correction-off 的 RZ 低残差已接近 reader/numerical floor, 因此只报告绝对值与相对差, 不把放大的相对末位差作为失败。这强化的是稳定的 reader-side boundary, 不关闭 deposition kernel root cause、current closure 或正式 order。

同一诊断扩展到 RCYLINDER/RSPHERE shape=1 后, 关闭轴修正可以显著降低 residual, 但 RSPHERE 仍可能略高于强 gate。两者都不支持直接修改全局默认值。

RSPHERE 的 64/128/256 resolution paired control 进一步显示: correction on 的 residual 为 $4.166e-2/1.390e-2/4.142e-3$, correction off 为 $2.420e-11/9.843e-11/7.461e-11$; 六个 field gate 都通过, 但六个 charge gate 都未闭合。因此这条证据只能说明 axis/resolution 组合敏感, 不能替代正式收敛研究或作为全局默认参数修改依据。该组 256 case 必须使用专用 warpx.rsphere executable; 若误用 warpx.3d, 会在 boundary-array parser 阶段失败, 不能作为物理结论。

RCYLINDER/RSPHERE 的 $\text{shape}=1/2/3/4$ 都可得到径向 E_r field gate, 但 $\rho/\text{div}E$ charge residual 仍高于 $1e-11$ 强 gate, 且最大值由轴向 cell 主导。径向 field 通过不能写成完整 Gauss-law PASS。

源码中 `boundary.verboncoeur_axis_correction` 的解析与 `ApplyInverseVolumeScalingToChargeDensity()` 的调用时机解释了这条敏感性: RZ/RCYLINDER 的轴体积因子是 $1/3$ 对 $1/4$, RSPHERE 是 $1/4$ 对 $1/8$ 。因此径向 field shape coverage 有运行证据, charge residual 的轴体积/诊断耦合也有源码映射, 但尚未形成跨 geometry、shape、resolution 的统一强守恒合同。

在已有源码分派表之外, 还需要一张“证据覆盖表”, 因为 geometry、shape/order、AMR、axis correction 和 implicit solver 是相互独立的维度。当前最强的运行级覆盖可压缩为:

family	geometry	shape/order	当前证据	证据范围
Esirkepov	1D_Z	1	field + charge PASS	Langmuir
Esirkepov	XZ	1/2/3/4	field + charge PASS	2D Langmuir siblings
Esirkepov	3D	1/2/3/4	64^3 shape= $1/2$ field + charge PASS、shape= $3/4$ field BOUNDARY; 128^3 refined shape= $2/3/4$ field + charge PASS	Langmuir base + refined controls
Esirkepov	XZ + AMR	1	field PASS; level charge BOUNDARY	2D MR overlay
Esirkepov	RZ	1/2/3/4	field PASS; correction-on charge BOUNDARY; correction-off refined PASS	axis correc- tion/resolution family
Esirkepov	RCYLINDER/RSPHERE	1/2/3/4	radial E_r PASS; shape= $1/2/3/4$ $\rho/\text{div}E$ charge observed but BOUNDARY	不含完整 charge/Gauss-law
Villasenor implicit	XZ	2	energy + Gauss-law PASS	native/filtered/PICMI siblings
Villasenor implicit	XZ	4	cropping Gauss-law PASS	near-boundary cropping
Villasenor implicit	RZ	2	build/runtime BOUNDARY	未进入物理计算

这张矩阵回答“证据在哪里、证据能支持什么”, 不是所有 Cartesian product 的回归缺口。当前仍不能声明: RZ correction-on charge 已闭合、RCYLINDER/RSPHERE 已有完整 charge contract、2D MR 已完成 route-count/intermediate-field 证明、RZ implicit Villasenor 已进入物理计算, 或 $3D$ shape= $3/4$ 已形成正式 convergence-order 证明。

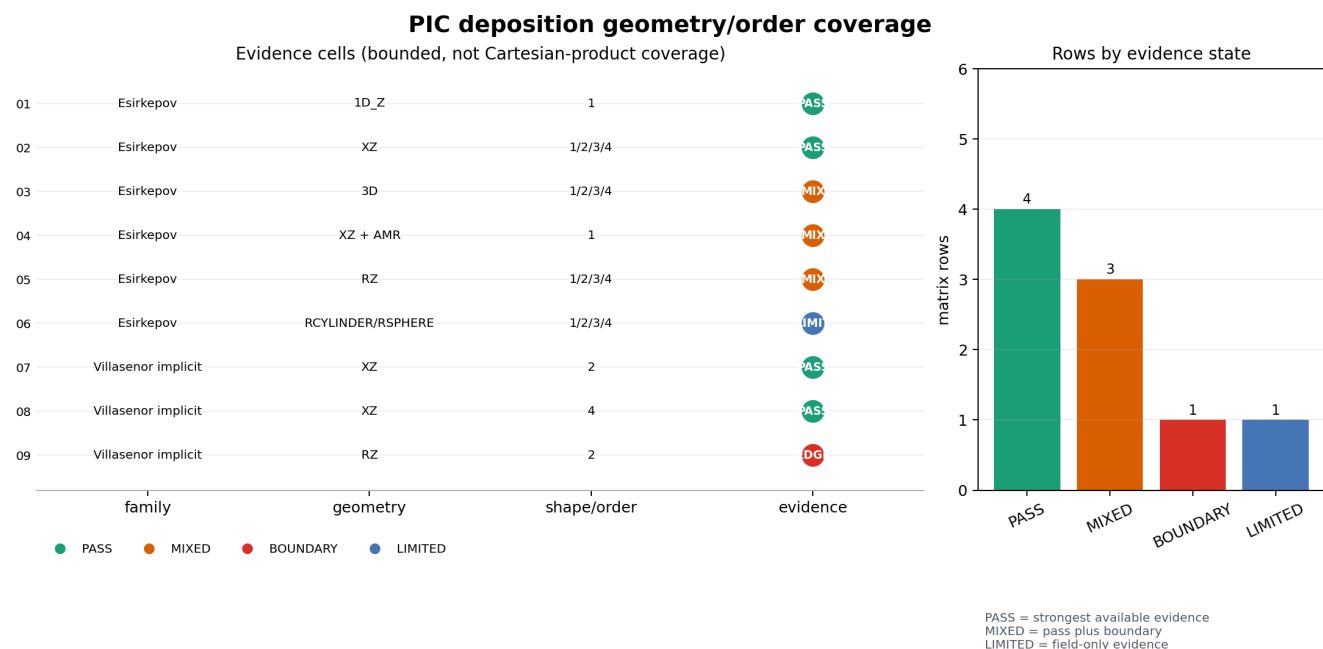


图 5-2: 当前 deposition geometry/order 证据矩阵的可视化。PASS 表示该行最强可用证据通过, MIX 表示同一行同时包含通过项和边界项, EDGE 表示构建或运行边界, LIMIT 表示只覆盖径向场而非完整 charge/Gauss-law。它展示的是九条证据行, 不是完整 geometry $\times$ shape/order 的笛卡尔积。

5.14.1 源码定位与结论范围

本章的源码路径和函数符号是阅读起点, 不是永久不变的 API。使用不同 WarpX 版本时, 先按函数名和调用关系定位, 再检查下表中的物理职责是否仍存在; 不要只因文件名相同就把后续版本当作本书所解释的等价实现。

按下面五个问题阅读源码, 比在一张宽表中比较路径更可靠:

1. 旧/新 rho 怎样区分时间层? 在 WarpXParticleContainer.cpp 查 icomp、time_shift_delta、LowerCorner 与 deposit_charge。这些入口支持本节的时间层解释, 但不能证明某次运行的 component 值一定正确。
2. 粒子为何能直接沉到另一层? 在 DepositCharge.H 查 depos_lev、rel_ref_ratio、GPU alias 和 CPU lockAdd。它们说明 level 与暂存路径存在, 不等于 CPU/GPU 数值结果已经逐点等价。
3. 隐式电流如何恢复端点? 在 CurrentDeposition.H 查两条 implicit 入口和 xp_np1 重建。它支持“端点恢复”和“共享守恒 kernel”是两层职责, 不能证明 RZ implicit runtime 已通过。
4. Villasenor 怎样处理多次 crossing? 在 Villasenor kernel family 查 crop_at_boundary、cell_crossings 和 num_segments。它支持 crossing-driven segment loop 的解释, 不能证明每个 geometry/order 组合都已运行。
5. shape 与径向几何在哪里分派? 在 ShapeFactors.H 和 ChargeDeposition.H 查 helper、shape 与 geometry 分支。它们把本节指向正确的实现职责, 但不替代 C++ 语义审计或完整笛卡尔积回归。

这些入口检查的作用是维持“正文解释能回到源码”的可追踪性, 而不是把函数名出现误写成物理验证。论文 publisher PDF 对照、完整 geometry/order runtime 和 RZ implicit 运行边界仍按前文分类保留。

5.14.2 覆盖范围与已知空白

上一张 coverage matrix 解决了“已有证据分布在哪里”, 但成书还需要明确记录“哪些组合仍然没有证据”。因此本节新增一份 **negative-space contract**: 它只登记已知缺口、当前分类和下一步证据入口, 不把缺少 runtime 结果的行写成 PASS。

缺口	当前分类	下一步证据入口
RZ 默认 axis correction 下的 charge residual	BOUNDARY	分离 axis-volume 与诊断路径; 在证据闭合前不修改默认值
RCYLINDER/RSPHERE 的完整 charge/Gauss-law	BOUNDARY	建立 geometry-specific charge consumer, 不能由径向 field PASS 代替

缺口	当前分类	下一步证据入口
2D MR transition-zone route-count	UNPROVEN	接入真实 intermediate-field/route ledger; schema fixture 不等于 runtime proof
RZ implicit Villasenor	PRE_PHYSICS_BOUNDARY	取得兼容 PETSc/AMReX build 后重跑, 不把 SIGILL 写成算法失败
Villasenor 非 XZ geometry/order family	PARTIAL	每次增加一个带独立 consumer 的 sibling
Vay geometry/order family	PARTIAL	将支持路径与 RZ/1D source guard 分开逐项补证据
跨 geometry/shape 的正式收敛阶	UNPROVEN	固定 observable、误差范数和 resolution family 后再做 study

Vay 的可用范围尤其需要按“能运行的条件”而不是算法名称来读。源码分派和官方测试入口确认其 Cartesian 2D/3D 路径, 以及 RZ/1D/implicit 的 guard。在该范围内, 单进程和两进程的 Cartesian Langmuir 分析都能通过指定的 `divE-rho/epsilon_0` gate, shape 扩展到 1..4 的 sibling 也有通过记录。这些结果只能支持“指定 Cartesian 配置可用”, 不能外推到 AMR、边界裁剪、RZ、1D、非 Cartesian geometry 或正式收敛阶。

对 AMR, 准确结论更强也更窄: WarpX 在初始化阶段显式拒绝 `Vay + mesh refinement`, 并非一次进入物理推进后的数值失败。读者应把它当作输入组合限制, 并在尝试运行前检查, 而不是用某个 Cartesian 通过案例替代这条 guard。

读者主线: 从守恒问题走到可解释的输入选择

到这里, 读者不需要按 v0.x 的时间顺序记住每一次新增实验。更有用的阅读方式是本章压缩成四个问题:

1. 粒子走过一段轨迹后, 网格上应该改变什么? 旧电荷、新电荷和半步电流必须满足离散连续性方程; 因此 Esirkepov 和 Villasenor-Buneman 的核心差别, 是它们怎样把端点形函数或轨迹 crossing 变成守恒的面通量。
2. 同一个算法为什么在不同几何上不能直接互相替代? Cartesian、RZ、RCYLINDER、RSPHERE 的网格自由度、轴处理和体积因子不同。某个几何的 `divE-rho` 通过, 只能证明该几何、shape、时间步和诊断 consumer 的组合。
3. 看到一个残差时, 先怀疑什么? 先分开 field error、all-cell charge、axis charge 和 off-axis charge, 再检查粒子状态、时间层、stencil 和 inverse-volume scaling。不要用 all-cell 平均掩盖轴向局部误差, 也不用 correction-off 的局部 PASS 推断默认 correction-on 已修复。
4. 怎样把证据转成输入决策? 先查 geometry/AMR/时间层是否允许该 deposition path, 再查 shape 和 guard-cell 预算, 最后选择与当前 observable 对应的 analysis。输入选择的结论应写成“在这些条件下可复现”, 而不是“这个算法永远更准确”。

后续小节保留代表性实验的数字、命令和分类, 适合作为证据索引; 第一次阅读可以先跳到 5.15 的结论和 5.16 的练习, 第二遍再用这些小节核对上述四个问题。

5.14.3 选择沉积算法: 先问约束, 再问精度

选择电流沉积算法时, 名称不是第一判断条件。应依次检查几何和网格布局、显式或隐式时间层、轨迹信息是否足够、以及可用的诊断量。下表给出读者可以直接用于输入设计的稳定结论。

选择面	Direct	Esirkepov	Villasenor-Buneman	Vay
离散目标	速度加权源项, 不自动满足离散连续性	由新旧形函数差构造守恒电流	由 cell crossing 分段构造面通量	专用两阶段 D-field 组织
轨迹信息	当前时间层速度	新旧端点对齐后的 shape	端点、crossing 与 segment fraction	显式 push 和专用 D 字段
时间层	显式/隐式均有, 但守恒性需另验	有显式/隐式入口	显式/隐式共享 segment backend	该源码快照仅显式
主要约束	不能由 current correction 自动升级为守恒算法	几何、shared-memory 与 collocation 分支需逐项查 guard	geometry/order 组合需逐项验证	Vay + AMR 在初始化阶段明确拒绝
典型诊断	非守恒对照	<code>divE-rho/epsilon_0</code> 、charge residual	能量/Gauss-law 与 crossing-sensitive case	Cartesian case 的 <code>divE-rho/epsilon_0</code>

这张表支持的是输入前的排查顺序，而不是算法排名。单个 Langmuir case 的通过不能推出 RZ axis、AMR、boundary crop、全部 shape 或其他诊断量同样可靠。

5.14.4 读懂证据：公式、源码和运行结果各回答什么

同一个“算法正确”的说法至少包含三件不同的事：论文或代数是否说明了离散构造，该源码快照是否含有相应实现入口，以及给定 case 是否通过指定 observable。三层证据必须并列，而不能互相替代。

证据层	它能回答的问题	它不能回答的问题
论文与公式	离散构造为什么应满足某种守恒或一致性	WarpX 的每个分支是否逐式相同，或某个 case 是否通过
源码快照	哪个 geometry、时间层和 guard 把算法接入主循环	所有输入、并行规模和数值参数下的物理正确性
producer + consumer	指定输入和指定误差范数下，输出是否满足 gate	未运行的 geometry、shape、AMR 或更强物理结论

因此，读者在引用本章的结论时应写明 scope。例如，Esirkepov 有预印本公式、当前 kernel 和代表性 runtime consumer 的三层交叉证据；这不等价于 CPC 定稿已逐式对照，也不等价于完整 geometry $\times$ order $\times$ AMR 覆盖。Villasenor-Buneman 的二维 implicit case 也不能替代 RZ runtime。Vay 的 Cartesian 2-rank family 通过，只能说明当前支持的 Cartesian 范围；它不改变源码对 AMR、RZ 与 1D 的限制。

5.14.5 RZ axis residual: 把局部诊断和算法错误分开

RZ Esirkepov 是本章最容易被误读的例子。默认 axis correction 下，代表性 64x128 $\rightarrow$ 128x256 $\rightarrow$ 256x512 family 的场误差随加密下降，shape=1 的 axis charge residual 约从 3.593e-3 降到 7.554e-4；但它仍主要位于 $r=0$ ，不能用场误差通过来宣布 charge closure。

关闭 axis correction 的某些 refined sibling 可以得到约 1e-11 的 charge residual，但高阶 shape 的 coarse field 又可能退化。因此它是一个诊断对照，不是可以全局套用的“修复开关”。非中性控制进一步表明，species rho 的轴向变化会随 shape 和 sampled-axis cancellation 改变 total rho 的表现结果；它收窄了问题所在，却没有给出 deposition kernel root cause。

读者应按以下顺序分析类似残差：先分离 field、all-cell charge、axis 与 off-axis residual；再检查粒子状态、时间层、RZ divergence stencil 与 inverse-volume scaling；最后才讨论 deposition、axis correction 或 diagnostics 的哪条路径需要更强证据。当前准确分类仍是 BOUNDARY，而不是“默认算法错误”或“默认算法已证明正确”。

5.14.6 收敛研究：描述性趋势不是正式阶数

本章现有 RZ 与 RSPHERE family 已足以比较相邻网格的误差趋势，也已经有两组独立 2-rank producer 的 correction-on repeat-slope 比较：14 项都在预注册容差内，最大绝对 slope 差为 2.0135e-11。这证明相同 reader-side norm 下的重复性，并不自动给出唯一的 formal numerical order。

正式收敛主张还必须同时固定 geometry、误差范数、时间步/粒子数/边界等控制变量、拟合区间和 primary observable；尤其不能用 all-cell residual 代替 axis residual。当前 correction-on 的 axis-charge boundary 仍开放，correction-off 因接近 numerical floor 只保留为负对照。读者可以把这些数据用作设计下一轮 refinement study 的模板，但不应把描述性 slope 写成程序或论文的正式收敛阶。

5.15 本章结论

沉积的物理底线是离散连续性方程，但读者不应把它理解成某一个 DepositCurrent() kernel 的孤立性质。它由形函数、粒子轨迹、旧/新电荷时间层、current kernel、AMR 同步和场边界共同决定。面对 divE-rho 残差、异常噪声或边界电荷时，可按以下顺序判断：

1. 先定义要守住的量。比较的是局部 rho、电流、离散连续性、Gauss law、场解析解，还是粒子统计矩？这些量的时间层和诊断 consumer 不同，不能把一个通过的场误差直接当作 charge closure。
2. 再选择与轨迹信息相容的电流算法。Direct 是直观的速度加权对照；Esirkepov 由新旧形函数差构造守恒电流；Villasenor-Buneman 按 crossing 分段构造面通量；Vay 使用专用的两阶段 D 场组织。几何、显式/隐式时间层、grid layout 和 AMR 限制先于“哪一个更精确”的比较。
3. 把 charge、current 与同步视为一条链。PhysicalParticleContainer::Evolve() 写入旧 rho component 0，轨迹进入 DepositCurrent(relative_time=-0.5 dt)，推进后状态写入新 rho component 1；随后 SyncCurrentAndRho() 处理 guard、物种求和、fine/coarse 合并、filter 和边界，再交给 field solver。跳过其中任一层，都不能把 tile 级沉积解释成求解器实际消费的源项。

4. 用与问题匹配的证据收束结论。解析 Langmuir 波和 $\text{div} \mathbf{E} - \rho / \epsilon_0$ 可检验指定输入下的场/源一致性；crossing 或公式恒等式可解释离散构造；源码入口说明可用分派。三者相互补充，不能互相替代。RZ axis、径向几何、AMR transition zone 和隐式路径仍应按各自的观察量保留边界，不从 Cartesian 通过案例外推。

第 6 章将从这里接手已经同步的 ρ/J ，讨论不同 Maxwell solver 如何消费它们；第 7 章继续解释边界、PML 和 AMR 如何改变 source 的有效定义；第 8 章则把本章涉及的场、粒子与守恒量组织成 diagnostics。

5.16 练习与源码定位

1. 连续性方程题：从 ρ 的 old/new 时间层出发，解释为什么 Direct current deposition 不能自动保证 $\Delta t \text{ div } \mathbf{J} = \rho_{\text{old}} - \rho_{\text{new}}$ ，而 Esirkepov/Villasenor 必须引入轨迹或 crossing 信息。
2. 源码定位题：分别定位 DepositCharge()、DepositCurrent() 和 SyncCurrentAndRho() 的入口，给每个函数写出一个“它负责什么”和一个“它不负责什么”的边界。
3. 观察量设计题：选择 Examples/Tests/langmuir/ 的一个 current-correction 或 Vay-deposition 变体，列出它比较的场量、source 一致性量和容差；写出一个该 case 不能证明的 geometry、AMR 或时间层结论。
4. 公式与运行边界题：从 5.11 的 old/new shape difference 推导一项局部恒等式，再与 Villasenor 的 crossing 分段和一个端到端 Gauss-law 案例比较。说明公式级恒等式、源码分派和端到端回归分别回答什么，为什么三者不能替代。

6. 电磁场求解器

场求解器把上一章已经同步的电荷与电流变成下一时刻的电磁场。本章不按“有哪些 .cpp 文件”展开，而按读者在设计输入或排查异常时真正需要回答的因果链组织：源项在什么时间层可用，选择了哪个场方程离散，边界/PML 怎样参与更新，最后用哪个观察量检验该选择。

阅读源码时，从 Source/Evolve/WarpXEvolve.cpp 的 WarpX::OneStep_nosub() 起步。该函数先完成粒子推进与 SyncCurrentAndRho(),再以 electromagnetic_solver_id 分成 FDTD 和 PSATD。Source/FieldSolver/WarpXPushFieldsEM.cpp 提供两条主入口：WarpX::EvolveB() / WarpX::EvolveE() 负责有限差分更新，WarpX::PushPSATD() 负责谱空间路径。WarpX::OneStep_JRhom() 则是 PSATD 的特殊时间模型：同一 PIC 步内多次沉积源项，但粒子只推进一次。

读者主线：从同步源项到可检验的场

1. 先固定源项时间层。在普通显式路径中，粒子推进后 J 位于半步，rho 有旧/新两个分量；SyncCurrentAndRho() 完成过滤、guard-cell 交换、AMR 层间处理与边界处理。场更新读取的是这条同步后的 source 链，而不是任意 tile 的局部数组。
2. 再选择场的时间与空间离散。FDTD 使用 EvolveF/G -> EvolveB(dt/2) -> EvolveE(dt) -> EvolveF/G -> EvolveB(dt/2) 的交错推进；PSATD 先把场和 source 变换到谱空间，经 PSATDPushSpectralFields() 更新后再反变换。两者不是同一更新式的不同开关。
3. 把边界视为更新的一部分。FDTD 的 EvolveB() / EvolveE() 同时按 fine/coarse patch 分派普通网格和 PML split field；PSATD 的主域谱推进之后会推进 PML 区域并施加场边界。PML、guard cell 与边界条件改变的是求解器实际消费的场状态，不能只在主域 curl kernel 之外事后理解。
4. 用与所选路径相符的观察量束束。FDTD 需要同时检查 CFL、色散与边界反射；PSATD 还必须检查 FFT、current correction/Galilean/Comoving 的组合限制；JRhom 的结论还依赖 source 时间模型和 subinterval 数。一个案例通过不等于另一几何、另一边界或另一 source 模型同样通过。

选择问题	首先阅读的入口	可以据此判断	不能据此断言
用交错或差分场更新	OneStep_nosub()、 EvolveB()、EvolveE()、 FiniteDifferenceSolver	半步 B / 整步 E 的调度与实际 curl 算子	所有 CFL、边界和网格分辨率下的 准确度
用谱空间推进	PushPSATD()、 SpectralSolver、 PsatdAlgorithm*	source/field 的 transform、 谱更新与 inverse transform 顺序	未测试的 current correction、 PML 或 AMR 组合
抑制 boosted-frame NCI	psatd.v_galilean、 Galilean algorithm	该设置选择 Galilean PSATD 且有 direct-deposition 前提	任意 deposition 或边界组合都会 抑制 NCI
使用多时间节点 source	OneStep_JRhom()、 psatd.JRhom	每个 subinterval 会重新沉积并 同步所需的 J/rho	普通 PSATD 和 JRhom 的数 值误差必然相同
吸收开放边界	PML、PML_RZ、 EvolveBPML()、 EvolveEPML()	PML split field 参与同一时间 推进	主域无反射或物理解已被证明

显式 FDTD 的经典时间交错可抽象为

$$\begin{aligned}\mathbf{B}^{n+1/2} &= \mathbf{B}^n - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^n, \\ \mathbf{E}^{n+1} &= \mathbf{E}^n + c^2 \Delta t \nabla_h \times \mathbf{B}^{n+1/2} - \frac{\Delta t}{\epsilon_0} \mathbf{J}^{n+1/2}, \\ \mathbf{B}^{n+1} &= \mathbf{B}^{n+1/2} - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^{n+1}.\end{aligned}$$

OneStep_nosub() 中的普通电磁分支还会在两次 B 推进的前后处理清洁场 F/G，并在每个必要阶段填充 guard cells。若介质模型不是 vacuum，E 的主入口会转为 MacroscopicEvolveE()；因此“FDTD”并不自动表示一条无介质、无边界辅助场的最小 Yee 更新。

选择路径前的检查表

- **Yee FDTD**. 选择交错网格上的有限差分 curl, 并用 `warpx.cfl` 与网格尺度共同约束时间步。CartesianYeeAlgorithm.H 给出空间差分, EvolveB.cpp / EvolveE.cpp 消费该差分; 应分别检查传播色散、守恒量与边界反射。
- **CKC 或 Nodal FDTD**. 它们仍通过 EvolveB/E 的调度框架, 却改变 Upward/Downward 差分或场的空间布局。先确认所选网格与 gather/deposition 约束, 再讨论“色散更低”是否对当前传播方向与分辨率有意义。
- **标准、Galilean 或 Comoving PSATD**. PSATD 需要 FFT 支持。非零 `psatd.v_galilean` 选择 Galilean 算法; 官方参数说明同时要求 direct current deposition。Comoving 也有自己的 source 与输入限制, 不能把二者都简称为“移动坐标 PSATD”。
- **JRhom PSATD**. `psatd.JRhom` 的字母指定 J 和 rho 的时间依赖, 数字指定 subinterval 数; 它只属于 PSATD, 且 source 的多次沉积是算法定义的一部分, 不是普通输出频率设置。
- **PML**. PML 不是一个求解器选项的尾部阻尼。FDTD 与 PSATD 分别有自己的 PML 更新入口; 验证时应把反射率/场能量与 PML 之外的主域误差分开报告。

本章随后先解释 FDTD curl 和 PML, 再进入 PSATD、JRhom、RZ、静电/静磁、Hybrid PIC 与 regression。每一节都区分: 公式解释什么、源码能定位什么、一个具体 regression 又实际检验了什么。

6.1 FDTD 差分算子: Yee、Nodal 与 CKC

在 Source/FieldSolver/FiniteDifferenceSolver/CartesianYeeAlgorithm.H 中, `T_Algo::Upward/Downward` 把 FDTD 模板连接到具体差分。Yee 的 UpwardDx 和 DownwardDx 分别是 staggered forward/backward difference:

```
return inv_dx*( F(i+1,j,k,ncomp) - F(i,j,k,ncomp) );
```

```
return inv_dx*( F(i,j,k,ncomp) - F(i-1,j,k,ncomp) );
```

Nodal grid 没有 Yee 那种 E/B 空间交错, 所以 UpwardDx 是中心差分, DownwardDx 直接调用同一个函数:

```
return 0.5_rt*inv_dx*( F(i+1,j,k,ncomp) - F(i-1,j,k,ncomp) );
```

CKC 的关键差异是 Upward 使用横向邻点加权扩展 stencil, 而 Downward 保持局部 backward difference。这对应理论文档中的

$$D_t \mathbf{B} = -\nabla^* \times \mathbf{E}, \quad D_t \mathbf{E} = \nabla \times \mathbf{B} - \mathbf{J}.$$

因此同一个 EvolveB.cpp 模板 kernel 在传入 CartesianYeeAlgorithm、CartesianNodalAlgorithm 或 CartesianCKCAlgorithm 时, 会得到不同的离散 curl。

本章的文献主线是 Yee、GodfreyJCP2014_PSATD、Lehe2016 和 VayJCP2013。Yee 1966 目前只能使用 indexed abstract: 它支持 finite-difference Maxwell、field-point placement、PEC boundary 和 conducting-cylinder example, 但不足以把 WarpX 的完整 Yee stencil 说成逐式来自 Yee 原文。CartesianYeeAlgorithm.H、FiniteDifferenceSolver.cpp、EvolveB.cpp 和 EvolveE.cpp 则说明现代实现怎样把本节的差分职责接入主循环; 这种源码对应不证明历史论文逐式等价。PSATD 与 Galilean NCI 的推导应分别回到各自论文和后文的离散方程阅读, 而不能由这条 FDTD 对应代替。

6.2 FDTD PML split-field 更新

PML 的目标是在计算区域边缘吸收入射电磁波。Berenger PML 的基本做法不是简单给整个场乘阻尼, 而是把场分量拆成不同方向的 split components, 并对这些分量施加匹配吸收。WarpX 的 FDTD PML 分量与更新入口分别位于 Source/BoundaryConditions/PMLComponent.H 和 Source/FieldSolver/FiniteDifferenceSolver/。

PML split components 的 component 编号定义在 Source/BoundaryConditions/PMLComponent.H:

```
/* In WarpX, the split fields of the PML (e.g. Eyx, Eyz) are stored as  
 * components of a MultiFab (e.g. component 0 and 1 of the MultiFab for Ey)  
 * The correspondence between the component index (0,1) and its meaning  
 * (yx, yz, etc.) is defined in the present file */
```

```
struct PMLComp {  
    enum { xy=0, xz=1, xx=2,  
           yz=0, yx=1, yy=2,  
           zx=0, zy=1, zz=2,  
           x=0, y=1, z=2 }; // Used for the PML components of F  
};
```

因此, $Ex(i, j, k, \text{PMLComp}::xy)$ 不是另一个物理电场分量, 而是 E_x 中由 y 方向 curl 项驱动的 split component。这个存储约定贯穿 `EvolveBPML.cpp`, `EvolveEPML.cpp` 和 `EvolveFPML.cpp`。

`EvolveBPML()` 明确把非 Cartesian FDTD PML 排除掉:

```
#if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER) || defined(WARPX_DIM_RSPHERE)
    amrex::ignore_unused(fields, patch_type, level, dt, dive_cleaning);
    WARPX_ABORT_WITH_MESSAGE(
        "PML only implemented in Cartesian geometry.");
#else
```

这是一条功能边界: 当前读到的 FDTD PML kernel 不能拿来解释 RZ 或 spherical 几何的 PML。进入 Cartesian 后, B 的 split update 仍然复用 Yee/Nodal/CKC 的 `T_Algo::UpwardDz*` 差分模板。例如 B_x 的更新为:

```
Bx(i, j, k, PMLComp::xz) += dt * (
    T_Algo::UpwardDz(Ey, coefs_z, n_coefs_z, i, j, k, PMLComp::yx)
    + T_Algo::UpwardDz(Ey, coefs_z, n_coefs_z, i, j, k, PMLComp::yz)
    + UpwardDz_Ey_yy);

Bx(i, j, k, PMLComp::xy) -= dt * (
    T_Algo::UpwardDy(Ez, coefs_y, n_coefs_y, i, j, k, PMLComp::zx)
    + T_Algo::UpwardDy(Ez, coefs_y, n_coefs_y, i, j, k, PMLComp::zy)
    + UpwardDy_Ez_zz);
```

它仍对应普通 Maxwell 方程中的

$$\partial_t B_x = \partial_z E_y - \partial_y E_z,$$

但 E_y 和 E_z 已经被拆成 PML components, 所以源码必须把相关 components 求和。UpwardDz_Ey_yy 和 UpwardDy_Ez_zz 只在开启 PML divergence cleaning 时加入。

`EvolveEPML()` 做相反的 curl(B) 更新, 并在 PML 中可选加入 F 修正和粒子电流项:

```
Ex(i, j, k, PMLComp::xz) -= c2 * dt * (
    T_Algo::DownwardDz(By, coefs_z, n_coefs_z, i, j, k, PMLComp::yx)
    + T_Algo::DownwardDz(By, coefs_z, n_coefs_z, i, j, k, PMLComp::yz) );
Ex(i, j, k, PMLComp::xy) += c2 * dt * (
    T_Algo::DownwardDy(Bz, coefs_y, n_coefs_y, i, j, k, PMLComp::zx)
    + T_Algo::DownwardDy(Bz, coefs_y, n_coefs_y, i, j, k, PMLComp::zy) );
```

这对应

$$\partial_t E_x = c^2(\partial_y B_z - \partial_z B_y),$$

只是每个参与 curl 的磁场分量也以 split components 形式存储。若 `pml_has_particles` 为真, `EvolveEPML.cpp` 还会读取 `pml_j_fp/cp` 并调用 `push_ex_pml_current` 等 helper, 使 PML 中传播的粒子电流进入电场更新。

最后, `EvolveFPML()` 更新 PML divergence-cleaning 标量:

```
F(i, j, k, PMLComp::x) += dt * (
    T_Algo::DownwardDx(Ex, coefs_x, n_coefs_x, i, j, k, PMLComp::xx)
    + T_Algo::DownwardDx(Ex, coefs_x, n_coefs_x, i, j, k, PMLComp::xy)
    + T_Algo::DownwardDx(Ex, coefs_x, n_coefs_x, i, j, k, PMLComp::xz) );
```

普通区域的 F 方程含有 $-\rho/\epsilon_0$ 项; 当前 PML F kernel 只读到 split E 的 divergence 累积。PML 中吸收系数、sigma profile、damping 因子和 current damping 的细节不在这三个 field update 文件中, 下一步要继续进入 `BoundaryConditions/PML.cpp` 和 `BoundaryConditions/PML_current.H`。

6.3 PML sigma profile、damping 与电流源项

Source/BoundaryConditions/PML.cpp 与 Source/Evolve/WarpXEvolvePML.cpp 负责把 PML profile 和主循环衔接起来。PML 的吸收 profile 由 FillLo() / FillHi() 生成:

```
Real offset = static_cast<Real>(glo-i);
p_sigma[i-slo] = fac*(offset*offset);
p_sigma_cumsum[i-slo] = (fac*(offset*offset*offset)/3._rt)/v_sigma;
if (i <= ohi+1) {
    offset = static_cast<Real>(glo-i) - 0.5_rt;
    p_sigma_star[i-sslo] = fac*(offset*offset);
    p_sigma_star_cumsum[i-sslo] = (fac*(offset*offset*offset)/3._rt)/v_sigma;
}
```

对应的 profile 是

$$\sigma(s) = Cs^2, \quad \int_0^s \sigma(s') ds' = \frac{Cs^3}{3}.$$

SigmaBox::ComputePMLFactorsE/B() 再把它转成指数阻尼:

```
p_sigma_star_fac[idim][i] = std::exp(-p_sigma_star[idim][i]*dt);
p_sigma_fac[idim][i] = std::exp(-p_sigma[idim][i]*dt);
```

DampPML_Cartesian() 在每个 PML tile 上调用 warpx_damp_pml_ex/ey/ez 和 warpx_damp_pml_bx/by/bz。这些 kernel 根据场的 staggered 位置选择 sigma_fac 或 sigma_star_fac, 对 split components 逐方向相乘。例如 Exy 乘 y 方向阻尼, Exz 乘 z 方向阻尼; 若开启 do_pml_dive_cleaning, Exx 也乘 x 方向阻尼。

若 PML 中有粒子电流, push_ex_pml_current() 会把 J_x 按横向 sigma 比例分给 Exy 和 Exz:

```
alpha_xy = sigjy[k-ylo]/(sigjy[k-ylo]+sigjz[l-zlo]);
alpha_xz = sigjz[l-zlo]/(sigjy[k-ylo]+sigjz[l-zlo]);
Ex(j,k,l,PMLComp:xy) = Ex(j,k,l,PMLComp:xy) - mu_c2_dt * alpha_xy * jx(j,k,l);
Ex(j,k,l,PMLComp:xz) = Ex(j,k,l,PMLComp:xz) - mu_c2_dt * alpha_xz * jx(j,k,l);
```

而 DampJPML() 使用的是 sigma_cumsum_fac, 不是场 damping 的 sigma_fac:

```
damp_jx_pml(i, j, k, pml_jxfab, sigma_star_cumsum_fac_j_x,
            sigma_cumsum_fac_j_y, sigma_cumsum_fac_j_z,
            xs_lo, y_lo, z_lo);
```

这说明 WarpX 对 PML 电流采用沿吸收层积分后的 damping, 而不是简单的局部 $e^{-\sigma \Delta t}$ 。

最后, PML::Exchange() 把 split fields 求和再与常规场交换:

```
MultiFab totpmlmf(pml.boxArray(), pml.DistributionMap(), 1, 0);
MultiFab::LinComb(totpmlmf, 1.0, pml, 0, 1.0, pml, 1, 0, 1, 0);
if (ncp == 3) {
    MultiFab::Add(totpmlmf, pml, 2, 0, 1, 0);
}
```

所以 PML 的完整链条是: 常规场进入 PML split component, PML 内 curl 更新, 乘 sigma damping, split components 求和回填常规边界。只看 EvolveEPML.cpp 或只看 DampPML() 都不足以说明 PML 的实际边界条件。

6.4 非 Cartesian FDTD: RZ、RCYLINDER 与 RSPHERE

Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceSolver.cpp 把 FDTD 分派到编译几何分支。RZ/RCYLINDER 和 RSPHERE 不走 Cartesian CKC/Nodal 分支, 而是只接受 Yee/HybridPIC:

```
#if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER)
    m_dr = cell_size[0];
    m_nmodes = WarpX::n_rz_azimuthal_modes;
    m_rmin = WarpX::GetInstance().Geom(0).ProbLo(0);
```

```

if (fdtd_algo == ElectromagneticSolverAlgo::Yee ||
    fdtd_algo == ElectromagneticSolverAlgo::HybridPIC ) {
    CylindricalYeeAlgorithm::InitializeStencilCoefficients( cell_size,
        m_h_stencil_coefs_r, m_h_stencil_coefs_z );
}

```

cylindrical Yee 的核心算子不是 $\partial_r F$ ，而是

$$\frac{1}{r} \frac{\partial(rF)}{\partial r}.$$

源码中对应：

```

return 1._rt/r * inv_dr*( (r+0.5_rt*dr)*F(i+1,j,k,comp) - (r-0.5_rt*dr)*F(i,j,k,comp) );

```

RZ 的 azimuthal mode 展开使 ∂_θ 变成 im ，所以实部/虚部会互相耦合。EvolveBCylindrical() 中 Br 的高阶 mode 更新为：

```

Br(i, j, 0, 2*m-1) += dt*(
    T_Algo::UpwardDz(Etheta, coefs_z, n_coefs_z, i, j, 0, 2*m-1)
    - m * Ez(i, j, 0, 2*m )/r );
Br(i, j, 0, 2*m ) += dt*(
    T_Algo::UpwardDz(Etheta, coefs_z, n_coefs_z, i, j, 0, 2*m )
    + m * Ez(i, j, 0, 2*m-1)/r );

```

轴上 $r = 0$ 不能直接除以 r ，源码显式使用正则化条件。例如 $Etheta(r=0, m=1) = -i Er(r=0, m=1)$ ：

```

Etheta(i, j, 0, 2*m-1) = Er(i, j, 0, 2*m );
Etheta(i, j, 0, 2*m ) = -Er(i, j, 0, 2*m-1);

```

这类条件是 RZ FDTD 的物理核心：轴上的场必须满足坐标正则性，而不是普通网格差分的延伸。

RSPHERE 使用 spherical operator：

$$\frac{1}{r^2} \frac{\partial(r^2 F)}{\partial r}.$$

源码为：

```

return 1._rt/(r*r) * inv_dr*( rph*rph*F(i,j,k,comp) - rmh*rmh*F(i-1,j,k,comp) );

```

对应的 EvolveFSpherical() 在轴上用 $6*Er/dr$ 正则化：

```

F(i, j, 0, 0) += dt * (
    - rho(i, j, 0, rho_shift) * inv_epsilon0
    + 6._rt*Er(i, j, 0, 0)/dr);

```

这说明非 Cartesian FDTD 不能作为 Cartesian FDTD 的坐标名替换来讲；必须把 metric factors、mode coupling 和 axis regularization 都写入公式和源码解读。

6.5 PSATD 谱求解主流程

PSATD 从 Source/FieldSolver/WarpXPushFieldsEM.cpp 的 PushPSATD() 进入。理论上，它把 Maxwell 方程写到 Fourier 空间：

$$\frac{\partial \tilde{\mathbf{E}}}{\partial t} = i\mathbf{k} \times \tilde{\mathbf{B}} - \tilde{\mathbf{J}}, \quad \frac{\partial \tilde{\mathbf{B}}}{\partial t} = -i\mathbf{k} \times \tilde{\mathbf{E}}.$$

在一个时间步内对线性部分解析积分，得到含

$$C = \cos(k\Delta t), \quad S = \sin(k\Delta t)$$

的更新式。源码入口是 WarpX::PushPSATD()，当前位于 Source/FieldSolver/WarpXPushFieldsEM.cpp：

```

// FFT of E and B
PSATDForwardTransformEB();

// FFT of F and G
if (WarpX::do_dive_cleaning) { PSATDForwardTransformF(); }
if (WarpX::do_divb_cleaning) { PSATDForwardTransformG(); }

// Update E, B, F, and G in k-space
PSATDPushSpectralFields();

// Inverse FFT of E, B, F, and G
PSATDBackwardTransformEB();

```

在此之前, PushPSATD() 会根据 current_correction、Vay deposition、periodic single box 和 mesh refinement 分支处理 J/rho:

```

PSATDForwardTransformJ(current_fp_string, current_cp_string);
PSATDForwardTransformRho(rho_fp_string, rho_cp_string, 0, rho_old);
PSATDForwardTransformRho(rho_fp_string, rho_cp_string, 1, rho_new);

::PSATDCurrentCorrection(finest_level, spectral_solver_fp, spectral_solver_cp);

PSATDBackwardTransformJ(current_fp_string, current_cp_string);

```

这里需要特别记住两个实现边界。第一, fft_periodic_single_box 分支完成 current correction 或 Vay deposition 的 k-space 处理; 非 periodic-single-box 分支则在 correction/Vay 之后按条件调用 SyncCurrent()、SyncRho() 或 SumBoundaryJ()。第二, PushPSATD() 的场阶段始终是 PSATDForwardTransformEB()、可选 RZ PML push、可选 F/G transform、PSATDPushSpectralFields()、E/B/F/G 反变换, 最后才对各层 PML 和物理边界作处理。读者应以这些函数阶段核对源码, 而不是依赖某个版本的行号。

SpectralSolver 本身只负责建立 k-space、spectral field storage 和选择具体算法。当前分派入口是 Source/FieldSolver/SpectralSolver/Spe

```

const SpectralKSpace k_space= SpectralKSpace(realspace_ba, dm, dx);

m_spectral_index = SpectralFieldIndex(
    update_with_rho, fft_do_time_averaging, time_dependency_J, time_dependency_rho,
    dive_cleaning, divb_cleaning, pml);

void SpectralSolver::pushSpectralFields(){
    algorithm->pushSpectralFields( field_data );
}

```

所以真正的 PSATD 更新在 PsatdAlgorithm* 子类中。以 PsatdAlgorithmGalilean.cpp 为例, 标准 PSATD 也通过 v_galilean=0 的情形进入同一套形式:

```

fields(i,j,k,Idx.Ex) = T2 * C * Ex_old
                      + I * c2 * T2 * S_ck * (ky * Bz_old - kz * By_old)
                      + X4 * Jx - I * (X2 * rho_new - T2 * X3 * rho_old) * kx;

```

(ky*Bz-kz*By) 是谱空间 curl 的 x 分量。X1/X2/X3/X4/T2 是一节需要展开的系数, 它们把标准 PSATD、Galilean PSATD、是否使用 rho、是否时间平均等分支折叠成统一更新式。

PSATD 还必须处理 staggered 网格位置。SpectralFieldData::ForwardTransform() 在实空间场不是 nodal 时乘相位因子:

```

if (!is_nodal_0) { spectral_field_value *= shift0_arr[i]; }
if (!is_nodal_1) { spectral_field_value *= shift1_arr[j]; }
if (!is_nodal_2) { spectral_field_value *= shift2_arr[k]; }

```

相位因子来自

```
pshift[i] = amrex::exp( I*sign*pk[i]*0.5_rt*t_dx_idim);
```

即 $e^{\pm i k \Delta x / 2}$ 。这一步是把 Yee staggered 数据映射到谱算法假定的位置; 没有这一步, 谱空间 curl 与实空间场的位置会错半格。

6.6 标准/Galilean PSATD 系数和 current correction

Source/FieldSolver/SpectralSolver/PsatdAlgorithmGalilean.cpp 实现 Galilean 分支。标准 PSATD 是 Galilean 实现的 $v_G = 0$ 极限；源码中

$$w_c = \mathbf{k}_c \cdot \mathbf{v}_G, \quad T_2 = e^{iw_c \Delta t}.$$

w_c 必须使用 centered modified k:

```
const amrex::Real w_c = kx_c[i]*vg_x +
#ifdef WARPX_DIM_3D
    ky_c[j]*vg_y + kz_c[k]*vg_z;
#else
    kz_c[j]*vg_z;
#endif
```

基础振荡系数是:

```
C(i,j,k) = std::cos(om_s * dt);

if (om_s != 0.)
{
    S_ck(i,j,k) = std::sin(om_s * dt) / om_s;
}
else
{
    S_ck(i,j,k) = dt;
}
```

T2(i,j,k) = theta_c * theta_c;

其中 om_s = c*|k_s|。X1-X4 把电流和电荷项折叠进统一更新式；源码显式处理 $k = 0$ 和 $w_c = 0$ 极限，避免除零。

current correction 的源码与官方参数文档逐项对应。标准分支为:

```
fields(i,j,k,Idx.Jx_mid) = Jx - (k_dot_J - I * (rho_new - rho_old) / dt)
    * kx / (k_norm * k_norm);
```

这正是

$$\hat{\mathbf{J}}_{corr} = \hat{\mathbf{J}} - \left(\mathbf{k} \cdot \hat{\mathbf{J}} - i \frac{\hat{\rho}^{n+1} - \hat{\rho}^n}{\Delta t} \right) \frac{\mathbf{k}}{k^2}.$$

Galilean 分支为:

```
const Complex rho_old_mod = rho_old * amrex::exp(I * k_dot_vg * dt);
const Complex den = 1._rt - amrex::exp(I * k_dot_vg * dt);

fields(i,j,k,Idx.Jx_mid) = Jx - (k_dot_J - k_dot_vg * (rho_new - rho_old_mod) / den)
    * kx / (k_norm * k_norm);
```

这里 rho_old_mod 是 $\rho^n \theta^2$, den 是 $1 - \theta^2$ 。因此 current correction 的目的不是平滑电流，而是投影掉违反谱空间连续性方程的纵向误差，使修正后的电流与 rho_old/rho_new 相容。

6.6.1 先按更新对象理解 PSATD 系数

读 PSATD 源码时，最容易犯的错误是从变量名开始背系数。更可靠的顺序是先问一个系数进入了哪一类更新：场的自由振荡、横向电流源、纵向电荷源，还是供 gather/diagnostic 使用的时间平均场。

系数族	服务的对象	读者应检查的时间层	不能与之混写的对象
C、S_ck Cartesian X1-X4	无源 Maxwell 振荡与谱 curl 普通或 Galilean PSATD 的 J、rho_old、rho_new 积分	旧 E/B 与当前步长 J 的中间层, rho 的两个端点	它们不是电流沉积算法 JRhom 的 Y1-Y8 与 RZ 的 X 系数
Psi1/Psi2/Y1-Y4	time-averaged E/B 输出	平均区间及 rho_old/rho_new old/mid/new 源项层	ordinary-field push 的同名 Y
JRhom Y1-Y8	子区间内常量、线性或二次源项		Galilean average-field 的 complex Y1-Y4

因此, Cartesian standard/Galilean PSATD 的场更新可先作为一个结构来读: C/S_ck 推进真空旋转, X4 接收横向电流, X2/X3 把新旧电荷端点接入纵向场, T2 只在 Galilean 表示中携带相位。 $k = 0$ 、 $\omega_c = 0$ 和退化分母的专门分支并不是实现细节的例外, 而是解析积分在零模和共振极限必须连续的条件。

打开 psatd.do_time_averaging=1 后,输出的 E_avg/B_avg 不是把两个普通场快照简单相加。WarpX 要求它与 psatd.update_with_rho=1 一起使用, 因为平均场的解析表达也同时依赖 J 与两个电荷端点。读者若只需要普通场推进, 应先停在 X1-X4; 若要解释 particle gather、平均场诊断或对应 regression, 才沿 Psi/Y 继续追到 PSATDScaleAverageFields() 与反变换路径。

6.6.2 Galilean PSATD 解决的是表示问题, 不是滤波开关

Galilean 坐标

$$\mathbf{x}' = \mathbf{x} - \mathbf{v}_{gal}t$$

把均匀漂移等离子体在数值网格中改写为近似静止的背景。旧电荷因而要携带相位, 离散连续性方程也从普通端点差分变成带 $\theta^2 = \exp(i\mathbf{k} \cdot \mathbf{v}_{gal}\Delta t)$ 的形式。这正是上面 rho_old_mod 出现的原因。

对 boosted-frame 问题, 读者应把选择过程分成三步: 先由物理问题确定背景等离子体在计算坐标中的漂移方向; 再让 v_galilean 接近该背景漂移, 而不是任意取一个移动速度; 最后用稳定性和物理量两类证据分别检查。Lehe et al. 的理论说明这种表示能消除主要的漂移 alias resonance; Kirchen et al. 的应用说明抑制 NCI 后仍须检查回变换的加速器物理量。需要进一步判断公式或适用范围时, 应直接回到两篇论文的正文, 并将其假设与当前输入、源码分派和输出观察量逐项对应。

filter、current correction 与 Galilean 表示因此必须分开: warpx.use_filter 主要压制短波 alias; psatd.current_correction 投影连续性残差并支撑 Gauss-law; Galilean 坐标改变源项在网格上的表示。它们可在同一输入卡出现, 但不能互相替代。Godfrey-Vay 的 fixed-grid NCI 分析、WarpX 的 NCIGodfreyFilter 名称和普通 PSATD filter 也不是同一个开关或同一个证明。

6.6.3 从 regression 反推可作出的结论

输入卡写着 PSATD 不足以说明“PSATD 正确”。读者应从 consumer 反问 producer 实际被检查了什么:

证据入口	主要 observable	可以支持的结论	仍不能支持的结论
analysis_galilean.py	最终场能量相对不稳定参考值	给定输入和分支下的 NCI 抑制	通用色散关系或所有 PSATD 组合稳定
current-correction 分支	场能量与 max divE-rho/eps0	稳定性和该路径的连续性/Gauss-law 投影	Godfrey 型 $\zeta(k)$ current scaling 已实现
analysis_psatd_CC1.py	JRhom CC1 的电场能量	该 consumer 的 NCI energy gate	其他 JRhom 时间模型或 charge closure
checksum-only case	输出是否与已知基线一致	workflow、写盘和回归可重复	独立的物理正确性断言

这张表也是本章的阅读纪律: 先把算法类、输入组合和 consumer 对齐, 再解释数值结果。论文、有限阶 PSATD 限制和实际阈值只用于界定各行的证据范围, 不改变这条由问题到 observable 的阅读顺序。

6.7 PSATD-JRhom: 多次源项沉积与一阶/二阶谱更新

Source/Evolve/WarpXEvolve.cpp 的 OneStep_JRhom() 将 PSATD-JRhom 从主循环接到谱算法。物理上, JRhom 处理的是一个 PIC 时间步内 J 和 rho 不一定满足“电流常量、电荷线性”的假设。WarpX 使用 psatd.JRhom 字符串指定时间依赖:

```

std::string JRhom_input;
pp_psatd.query("JRhom", JRhom_input);
if (!JRhom_input.empty()) {
    WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
        JRhom_input.length() >= 3,
        "psatd.JRhom = '" + JRhom_input + "' input string is too short to parse."
    );
    m_JRhom = true;
    // parse time dependency of J from first character
    if (JRhom_input[0] == 'C') {
        time_dependency_J = TimeDependencyJ::Constant;
    }
    else if (JRhom_input[0] == 'L') {
        time_dependency_J = TimeDependencyJ::Linear;
    }
    else if (JRhom_input[0] == 'Q') {
        time_dependency_J = TimeDependencyJ::Quadratic;
    }
}

```

第一个字符控制 J，第二个字符控制 rho，后续数字控制子区间数 m。例如 CL1 是标准 PSATD 源项假设，LQ4 表示 J 分段线性、rho 分段二次，并把大时间步拆成 4 个子区间。谱求解器分配时会把内部时间步改成

```

amrex::Real solver_dt = dt[lev];
if (WarpX::m_JRhom) { solver_dt /= static_cast<amrex::Real>(WarpX::m_JRhom_subintervals); }

```

因此 PsatdAlgorithmJRhom* 内部看到的是 $\delta t = \Delta t / m$ 。

JRhom 的外层 PIC loop 不走普通 PushPSATD()。它先推进粒子，但跳过普通沉积：

```

// Push particle from  $x^{\{n\}}$  to  $x^{\{n+1\}}$ 
// from  $p^{\{n-1/2\}}$  to  $p^{\{n+1/2\}}$ 
const bool skip_deposition = true;
PushParticlesandDeposit(cur_time, skip_deposition);

```

Source/Evolve/WarpXEvolve.cpp 的 OneStep_JRhom() 先把 E/B/F/G 变换到谱空间，按需清零平均场；随后对 rho 做初始沉积和 FFT，并在 J 非常量时先沉积、同步和变换 J。这是为后续子区间提供初始 source 时间层，而不是普通 PushPSATD() 的一次性 source 输入。

随后在每个子区间按时间依赖类型重新沉积 J/rho：

```

const int n_deposit = WarpX::m_JRhom_subintervals;
const amrex::Real sub_dt = dt[0] / static_cast<amrex::Real>(n_deposit);
const int n_loop = (WarpX::fft_do_time_averaging) ? 2*n_deposit : n_deposit;

for (int i_deposit = 0; i_deposit < n_loop; i_deposit++)
{
    if (time_dependency_J != TimeDependencyJ::Constant) { PSATDMoveJNewToJOld(); }

    const amrex::Real t_deposit_current = (time_dependency_J == TimeDependencyJ::Linear) ?
        (i_deposit-n_deposit+1)*sub_dt : (i_deposit-n_deposit+0.5_rt)*sub_dt;

    const amrex::Real t_deposit_charge = (time_dependency_rho == TimeDependencyRho::Linear) ?
        (i_deposit-n_deposit+1)*sub_dt : (i_deposit-n_deposit+0.5_rt)*sub_dt;

```

线性依赖使用子区间端点，常量和二次依赖使用中点；二次依赖还会额外沉积一次，形成 old/mid/new 三个时间层：

```

if (time_dependency_J == TimeDependencyJ::Quadratic)
{
    PSATDMoveJNewToJMid();
    mypc->DepositCurrent( m_fields.get_mr_levels_alldirs(current_string, finest_level), dt[0], t_deposit_
    SyncCurrent("current_fp");

```

```
    PSATDForwardTransformJ("current_fp", "current_cp");
}
```

谱数组的 old/mid/new component 由 SpectralFieldIndex 分配:

```
if (time_dependency_J == TimeDependencyJ::Quadratic)
{
    Jx_old = c++; Jy_old = c++; Jz_old = c++;
    Jx_new = c++; Jy_new = c++; Jz_new = c++;
    Jx_mid = c++; Jy_mid = c++; Jz_mid = c++;
}
else if (time_dependency_J == TimeDependencyJ::Linear)
{
    Jx_old = c++; Jy_old = c++; Jz_old = c++;
    Jx_new = c++; Jy_new = c++; Jz_new = c++;
}
```

二阶 JRhom kernel 把这些时间层组合成多项式系数:

```
const Complex a_jx = (J_quadratic) ? (Jx_new - 2._rt * Jx_mid + Jx_old) : 0._rt;
const Complex b_jx = (J_linear || J_quadratic) ? (Jx_new - Jx_old) : 0._rt;
const Complex c_jx = (J_linear) ? (Jx_new + Jx_old)/2._rt : Jx_mid;

const Complex a_rho = (rho_quadratic) ? (rho_new - 2._rt * rho_mid + rho_old) : 0._rt;
const Complex b_rho = (rho_linear || rho_quadratic) ? (rho_new - rho_old) : 0._rt;
const Complex c_rho = (rho_linear) ? (rho_new + rho_old)/2._rt : rho_mid;
```

这对应每个子区间内

$$\tilde{\mathbf{J}}(t) = \mathbf{a}_J \tau^2 + \mathbf{b}_J \tau + \mathbf{c}_J, \quad \tilde{\rho}(t) = a_\rho \tau^2 + b_\rho \tau + c_\rho.$$

电场更新式以 Ex 为例:

```
fields(i,j,k,Idx.Ex) = C * Ex_old
    + I * c2 * S_ck * (ky * Bz_old - kz * By_old)
    + Y3 * a_jx + Y2 * b_jx - S_ck/ep0 * c_jx
    + I * c2 * kx * sum_rho;
```

这里 (ky*Bz-kz*By) 是谱空间 curl(B), Y3/Y2/S_ck 分别积分二次、一次和常量电流源项, sum_rho 则是电荷密度多项式带来的纵向修正。

磁场更新式同样含有 k x J 的多项式积分:

```
fields(i,j,k,Idx.Bx) = C * Bx_old
    - I * S_ck * (ky * Ez_old - kz * Ey_old)
    - I * Y1 * (ky * a_jz - kz * a_jy)
    + I * Y5 * (ky * b_jz - kz * b_jy)
    + I * Y4 * (ky * c_jz - kz * c_jy );
```

JRhom 的支持边界也必须写清楚: 源码禁止 Vay deposition 与 JRhom 组合, 默认关闭 JRhom current correction, 并禁止 Galilean PSATD:

```
if (current_deposition_algo == CurrentDepositionAlgo::Vay) {
    WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
        m_JRhom == false,
        "Vay deposition not implemented with JRhom algorithm");
}

if (m_JRhom) { current_correction = false; }
```

在 OneStep_JRhom() 中, 二次 J 会在子区间中点额外沉积, rho 也按 old/new/mid 时间层移动; 每个子区间随后进入谱推进。若开启 time averaging, 平均场会在该循环结束后缩放并反变换回实空间。

```

if (m_JRhom)
{
    WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
        v_galilean_is_zero,
        "PSATD-JRhom algorithm not implemented with Galilean PSATD"
    );
}

```

所以 JRhom 不是“普通 PSATD 上再加一个系数表”，而是重排了源项采样和谱推进：粒子先完整推进，源项在多个相对时刻沉积，谱场按 old/mid/new 保存源项时间层，最后在每个子区间用解析积分推进 E/B/F/G。

6.7.1 JRhom 的选择依据是源项时间模型

psatd.JRhom 的三个字符不是性能档位，而是一个时间模型声明：第一个字符给出 J 的常量、线性或二次近似，第二个字符给出 rho 的近似，末尾数字给出一个 PIC 步内的子区间数。例如 CL1 是常量电流、线性电荷、一个子区间；更高阶组合则要求源码在端点和中点沉积足够的源项层。

选择 JRhom 前应依次确认：问题是否确实需要在一个 PIC 大步内解析更丰富的源项时间依赖；输入的沉积算法是否允许这一分支；以及后续要观察 ordinary fields 还是 time-averaged fields。只有这三个问题都明确，Y1-Y8 才有物理语义。把它们当成“比 X1-X4 更高阶的一组数”会掩盖真正变化的对象，即源项采样与子区间推进顺序。

二阶 JRhom 中，Y1-Y5 积分 ordinary E/B 更新内的二次、一次和常量电流项，Y6-Y8 只在 time averaging 打开时累加平均场的源项贡献。它们都使用子区间步长 $\delta t = \Delta t/m$ ，不是外层完整 PIC 步长。这是阅读 PsatdAlgorithmJRhomSecondOrder.cpp 时最重要的尺度检查。

6.7.2 组合限制与验证边界

JRhom 改变了沉积和谱推进顺序，因此不是所有普通 PSATD 选项都可叠加。实现明确禁止 Vay deposition 与 JRhom 组合，也禁止 JRhom 与 Galilean PSATD 组合，并默认关闭 JRhom current correction。遇到某个组合不支持时，应把它理解为算法假设不兼容，而不是将参数强行拼成一个“更稳定”的方案。

验证也必须按时间模型分层。analysis_psatd_CC1.py 对特定 Cartesian JRhom CC1 case 提供 energy gate；RZ Langmuir CL4 可提供解析场 gate；而 checksum-only 的 RZ JRhom case 只能证明 workflow。RZ JRhom 的 finite + energy 正负对照是补充证据，不能替代上游 CMake 已注册的 analysis。读者需要先匹配 JRhom 字符串、geometry、rank 与 consumer，再决定某条通过结果的外推范围。

6.8 RZ PSATD: Hankel transform、azimuthal modes 与 Ep/Em

RZ 谱求解器位于 Source/FieldSolver/SpectralSolver/。RZ PSATD 不能理解成“二维 PSATD”。它使用 azimuthal mode decomposition：

$$F(r, z, \theta) = \sum_m \Re(F_m(r, z)e^{im\theta}),$$

并且每个实空间 MultiFab 的 component 数为

$$n_{\text{comps}} = 2n_{\text{modes}} - 1.$$

源码中：

```

utils::parser::queryWithParser(pp_warpX, "n_rz_azimuthal_modes", n_rz_azimuthal_modes);
WARPX_ALWAYS_ASSERT_WITH_MESSAGE( n_rz_azimuthal_modes > 0,
    "The number of azimuthal modes (n_rz_azimuthal_modes) must be at least 1");
ncomps = n_rz_azimuthal_modes*2 - 1;
RZ spectral solver 入口选择标准、Galilean 或 PML 算法：
if (with_pml) {
    PML_algorithm = std::make_unique<PsatdAlgorithmPmlRZ>(
        k_space, dm, m_spectral_index, n_rz_azimuthal_modes, norder_z, grid_type, dt);
}
if (v_galilean[2] == 0) {

```

```

    algorithm = std::make_unique<PsatdAlgorithmRZ>(
        k_space, dm, m_spectral_index, n_rz_azimuthal_modes, norder_z, grid_type, dt,
        update_with_rho, fft_do_time_averaging, time_dependency_J, time_dependency_rho, dive_cleaning, dive_cleaning_rho,
    } else {
        algorithm = std::make_unique<PsatdAlgorithmGalileanRZ>(
            k_space, dm, m_spectral_index, n_rz_azimuthal_modes, norder_z, grid_type, v_galilean, dt, update_w
    }

```

RZ 的 k_z 来自 z 向 FFT, 而 k_r 来自径向 Hankel transform 的 Bessel roots. SpectralKSpaceRZ 只构造 z 向 k :

```

const int i_dim = 1;
const bool only_positive_k = false;
k_vec[i_dim] = getKComponent(dm, realspace_ba, i_dim, only_positive_k);

```

Hankel transformer 为每个 mode 建立三套 transform:

```

for (int mode=0 ; mode < m_n_rz_azimuthal_modes ; mode++) {
    dht0[mode] = std::make_unique<HankelTransform>(mode, mode, m_nr, rmax);
    dhtp[mode] = std::make_unique<HankelTransform>(mode+1, mode, m_nr, rmax);
    dhtm[mode] = std::make_unique<HankelTransform>(mode-1, mode, m_nr, rmax);
}

```

标量用 dht0. 横向矢量场先组合为

$$F_p = \frac{F_r - iF_\theta}{2}, \quad F_m = \frac{F_r + iF_\theta}{2},$$

源码中对应:

```

// temp_p = (F_r - I*F_t)/2
// temp_m = (F_r + I*F_t)/2
F_r_physical_array(i,j,k,mode_r) = 0.5_rt*(r_real + t_imag);
F_r_physical_array(i,j,k,mode_i) = 0.5_rt*(r_imag - t_real);
F_t_physical_array(i,j,k,mode_r) = 0.5_rt*(r_real - t_imag);
F_t_physical_array(i,j,k,mode_i) = 0.5_rt*(r_imag + t_real);

```

然后分别做 dhtp/dhtm:

```

dhtp[mode]->HankelForwardTransform(F_r_physical, mode_r, G_p_spectral, mode_r);
dhtm[mode]->HankelForwardTransform(F_t_physical, mode_r, G_m_spectral, mode_r);

```

所以 PsatdAlgorithmRZ 中的 E_p/E_m 不是 E_x/E_y , 而是由 E_r/E_θ 组合出的谱分量:

```

int const Ep_m = Idx.Ex + Idx.n_fields*mode;
int const Em_m = Idx.Ey + Idx.n_fields*mode;
int const Ez_m = Idx.Ez + Idx.n_fields*mode;
int const Bp_m = Idx.Bx + Idx.n_fields*mode;
int const Bm_m = Idx.By + Idx.n_fields*mode;
int const Bz_m = Idx.Bz + Idx.n_fields*mode;

```

RZ PSATD 的电场更新以 $E_p/E_m/E_z$ 为变量:

```

fields(i,j,k,Ep_m) = C*Ep_old
    + S_ck*(-c2*I*kr/2._rt*Bz_old + c2*kz*Bp_old - inv_ep0*Jp)
    + 0.5_rt*kr*rho_diff;
fields(i,j,k,Em_m) = C*Em_old
    + S_ck*(-c2*I*kr/2._rt*Bz_old - c2*kz*Bm_old - inv_ep0*Jm)
    - 0.5_rt*kr*rho_diff;
fields(i,j,k,Ez_m) = C*Ez_old
    + S_ck*(c2*I*kr*Bp_old + c2*I*kr*Bm_old - inv_ep0*Jz)
    - I*kz*rho_diff;

```

RZ 的谱散度写成

$$\nabla \cdot \mathbf{E} \rightarrow k_r(E_p - E_m) + ik_z E_z.$$

源码中 `update_with_rho=0` 时用它重构电荷项:

```
Complex const divE = kr*(Ep_old - Em_old) + I*kz*Ez_old;
Complex const divJ = kr*(Jp - Jm) + I*kz*Jz;
```

```
rho_diff = (X2 - X3)*PhysConst::epsilon_0*divE - X2*dt*divJ;
```

RZ current correction 也沿这个谱散度结构投影:

```
Complex const F = - ((rho_new - rho_old)/dt + I*kz*Jz + kr*(Jp - Jm))/k_norm2;
```

```
fields(i,j,k,Jp_m) += +0.5_rt*kr*F;
fields(i,j,k,Jm_m) += -0.5_rt*kr*F;
fields(i,j,k,Jz_m) += -I*kz*F;
```

反变换后, RZ 还要按 mode 对称性填充轴下 guard cells:

```
if (i < 0) {
    ii = -i - 1;
    if (icomp == 0) {
        // Mode zero is symmetric
        sign = +1._rt;
    } else {
        // Odd modes are anti-symmetric
        const auto imode = (icomp + 1)/2;
        sign = static_cast<amrex::Real>(std::pow(-1._rt, imode));
    }
}
```

这说明 RZ PSATD 的“正确性”同时依赖三件事: Hankel/Bessel 谱基、Ep/Em 横向矢量代数、以及轴上/轴下 mode 对称性。把 Cartesian PSATD 的 `kx, ky, kz` 公式机械删去一个方向, 得不到 WarpX 的 RZ 实现。

6.8.1 RZ 中先认清字段表示, 再读系数

RZ 的关键不是把三维 Cartesian 公式少写一个方向, 而是先把 (r, θ) 横向矢量改写为 p/m 组合, 再在每个 azimuthal mode 上使用 Hankel 变换。Ep/Em、Bp/Bm 和 Jp/Jm 因而是由物理横向分量组合得到的谱变量, 不是 Ex/Ey 的别名。

这会直接改变散度、current correction 和电荷项的写法: RZ 谱散度包含 $k_r(E_p - E_m) + ik_z E_z$, 修正电流也沿 $J_p - J_m$ 与 J_z 的组合投影。标准 RZ、Galilean RZ 和 RZ PML 可以共享基础的振荡思想, 却不能直接复用 Cartesian X1-X4 的更新式或验证结论。轴下 guard-cell 的 mode 对称性又是反变换后独立的一层条件。

对读者而言, 一个实用检查顺序是: 先看 `n_rz_azimuthal_modes` 与 `fields` 的 mode 组件数; 再定位 `PsatdAlgorithmRZ`、`PsatdAlgorithmGalileanRZ` 或 `PsatdAlgorithmPmlRZ` 中哪一类实际被构造; 最后才对照该类自己的 `rho/J` 时间模型和 `analysis`。这样不会把同名 X、Y 或同样叫 current correction 的路径误认成同一算法。

6.8.2 Comoving 与 Galilean 都有相位, 但问题不同

Galilean PSATD 选择的是随背景漂移移动的网格表示; comoving PSATD 则是 regular-domain 上另一套具有独立相位、波数分工和组合限制的算法。两者不能仅因都出现 Θ_2 或移动速度就互换。

comoving 分支要求 direct current deposition 与 `psatd.update_with_rho=1`, 并与 Esirkepov、Villasenor、Vay 以及 JRhom 组合受限。它的 C/S_ck 用有限阶 modified wave number, comoving 相位却使用另一组波数与速度; 这种双波数分工必须保留到解释退化极限和 current correction 时。`psatd.use_default_v_comoving` 还依赖 `warpx.gamma_boost`, 输入层的归一化速度会在算法内转换为 SI 速度。

因此读者不应把 comoving case 的默认 checksum 当作 Galilean NCI gate 的替代物。当前 comoving regression 的强度取决于它实际接入的 consumer; 它可以确认分支选择、输出与基线一致性, 却不能在缺少对应 energy/charge analysis 时宣布与 `analysis_galilean.py` 等价。

6.8.3 用算法选择表约束结论范围

需要回答的问题	优先确认的算法层	代表性证据
真空场如何在网格上推进? 源项在一个时间步内怎样近似?	FDTD stencil 或 Fourier/Hankel 谱基 ordinary、Galilean/comoving 或 JRhom 时间模型	解析场、色散或 PML residual analysis rho/J 时间层、算法类构造与兼容断言
连续性误差怎样处理?	current correction 的具体 geometry 分支	divE-rho/eps0 consumer, 而非参数名
漂移 NCI 如何压制? RZ 结果能否外推?	filter、表示、插值和时间步的组合 modes、axis symmetry、Ep/Em layout	NCI energy gate 与对应输入卡 RZ Langmuir、RZ Galilean 或 RZ PML 的同类 consumer

本章的结论不是“PSATD 比 FDTD 更好”，而是：不同谱基、源项时间模型与沉积/同步约束构成不同算法族，必须由对应 observable 验证。RZ Langmuir、Galilean NCI 和 PML residual-field analysis 能对各自 family 提供强断言；checksum 与文献 benchmark 则分别只承担 workflow 和理论背景的责任。读者应把公式、源码与运行检查分层使用，而不是把任何一层误当成完整证明。

6.9 静电与静磁求解器

WarpX 的 electrostatic 路径不再用 Maxwell curl 方程推进场，而是在每一步从当前粒子/流体源项重新解椭圆方程。最基本的 lab-frame 静电模式是

$$\nabla^2 \phi = -\rho/\epsilon_0, \quad \mathbf{E} = -\nabla \phi.$$

如果启用 labframe-electromagnetostatic，还会解磁矢势：

$$\nabla^2 \mathbf{A} = -\mu_0 \mathbf{J}, \quad \mathbf{B} = \nabla \times \mathbf{A}.$$

如果启用 relativistic，WarpX 对每个 species 用平均速度 $\beta = \langle \mathbf{v} \rangle / c$ 解修正 Poisson 方程：

$$[\nabla^2 - (\beta \cdot \nabla)^2] \phi = -\rho/\epsilon_0,$$

并按

$$\mathbf{E} = -\nabla \phi + \beta(\beta \cdot \nabla \phi), \quad \mathbf{B} = -\frac{1}{c} \beta \times \nabla \phi$$

重建场。这个模式不能把所有 species 的电荷先相加，因为不同 species 的平均速度不同。

参数入口在 WarpX::ReadParameters()。一旦 warpx.do_electrostatic 不是 none，Maxwell solver 会被关闭：

```
pp_warpx.query_enum_sloppy("do_electrostatic", electrostatic_solver_id, "-");
// if an electrostatic solver is used, set the Maxwell solver to None
if (electrostatic_solver_id != ElectrostaticSolverAlgo::None) {
    electromagnetic_solver_id = ElectromagneticSolverAlgo::None;
}
```

这句代码是模型边界：静电 PIC 不传播光波，也不描述激光/辐射传播。它用瞬时 Poisson 解代替全 Maxwell 更新。

solver 对象在 WarpX::WarpX() 构造期选择：

```
if ((WarpX::electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrame)
    || (WarpX::electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameElectroMagnetostatic))
{
    m_electrostatic_solver = std::make_unique<LabFrameExplicitES>(nlevs_max);
}
else if (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameEffectivePotential)
```

```

{
    m_electrostatic_solver = std::make_unique<EffectivePotentialES>(nlevs_max);
}
else
{
    m_electrostatic_solver = std::make_unique<RelativisticExplicitES>(nlevs_max);
}

```

每步入口是 WarpX::ComputeSpaceChargeField()。如果 reset_fields=true, E/B 先被清零, 再由静电/静磁求解器把自洽场加回:

```

if (reset_fields) {
    // Reset all E and B fields to 0, before calculating space-charge fields
    ABLASTR_PROFILE("WarpX::ComputeSpaceChargeField::reset_fields");
    for (int lev = 0; lev <= max_level; lev++) {
        for (int comp=0; comp<3; comp++) {
            m_fields.get(FieldType::Efield_fp, Direction{comp}, lev)->setVal(0);
            m_fields.get(FieldType::Bfield_fp, Direction{comp}, lev)->setVal(0);
        }
    }
}

```

```

m_electrostatic_solver->ComputeSpaceChargeField(
    m_fields, *mypc, myfl.get(), max_level );

```

6.9.1 Poisson 边界条件

PoissonBoundaryHandler 读取 boundary.potential_lo_x/hi_x/... 和 warpx.eb_potential(x,y,z,t), 再把 field boundary 转成 AMReX linear operator boundary。Multigrid 支持 periodic、PEC/Dirichlet、Neumann; open/PML 会被拒绝:

```

WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    (WarpX::field_boundary_lo[idim] != FieldBoundaryType::Open &&
     WarpX::field_boundary_hi[idim] != FieldBoundaryType::Open &&
     WarpX::field_boundary_lo[idim] != FieldBoundaryType::PML &&
     WarpX::field_boundary_hi[idim] != FieldBoundaryType::PML) ,
    "Open and PML field boundary conditions only work with "
    "warpx.poisson_solver = fft."
);

```

Dirichlet 电势由 setPhiBC() 写入 nodal phi 的物理边界:

```

if (dirichlet_flag[2*idim] && iv[idim] == domain.smallEnd(idim)){
    phi_arr(i,j,k) = phi_bc_values_lo[idim];
}
if (dirichlet_flag[2*idim+1] && iv[idim] == domain.bigEnd(idim)) {
    phi_arr(i,j,k) = phi_bc_values_hi[idim];
}

```

因此边界电势是解空间上的约束, 而不是加到 ρ 的体源项。

6.9.2 Lab-frame 静电

LabFrameExplicitES::ComputeSpaceChargeField() 先取出 rho_fp/rho_cp/phi_fp/Efield_fp, 把所有粒子电荷和流体电荷沉积到总 rho:

```

const MultiLevelScalarField rho_fp = fields.get_mr_levels(FieldType::rho_fp, max_level);
const MultiLevelScalarField rho_cp = fields.get_mr_levels(FieldType::rho_cp, max_level, skip_lev0_coarse_p);
const MultiLevelScalarField phi_fp = fields.get_mr_levels(FieldType::phi_fp, max_level);
const MultiLevelVectorField Efield_fp = fields.get_mr_levels_alldirs(FieldType::Efield_fp, max_level);

mpc.DepositCharge(rho_fp, 0.0_rt);

```



```

if (mfl) {
    const int lev = 0;
    mfl->DepositCharge(fields, *rho_fp[lev], lev);
}

```

随后同步电荷密度:

```

const Vector<std::unique_ptr<MultiFab> > rho_buf(num_levels);
auto & warpx = WarpX::GetInstance();
warpx.SyncRho( rho_fp, rho_cp, amrex::GetVecOfPtrs(rho_buf) );

```

lab-frame 中 $\beta=0$, 所以 `computeE()` 退化为普通的 $E=-\text{grad}(\phi)$:

```

const std::array<Real, 3> beta = {0._rt};

setPhiBC(phi_fp, warpx.gett_new(0));

computePhi(rho_fp, phi_fp, beta, self_fields_required_precision,
           self_fields_absolute_tolerance, self_fields_max_iters,
           self_fields_verbosity, is_igf_2d_slices, Efield_fp);

```

6.9.3 Relativistic self fields

Relativistic solver 对每个 species 分别求自场。核心差异是先沉积单 species 电荷, 再用该 species 的全局平均速度设置 β :

```

bool const local_average = false; // Average across all MPI ranks
std::array<ParticleReal, 3> beta_pr = pc.meanParticleVelocity(local_average);
std::array<Real, 3> beta;
for (int i=0 ; i < static_cast<int>(beta.size()) ; i++) {
    beta[i] = beta_pr[i]/PhysConst::c; // Normalize
}

```

然后用 species 自己的 self-field solver 参数求势并加场:

```

computePhi( amrex::GetVecOfPtrs(rho), amrex::GetVecOfPtrs(phi),
            beta, pc.self_fields_required_precision,
            pc.self_fields_absolute_tolerance, pc.self_fields_max_iters,
            pc.self_fields_verbosity, is_igf_2d_slices);

```

```

computeE( Efield_fp, amrex::GetVecOfPtrs(phi), beta );
computeB( Bfield_fp, amrex::GetVecOfPtrs(phi), beta );

```

`computeE()` 的 nodal 3D Ex 代码正是 $(\beta_i\beta_j - \delta_{ij})\partial_j\phi$:

```

Ex_arr(i,j,k) +=
    (beta_x*beta_x-1._rt)*0.5_rt*inv_dx*(phi_arr(i+1,j ,k )-phi_arr(i-1,j ,k ))
    + beta_x*beta_y      *0.5_rt*inv_dy*(phi_arr(i ,j+1,k )-phi_arr(i ,j-1,k ))
    + beta_x*beta_z      *0.5_rt*inv_dz*(phi_arr(i ,j ,k+1)-phi_arr(i ,j ,k-1));

```

`computeB()` 在 $\beta=0$ 时立即返回; 否则按 $-\beta \times \nabla\phi/c$ 生成磁场:

```

if ((beta[0] == 0._rt) && (beta[1] == 0._rt) && (beta[2] == 0._rt)) { return; }

Bx_arr(i,j,k) += PhysConst::inv_c * (
    -beta_y*inv_dz*0.5_rt*(phi_arr(i,j ,k+1)-phi_arr(i,j ,k-1))
    +beta_z*inv_dy*0.5_rt*(phi_arr(i,j+1,k )-phi_arr(i,j-1,k )));

```

6.9.4 Effective potential

Effective potential solver 把 Poisson 方程改为 variable-coefficient elliptic solve. 它先分配 cell-centered `effective_potential_sigma`:

```

fields.alloc_init(
    warpx::fields::FieldType::effective_potential_sigma, /*level=*/ 0,
    convert(rho->boxArray(), IntVect(AMREX_D_DECL(0,0,0))),

```

```
rho->DistributionMap(), 1, IntVect(AMREX_D_DECL(0,0,0)), 1.0_rt
);
```

ComputeSigma() 中的核心因子是

$$\frac{C_{EP}}{4} \omega_{ps}^2 \Delta t^2 = \frac{C_{EP}}{4} \frac{q_s n_s}{m_s \epsilon_0} \Delta t^2.$$

源码写作:

```
auto mult_factor = (
    C_SI * warpx.getdt(lev) * warpx.getdt(lev) / (4._rt * PhysConst::epsilon_0)
);
```

每个 species 对 sigma 的贡献为:

```
auto const q = std::abs(pc->getCharge());
auto const mult_factor_pc = mult_factor * q / pc->getMass();
```

```
sigma_arr(i, j, k, 0) += time_filter_param * mult_factor_pc * rho_cc;
```

最后调用专门的 variable-coefficient solver:

```
ablastr::fields::computeEffectivePotentialPhi(
    sorted_rho,
    sorted_phi,
    *sigma,
    required_precision,
    absolute_tolerance,
    max_iters,
    verbosity,
    warpx.Geom(),
    warpx.DistributionMap(),
    warpx.boxArray(),
    WarpX::grid_type,
    false,
    EB::enabled(),
    WarpX::do_single_precision_comms,
    warpx.refRatio(),
    post_phi_calculation,
    *m_poisson_boundary_handler,
    warpx.gett_new(0),
    eb_farray_box_factory
);
```

6.9.5 Magnetostatic vector Poisson

labframe-electromagnetostatic 的磁场不是 Maxwell curl 更新, 而是解 vector Poisson 后取 curl。入口明确要求无 mesh refinement:

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(this->max_level == 0,
    "Magnetostatic solver not implemented with mesh refinement.");
```

```
AddMagnetostaticFieldLabFrame();
```

它先清零并沉积所有 species 的电流:

```
for (int lev = 0; lev <= max_level; lev++) {
    for (int dim=0; dim < 3; dim++) {
        m_fields.get(FieldType::current_fp, Direction{dim}, lev)->setVal(0.);
    }
}
```

```
}
```

```
for (int ispecies=0; ispecies<mypc->nSpecies(); ispecies++){
    WarpXParticleContainer& species = mypc->GetParticleContainer(ispecies);
    if (!species.do_not_deposit) {
        species.DepositCurrent(
            m_fields.get_mr_levels_alldirs(FieldType::current_fp, finest_level),
            dt[0], 0.);
    }
}
```

再同步电流，设置矢势边界，调用 vector Poisson solver:

```
SyncCurrent("current_fp");

setVectorPotentialBC(m_fields.get_mr_levels_alldirs(FieldType::vector_potential_fp_nodal, finest_level));

computeVectorPotential(
    m_fields.get_mr_levels_alldirs(FieldType::current_fp, finest_level),
    m_fields.get_mr_levels_alldirs(FieldType::vector_potential_fp_nodal, finest_level),
    magnetostatic_solver_required_precision, magnetostatic_solver_absolute_tolerance,
    magnetostatic_solver_max_iters, magnetostatic_solver_verbosity
);
```

矢势的 PEC 边界条件按分量区分：法向 A_n 用 Neumann，切向 A_t 用 Dirichlet:

```
if ( WarpX::field_boundary_lo[idim] == FieldBoundaryType::PEC ) {
    if (ndotA) {
        lcbc[adim][idim] = LinOpBCType::Neumann;
        dirichlet_flag[adim][idim*2] = false;
    } else {
        lcbc[adim][idim] = LinOpBCType::Dirichlet;
        dirichlet_flag[adim][idim*2] = true;
    }
}
```

Poisson solve 后, EBCalcBfromVectorPotentialPerLevel::operator() 从 MLMG 取每个 A 分量的梯度，再按 curl 组合成 B。例如 A_x 对 B_y/B_z 的贡献:

```
mlmg[0]->getGradSolution({buf_ptr});

// Interpolate dAx/dz to By grid buffer, then add to By
this->doInterp(*m_grad_buf_e_stag[lev][2],
              *m_grad_buf_b_stag[lev][1]);
MultiFab::Add(*m_b_field[lev][1], *(m_grad_buf_b_stag[lev][1]), 0, 0, 1, 0 );

// Interpolate dAx/dy to Bz grid buffer, then subtract from Bz
this->doInterp(*m_grad_buf_e_stag[lev][1],
              *m_grad_buf_b_stag[lev][2]);
m_grad_buf_b_stag[lev][2]->mult(-1._rt);
MultiFab::Add(*m_b_field[lev][2], *(m_grad_buf_b_stag[lev][2]), 0, 0, 1, 0 );
```

逐项合起来就是

$$B_x = \partial_y A_z - \partial_z A_y, \quad B_y = \partial_z A_x - \partial_x A_z, \quad B_z = \partial_x A_y - \partial_y A_x.$$

因此这一路径的正确性依赖三个环节同时一致：电流沉积/同步给出正确 J，vector Poisson 解出符合边界条件的 A，post callback 再把 curl A 插值到 Bfield_fp 的实际 staggering。

6.10 Hybrid PIC: 广义 Ohm 定律与 B 场 RK 子步

Source/FieldSolver/FiniteDifferenceSolver/HybridPICModel/HybridPICModel.H 组织 WarpX 的 kinetic-fluid hybrid solver。这个路径不使用 Maxwell-Ampere 方程推进电场, 而是把电子视为流体、离子仍作为 kinetic particles, 用广义 Ohm 定律求电场:

$$\mathbf{E} = -\frac{1}{en_e} (\mathbf{J}_e \times \mathbf{B} + \nabla P_e) + \eta \mathbf{J} - \eta_h \nabla^2 \mathbf{J}.$$

其中准中性假设给出 $\rho = e n_e$, 总电流由忽略位移电流后的 Ampere 定律给出:

$$\mu_0 \mathbf{J} = \nabla \times \mathbf{B}.$$

电子电流不直接沉积, 而是由

$$\mathbf{J}_e = \mathbf{J} - \mathbf{J}_i - \mathbf{J}_{\text{ext}}$$

得到。源码里的 Jfield 已经在 CalculatePlasmaCurrent() 阶段扣除了外部电流, 所以 Ohm kernel 中只显式出现 $\mathbf{J} - \mathbf{J}_i$ 。

6.10.1 参数与辅助场

HybridPICModel 保存 hybrid solver 的所有模型参数和辅助场接口:

```
class HybridPICModel
{
public:
    HybridPICModel ();
    void ReadParameters ();
    void AllocateLevelMFs (...) const;
    void InitData (const ablastr::fields::MultiFabRegister& fields);
    void GetCurrentExternal ();
    void CalculatePlasmaCurrent (...) const;
    void HybridPICSolveE (...) const;
    void BfieldEvolveRK (...);
    void FieldPush (...);
    void CalculateElectronPressure () const;
```

它要求用户指定电子温度, 并在非等温多方闭合时要求 n0_ref:

```
utils::parser::queryWithParser(pp_hybrid, "gamma", m_gamma);
if (!utils::parser::queryWithParser(pp_hybrid, "elec_temp", m_elec_temp)) {
    Abort("hybrid_pic_model.elec_temp must be specified when using the hybrid solver");
}
const bool n0_ref_given = utils::parser::queryWithParser(pp_hybrid, "n0_ref", m_n0_ref);
if (m_gamma != 1.0 && !n0_ref_given) {
    Abort("hybrid_pic_model.n0_ref should be specified if hybrid_pic_model.gamma != 1");
}
```

电子温度从 eV 转成 J 后参与压力计算:

```
// convert electron temperature from eV to J
m_elec_temp *= PhysConst::q_e;
```

Hybrid PIC 需要额外场来存储电子压强、时间插值用的 ρ/J_i 、Ampere 电流和外部电流:

```
fields.alloc_init(FieldType::hybrid_electron_pressure_fp,
    lev, amrex::convert(ba, rho_nodal_flag),
    dm, ncomps, ngRho, 0.0_rt);
```

```
fields.alloc_init(FieldType::hybrid_rho_fp_temp,
```

```

    lev, amrex::convert(ba, rho_nodal_flag),
    dm, ncomps, ngRho, 0.0_rt);

fields.alloc_init(FieldType::hybrid_current_fp_plasma, Direction{0},
    lev, amrex::convert(ba, jx_nodal_flag),
    dm, ncomps, ngJ, 0.0_rt);

InitData() 还会记录 J/B/E 的 index type。这个细节决定 HybridPICSolveE.cpp 里每个物理量如何从自身 staggering 插值到 Ex/Ey/Ez 的位置:

amrex::IntVect Jx_stag = fields.get(FieldType::current_fp, Direction{0}, 0)->ixType().toIntVect();
amrex::IntVect Bx_stag = fields.get(FieldType::Bfield_fp, Direction{0}, 0)->ixType().toIntVect();
amrex::IntVect Ex_stag = fields.get(FieldType::Efield_fp, Direction{0}, 0)->ixType().toIntVect();

Jx_IndexType[idim] = Jx_stag[idim];
Bx_IndexType[idim] = Bx_stag[idim];
Ex_IndexType[idim] = Ex_stag[idim];

```

6.10.2 顶层 field update

Hybrid field update 的主入口是 WarpX::HybridPICEvolveFields()。它目前硬性限制为单 level:

```

WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    finest_level == 0,
    "Ohm's law E-solve only works with a single level.");

```

进入这个函数时, 粒子已经被推到 x^{n+1} 。接着沉积 ρ^{n+1} 和 $J_i^{n+1/2}$:

```

// The particles have now been pushed to their t_{n+1} positions.
// Perform charge deposition at t_{n+1} and current deposition at t_{n+1/2}.
HybridPICDepositRhoAndJ();

// Get the external current
m_hybrid_pic_model->GetCurrentExternal();

```

因为上一步末尾保存了 ρ^n 和 $J_i^{n-1/2}$, 源码可以构造中间时间层。 J_i^n 由两侧半步平均得到:

```

MultiFab::LinComb(
    *current_fp_temp[lev][idim],
    0.5_rt, *current_fp_temp[lev][idim], 0,
    0.5_rt, *m_fields.get(FieldType::current_fp, Direction{idim}, lev), 0,
    0, 1, current_fp_temp[lev][idim]->nGrowVect()
);

```

$\rho^{n+1/2}$ 同样由 ρ^n 和 ρ^{n+1} 平均:

```

MultiFab::LinComb(
    *rho_fp_temp[lev], 0.5_rt, *rho_fp_temp[lev], 0,
    0.5_rt, *m_fields.get(FieldType::rho_fp, lev), 0, 0, 1,
    rho_fp_temp[lev]->nGrowVect()
);

```

第一个半步用 E^n 推 $B^n \rightarrow B^{n+1/2}$, 第二个半步用 $E^{n+1/2}$ 推 $B^{n+1/2} \rightarrow B^{n+1}$ 。最后, 为了得到 E^{n+1} , 代码把 ion current 外推到 $n+1$:

```

MultiFab::LinComb(
    *current_fp_temp[lev][idim],
    -1._rt, *current_fp_temp[lev][idim], 0,
    2._rt, *m_fields.get(FieldType::current_fp, Direction{idim}, lev), 0,
    0, 1, current_fp_temp[lev][idim]->nGrowVect()
);

```

由于此时 $\text{current_fp_temp} = J_i^n = 0.5(J_i^{n-1/2} + J_i^{n+1/2})$, 上式就是

$$J_i^{n+1} = \frac{3}{2}J_i^{n+1/2} - \frac{1}{2}J_i^{n-1/2}.$$

6.10.3 Ampere 电流与 Ohm kernel

每个 B 场 RK stage 都通过 FieldPush() 闭合一次 B → J → E → B:

```
// Calculate J = curl x B / mu0 - J_ext
CalculatePlasmaCurrent(Bfield, eb_update_E);
// Calculate the E-field from Ohm's law
HybridPICSolveE(Efield, Jfield, Bfield, rhofield, eb_update_E, true);

// Push forward the B-field using Faraday's law
warpx.EvolveB(dt, subcycling_half, t_old);
warpx.FillBoundaryB(ng, nodal_sync);

CalculatePlasmaCurrent() 先用 finite-difference solver 从 B 计算 Ampere 电流, 再减去外部电流:
warpx.get_pointer_fdt_solver_fp(lev)→CalculateCurrentAmpere(
    current_fp_plasma, Bfield, eb_update_E, lev
);

if (m_has_external_current) {
    ablastr::fields::VectorField current_fp_external =
        warpx.m_fields.get_alldirs(FieldType::hybrid_current_fp_external, lev);
    for (int i=0; i<3; i++) {
        current_fp_plasma[i]→minus(*current_fp_external[i], 0, 1, 1);
    }
}
```

HybridPICSolveECartesian() 先把 J、J_i 和 B 插值到 nodal grid, 并计算 Hall 项:

```
// calculate enE = (J - J_i) x B
enE_nodal(i, j, k, 0) = (
    (jy_interp - jiy_interp) * Bz_interp
    - (jz_interp - jiz_interp) * By_interp
);
enE_nodal(i, j, k, 1) = (
    (jz_interp - jiz_interp) * Bx_interp
    - (jx_interp - jix_interp) * Bz_interp
);
enE_nodal(i, j, k, 2) = (
    (jx_interp - jix_interp) * By_interp
    - (jy_interp - jiy_interp) * Bx_interp
);
```

随后在每个 E 分量所在的 grid staggering 上除以 rho, 加入压力梯度、阻性和超阻性项。以 Ex 为例:

```
const Real rho_val = Interp(rho, nodal, Ex_stag, coarsen, i, j, k, 0);

if (rho_val < rho_floor && holmstrom_vacuum_region) {
    Ex(i, j, k) = 0._rt;
} else {
    const Real grad_Pe = (!solve_for_Faraday) ?
        T_Algo::UpwardDx(Pe, coefs_x, n_coefs_x, i, j, k)
        : 0._rt;

    const auto enE_x = Interp(enE, nodal, Ex_stag, coarsen, i, j, k, 0);
```

```

    const auto rho_val_limited = std::max(rho_val, rho_floor);

    Ex(i, j, k) = (enE_x - grad_Pe) / rho_val_limited;
}

```

solve_for_Faraday 是一个重要分支: 如果这个 E 只用于更新 B, 则 $\text{curl}(\text{grad } Pe)=0$, 压力梯度对 Faraday 更新没有贡献, 所以源码跳过它; 如果最终要求输出 E^{n+1} , 才把纵向压力项加回。

阻性和超阻性只在 solve_for_Faraday=true 时加入:

```

Ex(i, j, k) += eta(rho_val, jtot_val) * Jx(i, j, k);

auto nabla2Jx = T_Algo::Dxx(Jx, coefs_x, n_coefs_x, i, j, k)
    + T_Algo::Dyy(Jx, coefs_y, n_coefs_y, i, j, k)
    + T_Algo::Dzz(Jx, coefs_z, n_coefs_z, i, j, k);

Ex(i, j, k) -= eta_h(rho_val, btot_val) * nabla2Jx;

```

因此源码完整对应

$$\eta \mathbf{J} - \eta_h \nabla^2 \mathbf{J}.$$

6.10.4 电子压力与外部矢势 split field

电子压力闭合是多形式:

```

static amrex::Real get_pressure (amrex::Real const n0,
                                amrex::Real const T0,
                                amrex::Real const gamma,
                                amrex::Real const rho) {
    return n0 * T0 * std::pow((rho/PhysConst::q_e)/n0, gamma);
}

```

也就是

$$P_e = n_0 T_{e0} \left(\frac{n_e}{n_0} \right)^\gamma.$$

外部矢势由 ExternalVectorPotential 管理。用户提供空间矢势 $\mathbf{A}(x, y, z)$ 和时间函数 $s(t)$, 代码从中构造

$$\mathbf{B}_{\text{ext}} = s(t) \nabla \times \mathbf{A}, \quad \mathbf{E}_{\text{ext}} = -\frac{ds}{dt} \mathbf{A}.$$

源码用中心差分计算时间因子:

```

const amrex::Real scale_factor_B = m_A_time_scale[i](t);

const amrex::Real sf_l = m_A_time_scale[i](t-0.5_rt*dt);
const amrex::Real sf_r = m_A_time_scale[i](t+0.5_rt*dt);
const amrex::Real scale_factor_E = -(sf_r - sf_l)/dt;

```

然后累加到 hybrid 外部场:

```

AddExternalFieldFromVectorPotential(E_ext[lev], scale_factor_E, A_ext[lev],
    warpx.GetEBUpdateEFlag()[lev]);
AddExternalFieldFromVectorPotential(B_ext[lev], scale_factor_B, curlA_ext[lev],
    warpx.GetEBUpdateBFlag()[lev]);

```

在 HybridPICEvolveFields() 中, 若启用 split external fields, 推进前先从总 B 中减去外部 B, 结束时再把外部 E/B 加回。这一点防止 Ohm solver 把外部驱动场误当成自洽 plasma response。

这一路径的实现边界也很明确：目前 field solve 只支持单 level；RZ Ohm solver 只支持 $m=0$ ；`n_floor` 是除以密度时的硬下限；`holmstrom_vacuum_region` 会在低密度区置零 E 以抑制真空区波动。Hybrid PIC 的正确性不能只检查 `HybridPICsolveE.cpp`，还必须同时检查沉积时间层、Ampere current、RK 子步、外部场分裂和边界填充是否一致。

6.10.5 Fluids/ 与 HybridPICModel 不是同一条流体链

这里有一个很容易混淆的边界：WarpX 当前 worktree 里同时存在

- `Source/Fluids/*`
- `FieldSolver/FiniteDifferenceSolver/HybridPICModel/*`

它们都带有“fluid”语义，但职责完全不同。

`HybridPICModel` 是 field solver 内部的电子流体闭合。它不维护一套独立的 species state，也不通过 `WarpXFluidContainer` 推进电子。它做的是：

1. 从 $\text{curl } B / \mu_0$ 得到总 plasma current；
2. 扣掉外部电流和 kinetic ion current；
3. 用剩下的电子流体电流加上电子压强闭合广义 Ohm 定律，求出 E ；
4. 再用 Faraday 定律和 RK 子步推进 B 。

而 `Fluids/` 里的 `MultiFluidContainer` -> `WarpXFluidContainer` 则是另一条 runtime layer。它真正维护的是每个 fluid species 的 nodal

$$(N, NU_x, NU_y, NU_z),$$

并在每个 PIC step 里执行：

1. 从 `Efield_aux/Bfield_aux` gather 主场；
2. 复用粒子侧 `UpdateMomentumHigueraCary(...)` 加 Lorentz source；
3. 用 `AdvectivePush_Muscl()` 做 cold-fluid 守恒更新；
4. 把 qN 和 qNU/γ 再沉积回普通 `rho_fp/current_fp`。

所以 `Fluids/` 更像“额外 cold-fluid species 参与普通场沉积”，而不是“hybrid solver 的电子闭合实现”。这也是为什么：

- `WarpX.cpp` 里 `do_fluid_species` 和 `electromagnetic_solver_id == HybridPIC` 是两套独立 existence gate；
- `Fluids/` 会在 moving window 下整体平移并对新暴露 nodal box 重新 `InitData()`；
- 最直接的 validation 入口是 `langmuir_fluids` 这类 cold-fluid regression，而不是 `ohm_solver_*`。

6.11 FieldSolver regression 判据：从 analysis 脚本反读物理检查量

前面各节解释了场求解器的离散方程和源码路径。还需要回答一个实际问题：WarpX 自己怎样判断这些 solver 没有坏掉？答案不只在 `CMakeLists.txt` 里。`Examples/Tests/*/CMakeLists.txt` 告诉我们跑哪些输入文件和 checksum；真正带物理含义的判据在 `analysis*.py` 中。

因此本节只讨论源码/脚本判据，不把个别运行记录混入结论。FieldSolver 相关测试大致分三类：

- 明确 assert 物理量：NCI 场能、PSATD Gauss law、静电球 L2 误差、隐式能量守恒、Newton/GMRES 迭代数。
- 半物理半回归：Hybrid Ohm solver 的 RZ normal modes 和 ion beam instability 会比较谱采样或增长率 RMS 的历史值。
- 可视化加 checksum：Landau damping、magnetic reconnection、Cartesian Ohm EM modes、cylinder compression 主要生成物理图像并依靠 checksum 自动发现输出漂移。

6.11.1 NCI FDTD：场能增长是否被 corrector 压住

`Examples/Tests/nci_fDTD_stability/analysis_ncicorr.py` 的核心检查是读 plotfile 中的 E_x 、 E_z 、 B_y ，计算

$$\mathcal{E}_{\text{NCI}} = \sum_{\text{grid}} (E_x^2 + E_z^2 + c^2 B_y^2).$$

源码判据是：


```
use_MR = re.search("nci_correctorMR", fn) is not None
```

```
if use_MR:
    energy_corrector_off = 5.0e32
    energy_threshold = 1.0e28
else:
    energy_corrector_off = 1.5e26
    energy_threshold = 1.0e24
```

```
ex = ad0["boxlib", "Ex"].v
ez = ad0["boxlib", "Ez"].v
by = ad0["boxlib", "By"].v
energy = np.sum(ex**2 + ez**2 + scc.c**2 * by**2)
```

```
assert energy < energy_threshold
```

这条 regression 的输入骨架也值得写清。inputs_base_2d 不是空白模板，而是已经固定了：

- 2D periodic drifting plasma;
- algo.current_deposition = esirkepov;
- algo.particle_shape = 3;
- warpx.use_filter = 1;
- warpx.cfl = 1;
- warpx.do_subcycling = 1;
- 电子和离子都沿漂移方向取无量纲动量 $u_z = 1000$;
- base 层已经打开 particles.use_fdt_nci_corr = 1。

inputs_test_2d_nci_corrector 只是把它固定为单层 `amr.max_level = 0`，而 `inputs_test_2d_nci_corrector_mr` 则切到 `amr.max_level = 1` 并用 `warpx.fine_tag_lo/hi` 把整个域提升到 refined level。也就是说，这两条并不是“是否开 corrector”的 AB 对照，而是“同一 corrected drifting-plasma 骨架”在 non-MR 与 MR 配置下的稳定性检查。

还要明确一个当前注册边界：analysis_ncicorr.py 试图用

```
use_MR = re.search("nci_correctorMR", fn) is not None
```

来切换 non-MR 的 $1e24$ 阈值与 MR 的 $1e28$ 阈值，但当前 CMakeLists.txt 给两条活跃测试传入的参数都写成 `diags/diag1000600`。因此，从可见注册层看，MR 分支并没有被单独显式选通。保守的结论应是：

- non-MR 强断言是直接可见并可证实的；
- MR 变体的验证目标明确是“mesh refinement 下也要压制 NCI”，但 $1e28$ 那条阈值分支目前更像 analysis 脚本中的预留区分逻辑，而不是注册参数层已直接证明的独立入口。

这个量不是严格写成 SI 形式的电磁能，而是 NCI 增长指示量。脚本把 corrector 关闭时的 benchmark 能量量级也打印出来，说明测试要捕捉的是“数值 Cherenkov 不稳定性是否被压低很多个数量级”。它对应前面 FDTD EvolveE/B、NCI corrector/filter 和边界/同步状态的组合效果，而不是单独验证某一行 curl stencil。

6.11.2 NCI PSATD: 电场能量比与 Gauss law

PSATD 的 NCI 稳定性测试在 Examples/Tests/nci_psatd_stability/analysis_galilean.py。脚本先从 warpx_used_inputs 判断维度、current correction、time averaging 和 single-box FFT，然后设置不同 reference energy 与容差。

核心源码是：

```
energy = np.sum(scc.epsilon_0 / 2 * (Ex**2 + Ey**2 + Ez**2))
err_energy = energy / energy_ref
assert err_energy < tol_energy

if current_correction:
    divE = all_data["boxlib", "divE"].squeeze().v
    rho = all_data["boxlib", "rho"].squeeze().v / scc.epsilon_0
    err_charge = np.amax(np.abs(divE - rho)) / max(np.amax(divE), np.amax(rho))
    assert err_charge < tol_charge
```

这里 `energy_ref` 不是解析电磁能，而是同一测试在不稳定设置下的参考能量。例如 Galilean PSATD case 的参考来自 `psatd.v_galilean=(0,0,0)`，averaged Galilean case 的参考来自关闭 time averaging。判据是

$$\frac{\sum \epsilon_0 |\mathbf{E}|^2 / 2}{\mathcal{E}_{\text{unstable,ref}}} < \text{tol_energy}.$$

如果打开 `psatd.current_correction`，脚本还检查离散 Gauss law:

$$\epsilon_\rho = \frac{\|\nabla_h \cdot \mathbf{E} - \rho / \epsilon_0\|_\infty}{\max(\|\nabla_h \cdot \mathbf{E}\|_\infty, \|\rho / \epsilon_0\|_\infty)} < \text{tol_charge}.$$

这正对应前面 PSATD 章节中的两件事: `PsatdAlgorithmGalilean` 通过移动坐标系降低 NCI; current correction 通过谱空间投影修正 J，使更新后的 E 与 rho 满足 Gauss law。

6.11.3 Maxwell hybrid QED: 真空修正后的相速度偏移

`Examples/Tests/maxwell_hybrid_qed/analysis.py` 不是在检查辐射反作用、光子发射或 Breit-Wheeler 产额，而是在检查 hybrid-QED 修正后的 Maxwell 色散关系。输入文件固定了:

- `warpx.grid_type = collocated`
- `algo.maxwell_solver = psatd`
- `warpx.use_hybrid_QED = 1`
- `warpx.quantum_xi = 1.e-23`

并直接用 parser 外场构造一个叠加在静态背景场 `E_s` 上的高斯包络平面波:

```
warpx.Ey_external_grid_function(x,y,z) = "exp(-z**2/L**2)*cos(2*pi*z/wavelength) + Es"
warpx.Bx_external_grid_function(x,y,z) = "-sqrt((1+(12*xi*Es**2)/epsilon0)/(1+(4*xi*Es**2)/epsilon0))*exp(-z**2/L**2)*cos(2*pi*z/wavelength)"
```

analysis 从最终 `Ey(x_mid,z)` 线抽取脉冲峰值位置，进而计算模拟相速度:

```
EyQED = EyQED_2d[EyQED_2d.shape[0] // 2, :]
z_end = dsQED.domain_left_edge[1].v + np.argmax(EyQED) * dz
phase_velocity_pic = (z_end - z_start) / dsQED.current_time.v
```

然后与输入里同一组 `Es`、`xi` 对应的理论相速度比较:

```
phase_velocity_theory = scc.c / np.sqrt(
    (1.0 + 12.0 * xi * Es**2 / scc.epsilon_0) / (1.0 + 4.0 * xi * Es**2 / scc.epsilon_0)
)
error_percent = (
    100.0 * np.abs(phase_velocity_pic - phase_velocity_theory) / phase_velocity_theory
)
assert error_percent < 1.25
```

因此它验证的是:

$$v_\phi^{\text{PIC}} \approx v_\phi^{\text{hybrid QED}} = \frac{c}{\sqrt{(1 + 12\xi E_s^2 / \epsilon_0) / (1 + 4\xi E_s^2 / \epsilon_0)}}.$$

这条 test 的定位应当是“field solver / Maxwell hybrid QED / vacuum-dispersion benchmark”，而不是宽泛的“QED processes”。它和第 4 章那类粒子 QED regression 的差别很大: 这里没有粒子事件统计，只有带 vacuum-polarization 修正的场传播速度。

本书对 Hockney 1971 的使用是摘要级而不是全文级: 摘要公开了 NGP/CIC/HNGP/HCIC 的比较、collision/heating time 缩放、optimum path 和 K_2 系数。因此这些关系可以作为第 6 章稳定性设计语言的来源边界，但不能替代原文图表、拟合过程和完整误差预算。

QPM/PPPM 与 force-shaping 两篇 1974 摘要还提供了 solver-side 的历史补充: 前者说明 Gaussian cloud、potential shaping 和近邻 particle-particle correction 如何服务于低噪声或 sub-mesh resolution，后者说明 charge-sharing hierarchy 与 potential-correction coefficients 如何影响 force-law isotropy。它们只能作为摘要级来源，不能替代原文推导和图表。

6.11.4 K_4、QPM 与 thermal-plasma 长期 figure of merit

Birdsall Chapter 13 对 Hockney 2d2v 长时间实验的转述，还给出了一条比“提高 shape order 会降低噪声”更可操作的设计语言。在 optimum path 上，heating time 与 slowing-down time 的比值可以写成

$$\left(\frac{\tau_H}{\tau_s}\right)_{\text{opt}} = K_4 \left(\frac{\lambda_D}{\Delta x}\right)^2.$$

这里的 K_4 是 particle shape、Poisson operator 和 potential correction 组合的经验 figure of merit。它不应被误读成当前 WarpX 任意输入的 universal constant；它只描述 Birdsall 所转述的 thermal-plasma 2d2v 参数面和 optimum-path 拟合。转述中的量级对比是：标准 CIC 的 K_4 约为 100；QS weighting 加 9-point Poisson solver 时约为 150；再加入 potential correction、有效粒子半径约为 1.8--3 个网格尺度的 QPM 变体时可到约 3000。后者的含义不是“只多一个滤波开关”，而是 particle shape、场算子和势修正共同削弱了 mesh alias 对长期 heating 的耦合；Birdsall 转述的估计是，计算代价约增加到两倍，但 K_4 和 τ_H/τ_{pe} 的 figure of merit 可获得数量级提升。

同一组实验还把 field fluctuation 写成 mesh-aware 粒子数的缩放：

$$\frac{E_x^2/8\pi}{nmv_t^2} \propto \frac{1}{N_C}, \quad N_C = n [\lambda_D^2 + (R\Delta x)^2].$$

因此 N_C 同时连接三件事：有效粒子数、有限尺寸 cloud 对 Debye-scale physics 的替代，以及 field-noise / collision / heating 的长期尺度。当 $R\Delta x > \lambda_D$ 时，cloud 半径而不是单独的 Debye length 主导统计误差；这也是为什么只报告宏粒子数密度或只报告 $\lambda_D/\Delta x$ 都不足以描述 thermal-plasma 的数值健康度。

最后，Hockney 观察到 kinetic-energy 增量 $h(t)$ 近似随时间线性增长。该形状应解释为 stochastic heating 的长期积累，而不是自动解释成某个离散 mode 的瞬时爆炸。对 uniform_plasma、energy_conserving_thermal_plasma 和稳定性案例，较稳妥的 reader-side 问题应当是：漂移是否近似线性、在多少个 τ_s 内积累到可见，以及 τ_H/τ_s 是否足够大；不能只凭短时间总能量曲线就宣称热背景“长期稳定”。这些 K_4 数值和 QPM 结构来自 Birdsall 对 Hockney 结果的转述；Hockney-Eastwood 原书/发表版全文未被本章作为可逐页核对的来源，因此本节不宣称对原始图表逐页核对。

6.11.5 静电球：解析场 L2 误差与能量守恒

Examples/Tests/electrostatic_sphere/analysis_electrostatic_sphere.py 检查均匀带电电子球的库仑展开。球半径满足

$$\ddot{r} = \frac{a}{r^2}, \quad a = \frac{q_e q_{\text{tot}}}{4\pi\epsilon_0 m_e}.$$

脚本把解析反函数写成：

```
def v_exact(r):
    return np.sqrt(q_e * q_tot / (2 * pi * e_mass * epsilon_0) * (1 / r_0 - 1 / r))

def t_exact(r):
    return np.sqrt(r_0**3 * 2 * pi * e_mass * epsilon_0 / (q_e * q_tot)) * (
        np.sqrt(r / r_0 - 1) * np.sqrt(r / r_0)
        + np.log(np.sqrt(r / r_0 - 1) + np.sqrt(r / r_0))
    )

r_end = fsolve(func, r_0)[0]

def E_exact(r):
    return np.sign(r) * (
        q_tot / (4 * pi * epsilon_0 * r**2) * (abs(r) >= r_end)
        + q_tot * abs(r) / (4 * pi * epsilon_0 * r_end**3) * (abs(r) < r_end)
    )
```

然后沿三条坐标轴抽取 WarpX 电场，避开靠近边界的区域，计算相对 L2 误差：

```
L2_error = np.sqrt(sum((E_exact_grid - E_grid) ** 2)) / np.sqrt(
    sum((E_exact_grid) ** 2)
)
```

```
assert L2_error_x < l2_tolerance
assert L2_error_y < l2_tolerance
assert L2_error_z < l2_tolerance
```

普通 case 的 `l2_tolerance=0.05`, `emass_10` case 放宽到 0.096。如果粒子 openPMD 诊断里有 `phi`, 脚本还检查势能释放和总能量守恒:

```
assert Ep_f < 0.7 * Ep_i
assert abs((Ek_i + Ep_i) - (Ek_f + Ep_f)) < energy_fraction * (
    Ek_i + Ep_i
)
```

这组判据直接覆盖 `ElectrostaticSolvers` 的 Poisson solve、边界处理、粒子 `phi` 诊断和粒子-场能量一致性。与 NCI 测试不同, 它有明确解析解, 因此最适合作为静电求解器章节的物理闭环例子。

这里正好可以接回 Birdsall-Langdon 第一分卷 4-9 到 4-10 的两个老判断。第一, Poisson stencil 的“局部更高阶”不自动意味着整套 PIC 离散系统更准确, 因为真正决定误差的是 mover、shape、field differencing 和 solver 合起来的系统合同, 而不是某一个局部公式单独最优。第二, 在已经用 finite-size particles 和网格差分改写了短程库仑作用之后, 场能量更基础的记账式是

$$\text{ESE} \propto \sum_k \rho_k \phi_k^*$$

而不是简单把

$$\sum_k |E_k|^2$$

当成无条件等价的替代。因为一旦离散系统把 $\rho \rightarrow \phi \rightarrow E$ 的合同改成了 K^2 、 κ 、smoothing 和 staggered differencing 的版本, $|E_k|^2$ 与 $\rho_k \phi_k^*$ 的比值就会带上额外的离散因子。第 6 章后面遇到 electrostatic sphere、Pierce diode 和其它静电 benchmark 时, 应优先把 `rho`、`phi`、field solve 和总能量账本放在同一个检查框架里, 而不是只看场图像。

Chapter 8 则把这条判断推进成更系统的 finite-grid 理论: $K(k)$ 、 $\kappa(k)$ 、 $S(k)$ 和 alias sum 不是分散在不同实现角落里的“修正项”, 而是直接一起进入 grid-modified dielectric function。换句话说, field solver 在 PIC 里不是单独决定色散的; 它总是和 particle shape、sampled density、force interpolation 共同定义一条离散色散关系。所以本章讨论 Poisson、spectral solve、smoothing 和 field differencing 时, 不能只按“求解器精度”排序, 而必须同时追问这些算子会怎样改写 alias branches、Langmuir dispersion 和 warm-plasma damping 的数值边界。

Chapter 9/10 进一步把这条判断分成两半。第一, finite Δt 也会制造自己的 time aliases, 因此 numerical heating 不一定只是 spatial-grid aliases 的副产品; 当某条 plasma branch 靠近 $\pi/\Delta t$ 一带的时间 alias 时, branch-coupling 本身就能触发高噪声和非物理增热。第二, 若想得到 exact energy conservation, 关键不是“把 E 算得更准”, 而是让离散 Poisson 解、 $\sum_j \rho_j \phi_j$ 场能量账本和粒子受力共享同一套 reciprocity 合同。也正因为如此, energy-conserving 路线和 momentum-conserving 路线不是同一求解器上可自由互换的小选项, 而是两套不同的离散系统组织方式: 前者优先保证总能量账本, 后者更自然地保留零总力/动量结构, 而 long-wavelength dispersion、self-force 与 alias errors 则必须另行逐项审查。

Chapter 12 再把这件事推进到统计层。Birdsall 在 12-3 到 12-7 里说明: PIC 里的 thermal noise、Debye shielding、field correlation 和 numerical heating 不能只被看成“粒子数有限导致的随机噪声”, 而应写成带有 $S(k_p)$ 、 $\epsilon(k, \omega)$ 和时间 alias comb ω_g 的 fluctuation spectrum。此时 $1/2 \int \rho \phi$ 又一次成为更基础的能量变量, 因为它直接把 $(\rho^2)_{k, \omega}$ 通过 K^2 接到 field-energy density 上。更关键的是, Birdsall 进一步把 grid effects 写成 effective kinetic collision operator, 并用 H-theorem 说明: space-time grid 可以在 Maxwellian 本应最大熵的情形下继续制造 entropy, 这正是 nonphysical heating、drift drag 和 velocity diffusion 的统一统计图像。因此本章后面凡是谈 Poisson、PSATD、smoothing、Langmuir damping、uniform-plasma noise floor 或 NCI/stability 时, 都不该只问“色散关系对不对”, 还要追问这套离散合同在 $(\rho^2)_{k, \omega}$ 、 $1/2 \int \rho \phi$ 、drift / diffusion 和 entropy production 这几类观测量上会留下什么数值病灶。Chapter 13 则把这条统计图像压成了更直接的工程尺度。第一, thermal plasma 的 heating/cooling 不能只按 N_D 或 CFL 粗略估; 它还显式依赖 $\lambda_D/\Delta x$ 、 $v_t \Delta t/\Delta x$ 、shape order, 以及 mover 自身的 phase error。第二, damped equations of motion 既可能抑制某些高频 branch, 也可能通过 nonresonant drag term 制造 nonphysical cooling, 所以“总能量下降”并不自动代表数值健康。第三, Hockney 的 2d2v 长时间实验和 Abe 的摘要级 $\sigma(K_g)/\text{correlation-time}$ 观测说明: 真正有设计价值的量往往不是单独的 collision time τ_s 或 heating time

τ_H , 而是二者的比值 τ_H/τ_s 及其短时 fluctuation 边界; 这些历史结果不能直接变成当前 WarpX solver 的定量预测。也就是说, 本章后面只写“某求解器稳定”还不够; 更严谨的说法应当是, 它把 thermal-plasma 观测窗口放在了多少个 collision times 之前, 或者把 nonphysical heating / cooling 推迟到了多久之后。

Dawson 1983 对本章的补充则更偏“为什么 electrostatic solver 会自然长成这个样子”。这篇综述不是从 Poisson 方程本身出发, 而是先从 finite-size particles 与 coarse-grained density 讲起, 然后把 shape factor $\rightarrow$ charge sharing / multipole expansion $\rightarrow$ uniform-grid FFT $\rightarrow$ Fourier-space Poisson solve $\rightarrow$ inverse FFT $\rightarrow$ gather back to particle 写成标准 electrostatic particle-model contract。这样一来, 本章讨论 electrostatic / spectral 路线时就不该把 FFT-Poisson 看成孤立求解器技巧, 而应把它视为和 particle shape、source representation、field interpolation 同时定义的一整套离散系统。

同一篇综述对 electromagnetic / Darwin 路线也给出了一条很适合保留的高层边界。对 full electromagnetic model, 时间步首先受最高频 light mode 与 CFL 限制; 主动截断高频 k modes 的理由, 不只是“算得更快”, 而是把弱耦合短波 branch 从建模目标里剔除, 从而把时间分辨率留给真正关心的大尺度 collective physics。与此同时, Dawson 还明确批评了 space/time filtering 的根本不对称: 空间方向早已有 particle size、k-mode truncation 与 Fourier solve 这类成熟手段, 但时间方向并没有真正等价的 ω -space filtering, 因此 large-time-step / time-averaged routes 仍然只是不同程度的时间滤波妥协。

对 Darwin model, 他的判断更直接: 这条路线的目标不是更完整地逼近 Maxwell, 而是主动删去 displacement current 和不关心的 radiation branch, 以便在 Alfvén waves、pinches、ion-cyclotron 这类低频磁化问题上摆脱 light-wave time-step 限制。但它也不能靠“把 Maxwell 少一项再原样 leapfrog”获得, 因为直接这样做会因 different-current mutual inductance 而数值不稳定, 必须重新组织 transverse-field 方程。这正好说明本章后面遇到的 electrostatic、full EM、implicit、hybrid 或 Darwin-like low-frequency routes, 并不是同一个 solver 家族上的小微调, 而是针对不同 branch-retention 目标做出的不同模型组织。

Dawson 1983 在 numerical stability 小节里又给了一个比 Birdsall 更概括的总结: particle simulation 里的两类型数值不稳定, 本质上都来自 stroboscopic sampling。空间离散会把连续 density spectrum 投影到有限 field Fourier modes 上, 从而制造 spatial aliasing; 有限 Δt 则会把高频 branch 重新折叠成低频有效 branch, 形成 time aliasing。这样看, finite-size particles、short-wavelength cutoff 和足够小的 time step 并不只是零散经验, 而是在分别压制这两类 alias resonance。这个高层说法对本章很有用, 因为它把 electrostatic、full EM、PSATD、implicit 和后面所有 time-filtering / space-filtering 取舍都放回了同一组数值病灶。

同一篇综述在 Tests of the statistical theory of plasmas 的入口又补了一条很有用的校验边界: 对一维 electrostatic sheet model, 因其力律简单、无需 grid, 可以直接跟踪 point-particle dynamics 到接近 machine accuracy, 文中甚至给出长时间能量守恒到 10^{-12} 量级的代表性代码。这意味着后面凡是讨论 gridded electrostatic model 的 drag、diffusion、field fluctuations 或 transport coefficients, 都不该只拿解析理论作唯一标尺; 更基础的比较对象还包括这类无 grid、近 exact 的 particle benchmark。换句话说, Poisson solver、particle shape 和 field interpolation 的数值副作用, 很多时候应被理解成“相对于更 fundamental particle model 多引入了多少统计输运偏差”, 而不只是“场图看起来是否平滑”。

这条统计理论主线再往下走, 还有两个对本章特别实用的测量合同。第一, velocity diffusion 不是一条单斜率直线: Dawson 1983 明确把它分成 short-time 的 $\langle \Delta v^2 \rangle \propto \tau^2$ 阶段和 decorrelation 之后的近线性增长阶段。这意味着 diffusion coefficient 只有在进入 random-impulse regime 后才有稳定解释。第二, thermal field fluctuation 的第一层合同不是整张场图, 而是每个 Fourier mode 的 time-averaged modal energy; 对 point particles, 它满足 $KT/2$ 型 equipartition, 而 finite-size particle shape 会系统改写这一 fluctuation level。于是本章后面不论讨论 electrostatic noise floor、shape order, 还是 smoothing / spectral filtering, 都应追问它们怎样改写 modal fluctuation spectrum, 而不是只看总场能量是否变小。

再进一步, Dawson 1983 还明确说明: thermal-plasma wave diagnostics 至少要分成 power spectrum、time correlation 和 magnetized peak taxonomy 三层。power spectrum 的第一价值是把 Debye-cloud random continuum 和 collective plasma spike 分开, 而不是单纯“看哪里有峰”; 同时 $\Delta\omega \lesssim 1/T$ 又说明有限 run length 会直接限制谱结构的可解释性。对有外磁场的体系, 谱图里还会出现 Bernstein harmonics、upper-hybrid peak、可动离子时的 ion-cyclotron / lower-hybrid peaks, 以及 $\omega=0$ 的 convective-cell / charged-flux-tube 结构。于是本章后面讨论噪声底、shape order、smoothing、spectral filtering 或 magnetized fluctuation 时, 都不应只写“能量更小/更稳定”, 而应继续问: 谱是在 continuum 还是 discrete spike 上被改写、相关时间有多长、以及被改写的是哪一类 mode family。

除了均匀带电球, WarpX examples 里还有一个更偏工程器件侧、但同样有理论对照的静电强基准: Examples/Physics_applications/pierce_diode/。它把两平行板间的 1D Pierce diode 直接设到 Child-Langmuir 极限, 输入里:

- warpx.do_electrostatic = labframe
- boundary.potential_lo_z = 0
- boundary.potential_hi_z = extractor_voltage
- ions.flux = J_{CL}/q_e

也就是把注入通量直接固定成理论空间电荷限制电流。analysis 随后读取 openPMD 中的 ϕ 、 E_z 、 ρ 、 j_z 和离子 z/u_z , 并把 $\phi(z)$ 与 $J(z)$ 和 Child-Langmuir 理论解比较, 要求相对误差都低于 20%。

所以 pierce_diode 的意义不是泛泛的“静电应用案例”, 而是:

- Poisson solver
- fixed-potential conducting boundaries
- 连续粒子注入通量
- space-charge-limited diode steady profile

这四层在同一个 1D 理论基准下被同时闭合。

6.11.6 隐式 EM: 能量、Gauss law 与求解器迭代数

隐式 solver 的 regression 不是只看场图像, 而是直接读 reduced diagnostics。Examples/Tests/implicit/analysis_1d.py 对 1D Picard case 做总能量漂移检查:

```
field_energy = np.loadtxt("diags/reducedfiles/field_energy.txt", skiprows=1)
particle_energy = np.loadtxt("diags/reducedfiles/particle_energy.txt", skiprows=1)

total_energy = field_energy[:, 2] + particle_energy[:, 2]
delta_E = (total_energy - total_energy[0]) / total_energy[0]
max_delta_E = np.abs(delta_E).max()

if re.match("test_1d_semi_implicit_picard", test_name):
    tolerance_rel = 2.5e-5
elif re.match("test_1d_theta_implicit_picard", test_name):
    tolerance_rel = 1.0e-14

assert max_delta_E < tolerance_rel

inputs_test_1d_theta_implicit_picard 写出 101 个 reduced-diagnostic 样本; 官方 analysis_1d.py 对这条输入检查总能量漂移:
```

$$\max \left| \frac{W(t) - W(0)}{W(0)} \right| = 3.4784001 \times 10^{-15} < 10^{-14}.$$

这条结果把第 3 章的 implicit 时间合同接到了第 6 章的 field-solver gate: 粒子能量与场能量必须作为一个总账本检查, 不能只看某一类能量。还要注意官方 analysis_1d.py 通过 CMake 测试目录名选择容差; 直接在任意自定义目录执行会出现 tolerance_rel 未定义, 因此项目保留了独立、不依赖目录名的合同分析脚本。

theta_implicit_picard 要求接近机器精度, semi_implicit_picard 允许更大的能量误差。对 exactly energy-conserving implicit EM, analysis_implicit.py 还检查 Gauss law RMS:

相邻的 inputs_test_1d_semi_implicit_picard 同样输出 101 个 reduced-energy 样本, 但采用半隐式 EM 的实际容差合同:

scheme	$\max \Delta W / W_0 $	tolerance	status
theta-implicit Picard	$3.4784001 \times 10^{-15}$	10^{-14}	PASS
semi-implicit Picard	2.2569031×10^{-6}	2.5×10^{-5}	PASS

官方 analysis_1d.py 对两条输入分别应用相应阈值。这里不能把两档容差读成分析脚本不一致: 它们对应的是两个不同的时间离散/场推进合同。theta-implicit 分支在该基准上把粒子与场总账本压到机器精度量级; semi-implicit 分支则以 $2.5e-5$ 作为官方允许的能量漂移上界。

```
drho = (rho - epsilon_0 * divE) / e / ne0
drho2_avg = (drho**2).sum() / (nX * nY * nZ)
drho_rms = np.sqrt(drho2_avg)
```

```
assert drho_rms < tolerance_rel_charge
```

归一化形式是

$$\epsilon_{\text{Gauss,rms}} = \left[\frac{1}{N} \sum_i \left(\frac{\rho_i - \epsilon_0 (\nabla_h \cdot \mathbf{E})_i}{en_{e0}} \right)^2 \right]^{1/2}.$$

如果只看这些 diagnostics,很容易把 implicit case 误解成“普通场推进外加一个 nonlinear solver 黑箱”。实际上源码里,Gauss law 和能量误差背后还隐含着一条更具体的线性化装配链。Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp 的 PreLinearSolve() 在线性求解前会:

```
m_WarpX->DepositMassMatrices();

if (m_use_mass_matrices_jacobian) {
    FinishMassMatrices();
    SaveE();
}

if (m_use_mass_matrices_pc) {
    SyncMassMatricesPCAndApplyBCs();
    const amrex::Real theta_dt = m_theta*m_dt;
    SetMassMatricesForPC( theta_dt );
}
```

这几步不是普通缓存刷新,而是在把粒子响应拆成两套不同对象:

- current_fp_non_suborbit = J_0;
- MassMatrices_X/Y/Z = dJ/dE 的完整局域响应;
- MassMatrices_PC 是从主质量矩阵裁剪、通信、施边界、再乘上 $c^2\mu_0\theta\Delta t$ 后供 preconditioner 使用的近似系数场。

随后 Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp 通过 ComputeJfromMassMatrices() 将 linear stage 的电场明确写成

$$J(E) = J_{\text{suborbit}} + J_0 + MM(E - E_0),$$

其中 E_0 由 Efield_fp_save 保存。ComputeJfromMassMatrices() 不是抽象矩阵乘法,而是在每个 Jx/Jy/Jz 分量的真实 staggered grid 上,对 Ex/Ey/Ez-E0 做局域 stencil 卷积。再往下,Source/NonlinearSolvers/MatrixPC.H、JacobiPC.H 和 CurlCurlMLMGPC.H 分别把 MassMatrices_PC 当成稀疏矩阵条目、局域 Jacobi 权重或 MLMG 的 beta 系数来消费。

因此这些 implicit regression 实际同时在检查三层东西:

1. 粒子推进和 J_0/MM/J_{\rm suborbit} 分拆是否一致;
2. Jacobian 近似是否真的围绕同一个 E_0 线性化;
3. preconditioner 拿到的 MassMatrices_PC 是否已经是边界、通信和物理系数都正确处理过的线性算子系数。

再往下一层,Newton 真正送进 GMRES / PETSc 的也不是 R(U) 本身,而是

$$F(U) = U - b - R(U).$$

Source/NonlinearSolvers/NewtonSolver.H 的 EvalResidual() 明确写成:

```
m_ops->ComputeRHS( m_R, a_U, a_time, a_iter, false );
```

```
// Compute residual: F(U) = U - b - R(U)
```

```
a_F.Copy(a_U);
```

```
a_F -= m_R;
```

```
a_F -= a_b;
```

而 matrix-free Jacobian Source/NonlinearSolvers/JacobianFunctionMF.H 再用有限差分构造方向作用:

```
m_Z.linComb( 1.0, m_Y0, eps, a_dU ); // Z = Y0 + eps*dU
```

```
m_ops->ComputeRHS(m_R, m_Z, m_cur_time, -1, true );
```

```
// dF = dU - (R(Z)-R(Y0))/eps
```

```
a_dF.linComb( 1.0, a_dU, eps_inv, m_R0 );
```

```
a_dF.increment(m_R,-eps_inv);
```

因此 WarpX 的 JFNK 线性 solve 实际是在做

$$J_F(U_0) \delta U \approx \delta U - \frac{R(U_0 + \epsilon \delta U) - R(U_0)}{\epsilon},$$

而不是手工显式装配整块 Jacobian。接着 WarpX_PETSc.cpp:174-190,300-318 只把这两个 WarpX 回调接进 PETSc:

- applyMatOp 调 a_linop->apply(...)
- applyNativePC 调 a_linop->precond(...)

也就是说, PETSc 在这里不是重新定义物理, 而只是消费 WarpX 已经定义好的 residual、Jacobian 方向作用和 preconditioner apply。

若再切到 pc_petsc 这条支线, 结构还要再分一次: Source/FieldSolver/ImplicitSolvers/StrangImplicitSpectralEM.cpp 里, Strang split implicit spectral EM 的 nonlinear 右端不是 curl-curl 场更新, 而是直接

$$R(U) = -\frac{\Delta t}{2} \mu_0 c^2 J^{n+1/2},$$

因为 source-free Maxwell 部分已经由前后两次 spectral advance 吃掉。对应源码是:

```
a_RHS.Copy(FieldType::current_fp, warpx::fields::FieldType::None, allow_type_mismatch);
amrex::Real constexpr coeff = PhysConst::c2 * PhysConst::mu0;
a_RHS.scale(-coeff * 0.5_rt*m_dt);
```

而当 PETSc 不走 PCSHELL, WarpX_PETSc.cpp:115-140,342-391 会在 SNES Jacobian callback 里触发一次 assemblePCMatrix(), 把 WarpX 的 preconditioner 近似搬进显式 Mat P。这条链的关键不是 Jacobian A 被显式装配, 而是:

- A 仍然是 shell matrix, 继续通过 apply() 做 matrix-free Jacobian;
- 只有 P 被单独装配成 sparse matrix 给 PETSc PC 使用。

对应初始化代码是:

```
KSPSetOperators( m_ksp->obj, this->m_A->obj, this->m_P->obj );
```

MatrixPC::Assemble() 则把这块 P 写成

$$P \approx I + \nabla \times (\alpha \nabla \times \cdot) + M_{PC},$$

其中单位阵、curl-curl stencil 和 MassMatrices_PC 都通过 insertOrAdd() 累加到同一行的列条目里。也就是说, pc_petsc 路径不是“把整个 implicit 求解显式矩阵化”, 而只是把 preconditioner 近似显式矩阵化, 再交给 PETSc 处理。

若再往下追一层, Source/FieldSolver/ImplicitSolvers/WarpXSolverDOF.cpp 说明 MatrixPC 的每一行并不是抽象的“Ex/Ey/Ez 某个分量块”, 而是先由 WarpXSolverDOF 给 staggered Efield_fp 的每个有效点分配一对 {local, global} 自由度编号。这个编号还不是对整个 MultiFab 无差别铺开, 而是先经过 getFieldDotMaskPointer(...) 取回的 dot-mask 裁剪: 只有 mask 为真的位置才进入线性系统, 其他位置的 local/global 槽都保留为 invalid。于是 MatrixPC::Assemble() 里的

```
const int ridx_l = dof_arr(i,j,k,0);
const int ridx_g = dof_arr(i,j,k,1);
if (ridx_l < 0) { return; }
```

实际意思就是: 这一条矩阵行只对应一个被 dot-mask 接受的 staggered 电场自由度, 而 ridx_l 决定它在本 rank 的行号, ridx_g 决定它在全局稀疏矩阵里的真实列号。

在此基础上, Source/NonlinearSolvers/MatrixPC.H 再按几何把这一行写成局域 stencil。共有三层叠加:

1. 先无条件写单位对角 I;
2. 若 thetaDt>0, 再写 curl(alpha curl .) 的离散条目;
3. 若 m_include_mass_matrices=true, 最后再把 MassMatrices_PC 的同分量局域窗口写进去。

不同几何的差异主要体现在第二层。1D Z 几何下只有横向 Ex/Ey 行带三点二阶差分, Ez 不带 curl-curl; XZ / RZ 下 dir=0,2 不只是本分量三点模板, 还会额外跨到横向分量写四个 mixed-derivative 角点条目, 对应二维的 $\partial_x \partial_z$ 交叉导数; 3D 下每个分量行都会同时耦合到另外两个分量, 在两个横向方向上写二阶项和 mixed-derivative 项; RCYLINDER 则没有这类跨分量 mixed derivative, 但径向二阶项都显式带有 $1/\rho$

0.5/i 这类圆柱几何因子。所有这些条目都通过 insertOrAdd() 合并到同一行里，并逐项乘上 BC_mask_Edir_arr(...), 所以边界条件不是事后再修，而是在矩阵条目生成时就已经嵌入 stencil。

而 BC_mask_Edir_arr(...) 本身也不是临时判断得到的布尔开关,而是 Source/FieldSolver/ImplicitSolvers/ThetaImplicitEM.cpp 在 pc_petsc 模式下预先分配并写好的系数场。InitializeCurlCurlBCMasks() 会根据几何维度先决定每个 E 分量需要多少类 mask, 然后再把 PEC、PMC、Silver-Mueller、PECInsulator 甚至轴线 None 的边界重构系数直接写进这些分量里。所以 MatrixPC::Assemble() 在边界上不是“先写标准 stencil, 再删条目”, 而是直接把已经改写好的离散系数乘进对角项、邻点项和 mixed-derivative 项。

MassMatrices_PC 这边也有类似的“前处理后消费”结构。Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp 先按 deposition 算法、shape 和 mass_matrices_pc_width 得到完整的 Jacobian mass-matrix 窗口 m_ncomp_xx/yy/zz, 再裁出只供 preconditioner 使用的 m_ncomp_pc_xx/yy/zz。该实现只保留 xx/yy/zz 三个同分量块, 不显式保留 xy/xz/... 交叉块; 随后 PreLinearSolve() 再对 MassMatrices_PC 做同步、J 边界处理和 $c^2 \mu_0 \theta \Delta t$ 缩放。因此 MatrixPC::Assemble() 读到的 sigma_ii_arr 已经不是原始粒子沉积结果, 而是一个经过窗口裁剪、通信和边界条件处理的 diagonal-block 近似。

最后一步 Source/NonlinearSolvers/WarpX_PETSc.cpp 说明 pc_petsc 的矩阵提交流程是: WarpX 先按每个 rank 的 m_ndofs_l 创建 Mat P 的行块, 再由 assemblePCMatrix() 从 MatrixPC 取回 device 端的行存数组, 拷回 host, 逐行调用 MatSetValues(), 最后统一 MatAssemblyBegin/End。所以这条链的并行 ownership 仍然跟着 WarpXSolverDOF 的 local/global 编号走, PETSc 只负责把各 rank 的行提交拼成全局 sparse matrix, 而不重新定义行列的物理含义。

这些实现细节之所以值得追到测试层, 是因为 Examples/Tests/implicit/analysis_petsc_matrix.py 给了它们一个非常强的硬断言: inputs_test_2d_curl_curl_petsc_pc、inputs_test_rz_curl_curl_petsc_pc 和 inputs_test_rcylinder_curl_curl_petsc_pc 都把 jacobian.pc_type = pc_petsc 和 pc_petsc.type = lu 放在一起, 然后直接要求

```
assert total_gmres_iters == num_steps
assert total_newton_iters == num_steps
```

也就是每个时间步恰好只需要 1 次 Newton 和 1 次 GMRES。由于 LU 在这里是精确线性求解器, 这组 regression 实际不是在检验长期物理解, 而是在把 WarpXSolverDOF 编号、MatrixPC::Assemble() 几何条目、curl2_BC_mask、MassMatrices_PC 和 assemblePCMatrix() 提交链整体当成一个“应当等价于精确 PC”的矩阵装配系统来验收。只要这条断言失败, 就更应该先怀疑矩阵条目生成或提交, 而不是先怀疑 Maxwell 方程本身。

同一条结构判据现在应明确绑定到三条 benchmark 名, 而不再混成一个笼统的 implicit checksum 桶:

- test_2d_curl_curl_petsc_pc
- test_rz_curl_curl_petsc_pc
- test_rcylinder_curl_curl_petsc_pc

它们共享同一个 analysis_petsc_matrix.py, 区别只在几何维度和 MatrixPC 的装配分支。

相比之下, Examples/Tests/implicit/analysis_planar_pinch.py 的验证口径更综合。它一边读取 newton_solver.txt 约束平均 GMRES/Newton 和 Newton/step 迭代数, 一边把 field_energy.txt、particle_energy.txt 和 poynting_flux.txt 合成完整能量账本, 再用 plotfile 里的 divE 与 rho 做 Gauss 定律 RMS 误差检查。因此 planar pinch 这一组 regression 不只是 solver smoke test, 而是在同时检验: implicit 粒子-场耦合是否保持能量守恒, 边界 Poynting flux 是否正确计入能量账本, preconditioner 是否维持可接受效率, 以及最终的电荷约束是否仍在机器精度附近。

另外一组经常被忽略但同样关键的 regression 是 analysis_vandb_jfnk_2d.py 与 analysis_vandb_jfnk_2d_cropping.py。前者把 theta_implicit_em + newton + Villasenor 放在 2D periodic thermal plasma 下, 只检查两件事: 总能量机器精度守恒, 以及

$$\rho - \epsilon_0 \nabla \cdot E$$

的 RMS 误差仍在机器精度附近。它因此更像是对 ImplicitPushPX.cpp、CurrentDeposition.H 中 Villasenor 路径、以及 SyncCurrentAndRho() 后续消费链的专项守恒验收, 而不是对 pc_petsc 的矩阵装配验收。inputs_test_2d_theta_implicit_jfnk_vandb_filt 只是把 warpx.use_filter = 1 打开, 然后继续用同一个 analysis, 这等于给 filter 路径加了一条“不能破坏能量守恒和 Gauss 定律”的强回归约束。

analysis_vandb_jfnk_2d_cropping.py 则更偏向边界和粒子轨道纠正路径。对应输入把 PEC 场边界、absorbing 粒子边界、particles.crop_on_PEC_boundary = 1、implicit_evolve.particle_suborbits = 1 和 algo.current_deposition = villasenor 同时打开, 但 analysis 只保留一个最大局部误差断言:

```
assert drho_max < tolerance_max_charge
```

这表明它关心的不是闭域总能量，而是：当 particle cropping、PEC 边界、suborbit fallback 和 Villasenor charge-conserving deposition 一起出现时，局部 Gauss 定律还能不能被守住。换句话说，这个 regression 实际把第 5 章的 charge-conserving deposition、第 4 章的 implicit suborbit fallback，以及第 7 章的 PEC/cropping 边界语义绑成了一条共同的验证链。

因此 Examples/Tests/implicit/ 现在至少应分成五条不同验证线，而不应继续被写成一个统一的 implicit checksum 桶：

1. analysis_1d.py 验证 1D Picard 周期热等离子体的总能量漂移，其中 semi_implicit_picard 容差较宽，theta_implicit_picard 要求机器精度；
2. analysis_implicit.py 验证周期/对称边界下 exactly energy-conserving implicit EM 的总能量和 Gauss-law RMS 同时达到机器精度；
3. analysis_2d_psatd.py 验证 strang_implicit_spectral_em + psatd 的 spectral split 没有破坏总能量守恒；
4. analysis_planar_pinch.py 验证 planar pinch 的能量账本、边界 Poynting flux、平均 Newton/GMRES 迭代数和 Gauss 定律；
5. analysis_vandb_jfnk_2d.py 与 analysis_vandb_jfnk_2d_cropping.py 分别验证 JFNK + Villasenor 周期热等离子体守恒，以及 PEC cropping / suborbit fallback 组合下的局部 Gauss-law 约束。

这样看，inputs_test_2d_theta_implicit_jfnk_vandb_filtered 和 inputs_test_2d_theta_implicit_jfnk_vandb_picmi.py 也都不再是“新的 implicit 物理 benchmark”，而只是把同一条 JFNK/Villasenor 守恒合同分别延伸到 filter 路径和 PICMI front-end 映射路径。

PSATD 这边的验证树也应采用同样的分层，而不是把所有 nci_psatd_stability tests 都写成一个统一的“PSATD / spectral solver”桶。当前至少应分成三类：

1. analysis_galilean.py 族：2D/3D/RZ 的普通 Galilean、current_correction、current_correction + periodic_single_box_fft，以及 averaged Galilean 与其 hybrid-grid 版本。共同判据是把最终电场能量与一个已知不稳定参考值比较，并在 current_correction=1 时额外检查 divE-rho/\epsilon_0 的相对误差。
2. analysis_psatd_CC1.py：单独覆盖 test_3d_uniform_plasma_psatd_JRhom_CC1，验证 JRhom = CC1 配合 do_divb_cleaning/do_dive_cleaning 后的 NCI 抑制。
3. checksum-only 基线：test_2d_comoving_psatd_hybrid、test_2d_galilean_psatd_hybrid 和 test_rz_psatd_JRhom_LL2 当前都没有独立 analysis，应诚实记录为 workflow/输出基线，而不是强稳定性断言。

Planar pinch case 还必须把边界 Poynting flux 纳入能量账本：

```
dE = Efields + Eplasma + dE_poynting
rel_net_energy = np.abs(dE - dE[0]) / Eplasma
assert max_rel_net_energy < rel_net_energy_tol
```

```
assert total_gmres_iters / total_newton_iters < gmres_iters_tol
assert total_newton_iters / num_steps < newton_iters_tol
```

这说明隐式 field solver 的正确性至少有三层：离散能量守恒、Gauss law 约束、非线性/线性求解器效率。PETSc matrix 测试进一步把求解器结构变成硬断言：

```
assert total_gmres_iters == num_steps
assert total_newton_iters == num_steps
```

当 LU 作为精确求解器或预条件器时，每个时间步只需要 1 次 Newton 和 1 次 GMRES；如果这个断言失败，问题更可能出在矩阵装配、DOF 映射、PETSc bridge 或预条件器，而不是 Maxwell 方程本身。

6.11.7 Hybrid Ohm solver: 哪些是强判据，哪些只是输出回归

Hybrid Ohm solver 的测试更接近物理 benchmark。ohm_solver_em_modes/analysis_rz.py 先对 $E_\theta(r, z, t)$ 做径向 Hankel 投影、轴向 Fourier transform 和时间 Fourier transform：

```
def transform_spatially(data_for_transform):
    interp = RegularGridInterpolator(
        (info.z, info.r), data_for_transform, method="linear"
    )
    data_interp = interp((zg, rg))

    Fmz = np.einsum("ijkl,kl->ij", proj, data_interp)
    Fmn = fft.fftshift(fft.fft(Fmz, axis=1), axes=1)
```

```
return Fmn
```

```
F_kw = fft.fftshift(fft.fft(results, axis=0), axes=0)
```

它会画出 fast/slow branch 和热共振线。显式 assert 只比较谱上固定采样点：

```
amps = np.abs(F_kw[2, 1, len(kz) // 2 - 2 : len(kz) // 2 + 2])
assert np.allclose(
    amps, np.array([55.65891974, 31.29213566, 70.13683876, 15.395433])
)
```

所以这不是完整色散关系拟合，而是谱结构回归。Ion beam R instability 更接近增长率 benchmark：对 $B_y(z, t)$ 做空间 FFT，追踪 $m=4, 5, 6$ 模，并用 Munoz et al. 2018 Fig. 12a 的增长率在 $10 < t\Omega_i < 40$ 内拟合：

```
gamma4 = 0.1915611861780133
gamma5 = 0.20087036355662818
gamma6 = 0.17123024228396777
idx = np.where((t_grid > 10) & (t_grid < 40))

A4 = np.exp(np.mean(np.log(np.abs(field_kt[idx, 4] / sim.B0)) - t_points * gamma4))
m4_rms_error = np.sqrt(
    np.mean(
        (np.abs(field_kt[idx, 4] / sim.B0) - A4 * np.exp(t_points * gamma4)) ** 2
    )
)

assert np.isclose(m4_rms_error, 1.546, atol=0.01)
```

脚本注释说明这些容差来自测试创建时的误差，不是从理论直接推导出的严格误差上界。因此失败时不能只看 assert，需要重跑 full benchmark 并人工比较增长到饱和前的理论趋势。

另外几类 Hybrid Ohm 测试没有显式 assert：

- ohm_solver_ion_Landau_damping/analysis.py 画出 $|E_z(k_m, t)|/|E_z(k_m, 0)|$ 与 $\exp(-\gamma t)$ 的比较， γ 来自 Munoz et al. 2018 Fig. 14b 插图。
- ohm_solver_magnetic_reconnection/analysis.py 输出重联率

$$R(t) = \frac{\langle E_y \rangle}{v_A B_0}.$$

- ohm_solver_em_modes/analysis.py 对 Cartesian parallel/perpendicular EM modes 做二维 FFT 谱图。
- ohm_solver_cylinder_compression 在 CMake 中 analysis=OFF，只有 checksum。

这给本章一个重要限制：Hybrid PIC 章节不能把所有 regression 都写成“物理判据已严格验证”。更准确的说法是：RZ normal modes 和 ion beam instability 有脚本级硬断言；Landau damping、magnetic reconnection、Cartesian EM modes 和 cylinder compression 主要提供物理图像与输出回归线索。

6.11.8 具体 regression 入口索引

上面的验证讨论按物理检查量组织。实际维护时，还需要知道哪些 regression 入口正在覆盖这些检查。下表按当前 Examples/Tests 的 CMake 与 analysis 脚本整理，目的是让读者能从正文回到可运行测试，而不是只停留在抽象“有验证”的说法上。

family	代表输入 / 测试名	analysis 入口	主要判据	本章对应的源码风险
PML FDTD Yee	pml/inputs_test_2d_pml/analysis_pml_yee.py	diags/diag1000300	本步能量除以初始激光能量，反射率相对理论值误差 < 5%	EvolveBPML/EPML、sigma damping、PML::Exchange() 是否能吸收而不反射
PML FDTD CKC	pml/inputs_test_2d_pml/analysis_pml_ckc.py	diags/diag1000300	同样检查反射率，理论参考值不同，误差 < 5%	CKC stencil 与 split-field PML 是否组合正确

family	代表输入 / 测试名	analysis 入口	主要判据	本章对应的源码风险
PML PSATD / Galilean	pml/inputs_test_2d_pml_psatd / pml/analysis_pml_psatd inputs_test_2d_pml_xdiag/diag1000300	pml/analysis_pml_psatd	要求 iteration 50 的能量与脚本常量一致到 $1e-14$, 再要求最终反射率 $< 1e-6$	PushPSATD() 后的 PML::PushPSATD()、谱场回填和 PML 边界是否正确
PML restart	pml/inputs_test_2d_pml_restart / pml/analysis_pml_restart inputs_test_2d_pml_xdiag/diag1000300	pml/analysis_pml_restart	重启前后 plotfile 一致	PML split fields 和场 guard cells 是否可 checkpoint/restart
RZ PML PSATD	pml/inputs_test_rz_pml_psatd / pml/analysis_pml_psatd diags/diag1000500	pml/analysis_pml_psatd	$ E_z < 2.0$	PML_RZ::PushPSATD()、RZ spectral PML 和径向边界吸收
Galilean PSATD NCI	nci_psatd_stability/inputs_test_2d_nci_psatd / nci/analysis_nci_psatd inputs_test_2d_nci_psatd_xdiag/diag1000300	nci/analysis_nci_psatd	之比小于维度/分支容差; current correction 分支还检查 $\max \text{divE-rho}/\text{eps0} $ 相对误差	Galilean PsatdAlgorithm 是否抑制 boosted-frame NCI, 并保持 Gauss law
Averaged Galilean PSATD	inputs_test_2d_averaged_galilean_psatd / inputs_test_3d_averaged_galilean_psatd	inputs_test_2d_averaged_galilean_psatd	同样用电场能量比检查稳定性, time averaging 分支容差更宽	fft_do_time_averaging 的平均场回填是否稳定
PSATD-JRhom NCI	inputs_test_3d_uniform_plasma_psatd_JRhom / inputs_test_3d_uniform_plasma_psatd_JRhom diags/diag1000300	inputs_test_3d_uniform_plasma_psatd_JRhom	电场能量除以 $66e6$ 后 $< 1e-8$	OneStep_JRhom() 多次沉积与 PsatdAlgorithmJRhom* 源项积分是否抑制 NCI
RZ PSATD-JRhom smoke	inputs_test_rz_psatd_JRhom / inputs_test_rz_psatd_JRhom checksum	inputs_test_rz_psatd_JRhom	只有最终 plotfile checksum	RZ JRhom 当前是输出回归入口, 不应写成物理强判据
Langmuir FDTD / PSATD 2D	langmuir/inputs_test_langmuir_fDTD / langmuir/analysis_langmuir_fDTD diags/diag1000080	langmuir/analysis_langmuir_fDTD	Langmuir 场最大相对误差 < 0.0503 ; 四阶形函数分支 < 0.07 ; 部分分支追加 charge conservation	场求解器、沉积、gather 和 guard-cell 同步在解析 plasma wave 中是否闭合
Langmuir 3D / div cleaning	langmuir/inputs_test_langmuir_3D / langmuir/analysis_langmuir_3D diags/diag1000040	langmuir/analysis_langmuir_3D	网格场与粒子处场均与解析场比较, 误差 $< 5e-2$; div-cleaning 分支还检查 $dF/dt = \text{divE-rho}/\text{eps0}$ 到 $1e-2$	3D PSATD/FDTD、粒子场诊断和 divergence cleaning 的一致性
Langmuir RZ / RZ PSATD	langmuir/inputs_test_langmuir_RZ / langmuir/analysis_langmuir_RZ diags/diag1000080	langmuir/analysis_langmuir_RZ	Langmuir 场误差 < 0.12 , 并检查 RZ 粒子过滤诊断	RZ field solver、RZ PSATD/current correction/JRhom 和诊断过滤是否共同正确

这个索引表也暴露了一个写作边界: 有些 regression 是物理强判据, 例如 Langmuir 解析场、PML 反射率、NCI 电场能量比; 有些只是 checksum 或 restart 路径, 例如 RZ PSATD-JRhom smoke 和部分 PML restart。正文讨论“验证链”时要区分这两类证据, 不能把 checksum 说成完整物理验证。

6.11.9 从源码入口回查验证量

当读者需要把一条输入参数、一个离散更新式和一个测试结果连起来时, 最短的回查路线不是从所有类定义开始, 而是依次回答三个问题:

1. 哪一个顶层分派选择了这条算法? 从 WarpXEvolve.cpp 的 OneStep 路线开始, 确认这是 FDTD、标准/ Galilean PSATD、JRhom、静电/静磁、隐式还是 Hybrid PIC。
2. 该路线在一个时间步中消费什么 source? 对 FDTD/PML 回到 EvolveB/EvolveE 及 PML 更新; 对 PSATD 回到谱场推进与历史 J/rho; 对隐式路线回到 residual 和迭代器; 对 Hybrid 回到 Ohm kernel 和 B 场子步。

3. 哪个 **analysis** 量能回答当前物理问题? 反射率对应 PML, 电场能量与 $\text{div}E-\rho$ 对应 NCI/Gauss law, 解析 Langmuir 场对应波动误差, 总能量和迭代信息对应隐式离散守恒。checksum 只能回答输出是否回归, 不能替代这些量。

这条三问路线也给出阅读源码时的停止条件: 找到一个函数名或一次 **assert** 还不足以说明算法正确; 必须能说清它所在的时间层、它消费的场或 source, 以及该测试实际测量的 observable。反过来, 测试通过也不表示任意几何、边界、AMR 层数或沉积方式都被证明正确。

6.12 练习与运行验证

1. **solver 分派题**: 给定 `algo.maxwell_solver`、`psatd.JRhom`、`m_implicit_solver` 和 AMR subcycling 四个开关, 使用第 2 章决策图判断它们分别会落到哪一个 `OneStep` 入口, 并列出一个不允许的组合。
2. **源码与观察量题**: 从 `EvolveB/EvolveE`、`PushPSATD`、`ImplicitSolver::ComputeRHS` 中各选一个入口, 指出它消费的是 J/ρ 、谱空间历史 source 还是 nonlinear residual; 再为该入口选择一个能检验它的 observable, 并写出该 observable 不能证明的结论。
3. **最小运行设计题**: 比较官方 `Examples/Tests/implicit/inputs_test_1d_theta_implicit_picard` 与 `inputs_test_1d_semi_imp` 在运行前阅读它们共用的 `analysis_1d.py`, 记录总能量、两条容差和各自算法类型; 运行后解释为何 $1.0e-14$ 与 $2.5e-5$ 不能只被理解成“一个更好、一个更差”, 以及为什么两者都不能单独证明其他几何或边界下的 Gauss law。
4. **跨章诊断题**: 选择一个 PML 或 NCI case, 按照“分派 -> source 时间层 -> observable -> 不可外推范围”写一页检查表; 再指出其中哪一项需要第 5 章的沉积/同步知识, 哪一项需要第 7 章的边界/AMR 知识。

6.13 本章结论

场求解器的选择不是在“FDTD 还是 PSATD”之间选一个名称, 而是在以下四层约束之间做匹配:

1. **表示与几何**。Cartesian、RZ、RCYLINDER/RSPHERE 所允许的场表示、谱基和轴条件不同; 不能把 Cartesian Fourier 公式直接移到 RZ 的 Hankel/azimuthal-mode 表示中。
2. **source 的时间模型**。标准 PSATD、Galilean/averaged PSATD 与 JRhom 对 J/ρ 的时间处理不同; current correction、沉积选择和 source 同步因此属于求解器选择的一部分, 而不是后处理开关。
3. **边界、通信与耦合**。PML split fields、guard cells、AMR coarse/fine 同步、隐式 residual 与 Hybrid Ohm 子步都决定离散更新是否能在完整 PIC loop 中闭合。它们将在第 7 章按边界和 AMR 路线继续展开。
4. **与问题匹配的验证量**。PML 应看反射率或残余场, NCI 应看场能与 Gauss-law residual, Langmuir 应看解析场, 隐式算法应同时看能量、charge/Gauss-law 和迭代信息。checksum 只是一类输出回归证据。

因此, 本章最后的判断顺序是: 先确定几何和物理目标, 再确定 source 时间模型与允许的算法组合, 随后沿真实 `OneStep` 路线检查边界/同步, 最后用对应 observable 解释结果。若其中任一步缺失, 就不应把一次成功运行写成“某类场求解器已经普遍正确”。这条顺序把第 4、5 章的粒子推进和沉积带入 Maxwell 更新, 也为第 7、8 章的边界诊断和案例解释提供了共同坐标。

下表将本章的主要验证问题压缩为一张选择表:

求解器路径	analysis 量	主要检查
FDTD + NCI corrector	$\sum(E_x^2 + E_z^2 + c^2 B_y^2)$	数值 Cherenkov 是否被抑制
PSATD Galilean/current correction	电场能量比、 $\nabla \cdot E - \rho/\epsilon_0$	NCI 抑制和 Gauss law
Electrostatic Poisson	均匀球解析 E_r 的三轴 L2、粒子势能/动能	Poisson 场、边界和能量一致性
Implicit EM	总能量漂移、Gauss RMS、	隐式离散守恒和求解器结构
	Newton/GMRES 迭代数	
Hybrid Ohm	谱采样、增长率 RMS、阻尼/重联图像、checksum	Ohm solver 的物理 benchmark 和输出回归

这也说明, 场求解器的“正确性”不能只靠某一个 **assert**, 而要把连续方程、离散公式、源码时间层、边界/同步和 regression analysis 合起来看。否则, 单独贴 `EvolveE.cpp` 的 curl 更新式, 仍然无法证明真实 WarpX field solver 在完整 PIC loop 中保持物理一致。

7. 边界条件、PML 与 AMR

边界、PML、guard cell 与 AMR 不是若干彼此独立的开关：它们共同决定 Maxwell 更新和粒子推进如何在有限计算域、多个 patch 与多个网格层级中闭合。本章按一条读者可追踪的链展开：输入怎样定义拓扑，场与粒子怎样在边界采取动作，数据怎样经过 guard cells/PML/AMR 迁移，最后用什么诊断判断这一闭合。

关于 PML 的资料、WarpX 源码和 Cartesian/RZ 案例可以共同说明指定设置下的实现与可测结果；它们不能自动证明所有 PML 系数、所有 Galilean/cleaning 组合，或 transition zone 中每一条粒子路线都被验证。阅读本章时，始终把“源码能定位的职责”“案例实际比较的量”和“可外推的物理结论”分开。

边界条件在 PIC 中同时作用于场和粒子。场边界控制 Maxwell 方程如何在计算域边缘闭合；粒子边界控制宏粒子离开、反射、吸收、周期穿越或被记录的方式。二者不能混为一谈。

WarpX 官方理论文档将 PML、PEC、PMC、Silver-Mueller、周期边界和嵌入边界说明在 Docs/source/theory/boundary_conditions.rst。初次阅读时可先用下面的源码入口定位问题：

- Source/BoundaryConditions/
- Source/Particles/ParticleBoundaries.cpp
- Source/Particles/ParticleBoundaries_K.H
- Source/Evolve/WarpXEvolve.cpp::HandleParticlesAtBoundaries

第一次阅读应先沿本章的因果链理解边界为何同时跨参数解析、主循环分派、场数组镜像和粒子沉积；需要实现细节时，再从本节列出的文件与函数向下追踪。

本章的阅读路线：边界是一个闭合系统

读者第一次读本章时，可以把所有边界问题放进同一条因果链：

输入参数 -> field boundary / particle boundary -> guard cells 与 PML 子域
-> 场更新与粒子处理 -> rho/J 同步与 AMR 重建 -> 输出诊断

这条链解释了三个容易混淆的现象。第一，PEC、PML 和 embedded boundary 不是同一类对象：前者是场边界条件，第二者是吸收层的离散子域，第三者还要先生成 cut-cell 几何和粒子侧距离判定。第二，粒子被吸收、被记录或穿过周期边界，和场的 Maxwell 边界是否正确是两条需要分别验证的路径。第三，AMR 的难点不只是细网格上的局部更新，还包括 coarse/fine ownership、guard-cell 填充、moving window 和粒子/源项同步。

因此每个边界案例都应按以下顺序阅读：

1. 先确认边界参数的 owner、默认值和方向配对；
2. 再确认边界进入哪个场/粒子分派，以及需要多少 guard cells；
3. 最后选择与问题相符的 observable：反射率、残余场、能量账本、重启重复性、scraped-particle buffer 或 AMR route ledger。

本章后面的案例段不是另一套理论，而是这条链上不同节点的检验。通过一个 PML 反射率 gate，不代表 RZ 残余场、粒子入 PML 或 AMR transition-zone route ledger 也已证明；读者应始终沿 producer/consumer 和 observable 的边界解释结果。

7.0 源码入口地图

本章不能只按“边界条件”这个名词归类，因为 WarpX 中的边界语义会穿过参数解析、场数组 guard cell、PML split field、粒子删除/反射/记录、诊断和 AMR 重建。以下列出读代码时需要反复回查的入口：

问题	核心函数/文件	读者问题
边界解析	MakeWarpX()	field periodic 为何约束 particle boundary?
field 参数	FieldBoundaries	为何 periodic 必须在 lo/hi 成对闭合?
particle 参数	ParticleBoundaries	何时继承 periodic, 何时保持 absorbing?
场边界	WarpXFieldBoundaries	PEC、PMC、Silver-Mueller 和轴边界怎样分派?
源项边界	ApplyRhoJ	rho/J 何时镜像, 何时在导体内清零?
PML	PML / DampPML()	split field 和电流 damping 如何协同?
guard cells	WarpXComm / GuardCellManager	哪些物理和数值选择决定通信宽度?
AMR 重建	RemakeLevel()	重映射后哪些场、粒子和 buffer 必须一起重建?
scraping	BoundaryScrapingDiagnostics	粒子何时被记录, 而不只是被删除?

这些入口位于 WarpX 的 Source/BoundaryConditions/、Source/Parallelization/、Source/Particles/ 与 Source/Diagnostics/；后文以文件路径和函数职责说明它们的关系，而不依赖随版本漂移的固定行号。

边界章节的主线可按以下闭合链阅读：

1. WarpX::MakeWarpX() 调用 parse_field_boundaries(), 由 field 边界构造 periodicity array, 再据此调用 parse_particle_boundaries()。
2. 场路径分别应用 E/B 边界、PEC/PMC/PECInsulator/Silver-Mueller/axis 条件和反射型 rho/J 边界；PML 则维护 split fields 与电流 damping。
3. WarpXComm 的 FillBoundary 与 PML exchange 让边界数据进入相邻 patch；GuardCellManager 据此决定 guard 宽度，AMR regrid、RemakeLevel() 与 EB factory 再重建相应状态。
4. 粒子路由 HandleParticlesAtBoundaries 处理边界和 buffer；需要记录而非只删除的粒子进入 BoundaryScrapingDiagnostics。

因此，后续解释边界时要同时回答三类问题：输入参数如何被约束，场和粒子的运行时边界动作在哪里发生，以及这些动作怎样与 PML、guard cell、AMR 和 diagnostics 互交叉。每一节都应能回到一个可观察量，例如反射率、残余场、scraped-particle buffer、能量账本或 AMR 的中间 route 账本；没有对应观察量时，只能把结论写成源码路径或未闭合边界。

7.1 field / particle 边界不是两套彼此独立的输入

WarpX::MakeWarpX() 在构造单例之前，先解析 field boundary，再从中提取 periodic 掩码，最后才解析 particle boundary：

```
std::tie(field_boundary_lo, field_boundary_hi) =
    warpx::boundary_conditions::parse_field_boundaries();

const auto is_field_boundary_periodic =
    warpx::boundary_conditions::get_periodicity_array(field_boundary_lo, field_boundary_hi);

std::tie(particle_boundary_lo, particle_boundary_hi) =
    warpx::particles::parse_particle_boundaries(is_field_boundary_periodic);
```

源码入口：Source/WarpX.cpp。

这意味着 particle 边界不是“读完自己就结束”，而是依赖 field 边界的第二阶段配置。其后果有两条：

1. 某方向若 field 是 periodic，则 particle 必须两侧都 periodic；
2. 如果用户根本没写 boundary.particle_lo/hi，periodic 的 field 方向会自动把 particle 边界改成 periodic，而不是保留 absorbing。

对应的 field consistency 检查在 FieldBoundaries.cpp：

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    (is_lo_periodic == is_hi_periodic),
    "field boundary must be consistenly periodic in both lo and hi");
```

源码入口：Source/BoundaryConditions/FieldBoundaries.cpp。

而 field/particle 联合一致性检查在 ParticleBoundaries.cpp：

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    (particle_boundary_lo[idim] == ParticleBoundaryType::Periodic) &&
    (particle_boundary_hi[idim] == ParticleBoundaryType::Periodic),
    "field and particle boundary must be periodic in both lo and hi");
```

源码入口：Source/Particles/ParticleBoundaries.cpp。

因此，periodic 在 WarpX 中的真实语义不是“某一侧做周期延拓”，而是“整根坐标轴拓扑闭合”。

读参数时应把 boundary.field_*、boundary.particle_*、boundary.potential_*、PECInsulator parser、particles.crop_on_PEC_b 和 PML 参数作为同一组依赖来检查：先由 field periodicity 建立拓扑，再确认粒子侧继承和 PML/导体边界是否与所选 geometry 相容。

7.2 电磁 field boundary 的顶层分派

field boundary 参数解析完成后，真正把边界施加到场数组的入口在 WarpXFieldBoundaries.cpp。ApplyEfieldBoundary() 顶层首先按边界类型分派：


```

if (::isAnyBoundary<FieldBoundaryType::PEC>(field_boundary_lo, field_boundary_hi)) {
    PEC::ApplyPECToEfield(...);
}

if (::isAnyBoundary<FieldBoundaryType::PMC>(field_boundary_lo, field_boundary_hi)) {
    PEC::ApplyPECToBfield(...);
}

if (::isAnyBoundary<FieldBoundaryType::PECInsulator>(field_boundary_lo, field_boundary_hi)) {
    pec_insulator_boundary->ApplyPEC_InsulatorToEfield(...);
}

```

源码入口: Source/BoundaryConditions/WarpXFieldBoundaries.cpp。

ApplyBfieldBoundary() 除了 PEC/PMC/PECInsulator 外, 还处理 Silver-Mueller:

```

if (lev == 0) {
    if (subcycling_half == SubcyclingHalf::FirstHalf) {
        if (::isAnyBoundary<FieldBoundaryType::Absorbing_SilverMueller>(field_boundary_lo, field_boundary_hi)) {
            m_fdt_d_solver_fp[0]->ApplySilverMuellerBoundary(...);
        }
    }
}

```

源码入口: Source/BoundaryConditions/WarpXFieldBoundaries.cpp。

这说明 Silver-Mueller 不是通用 field boundary post-process, 而是挂在 Yee/FDTD 的 B first half-push 上的专用边界。

继续往下看其内部实现时, 还要再补一层认识: 它更新的不是域内最后一层 B, 而是物理域外第一层 guard cell, 并且按 Yee 交错把切向 E 递推到切向 B。因此检查 Silver-Mueller 时应同时查看 first-half B 更新、对应 guard cell 和离域后的残余场, 而非只看边界参数名称。

7.3 PEC / PMC 不只是场边界, 也是沉积对称性

官方理论文档对 PEC 的定义是: 边界上切向 E 与法向 B 为零; guard 区对场做奇偶镜像; rho 和平行电流的边界处理还取决于粒子边界是 reflecting 还是 absorbing。见 Docs/source/theory/boundary_conditions.rst。

这意味着 PEC/PMC 的章节写法不能只停留在“某些分量置零”:

- 对 E/B, 要讲边界值和 guard-cell 镜像;
- 对 rho/J, 要讲镜像沉积与 image charge / reflective deposition;
- 对粒子, 要讲 ApplyBoundaryConditions() 和沉积语义如何配套。

因此 PEC、PMC 与 PECInsulator 的阅读顺序应固定为: 先看 E/B 的镜像或约束, 再看 rho/J 的边界处理, 最后检查粒子反射、吸收或 cropping 的配套语义。三层缺一不可, 都不能把边界写成简单的“分量置零”。

PMC 还有一条很直接的场级 regression: Examples/Tests/pec/inputs_test_3d_pmc_field。它在 z 方向设 PMC、在局部区域初始化正弦 Ey/Bx 波包, 然后用 analysis_pec.py 检查反射后的 standing wave 是否达到理论上的 constructive interference 振幅 $\pm 2E_{in}$ 。因此这条测试验证的不是抽象“PMC 边界存在”, 而是 PMC 通过交换 PEC 的 E/B 角色后, 反射相位与站波振幅仍满足理论合同。

Silver-Mueller 的 regression 则是另一条完全不同的口径。Examples/Tests/silver_mueller/analysis.py 直接读取最终 Full diagnostics, 并要求所有场分量在脉冲离域后都满足

$$|E| < 0.01 \text{ V/m},$$

而这些输入里的入射激光峰值量级约为 10 V/m。所以这组测试检验的不是“反射后应形成某种驻波”, 而是

$$|E_{\text{reflected}}| \ll |E_{\text{incident}}|.$$

该 family 共有四条最小基准:

- test_1d_silver_mueller
- test_2d_silver_mueller_x

- test_2d_silver_mueller_z
- test_rz_silver_mueller_z

它们分别覆盖 1D 轴向出射、2D x 向出射、2D z 向出射，以及 RZ z 向出射；其中 RZ 版本还同时把 `r_lo = none` 的轴线正则性和 `absorbing_silver_mueller` 开放边界放到同一最小回归里。

PEC 与 PECInsulator 的 regression 边界也应和上面区分开。pec family 里至少有两组强 analysis:

1. test_3d_pec_field 与 test_3d_pec_field_mr 这两条分别用 analysis_pec.py 和 analysis_pec_mr.py 检查反射后 standing-wave 的 E_{y_max}/E_{y_min} 是否接近理论 $\pm 2E_{in}$ 。单级版本容差为 1%，MR 版本放宽到 5%。因此它们真正验证的是 PEC 场边界反射后的波振幅合同，而不是抽象“边界条件被支持”。
2. test_2d_pec_field_insulator_implicit 与 ..._restart 这两条走 analysis_pec_insulator_implicit.py，把 fieldenergy.txt 与 poyntingflux.txt 合成完整能量账本，并要求相对误差低于 10^{-13} 。因此它们真正验证的是 pec_insulator parser 边界、implicit 场推进和边界 Poynting flux reduced diagnostics 一起构成的精确能量记账合同。..._restart 版本说明从 checkpoint 恢复后同一合同仍成立，但它并不是独立的逐字段 restart 对照。

相比之下，test_3d_pec_particle 当前没有独立 analysis，仍应诚实记录为粒子侧 gather/deposition 的 checksum 基线，而不是被误写成强物理 benchmark。

PML 的物理目标是吸收入射电磁波，使开放边界尽量不反射。经典 PML 思想来自 Berenger。WarpX 在 OneStep_nosub 的场推进后处理 PML: FDTD 分支中，场推进后若 do_pml 为真，执行 DampPML() 并填充 E/B/F/G 的 moving-window guard cells。PSATD 分支中也有单独的 PML damping。

7.4 PML 在 WarpX 里是独立子域，不是单个边界公式

PML 类本身管理的是一整套 PML 子域、split fields 和阻尼系数缓存:

```
class PML
{
public:
    PML (... ,
        int ncell, int delta, ...,
        int pml_has_particles, int do_pml_in_domain,
        ...,
        bool do_pml_dive_cleaning, bool do_pml_divb_cleaning,
        ...);
```

源码入口: Source/BoundaryConditions/PML.H。

真正承载阻尼 profile 的核心数据结构是 SigmaBox，其中缓存了:

- sigma / sigma_star
- sigma_cumsum / sigma_star_cumsum
- sigma_fac / sigma_star_fac
- sigma_cumsum_fac / sigma_star_cumsum_fac

源码入口: Source/BoundaryConditions/PML.H。

SigmaBox 的 profile 在 PML.cpp 中按离边界距离平方增长:

```
Real offset = static_cast<Real>(glo-i);
p_sigma[i-slo] = fac*(offset*offset);
...
offset = static_cast<Real>(glo-i) - 0.5_rt;
p_sigma_star[i-sslo] = fac*(offset*offset);
```

源码入口: Source/BoundaryConditions/PML.cpp。

所以 warpx.pml_delta 控制的是阻尼增长深度，而不是简单的总厚度。

7.5 PML split field 与 PML 电流

在主循环里,PML 阻尼入口是 WarpX::DampPML(),Cartesian 实际工作函数是 DampPML_Cartesian()。见 Source/BoundaryConditions/WarpX

这个函数先取出 pml_E、pml_B、sigba 和每个分量的 stagger 信息，然后把它们送进 warpx_damp_pml_ex/ey/ez/bx/by/bz。例如 Ex 的 split 分量阻尼：

```
if (sy == 0) {
    Ex(i,j,k,PMLComp::xy) *= sigma_star_fac_y[j-ylo];
} else {
    Ex(i,j,k,PMLComp::xy) *= sigma_fac_y[j-ylo];
}
```

源码入口：Source/BoundaryConditions/WarpX_PML_kernels.H。

这说明 PML 不是给整个 E_x 统一乘一个阻尼系数，而是对 E_{xy} 、 E_{xz} 这类 split components 按其离散位置和方向分别阻尼。

如果进一步允许粒子进入 PML，即 warpx.pml_has_particles = 1，那么粒子电流还要按 split 方式注入 PML 电场。push_ex_pml_current() 的形式是：

```
alpha_xy = sigjy[k-ylo]/(sigjy[k-ylo]+sigjz[l-zlo]);
alpha_xz = sigjz[l-zlo]/(sigjy[k-ylo]+sigjz[l-zlo]);
Ex(j,k,l,PMLComp::xy) = Ex(j,k,l,PMLComp::xy) - mu_c2_dt * alpha_xy * jx(j,k,l);
Ex(j,k,l,PMLComp::xz) = Ex(j,k,l,PMLComp::xz) - mu_c2_dt * alpha_xz * jx(j,k,l);
```

源码入口：Source/BoundaryConditions/PML_current.H。

也就是说，PML 中的 J_x 不是直接加到“整体 E_x ”上，而是要分摊到与阻尼方向一致的 split components 上。

PML 的 regression 入口也不能继续混成单一 checksum 桶。当前最稳定的五条验证线是：

1. test_2d_pml_x_yee
 - analysis_pml_yee.py
 - 从最终全场重建总电磁能量
 - 计算反射率 $R = E_{end} / E_{start}$
 - 要求其相对理论 $5.683000058954201e-07$ 的误差低于 5%
2. test_2d_pml_x_ckc
 - analysis_pml_ckc.py
 - 做同一反射率检查
 - 但理论值变成 $1.8015e-06$
3. test_2d_pml_x_psatd 与 test_2d_pml_x_galilean
 - 共享 analysis_pml_psatd.py
 - 先从 diag1000050 复算初始电磁能量并要求与硬编码参考值一致到 $1e-14$
 - 再要求最终反射率低于 $1e-6$
 - 因而这里验证的是 PSATD/Galilean PSATD + PML 的低反射率合同
4. test_rz_pml_psatd
 - analysis_pml_psatd_rz.py
 - 不比较能量比，而是在脉冲离域后直接要求域内 $\max(|E_r|, |E_z|) < 2$
 - 它真正验证的是 RZ radial PML 的残余场衰减
5. test_2d_pml_x_yee_restart 与 test_2d_pml_x_psatd_restart
 - 复用顶层 Examples/analysis_default_restart.py
 - 逐字段比较 restart 与非 restart 输出
 - 因而这两条是在测 PML + solver 场景的 restart 可重复性，而不是新物理吸收判据

相比之下，test_3d_pml_psatd_dive_divb_cleaning 当前 analysis=OFF。它把 psatd + pml + do_dive_cleaning + do_divb_cleaning + do_pml_dive/divb_cleaning 组合在一起，但目前只应诚实记录为 workflow/output checksum 基线，不能写成与上面四条同等级的强吸收 benchmark。

7.5.1 用正确的 observable 判断 PML

PML 的问题不是“是否开了吸收边界”，而是出射波、不同 solver、几何和 restart 后的状态是否仍满足对应的物理目标。应先根据问题选择 observable，再解释 analysis 的结论。

目标	代表性 consumer	可以支持的结论	不能替代
Cartesian FDTD 反射	<code>analysis_pml_yee.py</code> 、 <code>analysis_pml_ckc.py</code>	末态能量反射率与相应理论值的偏差 小于 5%	PSATD、RZ 或其他入射角
Cartesian PSATD/Galilean 反射	<code>analysis_pml_psatd.py</code>	初始能量重建一致，末态反射率小于 1e-6	每个 PML 系数的逐项证明
RZ PML 残余场	<code>analysis_pml_psatd_rz.py</code>	脉冲离域后 $\max(E_r , E_z)$ < 2	Cartesian 反射率或完整轴向诊断
checkpoint/restart particles in PML	<code>analysis_default_restart.py</code> 专用 <code>analysis_particles_in_pml.py</code> 和绝对值复核	物理量输出序列与基线一致 指定层级/字段的残余量满足	新的吸收精度结论 粒子轨迹守恒或完整 AMR 覆盖

因此，一个低反射率 PASS 不能被写成“PML 在所有场景都正确”。它只说明指定 solver、输入、网格、诊断和阈值构成的组合通过。特别是 3D AMR particles-in-PML 的官方 signed gate 通过而更强 absolute gate 仍越界，说明 consumer 的定义本身也是结论范围的一部分。

7.5.2 从 split field 到 runtime evidence

前文的 PML、SigmaBox、DampPML() 与 `push_ex_pml_current()` 解释了吸收层如何在源码中被构造和推进：每个 split component 按自身 staggering 和阻尼方向更新，粒子电流也要分摊到对应的 split field。源码入口说明“程序有这条机制”，运行 consumer 才回答“这条机制在某个 case 中的可观测结果”。

读者应把 PML 证据固定为三层：理论或文献解释吸收的目标，源码路径解释系数和分派，regression/analysis 限定一个 measurable outcome。LeeCPC2015 的 accepted manuscript 与 PSATD-PML 源码可以支持机制和公式映射的讨论，但 publisher-formatted PDF 的逐式差异仍未完成；同样，Cartesian、RZ、cleaning 和粒子入 PML 也必须保持各自的 observable 边界。

7.6 Embedded boundary 先是几何初始化和辅助标记系统

前面讨论的 PML、PEC、PMC、Silver-Mueller 都作用在计算域外边界上，而 embedded boundary 的第一步不是“给某个边界类型分派更新公式”，而是先把几何对象嵌入到 AMReX cut-cell 数据结构。

运行时总开关在 EmbeddedBoundary/Enabled.cpp：

```
std::string eb_implicit_function;
bool eb_enabled = pp_warpx.query("eb_implicit_function", eb_implicit_function);

std::string eb_stl;
eb_enabled |= pp_eb2.query("geom_type", eb_stl);
```

源码入口：Source/EmbeddedBoundary/Enabled.cpp。

这说明 EB 的启用条件不是单一布尔量，而是：

1. 编译时必须有 AMREX_USE_EB；
2. 运行时必须提供 warpx.eb_implicit_function，或者走 eb2.geom_type / eb2.stl_file 的 AMReX EB2 参数路径。

几何真正初始化在 WarpX::InitEB()：

```
if (! impf.empty()) {
    auto eb_if_parser = utils::parser::makeParser(impf, {"x", "y", "z"});
    ParserIF const pif(eb_if_parser.compile<3>());
    auto gshop = amrex::EB2::makeShop(pif, eb_if_parser);
    amrex::EB2::Build(gshop, Geom(maxLevel()), maxLevel(), maxLevel()+20);
} else {
    amrex::ParmParse pp_eb2("eb2");
    if (!pp_eb2.contains("geom_type")) {
        std::string const geom_type = "all_regular";
        pp_eb2.add("geom_type", geom_type);
    }
    amrex::EB2::Build(Geom(maxLevel()), maxLevel(), maxLevel()+20);
}
```

源码入口: Source/WarpXInitEB.cpp。

这里的结构很清楚:

- 若给出 `warpx.eb_implicit_function`, WarpX 自己负责把解析函数包装成 `ParserIF` 再交给 `AMReX EB2::GeometryShop`;
- 若没有隐式函数, 就沿用 `eb2.*` 参数, 由 `AMReX EB2` 直接构几何;
- `maxLevel()+20` 是为了让 `EB2` 尽量向粗层 `coarsen`, 服务 `multigrid`, 而不是某个物理精度参数。

EB 几何建立后, WarpX 还会把 signed distance 场填进 field registry:

```
const amrex::EB2::IndexSpace& eb_is = amrex::EB2::IndexSpace::top();
for (int lev=0; lev<=maxLevel(); lev++) {
    const amrex::EB2::Level& eb_level = eb_is.getLevel(Geom(lev));
    auto const eb_fact = fieldEBFactory(lev);
    amrex::FillSignedDistance(*m_fields.get(FieldType::distance_to_eb, lev), eb_level, eb_fact, 1);
}
```

源码入口: Source/WarpXInitEB.cpp。

所以 `distance_to_eb` 不是附加诊断, 而是后续 `scraping`、近壁沉积和 `cut-cell` 处理可以直接复用的几何辅助场。

更关键的是, `EmbeddedBoundaryInit.*` 在初始化阶段就为后续 `solver / deposition` 准备了几类辅助标记:

- `MarkReducedShapeCells`: 若高阶 `shape` 可能覆盖到部分或完全 `cut cell`, 就强制降成一阶沉积;
- `MarkUpdateCellsStairCase`: 对非 ECT solver, 只要 `field` 自由度邻接到非 `regular cell`, 就停止更新;
- `MarkUpdateECellsECT / MarkUpdateBCellsECT`: ECT solver 不看 `stair-case` 邻域, 而是直接看对应 `edge length` 或 `face area` 是否为零。

例如 `MarkReducedShapeCells()` 的混合区域逻辑是:

```
if ( !flag(i_cell, j_cell, k_cell).isRegular() ) {
    reduce_shape = 1;
}
```

源码入口: Source/EmbeddedBoundary/EmbeddedBoundaryInit.cpp。

而 `MarkUpdateCellsStairCase()` 的核心是:

```
if ( !flag(i_cell, j_cell, k_cell).isRegular() ) {
    eb_update_flag = 0;
}
```

源码入口: Source/EmbeddedBoundary/EmbeddedBoundaryInit.cpp。

这说明 EB 在 WarpX 里的第一层实现不是“直接改 Maxwell 更新式”, 而是先把 `cut-cell` 几何转换成:

1. 粒子沉积是否降阶;
2. 场自由度是否允许更新;
3. `edge/face` 几何量是否还存在。

因此 `embedded boundary` 必须先作为几何与辅助标记系统阅读, 再进入 `WarpXFaceExtensions.cpp` 的 `face-extension` 稳定性标志、`intrusion` 判据和 `cut-face` 修正。把 EB 简化成某一个 `field boundary` 会遗漏它对数组布局和粒子距离判定的前置影响。

7.7 Embedded boundary 的 face extension: 把不稳定 cut face 变成 enlarged face

对 ECT solver 来说, 仅仅知道某个 `face` 被 `cut` 还不够, 因为部分 `cut face` 的有效面积可能小到破坏稳定性。WarpX 的处理不是简单禁用这些 `face`, 而是尝试把它们扩成 `enlarged face`。

初始化侧先在 `MarkExtensionCells()` 中定义两个标志:

```
flag_ext_face_data(i, j, k) = int(S(i, j, k) < S_stab && S(i, j, k) > 0);
if(flag_ext_face_data(i, j, k)){
    flag_info_face_data(i, j, k) = 0;
}
if(int(S(i, j, k) > 0 && !flag_ext_face_data(i, j, k))) {
```

```
    flag_info_face_data(i, j, k) = 1;
}
```

源码入口: Source/EmbeddedBoundary/EmbeddedBoundaryInit.cpp。

其中:

- flag_ext_face = 1 表示这个 cut face 本身不稳定, 必须扩展;
- flag_info_face = 0 表示它当前是借方面;
- flag_info_face = 1 表示它是可出借面积的稳定面;
- 在 extension 过程中, 被别人侵入的 lender 会再改成 2。

真正的 extension 在 WarpX::ComputeFaceExtensions() 里按三步进行:

```
::init_borrowing(m_borrowing[maxLevel()], Bfield);
ComputeOneWayExtensions();
ComputeEightWaysExtensions();
::shrink_borrowing(m_borrowing[maxLevel()], Bfield);
```

源码入口: Source/EmbeddedBoundary/WarpXFaceExtensions.cpp。

第一步是 one-way extension, 只允许从一个正交邻居一次性借满所需面积 S_{ext} 。如果存在这样的 lender, 就直接把 lender 的 S_{mod} 扣掉 S_{ext} , 把 borrower 的 S_{mod} 增加 S_{ext} , 并把 lender 标成 2。见 Source/EmbeddedBoundary/WarpXFaceExtensions.cpp。

第二步是 eight-ways extension。若单邻居借不满, 就在 3×3 邻域内筛选所有可用 lender, 按原始 face 面积比例分摊:

```
const amrex::Real patch = S_ext * ::GetNeigh(S, i, j, k, i_n, j_n, idim) / denom;
```

源码入口: Source/EmbeddedBoundary/WarpXFaceExtensions.cpp。

但 WarpX 还会反复剔除那些按该比例借出后会把自己 S_{mod} 减成非正的邻居, 因此 eight-ways 不是机械加权, 而是“保正性的面积分摊”。见 Source/EmbeddedBoundary/WarpXFaceExtensions.cpp。

如果 one-way 和 eight-ways 都失败, 就进入 BCK fallback:

```
if (flag_ext_face_max_lev_idim(i, j, k)) {
    S(i, j, k) = ::ComputeSStab<idim>(i, j, k, lx, ly, lz, dx, dy, dz);
    flag_info_face_max_lev_idim(i, j, k) = -1;
}
```

源码入口: Source/EmbeddedBoundary/WarpXFaceExtensions.cpp。

源码注释说明这是 Benkler-Chavannes-Kuster correction, 精度低于常规 ECT extension, 但仍优于纯 staircasing。

extension 的借用关系不会散落在若干数组里, 而是统一压进 FaceInfoBox:

```
struct FaceInfoBox {
    amrex::Gpu::DeviceVector<Neighbours> neigh_faces;
    amrex::Gpu::DeviceVector<amrex::Real> area;
    amrex::Gpu::DeviceVector<int> inds;
    amrex::BaseFab<int> size;
    amrex::BaseFab<int*> inds_pointer;
```

源码入口: Source/EmbeddedBoundary/WarpXFaceInfoBox.H。

它记录的是“这个 enlarged face 向哪些邻居借了多少面积”。后续 ECT B 更新时, WarpX::EvolveB() 把 m_flag_info_face 和 m_borrowing 直接送进 solver:

```
m_fdt_d_solver_fp[lev]->EvolveB( m_fields,
                                   lev,
                                   patch_type,
                                   m_flag_info_face[lev], m_borrowing[lev], a_dt );
```

源码入口: Source/FieldSolver/WarpXPushFieldsEM.cpp。

在 FiniteDifferenceSolver::EvolveBCartesianECT() 中, 不稳定 face 会先聚合 enlarged face 的有效电荷:

```

Venl_dim(i, j, k) = Rho(i, j, k) * S(i, j, k);
...
Venl_dim(i, j, k) += Rho(ip, jp, kp) * borrowing_dim_area[ind];
...
rho_enl = Venl_dim(i, j, k) / S_mod(i, j, k);

```

源码入口: Source/FieldSolver/FiniteDifferenceSolver/EvolveB.cpp。

因此, WarpX 的 face extension 不是“把几何修漂亮一点”,而是直接决定 ECT solver 如何构造 enlarged face 的有效 rho 并推进 B。

7.8 Embedded boundary 的粒子侧: signed-distance 判定、默认吸收与 scraped buffer

对粒子来说, embedded boundary 的关键并不是 face extension, 而是“什么时候认定粒子已经撞进了 EB, 以及撞进去之后怎样处理”。

ParticleScraper.H 的核心逻辑非常直接:

```

ablastr::particles::compute_weights<amrex::IndexType::NODE>(
    xp, yp, zp, plo, dxi, i, j, k, W);
amrex::Real const phi_value = ablastr::particles::interp_field_nodal(i, j, k, W, phi);

if (phi_value < 0.0)
{
    ...
    amrex::RealVect normal = DistanceToEB::interp_normal(i, j, k, W, ic, jc, kc, Wc, phi, dxi);
    DistanceToEB::normalize(normal);
    ...
    f(ptd, ip, pos, normal, engine);
}

```

源码入口: Source/EmbeddedBoundary/ParticleScraper.H。

这说明 WarpX 的粒子撞墙检测并不是拿粒子坐标去直接查解析几何, 而是:

1. 在 nodal distance_to_eb 上插值;
2. 若 `phi_value < 0`, 认定粒子进入了需要刮擦的 EB 区域;
3. 再用 `DistanceToEB::interp_normal()` 从 signed-distance 的离散梯度重建法向。

`DistanceToEB.H` 本身也只做这件事。它的 `interp_normal()` 对 `phi` 做差分加权, `normalize()` 再把法向单位化。也就是说, 粒子侧法向来自 signed-distance 场, 而不是 STL 面片直接投影。

默认处理器在 `ParticleBoundaryProcess.H` 里只有两个最小版本:

```

struct NoOp { ... };

struct Absorb {
    ...
    amrex::ParticleIDWrapper{ptd.m_idcpu[i]}.make_invalid();
}

```

源码入口: Source/EmbeddedBoundary/ParticleBoundaryProcess.H。

因此, 当前主源码链上的默认 EB 粒子边界语义其实很朴素: 不是复杂反射模型, 而是“把撞进 EB 的粒子标成 invalid”。真正删除发生在后续 `deleteInvalidParticles()` 或 `Redistribute()`, 而不是 `Absorb()` 本身立即擦除数据。

这一逻辑会在多处触发:

- `WarpXParticleContainer` 在 `Redistribute()` 后立刻做一轮 EB 吸收;
- `MultiParticleContainer::ScrapeParticlesAtEB()` 可以对所有 species 统一刮擦;
- `AddParticles.cpp` 在新增粒子或 flux 注入后, 也会先对临时容器做 `scrapeParticlesAtEB(..., Absorb())`。

因此 WarpX 的策略是“新粒子一旦进入容器, 就尽快排除已经落在 EB 内部的非法粒子”。

如果用户想把这些 scraped 粒子保留下来做诊断, 就可以设置:

- `<species_name>.save_particles_at_eb = 1`

官方文档说明这会吧撞到 EB 的粒子复制到 scraped particle buffer, 可供 BoundaryScrapingDiagnostic 或 Python 接口使用。见 Docs/source/usage/parameters.rst。

更重要的是, ParticleBoundaryBuffer.cpp 在把粒子放进 EB buffer 时, 不是简单复制当前状态, 而是用二分法沿粒子轨迹回溯到 $\phi = 0$ 的真实交点:

```
amrex::Real const dt_fraction = amrex::bisect( 0.0, 1.0,
  [=] (amrex::Real dt_frac) {
    ...
    UpdatePosition(x_temp, y_temp, z_temp, ux, uy, uz, -dt_frac*dt, mass);
    ...
    amrex::Real const phi_value = ablastr::particles::interp_field_nodal(i, j, k, W, phiarr);
    return phi_value;
  });
```

源码入口: Source/Particles/ParticleBoundaryBuffer.cpp。

随后它还会记录 scraping 发生的 step、时间偏移、真实时间和表面法向。也就是说, scraped particle buffer 保存的不是“死前最后一帧粒子”, 而是“与 EB 表面交点处的粒子诊断样本”。

因此, embedded boundary 的粒子侧主链可以概括为:

1. distance_to_eb 判定粒子是否进入 EB;
2. DistanceToEB 重建边界法向;
3. 默认 Absorb 仅把粒子标成 invalid;
4. 后续删除逻辑真正清除粒子;
5. 若启用 save_particles_at_eb, 则 ParticleBoundaryBuffer 回溯到 $\phi=0$ 交点并记录 scraped 事件。

Examples/Tests/embedded_circle/ 为这条链提供了一个实用但证据层级较弱的入口。它不是 electrostatic_sphere_eb 那类解析 ϕ /Er 强基准, 也不是 point_of_contact_eb 那类直接检查交点几何的强 analysis, 而是一个 2D circular EB workflow baseline:

- eb_implicit_function 定义圆形导体
- eb_potential = -10 进入静电求解
- 电子/氦离子都先 initialize_self_fields = 1
- 双物种 background_mcc 持续运行
- 两个 species 都打开 save_particles_at_eb = 1
- diag3 用 BoundaryScraping openPMD 写出 scraped 粒子

该 CMakeLists.txt 中的 test 没有独立 analysis.py, 只保留 checksum helper。因此它在本章里更适合承担:

- EB geometry + electrostatic + MCC + BoundaryScraping 的联合 workflow 基线

而不是承担解析电势或表面碰撞物理的强验证。

domain boundary buffer 的收集时机也要一起记住。WarpXEvolve.cpp 里先执行 mypc->ApplyBoundaryConditions(), 随后立刻调用 m_particle_boundary_buffer->gatherParticlesFromDomainBoundaries(*mypc, cur_time), 最后才在 EB 路径后统一 deleteInvalidParticles()。因此 buffer 记录依赖的是“粒子还在容器里、但已经越界或即将失效”的中间态, 而不是从被删除后的粒子列表回溯出来。对 save_particles_at_xlo/.../eb、BoundaryScrapingDiagnostic 和 Python buffer 接口来说, 这个顺序决定了 scraped 数据为什么能同时保留 step、时间偏移和边界法向。

周期边界有一个关键规则: 如果某个方向的场边界是 periodic, 该方向粒子边界也必须 periodic; 非周期边界则不要求 field 和 particle 边界字面一致。这一规则由本章 7.1 的参数解析和断言直接约束, 不能从“某次运行没有报错”反推。

AMR 的目标是把计算资源集中在物理上需要高分辨率的区域。但 PIC 的 AMR 比流体 AMR 更难, 因为宏粒子穿过 refinement interface 时会看到不同网格上的插值场, 容易产生 self-force、短波反射和电荷/电流不一致。WarpX 官方 Docs/source/theory/amr.rst 特别强调两个问题: mesh refinement interface 附近的 spurious self-force, 以及电磁波在粗细网格界面处的反射和放大。

在真正进入 coarse-fine interface 之前, 还必须先把并行层的 guard-cell 通信模型看清。WarpX 不是“所有字段都统一 FillBoundary 一次”这么简单, 而是把通信语义分成两类:

- E/B/F/G/Aux 这类场变量: guard cells 用 FillBoundary 风格的复制/同步;
- J/rho 这类源项: 重叠区域必须做 SumBoundary / ParallelAdd 风格的累加, 因为粒子靠近 box 边缘时, 同一 (i,j,k) 可能同时被沉积到一个 box 的 guard 区和另一个 box 的 valid 区。

这一层的统一预算由 Parallelization/GuardCellManager.* 管理。它先区分:

- `ng_alloc_*`: 每类 MultiFab 实际分配多少 guard cells;
- `ng_FieldSolver`、`ng_FieldGather`、`ng_UpdateAux`、`ng_MovingWindow` 等: PIC 循环不同阶段真正交换多少。

这些配额不是拍脑袋常数, 而是共同受以下因素控制:

- 粒子 shape 阶数与 subcycling;
- moving window 位移;
- Galilean / comoving 修正;
- NCI / bilinear filter;
- FDTD stencil 或 PSATD 局部 FFT stencil。

例如 `SyncCurrent()` 的源码注释就明确说明, 多层 AMR 下不能简单把 finer coarse-patch current 直接 `ParallelAdd` 到当前 level 的 fine patch, 因为 nodal overlap 会双计数; WarpX 为此引入临时 `fine_lev_cp` 和 `OwnerMask` 去重。也就是说, coarse-fine current 同步本身就已经是 AMR 物理一致性的一部分, 而不是纯粹 MPI 细节。

guard-cell 宽度由场求解、PML、滤波、particle gather/deposition 和 AMR 同时约束; 因此一项输入变更可能改变通信宽度, 即使主域更新公式不变。

继续往下看 `Parallelization/WarpXComm.cpp`, 会发现 WarpX 对 current / rho 的 coarse-fine 同步并不是简单的“restrict 一下再加回去”。`SyncCurrent()` 的大段注释明确说明:

- finest level 先把 fine-patch current restriction 到同层 coarse patch;
- 若有 current buffer, 则 coarse-patch current 先并入 buffer, 再把 buffer 当作更粗层的通信源;
- 更粗层接收 finer 数据时, 先写到临时 `fine_lev_cp`;
- 由于 nodal 点可能在多个 box 中重叠, 不能直接加回 `J_fp`, 而要借助 `OwnerMask` 只让 owner box 接管该点数据。

因此, WarpX 的 coarse-fine source 同步真实更接近:

restriction -> optional buffer merge -> temporary receive -> owner-mask de-dup -> same-level SumBoundary

而不是单一的 restriction/prolongation 二步法。

rho 路径在 `SyncRho()` 中基本平行, 只是 bilinear filter 与 SumBoundary 被折叠成 `ApplyFilterandSumBoundaryRho()`。这意味着 J 和 rho 虽然共享 AMR 同步框架, 但在 filter 实现上仍有细微差异。

继续顺着 `WarpXRegrid.cpp` 往下读时, 问题就不再是“数据如何同步”, 而会转成“DistributionMapping 改变后, fields、particles、EB、boundary buffer 和 diagnostics 如何整体重建”。这也是为什么 regrid 不能只用末态场图来判断正确性。

WarpXRegrid.cpp 的顶层入口是:

```
void
WarpX::CheckLoadBalance (int step)
{
    if (step > 0 && load_balance_intervals.contains(step+1))
    {
        LoadBalance();
        ResetCosts();
    }
    if (!costs.empty())
    {
        RescaleCosts(step);
    }
}
```

源码入口: `Source/Parallelization/WarpXRegrid.cpp`。

这说明 WarpX 的 load balance 不是“到点直接重分布”, 而是:

1. 周期性检查当前 step 是否命中 `load_balance_intervals`;
2. 若命中, 则执行 `LoadBalance()`;
3. 之后清零 costs;
4. timer 模式下还会继续对 costs 做 running-average 式重标定。

`LoadBalance()` 内部先按 level 构造候选 `DistributionMapping`, 支持:

- SFC
- knapsack

然后比较 `currentEfficiency` 和 `proposedEfficiency`, 只有满足:

`proposedEfficiency > load_balance_efficiency_ratio_threshold*currentEfficiency`

时才真正采纳新映射。源码入口: `Source/Parallelization/WarpXRegrid.cpp`。

因此, WarpX 当前策略更接近“收益足够大才搬”, 而不是粗暴地按固定周期强制改 rank 图。

更关键的是 `RemakeLevel()` 的边界。它当前只支持:

- `BoxArray` 不变;
- `DistributionMapping` 改变。

因为函数内部直接写着:

```
if (ba == boxArray(lev)) {
    ...
} else
{
    WARPX_ABORT_WITH_MESSAGE("RemakeLevel: to be implemented");
}
```

源码入口: `Source/Parallelization/WarpXRegrid.cpp`。

这意味着当前这里讨论的还不是“任意 AMR regrid”, 而是“同一 patch 拓扑下的 rank 重映射”。

一旦某个 level 真的采纳新映射, WarpX 重建的远不只是粒子容器。`RemakeLevel()` 里会依次重做:

- `m_fields.remake_level(lev, dm)`: field registry 的 level 场数据;
- EB 相关的 `m_eb_reduce_particle_shape`、`m_eb_update_E/B`、ECT `m_borrowing`;
- `m_field_factory[lev]` 与 `InitializeEBGridData(lev)`;
- PSATD 的 fine/coarse spectral solver real-space 容器;
- `m_accelerator_lattice[lev]->InitElementFinder(...)`;
- `current_buffer_masks / gather_buffer_masks` 与 `BuildBufferMasks()`;
- `multi_diags->InitializeFieldFunctors(lev)`。

源码入口: `Source/Parallelization/WarpXRegrid.cpp`。

随后若至少有一个 level 完成了 load balance, WarpX 才统一做:

```
mypc->Redistribute();
mypc->defineAllParticleTiles();
m_particle_boundary_buffer->redistribute();
reduced_diags->LoadBalance();
```

源码入口: `Source/Parallelization/WarpXRegrid.cpp`。

因此, WarpX 的 load balance 不是“先搬粒子再说”, 而是一次多子系统一致提交:

```
candidate DM -> efficiency check -> remake field/EB/solver/masks -> redistribute particles ->
redistribute boundary buffer -> refresh diagnostics
```

这一阶段的正确性依赖多子系统共同完成迁移; 只确认某个 particle count 或 field checksum 保持不变, 不能证明 boundary buffer、EB factory 和 diagnostics 的 ownership 也已同步。

7.9 AMR transition zone: 为什么最终 plotfile 不足以证明路由正确

AMR 的 transition zone 同时影响 gather 和 deposition。粒子在细网格 interior 时从 `E/Bfield_aux` gather 并向 `rho_fp/current_fp` deposition; 进入 buffer 后, gather 和 deposition 可以分别切换到 coarse `E/Bfield_cax` 与 `rho_buf/current_buf`。这两个分界不必相同, 因此“场看起来正常”不能证明粒子经过了正确的 coarse/fine route。

`PartitionParticlesInBuffers()` 是理解这条路径的关键入口。它在一个 tile 内给出 `nfine_gather` 与 `nfine_deposit`, 随后同步阶段才会把 buffer 与 coarse/fine 数据合并。因而强验证需要在分区之后、同步之前观察 route counts 或中间账本; 只读取最终 plotfile, 只能给出间接证据。

证据层	能说明什么	仍不能说明什么
源码路径	buffer mask、 <code>nfine_gather/nfine_deposit</code> 、 <code>aux/cax/fp/buf</code> 和同步入口存在且相互对应	每个 route 在真实 case 中都被独立命中
现有 MR case	subcycling、moving window、PML 或解析场 consumer 可验证整体运行完整性	<code>fine/coarse gather/deposit</code> 的逐粒子分区
route-count schema	专用 diagnostic 应检查 count、weight、 <code>rho/J</code> 与 post-sync closure	WarpX 已经输出这些数据
runtime activation	已有 AMR workflow 确实调用了 partition/sync 分支	没有 route id、pre-sync buffer 或 owner-mask 数值账本

因此，本章对 transition zone 的准确结论是：源码路径已核、整体 MR workflow 有运行证据、专用 route ledger 仍未实现。不要把末态 checksum、解析场误差或 profiling marker 写成 branch-level route proof。

7.10 本章练习与源码定位

1. **边界分派题**: 给定一个 field boundary 和一个 particle boundary, 分别沿 `parse_field_boundaries()`、`parse_particle_boundaries()` 定位它们如何进入 solver/particle container; 说明 periodic 继承为什么不能只看输入字符串。
2. **PML 证据题**: 对照 `pml/analysis_pml_yee.py`、`analysis_pml_psatd.py` 和 RZ analysis, 区分反射率强判据、末态 residual 判据和 checksum-only 证据。
3. **AMR route 题**: 阅读 `BuildBufferMasks()` 与 `PartitionParticlesInBuffers()`, 画出一个粒子分别进入 fine gather、coarse gather、fine deposit 和 coarse deposit 的条件; 说明为什么当前没有 dedicated route-count regression 时不能声称每条 route 已被单独验证。

7.11 本章结论

边界问题的正确读法不是先问“这个边界有没有打开”，而是顺序确认下列四件事：

1. **拓扑是否一致**。field 与 particle 的 periodicity 必须沿同一坐标轴成对闭合; PEC、PMC、Silver-Mueller 和 embedded boundary 则对应不同的场或几何条件，不能用同一组输入语义代替。
2. **离散更新是否在边界闭合**。PML 的 split fields、guard-cell 交换、`rho/J` 镜像或清零，以及粒子反射、吸收和 scraping 分属不同路径。需要分别知道哪一个数组在何时更新，而不能只观察最终粒子数或一个场快照。
3. **AMR 状态是否共同迁移**。regrid 或 load balance 会同时影响 fields、粒子、buffer masks、PML/solver 容器和 diagnostics。coarse/fine 的 gather 与 deposition 也可在不同条件下切换，因此一个平滑的末态场不是 route 正确性的充分证据。
4. **观察量是否匹配问题**。PML 可用反射率或残余场，边界粒子可用 scraping buffer，重启可用输出重复性，AMR transition zone 则需要分区后的 route count、weight、`rho/J` buffer、coarsened-fine 与 owner-mask 的共同账本。checksum 只能补充这些判据，不能替代它们。

这四层构成本章与前后章节的连接：第 5、6 章决定沉积 source 和 Maxwell 更新如何生成，第 7 章检查这些量如何穿过边界、通信和网格层级，第 8 章再选择诊断把结果转化为可解释的证据。当前 AMR case 可以支持整体 workflow 已经走通，但仍不能把它写成 transition zone 的逐 route 验证；在 `PartitionParticlesInBuffers()` 之后没有对应账本之前，这个边界必须保留。

8. 诊断、验证与案例

PIC 程序的可信度来自验证，而不是来自输入文件能跑完。一个最小验证闭环需要回答：

- 初始条件是否表达了目标物理问题；
- 网格、粒子数、时间步是否分辨关键尺度；
- 输出量是否足以检查守恒律和不稳定性；
- 结果是否能和解析解、benchmark、regression 或文献对比；
- 源码和分析脚本是否确实覆盖了所声称的运行路径。

本章不按文件格式罗列功能，而按一条读者可追踪的证据链展开：先定义想测的物理量，再确认它从哪些运行态生成，接着选择 **reader-side** 的比较对象，最后标出该比较能支持和不能支持的结论。Langmuir wave、uniform plasma 和 LWFA/PWFA 分别提供解析波、热背景与应用 workflow 三条主线；后半章再把同一方法落实到 full/reduced diagnostics、plotfile/openPMD/checkpoint 和边界粒子缓冲区。

在进入具体案例前，可以先记住 Dawson 1983 对 diagnostics 的一个老判断：simulation 的目标是 physics essence，而不是 detail。也就是说，diagnostics 的价值不在于“把所有字段和粒子都写出来”，而在于能否把大规模数值状态压成可解释的 observables、谱、守恒量和 reader-side 证据。对二维和三维模型，这种 diagnostics / visualization / postprocessing 的难度甚至可能不低于模型本身。WarpX 的 full diagnostics、reduced diagnostics、back-transformed diagnostics、checkpoint 以及 openPMD/plotfile reader-side analysis，都不该只按 writer 类型分类，而应按“是否真正提炼出目标 physics”来理解。

同一篇综述还给了 diagnostics 的另一条很有价值的组织方式：先分 measurements related to particle motion，再分 measurements related to waves。前者典型的是 distribution function、phase space、drag、velocity diffusion；后者典型的是 field fluctuation level、time correlations、power spectrum 与 nonuniform-plasma normal modes。这种分法比“plotfile/reduced/openPMD/BTD”更接近物理问题本身，因为它直接对应读者真正要问的量：是想测输运系数、相关时间、噪声底、谱线，还是想重建某个本征模的空间结构。后面各案例如果只停在“输出了哪类文件”，而不说明它到底在测哪一类物理量，diagnostics 章节就会失焦。

Dawson 1983 后面的统计理论 examples 又把这条 diagnostics 思路压得更具体：这些 drag、diffusion、field-fluctuation 和 correlation measurements，不只是“可以输出的量”，而是 simulation 用来直接检验 subtle plasma statistics 的观测合同。作者甚至特意把一维 electrostatic sheet model 提出来当高精度 benchmark，因为它不需要 grid、可把 point-particle dynamics 跟到近 machine accuracy。于是 diagnostics 章节里有一条很值得保留的边界：reader-side analysis 的对照对象不一定只有解析式，也可以是更 fundamental、近 exact 的 particle model。这一点对后面理解 noisy thermal backgrounds、transport coefficients 和 fluctuation measurements 特别重要。

把这条统计 diagnostics 再压实一点，Dawson 给出的最小测量合同其实已经很完整：

- drag
 - 不是看单粒子轨道，而是固定窄速度窗口后测群体平均速度衰减；
- velocity diffusion
 - 不是任意时段都能读系数，而要先识别 τ^2 的 short-time regime 和 decorrelation 后的近线性 regime；
- field fluctuations
 - 不是先看整张场图，而是先看每个 k mode 的 time-averaged modal energy 是否满足热平衡与 shape-modified fluctuation 预期。

这条合同对本书案例的意义很直接：后面不论是 uniform_plasma 的 noisy thermal background、Langmuir family 的 fluctuation floor，还是 thermal-plasma energy/stability families，都更应该被组织成“这些 reader-side measurements 能否稳定恢复理论里真正关心的统计量”，而不是“导出了哪些字段文件”。

如果再往前推进一层，Dawson 的 wave-side diagnostics 还要求继续区分：

- power spectrum:
 - 是 Debye-cloud random continuum 还是 collective plasma spike；
- time correlations:
 - 对应的 wave memory / decorrelation time 多长；
- magnetized peaks:
 - 是 Bernstein、upper-hybrid、ion-cyclotron、lower-hybrid，还是 $\omega = 0$ 的 convective-cell / charged-flux-tube 结构。

这对本章的直接约束是：thermal / noisy plasma diagnostics 不该只停在 field RMS 或总场能量上，而应继续追问谱线形状、linewidth、相关时间和 peak taxonomy。否则我们只能知道“有噪声”，却不知道噪声究竟来自随机 continuum、热平衡模、磁化谐波，还是低频结构化 cells。

对 nonuniform plasma，Dawson 又把这条 diagnostics 合同推进了一步：reader-side analysis 的目标不只是标出某个 ω 上“有一条峰”，而是重建该峰对应的空间波函数。做法是先记录 $\phi(\mathbf{r}, t)$ 、 $\mathbf{E}(\mathbf{r}, t)$ 或 $\mathbf{B}(\mathbf{r}, t)$ ；若系统在某个方向上均匀，就先沿该方向 Fourier 分解，再在剩余坐标上分析 $\phi(k_x, y, \omega)$ 这类量。对离散谱线 ω_1 ，可以把信号分别与 $\sin \omega_1 t$ 和 $\cos \omega_1 t$ 做相关积分，从而恢复 mode amplitude 和 phase profile。

这里有个很硬的 measurement boundary: 积分窗口 T 必须短于该 mode 的 damping time, 否则初始 coherent oscillation 衰减后, 由随机粒子运动重新激发的任意相位会把空间相位结构洗掉; 长运行应拆成多个短窗口再平均, 而不是简单延长一次积分。对连续谱也不能一概当噪声处理, 因为其中既可能出现局域在某一小块等离子体区域的 localized oscillations, 也可能只是 random particle motion 的 continuum; 后者就必须继续测 $\delta v(\mathbf{v}, x, \omega)$ 这类 kinetic quantity, 而不能只停在势场或电场谱图。

这一点又和 noisy start / quiet start 的工程边界连在一起。Dawson 明确指出, 对 weak instability, random start 的主要问题不只是“图更吵”, 而是它会直接限制增长率测量的动态范围: 给定 k 模的初始涨落通常是 $N^{-1/2}$ 量级, 而弱不稳定最终可能只长到不到百分之一到几个百分点, 于是总共可用的指数增长窗口只有有限的 γt 。作者给出的数量级判断是 $\gamma t \sim \frac{1}{2} \ln N$; 即便 $N = 10^5$, 典型也只有大约 5 个 e-foldings, 因此增长率往往只能测到二十个百分点量级, 对更弱的不稳定性甚至会被 natural noise 直接淹没。更具体地说, 纯随机空间加载还会强烈过激发 small- k long-wavelength electrostatic modes, 因为它没有体现 Debye shielding 和局域电中性; 这说明 quiet-start 或 cell-neutral loading 的意义不只是“让初值更平滑”, 而是把 weak-effect measurements 的可识别动态范围从噪声底里救出来。

Dawson 统计诊断链: 从 modal energy 到 normal-mode reconstruction

Dawson 1983 的统计理论 examples 还给出了一条可以直接移植到现代 reader-side analysis 的 wave-side 诊断链。第一层不是把整张场图压成一个 RMS, 而是对每个 Fourier mode 计算 time-averaged modal energy; 第二层把同一 mode 的时间序列变成 power spectrum, 用来区分随机粒子运动形成的连续谱和 collective plasma oscillation 形成的离散尖峰; 第三层计算时间相关函数, 测量 phase memory 和 decorrelation; 第四层在非均匀等离子体中重建 mode 的空间波函数。四层分别回答“能量有多大、是哪类频率结构、记忆持续多久、空间上究竟是哪一个本征模”。

对单个波数 k , 相关函数可写为

$$C(k, \tau) = \lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T E(k, t) E(k, t + \tau) dt,$$

其对应的谱密度满足 Wiener-Khinchine 型关系

$$G(k, \omega) = 4 \int_0^\infty C(k, \tau) \cos(\omega \tau) d\tau.$$

因此 power spectrum 和 time correlation 不是两个互不相关的后处理图, 而是同一 fluctuation process 的频域/时域表示。有限 run length T 还给出不可绕过的频率分辨率边界 $\Delta\omega \simeq 1/T$: 如果 $1/T$ 大于目标谱线宽度, 所谓 peak width 主要是窗函数和有限样本造成的, 不能直接当成物理 damping rate。对长期运行, 应按多个短窗口分别估计, 再把统计量汇总, 而不是盲目延长一个相位已经失真的积分窗口。

在磁化等离子体中, peak taxonomy 本身也是物理结果。Bernstein harmonics、upper-hybrid、ion-cyclotron、lower-hybrid, 以及 $\omega = 0$ 附近的 convective-cell / charged-flux-tube 结构, 不能统一归类为“噪声峰”; 它们需要结合外磁场、species mobility 和空间结构共同解释。对非均匀等离子体, 若沿均匀方向先做 Fourier 分解, 可在剩余坐标上构造 $\phi(k_x, y, \omega)$; 再把离散频率 ω_1 的信号分别与 $\sin \omega_1 t$ 、 $\cos \omega_1 t$ 做相关积分, 就能恢复 mode amplitude、phase 和空间波函数。这里的积分窗口必须短于该 mode 的 damping time, 否则初始 coherent oscillation 衰减后, 随机粒子运动重新激发的任意相位会把空间结构洗掉。

continuous spectrum 也不能自动当成无意义的背景。它可能包含局域在某一小块等离子体区域的真实振荡, 也可能只是随机粒子运动的 continuum; 后者需要进一步观察 $\delta v(\mathbf{v}, x, \omega)$ 等 kinetic observable, 而不是只凭势场或电场谱下结论。对 weak instability, 随机初态的 $N^{-1/2}$ 模涨落还会消耗可用的指数增长窗口, 数量级上 $\gamma t \sim \frac{1}{2} \ln N$; quiet-start / cell-neutral loading 的价值因此是提高可识别动态范围, 而不是保证所有后续演化都更物理。上述统计链来自 Dawson 1983 的相关章节; 它支撑的是 diagnostics 设计原则, 不替代 WarpX 各案例已有的具体 runtime gate。此外, Birdsall 1985 的 13-6 提醒我们: 即使线性介电关系显示稳定, 相对漂移仍可能把自由能转入非线性相空间 clump 与 density hole; 因此接近稳定阈值的案例还应保留 phase-space correlation 观察, 而不能只看场能量或把它直接归类为 NCI。

再往实现层压一步, Dawson 给的 quiet-start recipe 也不是抽象建议, 而是明确的 phase-space construction: 把相空间切成 cells, 把每个空间 cell 内的目标速度分布 $P(\mathbf{v})$ 归一到该 cell 的粒子数, 再把 $P(\mathbf{v})$ 分成等面积小区间, 每个区间放一个粒子并赋予相应代表速度。对任意目标分布, 还可以先构造 cumulative map $\mathbf{y}(\mathbf{v}) = \int_{-\infty}^{\mathbf{v}} P(\mathbf{v}') d\mathbf{v}'$, 再用其反函数把 $[0, 1]$ 上的均匀变量映射成所需速度分布。这说明 diagnostics 一侧讨论 noisy/quiet starts 时, 不能只写“quiet start 降噪”, 还要看到它真正交换掉了什么: 它用更规则的有限粒子 phase-space covering 换取更大的 weak-effect dynamic range, 但简单的 equal-area placement 对 tail 或低密度关键区域的分辨能力有限, 于是后面才需要 weighted particles / many-size electrons 继续补这条短板。

Langmuir wave

入口: Examples/Tests/langmuir/inputs_test_1d_langmuir_multi

关键设置:

- `max_step = 80`
- `geometry.dims = 1`
- `boundary.field_lo = periodic`
- `boundary.field_hi = periodic`
- `algo.field_gathering = energy-conserving`
- `algo.current_deposition = esirkepov`
- 电子和正电子两个 species, 密度 $n_0 = 2.e24$

这个例子适合检查等离子体振荡频率、沉积和 Gauss 定律误差。输入中定义

$$\omega_p = \sqrt{\frac{2n_0 e^2}{\epsilon_0 m_e}},$$

这里的因子 2 来自电子和正电子两种可动带电粒子的对称贡献。动量微扰用正负相反的三角函数给出, 使两个 species 产生相反响应。

图 8-1 给出 81 个快照序列末态的数值电场与解析场对照。两幅图使用同一纵轴尺度: 左图为 simulation, 右图为 theory。这个图只承担 reader-side 的波形 sanity check; 严格的场误差、最终 Gauss-law 误差和频率拟合数值仍以紧随其后的 gate 为准。

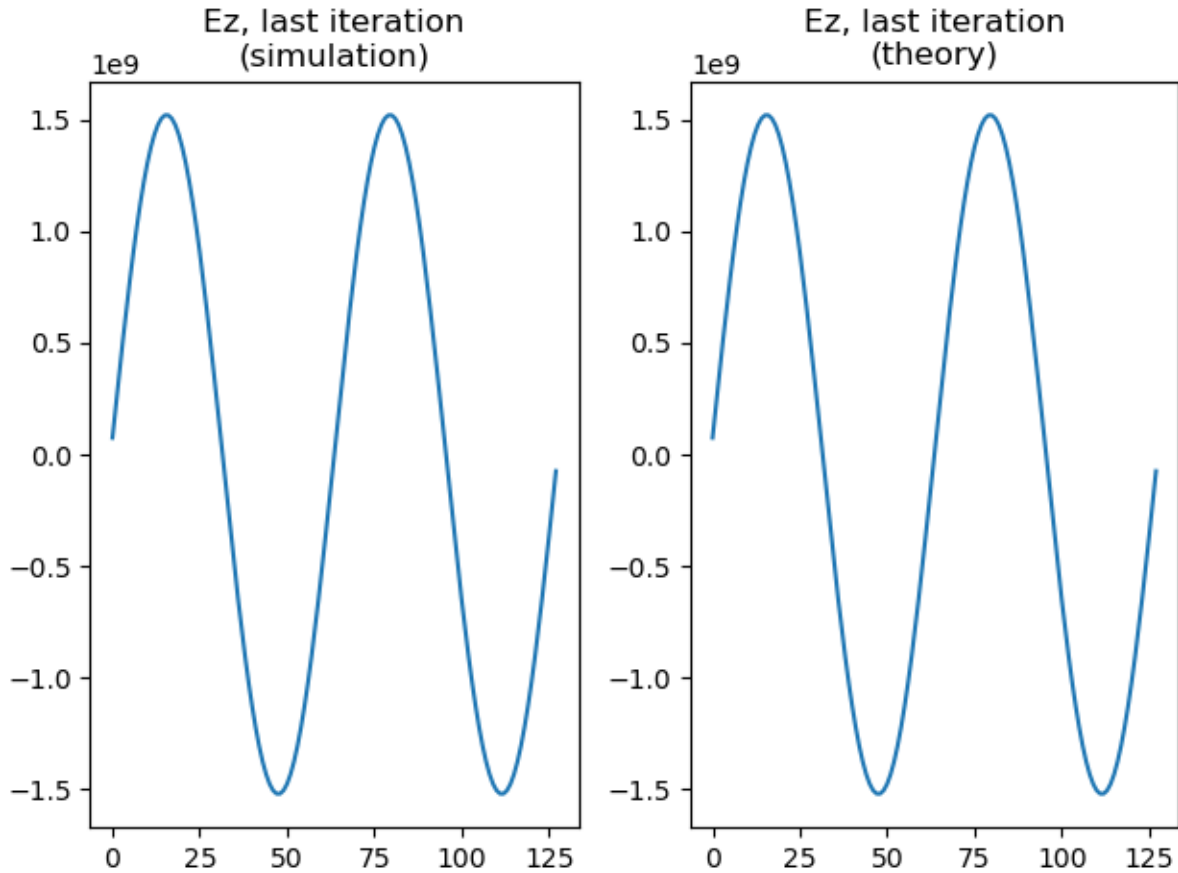


图 8-1 由官方 Langmuir 输入和 reader-side analysis 生成; 发布图像位于 `manuscript/assets/figures/`, 正文不依赖临时运行目录。

Langmuir 验证树已经比这个 1D 入口更大。1D/2D/3D/RZ 原生输入族分别复用 `analysis_1d.py`、`analysis_2d.py`、`analysis_3d.py`、`analysis_rz.py`, 因此共享同一个“解析场解逐点比较”的主合同; 其中 3D 版本还额外检查 selective particle output 和 openPMD 粒子位置上的 $E_x/E_y/E_z$ 场采样。`analysis_utils.py` 又把 charge-conservation 检查做成条件分支, 只在 Esirkepov、Vay deposition 或 PSATD current-correction 这些适用组合下强制比较 $\nabla \cdot \mathbf{E}$ 与 ρ/ϵ_0 。与之并列的 `langmuir_fluids` 则是另一棵冷流体验证树: 它不只看 E , 还把 J 和 ρ 一起与解析冷流体解比较。需要单独记住的是, 2D/3D/RZ 的 PICMI 变体目前大多仍是 `analysis=OFF` 的前端 + checksum scaffold, 不应和原生输入的强大物理断言混成同一等级。

从应用综合章的角度, Langmuir wave 的价值不只是“有一个 textbook 解析解”, 而是它把四条核心数值主线挂到了同一个最小物理问题上:

1. 初始化
 - NUniformPerCell
 - profile = constant
 - parse_momentum_function 共同决定冷等离子体微扰如何进入粒子。
2. 粒子推进与沉积
 - energy-conserving gather
 - Esirkepov、Vay deposition
 - momentum-conserving gather 在这个最小问题上暴露 $\nabla \cdot \mathbf{E} - \rho/\epsilon_0$ 误差。
3. 场求解
 - FDTD
 - PSATD
 - current_correction
 - JRhom 都能在同一解析波形下做最小分支验证。
4. 诊断与 reader-side analysis
 - plotfile/full diagnostics
 - openPMD
 - selective particle output
 - on-particle Ex/Ey/Ez 都已经现有 analysis 脚本消费。

这也是为什么 Examples/Tests/langmuir/ 不只是一个普通回归目录，而是应用综合章最适合先理解的第一条主线。它把：

冷等离子体解析振荡

```
-> parser 初始化
-> gather / pusher / deposition / solver
-> diagnostics / openPMD reader
-> MR / PSATD / current correction / JRhom / PICMI / fluids 分支
```

压在了同一个最小问题上。

归档的 1D 运行产物表明，这条主线不只停在源码和 analysis 脚本层；它产生 diags/diag1000080，并可用同一组解析式复核核心断言：

- 解析场相对误差 error_rel = 1.70e-3 < 5e-2
- $\nabla \cdot \mathbf{E} - \rho/\epsilon_0$ 相对误差 8.35e-12 < 1e-11

因此 Langmuir wave 是运行级强基准，而不只是“源码上看起来应该能验证”的基准。读者可从 Examples/Tests/langmuir/inputs_test_1d_langmuir 出发，用与所用 WarpX 版本匹配的 build 和 analysis_1d.py 重建同一条验证链；后文的频率拟合说明补充了逐快照证据。

Uniform plasma

入口：Examples/Physics_applications/uniform_plasma/inputs_test_2d_uniform_plasma

关键设置：

- max_step = 10
- geometry.dims = 2
- 周期场边界
- 单电子 species
- 常密度 1.e25
- gaussian 动量分布, ux_th = uy_th = uz_th = 0.01

这个例子更适合检查并行分解、粒子噪声、诊断输出和性能路径。因为物理结构简单，任何明显的非均匀场增长、粒子丢失或能量异常都容易被发现。

但 uniform_plasma 目录里的 regression 边界需要写得更精确。test_2d_uniform_plasma 和 test_3d_uniform_plasma 在 CMakeLists.txt 中都没有独立 analysis，实际只依赖顶层 Examples/analysis_default_regression.py 提供 checksum 基线；因此它们更像是“full diagnostics / 并行噪声 / 最小 workflow 稳定性”基准，而不是独立的热等离子体物理 hard assert。真正的强断言在 test_3d_uniform_plasma_restart：它从 chk000006 恢复，再用 Examples/analysis_default_restart.py 逐字段比较 restart 与非 restart 输出，要求相对误差低于 1e-12。另外，名字里带 uniform_plasma 的 test_3d_uniform_plasma_psatd_JRhom_CC1 并不属于这个应用目录，而是 nci_psatd_stability 里的 PSATD 稳定性回归，它检查的是 JRhom=CC1 + div cleaning 后电场能量是否足够小，从而证明 NCI 被压制，而不是均匀热等离子体本身的统计性质。

这意味着 uniform_plasma 在应用综合章里最准确的角色不是“单一 physics benchmark”，而是最小热背景 workflow：

1. 均匀、周期、单 species 的 thermal background

- 给粒子噪声和并行划分提供最低复杂度基线;
2. full diagnostics
 - 给 plotfile/openPMD 风格输出提供最小 reader-side 骨架;
 3. checkpoint/restart
 - 给 analysis_default_restart.py 提供一条极干净的 field-level reproducibility 基准。

若把“噪声、能量、性能和诊断”拆开，证据来源其实是分层的：

- 噪声、性能背景、writer/checkpoint:
 - 主要来自 Examples/Physics_applications/uniform_plasma/
- 热等离子体总能量强断言:
 - 主要来自相邻 Examples/Tests/energy_conserving_thermal_plasma/
- PSATD/JRhom 稳定性强断言:
 - 主要来自相邻 Examples/Tests/nci_psatd_stability/

因此 uniform_plasma 的真正价值，不是它自己包含了所有强 analysis，而是它把：

均匀热背景

-> 粒子噪声 / 并行稳定性
 -> full diagnostics / checkpoint
 -> restart reproducibility
 -> 与能量守恒、PSATD 稳定性测试树的边界

压成了本书第二条适合系统学习的应用主线。

归档的 2D 运行从 Examples/Physics_applications/uniform_plasma/inputs_test_2d_uniform_plasma 生成 diags/diag1000010。它说明最小 workflow 可以落盘，但这里必须保持验证分级：

- 主程序运行成功，只能证明 workflow、writer、最小噪声背景和输出路径正常；
- 官方 regression 本来就只有 analysis_default_regression.py --path diags/diag1000010 这一层 checksum；
- 这条 2D case 的现有证据不包含独立的 openPMD 读取验证；需要该格式时，应按后文的 openPMD reader 案例另行执行 consumer。

因此 uniform_plasma 的证据等级应表述为：

- 已有运行级 baseline；
- 尚不是独立物理解强断言；
- 它的强 physics closure 仍需借相邻 restart、energy_conserving_thermal_plasma 和 nci_psatd_stability 三棵树来补齐。

Checkpoint/restart 的运行证据

对同一组 3D 输入，基线从第 0 步推进到第 10 步，并在第 6 步写出 diags/chk000006；restart sibling 从该 checkpoint 继续推进到第 10 步。两条路径最终都写出 diags/diag1000010，可据此直接比较末态。

官方 Examples/analysis_default_restart.py 与独立 reader-side 比较都会对两个末态 plotfile 的 level-0 covering grid 逐字段比较。共比较 37 个 field，包含 Bx/By/Bz、Ex/Ey/Ez、jx/jy/jz、rho，以及 electrons 的粒子位置、动量、权重和粒子 ID；独立对照的最大绝对误差为 $2.4414\text{e-}4$ ，最大相对误差为 $2.8631\text{e-}16$ ，通过官方 $1\text{e-}12$ 容差。绝对误差来自量纲较大的场/电流数组，不能脱离相对误差单独解释。

这一证据有两个必须同时保留的边界。第一，它直接证明的是 checkpoint 状态恢复、粒子/场续跑和末态 diagnostics 的 reproducibility，不是热平衡能量守恒或某个解析波的 physics gate。第二，WarpX 的 CMake 注册把该测试配置为 2-rank MPI；使用 MPICH mpiexec -n 2 按官方兄弟目录布局执行后，官方 analysis_default_restart.py 对 37 个 field 全部通过，独立 reader-side 对照的最大相对误差为 $2.8631\text{e-}16 < 1\text{e-}12$ 。但仓库 checksum API 的 rank-specific 聚合参考与 2-rank producer 不一致，最大相对差为 $3.20\text{e-}2$ ；因此这里应写成“2-rank restart reproducibility evidence”，同时保留 checksum 非通过边界，不能把逐字段 restart pass 扩大成 checksum pass。

为解释这一 checksum 边界，可在相同输入下分别生成 1-rank、2-rank 的非-restart 基线，再比较相同时间的输出。两套 producer 的粒子总权重完全一致；field energy 相对差为 $1.9379\text{e-}2$ ，particle kinetic energy 相对差为 $8.9170\text{e-}4$ ，total energy 相对差为 $6.2269\text{e-}4$ ，physical-field 最大 L2 相对差为 1.0185。因此该 thermal/randomized uniform-plasma case 的 rank-invariant field contract 明确不成立，checksum 差异不能被解释成 restart 失败；只有 2-rank restart 的 plotfile-to-plotfile 一致性可写成通过证据。

图 8-11 将这条边界压成两个可读的面板：左图显示 2-rank 相对 1-rank 的全局能量比，右图显示 B/E/J/rho 各组 physical field 的最大 L2 相对误差。左图说明粒子动能和总能量仍接近，但右图说明逐场 rank-invariant gate 并未成立；虚线是参考值或 $1\text{e-}12$ machine-level gate，不

是本案例已经通过的物理阈值。

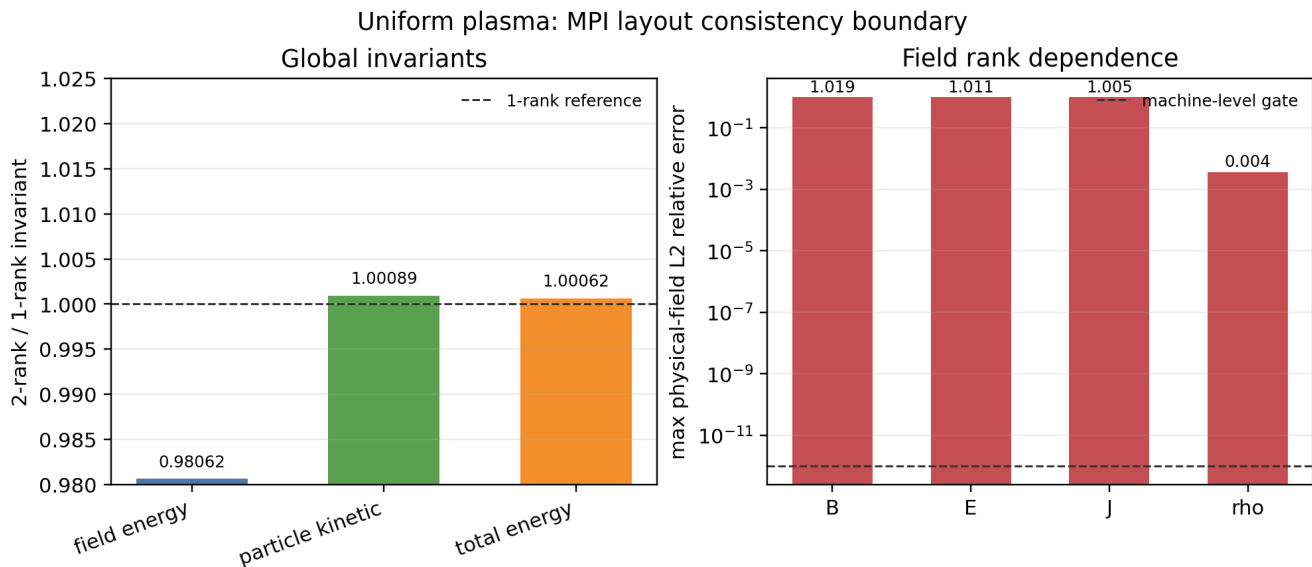


图 8-11 由同一组输出的 reader-side 比较重建；该图展示的是并行证据边界，不是新的强 physics benchmark。

LWFA/PWFA

应用综合章的下一条主线不应再写成单独的 `plasma_acceleration`。在案例库中，更准确的组织方式是把：

- `Examples/Physics_applications/laser_acceleration/`
- `Examples/Physics_applications/plasma_acceleration/`

并排视作同一类 wakefield acceleration runtime architecture 的两个分支：

1. LWFA
 - laser-driven
2. PWFA
 - beam-driven

它们共享的不是统一的 physics hard assert，而是非常相近的工程关注点：

- moving window
- boosted frame
- diagnostics
- mesh refinement
- PICMI/native front-end split

这一节现在还需要保留一条更早的文献边界：当前已经开始精读的 Tajima-Dawson 1979 并不是现代 `laser_acceleration` family 的 analysis blueprint，而是这条应用线最早期的 scaling baseline。它把 `laser pulse` -> `ponderomotive wake` -> `trapping` -> `acceleration` 这条最小物理闭环压得很硬，并给出

$$v_p = v_g^{EM} = c \sqrt{1 - \frac{\omega_p^2}{\omega^2}},$$

$$L_t = \frac{\lambda_w}{2} = \frac{\pi c}{\omega_p},$$

$$eE_L \cong mc\omega_p, \quad \gamma_{\max} \simeq 2 \frac{\omega^2}{\omega_p^2}, \quad l_a \cong 2 \frac{\omega^2 c}{\omega_p^3}.$$

这些式子最适合在本章里承担 LWFA earliest scaling baseline 的角色：它们解释为什么 wake phase velocity、dephasing、加速长度和 underdense-plasma driver 是同一条物理主线；但它们并不直接验证当代 WarpX 的 moving window、boosted frame、mesh refinement、openPMD 或 PICMI 前端实现。

同时，这篇文章也不是只有解析公式。当前精读已经确认它还给出了一个最小 relativistic electromagnetic PIC demonstration: 1 1/2-D、one spatial dimension、three velocity/field dimensions、Gaussian finite-size particles、固定离子背景，并通过扫描 ω/ω_p 去对照最早期 scaling。文中数值结果至少压了三件事：

1. wake longitudinal field 可达到

$$E_L \sim 0.6 \frac{mc\omega_p}{e},$$

即冷等离子体 wave-breaking 级上限的大约 60%；

2. driver spectrum 会裂成多峰，作者明确解释为 successive / multiple forward Raman scattering，并把它和 photon deceleration、wake emission 联系起来；
3. simulation 中的最大电子能量随 $(\omega/\omega_p)^2$ 的变化基本贴合解析式，只是在高端开始受有限系统大小和周期边界污染。

这进一步说明：Tajima-Dawson 1979 能作为 LWFA 的 earliest scaling 与 minimal EM-PIC demonstration 文献入口，但它仍不能替代现代 WarpX laser_acceleration family 的 runtime regression 合同。

文末还要再保留三条降级边界。第一，原文的 feasible within present-day technology 只是 1979 年语境下的工程可行性判断，后面立刻又承认 short-pulse shaping 仍需改进。第二，作者明确保留了 $\Delta\omega = \omega_p$ 的 two-laser / beat-wave alternative，因此这篇文章更准确地支撑的是早期 wakefield family，而不是今天单一路径的单脉冲 LWFA。第三，pulsar atmosphere / cosmic-ray source 的段落只应当看作历史语境下的 speculative extrapolation，不能进入现代 WarpX 应用合同。

LWFA: laser_acceleration 是 runtime matrix, 不是统一 wake benchmark

laser_acceleration/README.rst 明确写的是 laser-wakefield acceleration，但 Analyze 章节仍是 TODO。配合 CMakeLists.txt 可以看出，当前这组目录更像是：

- 1D/2D/3D/RZ
- boosted frame
- moving window
- refined patch
- PICMI / Python callback
- openPMD diagnostics

这些路径的 LWFA runtime matrix。

当前只有三条局部强 analysis：

1. analysis_1d_fluid_boosted.py
 - 检查 boosted 1D 冷流体 $E_z/J_z/\rho/V_z$ 是否贴理论；
2. analysis_refined_injection.py
 - 检查 refined injection 的总粒子数和 refinement-edge 前方 ρ 均匀性；
3. analysis_openpmd_rz.py
 - 检查 RZ openPMD diagnostics 的 mesh shape、species ordering 和 $\rho_{<species>}$ 物理中心。

其余大多数 active tests 都是 analysis = OFF 加 checksum baseline。因此，当前 laser_acceleration 目录更适合在本章承担：

- moving-window / boosted LWFA skeleton
- diagnostics / openPMD / MR / PICMI 路径覆盖

而不是完整的 wake amplitude、beam energy gain 或 laser diffraction 强 benchmark。

PWFA: plasma_acceleration 是 workflow matrix, 不是解析 wake benchmark

plasma_acceleration/README.rst 明确写的是 beam-driven wakefield acceleration，而不是 generic laser-plasma acceleration。这一点非常重要，因为它决定了这里的 driver 不是 laser antenna，而是 relativistic bunch。

当前目录的另一个硬边界是：

- Analyze 仍是 TODO
- Visualize 仍是 TODO
- 3D PICMI boosted-frame 等价性也被 README.rst 自己标成 TODO

同时, CMakeLists.txt 中所有 active tests 都是:

```
OFF # analysis
"analysis_default_regression.py --path ..."
```

因此 plasma_acceleration 当前最准确的角色是:

- PWFA workflow matrix
- beam-driven wakefield application baseline

它覆盖了:

- moving window
- boosted frame
- rigid bunch
- density ramp / plasma channel
- particles.use_fdttd_nci_corr = 1
- mesh refinement
- hybrid grid
- PICMI front-end

但当前并没有目录内统一的 wakefield physics hard assert。尤其要保留一个源码树边界: 3D PICMI 输入文件目前仍未像 native 输入那样真正使用 boosted frame, 所以不能把它说成 native boosted PWFA 的等价前端。

LWFA/PWFA 案例能够支持的结论

将 LWFA/PWFA 重新收束后, 案例证据支持的最强结论是:

1. laser_acceleration
 - 是 LWFA runtime matrix
 - 强 analysis 只覆盖局部合同
2. plasma_acceleration
 - 是 PWFA workflow matrix
 - 当前 active tree 全部 checksum-only
3. 二者共享的主线不是统一 benchmark, 而是:
 - moving window
 - boosted frame
 - diagnostics
 - MR
 - PICMI/native front-end split

这也意味着, 后续书稿如果从应用角度组织 wakefield acceleration, 最自然的章节结构不是简单按目录分, 而是:

```
wakefield acceleration runtime architectures
-> laser-driven branch (LWFA)
-> beam-driven branch (PWFA)
```

Laser ion / plasma mirror / RPA/TNSA

这一条应用主线必须写得比目录名更谨慎。案例库中真正可落到 application tree 的 laser-target 入口只有两个:

- Examples/Physics_applications/laser_ion/
- Examples/Physics_applications/plasma_mirror/

而 RPA/TNSA 当前并没有独立应用目录或回归树, 只是:

- laser_ion/README.rst 背后的物理机制标签;
- Docs/source/glossary.rst 里的术语定义。

laser_ion: 最强的 laser-target application entry

laser_ion 的真实角色不是“已经证明某条 ion-acceleration scaling”, 而是:

- Gaussian laser

- planar solid-density target
- full diagnostics
- time-averaged diagnostics
- reduced diagnostics
- PICMI front-end

这条组合工作流的应用入口。

当前它在 CI 里的最硬断言来自 `analysis_test_laser_ion.py`, 检查的是:

- `diagInst` 最后 5 个瞬时 Ez snapshot 的时间平均
- 与 `diagTimeAvg` 的原位 time-averaged Ez 是否逐点一致

因此它当前最强的 regression 合同是:

- diagnostics time-average consistency

而不是:

- TNSA cutoff energy
- RPA threshold
- ion conversion efficiency

README 里的 `analysis_histogram_2D.py` 和 `plot_2d.py` 仍然很重要, 但它们当前属于 user-facing post-processing helper, 不是活跃 CI regression 本体。

还要再保留一个细边界: `laser_ion` 确实已经有 PICMI 版输入, 但其 reduced diagnostics 能力和 native 版并不完全对齐, 例如 PICMI 脚本里仍留有 `ParticleHistogram2D` 的 TODO。因此更准确的说法是:

- PICMI 已覆盖主工作流与 `analysis_test_laser_ion.py` 合同;
- 但前端能力还没有完全追平 native input。

plasma_mirror: laser-solid surface-plasma workflow baseline

`plasma_mirror` 当前应用语义很明确:

- laser-solid interaction
- surface plasma
- planar overdense target

但验证层级明显更弱:

- 只有 `test_2d_plasma_mirror`
- `analysis = OFF`
- checksum helper
- 没有 PICMI
- README.rst 的 Analyze/Visualize 仍是 TODO

因此它更准确的角色是:

- laser-solid surface-plasma workflow baseline

而不是:

- reflectivity benchmark
- high-harmonic benchmark

RPA/TNSA: 当前属于物理解释层, 不属于独立应用目录层

这条边界如果不写清, 很容易把文献中的机制标签误写成案例库已有的独立 examples。最强、也最保守的结论只能是:

1. `laser_ion`
 - 是激光打固体平面靶的应用骨架;
2. RPA/TNSA
 - 是理解这类骨架时需要引入的机制标签;
3. 当前 Examples/ 中

- 没有独立 `rpa_*`
- 也没有独立 `tnsa_*` application tree。

因此，这条应用综合章更准确的组织方式是：

```
laser-target applications
-> laser_ion
-> plasma_mirror
-> RPA/TNSA as mechanism labels, not standalone application trees
```

Capacitive discharge

这条应用线不应混成普通 `collision/*` 附属条目。它更准确的角色是：

- PIC-MCC low-temperature plasma application tree

因为它同时把以下对象接到同一应用骨架上：

- parallel-plate electrostatic discharge
- `background_mcc`
- 可选 DSMC ionization 分支
- PICMI front-end
- Python callback Poisson solver
- Turner benchmark profile 对照

1D PICMI 是当前最强的 Turner benchmark 入口

当前最强的两条 active tests 是：

- `test_1d_background_mcc_picmi`
- `test_1d_dsmc_picmi`

它们共用同一条脚本：

- `inputs_base_1d_picmi.py`

这不是普通薄输入卡，而是完整的 benchmark driver：

1. 选择 Turner case N=1..4
2. 组装 1D electrostatic grid
3. 可选安装 Python level Poisson solver callback
4. 打开 `background_mcc`
5. 可选把 ionization 切成 DSMC
6. 累积离子密度并写出 `ion_density_case_N.npy`

当前 `CI --test --pythonsolver` 模式下还会显式确认：

- callback solver 已经实际运行；
- `he_ions` 的 `z` 坐标访问链可用。

因此这条应用树在工程上也不只是低温等离子体 benchmark，同时还是清晰的：

- PICMI + Python callback Poisson solver

应用入口之一。

当前最硬断言是 case-1 ion-density profile

`analysis_1d.py` 和 `analysis_dsmc.py` 当前都直接读取：

- `ion_density_case_1.npy`

并与内置 Turner case-1 参考离子密度 profile 做 `allclose`。

因此该案例最强的 physics contract 是：

- final averaged ion density profile

- against Turner benchmark case 1

而不是笼统的“有 MCC test”或“有 DSMC test”。

DSMC 版不是孤立小 test, 而是同一 benchmark scaffold 的分支

这两条 1D tests 共享同一应用骨架, 区别只在于 collision realization:

1. `test_1d_background_mcc_picmi`
 - `background_mcc`
 - external Python Poisson solver callback
 - Turner case-1 profile 对照
2. `test_1d_dsmc_picmi`
 - 把 ionization 切到 DSMC 分支
 - 仍回到同一 Turner case-1 profile 对照

因此更准确的表述是:

- DSMC 分支已经在同一低温等离子体 benchmark scaffold 里被强对照覆盖

而不是只证明“DSMC can run”。

2D native / PICMI 当前仍主要是 workflow baseline

与 1D 强对照相比, 当前 2D 分支只有:

- `test_2d_background_mcc`
- `test_2d_background_mcc_picmi`

并且两者都是:

- `analysis = OFF`
- checksum helper

因此它们当前只能诚实记成:

- 2D capacitive-discharge workflow baseline

而不是 2D Turner 强 benchmark。另一个必须保留的边界是:

- `test_2d_background_mcc_dp_psp`

当前整条 `add_warpx_test(...)` 仍被注释掉, 所以它只能作为遗留分支记录, 不能再冒充活跃 test。

既有 plasma_acceleration 目录边界

入口: `Examples/Physics_applications/plasma_acceleration/inputs_test_3d_plasma_acceleration_boosted`

这一组也需要避免被过度解读。plasma_acceleration family 在 `CMakeLists.txt` 中所有活跃 tests 都是 `analysis = OFF`, 只复用目录内的 `analysis_default_regression.py` 做 checksum。因此它们不是“PWFA 解析 benchmark”, 而是应用 workflow 基线。

但它们并不空泛。原生输入和 PICMI 输入合起来, 已经覆盖了:

- moving window
- boosted frame
- rigid-injected driver/beam
- `particles.use_fdt_nci_corr = 1`
- level-1 refined patch / `add_refined_region(...)`
- momentum-conserving gather 分支
- `grid_type = hybrid`
- field / particle diagnostics

因此这组例子当前真正承担的角色是: 给 beam-driven wakefield acceleration 的 runtime matrix 保留稳定输出基线, 而不是直接对 wake amplitude、dephasing 或 beam loading 做强物理断言。还有一个需要显式保留的源码树边界是: `README.rst` 目前明确写着 3D PICMI 版“应该像原生输入一样使用 boosted frame, 但仍是 TODO”。所以 `inputs_test_3d_plasma_acceleration_picmi.py` 当前只能诚实记成 non-boosted PICMI scaffold, 不能误写成原生 boosted PWFA 的等价前端。

Magnetic reconnection

磁重联这条应用线最准确的入口不是一般 `Fluids/`，而是：

- `Examples/Tests/ohm_solver_magnetic_reconnection/`

它依赖的是：

- `picmi.HybridPICSolver`
- `HybridPICModel`

也就是：

- kinetic ions
- electron-fluid Ohm closure
- Faraday + RK 子步推进 B

而不是 `WarpXFluidContainer` 那条额外 cold-fluid species runtime layer。这个边界必须写死，因为两条实现承担的职责不同：

- `Fluids/`
 - 自己维护 nodal N/NU
 - gather 主场
 - 再把 ρ/J 沉积回普通场寄存器；
- `HybridPICModel`
 - 则是在 field solver 内部从总电流、离子电流和电子闭合关系反推出 E。

因此 `magnetic_reconnection` 在应用综合章里的正确角色是：

- hybrid-PIC space-plasma application

而不是：

- fluid/PIC coupling demo

force-free sheet + reduced FieldProbe

`inputs_test_2d_ohm_solver_magnetic_reconnection_picmi.py` 当前不是薄输入卡，而是完整应用 driver。它同时定义了：

- 2D Cartesian 几何
- x 周期、z 方向 `dirichlet/reflecting`
- force-free-sheet 解析初始 $B_x/B_y/B_z$
- `plasma_resistivity`
- `substeps`
- kinetic ion loading
- reduced diagnostic `FieldProbe`

其中最重要的 diagnostics 不是 full plotfile，而是：

- `plane.dat`

它来自 X 点附近的 reduced `FieldProbe`，专门供 reader-side analysis 提取重联率。

当前 analysis 是 observable extraction，不是强 assert

`analysis.py` 当前直接从 `plane.dat` 读取 E_y ，构造：

$$R(t) = \frac{\langle E_y \rangle}{v_A B_0}.$$

然后输出：

- `reconnection_rate.png`

在非 `--test` 模式下还会进一步生成：

- `mag_reconnection.mp4`

但这条脚本没有显式数值 `assert`。因此它当前只能被诚实归类为：

- `physics-informed visualization / observable extraction`

而不是：

- `hard numerical benchmark`

`checksum` 仍是 `active coverage` 的另一半

`CMakeLists.txt` 里这条 `test` 还会同时跑：

- `analysis_default_regression.py --path diags/diag1000020`

所以当前 `active coverage` 的真实结构是：

1. `analysis.py`
 - 提取重联率并可视化；
2. `checksum helper`
 - 兜底历史输出稳定性。

这也解释了它和邻近 `ohm_solver_*` 条目的分工：

- `ohm_solver_em_modes、ion_beam_instability`
 - 更偏局部 `solver correctness` 的强 `regression`；
- `magnetic_reconnection`
 - 更偏 `hybrid-PIC` 代表性物理案例和输出回归。

因此，这条应用线在当前书稿里最准确的结论应写成：

```
magnetic_reconnection
= HybridPICModel application line
= force-free-sheet + reduced FieldProbe + reconnection-rate extraction
= physics-informed visualization + checksum
!= scalar hard-assert benchmark
```

Beam-beam / luminosity / FEL / ion extraction

这一条应用综合主线最容易被误写成单一“束流例子”列表，但案例库中的四类入口其实承担着不同层级的合同：

- `DifferentialLuminosity`
- `beam_beam_collision`
- `free_electron_laser`
- `ion_beam_extraction`
- `accelerator_lattice`

更准确的组织方式应是：

```
beam and accelerator applications
-> luminosity diagnostics benchmark
-> collider-QED workflow baseline
-> FEL boosted-frame radiation benchmark
-> electrostatic ion-source extraction
-> accelerator-lattice optics regression
```

DifferentialLuminosity: reduced-diagnostic 强谱基准

`Examples/Tests/diff_lumi_diag/` 当前是这条应用线里最强的 `diagnostics regression`。它的 `analysis.py` 不是普通画图脚本，而是同时读取：

- 一维文本表 `DifferentialLuminosity_beam1_beam2.txt`
- 二维 `openPMD` 网格 `DifferentialLuminosity2d_beam1_beam2/`

然后直接构造两束 `Gaussian beams` 的解析 `luminosity` 谱：

- dL/dE
- $d^2L/dE_1 dE_2$

再与 diagnostics 做显式误差比较和 `assert`。因此它的角色必须写成：

- `reduced-diagnostic strong benchmark`

而不是一般的 beam application helper。

beam_beam_collision: collider-QED 应用骨架

与 `DifferentialLuminosity` 相比, `Examples/Physics_applications/beam_beam_collision/` 当前证据层要弱得多：

- active regression 只有 checksum helper;
- 没有独立 `analysis.py`;
- `plot_fields.py` / `plot_reduced.py` 只是后处理可视化脚本。

但它并不空泛。它当前真正把这些路径绑在一起：

- `warpx.do_electrostatic = relativistic`
- 两束 125 GeV 电子/正电子 Gaussian bunch 对撞
- `initialize_self_fields = 1`
- Quantum Synchrotron
- Breit-Wheeler
- `ColliderRelevant`
- `ParticleNumber`
- openPMD full diagnostics

因此它最准确的定位是：

- `collider-QED application baseline`

而不是 luminosity 强谱基准。

free_electron_laser: boosted rigid-beam + undulator + BTD 的强 benchmark

`free_electron_laser` 当前不是普通 laser example, 因为它本质上没有 laser antenna。源码与官方案例表明：

- 核心是 `RigidInjectedParticleContainer`
- `particles.By_external_particle_function(...)` 提供 undulator 外加粒子磁场
- `BackTransformed` diagnostics 与 boosted-frame full diagnostics 的一致性

当前最强断言来自 `analysis_fel.py`：

1. 对 $\log(E_x^2)$ 的线性增长区做拟合, 要求 gain length 接近 0.22 m;
2. 在 lab-frame 与 boosted-frame diagnostics 上做 FFT, 要求 radiation wavelength 满足 undulator 理论值。

因此它当前最准确的角色是：

- `boosted rigid-beam radiation benchmark`

这条应用线也正好和 Dawson 1983 里的经典 FEL 例子接上。那篇综述把 free-electron laser 专门当作 relativistic electromagnetic particle model 的代表问题: lab frame 下给出

$$\lambda \simeq \frac{\lambda_0}{2\gamma^2},$$

而在 beam frame 下又把同一过程重写成 pump electromagnetic wave 衰变成 electromagnetic wave 加 plasma wave 的 Raman-like 参数不稳定, 并要求满足

$$k_{\text{pump}} = k_{\text{EM}} + k_p, \quad \omega_{\text{pump}} = \omega_{\text{EM}} + \omega_p(k_p).$$

它的历史 simulation 结果还明确展示了 matching-condition 谱证据、约 36% 的 longitudinal current 下降、约 30% 的束流能量转成辐射, 以及 $2\lambda_0$ backward mode 的危险性。对本章来说, 这组文献证据的作用不是替代当前 `analysis_fel.py`, 而是把 WarpX 这条 boosted rigid-beam + undulator + BTD benchmark 放回更早的 relativistic EM-PIC 谱系里理解。

如果再把图像层次压得更明确, Dawson 1983 这条 FEL 历史线已经形成一个很完整的 diagnostics contract:

- Fig.54
 - 给最小装置和 lab-frame / beam-frame 两种物理图像;
- Fig.55
 - 用 EM / electrostatic spectra 检查 matching condition;
- Fig.56
 - 用 EM energy、electrostatic energy 和 longitudinal current 的同步演化检查 gain、beam degradation 与 saturation;
- Fig.57
 - 再把 trapping-based efficiency estimate

$$\eta = \frac{\gamma_0 - \gamma_{ph}}{\gamma_0 - 1}$$

及其大- γ 近似

$$\eta \simeq \omega_{po}(2k_0 c \gamma^{3/2})^{-1}$$

与 simulation 对照。

也就是说, 这组经典图像已经把 mechanism verification、nonlinear saturation 和 rough efficiency scaling 串成一条最小 reader-side 论证链。当前 WarpX `free_electron_laser` 的价值, 则是把这条历史论证链重新落到 boosted-frame implementation、BTD 重建和 gain-length / wavelength regression 上。

ion_beam_extraction: EB electrostatic extraction 的强应用入口

Examples/Physics_applications/ion_beam_extraction/ 当前是这条应用线里最直接的 electrostatic + embedded-boundary beam-source application。

它把:

- plasma source
- boundary.potential_*
- warpx.eb_potential(x,y,z,t)
- electrostatic solver
- embedded-boundary electrode geometry
- 持续 boundary injection

接成了一条完整链。

当前 `analysis_ion_beam_extraction.py` 会直接检查抽出离子束尾部能量是否接近 40 keV, 因此它不是 checksum baseline, 而是:

- electrostatic EB extraction strong application check

accelerator_lattice: beamline optics 的强回归层

如果只写 collider、FEL 和 extraction, 这条总节还少了一层真正对应加速器模块本身的强验证。当前最自然的入口是:

- Examples/Tests/accelerator_lattice/

这里的 `analysis.py` 会重新读回:

- lattice.elements
- line*
- drift*
- quad*

然后按解析 hard-edged quadrupole optics 逐段积分, 并要求最终粒子轨道和解析解足够接近。它同时覆盖:

- lab frame
- boosted frame
- moving window

所以它在这条总节里最准确的角色是:

- beamline optics strong regression

诊断在源码中的位置

WarpX::Evolve 中诊断不是附加脚本, 而是时间步的一部分: 每一步先由 `multi_diags->NewIteration()` 重置迭代状态; 随后根据 `DoComputeAndPack()` 和 `reduced_diags->DoDiags()` 判断是否需要同步粒子速度、计算/打包 reduced 或 full diagnostics; 最后通过 `FilterComputePackFlush()` 写出, 并在最终时间步或中断时冲刷剩余数据。源码入口为 `Source/Evolve/WarpXEvolve.cpp`、`Source/Diagnostics/MultiDiagnostics.cpp` 与 `Source/Diagnostics/Diagnostics.cpp`。

读源码时应区分三个职责入口: `Source/Diagnostics/` 负责对象生命周期和 `writer`, `Examples/Tests/` 中的 `analysis*.py` 定义具体物理或输出比较, `Regression/Checksum/` 只保存指定输出的回归基线。三者不能互相替代。

更底层地看, `Source/Diagnostics` 顶层其实分成四层角色:

1. `MultiDiagnostics` 负责读取 `diagnostics.diags_names`, 按 `<diag>.diag_type` 把每个 `diagnostics` 实例化成 `FullDiagnostics`、`BTDiagnostics` 或 `BoundaryScrapingDiagnostics`, 并在主循环里统一分派。
2. `Diagnostics` 提供统一模板骨架: `InitData()`、`FilterComputePackFlush()`、`ComputeAndPack()`、`Flush()`。也就是说, 所有 `diagnostics` 都要经过“先决定是否 `compute/pack`, 再决定是否 `flush`”的同一套阶段。
3. `FullDiagnostics` 负责把 `fields_to_plot`、`particle_fields_to_plot` 和 `species` 输出需求映射成具体 functors, 再把结果堆叠进输出 `MultiFab`。
4. `ParticleDiag` 不是真正的粒子数据缓冲区, 而是每个 `species` 的输出配置对象: 它记录变量选择、`random_fraction` / `uniform_stride` / `parser filter`、附加粒子场请求, 以及粒子来源容器指针。

一个关键实现边界是: 普通 `FullDiagnostics` 的粒子输出, 并不像 `back-transformed diagnostics` 那样先 `pack` 到独立粒子 `buffer`。它更多是把 `ParticleDiag` 作为 `species` 句柄和过滤配置传给 `writer`; 真正的粒子变量裁剪和过滤发生在 `FlushFormatPlotfile.cpp` / `WarpXOpenPMD.cpp` 的写出阶段。只有 `BTDiagnostics` 这类带 `snapshot buffering` 的 `diagnostics`, 才会真正分配 `m_particles_buffer` 和 `ComputeParticleDiagFunctor`。

这也是为什么 `Diagnostics::ComputeAndPack()` 虽然同时有 `field-functor loop` 和 `particle-functor loop`, 但对普通 `full diagnostics` 来说, 后者默认是空的。不要把“所有 `diagnostics` 都有独立粒子 `pack` 阶段”当成 `WarpX` 的统一事实。

继续往下拆, 会看到 `diagnostics` 模块其实还有另一条容易混淆的边界。 `ComputeDiagFunctors/` 是字段计算层: `JFunctor`、`RhoFunctor`、`PhiFunctor` 直接把 `j/rho/phi` 这类量写进 `diagnostics MultiFab`; `ParticleReductionFunctor` 虽然读取粒子, 但它输出的仍然是 `cell-centered MultiFab`, 因此也属于字段计算层, 而不是粒子 `writer`。

真正的粒子过滤发生在 `writer` 阶段。无论是 `WarpXOpenPMD.cpp` 还是 `FlushFormatPlotfile.cpp`, 都会先从 `ParticleDiag` 取出:

- `random_fraction`
- `uniform_stride`
- `parser filter`
- `geometry filter`

然后创建一个临时粒子容器 `tmp`, 通过 `tmp.copyParticles(...)` 把通过过滤的粒子复制进去, 再把 `tmp` 写出。因此 `ParticleDiag` 构造阶段只是记录过滤规则; 真正应用过滤是在 `flush` 的时候。

`phi`、`Ex/Ey/Ez`、`Bx/By/Bz on particles` 也不属于 `field functor` 层, 而是 `writer` 在过滤后的 `tmp` 上再做一次 `gather`。 `ParticleIO.cpp` 明确限制这些附加粒子场只允许 `diag_type = Full`, 因为对 `BackTransformed` 或 `BoundaryScraping` 这类带粒子缓冲区的 `diagnostics` 来说, 粒子被写出的时间并不等于粒子被收集的时间, 此时再 `gather` 场会产生时间层错配。

`ReducedDiags/` 又是另一套平行体系。它不继承 `Diagnostics`, 也不走 `MultiFab + ParticleDiag + FlushFormat` 这条主线, 而是由 `MultiReducedDiags` 读 `warpX.reduced_diags_names` 后, 按 `<reduced_diag_name>.type` 分派到 `FieldEnergy`、`ParticleEnergy`、`ParticleHistogram`、`FieldProbe`、`LoadBalanceCosts` 等具体类型。它们共享的核心抽象不是字段/粒子快照, 而是一段 `m_data` 向量和统一的表格写出协议: 第一列是 `step`, 第二列是时间, 后面各列是 `reduced quantity`。

这类 `reduced diagnostics` 里, 很多类型是“到点现算即写”, 例如 `FieldEnergy` 和 `ParticleEnergy`; 但也有例外, 例如 `FieldPoyntingFlux` 会维护跨时间步累积的积分量, 因此还要实现 `WriteCheckpointData()` / `ReadCheckpointData()`, 把内部状态写进 `checkpoint` 并在 `restart` 时恢复。也就是说, `diagnostics` 模块里并不是只有 `BTDiagnostics` 才有跨步状态, 部分 `reduced diagnostics` 也有。

`checkpoint format` 本身也不能等同于普通 `diagnostics` 格式。 `FlushFormatCheckpoint.cpp` 实际写出的是 `WarpX` 的 `restart state`: `E/B`、`coarse/fine patch fields`、同步后的 `current`、`PML` 数据、粒子状态, 以及 `reduced diagnostics` 额外的 `checkpoint` 数据。它并不是把某个 `diagnostics` 的 `m_mf_output` 换一种文件格式落盘, 而是直接序列化运行态。

BTDiagnostics 则是另一类“有状态机”的 diagnostics。它几乎每一步都可能执行 DoComputeAndPack(), 把 cell-centered 后的场切成一片片 lab-frame slice, 逐步填进 snapshot buffer; DoDump() 判断的也不是单纯的时间间隔, 而是“当前 buffer 是否已满”“最后一个有效 z-slice 是否已经填到”以及“结束时是否需要强制冲刷剩余 buffer”。因此 BTD 不能被理解成另一种普通 full diagnostics, 它本质上是一套 slice / buffer 的累积和 flush 机制。

再往 reduced diagnostics 的具体类型里看, 会发现它们虽然共用 ReducedDiags 骨架, 但实现形态并不统一。FieldProbe 内部维护的不是一个标量数组, 而是一套专门的 FieldProbeParticleContainer: point/line/plane 三种几何最终都会在 InitData() 中转成一批 probe particles, 再在 ComputeDiags() 里对 Efield_aux/Bfield_aux 做 gather。因此它测到的不是推进器主寄存器的原始 fp 场, 而是粒子侧真正会看到的 aux 场; do_moving_window_FP 也不是事后回推场, 而是直接平移这批 probe particles。若 integrate = 1, 它还会在每一步把采样值乘 dt 累加到 probe-particle SoA 中, 到输出步才写出累计量。

ParticleHistogram 和 ParticleHistogram2D 又是另一种 reduced diagnostics。前者是 parser 驱动的一维 weighted histogram: 对每个粒子先算 histogram_function(t,x,y,z,ux,uy,uz), 再按 floor((f-bin_min)/bin_size) 落 bin, 默认累加粒子权重 w, 最后在 MPI 归约之后再 max_to_unity 或 area_to_unity 归一化。后者虽然还挂在 ReducedDiags/ 下, 但 writer 已完全变成 openPMD mesh 输出: histogram_function_abs、histogram_function_ord 决定二维坐标, value_function 决定每个粒子向该 bin 累加什么值, 并且 writer 会连同轴标签、bin spacing、global offset 和 parser 字符串一起写进 openPMD 元数据。因此二维 histogram 不是“多一列文本表”, 而是真正的带坐标二维诊断场。

LoadBalanceCosts 和 LoadBalanceEfficiency 则属于性能/并行态 diagnostics。前者输出粒度是 box, 而不是 rank: 每个 box 会写 cost、proc、lev、i_low/j_low/k_low、num_cells、num_macro_particles, GPU 运行时还会附带 gpu_ID, 再通过额外的 MPI_Gatherv 收集 hostname。Heuristic 模式下它会先调用 ComputeCostsHeuristic() 重建 heuristic cost; Timers 模式下则直接导出当前 timers 成本。因此它真正暴露的是 WarpX load-balance 决策所依据的 box-level 负载分布, 而不只是一个抽象效率数字。LoadBalanceEfficiency 相比之下非常薄, 它只是把 warpx.getLoadBalanceEfficiency(lev) 的结果按 level 写出来, 用于快速检查某次重分配前后是否更均衡。

对应的 regression 也不是同一种口径。analysis_reduced_diags_impl.py 会从 full plotfile 重新计算 FieldEnergy、ParticleEnergy、FieldReduction 等 compact observable, 再与 reduced diagnostics 文本结果比较, 所以它主要验证 reduced diagnostics 与 full-state reference 是否一致。analysis_reduced_diags_load_balance_costs.py 则完全不读 plotfile, 而是直接从 LBC.txt 重建每个 rank 的总成本, 并只断言 load-balance 之后

$$\text{efficiency}_{\text{before}} < \text{efficiency}_{\text{after}}.$$

这说明 reduced diagnostics 在 WarpX 里既有“物理量压缩输出”的一面, 也有“把并行运行态暴露给后处理”的一面, 不能把它们都当成同一种小型文本统计表。

如果把 diagnostics 再按 writer 分成一层, 会看到 diag_type 和 format 是两套独立分派。MultiDiagnostics 先按 diag_type 把对象构造成 FullDiagnostics、BackTransformed 或 BoundaryScrapingDiagnostics; 之后 Diagnostics::InitDataBeforeRestart() 再按 format 选择:

- FlushFormatPlotfile
- FlushFormatOpenPMD
- FlushFormatCheckpoint

这三条 writer 路径虽然共用 flush 调度时机, 但服务目标已经不同。plotfile 路径会把 diagnostics 已经 pack 好的 cell-centered MultiFab 通过 WriteMultiLevelPlotfile(...) 写出; 若打开 plot_raw_fields = 1, 还会额外把原始 staggered/raw fields 写进 raw_fields/ 子目录, 因此它既能给普通分析用, 也能给底层网格调试用。openPMD 路径在 fields 侧同样写 diagnostics 视图, 但在粒子侧比 plotfile 更强: 它可以在 writer 中对过滤后的临时粒子容器再次 gather phi、Ex/Ey/Ez、Bx/By/Bz。不过这项能力只允许 diag_type = Full, 因为 ParticleIO.cpp 明确禁止对 BackTransformed 或 BoundaryScraping 这类“收集时刻和写出时刻不一致”的缓冲型 diagnostics 再去 gather 场。

checkpoint 路径则完全不是“另一种 diagnostics 文件格式”。FlushFormatCheckpoint::WriteToFile() 基本不消费 m_mf_output, 而是直接从 warpx.m_fields 序列化真实运行态:

- Efield_fp/Bfield_fp
- E_old
- synchronized current_fp/current_cp
- coarse patch fields
- time-averaged fields
- PML 数据
- 完整 species 与 lasers 粒子状态
- distribution mapping

- reduced diagnostics 的 checkpoint state

因此 checkpoint 的真正对象是 restart persistence,而不是用户筛选后的诊断视图。也正因为如此,FullDiagnostics::ReadParameters() 对 `format = checkpoint` 做了比文档更强的源码约束: 不能自定义 `fields_to_plot`、不能裁剪 `diag_lo/diag_hi`、不能做 `coarsening_ratio`、不能指定 `species` 子集,也不能开 `raw fields`。它要求的是“全量可恢复状态”,不是“最小可读输出”。

从官方 examples 看,这三类 writer 的最小输入骨架已经比较稳定:

- 普通 plotfile: 只写 `diag1.diag_type = Full` 和一组 `fields_to_plot` 即可, `format` 缺省就是 `plotfile`。
- openPMD: 在 full diagnostics 上再加 `diag1.format = openpmd` 和 `diag1.openpmd_backend = h5/bp*,laser_ion` 已经给出了带 field filtering 的最小可复用骨架。
- checkpoint: 通常并行放一个 `diag1` 和一个 `chk`, 后者写 `chk.diag_type = Full`、`chk.format = checkpoint`; 重启则用 `amr.restart = "../.../chk000XXX"` 接回。

对本章最相关的 reduced diagnostics, 也可以直接从官方 examples 抽出最小运行入口:

- FieldProbe: Examples/Tests/reduced_diags/inputs_test_3d_reduced_diags 和 `laser_ion` 都给了 point/line 的最小参数骨架。
- ParticleHistogram2D: `laser_ion` 已经给了 `histogram_function_abs/ord` 与 `value_function = "w"` 的二维相空间例子。
- LoadBalanceCosts: Docs/source/usage/workflows/plot_distribution_mapping.rst 与 Examples/Tests/reduced_diags/inputs_test_3d_reduced_diags 已经构成最小“生成 + 画图/验证” workflow。

如果要看 FieldProbe 的强 analysis regression, 官方测试树中还有一组比这些“最小骨架”更直接的条目: Examples/Tests/field_probe/. 它不是只检查文件格式, 而是把 line FieldProbe 接到单缝衍射 benchmark 上。analysis 会从 `FP_line.txt` 读出 step 500 的积分电磁通量, 再与解析 sinc^2 衍射包络比较, 并要求平均相对误差小于 2.5%。按所检验的官方输入复现后, 这条线暂时不能作为“已通过”的例子: 1-rank 和官方 2-rank MPI 配置产生完全一致的 `FP_line.txt`, 但 `analysis.py` 的平均误差都是 3.6703%, 最大选点误差为 10.0843%; 独立 reader-side 比较使用相同选点和分母, 得到相同的结论。

这里的失败本身是诊断章节需要保留的证据。它说明 reduced diagnostic 的 writer 链已经接通: 201 个 probe 点、step 500 的积分通量和 1/2-rank 一致性都成立; 但这还不足以证明 FieldProbe 的物理量与解析衍射包络一致。随后将网格从官方的 $\lambda/16$ 加密到 $\lambda/32$, 并在相同物理时间的 step 1000 取样, 官方同口径误差降为 0.3533%, 最大选点误差为 1.0414%, 通过 2.5% gate。

因此最稳妥的成书结论是: 原始 coarse case 的“输出链通过、解析 physics gate 未通过”是真实结果; 网格加密后的通过结果支持 coarse-grid 离散误差是主因, 但不能把 refined case 的结果反写成原始官方输入已通过。关闭 filter 只能把误差略降至 3.5910%, 而 `interp_order=0` 在所检验版本中产生零通量, 后者应作为另一个需要单独审计的 raw-field gather 边界, 而不是有效的物理改进方案。

图 8-3 将这条边界画成两个并排面板: 左图是官方 analysis 使用的平均误差和 2.5% gate, 右图是 40 个选点中的最大误差。颜色只表示当前报告的 pass/fail 状态; refined 的通过来自 $\lambda/32$ 、相同物理时间的 step 1000 对照, 不是对 coarse 输入的重写。



图 8-3 由同一组 FieldProbe 输出的 reader-side 比较重建。

同一层里，官方测试树中还有两组更偏束流诊断的强 regression。Examples/Tests/collider_relevant_diags/ 不是普通 reduced-output 烟雾测试，而是把 ColliderRelevant 与 ParticleExtrema 并排打开，然后用解析粒子样本逐项核对 chi_min/max/ave、theta_x/theta_y 的 min/ave/max/std，再从 full openPMD 的 rho_beam_e/rho_beam_p 重建 dL/dt 与 reduced output 交叉验证。也就是说，这组例子验证的不是“表格写出来了”，而是 collider-oriented reduced quantities 的定义和聚合合同本身。

Examples/Tests/diff_lumi_diag/ 则把 reduced diagnostics 进一步推进到带解析谱对照的束流物理量：一维 DifferentialLuminosity 文本表和二维 DifferentialLuminosity2D openPMD 网格同时输出，analysis 直接构造两束高斯束流碰撞的解析 dL/dE 与 d^2L/dE_1dE_2 ，再分别比较 1D/2D diagnostics。对本章来说，这组例子非常有价值，因为它把“reduced diagnostics 可以是纯文本列，也可以是 openPMD 网格”这件事，用同一个物理 benchmark 明确落地了。

与之相邻、但等级更弱的一组是 Examples/Physics_applications/beam_beam_collision/。这组例子同样使用 collider 场景、Quantum Synchrotron、Breit-Wheeler、ColliderRelevant 与 ParticleNumber reduced diagnostics，但当前活跃 regression 只有 analysis_default_regression.py 提供 checksum，没有独立 analysis.py。因此它更准确地是一个 collider-QED application baseline：验证 relativistic electrostatic self-field、beamstrahlung、coherent pair generation 和 reduced diagnostics 的联合工作流能否稳定接通，而不是对 luminosity 谱或 Yakimenko 2019 结果做强物理断言。目录里的 plot_fields.py 和 plot_reduced.py 也只是说明用户应如何可视化 $|E|/|B|$ 、次级对密度、每个 beam particle 的 photon 数与 NLBW pair 数；它们不应被当成 regression analysis。

Examples/Tests/reduced_diags/ 本体当前则应再拆成两棵验证树。test_3d_reduced_diags 走 analysis_reduced_diags_impl.py：它不是只看 reduced text 文件能不能写出来，而是从 full plotfile 重新计算 FieldEnergy、ParticleEnergy、FieldMomentum、ParticleMomentum、FieldMaximum、RhoMaximum 和 parser 驱动的 FieldReduction，再与 EF/EP/PF/PP/MF/MR/NP/FR_*/Edotj.txt 逐项对照。除 field energy 因 staggered-vs-cell-centered 差异放宽到 0.3 之外，其余量默认要求 $1e-12$ 。因此这条 regression 真正验证的是 compact observable 的物理定义和 full-state reference 一致性。

与之并列的 test_3d_reduced_diags_load_balance_costs_* 则完全不是物理场解对照。analysis_reduced_diags_load_balance_costs 根本不读 plotfile，而是直接从 LBC.txt 重建每个 rank 的总成本，再只断言 load balance 前后的效率满足 efficiency_before < efficiency_after。因此这组 tests 真正验证的是 LoadBalanceCosts 是否把并行运行态忠实暴露给 reduced output，而不是某个固定电磁场或粒子分布是否被精确重现。

这里还要特别写清两个边界。第一，test_3d_reduced_diags_load_balance_costs_timers_psatd 这个名字并没有真的把 solver 切到 psatd；它的 input 仍然只是 inputs_base_3d + algo.load_balance_costs_update = Timers。第二，test_3d_reduced_diags_single_precision 也只看到 analysis_reduced_diags_impl.py 里预留了 single_precision=True 的放宽容差代码路径，但没有看到 active CMake test/input。因此这两条都不能被夸大成活跃的强 regression。

这一层补上以后，第 8 章里 diagnostics 的主线已经不只是“有哪些类”，而是能同时回答：

1. 这个 diagnostics 对象属于哪条计算主线；
2. flush 时走哪种 writer；
3. 最终落盘的是 diagnostics 视图、标准化交换格式，还是 restart 运行态。

如果进一步按“落盘后怎么读”来分，这三类 writer 也已经形成稳定分工。plotfile 典型目录是：

```
diag1NNNNNN/
  Header
  Level_*/
  <species>/
  warpx_job_info
  WarpXHeader
  raw_fields/    # optional
```

主读取工具是 yt，WarpX 文档里的标准入口就是：

```
import yt
ds = yt.load('./diags/plotfiles/plt00000/')
```

如果只是要常规 field/particle 分析、AMR-aware 后处理，plotfile + yt 仍然是最顺手的默认组合。只有当打开 plot_raw_fields = 1 想看原始 staggered 网格时，才需要额外走 Tools/PostProcessing/read_raw_data.py 这条 raw reader。

openPMD 的目录则更扁平，典型是：

```
diag1/
  paraview.pmd
```

```
openpmd_%06T.h5|bp5|bp4|json
```

fields/particles 的层级主要在文件内部，而不是目录树展开。读者侧主工具是：

- openPMD-viewer
- openPMD-api

前者适合快速浏览和 Jupyter 交互，后者适合保留完整 metadata、做并行/chunk 读取以及与外部数据生态对接。文档还专门提醒：yt 也能读一部分 openPMD HDF5，但没有 mesh refinement 支持，因此不能把它当成 plotfile 读法的完全替代。

checkpoint 则根本不是“分析目录”，而是 WarpX 自己的 restart contract。其目录会更像：

```
chkNNNNNNN/  
  WarpXHeader  
  warpx_job_info  
  Level_*/  
  <species>/  
  <lasers>/
```

里面包含的是 E/B 主寄存器、current、coarse patch、PML、完整 species 和 lasers，以及 distribution mapping 和 reduced diagnostics checkpoint state。它的首要读取者不是 Python 数据分析工具，而是：

```
amr.restart = "../.../chk000XXX"
```

也就是说，checkpoint 的主用途是“接着跑”和“恢复”，不是“直接分析”。这一点和 plotfile/openPMD 必须明确分开。

因此，对输出格式的选择可以直接收敛成三条经验：

1. 想做常规物理分析，优先 plotfile。
2. 想做标准化交换、粒子上附加场、RZ mode 或 richer metadata，优先 openPMD。
3. 想保留完整运行态并支持 restart，只能用 checkpoint。

restart/ 目录里还有两条很窄但很有代表性的 PICMI regression，把这三条经验压到了更细的接口层。inputs_test_2d_id_cpu_read_picmi.py 虽然也挂了 checkpoint 组件，但当前强断言其实是脚本内直接读取 pti["idcpu"] 并用 unpack_ids/unpack_cpus 验证粒子标识解包合同；inputs_test_2d_runtime_components_picmi.py 则把 picmi.Checkpoint(...)、amr.restart=... 参数解析和动态 newPid runtime component 放在一起，证明 checkpoint front-end 接线与 runtime-attribute 写入合同可以共存，但对应的 test_2d_runtime_components_picmi_restart 仍是 FIXME scaffold。这说明 checkpoint/PICMI 这条线已经有最小 regression 证明“前端能接上”，但还没有把“restart 后动态 runtime attrs 仍完全一致”升级成活跃强断言。

BackTransformed diagnostics 还有一条很值得保留的 RZ 强基准：Examples/Tests/btd_rz/。它不是只检查 RZ BTD 目录结构，而是从 back_rz openPMD 文件读取 back-transformed 轴上场剖面，直接拟合 boosted-frame Gaussian laser 还原到 lab frame 后的振幅、波长、包络持续时间和相位中心。因此这组例子说明：RZ BackTransformed diagnostics 已经不仅有 writer 合同，还有明确的物理重建合同。

checkpoint/restart 这条线也有两类很值得区分的最小基准。test_3d_acceleration / test_3d_acceleration_restart 是最严格的一类：analysis 逐字段比较 restart 与非 restart plotfile，要求最大相对误差低于 1e-12，因此它真正验证的是 acceleration 基线上的 restart 可重复性，而不是某个独立“加速物理”现象。test_3d_eb_picmi 则更像一条前端 scaffold：它把 PICMI、embedded boundary、checkpoint 与 amr.restart=... 放进同一个最小脚本中，但当前活跃 test 仍主要依赖 checksum，显式 restart 变体还停在 FIXME。因此这条线目前证明的是“EB + PICMI + checkpoint 配线能接上”，而不是“EB restart 后所有状态都已有独立强断言覆盖”。

这条 restart_eb 边界还需要再强调一层：目录里虽然已经放着 analysis_default_restart.py，而且注释掉的 test_3d_eb_picmi_restart 也明确准备好了

```
amr.restart = "../test_3d_eb_picmi/diags/chk000030"
```

与逐字段 restart 对照的调用方式，但当前活跃注册仍只有 test_3d_eb_picmi 本体，且 analysis = OFF。因此在正文里更准确的说法应是：

1. restart_eb 已经有完整的“未来强 restart regression”脚手架；
2. 当前活跃 CI 仍只证明 EB + PICMI + checkpoint 输出链稳定；
3. 还不能把这条线表述成“EB restart 已完成 field-level reproducibility 验证”。

同一个 restart/ 目录里还有一条更细、但很容易被误归到“纯 PSATD benchmark”的分支：test_3d_acceleration_psatd*。这些输入并不是单独拿出来验证谱色散关系，而是把同一条 3D boosted acceleration workflow 切到：

- algo.maxwell_solver = psatd
- psatd.use_default_v_galilean = 1

并在另一支里再额外打开：

- `psatd.do_time_averaging = 1`

然后继续复用同一个 `analysis_default_restart.py` 做逐字段 restart 对照。也就是说，这几条回归真正证明的是：

1. PSATD + Galilean acceleration workflow 的 checkpoint/restart 可重复性；
2. time-averaged PSATD update 打开后，同一 workflow 的 restart 可重复性。

它们不是新的色散理论强基准，而是“更复杂 solver path 仍能完整写盘并无漂移恢复”的 diagnostics/restart 合同。

BoundaryScrapingDiagnostics 与 Python scraped-particle buffer

边界诊断是这一章里一个容易误解的特殊分支。它不是普通 full diagnostics 的“少画几列字段”，而是单独的 `diag_type=BoundaryScraping`。

`MultiDiagnostics.cpp` 读到这个 `diag_type` 时，会直接构造：

```
alldiags[i] = std::make_unique<BoundaryScrapingDiagnostics>(i, diags_names[i], diags_types[i]);
```

而 `BoundaryScrapingDiagnostics::ReadParameters()` 又立刻把默认 field 输出关掉：

```
m_varnames_fields = {};  
m_varnames = {};  
m_num_buffers = AMREX_SPACEDIM*2;  
if (eb_enabled) { m_num_buffers += 1; }
```

这说明它的输出对象不是普通场变量，而是每个边界各自那份 `ParticleBoundaryBuffer`。

进一步看 `InitializeParticleBuffer()`，它对每个 species、每个 boundary 都直接取：

```
WarpXParticleContainer::Base* bnd_buffer =  
    particle_buffer.getParticleBufferPointer(species_name, i_buffer);  
m_output_species[i_buffer].push_back(ParticleDiag(m_diag_name, species_name, pc, bnd_buffer));
```

所以 `BoundaryScrapingDiagnostics` 不是重新扫主粒子容器，而是直接消费前面边界处理阶段已经收集好的 scraped event。

这个 diagnostics 目前还有两个硬边界：

- 必须编译 `openPMD`；
- `<diag>.format` 必须是 `openpmd`。

写出时，`Flush(i_buffer)` 会把每个边界单独写到：

```
const std::string file_prefix =  
    m_file_prefix + "/particles_at_" + particle_buffer.boundaryName(i_buffer);
```

因此目录天然会分成 `particles_at_xlo`、`particles_at_zhi`、`particles_at_eb` 等子目录。更关键的是，写完后它立即：

```
particle_buffer.clearParticles(i_buffer);
```

也就是说，`BoundaryScrapingDiagnostics` 对 `ParticleBoundaryBuffer` 是“写出并消费”的语义，而不是只读观察。

Python 接口走的是同一份底层状态。`Source/Python/WarpX.cpp` 只是把 `WarpX` 单例里的：

```
wx.GetParticleBoundaryBuffer()
```

直接暴露给 `sim.extension.get_particle_boundary_buffer()`。高层 `ParticleBoundaryBufferWrapper` 再把它封成：

- `get_particle_boundary_buffer_size(...)`
- `get_particle_boundary_buffer(...)`
- `get_particle_scraped_this_step(...)`
- `clear_buffer()`

其中 `get_particle_scraped_this_step()` 并不是单独的“本步队列”，它只是用 `stepScraped == getistep(level)` 对累计 buffer 做一次筛选。官方 Python 文档也明确要求用户手动 `clear_buffer()`，否则内存会持续增长。

因此，边界 scraped particle 的消费侧必须记住三个事实：

1. `BoundaryScrapingDiagnostics` 只写粒子，不写场，而且当前只支持 `openPMD`。
2. Python wrapper 和 diagnostics 共用同一个 `ParticleBoundaryBuffer`，不是两份独立副本。

3. diagnostics flush 后会自动清空对应 boundary buffer; Python 路径则需要用户自己清空。

这一点对二次发射、探测器统计和边界通量诊断尤其关键，因为它决定了“什么时候读到哪些粒子”，本质上取决于 buffer 的消费时机，而不只是取决于边界物理本身。

固定模板案例页

为了避免 diagnostics 章节一直停留在“原理说明”，现在把四类最常用输出都压成同一模板：

1. 最小输入片段。
2. 典型目录树。
3. 读取入口。
4. 适用场景。

模板 A: plotfile

最小输入片段：

```
diagnostics.diams_names = diag1
```

```
diag1.diag_type = Full
diag1.intervals = 10
diag1.fields_to_plot = Ex Ey Ez Bx By Bz rho
diag1.write_species = 1
```

典型目录树：

```
diags/diag1/
  diag1000000/
    Header
    Level_0/
      <species>/
        warpx_job_info
        WarpXHeader
        raw_fields/ # optional
```

读取入口：

```
import yt
ds = yt.load("./diags/diag1/diag1000000/")
```

适用场景：

- 常规 field/particle 分析。
- 需要 yt 的 AMR-aware 工作流。
- 需要额外读取 raw_fields/ 做 staggered-grid 调试。

模板 B: openPMD

最小输入片段：

```
diagnostics.diams_names = diag1
```

```
diag1.diag_type = Full
diag1.format = openpmd
diag1.openpmd_backend = h5
diag1.intervals = 10
diag1.fields_to_plot = Ex Ey Ez Bx By Bz rho
diag1.write_species = 1
```

如果需要 writer 阶段再把场 gather 到粒子：


```
diag1.plot_phi = 1
diag1.plot_E = 1
diag1.plot_B = 1
```

典型目录树:

```
diags/diag1/
  paraview.pmd
  openpmd_000000.h5
  openpmd_000010.h5
```

读取入口:

```
from openpmd_viewer import OpenPMDTimeSeries
ts = OpenPMDTimeSeries("./diags/diag1/")
```

适用场景:

- 标准化数据交换。
- richer metadata 或 openPMD 生态工具链。
- full diagnostics 下的 phi / E / B on particles。

模板 C: checkpoint

最小输入片段:

```
diagnostics.diags_names = chk
```

```
chk.diag_type = Full
chk.format = checkpoint
chk.intervals = 100
```

重启入口:

```
amr.restart = ./diags/chk/chk000100
```

典型目录树:

```
diags/chk/
  chk000100/
    WarpXHeader
    warpx_job_info
    Level_0/
      <species>/
      <lasers>/
```

读取入口:

首要读取者不是 Python, 而是 WarpX 本体的 restart。

适用场景:

- 中断后续跑。
- 完整运行态持久化。
- reduced diagnostics 的 checkpoint state 恢复。

模板 D: BoundaryScraping/openPMD

最清楚的官方骨架来自 thomson_parabola_spectrometer:

```
diagnostics.diags_names = screen
```

```
screen.diag_type = BoundaryScraping
screen.format = openpmd
screen.intervals = 1
```

```
hydrogen1_1.save_particles_at_zhi = 1
carbon12_6.save_particles_at_zhi = 1
carbon12_4.save_particles_at_zhi = 1
```

典型目录树:

```
diags/screen/
  particles_at_zhi/
    paraview.pmd
    openpmd_000001.h5
    openpmd_000002.h5
    ...
```

如果是 embedded boundary scraping, 则目录会变成 `particles_at_eb/`。

读取入口:

```
from openpmd_viewer import OpenPMDTimeSeries
series = OpenPMDTimeSeries("./diags/screen/particles_at_zhi/")

point_of_contact_eb/analysis.py 也用同一路径读取:

ts_scraping = OpenPMDTimeSeries("./diags/diag2/particles_at_eb/")
```

适用场景:

- 探测器或屏幕 hit 记录。
- 吸收边界通量统计。
- EB 接触点位置和法向验证。
- 需要把 scraped particles 落成持久文件而不是只在 Python callback 里临时消费。

和前三类相比, 这一类还要额外记住:

1. 只支持 openPMD。
2. 只写粒子, 不写场。
3. species 必须先开 `save_particles_at_xlo/.../eb`。
4. writer filter 在 flush 时生效, 因此 `plot_filter_function` 里的 `t` 是写出时间, 不是撞边界时间。

`thomson_parabola_spectrometer` 还需要再往前走一步理解。它不只是 `BoundaryScraping/openPMD` 的模板例子, 也是 active 强 analysis: `CMakeLists.txt` 同时运行 `analysis.py` 和 checksum helper. `analysis.py` 会从 `screen/particles_at_zhi/` 读取 detector hits, 再从 `diag0` 的初始 full diagnostic 读取 `uz/id/mass`, 按粒子 id 回连出每个 detector hit 对应的初始能量, 最终在 screen 平面上重建按 species 和入射能量着色的离子分离图。因此这组例子真正验证的是:

1. `BoundaryScraping` 在 zhi 屏面的 detector-hit 持久化
2. openPMD 读取链
3. id 跨 diagnostics 回连
4. 解析 E_x/B_x 场驱动下的 test-particle TPS optics

所以它不应再被归到笼统的 PEC / conducting boundary 桶里。

这样整理后, 第 8 章里关于 writer 的内容已经不再只是抽象分类, 而是能直接给出“要什么输出, 就怎么写输入、怎么找目录、怎么读文件”。

Python 边界 buffer 的最小消费模板

如果不想先把 scraped particles 写成 openPMD, 而是直接在 Python 里消费 `ParticleBoundaryBuffer`, 官方 examples 给出两种典型模式。

模式 A: 运行结束后统一检查

`particle_boundary_scrape` 给的是最小自检骨架。species 只要打开:

```
electrons = picmi.Species(
    ...,
    warpx_save_particles_at_xhi=1,
```

```
warpx_save_particles_at_eb=1,
)
```

模拟结束后直接：

```
from pywarpx import particle_containers

particle_buffer = particle_containers.ParticleBoundaryBufferWrapper()
n = particle_buffer.get_particle_boundary_buffer_size("electrons", "eb")
weights = particle_buffer.get_particle_boundary_buffer("electrons", "eb", "w", 0)
total_weight = sum(w.sum() for w in weights)
particle_buffer.clear_buffer()
```

这里三个实现细节必须记住：

1. `get_particle_boundary_buffer_size()` 给的是累计到当前时刻的 scraped 数，不是本步增量。
2. `get_particle_boundary_buffer()` 返回的是“每个 tile 一条数组”的列表，不会自动拼平成一块大数组。
3. Python 路径不会自动清空 buffer，必须自己 `clear_buffer()`。

模式 B: callback 或在线控制里的即时事件流

`spacecraft_charging` 给的是“文件化 writer 和 Python 在线消费并存”的例子。它一边开：

```
part_scraping_boundary_diag = picmi.ParticleBoundaryScrapingDiagnostic(
    name="diag2",
    period=-1,
    species=[electrons, protons],
    warpx_format="openpmd",
)
```

一边又在 Python 里直接读同一个 EB buffer：

```
particle_buffer = ParticleBoundaryBufferWrapper()
weights = particle_buffer.get_particle_boundary_buffer(species, "eb", "w", 0)
sum_weights_over_tiles = sum([w.sum() for w in weights])
ntot = float(mpi.COMM_WORLD.allreduce(sum_weights_over_tiles, op=mpi.SUM))
```

这里它把 `period=-1` 设成只在末尾 flush 一次，目的就是在模拟中途保留完整 in-memory buffer，供 Python 侧累计计算 spacecraft 收集到的净电荷。

它的 analysis 还会继续从 full diagnostics 里抽取每个输出步的最小势值，拟合

$$\phi(t) = v_0 (1 - e^{-t/\tau}),$$

并要求拟合得到的 `v0` 和 `tau` 分别落在 4% 与 20% 的容差内。于是这组例子在第 8 章里最准确的定位就不是“演示 Python 能访问 buffer”，而是：

- boundary buffer 在线消费
- Python 动态改写 EB potential
- 以及 electrostatic diagnostics 最终能否给出正确的 charging 时间尺度

三者联动的应用级 regression。

如果逻辑只想处理“本步刚撞边界的粒子”，更直接的高层接口是：

```
weights_this_step = particle_buffer.get_particle_scraped_this_step(
    "electrons", "eb", "w", 0
)
```

它本质上只是按 `stepScraped == current_step` 对累计 buffer 做过滤，但这已经足够支撑：

- secondary emission;
- 每步边界通量统计;
- callback 驱动的边界反应模型。

这和 BoundaryScrapingDiagnostics 的分工

现在 diagnostics 和 Python 两条路径的边界已经可以写得很清楚：

1. BoundaryScrapingDiagnostics 负责文件化持久输出，flush 后自动清空对应 boundary buffer。
2. ParticleBoundaryBufferWrapper 负责 Python 即时访问，默认不会自动清空。
3. 两者共用同一份 ParticleBoundaryBuffer，不是两份副本。

因此，如果 diagnostics 设成频繁 flush，Python 侧看到的累计事件就会被 writer 周期性截断；如果 diagnostics 只在末尾 flush，甚至根本不开 writer，Python 才能把它当成长时间累积的事件池来用。

Python field wrapper 的最小强回归

python_wrappers/inputs_test_2d_python_wrappers_picmi.py 覆盖的是另一条 Python 接口线。它既不是粒子 callback，也不是单纯 writer smoke test，而是直接验证：

- sim.fields.get(...)
- MultiFabRegister
- valid-domain 字段
- PML split fields
- divergence-cleaning 标量

能否在 Python 侧被稳定访问。

这条 regression 在 CMakeLists.txt 里没有独立 analysis.py：

```
add_warpx_test(
    test_2d_python_wrappers_picmi
    ...
    OFF
    "analysis_default_regression.py --path diags/diag1000100"
    OFF
)
```

所以如果只看 CMake，会误以为它只是 checksum-only。实际上强断言全部写在 input 脚本内部。脚本在

```
sim.initialize_inputs()
sim.initialize_warpx()
```

之后，直接抓出：

```
Ex = sim.fields.get("Efield_fp", dir="x", level=0)
...
Expml = sim.fields.get("pml_E_fp", dir="x", level=0)
...
Fpml = sim.fields.get("pml_F_fp", level=0)
Gpml = sim.fields.get("pml_G_fp", level=0)
```

然后给 valid domain 里的 E/B/F/G 填一个平滑 unit pulse，推进 100 步，最后不是看图片，而是逐分量检查 benchmark：

```
def check_values(benchmark, data, comp, rtol, atol):
    passed = np.allclose(
        benchmark, np.sum(np.abs(data[(), (), comp])), rtol=rtol, atol=atol
    )
    assert passed
```

这里 data[(), (), comp] 的意思也很关键：

- () 取 valid + ghost 全范围；
- comp 用来访问 PML split-field 的不同 component。

脚本会依次断言：

- Ex/Ey/Ez
- Bx/By/Bz

- F/G
- pml_E_fp
- pml_B_fp
- pml_F_fp
- pml_G_fp

的每个相关 component 都与固定 benchmark 一致，而且若干应为零的 component 也显式要求保持为零。于是这条 test 的真实定位应当是：

- Python field wrapper / PML split-field access

而不是过粗的 Python API / callbacks。它验证的是 pywarpx 经由 MultiFabRegister 暴露出来的非 owning MultiFab 视图，在 valid domain、ghost cell、PML 和 cleaning 字段这几层上都没有被包装错。

MPI transport 环境注意

某些 MPI/OFI 组合会受网络接口选择影响；在这类环境中，小规模复现实验可设置：

`FI_PROVIDER=tcp`

这不是物理参数，而是 MPI transport 的环境兼容设置。复现实验应记录环境变量、binary 版本、输入文件、输出目录和分析脚本；不要把它误当成会改变 PIC 物理模型的输入参数。

Langmuir 与均匀等离子体的运行证据

下面三项 reader-side 比较分别覆盖 Langmuir 时间采样、均匀等离子体守恒统计和二者的分析入口：

1. 对同一份 Langmuir 官方输入，将 `diag1/openpmd.intervals` 从 40 改为 1，运行 80 步，得到 81 个逐步快照；
2. 对 Ez 的目标空间模做投影，用两正交分量拟合时间频率，并逐快照计算 `divE-rho/epsilon_0`；
3. 用 `yt` 读取 uniform-plasma 初末 plotfile，统计粒子数、粒子总权重、场能、粒子动能和总能量。

Langmuir 的受控复现实验给出：

- 81 个快照；
- 解析 `omega_p=1.128292045086e14`，拟合值为 `1.128697661742e14`；
- 相对频率误差 `3.595e-4`；
- 官方 `analysis_1d.py` 在同一族运行上给出最大相对误差 `1.70e-3`、最终 `divE-rho/epsilon_0` 误差 `8.35e-12`，均通过原始阈值；
- 逐步 reader-side 扫描的最大守恒误差为 `3.149e-10`，发生在中间快照，因此不能把“每一步都满足官方 `1e-11` 阈值”写成已验证事实。

Uniform-plasma 的受控复现实验给出：

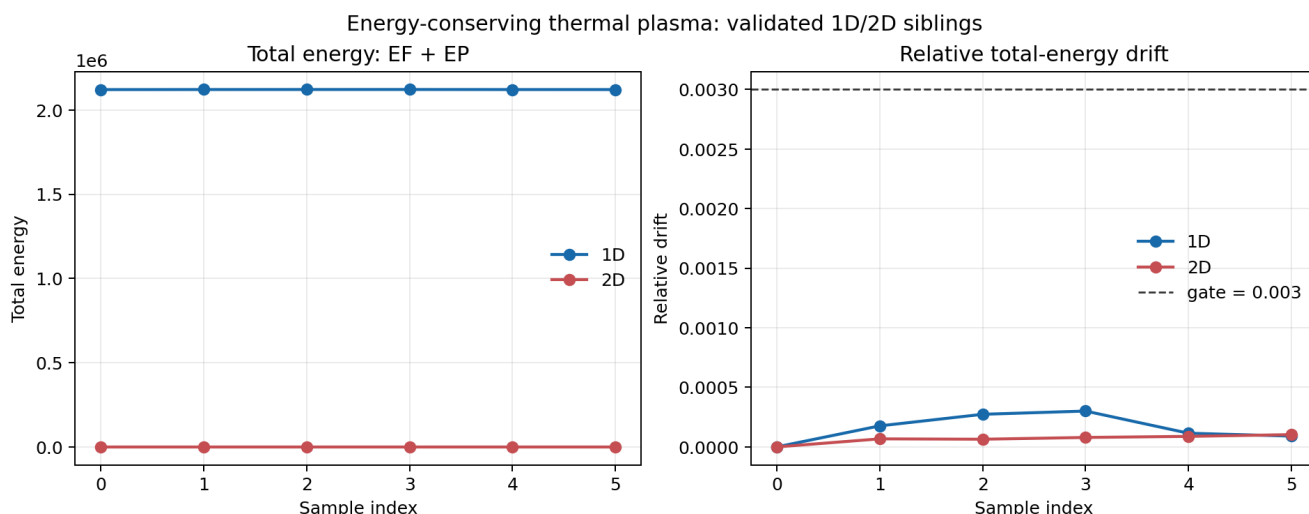
- 初末粒子数均为 65536；
- 粒子总权重相对变化为 0；
- 初始场能量为零，所以场能相对变化率有意标记为 `undefined (zero baseline)`；
- 粒子动能变化约 7.06%，总能量变化约 1.97%。

将同一输入延长到 100 步、每 10 步写出一个 plotfile 后，粒子总权重在全部 11 个快照中保持不变，末态总能量相对初态变化 `1.387e-2`，时间序列中的最大绝对相对偏差为 `2.518e-2`。这比单看 10 步终点更能说明短时热背景统计范围，但仍不足以构成热平衡能量守恒 gate；后者应与 `energy_conserving_thermal_plasma` 的专门 analysis 绑定。

官方 2D `energy_conserving_thermal_plasma` 输入运行 500 步，产生 `EF.txt` 和 `EP.txt` 六个 reduced-energy 样本；其 `analysis.py` 通过，独立 reader-side 计算同一 `EF+EP`，得到最大总能量相对漂移 `1.031e-4`，低于官方 `3.000e-3` 阈值。由此可以把两类证据明确分开：uniform-plasma 负责粒子数、I/O 和热背景 workflow 统计，energy-conserving-thermal-plasma 才负责 energy-conserving gather 的强能量漂移 gate。

同一官方 family 的 1D sibling 也在 500 步运行中通过官方 analysis，最大漂移为 `3.009e-4`。1D/2D 都使用 `EF+EP、0.003` 阈值和 6 个采样点，但不应把两种几何的物理轨迹误写成数值等价。

图 8-2 把 1D/2D 的 `EF+EP` 和归一化漂移放在同一张图里。左图保留不同几何的能量尺度差异，右图直接与共同的 `0.003 gate` 对照；因此读者可以同时看到“能量在场能与粒子能之间交换”和“总能量误差仍被 gate 约束”这两个不同层次。



Reduced diagnostics 与 full-state reference 对照

按官方 2-rank 配置执行 `Examples/Tests/reduced_diags/inputs_test_3d_reduced_diags` 后,末态为 `diags/diag1000200`。官方 `analysis_reduced_diags.py` 从该 plotfile 重新计算粒子能量/动量、场能/动量、场最大值、rho 最大值、粒子数以及 `FR_Max/FR_Min/FR_Integral/Edotj` 等 parser-driven FieldReduction,再逐项与 `EP/EF/PP/PF/MF/MR/NP/FR_*/Edotj.txt` 对照。

共比较 60 个 reduced observable, 官方 analysis 通过。除 field energy 外, 最大相对误差为 $4.125\text{e-}13$; field energy 的相对误差为 $2.483\text{e-}1$, 仍低于官方为 staggered Yee reduced energy 与 cell-centered plotfile reference 设置的专用 0.3 容差。独立 reader-side 比较给出相同的误差分层。

这条证据的准确含义是“compact reduced observable 与 full-state reference 的定义和 writer 输出一致”, 不是说 60 个量都构成独立物理守恒定律。尤其 field energy 的 24.8% 误差必须保留其 staggered/cell-centered 离散表示边界, 不能被误读成普通量的数值精度。

图 8-4 将这两个误差层分开画出: 左图在对数尺度下显示 59 个非 field-energy observable 的排序误差, 右图单独显示 staggered field-energy 的 0.2483 误差与 0.3 专用 gate。这样图表本身就保留了“普通量接近机器精度、field energy 仍有离散表示差异”的证据结构, 而不是只给出一个混合最大值。

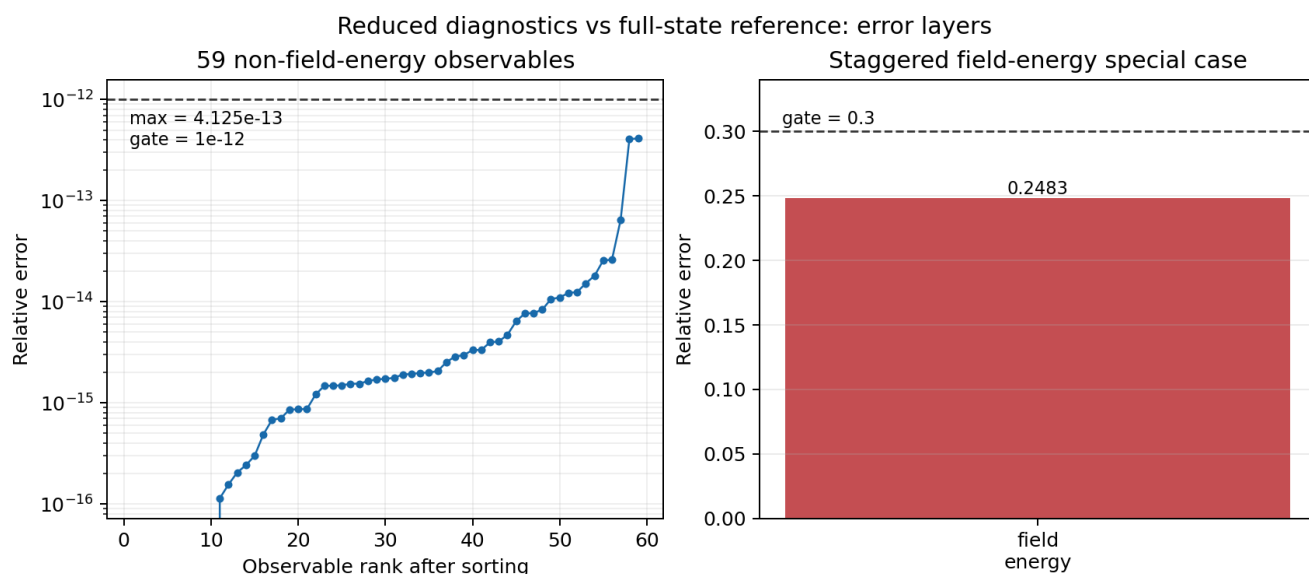


图 8-4 由同一组 reduced diagnostics 与 full-state 输出的 reader-side 比较重建。

LoadBalanceCosts: 性能诊断的 efficiency gate

同一 `reduced_diags` family 还包含一条不读取 `plotfile` 的性能诊断分支: `inputs_base_3d` 使用 $128 \times 32 \times 128$ 网格、16 个 box、`algo.load_balance_intervals = 2`, 并由 `LoadBalanceCosts` 将每个 box 的 `cost`、`rank`、`level`、几何位置和粒子数写入 `LBC.txt`。官方 `analysis` 按 `rank` 汇总 box `cost`, 再计算

$$\eta = \frac{1}{N_{\text{rank}}} \sum_r \frac{C_r}{\max_{r'} C_{r'}}.$$

它比较第 1 行 (load balance 前) 与第 2 行 (load balance 后), 要求 `eta_after > eta_before`。按官方 2-rank 配置分别运行 `Heuristic` 与 `Timers` 两个 sibling:

	cost source	before	after	result
	Heuristic	0.625252	1.000000	PASS
	Timers	0.744780	0.996162	PASS

因此 `LoadBalanceCosts` 的强合同不是“有一个 `LBC` 文本文件”, 而是它能把 box-level `cost` 通过 `MPI` 汇总成 rank-level efficiency, 并观察到重分配后的效率改善; 这属于性能/并行态验证, 不应与 `reduced_diags` 的 compact physical observable 对照混成同一类 gate。

图 8-6 将两条 `cost source` 的 rank-level efficiency 直接画成 before/after 对照。Heuristic 从 0.625252 提升到 1.0, Timers 从 0.744780 提升到 0.996162; 图表表达的是负载重分配后的性能改善, 不是场或粒子物理精度。

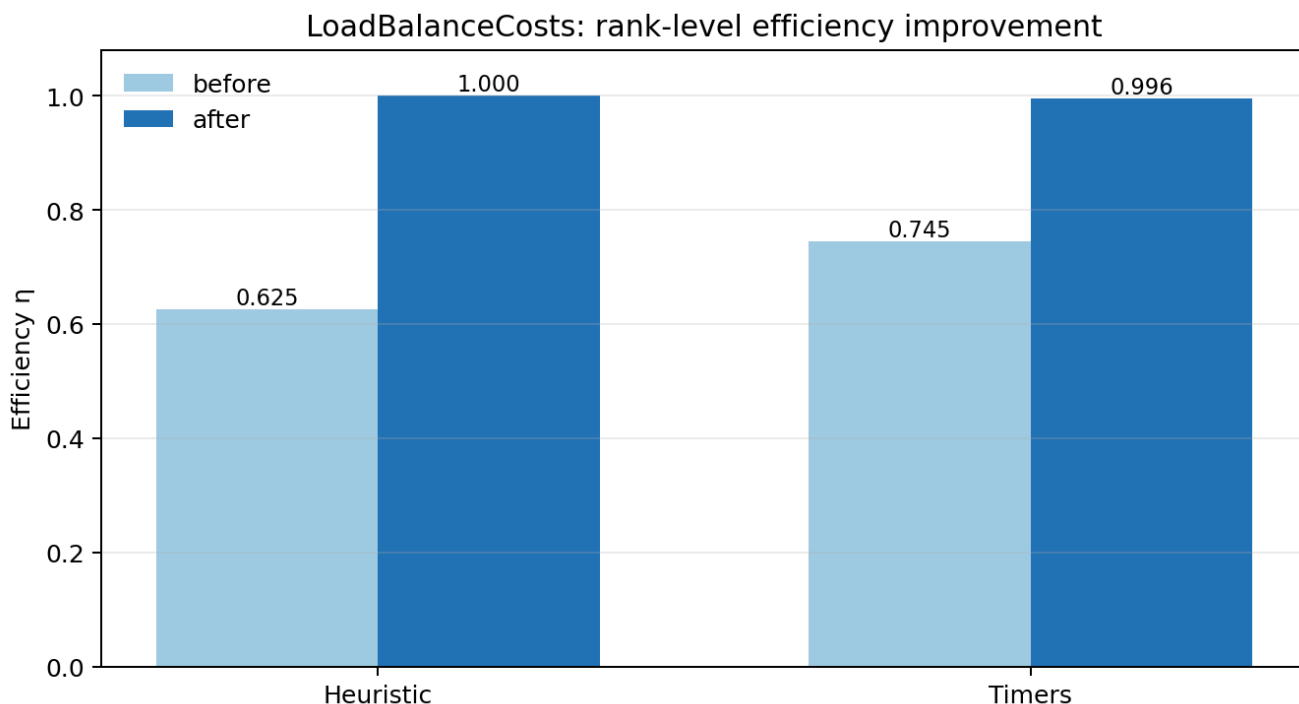


图 8-6 由 `LBC.txt` 的 reader-side 汇总重建。

ColliderRelevant: 束流统计与 luminosity-rate 聚合同

`Examples/Tests/collider_relevant_diags/` 提供了另一条强 reduced-diagnostics regression。它同一 3D、2-rank 输入中同时输出 `ColliderRelevant_beam_e_beam_p`、`ParticleExtrema_beam_e/beam_p` 和 `openPMD full state`。官方 `analysis.py` 使用输入中的三个解析宏粒子样本, 逐项检查每个 beam 的 `chi_min/max/ave`、位置均值/标准差和 `theta_x/theta_y` 的 `min/ave/max/std`, 并把 `ParticleExtrema` 与 `ColliderRelevant` 交叉核对; 随后从 `rho_beam_e`、`rho_beam_p` 和粒子电荷重建

$$\frac{dL}{dt} = 2c \Delta V \sum_i \frac{\rho_{e,i}}{q_e} \frac{\rho_{p,i}}{q_p}.$$

在两次 openPMD iteration 中, $dL/dt = 4.42662301265625e8$, 与 reduced text 的对应两行完全一致; ColliderRelevant 为 2 行/33 列, 两个 ParticleExtrema 文件各为 2 行, 官方 analysis 与独立 reader-side 比较均通过。该结果验证的是 collider-oriented quantity 的定义、统计和聚合 writer 合同, 不等于已经完成 diff_lumi_diag 的解析谱 benchmark, 也不等于 beam_beam_collision 的 QED 应用级物理复现。

图 8-7 将两个 openPMD iteration 的 dL/dt 交叉结果叠加显示。两个 reader-side reconstruction 点与 ColliderRelevant reduced 点完全重合, 且 JSON 报告给出的相对误差均为 0; 这张图验证的是聚合定义和 writer/reader 对齐, 不是 luminosity 随束流演化的独立动力学 benchmark。

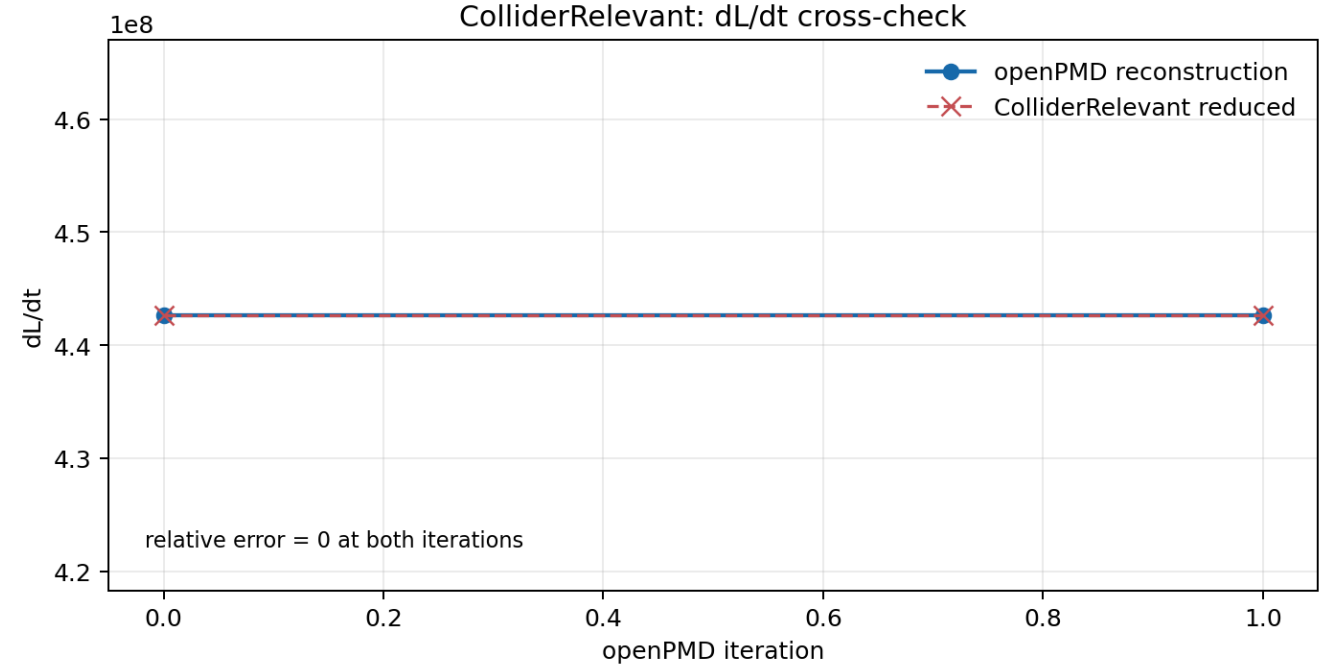


图 8-7 由同一 openPMD 与 reduced-output 对照重建。

DifferentialLuminosity: 1D/2D 解析谱与 AMR 对照

Examples/Tests/diff_lumi_diag/ 把 reduced diagnostics 推进到能量微分 luminosity 的解析 benchmark。共享的 inputs_base_3d 设定两束相向高斯束, 1D DifferentialLuminosity 输出总能量谱, 2D DifferentialLuminosity2D 输出两个入射束能量的二维网格; 官方 analysis.py 分别用高斯束解析式比较末态 1D 与 2D 结果。这个分析依赖 openpmd_viewer 读取 2D openPMD series, 而不是把二维数据误当作普通文本列。

按官方 2-rank 配置完成三组 sibling, 三组都在 step 80 输出 128 个 1D 能量 bin 和 128 x 128 的 2D 网格:

case	AMR max level	1D error / tolerance	2D error / tolerance	result
leptons	0	0.8903% / 2.0%	2.6890% / 4.0%	PASS
leptons + AMR	1	0.9796% / 2.0%	3.0042% / 4.0%	PASS
photons	0	2.0119% / 2.1%	4.9327% / 6.0%	PASS

这些比较使用独立 reader-side analysis。这组结果补上了束流诊断链中的解析谱 physics gate; AMR sibling 的意义是验证中心细化区域仍能保持相同的 reduced luminosity 定义, 而不是宣称 AMR 与 uniform-grid 轨迹逐点相同。

图 8-5 将三组 sibling 的 1D/2D 相对误差和各自 gate 并列展示。photons 使用报告中独立的 2.1%/6.0% 容差, 不能直接套用 leptons 的阈值; 三组柱状值均低于对应虚线 gate, 图表只表达解析谱误差合同, 不把 reduced writer 的文件形状误写成额外物理结论。

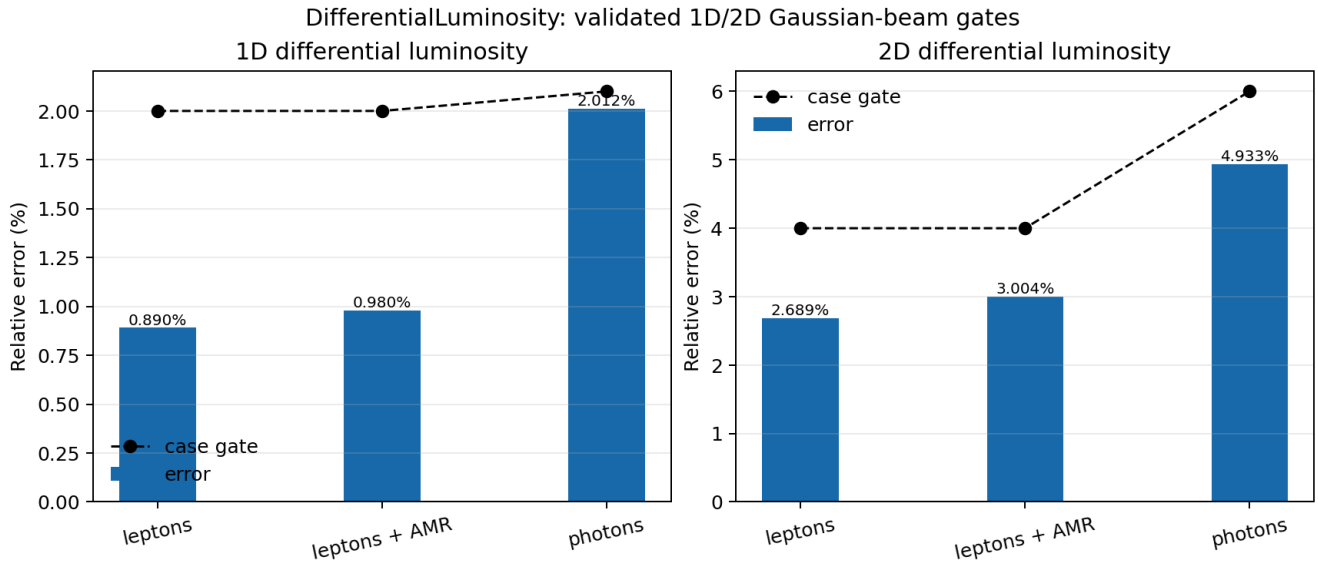


图 8-5 由三组解析谱比较的误差结果重建。

ParticleHistogram2D: 二维 openPMD writer 合同

官方测试树目前没有单独的 ParticleHistogram2D CMake regression; 可用 Examples/Physics_applications/laser_ion/ 的官方 2-rank application 作为完整 producer。输入同时配置 PhaseSpaceIons 和 PhaseSpaceElectrons 两个二维 histogram: 两者都使用 z 作为 abscissa, uz 作为 ordinate, 1000 x 1000 bins, `value_function = w`, 而 electrons 还增加 $\sqrt{x^2+y^2} < 1e-6$ filter。官方 CMake analysis 只负责 time-averaged field 与 instantaneous field 的一致性, 因此不能单独被写成 histogram physics gate; 独立 reader-side 检查则验证 histogram series 的 writer 语义。

在受控复现实验中, 两个 series 都写出 0 和 100 两个 BP5 iteration, 数据形状均为 1000 x 1000, axis labels 为 uz/z , 所有数据有限且存在非零 bin; 官方 time-average analysis 通过。PhaseSpaceIons.txt 与 PhaseSpaceElectrons.txt 的大小均为 0, 这是预期结果: ParticleHistogram2D::WriteToFile() 绕过基类逐行文本 writer, 直接创建 reducedfiles/<name>/openpmd_%T.<backend> series, 因此空 .txt companion 不能被误判成 histogram 丢失。

这条证据的边界也需要保留: 它证明二维 histogram 的配置、openPMD layout、轴元数据和 writer 输出链成立, 不等于已经用独立解析分布证明 laser-ion phase-space 的物理收敛性; 后者需要更高分辨率/粒子数以及针对相空间分布的物理参考结果。

在不改变这条边界的前提下, reader-side analysis 对 BP5 数组做加权统计。PhaseSpaceIons 的总权重从 iteration 0 的 $3.9975794429219594e18$ 到 iteration 100 的 $3.997579442921919e18$, 相对变化约 $1.0e-14$; $\text{std}(z)$ 保持在 $1.47204e-6$ m, 而 $\text{std}(uz)$ 从数值零增长到 $4.78558e-4$ 。受径向 filter 影响的 PhaseSpaceElectrons 总权重从 $5.30929e17$ 变为 $5.32861e17$, $\text{std}(uz)$ 从 0.196998 变为 0.199300 。这些统计量把“相空间图发生了什么变化”从视觉判断推进成可复现的 weighted-moment 摘要, 但仍不能替代更高分辨率/粒子数的 convergence study。

随后又做了一个匹配物理时间的 producer 对照: baseline 使用 384×512 网格, $dt=1.083064693e-16$ s, 100 步, refined 使用 768×1024 网格, $dt=5.415323467e-17$ s, 200 步; 两者最终时间差只有 $3.31e-29$ s。reader-side analysis 对两个 BP5 series 的总权重、 $\text{std}(z)$ 和 $\text{std}(uz)$ 设置了 $1e-3/1e-2/5e-2$ 的局部稳定性阈值, ions/electrons 均通过: $\text{std}(z)$ 相对差为 $5.51e-4/2.02e-4$, $\text{std}(uz)$ 相对差为 $2.04e-3/4.47e-2$ 。这是“网格加密后 reader-side 加权宽度仍稳定”的局部证据, 不是严格的物理收敛阶证明; 两套 producer 都是单进程, 运行结束时的 OFI MPI_Finalize 环境尾噪声不影响已写出的 BP5 数据读取。

同一比较还记录了 1×1 particles-per-cell 的负对照。它的 ions 宽度仍接近 baseline, 但 electrons 总权重相对差为 $1.9471e-3$, 超过 $1e-3$ 阈值, 因此 weighted_width_stability 被拒绝。这个负结果应保留为低采样负对照, 而不是被静默删除; 高粒子数区间需要与它分开判断。

为判断该负结果是否只是单个低采样点, 比较 $1 \times 1/2 \times 2/4 \times 4/8 \times 8$ 四档 particles-per-cell 的相邻档位: $1 \times 1 \rightarrow 2 \times 2$ 是预期负对照, $2 \times 2 \rightarrow 4 \times 4$ 与 $4 \times 4 \rightarrow 8 \times 8$ 是高粒子数局部稳定性 gate。electrons 总权重差依次为 $1.9471e-3$, $4.2685e-4$, $3.6534e-4$, ions/electrons 的总权重、 $\text{std}(z)$ 、 $\text{std}(uz)$ 在高粒子数 pair 均通过。这支持“增加粒子数后该 reader-side 统计总体更稳定”的方向性判断, 但仍是单进程、单一激光离子 case 的局部矩比较, 不足以给出正式收敛阶或上游 regression gate。

图 8-8 直接从 BP5 series 读取 PhaseSpaceIons 与 PhaseSpaceElectrons 的 iteration 0/100 数据, 并按每个面板的非零 bin 裁剪显示范围。它展示的是 writer 实际落盘的 $uz-z$ 相空间结构和随 iteration 的变化; 由于每个面板使用独立对数颜色归一化, 颜色不能用于跨 species 或跨 iteration 的绝对产额比较。完整数组仍为 1000×1000 , 空 .txt sidecar 也仍是预期的 writer 路径边界。

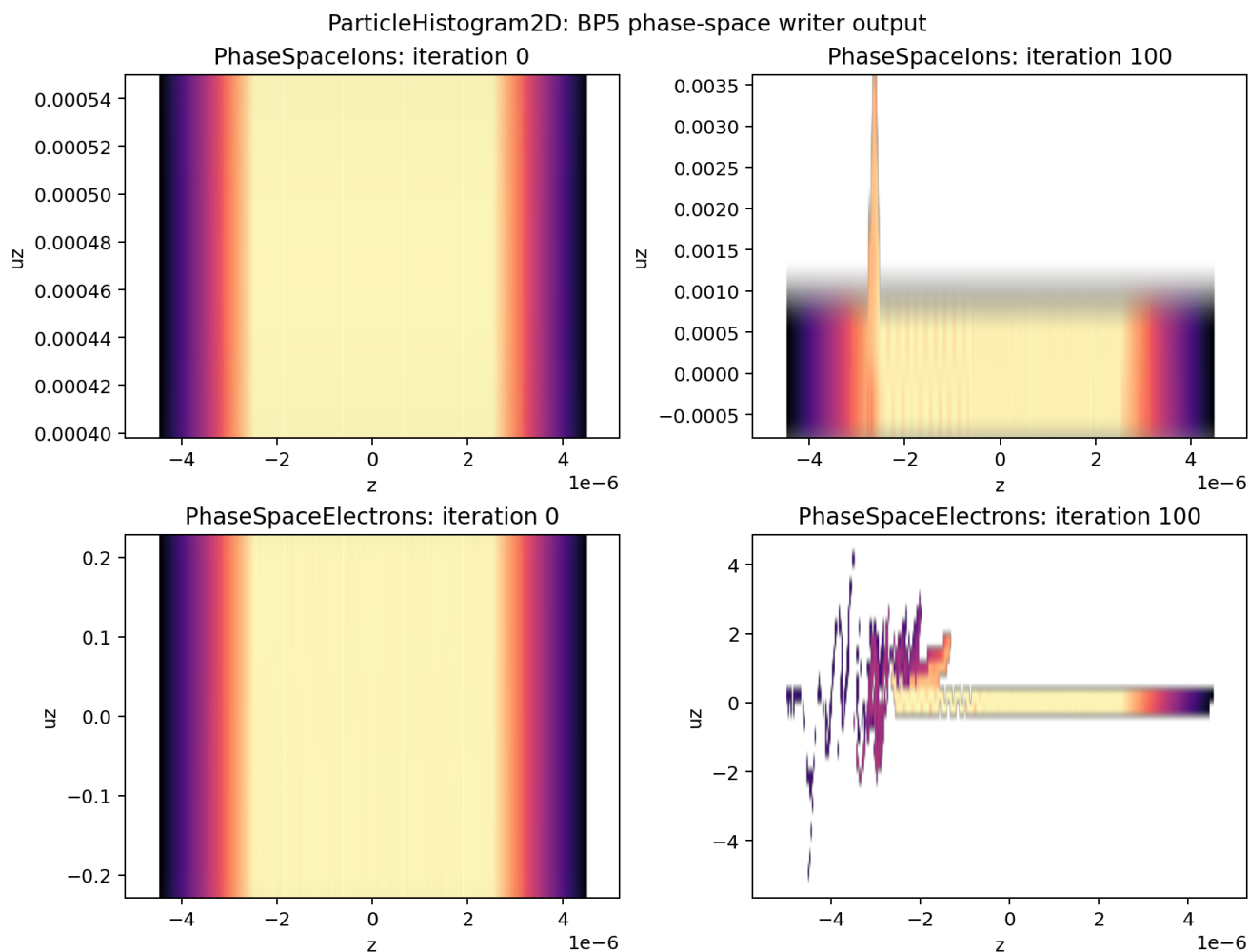
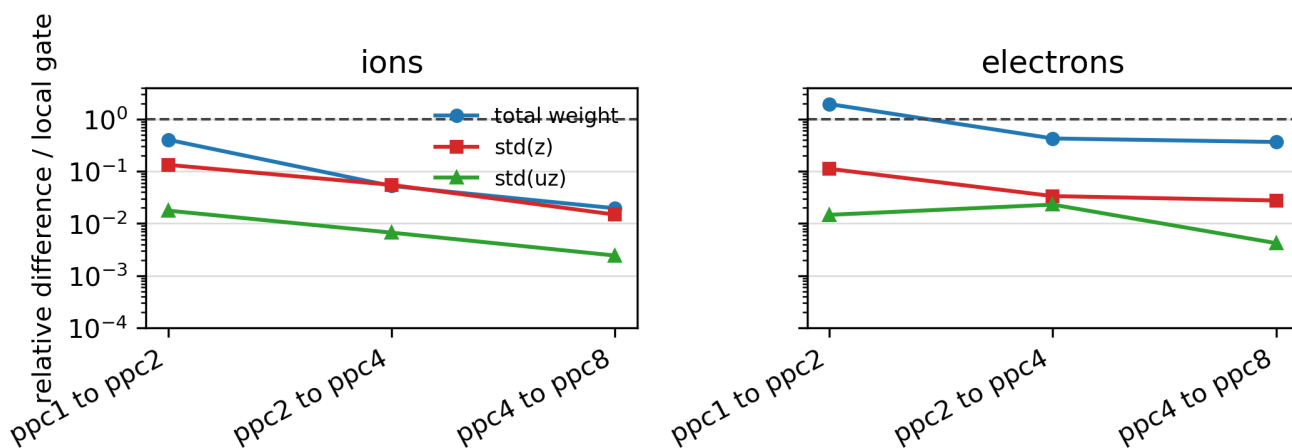


图 8-12 用同一份四档 pairwise contract 把粒子数敏感性归一化到各自局部 gate: 虚线是 gate 边界, 纵轴小于 1 表示通过。1x1 $\rightarrow$ 2x2 的电子总权重仍越过边界, 2x2 $\rightarrow$ 4x4 和 4x4 $\rightarrow$ 8x8 则落在边界以内; 新增 trend contract 将这种“预期负对照 + 高粒子数局部通过”固定为可复查状态, 但不把单一 case 的 reader-side 矩合同写成正式收敛阶。

ParticleHistogram2D particle-count sensitivity



Dashed line = gate boundary; values below 1 pass the local reader-side gate

图 8-8 由 BP5 series 的 reader-side 读取重建。

BeamRelevant: 束流矩与截断高斯束合同

BeamRelevant 是文本型束流诊断, 和 ColliderRelevant 的逐粒子 χ/θ 统计不同。3D 路径固定输出 22 个物理量: 位置与动量均值、 γ 均值、位置/动量/ γ rms、三方向 emittance、Twiss α/β 以及总 charge; 连同步列和时间列共 24 列。其实现先按粒子权重做并行归约, 再从二阶矩构造 rms、emittance 和 Twiss 量, 因此最小验证应同时检查 schema、权重聚合和几何分布, 而不是只检查文件存在。

以官方 initial_distribution 中的 beam 参数为基准, 可以用只包含该 beam、`bmmntr = BeamRelevant` 的 3D、1-rank、`max_step=0` 输入检查初始化束流矩。reader-side analysis 对 `bmmntr.txt` 的独立读取给出 1 行/24 列: $z_{\text{cut}}=2$ 的截断高斯束 charge 期望值为 $-9.544997\text{e-}21$ C, 实测为 $-9.544980\text{e-}21$ C, 相对误差 $1.77\text{e-}6$; 横向 rms 为 $0.249884/0.249765$ m, 纵向 rms 为 0.220356 m, 均通过 2% gate; 均值、emittance、Twiss 相关输出均有限且满足正值边界。

图 8-9 将这个初始化检查的两个主要物理量画出来: 左图是实际总 charge 相对于截断高斯期望值的比值, 右图是三个位置 rms 与解析目标的对照。图中没有把单行输出扩展成虚假的时间演化; γ 、emittance 和 Twiss 量仍只要求满足有限性与正值边界。

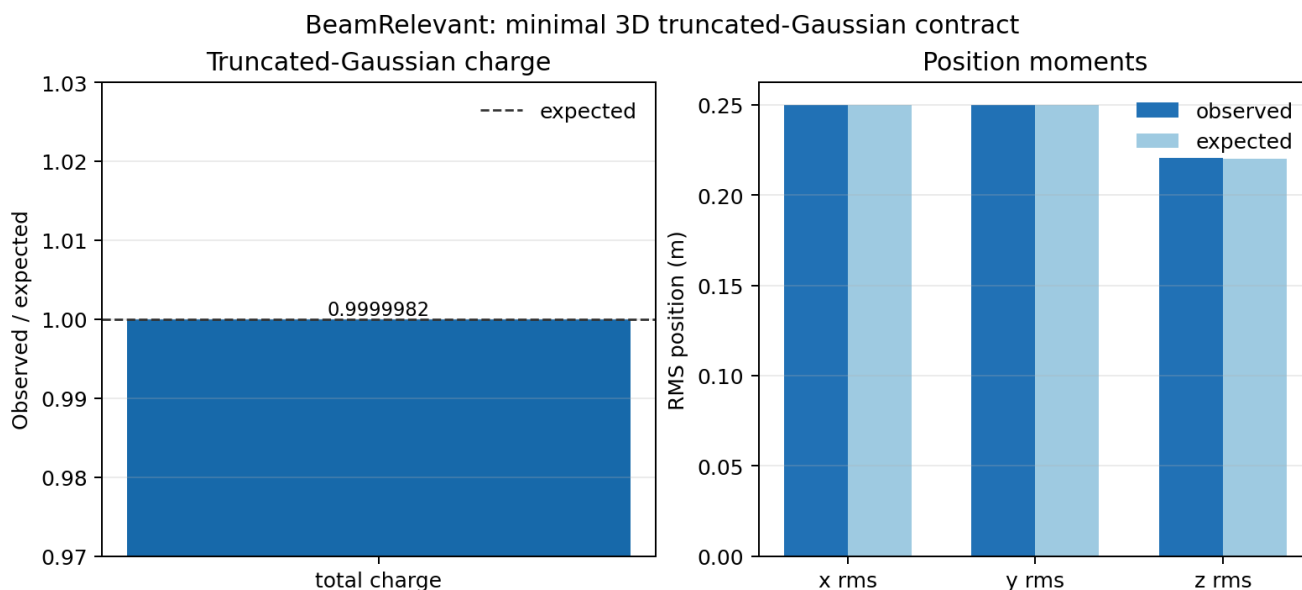


图 8-9 由 reader-side analysis 对对应的束流矩检查结果重新生成。

Native external-file Gaussian beam: 束斑理论包络检查

`gaussian_beam/CMakeLists.txt` 中的 `native test_3d_focusing_gaussian_beam_from_openpmd` 仍引用目录内不存在的 `analysis.py`, 所以不能把它表述为已恢复的上游 CMake regression。可用的物理检查是: 由 `prepare` 脚本和 `native input` 生成 `plotfile` 与 BP5 openPMD 输出, 再用 reader-side analysis 独立读取 iteration 0 的 $x/y/z/w$, 与理论束斑包络比较。

该输入产生 1,999,966 个宏粒子、总权重 $1.999966\text{e}10$ 和 81 个有效 z slice; 按 focal-distance 理论包络计算的最大相对误差为 $\sigma_x = 3.0515\text{e-}2 < 0.051$ 、 $\sigma_y = 3.6214\text{e-}2 < 0.038$ 。官方 `analysis_focusing_beam.py` 也能处理同一输出。这个结果支持外部文件初始化后的束斑包络检查, 但不表示缺失的上游 `analysis.py` 已恢复。

图 8-10 直接从同一 BP5 iteration 0 重建每个 z slice 的加权 σ_x 、 σ_y , 并与 focal-distance 理论包络叠加。它展示外部文件初始化后的真实粒子输出与理论束斑的一致性, 而不是替代缺失的上游 `analysis.py`。

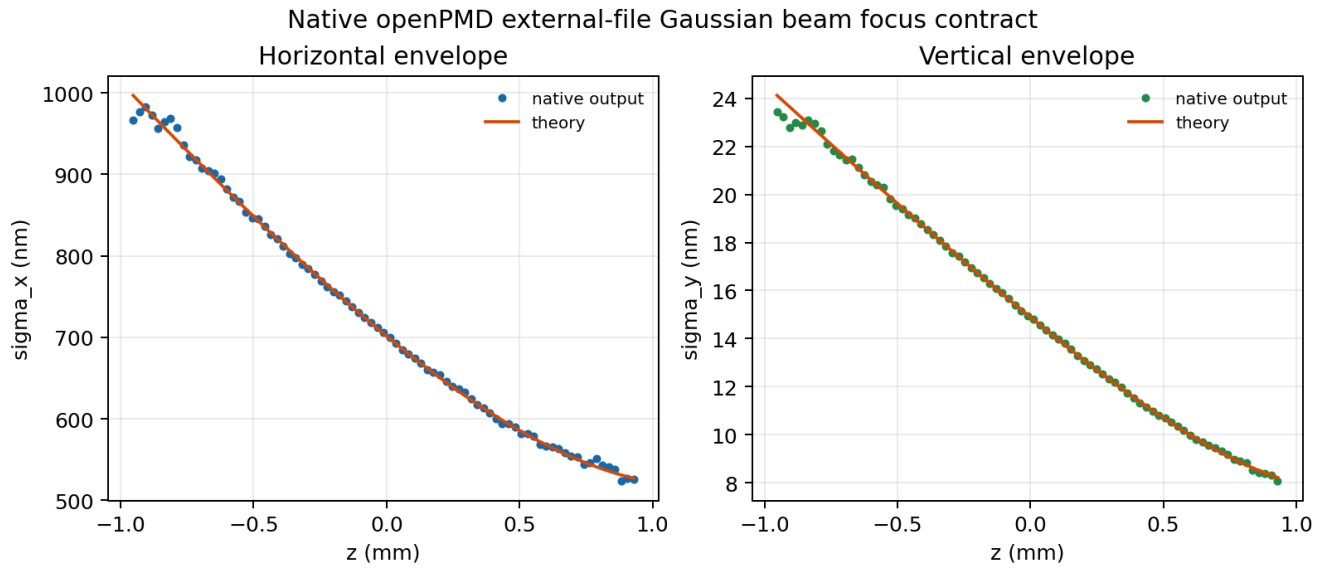


图 8-10 由 reader-side analysis 从对应的 Gaussian beam 输出重新生成。

完整官方 Examples/Tests/initial_distribution/ input 已由对应源码重建的 binary 复现。producer 和官方 analysis.py 均以 exit code 0 结束，10 类分布的最大相对差为 $1.8931\text{e-}2 < 0.02$ 。仓库 checksum 默认 $\text{rtol}=1\text{e-}9$ 观察到最大相对差 $3.18\text{e-}3$ ，反映随机采样而非初始化失败；在显式记录的 $\text{rtol}=5\text{e-}3$ sampling tolerance 下通过。因此该案例的结论是“官方分布 analysis 通过、随机 checksum 有条件通过”，但不宣称确定性 $1\text{e-}9$ checksum 相等。

第 8 章验证矩阵：观察量、结论与边界

详细输入、命令、环境和原始报告属于复现实验材料；正文只保留读者解释结果所需的观察量、结论与限制。

案例/诊断	观察量与可支持结论	仍需保留的边界
Langmuir	场误差 $1.70\text{e-}3$ 、最终守恒 $8.35\text{e-}12$ 、频率误差 $3.595\text{e-}4$ ；解析波与守恒链均通过	不替代多模式或完整色散研究
Uniform plasma restart	37 个 field 的末态相对误差 $2.8631\text{e-}16 < 1\text{e-}12$ ；checkpoint/restart 可重复	不证明热平衡或跨 rank 场一致性
Uniform plasma MPI	粒子总权重一致，但 rank-invariant field gate 不通过	不能把 checksum 差异写成 restart 失败
Thermal plasma FieldProbe	EF+EP 共同漂移 < 0.003 coarse 3.6703% 失败；matched-time refined 0.3533% 通过	仅覆盖指定 family 和时间窗 refined 结果不能反写为 coarse 输入通过
Reduced observables	60 项与 full-state reference 对照；非 field-energy $< 1\text{e-}12$	field-energy 使用独立的 < 0.3 容差
LoadBalanceCosts ColliderRelevant / DifferentialLuminosity	$\text{eta_after} > \text{eta_before}$ 束流统计与 1D/2D 解析谱交叉验证	验证效率改善，不验证场精度 不替代 collider-QED 应用级复现
ParticleHistogram2D / BeamRelevant	writer schema、有限非零数据、截断 Gaussian charge/rms	不给出粒子数正式收敛阶
Initial distribution / Gaussian beam	分布 analysis 与束斑理论包络通过	随机 checksum 与缺失上游 analysis 保持边界
RZ sphere / RZ multimode	场、rho-volume charge 与 mode writeback 的有界检查	不替代完整 RZ 诊断矩阵

这张表中的“通过”只表示对应列出的 gate 通过。例如 FieldProbe 的 coarse 输入仍然是失败证据，完整 initial-distribution 的随机 checksum 也不等价于确定性 $1\text{e-}9$ 回归；这样读者可以从同一张表直接区分强 physics analysis、writer/schema contract、性能 gate 和采样统计边界。任何摘要都不替代下表所指向的输入、analysis 和原始诊断输出。

本章的证据等级应按诊断问题分开理解：Langmuir 提供解析频率、场误差和最终守恒；uniform plasma 提供粒子数、能量统计和 checkpoint/restart 逐字段一致性，但短时总能量变化不等于热平衡守恒；FieldProbe 的 `lambda/32 matched-time` 对照通过解析 gate，而官方 `lambda/16 coarse case` 仍是失败证据；`reduced_diags` 将 compact observable 与 full-state reference 逐项对照，`LoadBalanceCosts` 则只验证效率改善；`ColliderRelevant`、`DifferentialLuminosity`、`ParticleHistogram2D` 和 `BeamRelevant` 分别验证其统计、谱或 writer 定义。RZ 多模 Langmuir 和 native Gaussian sibling 是有界案例证据，不能替代各自缺失的官方 analysis。每一项都必须沿验证矩阵中的 producer、consumer、observable 和限制阅读，不能用“已经运行”替代物理结论。

8.14 从诊断入口到可解释证据

面对一个新案例，先不要从“它写出了哪个文件”开始判断结果，而要依次回答三个问题：

1. 要测的物理量是什么，处在哪个时间层？主循环决定诊断何时采样；Full、BTD 与 BoundaryScraping 的差别决定你得到的是全状态、回到实验室系的快照，还是粒子边界事件。
2. 这个量怎样从模拟状态变成输出？ComputeDiagFuncctors 负责字段派生量，粒子采样和 flush 决定归约或 writer 的时机，OpenPMD iteration 则给出可由分析器读取的时间序列坐标。
3. 输出能够支持什么结论？解析 comparison 是 physics evidence，writer/schema 检查说明数据形状可消费，checksum 说明指定输出回归，performance gate 只说明效率指标。它们不能互相升级。

因此，MultiDiagnostics 或 WarpXOpenPMD 的入口存在，只说明诊断链被接入，不能反向证明每个下游案例都已通过。阅读验证矩阵时，应先从观察量和预期物理行为选择 case，再追踪生成该量的 consumer；不要把“文件已生成”误写为“物理机制已验证”。

8.14.1 三类 reduced diagnostics 的最小起点

三类 reduced diagnostics 的最小输入可以这样起步：

- FieldProbe: 官方 reduced-diags 测试中的 point/line/plane 骨架；
- ParticleHistogram2D: laser-ion 测试中的 z-uz openPMD mesh；
- LoadBalanceCosts: `LBC.type = LoadBalanceCosts` 与官方 efficiency analysis。

这些最小输入只回答“怎样产生该类输出”。它们不替代 FieldProbe 的解析 diffraction gate，不把 ParticleHistogram2D 的 writer/schema 变成物理收敛证明，也不把 LoadBalanceCosts 的效率比较与场精度混为同一类 physics gate。

8.15 练习与复现实验

1. 证据分层题：从验证矩阵中各选一个 physics gate、writer/schema 检查和 performance gate，说明它们的 producer、analysis 量和“不能支持的结论”。
2. reader-side 复现题：使用 reader-side analysis 或 reader-side analysis 读取一个案例输出，列出输入字段、输出文件和独立检查项。
3. 失败边界题：解释为什么 FieldProbe coarse failure、uniform-plasma reader-side 能量漂移和 initial-distribution binary mismatch 都应保留在书中，而不能简单从验证矩阵中删除。

8.16 延伸验证路线

- 沿 ComputeDiagFuncctors/、ParticleIO、WarpXOpenPMD 和 FlushFormats/ 分开追踪字段计算、粒子采样与 writer 生命周期，避免把文件格式当成物理诊断。
- 以 FieldProbe、ParticleHistogram2D 和 LoadBalanceCosts 的最小输入为例，分别复现解析 gate、writer/schema contract 和 performance gate。
- 用更长的 uniform-plasma 时间窗，并结合 energy_conserving_thermal_plasma 的 analysis，区分短时统计漂移与可解释的总能量结论。
- 改变 Langmuir 的模式数和拟合时间窗，检验频率测量对 sampling window 的敏感性，避免把单一窗口拟合误读为完整色散验证。

8.17 本章结论

诊断的价值不在于输出文件越多，而在于每一个输出都与明确的物理问题、时间层和判据相连。读者可以用下面四步设计或审读一个诊断：

1. 先写问题和比较对象。例如是 Langmuir 的解析频率、PML 的残余场、束流的相空间矩，还是 checkpoint/restart 的可重复性；不同问题要求不同 reference，而不是同一个通用 checksum。
2. 再选状态与时间层。确定需要全场、粒子、reduced observable、BTD 还是 boundary event，并明确采样发生在推进、同步或输出的哪一个阶段。
3. 为结论配对证据等级。物理 gate、writer/schema、性能量和输出回归应分别报告；只有与理论或独立 reference 比较的量才能支持相应的物理断言。

4. **保留失败与不可外推范围。**FieldProbe coarse failure、随机初始化的有限采样误差、跨 rank 的 field 差异和缺失的上游 analysis 都是结果的一部分。删除它们会让验证矩阵看似更整齐，却会削弱读者判断可信度的能力。

这条路线把第 4–7 章的算法、沉积、场更新与边界条件接到第 8 章的可观测量上：只有先知道一个量在何处产生、何时同步、由哪个 writer 或 analysis 消费，才能解释它究竟在验证哪一段 PIC 链。第 9 章将进一步说明如何为这些结论选择文献证据，并区分全文、摘要和源码案例各自能支持的范围。

9. 文献路线与延伸阅读

本书的文献不是装饰，也不是章节末尾统一贴一串 BibTeX。它真正承担三类职责：

1. 给物理结论提供一手来源。
2. 给数值算法和代码实现提供历史与方法边界。
3. 给独立分析、benchmark 和 regression 判据提供外部对照。

因此本章不把文献写成“推荐书单”，而是按证据强度和章节用途组织阅读路线：哪些来源能够支撑公式与机制，哪些只可提供历史线索，哪些问题仍应保留为开放边界。检索到的题名、DOI 或摘要只能帮助定位主题，不能替代全文阅读、公式核对和章节证据。

本章的读者用法：文献是论证工具，不是书目清单

读者不需要先把全部文献读完再学习 PIC。每篇文献在本书中只承担一个明确任务：解释一个物理概念、给出一个离散算法、提供历史边界，或作为某个诊断量的外部参照。遇到文献引用时，建议连续问三件事：

1. 这篇来源支持正文中的哪一句话，支持的是公式、机制还是历史事实？
2. 相关 WarpX 源码与运行案例分别提供了哪一层独立证据？
3. 哪一步仍然只是相似性或摘要级线索，不能写成“论文已经证明 WarpX 实现”？

这样读，文献路线就服务于教程的理解路径：第 1-2 章优先建立 kinetic/PIC 基础，第 4-6 章用 pusher、deposition 和 solver 文献解释离散选择，第 7 章用边界文献校准 PML/AMR 的历史语义，第 8 章再把外部理论与实际诊断对照。未取得全文或尚未完成逐段核对的来源只定义结论边界，不改变读者从概念到实现再到验证的主线。

9.1 文献证据的使用层级

本书使用的文献证据有四层，强度不能混写：

层级	本书中的典型形态	可支持的写法	限制
A. 全文可核查	全文、可检索文本、图像与逐段阅读笔记均可相互核对	可直接作为正文一手证据	仍需对照具体公式、图和段落，而不是只看中文摘要
B. 已取得待精读	全文可读，但尚未完成逐段笔记或章节对照	可作为“已取得、待精读”的明确线索	不能把具体公式或图表当成已核实正文
C. 书目信息/摘要线索	DOI、题名、摘要和可访问的书目信息	可作为取得边界或延伸阅读线索	不能把摘要内容冒充成论文正文结论
D. 旁证或相关文献	主题相关但不是当前章的主引用，或并非同一 bibliographic item	可作背景、旁证、术语线索	不能替代主引用本身

阅读和写作时应遵守：

- 只有 A 层全文可核查来源，才允许在正文里写成“已核实的一手证据”。
- B 层来源只能写成“已取得但尚待逐段讲解”。
- C 层和 D 层只能写成获取线索或背景边界，不能抬成正文论证。

这条规则尤其影响尚未完成全文或版本差异核对的 Hockney-Eastwood、Yee 1966、Esirkepov 2001、Villasenor-Buneman 1992 和 LeeCPC2015。

9.2 支撑各章的核心文献

本书已有可核查全文来源的核心文献主要集中在三条主线。

9.2.1 PIC foundations

可供深入阅读的起点是 Birdsall-Langdon 1985、Tajima-Dawson 1979、Dawson 1983、Vranic 2015 和 Muraviev 2021。第一次阅读应先从本章标出的概念和结论进入，而不是按年代或关键词机械浏览。

Tajima-Dawson 1979 用于解释最早的 driver -> wake -> trapping -> acceleration 与 LWFA scaling。它给出历史上的物理链条，而不替代现代 WarpX case 的输入、诊断和 regression。

这些基础文献主要支撑：

- 第 1 章的 superparticle、weighted particles、finite-size particles、quiet start、噪声与 heating 讨论；
- 第 2 章的最小 PIC loop、electrostatic / EM / Darwin 模型边界；
- 第 8 章中 diagnostics、spectrum、correlation time、weak instability dynamic range 的讨论；
- LWFA 最早 scaling baseline 的历史入口。

Vranic 2015 进一步补上了粒子 merge/resampling 这条应用线。它支撑第 4 章对“两粒子局部守恒 merge”的一手文献解释，但不替代 WarpX VelocityCoincidenceThinning 的逐行等价或专门运行验证。

Muraviev 2021 将这条应用线扩展为完整的 resampling 方法谱系。它支撑第 4 章对 agnostic down-sampling、局部权重噪声、严格守恒 thinning 和 merge/cluster 权衡的一手文献解释，但论文的 PICADOR/hi-chi QED cascade 结果不替代 WarpX 运行证据。

Birdsall-Langdon 1985 适合用作基础概念的回查书，而不是被误当作本书每个实现细节的直接来源；遇到具体 solver、pusher 或 deposition 公式时，仍应回到该算法的原始论文和对应源码。

9.2.2 Particle pusher

可供深入阅读的核心论文是 Vay 2008 和 Higuera-Cary 2017。

这两条线主要支撑：

- 第 4 章对 Vay pusher 与 Higuera-Cary pusher 的源码讲解；
- Source/Particles/Pusher/UpdateMomentumVay.H 与 UpdateMomentumHigueraCary.H 的公式对表；
- “相对论精度”和“结构保持”两条不同的算法目标。

但这条模块还没有完成 Boris 原始文献闭环，因此第 4 章仍不是“推进器历史谱系全闭环”。

9.2.3 PSATD / Galilean / boosted-frame / NCI

可供深入阅读的核心论文是 Godfrey 2014、Kirchen 2016 和 Lehe 2016。Vay 2014 的综述可作为第 4 章 Boris/Vay 谱系和第 6 章 PSATD/NCI 机制的统一入口；它不替代 WarpX 的源码交叉核对、案例验证或论文图形逐点复现。

这三条线构成第 6 章最完整的论文主干：

- Godfrey 2014: fixed-grid PSATD 的 NCI 策略分类；
- Lehe 2016: Galilean coordinates 消除 NCI 的核心离散论证；
- Kirchen 2016: boosted-frame workflow 与稳定离散表示之间的应用层连接。

Andriyash 2016 为 quasi-cylindrical Fourier-Bessel basis、PSATD 解析时间推进、 $m\pm 1$ 横向 mode coupling 和 current-correction 公式提供全文依据；但 PLARES-PIC 与 WarpX 的函数级等价、WarpX 运行复现和论文图逐点复现仍保持边界。

因此，第 6 章的阅读可以从论文的离散假设出发，再回查相应源码和案例；仍未完成的运行覆盖不会被写成已经证明的算法结论。

9.3 关键来源的已知边界

以 TajimaDawson1982 为例，应把“正式来源的书目信息已确认”和“可逐段核查的正文已取得”分开。Crossref/AIP 元数据确认 *AIP Conference Proceedings* 91(1):69–93 与 DOI 10.1063/1.33805，但本书覆盖的材料中没有可合法逐页核查的 publisher PDF。因此，相关会议稿只能作为主题旁证，不能替代 Tajima–Dawson 的正式条目。

这份相关会议稿可在有限范围内解释 beat-wave 共振、前向 Raman 散射、电子俘获、退相位、自聚焦、丝化和相对论前向 Brillouin 散射；但它是单作者相关会议稿，不能替代正式 Tajima–Dawson AIP 条目。

如果按“缺少它会让哪一章的论证最薄弱”排序，优先关注的不是更多新论文，而是以下几条老而关键的一手来源。

缺口	可用证据	主要影响章节	可替代程度
Hockney–Eastwood 原书	仅有书目信息与相关论文线索；尚无 可逐段核查的全文	第 1、2、5、6 章	只能部分由 Birdsall 1985 与 Dawson 1983 补足
Yee 1966	DOI 已确认；尚无全文	第 2、6 章	可由源码与后继 FDTD 文献支 撑，但缺原始历史入口
Esirkepov 2001	作者预印本及其公式材料可核查；仍 缺 CPC 发表版对照	第 5 章	可支撑预印本层面的解释，不能宣称 CPC 定稿已核对
Villasenor–Buneman 1992	论文全文及公式阅读材料可核查	第 5 章	可支撑公式与源码对照；仍需逐图、 逐记号复核

缺口	可用证据	主要影响章节	可替代程度
Andriyash 2016	全文公式材料可核查；主题为 quasi-cylindrical Fourier-Bessel PSATD	第 6 章 RZ PSATD	可支撑公式解释；不能推出 PLARES-PIC 与 WarpX 等价或运行复现
LeeCPC2015	accepted manuscript 可核查；仍缺 publisher-formatted CPC PDF	第 7 章	可支撑 accepted-manuscript 与源码的对应，不能宣称发表版逐系数等价

这六条中，LeeCPC2015 的边界最容易被误读：accepted manuscript 已可精读，所以它足以解释文中对应的 PML 思想；但发表版的排版、系数和版本差异尚未逐项核对，因此不能借此宣称已经完成 publisher-formatted CPC PDF 的对照。

9.4 各章的文献覆盖范围

全书各章的文献覆盖范围并不均匀。

章节	文献覆盖程度	主要已核查来源	主要边界
第 1 章 动理学模型	中等	Birdsall 1985、Dawson 1983	Hockney-Eastwood、更细的 particle-mesh heating 原始文献
第 2 章 PIC 总循环	中等	Birdsall 1985、Dawson 1983	Yee 1966 原始入口
第 3/3A 章 主循环与初始化	中低	以源码为主	需要把基础文献和工程论文绑定得更明确
第 4 章 粒子推进器	中高	Boris 1970 书目信息、Birdsall 1985、Vay 2008、Higuera-Cary 2017	原始 Boris 1970 会议论文 PDF 仍缺
第 5 章 沉积与形函数	中等	Esirkepov 与 Villasenor 的 charge-conserving 方法	Esirkepov 还缺 CPC 定稿对照；两种构造不能由单一案例互相替代
第 6 章 场求解器	高	Vay--Godfrey 2014、Godfrey 2014、Lehe 2016、Kirchen 2016	仍需把文献中的离散假设与各个具体求解器配置分别对应
第 7 章 边界、PML 与 AMR	中等偏低	Berenger 1994/1996、WarpX 源码与代表性案例	LeeCPC2015 出版社排版正文仍缺
第 8 章 诊断、验证与案例	中等	Dawson 1983 的诊断思路	还缺更多 case-specific benchmark papers
第 9 章 文献路线	本章即路线图	A/B/C/D 证据层级	新来源必须先判断证据层级，再进入章节论证

这个表最重要的结论是：相较于第 6 章，第 5 章和第 7 章更需要补强可逐段核查的一手文献。

9.5 延伸阅读的优先顺序

延伸阅读不应泛泛地“多找一些相关论文”，而应按它能补强哪一章的论证排序：

1. Esirkepov 2001 的 CPC 定稿 PDF
 - 预印本足以支撑本书的公式解释；与 2001 CPC 发表版逐项对齐仍是独立任务。
2. Yee 1966
 - 直接补第 2 / 6 章里的原始 FDTD 入口。
3. LeeCPC2015 正文 PDF
 - 直接补强第 7 章的 PML 一手文献。
4. Hockney-Eastwood 或其 article-level fallback

- 继续补第 1 / 2 章的 particle-mesh foundations。
5. Boris 原始文献
- 书目信息已核实，但原始 proceedings 全文尚未成为可逐页核查的材料。
 - 在获得合法全文前，不把 Birdsall 的二手推导写成 Boris 原文证据。

该顺序反映了证据分布：第 6 章已有较完整的论文主干，而第 5、7 章的原始文献支撑相对更薄。

9.6 用一张文献判读卡连接论文、源码与算例

不要把论文清单当成阅读成果。每读一篇主引用，都写一张不超过一页的**文献判读卡**；它的作用是把论文中的论断，接回本书的章节、WarpX 的实现和可观测量。卡片至少回答四个问题：

问题	读者应记录的内容	用途
这篇论文研究什么	一句物理问题、模型假设和适用尺度	防止把不同几何、近似或时间尺度混在一起
它给出什么证据	一个公式、图或定量判据及其上下文	区分原文结论与二手转述
它怎样接到本书	对应章节、源码职责和一个输出观察量	防止把论文机制直接写成程序行为
它还不能说明什么	版本、离散、几何、参数或运行覆盖的缺口	防止把局部证据放大成普遍结论

例如，读 Esirkepov 2001 时，卡片应把连续性方程与第 5 章的形函数/电流沉积并列；读 Lehe 2016 时，应把 Galilean 表示的离散假设与第 6 章的 PSATD 选择条件并列；读 LeeCPC2015 时，则应把 PML 的公式与第 7 章的残余场观察量并列。这样，读者获得的不是是一份资料清单，而是一组可检验的论证连接。

9.7 两条深读路线

下面两条路线把延伸阅读变成可检验的学习任务，而不是资料收集清单。

路线 A：第 5 章沉积文献

阅读任务：

- 先用 Esirkepov 2001 预印本追踪连续性方程、形函数和电流构造之间的关系；
- 再用 Villasenor 与 Esirkepov 的公式比较“轨迹分段”和“old/new shape difference”两种守恒组织；
- 最后回到第 5 章，分别写出论文、源码和测试各自能证明什么。

完成标志：

- 能说明为什么预印本公式不能自动证明 CPC 定稿逐式一致；
- 能用一个指定 case 的 observable 说明运行通过为什么不等于全部 geometry/order 覆盖。

路线 B：第 7 章 PML 文献

阅读任务：

- 用 Berenger 1994/1996 理解 split-field PML 的吸收机制；
- 对照 LeeCPC2015 的 accepted manuscript，定位高阶有限差分或伪谱 PML 的适用条件；
- 再回查 WarpX PsatdAlgorithmPml.cpp，区分论文公式、程序分派和指定 regression 的角色。

完成标志：

- 能指出 accepted manuscript 支持的具体论断；
- 不会把它误写成 publisher-formatted CPC PDF 的逐式核对。

第一次做文献深读时，建议先走路线 A：它能直接连接第 5 章的连续性方程、形函数和运行观察量；路线 B 更适合已完成第 6–7 章源码阅读、准备分析 PML 表示与边界条件的读者。

9.7.1 深读笔记的核查边界

文献判读卡与 A/B/C/D 层级一致，只说明你的材料分类没有自相矛盾；它不证明逐式审校已经完成，不证明预印本与出版社版本逐页等价，也不把 WarpX 的一个运行结果升级为论文全部物理结论的验证。判断引用强度时，应始终回到具体全文、公式、源码职责和案例观察量，而不是把卡片本身当成证明。

9.8 如何阅读证据边界

当正文出现“尚未证明”“仅适用于”或“不能外推”时，先判断它属于哪一种边界。不同边界要求不同的下一步，不能用更多文字把它们混成一个模糊的保留意见。

边界类型	它实际表示什么	读者可以说什么	读者不能说什么
文献边界	目标版本的全文或逐段核对不可得	已有材料支持的机制或历史线索	原始论文的逐式或逐图结论
实现边界	源码职责已知，但相应路径没有独立运行观察	这段代码在生命周期中的角色	该路径已经在目标物理问题中正确工作
数值边界	已有指定输入、观察量和容差下的比较	该案例在该 gate 下通过或失败	任意参数、几何或分辨率下都正确
收敛边界	重复计算或局部趋势存在，但关闭条件未满足	误差趋势或重复性与给定比较相符	已得到正式收敛阶或物理正确性证明

第 8 章的验证矩阵正是这张边界卡的运行版：它把 producer、consumer、observable 与限制并列。重复计算的 slope 一致性只能说明该组重复计算相符；它不等于 formal numerical order，也不等于 axis charge correctness。读者据此回到前八章时，应把每一句强结论都还原为“来源、实现、观察量、适用范围”四项，而不是只记住一个 PASS 标签。

9.9 本章结论

本书的文献部分已经能支撑核心主线，但还没有让所有一手来源都达到逐段核查的程度。更准确的概括是：

- foundations 线已有 Birdsall 1985、Dawson 1983、Tajima-Dawson 1979
- pusher 线已有 Vay 2008、Higuera-Cary 2017
- PSATD/NCI 线已有 Godfrey 2014、Lehe 2016、Kirchen 2016
- PML 线有较强源码与案例证据，但缺 LeeCPC2015 的出版社排版正文
- deposition 线的 Esirkepov 2001 作者预印本与 Villasenor-Buneman 1992 全文可供公式核查；Esirkepov 的 publisher-formatted CPC PDF 仍是明确的访问边界

因此，这条路线图的核心约束不是“再多下载一些论文”，而是：

1. 优先补能直接改变章节可信度的 primary sources；
2. 严格区分可逐段核查的正文材料和仅有书目信息的线索；
3. 将取得全文、阅读公式和检查章节论断视为同一条证据链。

做到这三点，第 9 章才不是附录式书单，而是读者判断全书证据质量的导航章。

9.10 练习与复核

9.10.1 证据层分类练习

从以下五项中各选一项，分别判断它属于 A、B、C 或 D 层，并写出判断所依据的书中路径：Birdsall 1985、Yee 1966、Esirkepov 2001 作者预印本、Tajima 1982 FNAL 相关会议稿、LeeCPC2015 accepted manuscript。答案必须同时写出“可以支持的句子”和“不能支持的句子”。例如，不能因为某项有 DOI 或摘要，就把它写成“已完成全文精读”。

9.10.2 证据边界复核练习

选择一条 A 层来源和一条 C 层来源，各写一张文献判读卡，分别列出它们支持与不支持的结论。解释为什么材料分类一致只能说明阅读路线自洽，不能证明论文出版社版本已取得，也不能证明 WarpX 运行已复现论文全部结论。

9.10.3 延伸阅读排序练习

从 Hockney-Eastwood、Yee 1966、Esirkepov 2001 CPC 定稿、LeeCPC2015 publisher PDF 和 Boris 1970 原始 proceedings 中选出下一项优先阅读或取得的来源。用三列短表说明：它影响哪一章、现有哪一级证据、取得后会澄清哪一个具体边界。若目标仍受访问或许可限制，必须把“继续取得全文”和“先用现有证据理解正文”分成两个独立动作。

附录 A：符号、时间层与源码变量

本附录服务于正文阅读和源码检索。公式中的符号采用连续模型或离散模型的常用记号；反引号中的名称则优先对应 WarpX 源码、输入文件或 diagnostics 输出中的实际字段。除特别说明外，SI 制单位适用，粒子权重 w_p 表示一个宏粒子代表的真实粒子数。

使用本附录时，先区分三件事：数学符号描述的物理量、WarpX 内部数组实际保存的量、输入文件或 diagnostics 对同一名称采用的单位。它们常常有关联，但不必完全相同；尤其是 ux ， uy ， uz 不能只按变量名猜测单位。

A.1 连续模型符号

符号	含义	常见单位或类型
s	species 索引，例如 electron、ion、photon	无量纲整数
$f_s(\mathbf{x}, \mathbf{p}, t)$	species s 的相空间分布函数	依定义而定
q_s, m_s	species 电荷和静质量	C、kg
$\mathbf{x} = (x, y, z)$	物理空间位置	m
$\mathbf{p}$	物理动量	kg m s^{-1}
$\beta = \mathbf{v}/c$	无量纲速度	无量纲
$\mathbf{u} = \mathbf{p}/m = \gamma \mathbf{v}$	proper velocity; WarpX 粒子容器内部的 ux ， uy ， uz 以此量保存	m s^{-1}
$\mathbf{p}/(mc) = \mathbf{u}/c = \gamma \beta$	无量纲归一化动量；多数 species 输入的 $\mathbf{u}$ 参数采用这一约定	无量纲
$\mathbf{v}$	粒子速度	m s^{-1}
γ	Lorentz 因子， $\gamma = (1 - v^2/c^2)^{-1/2}$	无量纲
c	真空光速	m s^{-1}
$\mathbf{E}, \mathbf{B}$	电场和磁场	V m^{-1} 、T
$\rho, \mathbf{J}$	电荷密度和电流密度	C m^{-3} 、 A m^{-2}
ϵ_0, μ_0	真空介电常数和磁导率	SI 常数
w_p	宏粒子权重，代表的真实粒子数	无量纲或按模型定义
S	粒子到网格的空间形函数	按离散归一化定义

ux ， uy ， uz 的两层约定

对有质量粒子，三种写法的关系是

$$\mathbf{p} = m\mathbf{u}, \quad \mathbf{u} = \gamma\mathbf{v}, \quad \frac{\mathbf{u}}{c} = \frac{\mathbf{p}}{mc} = \gamma\beta,$$

并且

$$\gamma = \sqrt{1 + \left| \frac{\mathbf{u}}{c} \right|^2}.$$

这解释了两个表面矛盾、实际一致的源码约定。

1. Source/Particles/WarpXParticleContainer.H 将粒子容器中的 ux ， uy ， uz 说明为 proper velocity，即 $\gamma\mathbf{v}$ ，内部量纲为 m/s。
2. Docs/source/usage/parameters.rst 将 single_particle_u、multiple_particles_u* 和常见 species momentum distribution 的输入定义为 $\gamma\mathbf{u}$ ，即无量纲的 $\gamma\mathbf{u}/c$ 。初始化时会乘以 c ，再写入内部粒子数组；PlasmaInjector.cpp 中的 multiple_particles_ux 转换正是这一边界的例子。

因此，阅读输入文件时把 $uz = 10$ 理解为

$$\gamma\beta_z = 10$$

；阅读粒子容器或以 SI 单位计算能量的 kernel 时，把内部 uz 理解为

$$\gamma v_z = 10c$$

。诊断 parser、openPMD 记录和不同输出格式还可能按其文档以

$$\gamma v/c$$

暴露动量分量；遇到 `ux` 时，先确认它来自输入、容器还是输出 metadata，再进行单位换算。

A.2 网格、时间层与离散量

符号	含义	在程序中的对应
t^n	第 n 个整数时间层	<code>step</code> 、 <code>istep</code>
$t^{n+1/2}$	半时间层，常用于 leapfrog 电流或磁场	$J^{\{n+1/2\}}$ 等时间层语义
Δt	时间步长	<code>dt</code> 、 <code>warpx.const_dt</code> 或派生时间步长
$\Delta x, \Delta y, \Delta z$	各方向网格间距	<code>amr.n_cell</code> 、几何 <code>cell_size</code>
i, j, k	网格单元或节点索引	AMReX <code>IntVect</code> 分量
ℓ	AMR level 索引, $\ell = 0$ 为最粗层	<code>lev</code> 、 <code>level</code>
r	RZ/cylindrical 几何中的径向坐标	<code>m</code>
<code>rank</code> 或 r_{MPI}	MPI rank 或 rank 索引；不用裸 r 与径向坐标混写	<code>ParallelDescriptor::MyProc()</code> 等
V_i	网格单元体积	Cartesian 中常为 $\Delta x \Delta y \Delta z$
ρ_i^n	单元/节点 i 在时间层 n 的电荷密度	<code>rho</code> 、 <code>rho_fp</code> 、 <code>rho_buf</code>
$\mathbf{J}_i^{n+1/2}$	时间层 $n + 1/2$ 的电流密度	<code>current_fp</code> 、 <code>current_buf</code>
$\nabla_h \cdot \mathbf{J}$	离散散度	由 <code>staggering</code> 和差分 <code>stencil</code> 决定
$S_i(\mathbf{x}_p)$	粒子位置对网格量 i 的形函数值	<code>ShapeFactors</code> 中的 <code>shape</code> 权重

在 AMR 语境中，`_fp` 与 `_cp` 通常分别标记 fine patch 和 coarse patch，`_aux` 则标记供粒子 gather 使用的辅助场。它们不是同一物理场的任意别名：粗细网格覆盖、guard cell 和同步阶段决定某个数组何时有效。第 3、5 和 7 章中的 `current_fp/current_buf`、`rho_fp/rho_buf` 与 `Efield_aux/Bfield_aux` 都应按这个生命周期来读。

A.3 沉积、推进与场求解记号

符号或名称	含义	阅读提示
$\rho^n \rightarrow \mathbf{J}^{n+1/2} \rightarrow \rho^{n+1}$	电荷守恒 current-deposition 主线的时间层关系	不能把 charge deposition 和 current deposition 当成同一个 kernel
<code>old / new</code>	粒子在一个推进步或 segment 两端的形函数/位置状态	Esirkepov、Villasenor kernel 中常见
<code>relative_time</code>	在同一步内取样粒子位置的相对时间参数	影响 source 的时间层，而不是额外推进粒子
<code>icomp</code>	charge component 或时间层分量选择	需结合调用入口解释，不能只按名字猜测
<code>depos_lev</code>	charge 沉积目标 AMR level	coarse-buffer 粒子可能直接沉积到 <code>lev-1</code>
<code>current_fp</code>	fine-patch current buffer	主要对应 fine patch 粒子沉积
<code>current_buf</code>	coarse-buffer current buffer	后续还要经过同步/整理
<code>rho_fp / rho_buf</code>	fine-patch / coarse-buffer charge buffer	是 source route 的观测面，不等于最终 plotfile 字段
<code>Efield_fp / Bfield_fp</code>	fine-patch 电场/磁场	不能直接假定就是粒子 gather 的数组
<code>Efield_cp / Bfield_cp</code>	coarse-patch 电场/磁场	在 mesh refinement 近边界的 gather/sync 语义要结合 level 与 patch type 判断
<code>Efield_aux / Bfield_aux</code>	供粒子 gather 的辅助场	由 <code>UpdateAuxiliaryData</code> 等路径从 patch 场构造；只定义 E/B，不把它误认为 current 或 charge 容器
<code>Direct</code>	直接速度加权电流沉积	简单但不自动满足离散连续性方程
<code>Esirkepov</code>	基于轨迹与形函数差分的 charge-conserving current deposition	重点看 density decomposition 和 prefix accumulation

符号或名称	含义	阅读提示
Villasenor	基于 cell crossing segment 的 charge-conserving current deposition	重点看 crossing、segment 和局部 face flux
Vay	Vay current deposition / spectral 相关路径	常与 PSATD、current correction 组合讨论
FDTD	有限差分场求解器	依赖 staggered grid 和 CFL 限制
PSATD	pseudo-spectral analytical time-domain 求解器	重点看谱空间系数、源项时间模型和周期边界假设
JRhom	PSATD 的多次 J/ρ 时间采样路径	不是多次粒子 push, 而是同一轨迹的多时刻源项
PML	吸收边界层	通过 split-field 或等价谱推进吸收出射波

A.4 诊断、文件与验证术语

名称	含义
plotfile	WarpX/AMReX 网格和粒子状态的可重启或后处理输出
openPMD	粒子、网格或 reduced diagnostics 的结构化输出接口
diagNNNNNNN	diagnostics 输出的迭代目录, 例如 diag1000000
reduced_diags	不保存完整场/粒子, 而输出聚合标量或低维量的 diagnostics
FieldProbe	沿指定几何路径采样场并做积分或线探针输出的诊断
BoundaryScrapingDiagnostics	记录穿过粒子边界的粒子及其统计量
producer	产生字段、粒子或 reduced output 的运行时路径
consumer	读取产物并执行物理、格式或性能断言的分析脚本
physics gate	对解析解、守恒量、谱或物理量的数值容差断言
writer gate	对文件、字段、维度、iteration、轴和有限非零数据的断言
checksum gate	对最终输出做确定性校验; 不能自动等价于物理正确性
reader-side analysis	从已有 plotfile/openPMD 重新读取数据并验证合同的分析层

A.5 常用缩写

缩写	全称	本书中的语境
PIC	Particle-In-Cell	粒子-网格数值方法
AMR	Adaptive Mesh Refinement	多层网格和 coarse/fine 同步
CFL	Courant-Friedrichs-Lewy	显式场推进稳定性限制
NCI	Numerical Cherenkov Instability	boosted-frame / PSATD 中的数值不稳定性
RZ	cylindrical geometry with azimuthal modes	WarpX 的轴对称/模式几何
EB	Embedded Boundary	嵌入边界和 cut-cell 几何
MPI	Message Passing Interface	rank 间并行通信
GPU	Graphics Processing Unit	device kernel 和 GPU memory 路径
QED	Quantum Electrodynamics	高场量子辐射和 pair-production 模型
LWFA / PWFA	Laser / Plasma Wakefield Acceleration	激光/束流驱动尾场应用

A.6 使用规则

- 看到 u_x , u_y , u_z 时, 先确认它来自输入、内部粒子容器还是 diagnostics; 分别按 $\gamma\beta$ 、 $\gamma\mathbf{v}$ 或该输出的 metadata 解释, 不能把数值直接混用。
- 看到 ρ 、 current_fp 或 current_buf 时, 先判断它属于 local kernel、level buffer、同步后场, 还是最终 diagnostics 输出; 同名物理量可能处于不同生命周期阶段。
- 看到上标 n 、 $n + 1/2$ 或 relative_time 时, 先确认粒子位置、场、current 和 charge 是否处在同一个时间层; 不能只根据变量名推断守恒性。